# A2 Software Systems Development Coursework guide

Build the bike shop system one chapter at a time, then turn it into your own system. The page numbers below are the page numbers your PDF reader shows.

## Contents


| | | |
|---|---|---|
| 0 | Start here | page 2 |
| 1 | Your workspace | page 14 |
| 2 | Plan the tables | page 43 |
| 3 | Build the database | page 61 |
| 4 | Stored procedures | page 151 |
| 5 | Connect C# | page 240 |
| 6 | Add, edit, delete, display | page 322 |
| 7 | Bookings | page 421 |
| 8 | Stock | page 546 |
| 9 | Report across tables | page 762 |
| 10 | Testing evidence and user guide | page 842 |
| 11 | Mistakes that cost marks | page 867 |
| 12 | Make it yours | page 876 |

CHAPTER 0 OF 12

# Start here

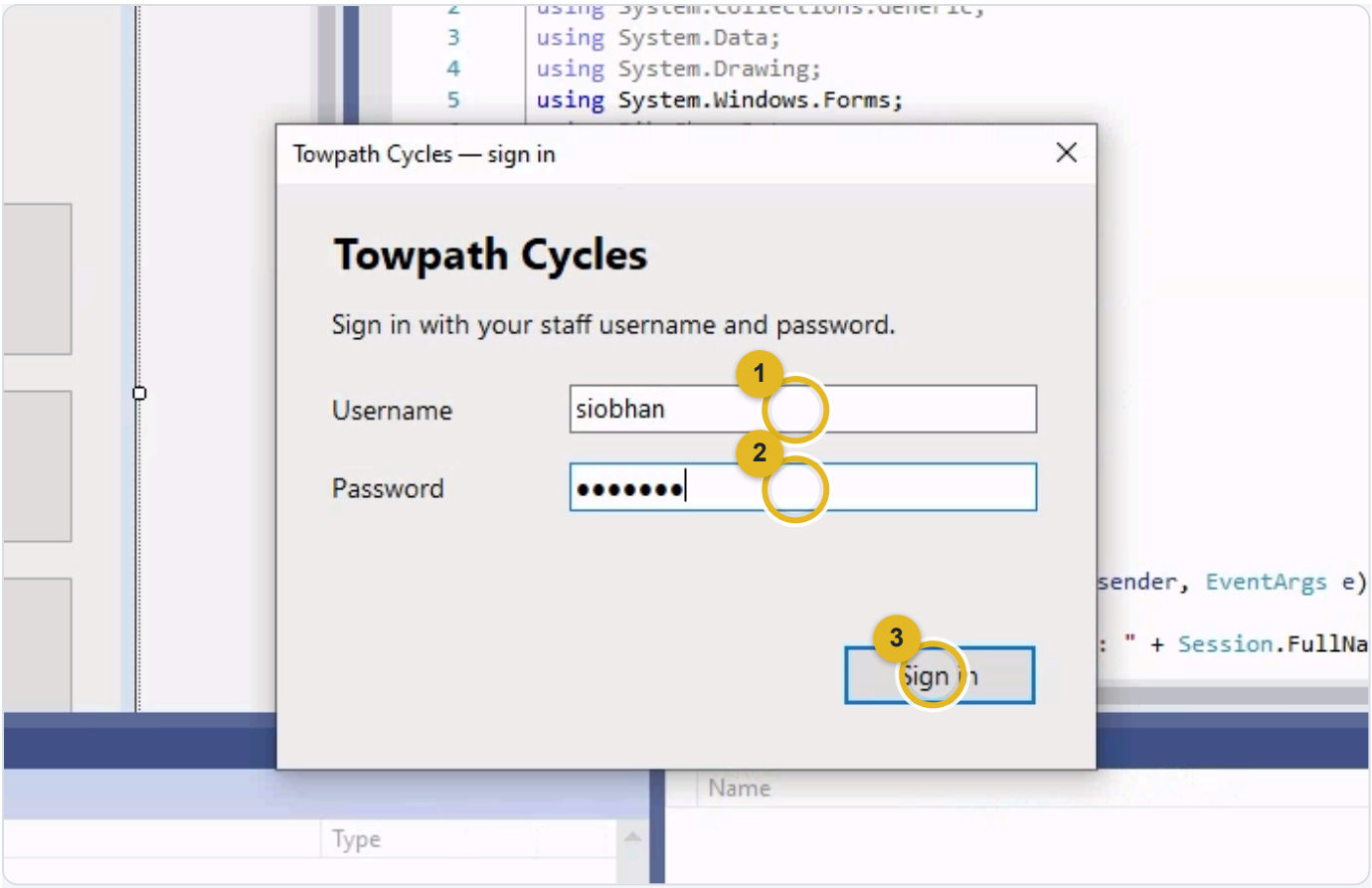
What you are making, how the guide works, and the two rules that never break.

STAGE 1 OF 2 · WHAT YOU ARE MAKING · SCREEN 1 OF 3

## See the shop and the program you will build for it.

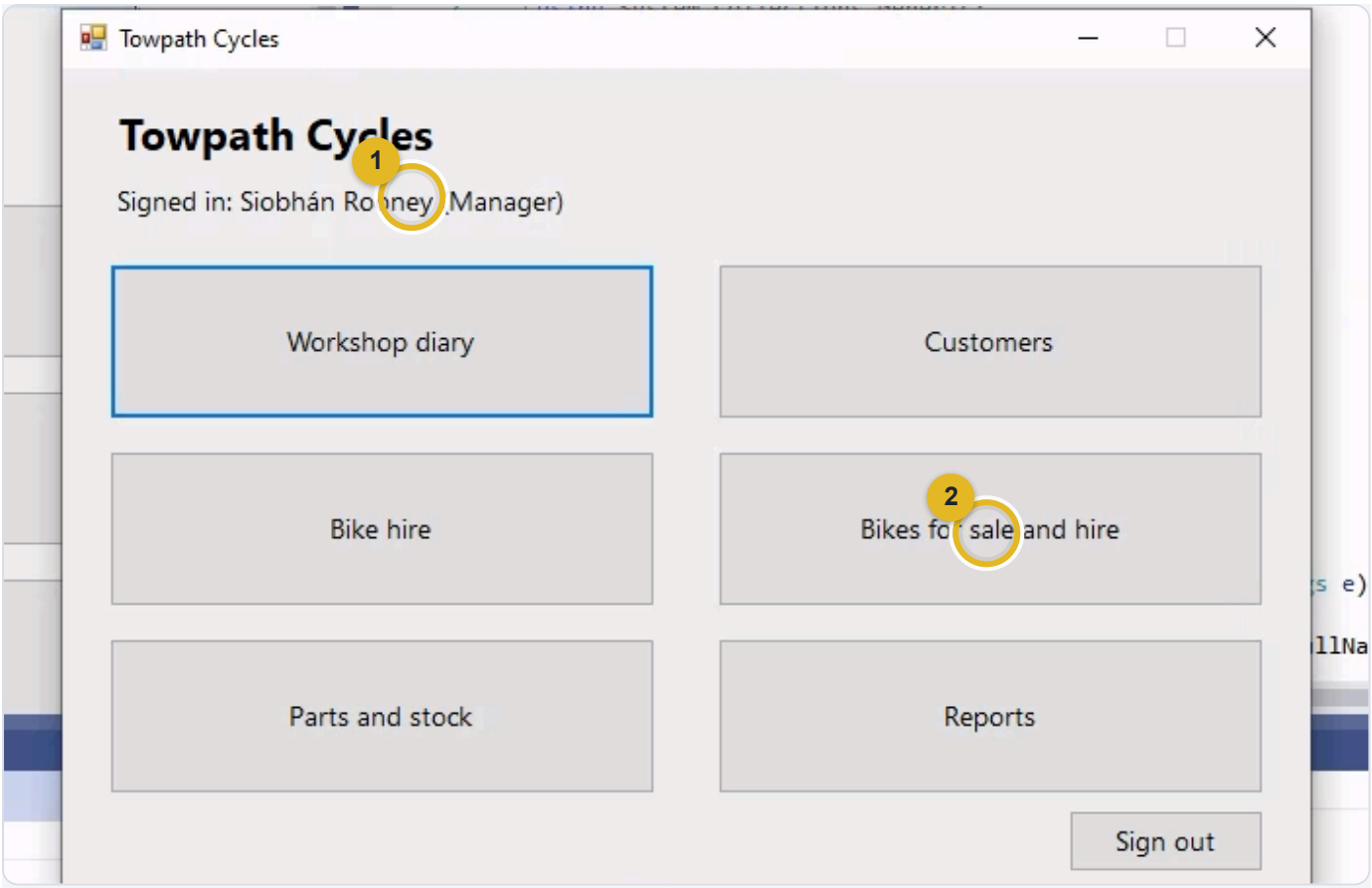
Towpath Cycles is a small bike shop beside the canal towpath in Newry. Its mechanics repair bikes, and customers book a time with a mechanic for each repair. The shop also hires out bikes by the day and sells bikes of its own. It keeps spare parts on the shelf, sells them, and orders more from its suppliers. The shop is made up, and it holds every kind of booking and stock that a CCEA case study asks for.

In this guide you build a program for the staff of Towpath Cycles. The program keeps everything in a database, which is a store of information kept in tables. A table is a grid where each row is one thing, such as one customer or one booking. The six pictures below are the finished program, signed in as Siobhán Rooney, the manager.



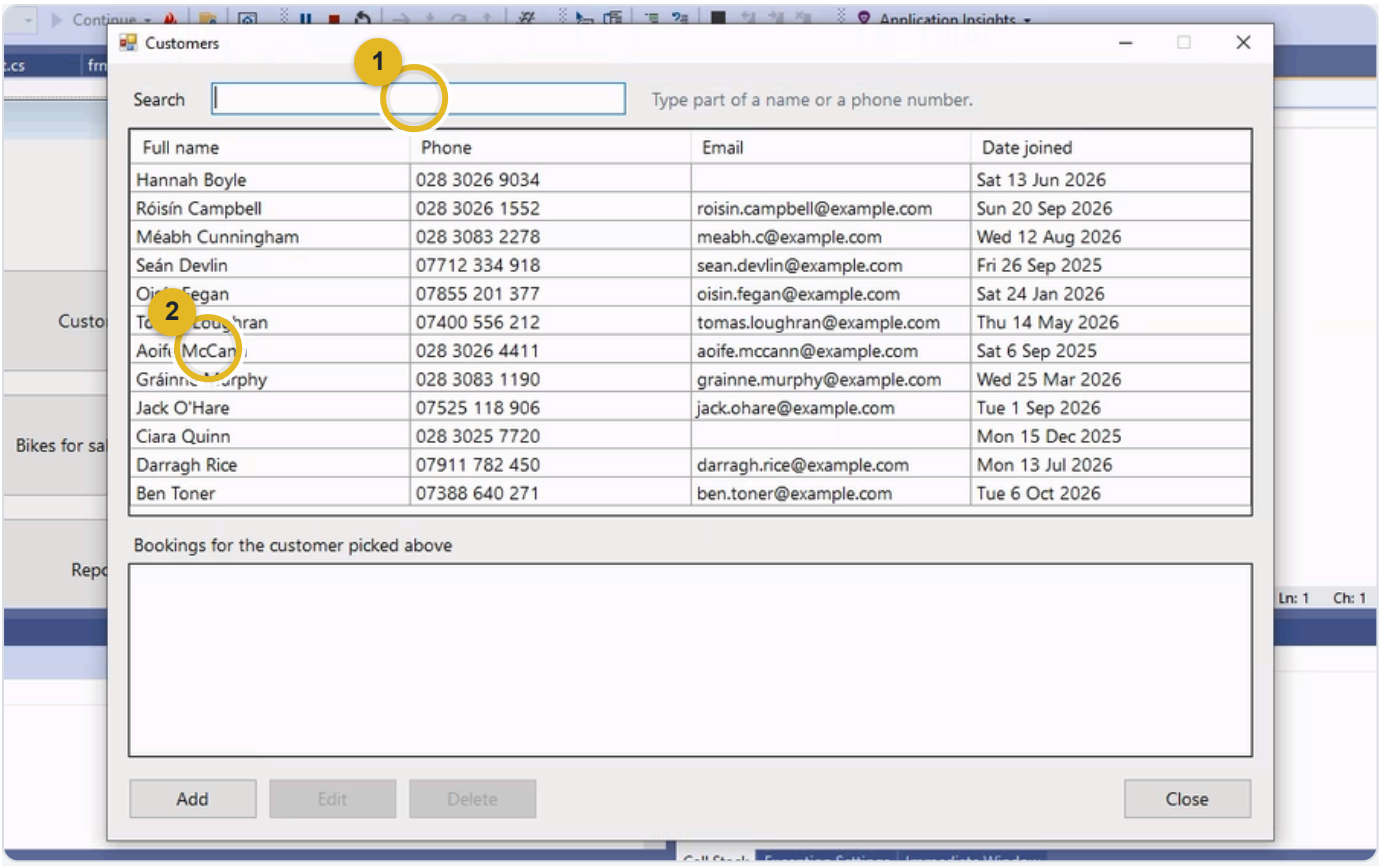

The sign-in window. Every member of staff signs in with their own username, so the program knows who did what.

1. Type the username.
2. Type the password.
3. Click Sign in.



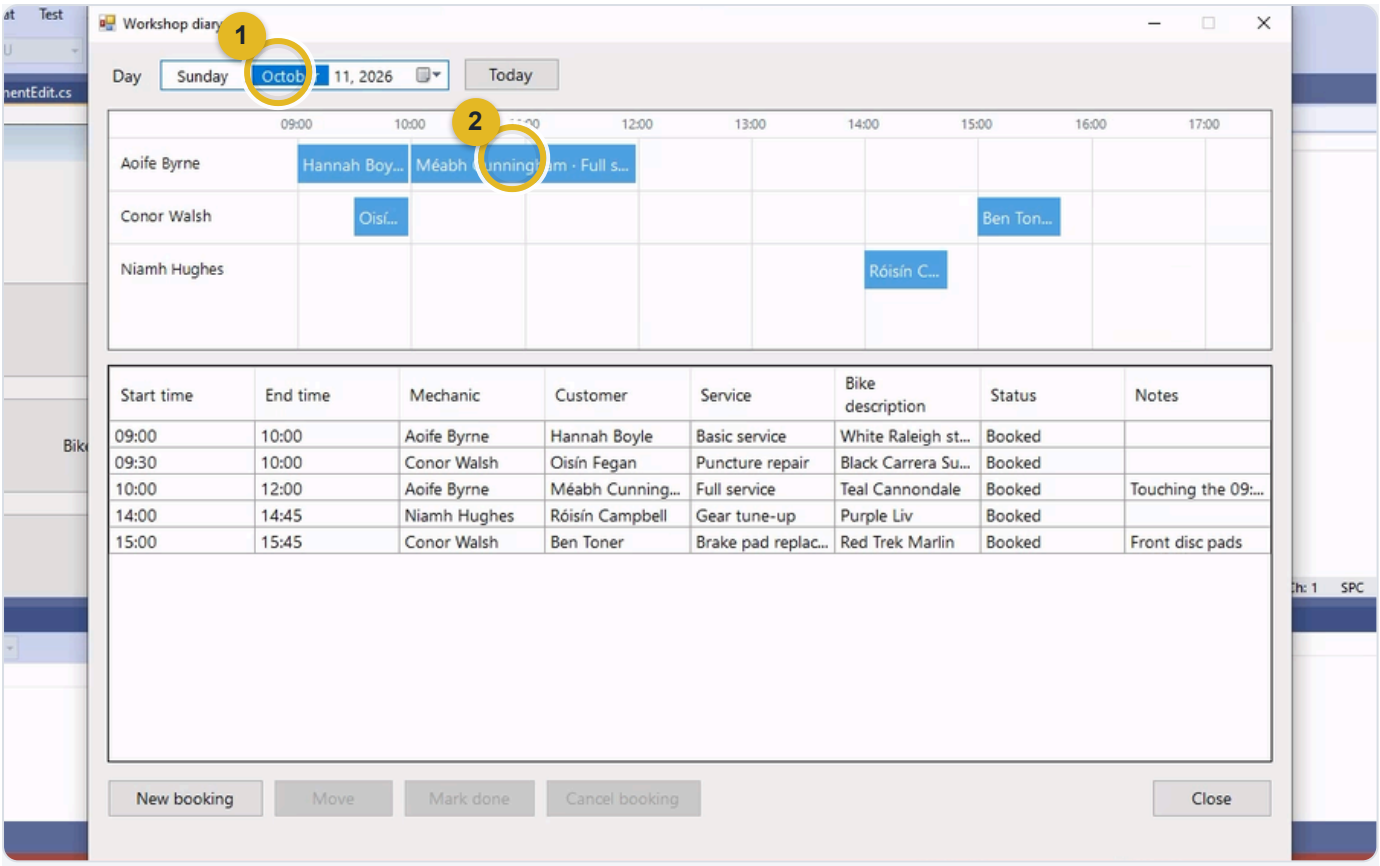

The main menu. One button opens each screen of the program.

1. Who is signed in.
2. One button for each screen.



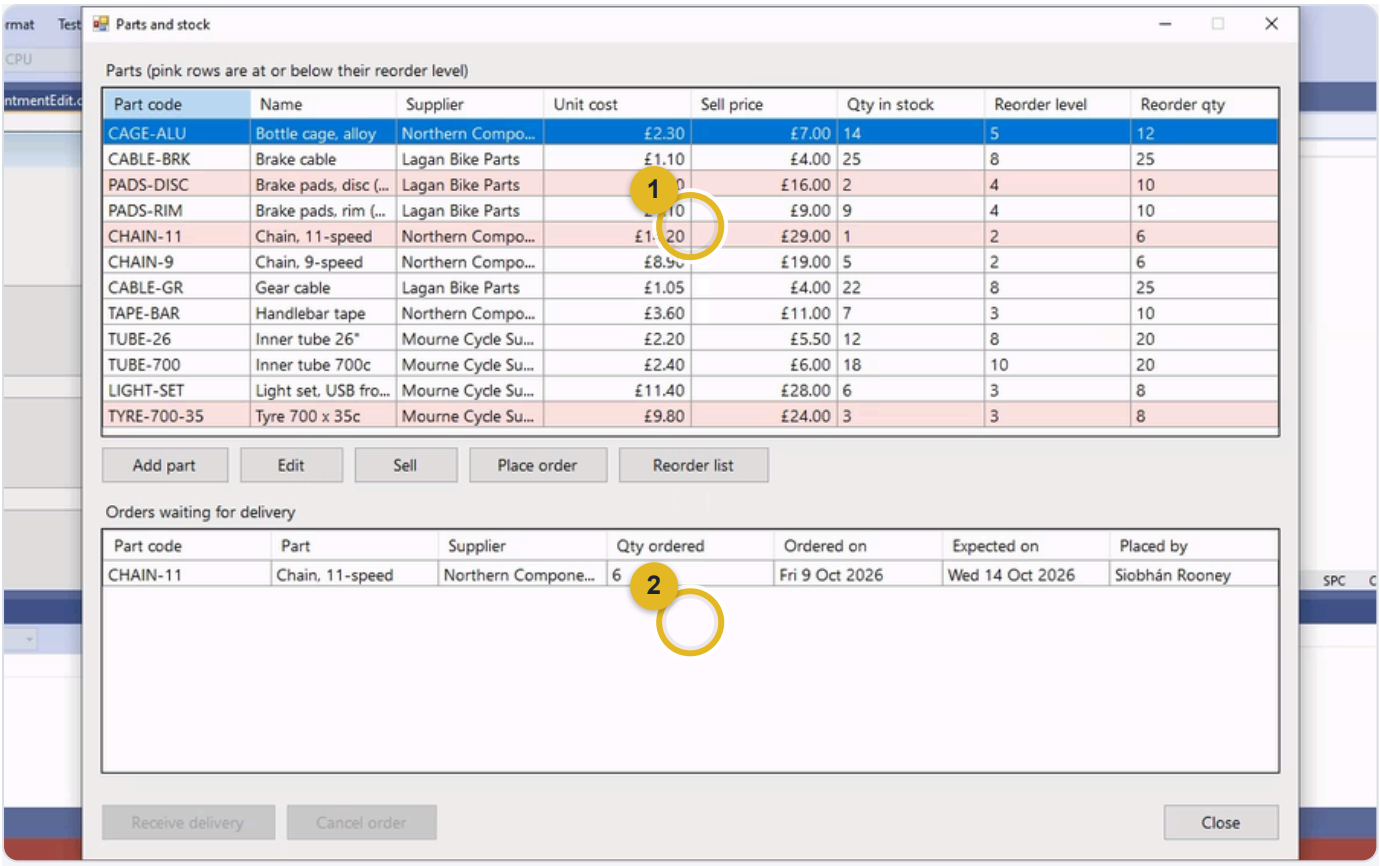

The customers screen. You build it in chapter 6.

1. The Search box narrows the list as you type.
2. Customers are shown by name, never by number.



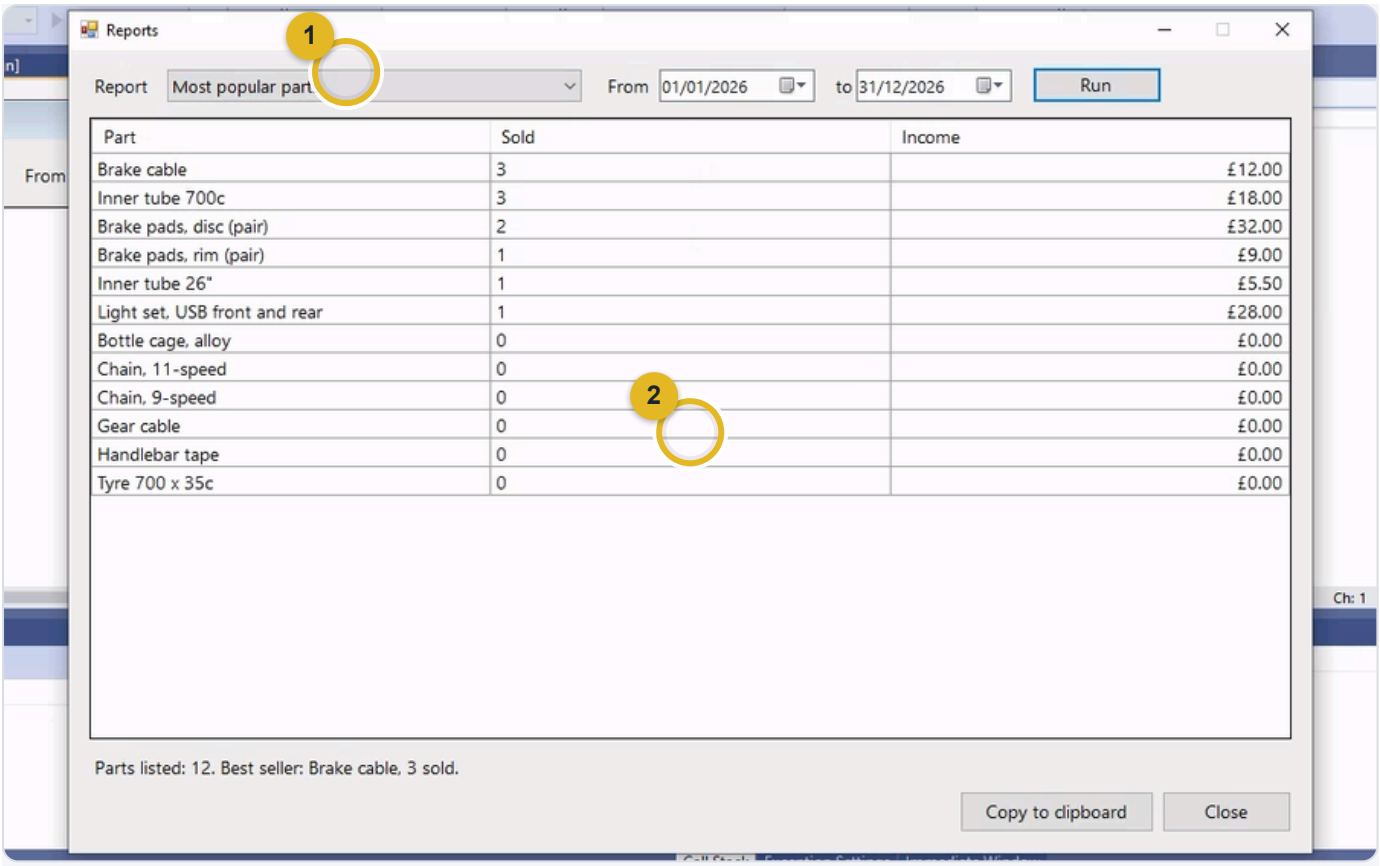

The diary of workshop bookings. You build it in chapter 7, and it can never book one mechanic twice at the same time.

1. Pick a day here.
2. One lane for each mechanic, with each booking drawn as a bar.




The parts and stock screen. You build it in chapter 8, and its counts are always right.

1. Pink rows are parts that are running low and need to be ordered.
2. Orders from suppliers that have not arrived yet.




The reports screen. You build it in chapter 9, and each report joins several tables for a range of dates.

1. Pick a report and two dates.
2. The answer, worked out from several tables at once.

STAGE 1 OF 2 · WHAT YOU ARE MAKING · SCREEN 2 OF 3

# Read the two rules that never break.

Every case study has bookings and stock, and the examiner checks two things about them harder than anything else. Each rule is written below in one sentence, with what goes wrong in the shop without it.

| | |
|---|---|
| **RULE THAT NEVER BREAKS · NO DOUBLE BOOKING** | One mechanic can never be booked for two repairs that overlap in time. |

Without this rule, two customers arrive at 13:00 for the same mechanic, and one of them is sent home.

| | |
|---|---|
| **RULE THAT NEVER BREAKS · STOCK IS ALWAYS RIGHT** | The number of each part in stock always matches what is really on the shelf, and it can never go below zero. |

Without this rule, the program sells three brake cables when the shelf holds two, and the count says minus one.

Chapter 7 builds the first rule and chapter 8 builds the second. In both chapters the database itself refuses the bad booking or the bad sale, so no screen of the program can get round it.

# See where the fifty marks come from.

The coursework is marked out of 50 marks, split into three parts called assessment objectives, AO1, AO2 and AO3.

AO1 is worth 10 marks, for your project plan and your list of what the users need.

AO2 is worth 24 marks, for the working program and the evidence that it works.

AO3 is worth 16 marks, for how well you look back at your plan and judge what you made.

The top band of AO2, Level 4, is 19 to 24 marks, and it asks for each of the following.

The program works and covers the whole case study, with screens that are easy to use.

It can add, change, delete and show records from several tables that are linked to each other.

It has a report that brings together records from several linked tables and works with them, such as adding up totals.

You hand in the script that makes and fills the database, and all the code of the program.

You write a user guide that shows a member of staff how to use every screen, step by step.

You write a test plan that covers the whole program, with the result of every test.

This guide builds every one of those, in the bike shop first, so that you can build the same for your own case study.

# Learn how to move through a chapter and tick your steps.

Each chapter is split into stages, and each stage is a short run of screens. A screen holds one thing to read or one small group of steps to do. The line above each screen heading says which stage you are in and which screen of that stage you are on.

The strip of numbered stages under the chapter title shows the whole chapter. Each stage has a row of small dots, one dot for each screen, and the dot for the screen you are on is lit. Click a stage or a dot to go straight to it.

At the foot of every screen, the Back button goes to the screen before and the Next button goes to the screen after. The Next button shows the heading of the screen it leads to. The left and right arrow keys on the keyboard do the same. If you close the guide and open the chapter again, it opens on the screen you were last on.

Every step has a small numbered box at its left. Click the box when you have done the step, and it turns into a tick. Ticks are saved to your name when you are signed in, so they are still there on another computer and in another lesson. When every step on a screen is ticked, the dot for that screen is ticked too.

The last screen of every chapter counts your ticks, and it has a button that says Clear my ticks. Clear my ticks asks you first, and then takes every tick off that chapter, so that you can work through it again from the start. The same screen has the button that marks the chapter as done, and the button that goes on to the next chapter.

Click the numbered box at the left of this step.

> **You should see** The box turns into a tick, and the dot for this screen in the stage strip is ticked too.

# Learn what the four labels on the code mean.

From chapter 3 onward, the guide shows you code, which is the lines a computer follows. Every block of code has a label at its top, and the label tells you what to do with it before you read a line.

**Copy this one**

You read this code and copy it into your file with the Copy button. It is used for the long parts that every program needs, and for the first time the guide shows a new idea in full.

**Type this one**

You type the yellow lines yourself, letter by letter. These blocks are short, usually ten lines or fewer, and the grey lines round them can still be copied. The yellow lines are the ones your own project needs, with your own names in them.

**Your turn**

There is no code to copy here. The guide tells you what the code must do, the test to run, and the message you should see when it works. If you get stuck, a closed box under it that starts with Stuck? Show me holds the answer.

**Read this one**

You read this code to see how something works, and you do not copy it or type it. It has no Copy button.

Typing the short blocks matters, because those lines are the ones you change when you build your own case study. When your fingers have typed a line once, you notice each word in it, and you can find the same line in your own code later. Every block also has a button that says Walk me through it, which lights up one part of the code at a time and explains it in plain words.

# Find the one download, then start chapter 1.

There is one file to download for the whole coursework. It is called BikeShop-sample-data.sql, and it holds sample data, which is made-up records for the shop, such as twelve customers and the parts on the shelf.

You will use it in chapter 3, and the button for it is there too. It is also in the Sample data box on the front page of this guide, under the list of chapters.

Everything else you build yourself, chapter by chapter. Chapter 1 sets up the place where you work: SQL Server Management Studio (SSMS), Visual Studio 2019 and your OneDrive folder, inside the Programming Environment.

Click the button that says Mark chapter 0 as done, at the foot of this screen.

> **You should see** The button changes to say that chapter 0 is done.

Click the button that says Go on to chapter 1, at the foot of this screen.

> **You should see** Chapter 1, Your workspace, opens on its first screen.

**Words used in this chapter**

| | |
|---|---|
| `Database` | A store of information kept in tables, which the program reads from and writes to. |
| `Table` | A grid with named columns, where each row is one thing, such as one customer. |
| `Record` | One row of a table, such as one customer or one booking. |
| `Program` | A set of screens that staff click and type in, which you write in Visual Studio 2019. |
| `Code` | The lines a computer follows, written in a language such as SQL or C#. |
| `Script` | A file of SQL lines that the database runs from the top to the bottom. |
| `Sample data` | Made-up records with real-looking names and dates, so that you have something to test with. |
| `Double booking` | Two bookings for the same mechanic that overlap in time. |
| `Stock` | The number of each part the shop has on its shelf. |
| `Assessment objective` | One of the three parts the marks are split into, called AO1, AO2 and AO3. |
| `Level 4` | The top band of marks in an assessment objective. |
| `Stage` | One part of a chapter, made up of a few screens. |
| `Tick` | The mark you put on a step when you have done it, saved to your name. |

CHAPTER 1 OF 12

# Your workspace

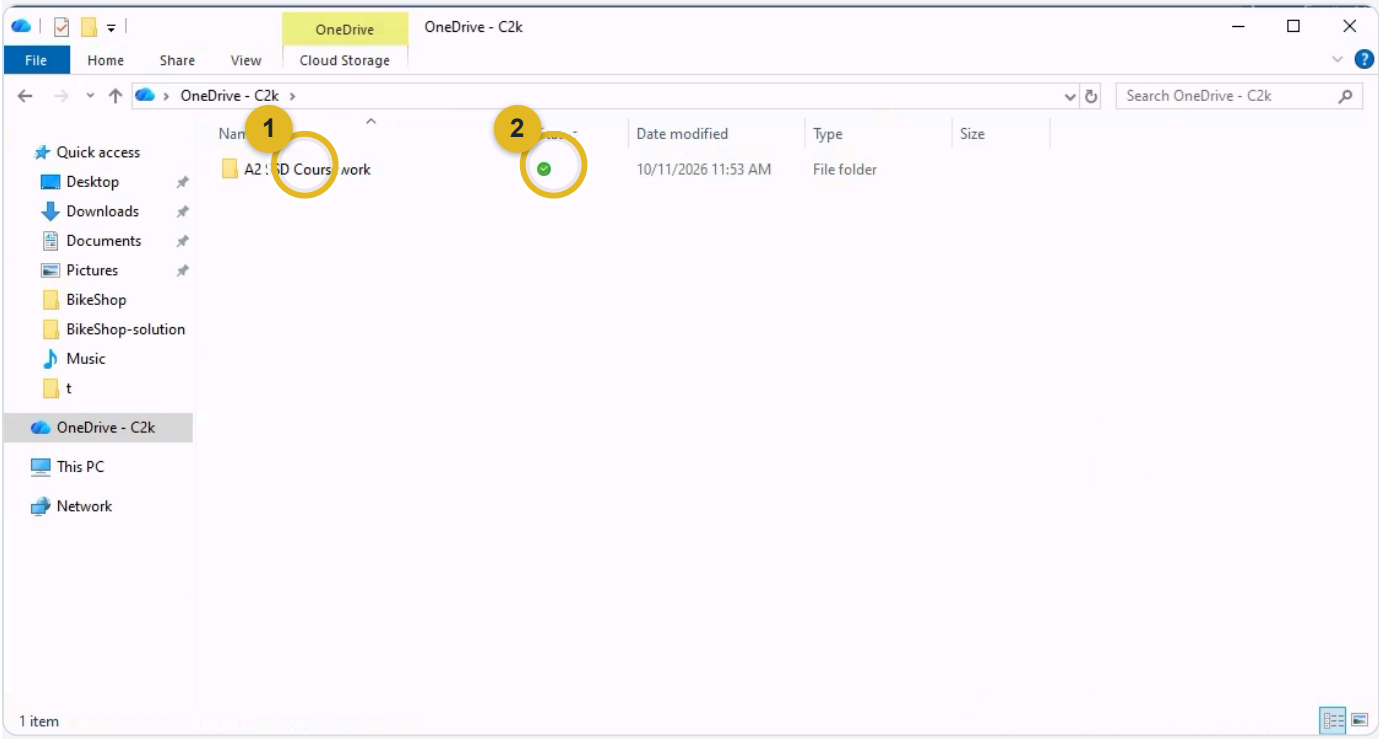
SSMS, Visual Studio and OneDrive inside the Programming Environment, and where your files live.

STAGE 1 OF 5 · YOUR FOLDER · SCREEN 1 OF 3

## See what you will set up in this chapter.

Before you build anything, three things need to be in place. The first is a folder in your school OneDrive, where every file for this coursework lives. The second is SQL Server Management Studio (SSMS), connected to the database server. The third is Visual Studio 2019, checked so that it can make the kind of program this guide builds.

You do all of this inside the Programming Environment, the Windows desktop you are signed in to now. By the end of the chapter you will have run your first line of SQL, and one empty file called BikeShop.sql will be waiting in your own folder.




Your coursework folder in OneDrive. The green tick means OneDrive has copied it to your school account.

1. The folder A2 SSD Coursework.
2. The green tick in the Status column.

# Check that you have these ready.

**Before you start**

- You are signed in to the Programming Environment.
- This guide is open in the Edge browser inside the Programming Environment, beside the other windows.
- You have read chapter 0, so you know how the screens, the steps and the ticks work.

**Words used in this chapter**

| | |
|---|---|
| `Programming Environment` | The Windows desktop you reach through C2k, where SQL Server and the programming tools are installed. |
| `OneDrive` | Storage on your school account. A file saved there can be opened from any computer you sign in on. |
| `Folder` | A named place that holds files, the way a ring binder holds pages. |
| `File Explorer` | The Windows window that shows your folders and files. |
| `Green tick` | The mark OneDrive puts beside a file or folder once it has copied it to your school account. |
| `SQL` | The language you use to make a database and to ask it questions. |
| `Database` | A set of tables that keeps the shop's records, such as its customers and its bookings. |
| `Server` | A program that keeps databases and answers the SQL sent to it. It has no window of its own. |
| `LocalDB` | A small copy of SQL Server that starts by itself when a program connects to it. |
| `SQL Server Management Studio (SSMS)` | The window you use to type SQL, send it to the server, and read the answer. |
| `Connect` | Link SQL Server Management Studio (SSMS) to one server, so that the SQL you type goes to that server. |
| `Query` | A question or an instruction written in SQL, such as a request for the server's version. |
| `F5` | The key that runs the SQL in the window. It does the same as the Execute button. |
| `Script` | A file of SQL lines that the server runs from the top to the bottom. Its name ends in .sql. |
| `Visual Studio 2019` | The program you use to build the shop's windows and buttons, and the C# code behind them. |

| | |
|---|---|
| `C#` | The programming language you use in class, written C and a hash sign and said C sharp. |
| `Template` | A ready-made starting point that Visual Studio 2019 fills in when you make a new project. |
| `Project` | The set of files that make one program. |
| `Solution folder` | The folder Visual Studio 2019 makes for your project, with every file of the program inside it. |
| `bin and obj` | Two folders Visual Studio 2019 makes each time it builds the program. They are made again from your code, so you leave them out when you hand in. |
| `Zip` | Pack a folder into one smaller file that can be handed in. |

# Make your coursework folder in OneDrive.

Your coursework folder is called A2 SSD Coursework, and it sits inside OneDrive - C2k, your school OneDrive. OneDrive keeps a copy of everything in this folder on your school account. That means you can reach it from any computer you sign in on, and it is safe if one computer breaks. Every file you make in this guide goes in this folder: the script in chapter 1, the program in chapter 5, and your screenshots.

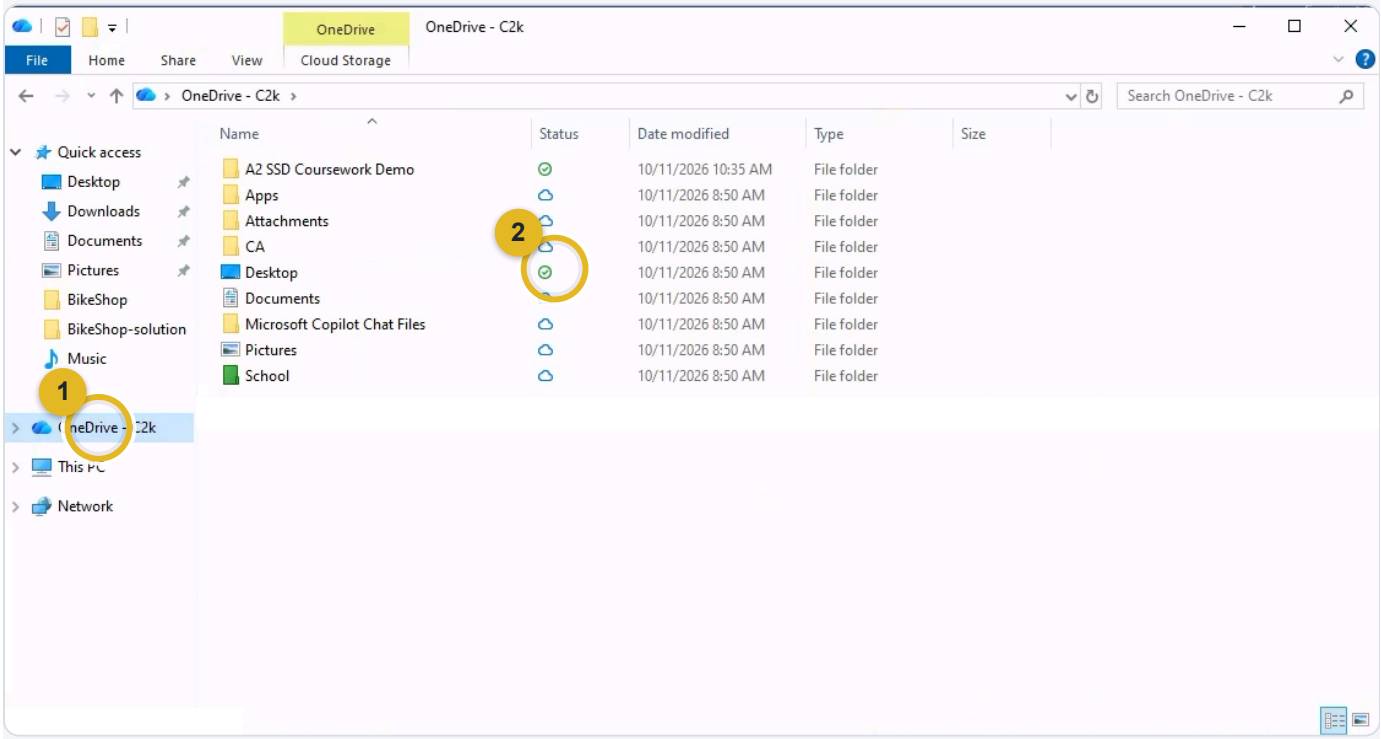
Press the Windows key and the E key together.

**You should see** File Explorer opens. It is a window with a list of places down its left side.

**IF IT GOES WRONG** Nothing opens

Click the yellow folder picture on the taskbar at the bottom of the screen instead. It opens File Explorer too.

Click OneDrive - C2k in the list down the left side of File Explorer.




File Explorer, open at your school OneDrive. Everything you make for this coursework goes in here.

1. Click OneDrive - C2k in the list down the left side.

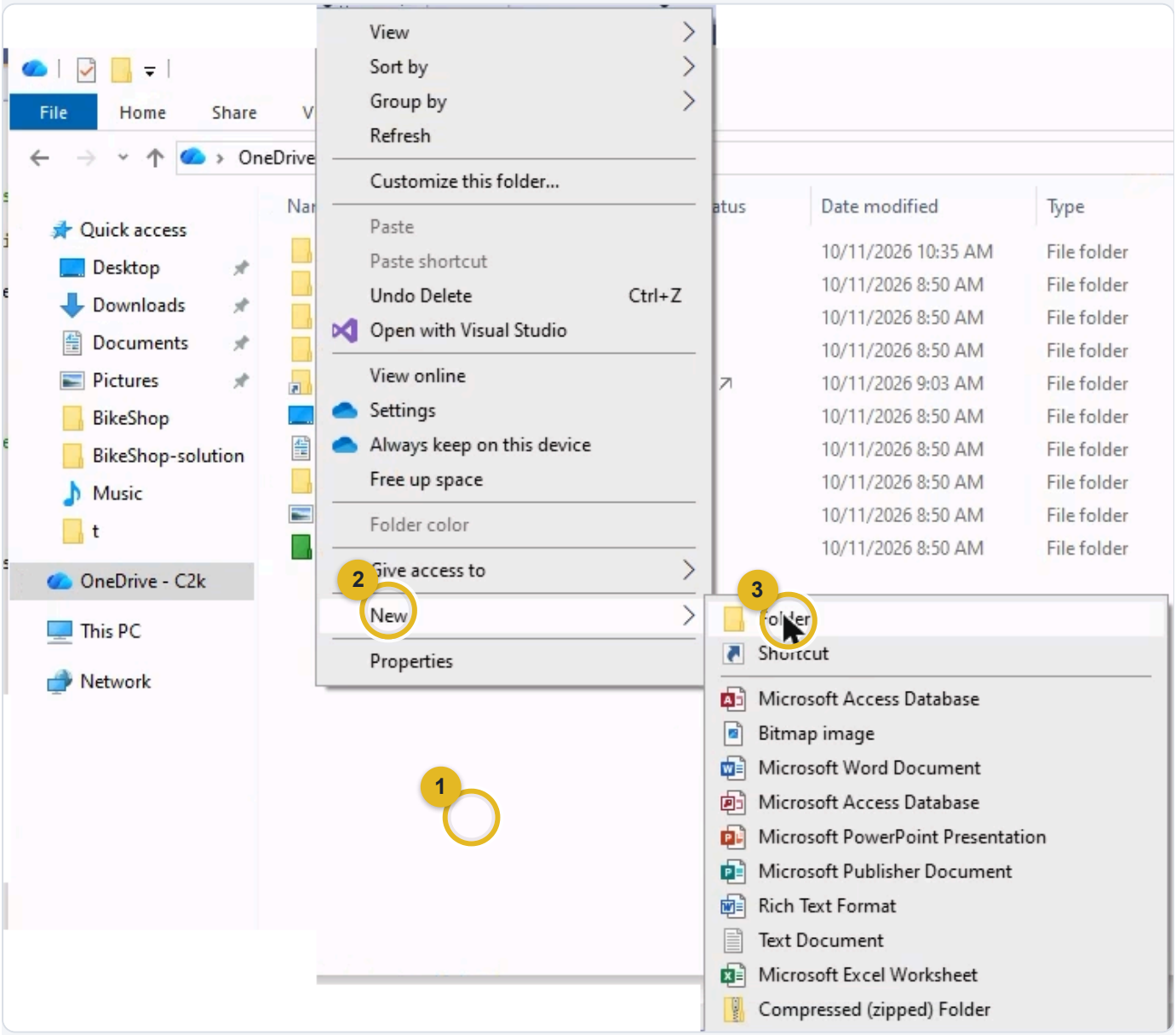
2. Your OneDrive files and folders are listed here.

**You should see** The right side lists the files and folders in your school OneDrive. It may be empty.

**IF IT GOES WRONG** There is no OneDrive - C2k in the list

OneDrive has not started yet. Wait one minute, close File Explorer, and open it again with the Windows key and E. If it is still missing, tell your teacher before you go on.

Right-click an empty white space on the right side, below any files.




Right-click an empty space in OneDrive - C2k, point at New, and click Folder.

1. Right-click an empty white space below the files.

2. Point at New.

3. Click Folder.

> **You should see** A menu opens, and New is one of the lines in it.

Point at New in the menu.

> **You should see** A small menu opens beside New, with Folder at the top of it.

Click Folder in the small menu.

> **You should see** A new folder appears, with its name, New folder, highlighted in blue and ready to be typed over.

Type A2 SSD Coursework and press Enter.

> **You should see** The folder is named A2 SSD Coursework, with a capital A, capital SSD and a capital C.

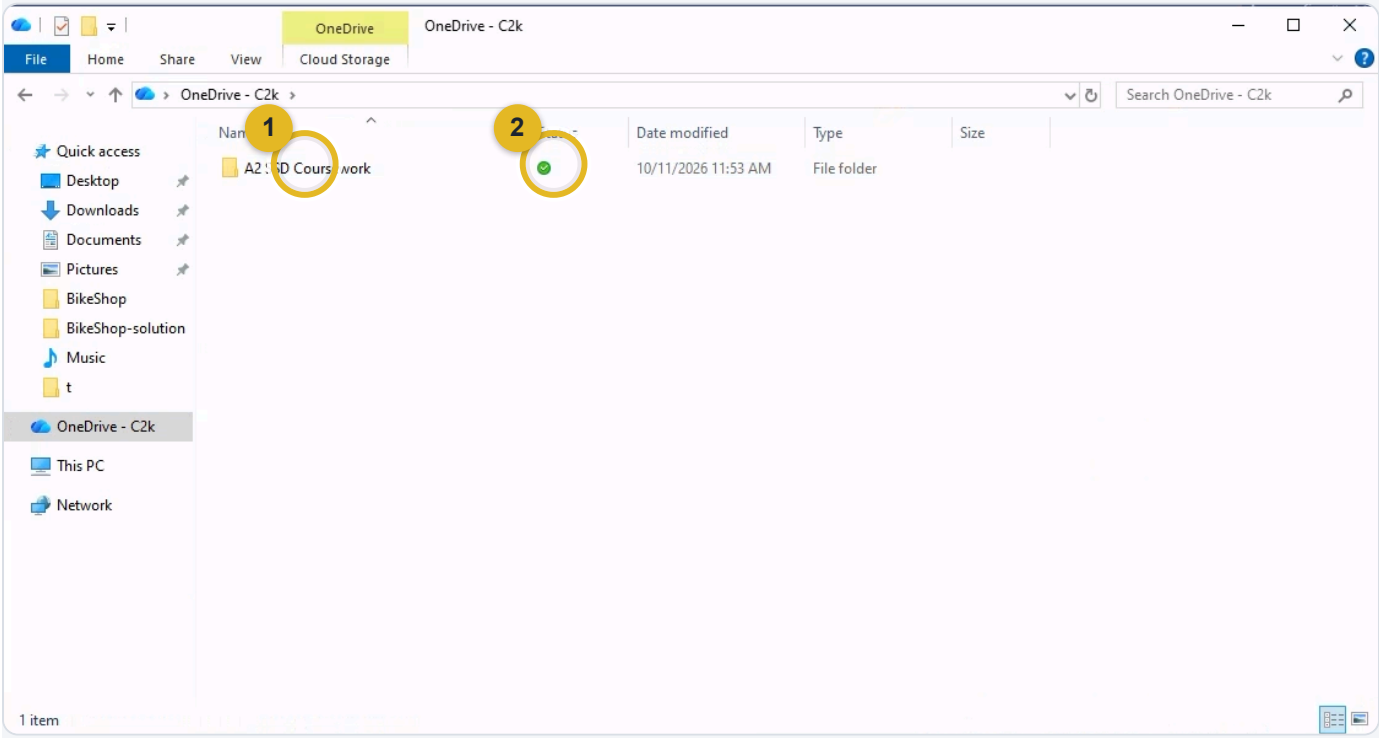
**IF IT GOES WRONG** The name is spelled wrongly

Right-click the folder, click Rename, type A2 SSD Coursework, and press Enter.

**IF IT GOES WRONG** The folder is still called New folder

The name was not highlighted when you typed. Right-click the folder, click Rename, type A2 SSD Coursework, and press Enter.

Wait until a green tick appears beside the folder, in the Status column.




Your coursework folder in OneDrive. The green tick means OneDrive has copied it to your school account.

1. The folder A2 SSD Coursework.

2. The green tick in the Status column.

**You should see** A green tick sits beside A2 SSD Coursework. OneDrive has copied the folder to your school account, so it will be there in every session.

**IF IT GOES WRONG** Two blue arrows going round in a circle

OneDrive is still copying the folder. Wait a few seconds, and the arrows turn into a green tick.

**IF IT GOES WRONG** A red cross beside the folder

OneDrive could not copy the folder. Check that you are still signed in to OneDrive, then tell your teacher, because your work will not be safe until the tick is green.

# Open SQL Server Management Studio (SSMS) and connect it to the database server.

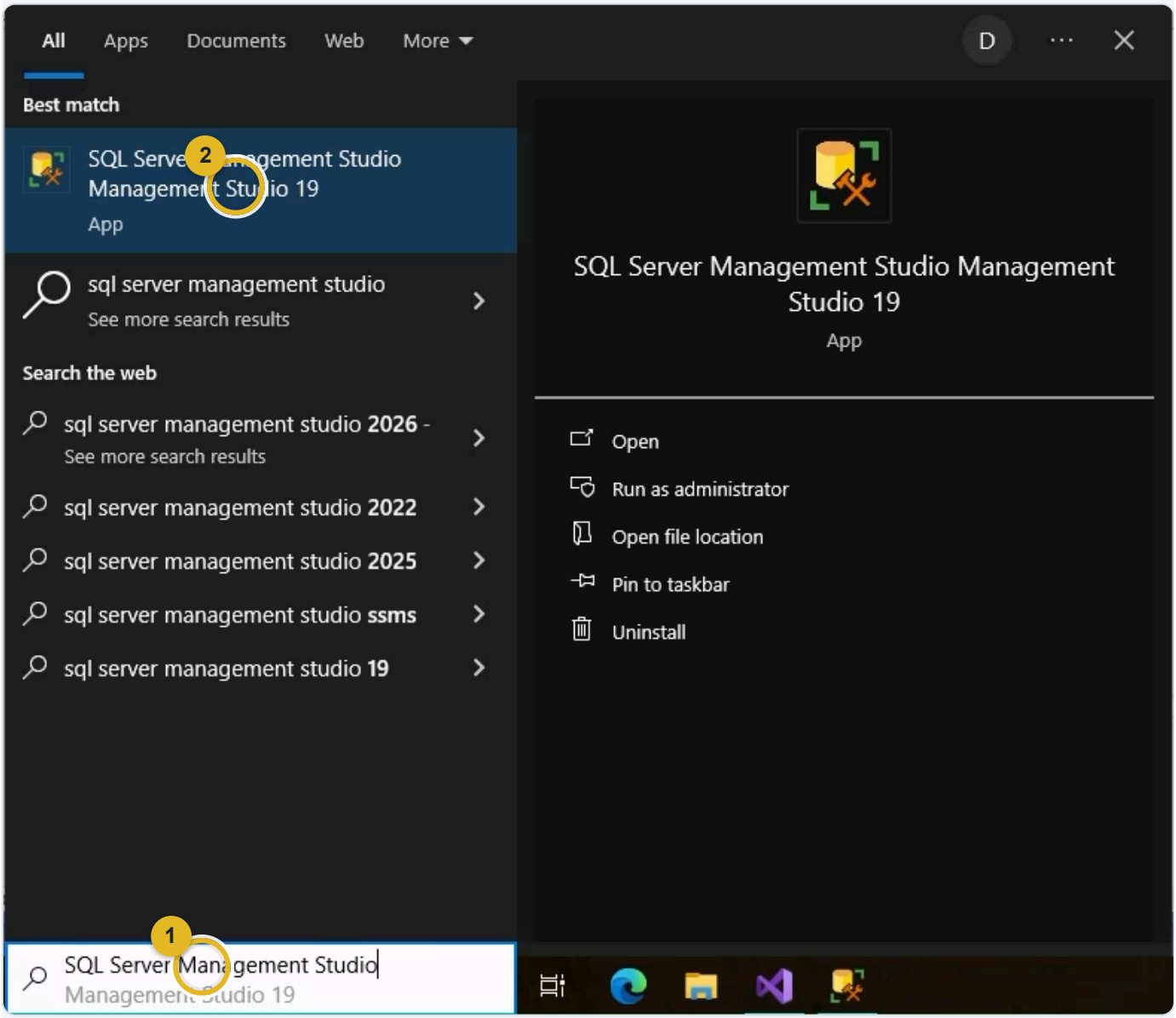
SQL Server is the program that keeps your database. It runs quietly in the background and has no window of its own. SQL Server Management Studio (SSMS) is the window you use to talk to it: you type SQL, press F5, and the server's answer appears underneath.

The server in the Programming Environment is called `(localdb)\mssqllocaldb`. It is LocalDB, a small copy of SQL Server that starts by itself when SQL Server Management Studio (SSMS) connects to it. You type this name once each time you open SQL Server Management Studio (SSMS), so it is worth getting every character right: the brackets, the backslash, and no spaces.

Press the Windows key on the keyboard.

> **You should see** The Start menu opens, with a search box ready for typing.

Type SQL Server Management Studio.




Typing the name in the Start menu finds the app. Click the top result.

1. The search box, with the name typed in it.

2. Click SQL Server Management Studio at the top of the list.

**You should see** SQL Server Management Studio appears at the top of the list, with a version number after its name.

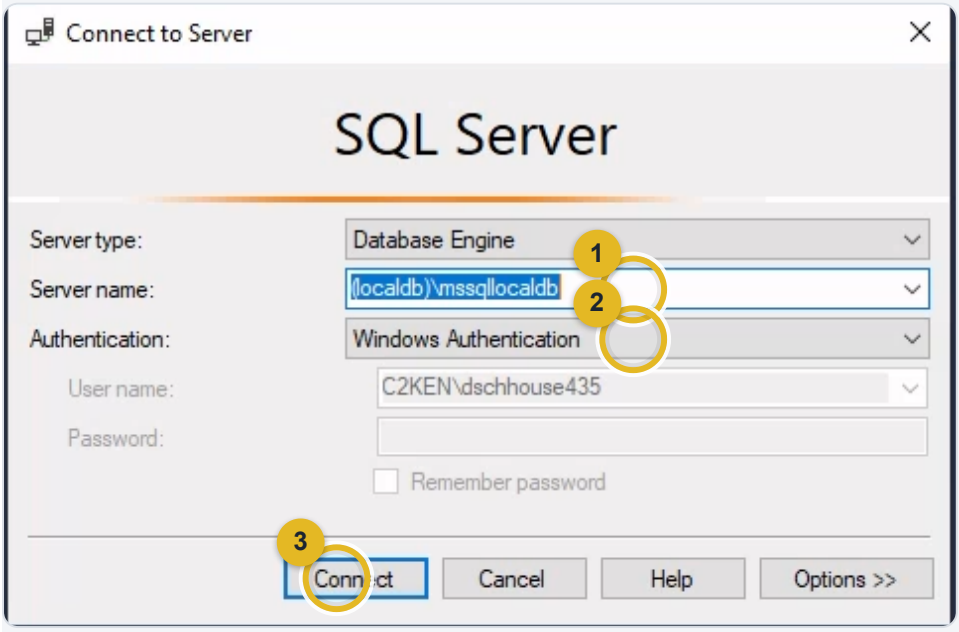
**IF IT GOES WRONG** Nothing called SQL Server Management Studio in the list

Check the spelling, then try typing Management Studio on its own as the search. If it is still missing, tell your teacher: the Programming Environment needs it for chapters 3 to 9.

Click SQL Server Management Studio at the top of the list.

> **You should see** SQL Server Management Studio (SSMS) opens. After a few seconds the Connect to Server box appears in the middle of it.

Triple-click the Server name box, so that anything already in it is highlighted.




The Connect to Server box. It opens every time you start SQL Server Management Studio (SSMS).

1. Type (localdb)\mssqllocaldb in the Server name box.
2. Leave Authentication as Windows Authentication.
3. Click Connect.

> **You should see** The text in the Server name box is highlighted in blue, or the box is empty.

Type `(localdb)\mssqllocaldb` in the Server name box.

> **You should see** The box reads `(localdb)\mssqllocaldb` : the word localdb in round brackets, a backslash, then mssqllocaldb, with no spaces.

Look at the Authentication box, under the Server name box.

> **You should see** Authentication says Windows Authentication. That means the server knows you by the account you signed in with, so there is no password to type.

**IF IT GOES WRONG** It says SQL Server Authentication

Click the Authentication box and choose Windows Authentication from the list.

Click the Connect button.

**You should see** The Connect to Server box closes. Object Explorer, down the left side, shows `(localdb)\mssqllocaldb` at the top with a Databases folder under it.

**IF IT GOES WRONG** A red box that says 'Cannot connect to (localdb)\mssqllocaldb'

LocalDB can take a few seconds to start the first time. Click OK, wait ten seconds, and click Connect again. If it still fails, compare the Server name box with `(localdb)\mssqllocaldb`, character by character.

**IF IT GOES WRONG** A red box that names a network-related or instance-specific error

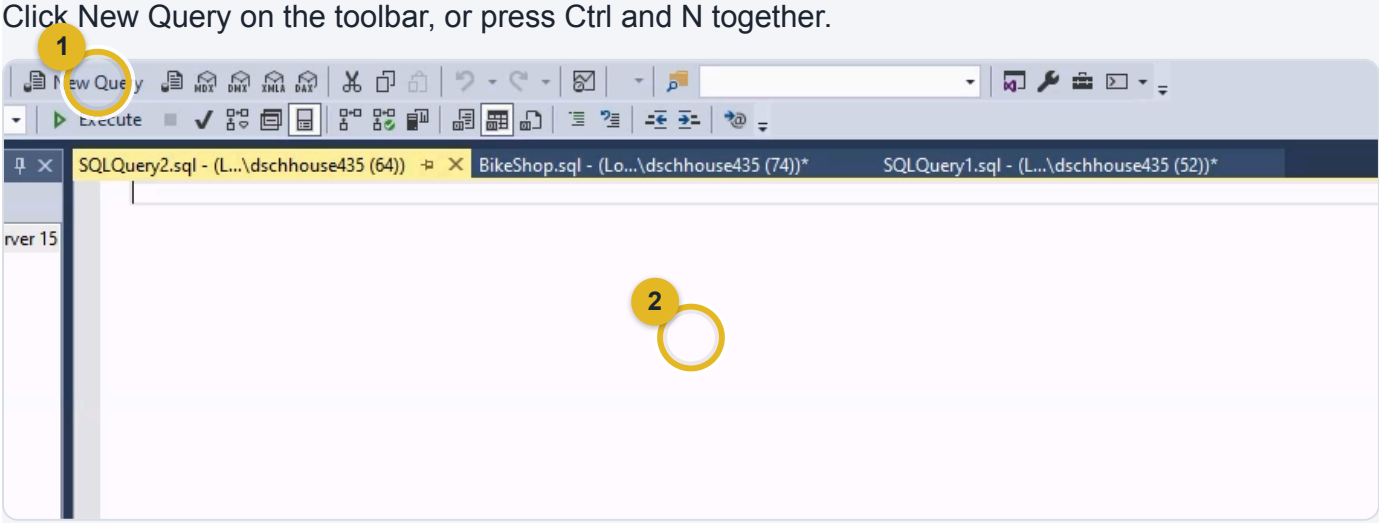
The server name is spelled wrongly, so SQL Server Management Studio (SSMS) looked for a server that does not exist. Click OK, and retype the name exactly as `(localdb)\mssqllocaldb`.

# Run your first line of SQL, which asks the server for its version.

A query is a question or an instruction written in SQL. You type it in a query window and press F5 to run it. Your first query is one line: `SELECT @@VERSION;` Type it exactly, with two at signs and a semicolon at the end.

Each part of the line has a job. SELECT means show me. @@VERSION is a value the server always knows about itself: its own name and version number. The two at signs mark it as one of the server's own values. The semicolon ends the instruction, the way a full stop ends a sentence. If one row comes back, SQL Server Management Studio (SSMS) is connected and the server is answering.

Click New Query on the toolbar, or press Ctrl and N together.




New Query opens an empty window where you type SQL.

1. Click New Query, or press Ctrl and N together.

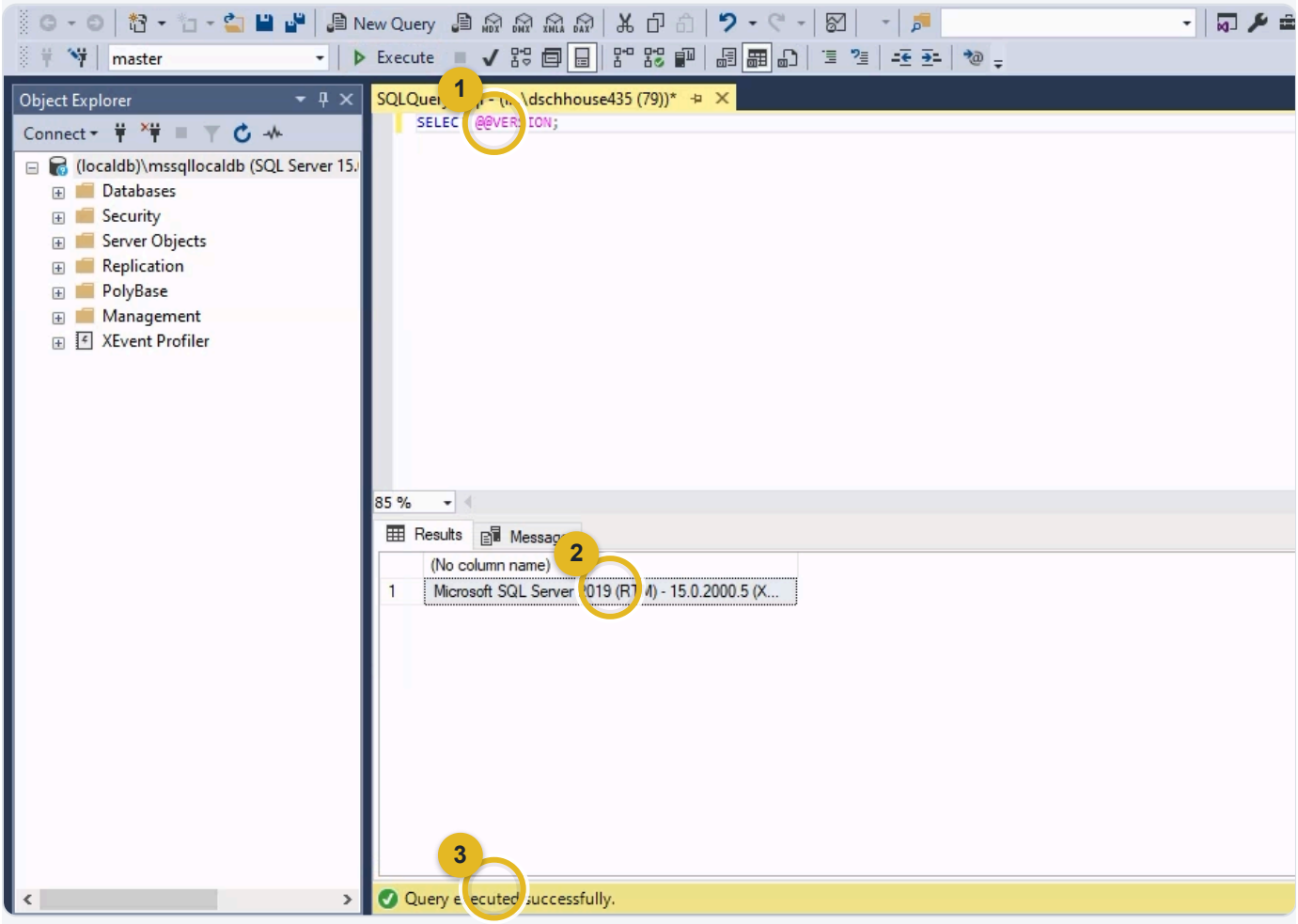
2. Type into the empty window.

> **You should see** A tab named SQLQuery1.sql opens with an empty white window, and the cursor blinks in it.

Type `SELECT @@VERSION;` in the empty window.

> **You should see** The line reads `SELECT @@VERSION;` with SELECT coloured blue and @@VERSION coloured pink.

Press F5 to run the query.




Your first query and its answer. One row came back from the server, so SQL Server Management Studio (SSMS) is connected and working.

1. The line you typed.
2. One row, starting Microsoft SQL Server.
3. Query executed successfully.

**You should see** A Results tab opens under the window with one row. Its one box holds a long line that starts with Microsoft SQL Server, then a year and a version number. The bar at the bottom says Query executed successfully.

**IF IT GOES WRONG** Red text that says 'Must declare the scalar variable'

@@VERSION is spelled wrongly, so the server does not recognise it. Check that the line has two at signs and the whole word VERSION, then press F5 again.

**IF IT GOES WRONG** Red text that says 'Incorrect syntax near'

A character is missing or extra. Compare your line with `SELECT @@VERSION;` character by character, then

press F5 again.

# Empty the window and save it as BikeShop.sql in your folder.

This query window becomes your script. A script is a file of SQL lines that the server runs from the top to the bottom, and its name ends in .sql. In chapter 3 you fill it with the tables of your database, so it starts empty. Clear the test line out of the window, then save the empty window into your OneDrive folder as BikeShop.sql.

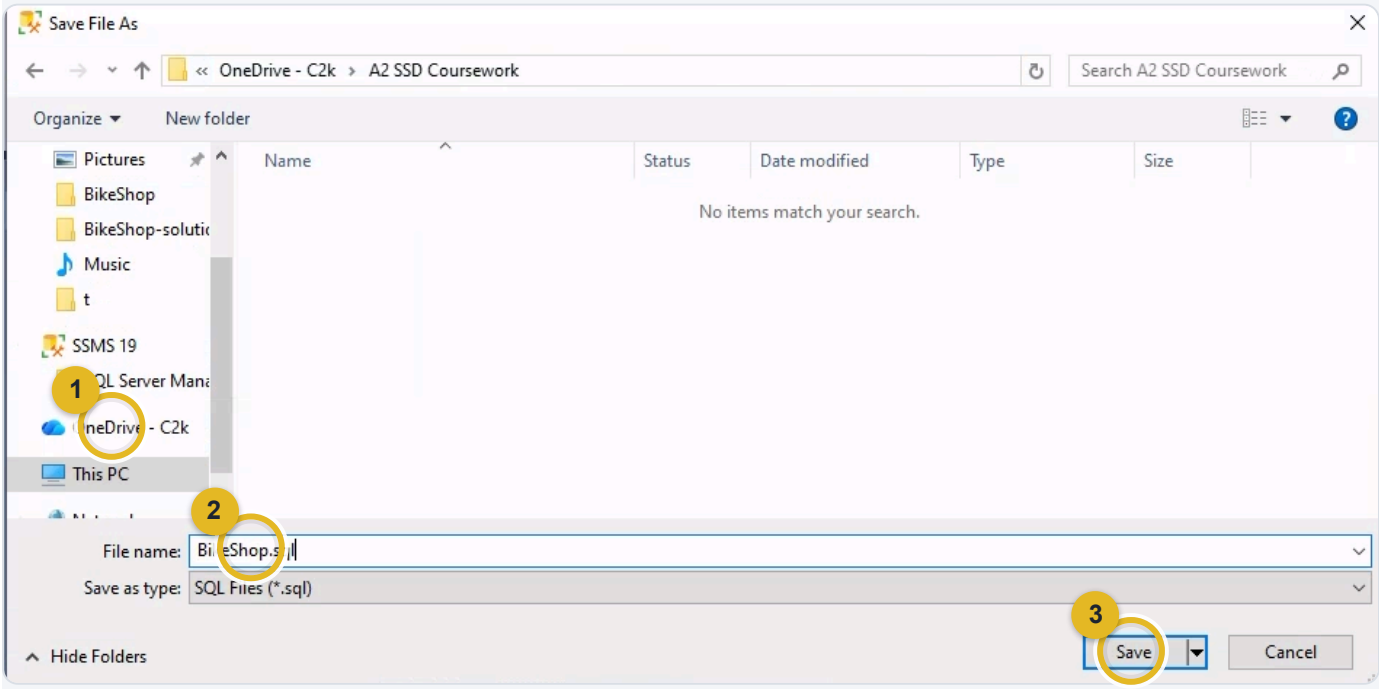
Press Ctrl and A together, with the cursor in the query window.

> **You should see** The SELECT line is highlighted in blue.

Press the Delete key.

> **You should see** The query window is empty. The Results tab underneath stays until you run something else.

Press Ctrl and S together to save the window.




Saving the empty script into your OneDrive coursework folder, so that it is there in every session.

1. Click OneDrive - C2k on the left.
2. Type BikeShop.sql in the File name box.
3. Click Save.

> **You should see** The Save File As box opens, because this window has never been saved before.

Click OneDrive - C2k in the list down the left side of the Save File As box.

> **You should see** The middle of the box lists your OneDrive, and A2 SSD Coursework is in the list.

Double-click the A2 SSD Coursework folder.

> **You should see** The box opens inside A2 SSD Coursework, and the folder is empty.

Triple-click the File name box, so that the name SQLQuery1.sql is highlighted.

> **You should see** SQLQuery1.sql is highlighted in blue in the File name box.

Type BikeShop.sql in the File name box.

> **You should see** The File name box reads BikeShop.sql, with a capital B and a capital S.

Click the Save button.

**You should see** The Save File As box closes, and the tab at the top of the query window is now named BikeShop.sql.

**IF IT GOES WRONG** The file was saved in the wrong folder

Click File at the top left, then Save BikeShop.sql As…, and save it again into A2 SSD Coursework. Then delete the copy in the wrong folder with File Explorer.

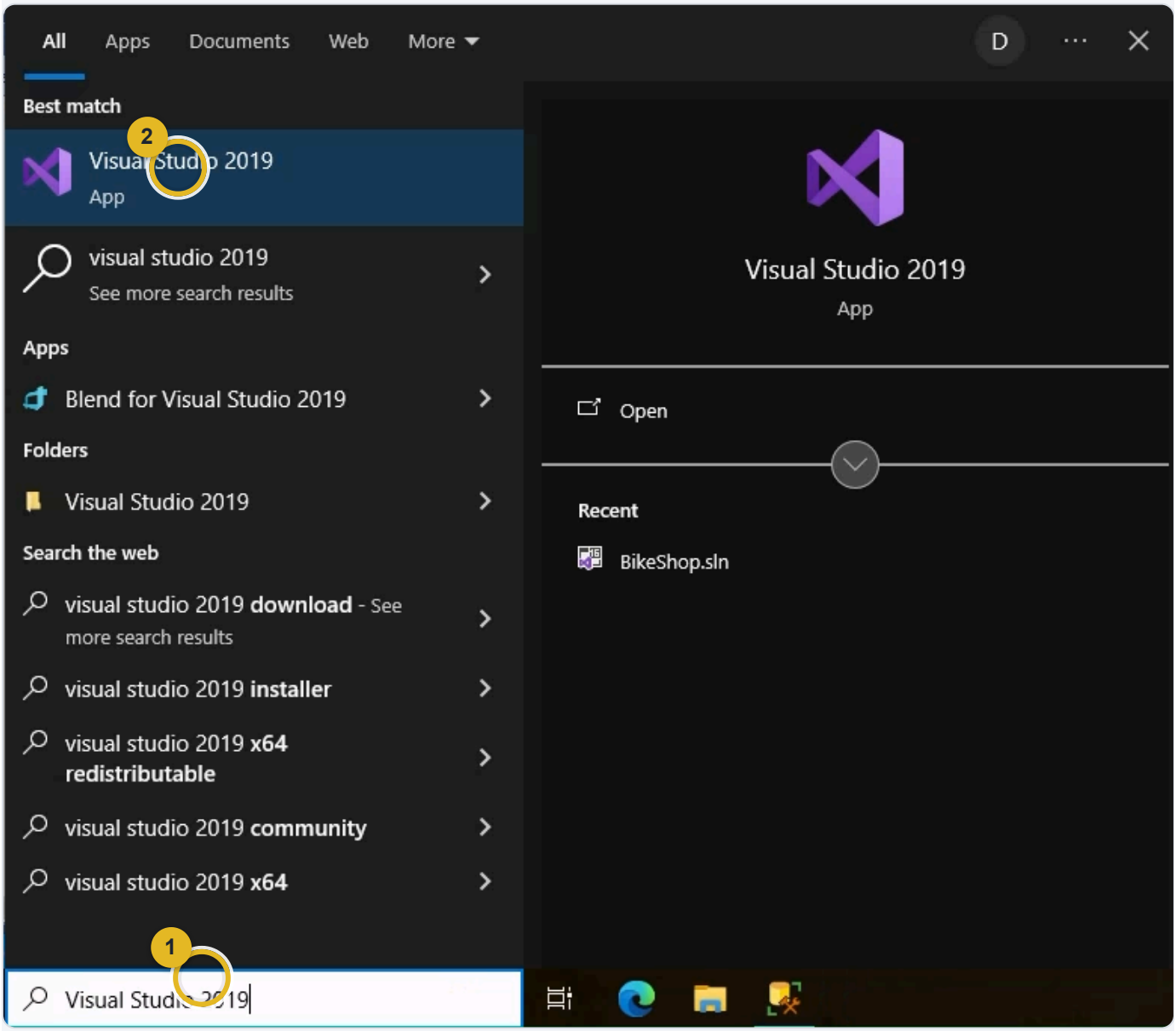
# Open Visual Studio 2019 and check that the Windows Forms template is there.

Visual Studio 2019 is where you build the program that the shop's staff use: the windows, the buttons and the C# code behind them. You make that program in chapter 5. Today you check one thing. The Programming Environment must have the template this guide uses, which is Windows Forms App (.NET Framework) for C#. A template is a ready-made starting point that Visual Studio 2019 fills in for a new project. You look for it, then go back without making anything.

Press the Windows key on the keyboard.

> **You should see** The Start menu opens, with a search box ready for typing.

Type Visual Studio 2019.




The same search finds Visual Studio 2019. Click the top result.

1. The search box, with the name typed in it.

2. Click Visual Studio 2019 at the top of the list.

> **You should see** Visual Studio 2019 appears at the top of the list.

Click Visual Studio 2019 at the top of the list.

**You should see** Visual Studio 2019 opens its start window, with Create a new project on the right.

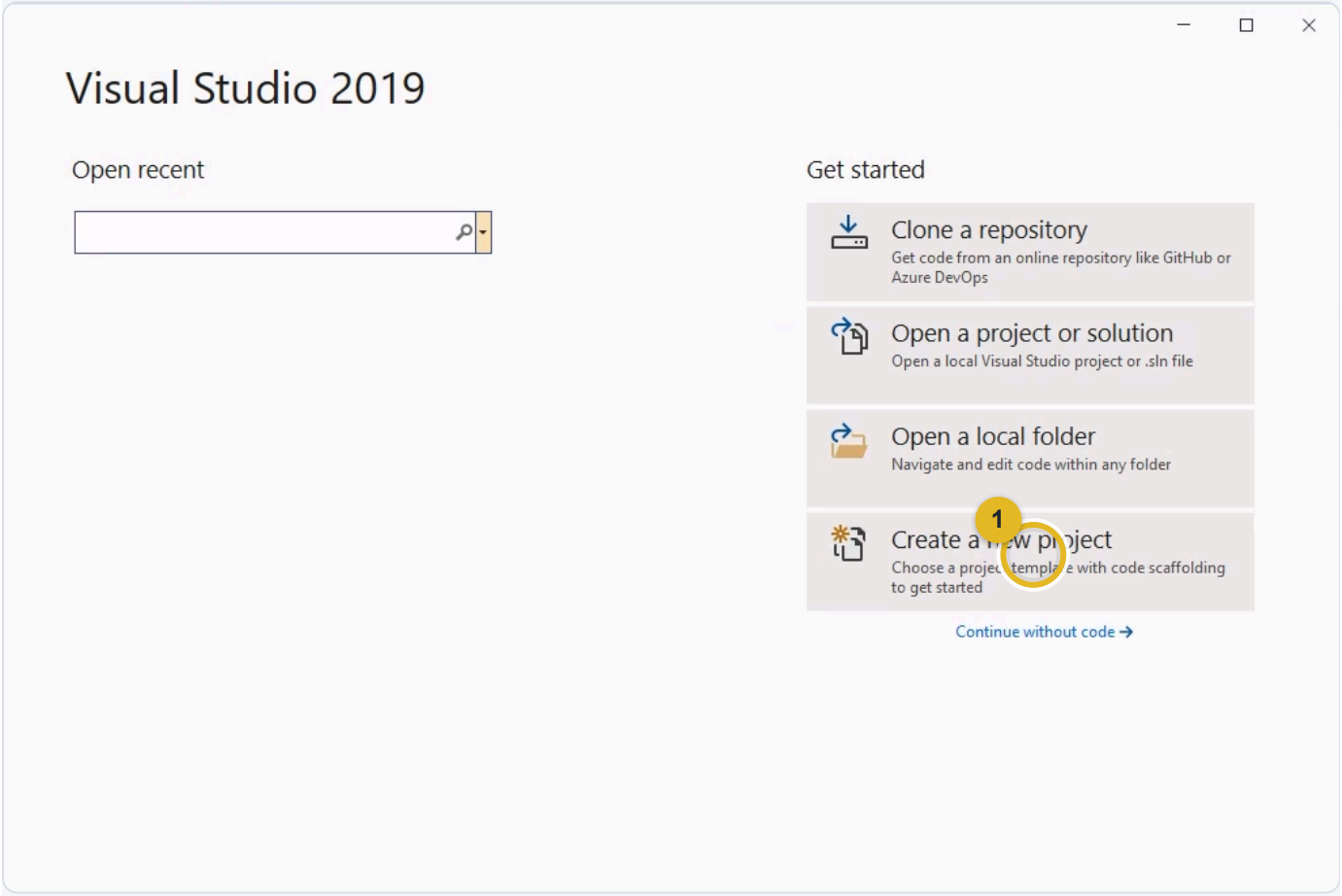
**IF IT GOES WRONG** A box asks you to sign in with a Microsoft account

Click Not now, maybe later. You do not need an account for anything in this guide.

**IF IT GOES WRONG** A box asks you to choose a colour theme

Click the blue button at the bottom of the box, whose name begins with the word Start. Any theme works with this guide.

Click Create a new project.

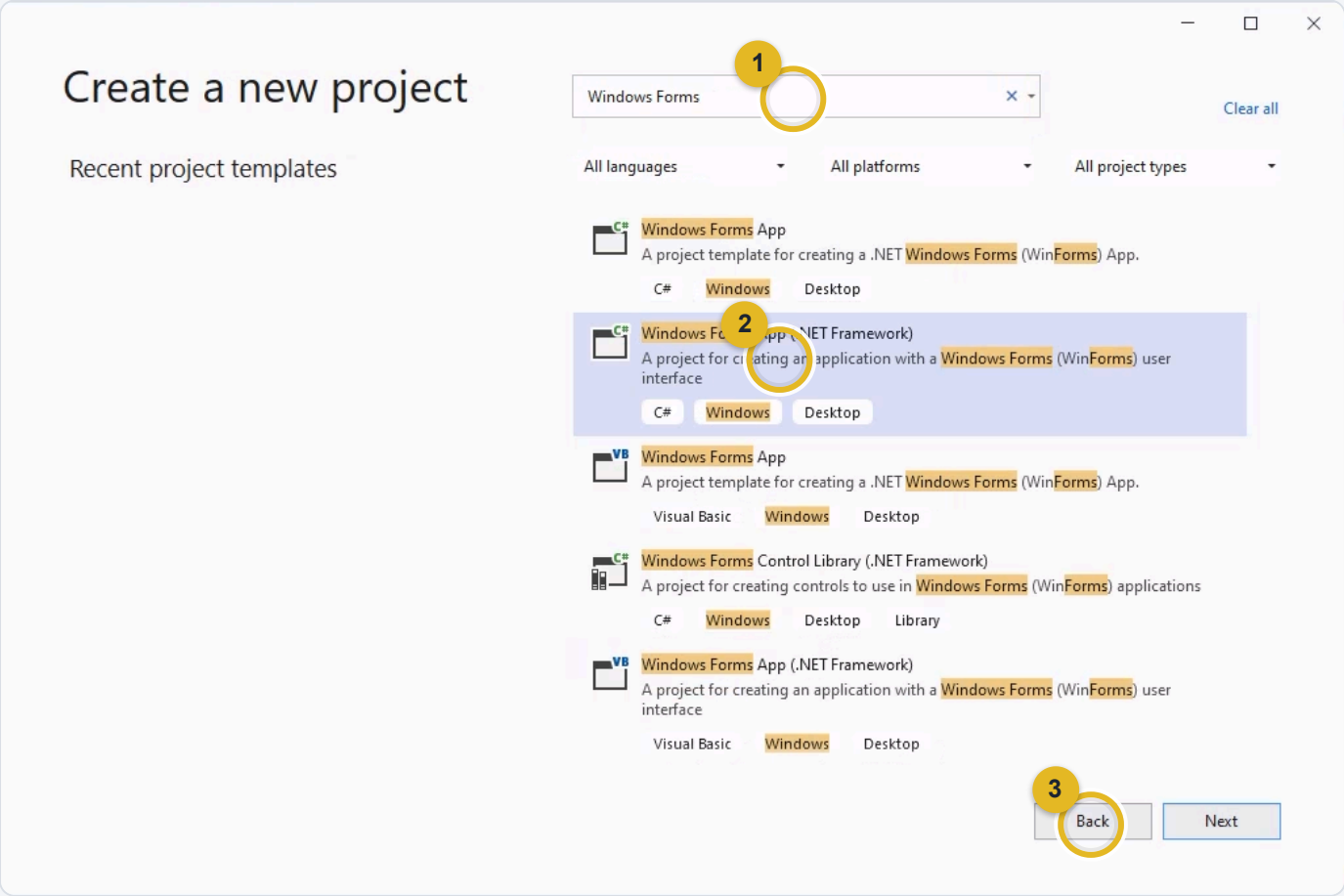



The start window of Visual Studio 2019. It opens every time you start the program.

1. Click Create a new project.

**You should see** The Create a new project window opens, with a search box at the top and a long list of templates.

Type Windows Forms in the search box at the top.




Windows Forms App (.NET Framework), the one marked C#, is the template this guide uses. The one without (.NET Framework) is a different kind of project.

1. Type Windows Forms in the search box.
2. Windows Forms App (.NET Framework), with C# under it.
3. Click Back. Today you only look.

**You should see** The list shrinks to the Windows Forms templates. One of them is Windows Forms App (.NET Framework), with C#, Windows and Desktop under its name.

**IF IT GOES WRONG** There is no Windows Forms App (.NET Framework) in the list

The Programming Environment is missing the part of Visual Studio 2019 that makes Windows programs. Tell your teacher now. Chapters 2, 3 and 4 do not need Visual Studio 2019, so you can carry on with those while it is fixed.

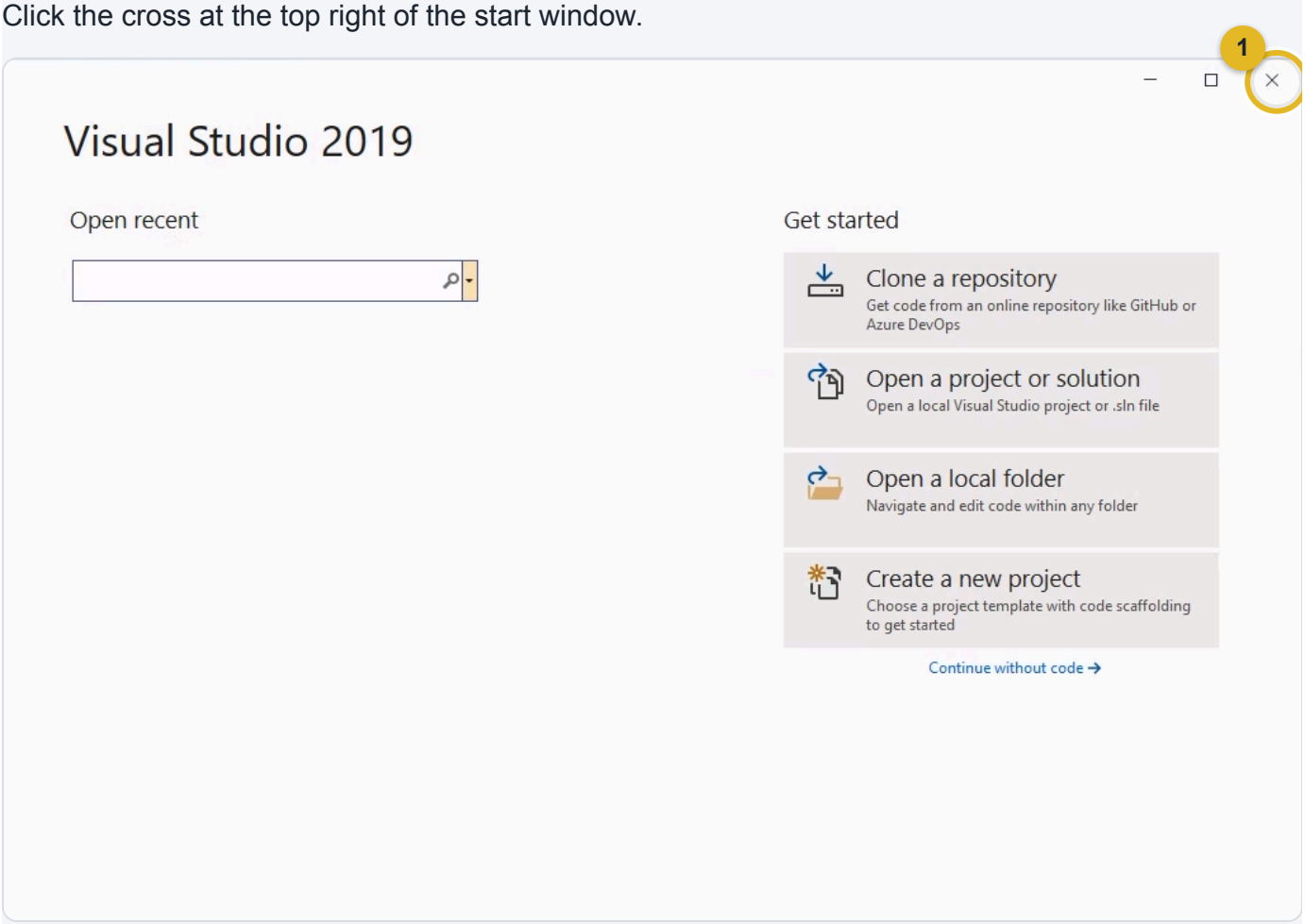
**IF IT GOES WRONG** Only Windows Forms App, without (.NET Framework)

Scroll down the list. The template with (.NET Framework) in its name is a different one, and it is the one this guide uses.

Click Back at the bottom right, without choosing a template.

**You should see** The start window comes back, and no project has been made.

Click the cross at the top right of the start window.




Back returns you to the start window. The cross at its top right closes it.

1. Click the cross at the top right.

**You should see** Visual Studio 2019 closes.

**IF IT GOES WRONG** A large empty Visual Studio 2019 window is still open

Closing the start window can leave the main window open with nothing in it. Click the cross at its top right, and Visual Studio 2019 closes.

# Read the list of what you hand in at the end.

At the end of the coursework you hand in five things. Every one of them is made in your A2 SSD Coursework folder, so nothing has to be found or copied at the last minute.

**The five things you hand in**

The first is BikeShop.sql, the script that makes your database. The second is the BikeShop solution folder from Visual Studio 2019, zipped into one file, with its bin and obj folders left out. The third is your write-up. The fourth is your user guide. The fifth is your test evidence, which is the screenshots you take at the end of each chapter.

Visual Studio 2019 makes the bin and obj folders every time it builds your program. They are large, and they are made again from your code, so the examiner does not need them. You zip the solution folder once, at the very end, when the program is finished.

# Read when to run your script.

From chapter 3 onward, BikeShop.sql makes your whole database: the tables, the sample data, and later the stored procedures, which are named blocks of SQL saved in the database. The database it makes lives on the server `(localdb)\mssqllocaldb` inside the Programming Environment, not in your OneDrive folder. The script is the part that lives in OneDrive.

When you sign out of the Programming Environment, the database on the server may be gone the next time you sign in. The box below says what to do about it, and it is the same for every chapter.

**When to run your script**

BikeShop.sql is the master copy of your database. Run it at the start of every session; it takes two seconds.

# Read what changes if you work at home.

Some pupils install SQL Server Express, SQL Server Management Studio (SSMS) and Visual Studio 2019 on a computer at home. If you do, one thing changes, which is the server name. At home the server is your own, such as `.\SQLEXPRESS`. The full stop before the backslash means this computer, and SQLEXPRESS is the name SQL Server Express gives itself when it installs.

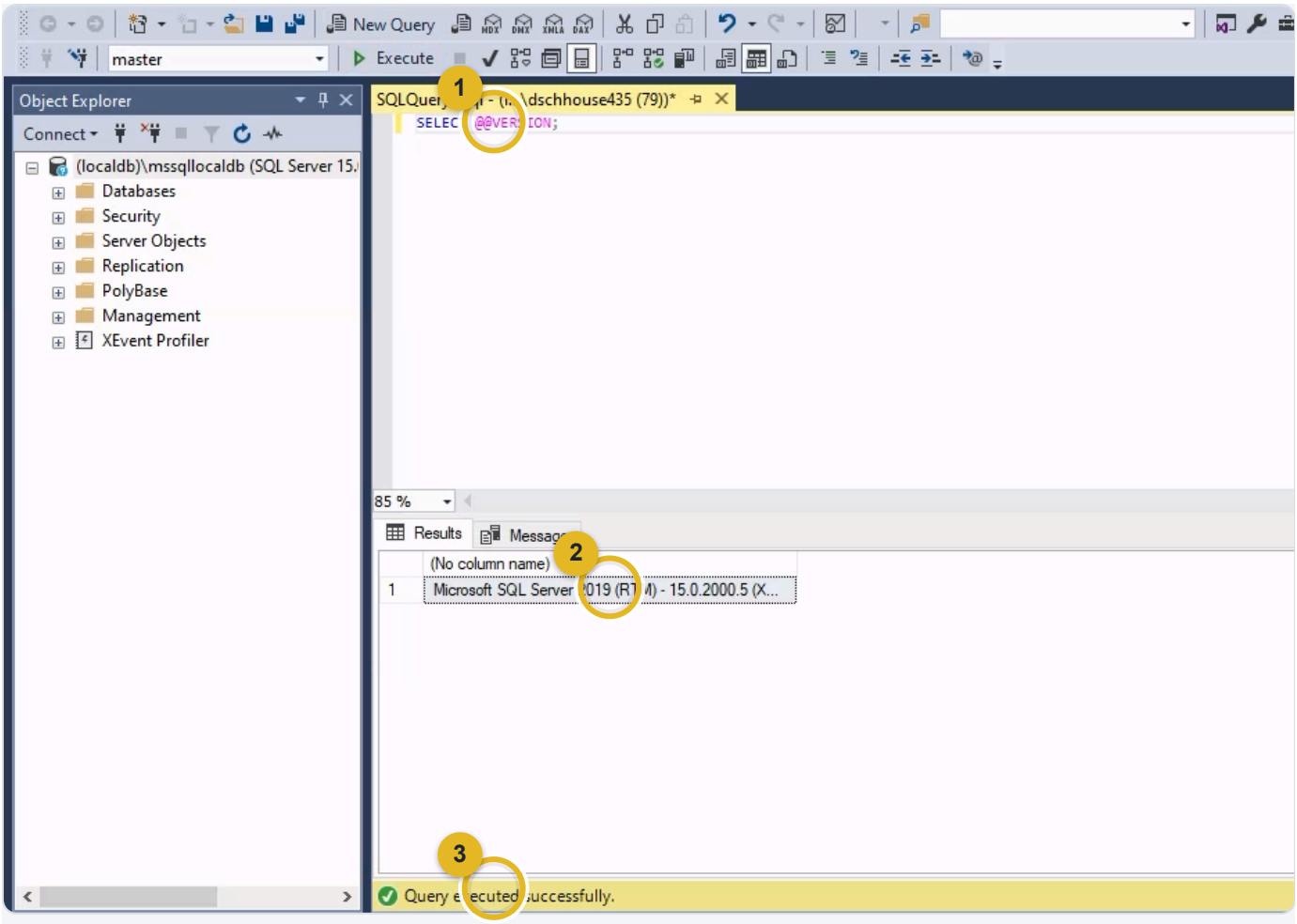
**If you work at home**

Type your own server name in the Server name box when SQL Server Management Studio (SSMS) asks you to connect. In chapter 5 you write the server name a second time, in a settings file called App.config, and at home you change it there too. The database at home is a separate one, so run BikeShop.sql there to make it. Nothing else changes: the script, the code and the steps in this guide are the same.

# Tick the table: the folder, the script, the server and the template.

Every row below is a check you do now, in the Programming Environment. Two rows are mistakes made on purpose, and a refusal there is a pass, not a fail: it proves that SQL Server Management Studio (SSMS) is talking to a real server that checks what you send it.

| | THE TEST | WHAT MUST HAPPEN | HOW TO DO IT, AND WHAT YOU SHOULD SEE |
|---|---|---|---|
| | Your coursework folder is in OneDrive with a green tick | **Must be allowed** | Press the Windows key and E together, and click OneDrive - C2k on the left. You see A2 SSD Coursework in the list, with a green tick in the Status column. |
| | BikeShop.sql is saved in the folder | **Must be allowed** | Double-click A2 SSD Coursework in File Explorer. You see BikeShop.sql in it, with a green tick beside it. |
| | A wrong server name is refused | **Must be refused** | Close SQL Server Management Studio (SSMS) with the cross at its top right. Start it again from the Start menu. In the Server name box type `(localdb)\mssqllocal`, with the last two letters missing, and click Connect. After a few seconds a red box names a network-related or instance-specific error, and nothing connects. Click OK. |
| | The right server name connects | **Must be allowed** | In the same Connect to Server box, type `(localdb)\mssqllocaldb` in the Server name box and click Connect. Object Explorer shows `(localdb)\mssqllocaldb` with a Databases folder under it. |
| | The server answers your first query | **Must be allowed** | Click New Query, type `SELECT @@VERSION;` and press F5. The Results tab shows one row that starts with Microsoft SQL Server. |
| | A misspelled query is refused | **Must be refused** | Press Ctrl and A in the same window, type `SELECT @@VERSIO;` with the N missing, and press F5. The Messages tab shows red text: Must declare the scalar variable "@@VERSIO". Nothing else happens. |
| | The Windows Forms template is there | **Must be allowed** | Start Visual Studio 2019, click Create a new project, and type Windows Forms in the search box. You see Windows Forms App (.NET Framework) with C# under its name. Click Back and close Visual Studio 2019. |




Your first query and its answer. One row came back from the server, so SQL Server Management Studio (SSMS) is connected and working.

1. The line you typed.
2. One row, starting Microsoft SQL Server.
3. Query executed successfully.

# Finish the chapter.

Seven ticks mean your workspace is ready. Close the query window without saving it, because the test lines in it are not part of your script. Your one saved file, BikeShop.sql, is empty and waiting in your OneDrive folder. Chapter 3 opens it and fills it with the tables.

**Before you start**

- A2 SSD Coursework is in your OneDrive, with a green tick.
- BikeShop.sql is saved in A2 SSD Coursework, and it is empty.
- SQL Server Management Studio (SSMS) connects to `(localdb)\mssqllocaldb`, and `SELECT @@VERSION;` returns one row.
- Visual Studio 2019 has the Windows Forms App (.NET Framework) template for C#.

**Make it yours**

Your own scenario uses the same folder and the same server. When you build your own database in place of the bike shop, give the script your scenario's name, such as DogGroomer.sql, and save it in A2 SSD Coursework beside BikeShop.sql.

CHAPTER 2 OF 12

# Plan the tables

From the scenario to tables, keys and the columns each one needs.

STAGE 1 OF 5 · FROM THE STORY TO THE TABLES · SCREEN 1 OF 3

## See what you will plan in this chapter, and learn the words it uses.

This chapter uses no software. You read the story of a bike shop, pick out the things the shop needs to remember, and plan the tables that will hold them. By the end of the chapter you have a drawing of ten tables, with their columns and the lines that join them. In chapter 3 you type that plan into SQL Server Management Studio (SSMS), and the server makes the tables.

Planning first saves you hours. A table that is wrong on paper takes a minute to fix with a pencil. The same mistake found after the program is written means changing the table, the stored procedures and the screens.

**Before you start**

- Chapter 1 is finished: your OneDrive coursework folder exists, and SQL Server Management Studio (SSMS) has run SELECT @@VERSION once.
- You have a pen and two sheets of paper. One sheet is for the Towpath Cycles plan as you read it, and one is for your own case study at the end.
- This guide is open in the Edge browser. Tick each step as you do it, and the chapter remembers where you are.

**Words used in this chapter**

| | |
|---|---|
| `Stored procedure` | A saved set of instructions the database runs when your program asks, such as adding a booking. |
| `Database` | A store of tables kept by the server, such as BikeShop, which your program reads and changes. |
| `Noun` | A naming word for a person, place or thing, such as customer, bike or booking. |
| `Table` | A grid with named columns, where each row is one thing, such as one customer. |
| `Row` | One line of a table: one customer, one bike or one booking. |
| `Column` | One named box in every row, such as Phone, with a type that says what can go in it. |
| `Booking` | A row that holds a person or a thing for a time, such as a mechanic from 13:00 to 13:30. |

| | |
|---|---|
| Stock | The things a shop keeps to sell, such as inner tubes on a shelf or bikes on the floor. |
| Primary key | The column that holds a different number for every row, so that each row can be named by its number. On a plan it is marked PK. |
| Foreign key | A column that holds the number of a row in another table, such as the customer a booking is for. On a plan it is marked FK. |
| Parent and child | The table being pointed at is the parent, such as Customer. The table that holds the foreign key is the child, such as Appointment. |
| Type | What a column may hold: a whole number, text, money, a date, or a yes or no. |
| INT | A type for whole numbers, such as 3 or 120. |
| VARCHAR | A type for text, with the most letters allowed written in brackets, such as VARCHAR(40). |
| DECIMAL | A type for money: DECIMAL(8,2) allows eight digits, two of them after the point. |
| FLOAT | A type for very large or very small numbers that stores a close guess, so it is never used for money. |
| DATE and DATETIME | DATE holds a day, such as 12 October 2026. DATETIME holds a day and a clock time together, such as 12 October 2026 at 13:00. |
| BIT | A type that holds a yes or a no, written as 1 or 0. |
| NULL and NOT NULL | NULL is an empty box. NOT NULL on a column means the box can never be left empty. |
| CHECK | A rule written into the table that refuses a row that breaks it, before any program runs. |
| UNIQUE | A rule that no two rows can hold the same value in that column, such as two staff with one username. |

# Read the story of Towpath Cycles, the shop this guide builds.

Every case study the examiner sets is a short story about a business. The story for this guide is about Towpath Cycles, a bike shop beside the canal towpath in Newry. Read the story below once from start to end. Then read it again with a pen in your hand.

**Towpath Cycles, Newry**

Towpath Cycles is a bike workshop and hire shop beside the canal in Newry. Customers book their own bikes in for a service, such as a puncture repair or a full service, and each service takes a set number of minutes and has a set price. Each service appointment is with one mechanic, at a start time and an end time between 08:30 and 17:30, in steps of 15 minutes. The shop also hires out bikes by the day: a customer takes one bike from a start date to an end date and pays a deposit of 20 per cent. Other bikes are for sale, and each one has its own frame number, so it can be reserved for one customer and sold only once. The shop keeps parts on its shelves, such as inner tubes and brake pads, and it orders more from its suppliers when the count runs low. Every sale of a part takes it off the shelf, and every delivery from a supplier puts more on. Staff log in as a mechanic, a counter assistant or the manager, and the shop records who made each booking, order and sale.

Two rules matter more than anything else in the story. One mechanic can never be booked twice at the same time, and one bike can never be hired twice on the same day. The count of each part on the shelf must always be right, and it can never go below zero.

# Pick out the nouns, and keep the ones the shop has many of.

A table holds one kind of thing, and each row is one of those things. So the tables hide inside the nouns of the story. Underline every noun in the Towpath Cycles story, then ask the same question of each one: does the shop keep many of these, each with its own details?

A customer passes the test. The shop has many customers, and each has a first name, a last name and a phone number, so Customer is a table. Mechanic passes too, but a mechanic is one kind of member of staff, and so are the counter assistant and the manager. All three go in one Staff table, with a column called Role that says which kind each person is.

Three kinds of noun do not become tables. Towpath Cycles, the canal and Newry are one of each, so a table for any of them would have only one row. A phone number, a price and a frame number are details of something else, so each becomes a column in that thing's table. A service such as a puncture repair is a table, because the shop has seven of them, each with its own minutes and price.

**The ten tables that come out of the story**

Customer, Staff, ServiceType, Appointment, Hire, Bike, Part, Supplier, PartOrder and PartSale. Each name is one word with no spaces, and each is singular: Customer, not Customers, because each row is one customer. PartOrder records one order of a part from a supplier, and PartSale records one sale of a part to a customer.

# Learn the two booking shapes: booked by time, and booked by day.

Almost every case study has something that is booked. Towpath Cycles has both of the two kinds, so that you see each one once.

**Booked by time: Appointment**

A service appointment books a person, the mechanic, for part of one day. Each row holds a start time and an end time, such as 13:00 to 13:30, so its type is DATETIME, a day and a clock time together. A clash is two appointments for the same mechanic whose times overlap. In your case study the same shape might be a driving lesson, a dentist's appointment or a room booked by the hour.

**Booked by day: Hire**

A hire books a thing, one bike, for whole days. Each row holds a start date and an end date, the day the bike comes back, so its type is DATE. A clash is two hires of the same bike whose days overlap. In your case study the same shape might be a holiday cottage, a hired van or a hotel room.

**RULE THAT NEVER BREAKS · NO DOUBLE BOOKING**

One mechanic can never be booked twice at the same time, and one bike can never be hired twice on the same day. Chapter 7 builds the check that refuses a double booking.

# Learn the two stock shapes: counted stock, and one-off stock.

Almost every case study has stock too. Again there are two kinds, and Towpath Cycles has both.

**Counted stock: Part**

An inner tube is one of many that are all the same, so the shop keeps a count of them, not a row for each tube. The Part table has a column called QtyInStock, which holds the count. The count goes down when a part is sold and goes up when a delivery from a supplier arrives. When the count falls to ReorderLevel or below, the part goes on the list to order. In your case study the same shape might be tins in a food bank, tickets in a box office or books of one title in a shop.

**One-off stock: Bike**

A bike for sale is one of a kind, with its own frame number, so it gets its own row in the Bike table. Instead of a count, each bike has a status. The status goes from Available to Reserved to Sold, and a sold bike can never be sold again. In your case study the same shape might be a used car, a painting or a puppy for rehoming.

**RULE THAT NEVER BREAKS · STOCK IS ALWAYS RIGHT**

The count of a part on the shelf is always right, and it can never go below zero. Chapter 8 builds the check that refuses a sale of more parts than the shelf holds.

# See the people and lists that sit around the four shapes.

The four shapes never stand alone. Every booking is for a customer and is made by a member of staff. Every part comes from a supplier. So each case study also has the people and the lists that the four shapes point at.

**The people and the lists**

Customer holds the people the shop serves. Staff holds the people who work there, with the username and password each one logs in with. ServiceType is a list of the seven services the workshop does, with the minutes and the price of each. Supplier is a list of the three firms the shop buys parts from.

**The records of stock moving**

PartOrder holds one row for each order the shop sends to a supplier, and PartSale holds one row for each sale of a part to a customer. These two tables are the history behind the count in Part. Every change to QtyInStock is caused by a row in one of them.

Bike does a second job. The bikes for hire are rows in the Bike table too, with Hire in a column called Purpose, so that Hire can point at the one bike it books.

# Give every table a number column, its primary key.

Every row needs a name that no other row shares. A person's name is not good enough. Towpath Cycles has a customer called Aoife McCann and a mechanic called Aoife Byrne, and next year it may have two customers called Aoife McCann.

So every table gets a number column, and it is always the first column. Its name is the table name followed by ID: CustomerID in Customer, BikeID in Bike, HireID in Hire. This column is the primary key, which means the column that holds a different number for every row.

Nobody types the number. When a new row is added, the table gives out the next number itself: the first customer is 1, the second is 2, and so on. A number is never given out twice, even after its row is deleted, so a number always means the same row.

# Point at a row in another table by its number, with a foreign key.

A booking needs to say which customer it is for. It does that by holding the customer's number, not the customer's name. A column that holds the number of a row in another table is called a foreign key. A foreign key is the number of the other row.

**One real row of the Appointment table**

The first appointment in the sample data holds CustomerID 1, MechanicID 3 and ServiceTypeID 6. Customer number 1 is Aoife McCann. Staff number 3 is Aoife Byrne, a mechanic. ServiceType number 6 is the Basic service, 60 minutes for 40 pounds. So the row says that Aoife Byrne gave Aoife McCann's blue Trek hybrid a basic service.

A foreign key usually has the same name as the column it points at, such as CustomerID in Appointment pointing at CustomerID in Customer. When one table points at the same table twice, each column gets a name that says why. Appointment points at Staff twice: MechanicID for the mechanic who does the work, and BookedByStaffID for the person who made the booking.

The table being pointed at is called the parent, and the table that holds the foreign key is called the child. Customer is a parent, and Appointment is its child. A child row can only point at a parent row that exists, so the database refuses a booking for customer 99 when there is no customer 99.

# Store each name once, in one table.

A customer's name lives in the Customer table and nowhere else. Every other table that needs the customer holds the CustomerID. Here is why the name is never stored twice.

Suppose Appointment held the words Aoife McCann instead of the number 1. If she changes her surname, someone has to change every booking she ever made. Miss one, and the shop has two customers where it had one. A spelling slip in a single booking does the same.

With the number, her name is written in one place. A change to that one row shows up in every booking, every hire and every sale at once.

**Names on the screen, numbers in the tables**

The tables store numbers, and the screens show names. When your program shows a booking, a stored procedure looks up the number and shows Aoife McCann. The person using your program never sees or types an ID number. Chapter 4 shows how a stored procedure does that look-up.

STAGE 4 OF 5 · TYPES AND RULES · SCREEN 1 OF 3

# Choose a type for every column.

Every column has a type, which says what may go in it. The type is a rule too: a column for whole numbers refuses the word Tuesday.

**Whole numbers and text**

Counts and ID numbers are INT, the type for whole numbers, such as QtyInStock and CustomerID. Names are VARCHAR, the type for text, with the most letters allowed in brackets: FirstName is VARCHAR(40). A phone number is text too, not a number, because 028 3026 4411 starts with a 0 and has spaces, and a whole number would lose both.

**Money is DECIMAL, never FLOAT**

Every price, cost and total is DECIMAL, the type that keeps the exact pennies. Price in ServiceType is DECIMAL(8,2): up to eight digits, two of them after the point. FLOAT stores a close guess instead. In FLOAT, 0.1 added to 0.2 comes out as 0.30000000000000004, so a till that adds up in FLOAT is a fraction of a penny wrong.

**Dates are DATE or DATETIME, never text**

A hire is booked by the day, so StartDate and EndDate are DATE. An appointment is booked by the clock, so StartTime and EndTime are DATETIME, a day and a time together. A date stored as text goes wrong in three ways. 01/02/2026 could be the first of February or the second of January. 31/02/2026 would be let in, although February has no 31st. And the text 10/01/2026 sorts before 9/01/2026, because the 1 comes before the 9.

A yes or no column, such as IsActive in Staff, is BIT, which holds 1 for yes and 0 for no.

# Let the table refuse bad data before any program runs.

A rule can be written into the table itself. The table then refuses any row that breaks the rule, with red text that names the rule. No program has to remember to check, because every program that ever adds a row meets the same rule.

**NOT NULL: the box must be filled**

Every column is NOT NULL unless an empty box means something. A customer must have a first name, a last name and a phone number. Email may be empty, because Ciara Quinn and Hannah Boyle gave the shop no email address.

**CHECK: the value must pass a test**

A CHECK is a test the row must pass. The status of an appointment must be one of three words: Booked, Done or Cancelled. The end time must be after the start time. And QtyInStock in Part must be 0 or more, so the count can never go below zero, whatever any program does.

**UNIQUE: no two rows the same**

A UNIQUE column refuses a value that another row already holds. Two members of staff can never share a username, and two bikes can never share a frame number.

In chapter 3 you see a CHECK at work. You try to add a booking with the status Maybe, and the table refuses it in red before anything is saved.

# Read the plan of one table, written out in full.

Before you draw all ten tables, here is the plan of one of them, the Customer table, written out column by column. Each line gives the name of the column, its type and its rule. Your plan of each table in your case study looks like this one.

**The plan of the Customer table**

CustomerID is an INT that the table gives out itself, and it is the primary key.

FirstName is a VARCHAR(40), and it is NOT NULL.

LastName is a VARCHAR(40), and it is NOT NULL.

Phone is a VARCHAR(15), it is NOT NULL, and a CHECK refuses a number shorter than ten characters.

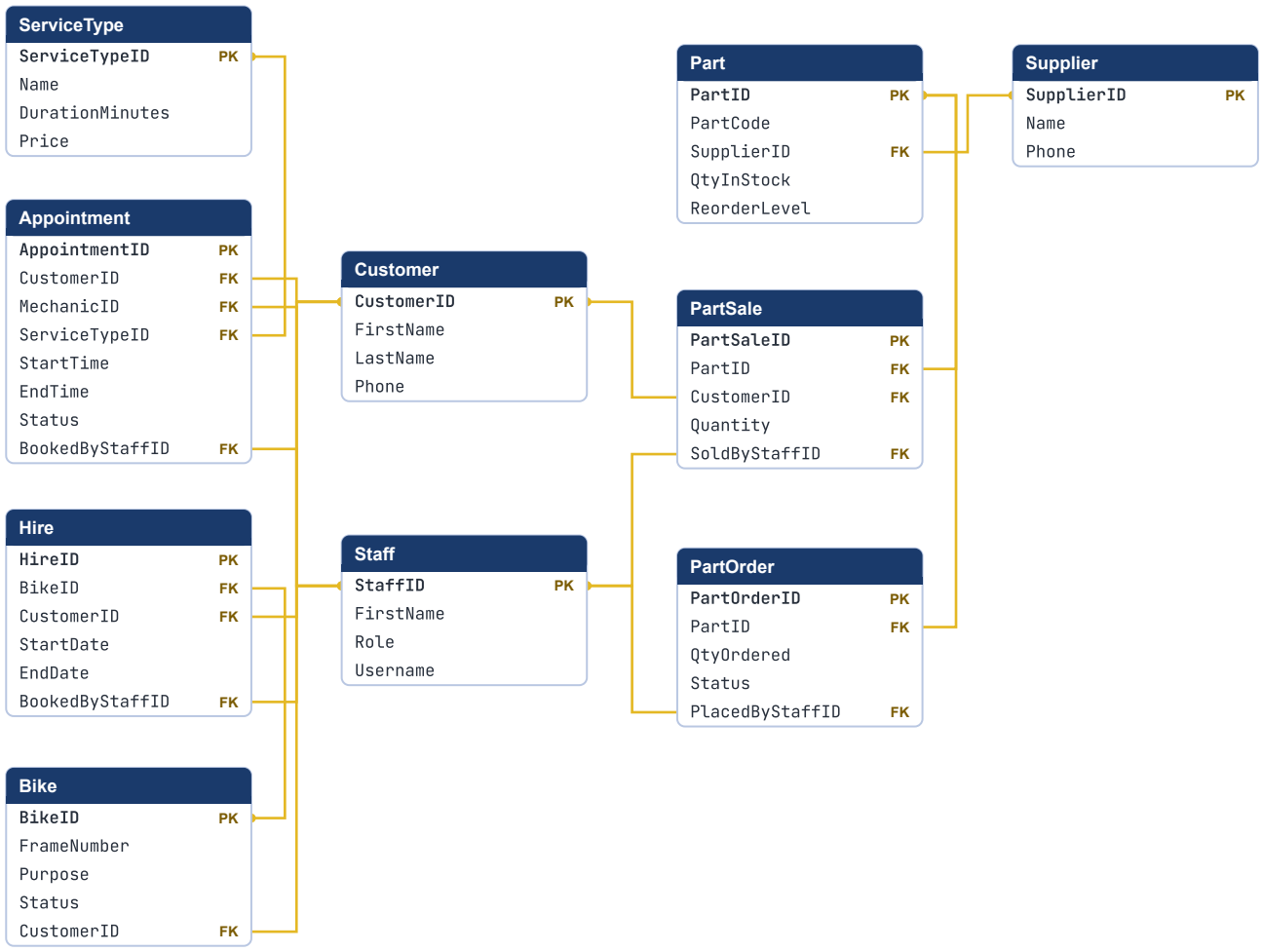
Email is a VARCHAR(100), and it may be NULL, because some customers give no email address.

DateJoined is a DATE, it is NOT NULL, and the table fills in today's date if no date is given.

Customer points at no other table, so it has no foreign key. Appointment, by contrast, has four foreign keys: CustomerID, MechanicID, ServiceTypeID and BookedByStaffID.

# See the plan of all ten Towpath Cycles tables.

Here is the whole plan as a drawing. Each box is one table, with its name in the dark bar at the top. Each box lists the primary key, marked PK, every foreign key, marked FK, and two or three of the other columns.




The plan of the ten tables. A gold line runs from each foreign key to the number column it points at, and the dot sits at the table being pointed at. Customer and Staff have the most lines, because almost everything in the shop is done for a customer by a member of staff.

Follow one line to read the drawing. The line from CustomerID in Appointment ends at CustomerID in Customer, so every appointment is for one customer. The five lines into Staff come from the people who do the work and make the bookings: the mechanic, and the staff who book appointments, book hires, sell parts and order parts. In chapter 3 the server checks every one of these lines, and it refuses a row that points at a parent that is not there.

# Your turn: plan the tables for your own case study.

**Your turn**

Take your own case study and turn it into a plan, the way the Towpath Cycles story became ten tables. Use a clean sheet of paper. This plan is yours, so there is no answer to open: the Towpath Cycles list below shows what a finished one looks like.

Read your case study once from start to end without writing anything.

> **You should see** You can tell someone what the business does and what it books or sells.

Read the case study again and underline every noun.

> **You should see** Every person, place and thing in the case study is underlined.

Write each underlined noun once, on its own line, down the left of your sheet.

> **You should see** Your sheet has a list of nouns, with no noun written twice.

Cross out each noun that the business has only one of, such as its own name or its town.

> **You should see** The nouns left are the things the business has many of, and their details.

Move each noun that is a detail of another noun, such as a phone number, to sit under that noun.

> **You should see** The nouns left down the left edge are your tables, and the details under them are their first columns.

Write a shape beside each table: booked by time, booked by day, counted stock, one-off stock, a person, a list, or a record of stock moving.

> **You should see** Every table has a shape, and you can see which of the four shapes your case study uses.

Draw a box for each table, with its ID column first, marked PK.

> **You should see** Each table is a box, and its first line is its name followed by ID, such as LessonID.

Write each foreign key in the box of the child table, marked FK.

> **You should see** Every booking and every sale has a column ending in ID for each person and list it needs.

Draw a line from each foreign key to the primary key of its parent table.

> **You should see** Every booking and every sale points at the people and the lists it needs, by number.

**What a finished list looks like, for Towpath Cycles**

Booked by time: Appointment, which books a mechanic from a start time to an end time.

Booked by day: Hire, which books one bike from a start date to an end date.

Counted stock: Part, with QtyInStock and ReorderLevel.

One-off stock: Bike, with a frame number and a status that goes from Available to Reserved to Sold.

The people: Customer and Staff.

The lists: ServiceType, the services the workshop does, and Supplier, the firms the shop buys from.

The records of stock moving: PartOrder, for parts coming in, and PartSale, for parts going out.

Your case study may not use all four shapes. A case study with no stock has no counted stock and no one-off stock, and that is fine. Most case studies have at least one booking shape and at least one stock shape, so look twice before you leave one out.

STAGE 5 OF 5 · PROVE IT: THE PLAN OF THE TEN TABLES, AND YOUR OWN · SCREEN 3 OF 3

# Tick the table: check your own plan against eight questions.

Check your own plan against every row below. The first four rows are things your plan must have. The last four are mistakes your plan must not have, and each one costs marks if it reaches your finished program.

| THE TEST | WHAT MUST HAPPEN | HOW TO DO IT, AND WHAT YOU SHOULD SEE |
|---|---|---|
| Every table has a number column | Must be allowed | Look at the first line of each box on your plan. It is the table name followed by ID, such as CustomerID, and it is marked PK. |
| Every booking has a start and an end | Must be allowed | Find each table you marked as booked by time or booked by day. A booking by time has a start time and an end time, both DATETIME. A booking by day has a start date and an end date, both DATE. |
| Every link is a number with a line | Must be allowed | Find each column marked FK. It ends in ID, it is an INT, and a line on your plan runs from it to the primary key of its parent table. |
| Every counted stock has a rule against going below zero | Must be allowed | Find the count column in each counted stock table, such as QtyInStock in Part. Beside it you have written a CHECK that says the count must be 0 or more. |
| A name stored twice | Must be refused | Look in every table except the people tables for a column such as CustomerName or MechanicName. Your plan must have none. If you find one, replace it with the ID of that person, such as CustomerID. |
| Money planned as FLOAT | Must be refused | Look at every price, cost and total on your plan. Each must be DECIMAL with two places after the point, such as DECIMAL(8,2). If you find a FLOAT, change it to DECIMAL. |
| A date planned as text | Must be refused | Look at every date and time on your plan. Each must be DATE or DATETIME. If you find a date planned as VARCHAR, change it, because text would let 31/02/2026 in. |

| THE TEST | WHAT MUST HAPPEN | HOW TO DO IT, AND WHAT YOU SHOULD SEE |
|---|---|---|
| A status that could be any word | **Must be refused** | Look at every status column on your plan. Each has a CHECK with its list of allowed words, such as Booked, Done or Cancelled. If one has no list, write the list beside it, or the table would let the word Maybe in. |

Keep your plan beside you for chapter 3. When your plan passes all eight rows, it is ready to be typed, and chapter 3 builds the Towpath Cycles tables the same way, one table at a time.

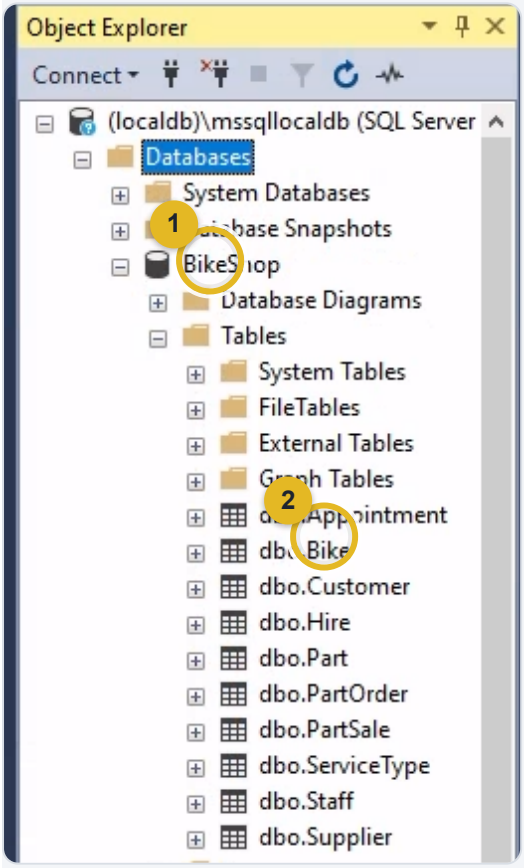
# Build the database

The script that creates every table and fills it with sample data, and can be run again.

STAGE 1 OF 6 · A SCRIPT YOU CAN RUN AGAIN · SCREEN 1 OF 4

## See what you will build in this chapter.

In chapter 2 you planned ten tables on paper. In this chapter you make them, by writing one script, BikeShop.sql, that the server runs from top to bottom. By the end of the chapter the script makes the database, makes the ten tables, puts in the sample data, and prints the words BikeShop is ready. Then you ask the Customer table for its rows and see twelve names come back.




Object Explorer, on the left of SQL Server Management Studio (SSMS). Right-click Tables and click Refresh to see the tables your script has made. The dbo. in front of each name is added by the server, and you never type it.

1. Open Databases, then BikeShop.
2. Tables: the ten tables, in alphabetical order.

The script has one habit that matters more than any other: it can be run again. Every time you press F5 it throws the old tables away and makes fresh ones with the same sample data. That means a mistake is never serious. If a table is wrong, you fix the script and run it again. If a test

in a later chapter leaves the data in a mess, you run it again. The script is the master copy of your database, and it lives in your OneDrive folder.

# Learn these words before you start.

**Before you start**

- Chapter 2 is finished: your plan of the ten tables, with their columns and rules, is beside you.
- SQL Server Management Studio (SSMS) is open and connected, as in chapter 1.
- BikeShop.sql is saved in your OneDrive coursework folder, and it is still empty.
- This guide is open in the Edge browser beside you. Tick each step as you do it, and the chapter remembers where you are.

**Words used in this chapter**

| | |
|---|---|
| `Script` | A file of SQL lines that the server runs from the top to the bottom. |
| `Table` | A grid with named columns, where each row is one thing, such as one customer. |
| `Column` | One named box in every row, with a type that says what can go in it, such as a number or a date. |
| `Row` | One line of a table: one customer, one bike, one booking. |
| `INT` | A type for whole numbers, such as 3 or 120. |
| `VARCHAR` | A type for text, with the most letters allowed in brackets, such as VARCHAR(40). |
| `DECIMAL` | A type for money and other numbers with a decimal point: DECIMAL(8,2) allows eight digits, two of them after the point. |
| `DATE and DATETIME` | DATE holds a day, such as 12 October 2026. DATETIME holds a day and a clock time together, such as 12 October 2026 at 09:30. |
| `BIT` | A type that holds a yes or a no, written as 1 or 0. |
| `NULL` | An empty box. NOT NULL after a column means the box can never be empty. |
| `IDENTITY` | The table gives each new row the next number itself, so nobody types it. |
| `PRIMARY KEY` | The column that names each row, so that no two rows can share a number. |
| `FOREIGN KEY` | A column that holds the number of a row in another table, such as the customer a booking is for. |
| `CHECK` | A rule written into the table that refuses a row that breaks it, before any program runs. |
| `UNIQUE` | A rule that no two rows can hold the same value in that column. |
| `DEFAULT` | The value a column takes when a row is added without one. |
| `GO` | A line on its own that sends everything above it to the server, so the next lines start fresh. |
| `Sample data` | Made-up rows, with real-looking names and dates, that fill the tables so that you have something to test with. |

# Read why the script drops the old tables first, children before parents.

A script that can be run again has to deal with what is already there. The second time it runs, the ten tables exist, and CREATE TABLE would be refused with the words there is already an object named Customer. So the script starts by dropping every table it is about to make. DROP TABLE IF EXISTS means: throw the table away if it is there, and say nothing if it is not. The first run finds nothing to drop and carries on.

**Children before parents**

A table that holds a foreign key points at another table. Appointment points at Customer, because every booking is for one customer. Call Appointment the child and Customer the parent. The server will not drop a parent while a child still points at it, so the children go first. That is why the ten DROP TABLE lines run in the opposite order to the ten CREATE TABLE lines: PartSale is dropped first and Customer last, and Customer is made first and PartSale last.

The same order matters when the tables are made. A foreign key can only point at a table that already exists, so Customer and Staff come before Appointment, and Supplier before Part. The order in this chapter is the order that works. Keep it.

# Copy the top of the script into BikeShop.sql.

The top of the script does three things: it makes the database if it is not there, it moves into it, and it drops the ten old tables. Read each part, then copy the whole block into your empty BikeShop.sql and run it once. The tables come next, one at a time.

Copy this one **The top of BikeShop.sql: the database, and the ten DROP TABLE lines** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
/* ====================================================================
   BikeShop.sql  –  Towpath Cycles (workshop + bike hire), Newry
   A2 SSD coursework demo database.  SQL Server 2019 / LocalDB.

   This script is RERUNNABLE: run it from the top any time and you get
   a fresh database with the same sample data.  Keep it in your
   OneDrive folder – it is the master copy of your database.

   Order:  1 create the database   2 drop old tables (children first)
           3 create tables         4 sample data        5 procedures
   ==================================================================== */

IF DB_ID('BikeShop') IS NULL
    CREATE DATABASE BikeShop;
GO
USE BikeShop;
GO

/* ---- 2. Drop old tables, children first ---- */
-- A child table holds the foreign key, so it goes before its parent.
DROP TABLE IF EXISTS PartSale;
DROP TABLE IF EXISTS PartOrder;
DROP TABLE IF EXISTS Part;
DROP TABLE IF EXISTS Supplier;
DROP TABLE IF EXISTS Hire;
DROP TABLE IF EXISTS Appointment;
DROP TABLE IF EXISTS Bike;
DROP TABLE IF EXISTS ServiceType;
DROP TABLE IF EXISTS Staff;
DROP TABLE IF EXISTS Customer;
GO

/* ---- 3. Tables ---- */
```

**1 The note at the top**

Lines 1 to 11

Everything between the slash-star at the start and the star-slash at the end is a note for a person, and the server skips it. The note names the shop in this guide, Towpath Cycles in Newry, and says what the file is, that it can be run again, and the order of its five parts. When you build your own scenario, you change the shop name and the town in this note to yours.

**2 Make the database if it is not there**

Lines 13 to 15

DB_ID looks up a database by name and gives NULL, meaning nothing, if there is no such database. So the second line, CREATE DATABASE BikeShop, only runs the first time. GO sends those lines to the server before anything else happens.

**3 Work inside it**

Lines 16 to 17

USE BikeShop tells the server that every line after this belongs to the BikeShop database, not to master, the database the server starts in. The GO under it sends the USE line to the server before the DROP TABLE lines run.

**4 Drop the ten old tables, children first**

Lines 19 to 31

This part has ten lines, one per table, and each says DROP TABLE IF EXISTS and a table name. The order runs from PartSale, which points at three other tables, down to Customer, which points at none. The comment line with the two dashes is a note for a person. GO ends this part.

**5 A heading for the tables**

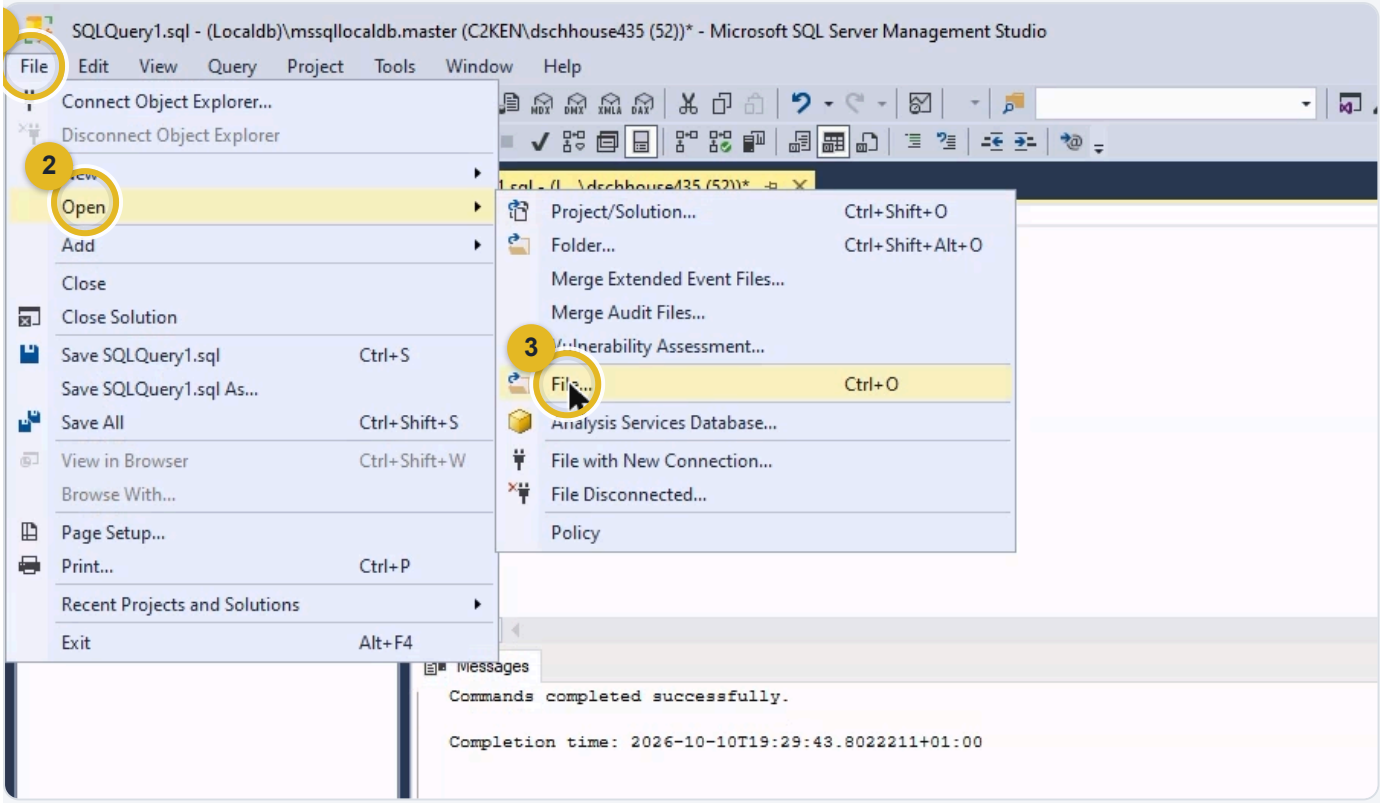
Line 33

This is one more note line, so that you can find the tables in a long script. The first CREATE TABLE goes on the empty line under this heading.

In SQL Server Management Studio (SSMS), click File at the top left.

**You should see** The File menu opens.

Point at Open in the File menu.




The File menu in SQL Server Management Studio (SSMS). Files open from here.

1. Click File at the top left.

2. Point at Open.

3. File… opens the box where you choose the file.

> **You should see** A small menu opens beside Open, and File… is in it.

Click File… in the small menu.

> **You should see** The Open File box opens.

In the Open File box, double-click BikeShop.sql in your OneDrive coursework folder.

> **You should see** A tab named BikeShop.sql opens, and it is empty.

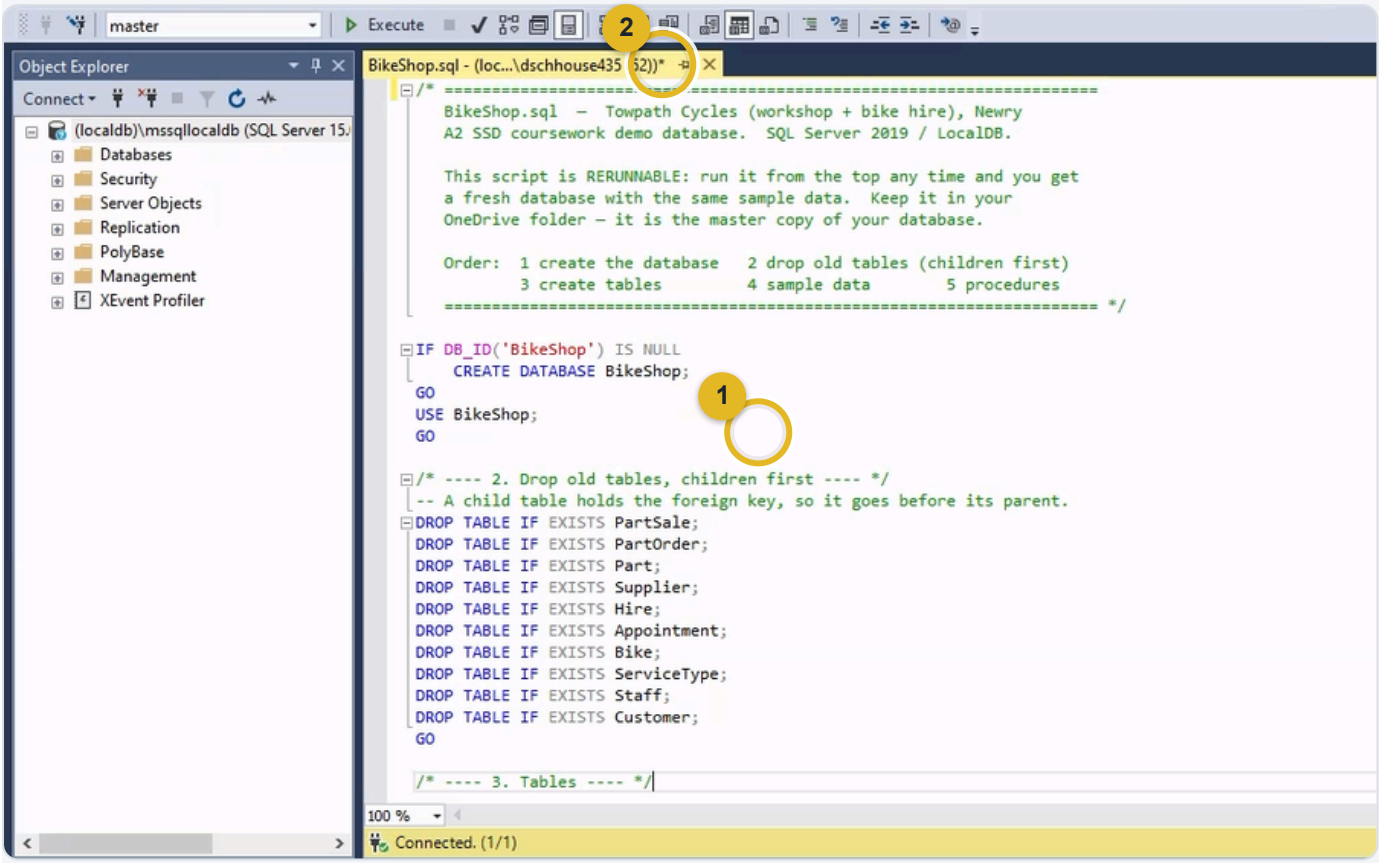
Press the Copy button under this step.

> **You should see** The button reads Copied for a moment.

Click in the empty BikeShop.sql window.

> **You should see** The cursor blinks at the top of the empty window.

Press Ctrl and V together.




The top of the script, pasted into the empty BikeShop.sql. The star after the tab name means it is not saved yet.

1. The pasted lines, from the note at the top down to the heading for the tables.

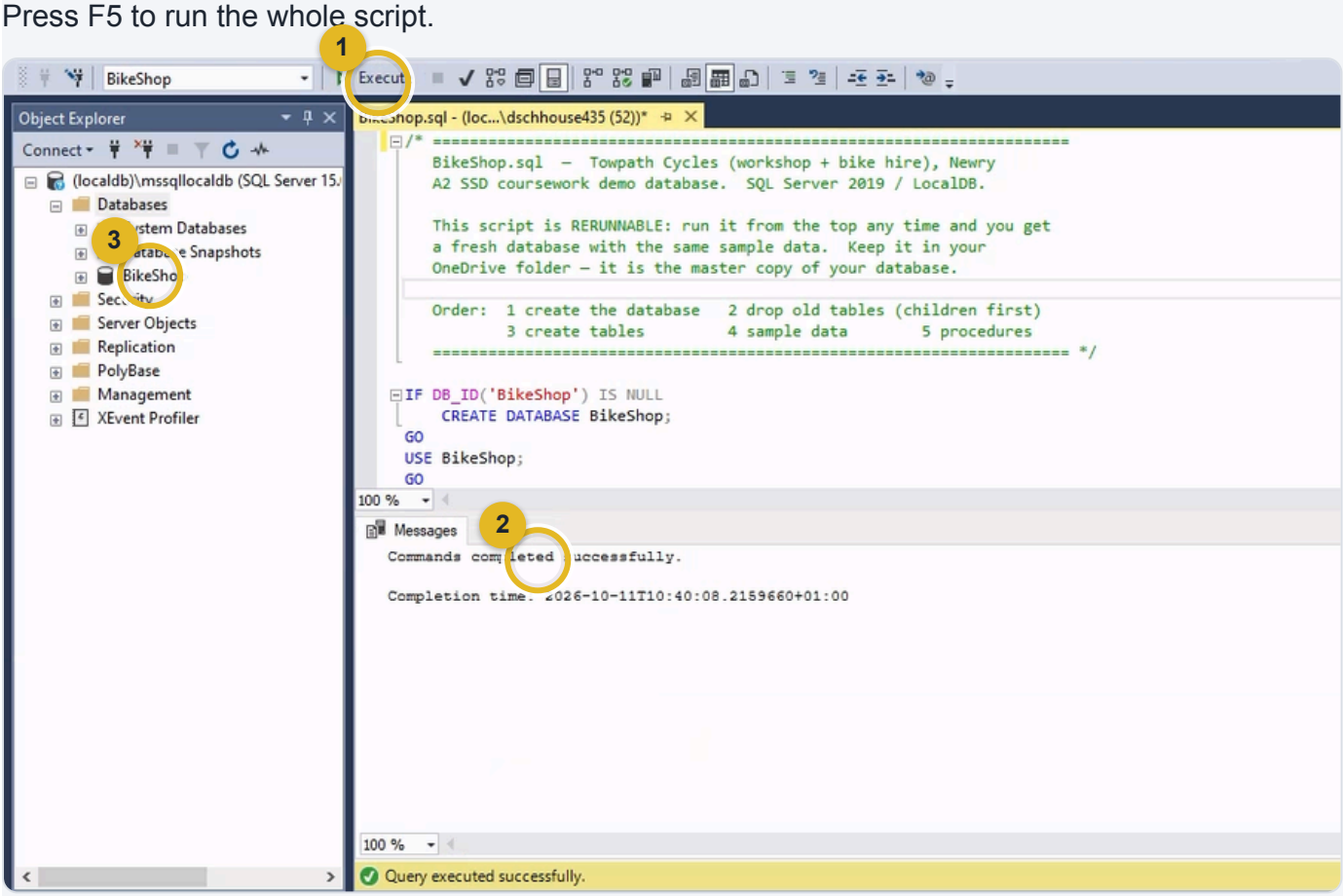
2. The star after BikeShop.sql means the file needs saving.

> **You should see** The top of the script is in your window, from the note at the top down to the heading for the tables, and the tab name has a star after it.

Press Ctrl and S together to save the script.

> **You should see** The star after BikeShop.sql on the tab disappears, because the file is saved.

Press F5 to run the whole script.




Running the whole script with nothing highlighted. Commands completed successfully means every line ran, and the database appears in Object Explorer.

1. Click Execute, or press F5, with nothing highlighted.
2. The Messages pane says: Commands completed successfully.
3. BikeShop appears under Databases.

> **You should see** The Messages pane says: Commands completed successfully. In Object Explorer, BikeShop appears under Databases, with nothing in its Tables folder yet.

**IF IT GOES WRONG** Red text that says 'CREATE DATABASE permission denied'

You are connected to the wrong server. Close the window, start SQL Server Management Studio (SSMS) again and connect to (localdb)\mssqllocaldb, as in chapter 1.

**IF IT GOES WRONG** BikeShop is not under Databases

Object Explorer does not look again by itself. Right-click Databases and click Refresh.

STAGE 2 OF 6 · CUSTOMER, STAFF AND SERVICETYPE · SCREEN 1 OF 3

# Read the Customer table part by part, then add it to your script.

The first table is Customer, and four other tables point at it. Read it slowly, because every table in this chapter is built out of the same pieces: a number column that fills itself, columns with a type and NOT NULL, a DEFAULT, a PRIMARY KEY, and a CHECK rule.

Copy this one **Customer: the first table** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
CREATE TABLE Customer (
    CustomerID   INT IDENTITY(1,1) NOT NULL,
    FirstName    VARCHAR(40)  NOT NULL,
    LastName     VARCHAR(40)  NOT NULL,
    Phone        VARCHAR(15)   NOT NULL,
    Email        VARCHAR(100) NULL,
    DateJoined   DATE          NOT NULL
        CONSTRAINT DF_Customer_DateJoined DEFAULT (CAST(GETDATE() AS DATE)),
    CONSTRAINT PK_Customer PRIMARY KEY (CustomerID),
    CONSTRAINT CK_Customer_Phone CHECK (LEN(Phone) ⩾ 10)
);
GO
```

**1 The table name and its number column**

Lines 1 to 2

CREATE TABLE and a name start the table. The bracket opens the list of columns. The first column is the number that names each row. IDENTITY(1,1) means the table gives the number out itself, starting at 1 and going up by 1, so you never type it. NOT NULL means the box can never be left empty.

**2 Three text columns that must be filled**

Lines 3 to 5

FirstName and LastName hold up to 40 letters each. Phone holds up to 15, because a number such as 028 3026 4411 has spaces in it and is text, not a number to add up. NOT NULL on all three: a customer with no name or no phone number is refused.

**3 A column that may be empty**

Line 6

Email says NULL, not NOT NULL, so a customer without an email address can still be added. The box is left empty, and the word NULL shows in it when you look.

**4 A date that fills itself in**

Lines 7 to 8

DateJoined is a DATE, and it has a DEFAULT. GETDATE() gives the date and time right now, and CAST turns that into a date with no clock part. So a new customer is stamped with today's date without anyone typing it. The DEFAULT has a name, DF_Customer_DateJoined, so that an error can name it.

**5 The primary key**

Line 9

CustomerID is the column that names each row. PRIMARY KEY means no two customers can have the same number, and a booking can point at a customer by it. The rule is named PK_Customer: PK for primary key, then the table.

**6 The phone number check**

Lines 10 to 12

LEN counts the characters in the phone number. The CHECK refuses any row whose phone number is shorter than ten characters, so 12345 can never get in. The rule is named CK_Customer_Phone: CK for check, the table, then what it checks. The closing bracket and the semicolon end the table. GO sends everything above it to the server before the next table starts.

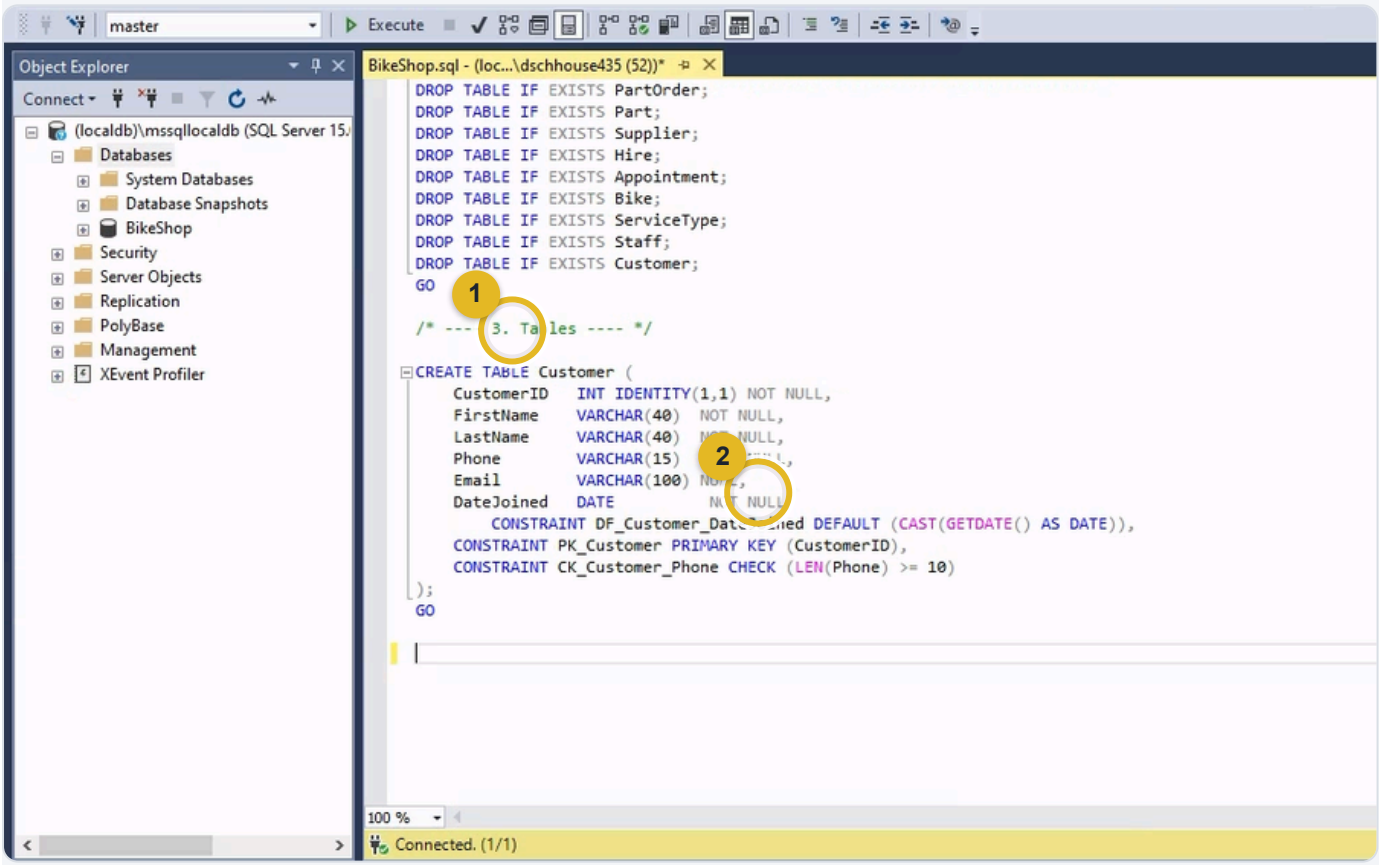
Press the Copy button under this step.

**You should see** The button reads Copied for a moment.

Click on the empty line under the heading for the tables, at the very end of the script.

**You should see** The cursor blinks on the empty line under the line that says 3. Tables, with nothing after it.

Press Ctrl and V together.




The Customer table pasted under the heading for the tables. Every table starts with CREATE TABLE and ends with GO, and the next one goes on the empty line under that GO.

1. The heading for the tables, with the Customer table pasted under it.

2. The pasted Customer table, from CREATE TABLE Customer down to GO.

**You should see** The new table is at the end of your script, from CREATE TABLE down to its own GO.

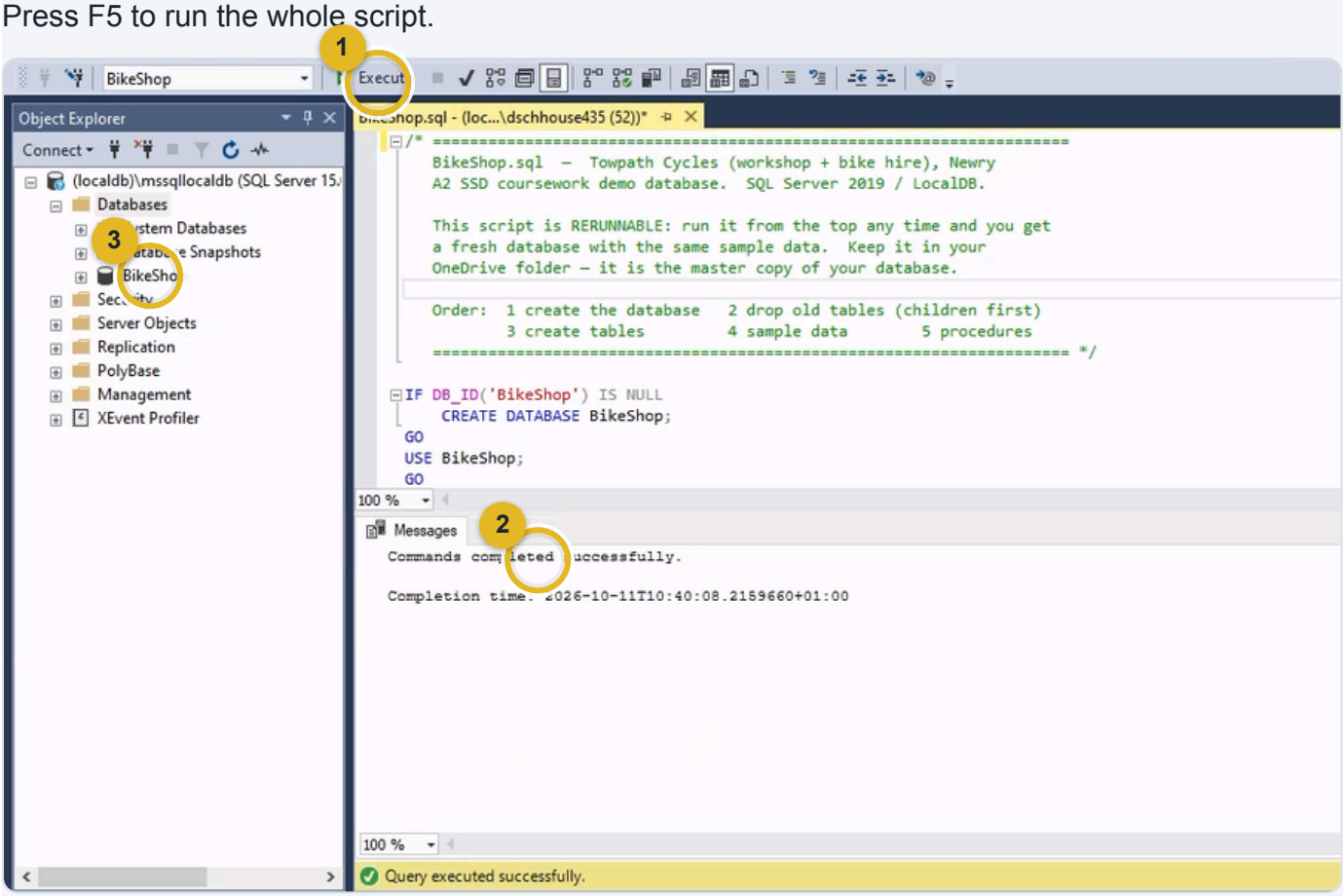
Press Ctrl and S together to save the script.

> **You should see** The star after BikeShop.sql on the tab disappears, because the file is saved.

Click anywhere in the script so that nothing is highlighted.

> **You should see** The cursor blinks in the script, and no line is highlighted in blue.

Press F5 to run the whole script.




Running the whole script with nothing highlighted. Commands completed successfully means every line ran, and the database appears in Object Explorer.

1. Click Execute, or press F5, with nothing highlighted.

2. The Messages pane says: Commands completed successfully.

3. BikeShop appears under Databases.

**You should see** The Messages pane says: Commands completed successfully. Every table so far was dropped and made again.

**IF IT GOES WRONG** Red text that says 'Incorrect syntax near …'

A bracket, a comma or a quote mark is missing or doubled. The line number in the red text is the line to look at. Compare it with the block above, character by character.

**IF IT GOES WRONG** Red text that says 'There is already an object named … in the database'

The DROP TABLE line for that table is missing from the top of the script, so the old table was still there. Check the ten DROP TABLE lines against the block in stage 1.

**IF IT GOES WRONG** Red text that says 'Could not drop object … because it is referenced by a FOREIGN KEY constraint'

A table is being dropped before a table that points at it. Put the DROP TABLE lines back in the order shown in stage 1: PartSale first, Customer last.

**IF IT GOES WRONG** Red text that says 'Foreign key … references invalid table …'

A table is being made before the table it points at. The tables must be in the order of this chapter: Customer, Staff, ServiceType, Appointment, Bike, Hire, Supplier, Part, PartOrder, PartSale.

STAGE 2 OF 6 · CUSTOMER, STAFF AND SERVICETYPE · SCREEN 2 OF 3

## Add the Staff table, which has a UNIQUE rule and a list of roles.

Staff is the people who use the program: a manager, counter staff and mechanics. Two things are new. A UNIQUE rule stops two people sharing a username, and the first CHECK with a list in it says a role must be one of three words.

Copy this one **Staff: the people who sign in** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
CREATE TABLE Staff (
    StaffID      INT IDENTITY(1,1) NOT NULL,
    FirstName    VARCHAR(40) NOT NULL,
    LastName     VARCHAR(40) NOT NULL,
    Role         VARCHAR(10)  NOT NULL,
    Username     VARCHAR(30)  NOT NULL,
    -- a real system stores a hash; say so in your user guide
    Password     VARCHAR(30)  NOT NULL,
    IsActive     BIT          NOT NULL CONSTRAINT DF_Staff_IsActive DEFAULT (1),
    CONSTRAINT PK_Staff PRIMARY KEY (StaffID),
    CONSTRAINT UQ_Staff_Username UNIQUE (Username),
    CONSTRAINT CK_Staff_Role CHECK (Role IN ('Mechanic', 'Counter', 'Manager'))
);
GO
```

### 1 The same start as Customer

Lines 1 to 4

This starts the same way as Customer: a number column that fills itself, then a first name and a last name that must be filled.

### 2 The job

Line 5

Role is one of three words, Mechanic, Counter or Manager, and it decides which screens a person can open in later chapters.

### 3 The sign-in details

Lines 6 to 8

Username and Password are what a person types to sign in. The comment line is a reminder for your user guide: a real system would scramble the password before storing it, and this coursework keeps it plain and says so.

### 4 A yes or no column

Line 9

BIT holds a 1 for yes or a 0 for no. The DEFAULT of 1 means a new member of staff is active unless told otherwise, and someone who leaves is set to 0 rather than deleted.

### 5 The key and the UNIQUE rule

Lines 10 to 11

StaffID is the primary key. UQ_Staff_Username says no two rows can have the same username, so a sign-in always finds one person.

### 6 The list of roles

Lines 12 to 14

IN followed by a list in brackets means: the value must be one of these. A role of Boss is refused by the table. The closing bracket and the semicolon end the table. GO sends everything above it to the server

before the next table starts.

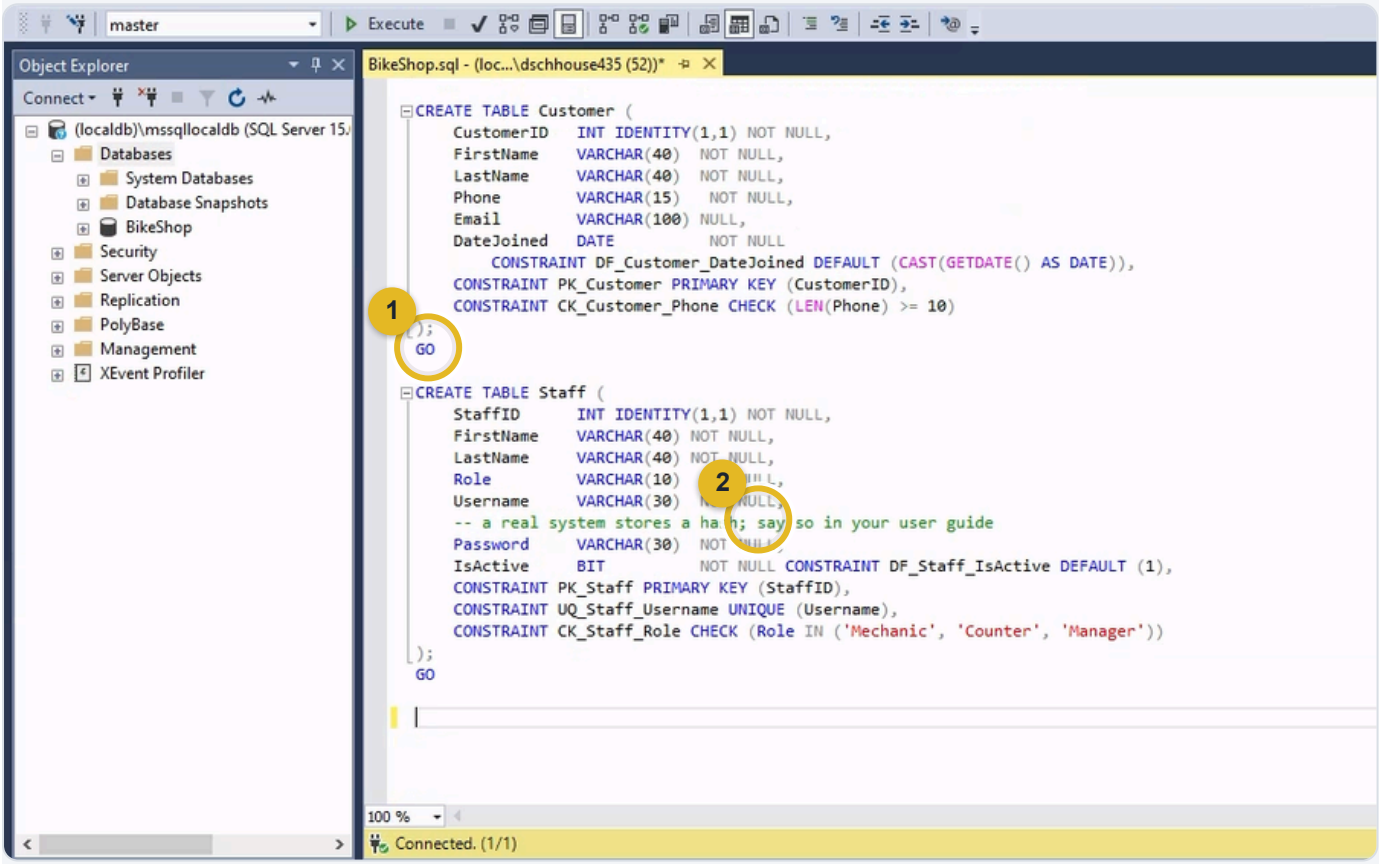
Press the Copy button under this step.

> **You should see** The button reads Copied for a moment.

Click on the empty line under the last GO, at the very end of the script.

> **You should see** The cursor blinks on an empty line under the last GO, with nothing after it.

Press Ctrl and V together.




The Staff table pasted at the end of the script, under the GO that ends the Customer table. The next table goes on the empty line under its own GO.

1. The GO that ends the Customer table.

2. The pasted Staff table, from CREATE TABLE Staff down to GO.

> **You should see** The new table is at the end of your script, from CREATE TABLE down to its own GO.

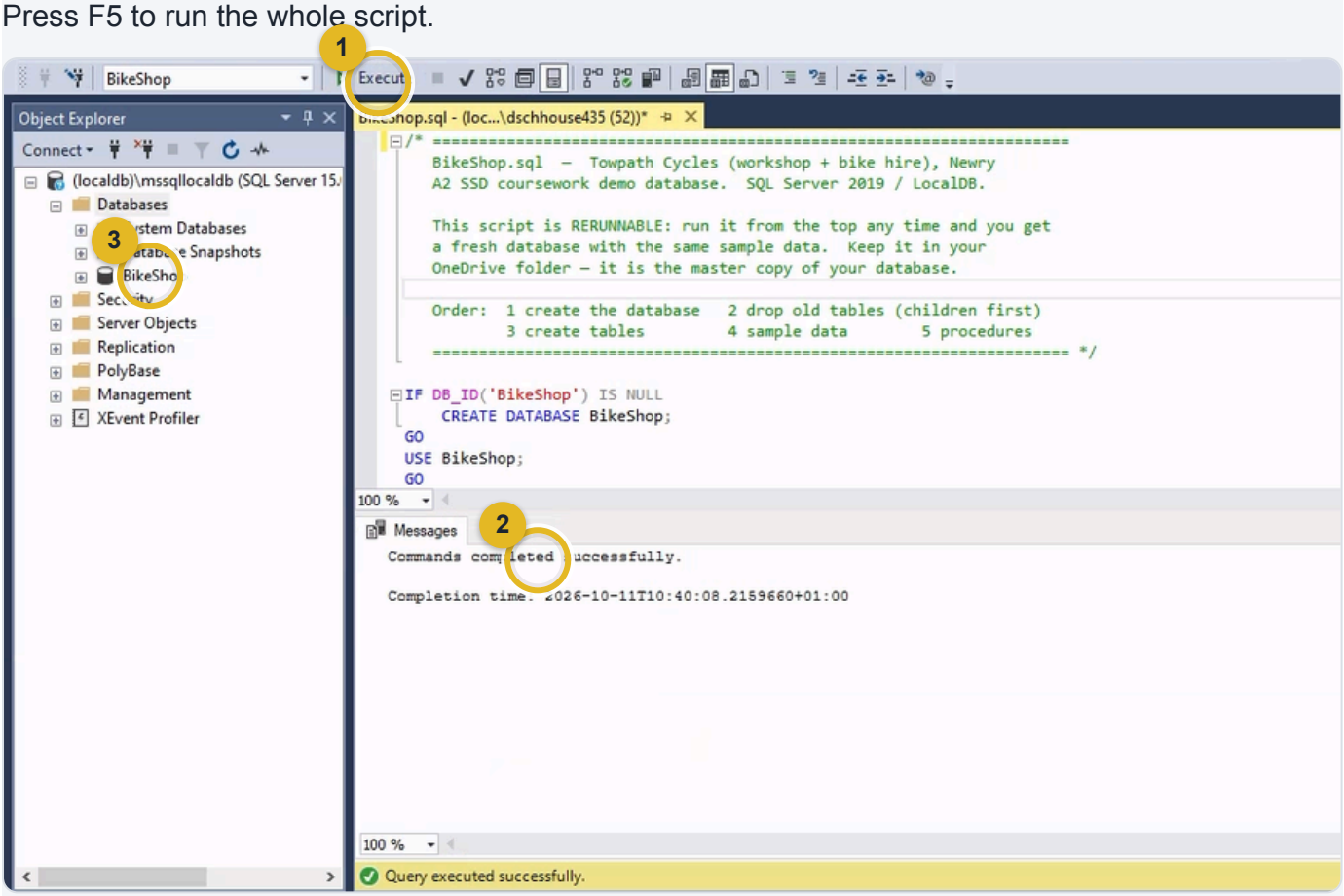
Press Ctrl and S together to save the script.

> **You should see** The star after BikeShop.sql on the tab disappears, because the file is saved.

Click anywhere in the script so that nothing is highlighted.

> **You should see** The cursor blinks in the script, and no line is highlighted in blue.

Press F5 to run the whole script.




Running the whole script with nothing highlighted. Commands completed successfully means every line ran, and the database appears in Object Explorer.

1. Click Execute, or press F5, with nothing highlighted.

2. The Messages pane says: Commands completed successfully.

3. BikeShop appears under Databases.

**You should see** The Messages pane says: Commands completed successfully. Every table so far was dropped and made again.

**IF IT GOES WRONG** Red text that says 'Incorrect syntax near …'

A bracket, a comma or a quote mark is missing or doubled. The line number in the red text is the line to look at. Compare it with the block above, character by character.

**IF IT GOES WRONG** Red text that says 'There is already an object named … in the database'

The DROP TABLE line for that table is missing from the top of the script, so the old table was still there. Check the ten DROP TABLE lines against the block in stage 1.

**IF IT GOES WRONG** Red text that says 'Could not drop object … because it is referenced by a FOREIGN KEY constraint'

A table is being dropped before a table that points at it. Put the DROP TABLE lines back in the order shown in stage 1: PartSale first, Customer last.

**IF IT GOES WRONG** Red text that says 'Foreign key … references invalid table …'

A table is being made before the table it points at. The tables must be in the order of this chapter: Customer, Staff, ServiceType, Appointment, Bike, Hire, Supplier, Part, PartOrder, PartSale.

# Type the two checks of the ServiceType table.

ServiceType is the list of jobs the workshop does, such as a puncture repair at 15.00 for 30 minutes. It is short, so this is the table where you type the rules yourself. The copy button leaves a gap for the yellow lines, and you type them in.

**Type this one** **ServiceType: the jobs the workshop does** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
CREATE TABLE ServiceType (
    ServiceTypeID   INT IDENTITY(1,1) NOT NULL,
    Name            VARCHAR(40) NOT NULL,
    DurationMinutes INT          NOT NULL,
    Price           DECIMAL(8,2) NOT NULL,
    CONSTRAINT PK_ServiceType PRIMARY KEY (ServiceTypeID),
    CONSTRAINT UQ_ServiceType_Name UNIQUE (Name),
    CONSTRAINT CK_ServiceType_Duration CHECK (DurationMinutes BETWEEN 15
        AND 480 AND DurationMinutes % 15 = 0),
    CONSTRAINT CK_ServiceType_Price CHECK (Price >= 0)
);
GO
```

**1 The table name and its number column**

Lines 1 to 2

This starts the same way as the two tables before it, with a number that the table gives out itself.

**2 What a job is called, how long it takes and what it costs**

Lines 3 to 5

Name is text. DurationMinutes is a whole number of minutes. Price is DECIMAL(8,2): up to eight digits with two after the point, which is how money is stored, because a DECIMAL never rounds a penny away.

**3 The key and the UNIQUE rule**

Lines 6 to 7

ServiceTypeID is the primary key, and no two jobs can share a name.

**4 The two checks: the yellow lines**

Lines 8 to 12

BETWEEN 15 AND 480 means the length must be from a quarter of an hour to eight hours. The percent sign gives the remainder after dividing, so DurationMinutes % 15 = 0 means the length divides exactly by 15: the diary runs in quarter hours. AND joins the two tests, so both must be true. The second CHECK refuses a price below zero. The closing bracket and the semicolon end the table. GO sends everything above it to the server before the next table starts.

**Why you type it** The two CHECK rules are the part of a table that you will write most often in your own scenario. Typing BETWEEN, the remainder sign and a comparison once, with your own hands, is how the shape sticks.

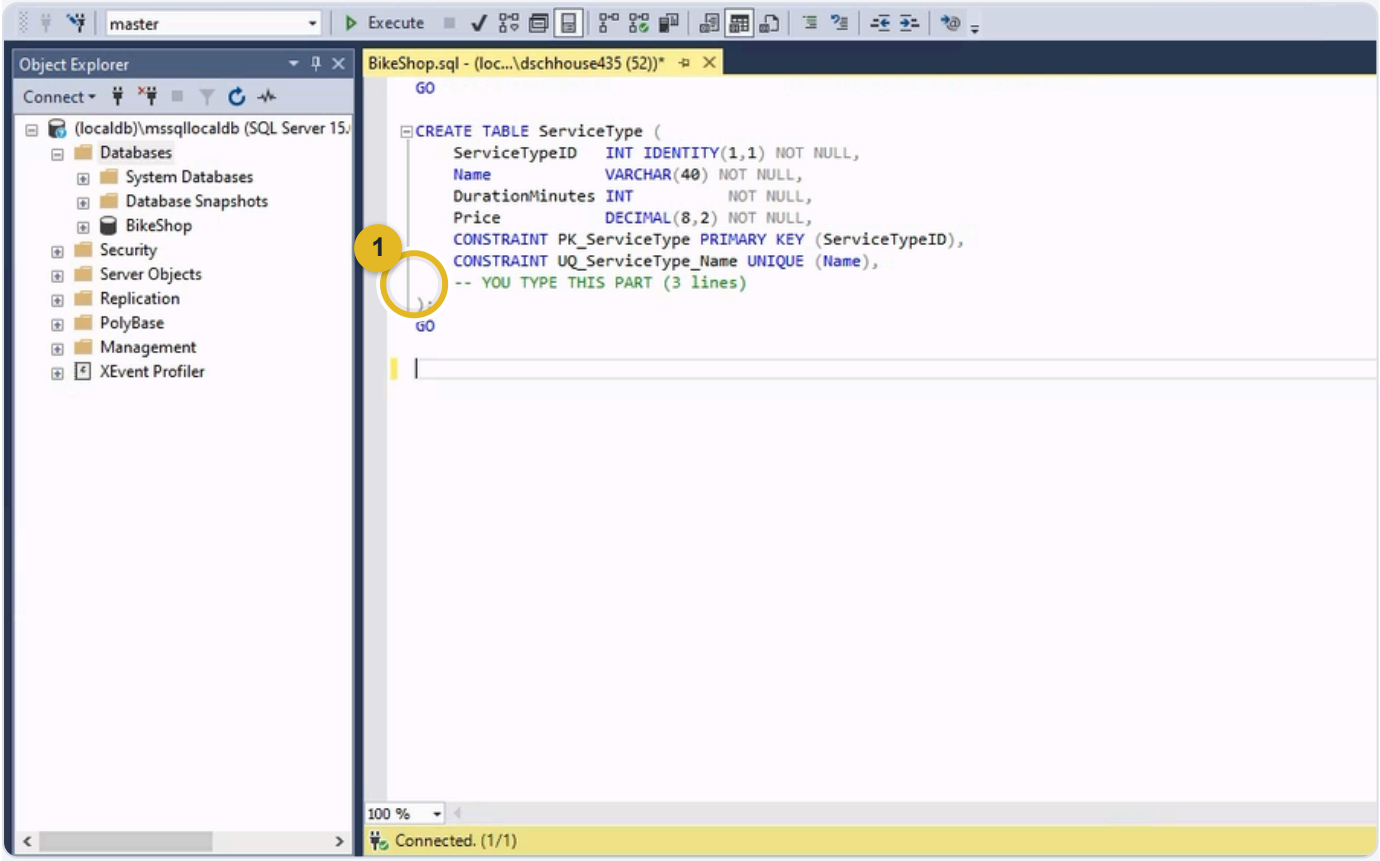
Press the Copy button under this step.

**You should see** The button reads Copied for a moment.

Click on the empty line under the last GO, at the very end of the script.

> **You should see** The cursor blinks on an empty line under the last GO, with nothing after it.

Press Ctrl and V together.




After the paste. Everything is in except the yellow lines. The marker line shows where they go.

1. The marker line that says YOU TYPE THIS PART. The yellow lines go in its place.

> **You should see** The table is at the end of your script, with a line that says YOU TYPE THIS PART where the yellow lines go.

Type the yellow lines in place of the line that says YOU TYPE THIS PART, so that no marker line is left.

> **You should see** The marker line is gone, and the lines you typed sit where it was, inside the table.

**IF IT GOES WRONG** A red wavy line under a word

A word is misspelled. Compare each line you typed with the same yellow line in the code block above, letter by letter, and check that your line ends the way that line ends.

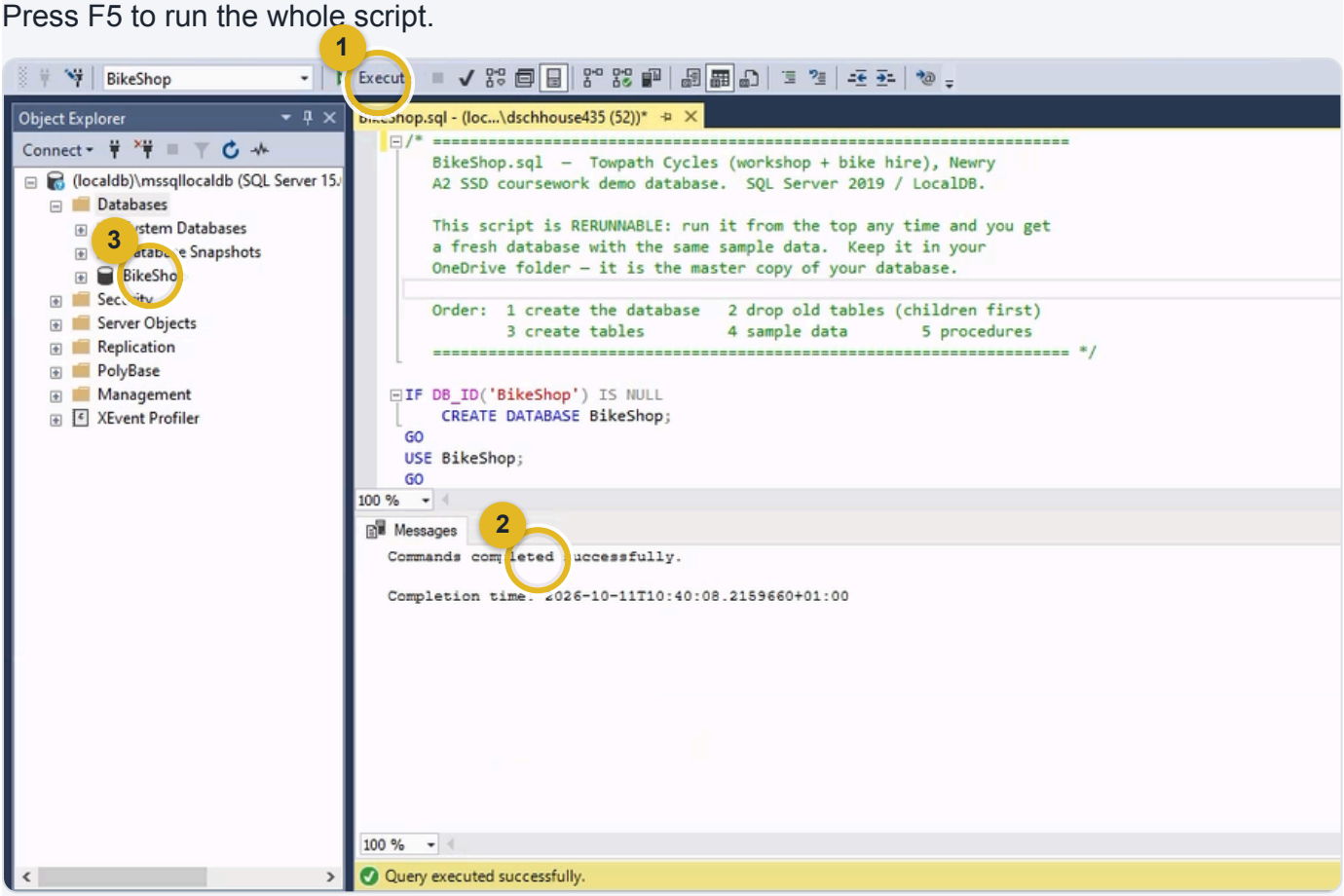
Press Ctrl and S together to save the script.

> **You should see** The star after BikeShop.sql on the tab disappears, because the file is saved.

Click anywhere in the script so that nothing is highlighted.

> **You should see** The cursor blinks in the script, and no line is highlighted in blue.

Press F5 to run the whole script.




Running the whole script with nothing highlighted. Commands completed successfully means every line ran, and the database appears in Object Explorer.

1. Click Execute, or press F5, with nothing highlighted.

2. The Messages pane says: Commands completed successfully.

3. BikeShop appears under Databases.

**You should see** The Messages pane says: Commands completed successfully. Every table so far was dropped and made again.

**IF IT GOES WRONG** Red text that says 'Incorrect syntax near …'

A bracket, a comma or a quote mark is missing or doubled. The line number in the red text is the line to look at. Compare it with the block above, character by character.

**IF IT GOES WRONG** Red text that says 'There is already an object named … in the database'

The DROP TABLE line for that table is missing from the top of the script, so the old table was still there. Check the ten DROP TABLE lines against the block in stage 1.

**IF IT GOES WRONG** Red text that says 'Could not drop object … because it is referenced by a FOREIGN KEY constraint'

A table is being dropped before a table that points at it. Put the DROP TABLE lines back in the order shown in stage 1: PartSale first, Customer last.

**IF IT GOES WRONG** Red text that says 'Foreign key … references invalid table …'

A table is being made before the table it points at. The tables must be in the order of this chapter: Customer, Staff, ServiceType, Appointment, Bike, Hire, Supplier, Part, PartOrder, PartSale.

# Read the Appointment table part by part, then add it to your script.

Appointment is the diary: one row is one booking of one bike for one mechanic at one time. It is the first table that points at other tables, and it points at three. Read it part by part, because the four FOREIGN KEY lines and the two CHECK rules at the end are the rules that keep the diary honest.

Copy this one **Appointment: one row per booking** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
CREATE TABLE Appointment (
    AppointmentID   INT IDENTITY(1,1) NOT NULL,
    CustomerID      INT           NOT NULL,
    MechanicID      INT           NOT NULL,
    ServiceTypeID   INT           NOT NULL,
    StartTime       DATETIME      NOT NULL,
    EndTime         DATETIME      NOT NULL,
    BikeDescription VARCHAR(80)  NOT NULL,
    Notes           VARCHAR(200) NULL,
    Status          VARCHAR(10)   NOT NULL
        CONSTRAINT DF_Appointment_Status DEFAULT ('Booked'),
    BookedByStaffID INT           NOT NULL,
    BookedOn        DATETIME      NOT NULL
        CONSTRAINT DF_Appointment_BookedOn DEFAULT (GETDATE()),
    CONSTRAINT PK_Appointment PRIMARY KEY (AppointmentID),
    CONSTRAINT FK_Appointment_Customer    FOREIGN KEY (CustomerID)
        REFERENCES Customer (CustomerID),
    CONSTRAINT FK_Appointment_Mechanic    FOREIGN KEY (MechanicID)
        REFERENCES Staff (StaffID),
    CONSTRAINT FK_Appointment_ServiceType FOREIGN KEY (ServiceTypeID)
        REFERENCES ServiceType (ServiceTypeID),
    CONSTRAINT FK_Appointment_BookedBy    FOREIGN KEY (BookedByStaffID)
        REFERENCES Staff (StaffID),
    CONSTRAINT CK_Appointment_Times  CHECK (EndTime > StartTime),
    CONSTRAINT CK_Appointment_Status CHECK (Status IN ('Booked', 'Done',
        'Cancelled'))
);
GO
```

**1 The table name and its number column**

Lines 1 to 2

AppointmentID is a number that the table gives out itself.

**2 Three numbers that point at other tables**

Lines 3 to 5

CustomerID is the number of a row in Customer. MechanicID is the number of a row in Staff. ServiceTypeID is the number of a row in ServiceType. Each is a plain INT here; the lines near the end say which table each one points at.

**3 When the job starts and ends**

Lines 6 to 7

DATETIME holds a day and a clock time together, such as 12 October 2026 at 09:30, which is what a diary needs. Both must be filled.

**4 What the bike is, and any notes**

Lines 8 to 9

BikeDescription is up to 80 letters, such as Blue Trek hybrid, and must be filled. Notes may be left empty.

**5 The status, Booked unless told otherwise**

Lines 10 to 11

Status is one short word. The DEFAULT means a new booking is Booked without anyone saying so.

**6 Who booked it and when**

Lines 12 to 14

BookedByStaffID is the number of the member of staff who made the booking. BookedOn is stamped with the date and time right now by its DEFAULT, GETDATE(), with the clock part kept this time.

**7 The primary key**

Line 15

AppointmentID names each booking.

**8 The first two foreign keys**

Lines 16 to 19

FOREIGN KEY (CustomerID) REFERENCES Customer (CustomerID) means: the number in CustomerID must be the number of a real row in Customer. A booking for customer 99, who does not exist, is refused. The second one says the same for MechanicID, which must be a real row in Staff. Each rule is named FK, the table, then what it points at.

**9 Two more foreign keys**

Lines 20 to 23

ServiceTypeID must be a real job in ServiceType. BookedByStaffID must be a real member of staff, so this table points at Staff twice: once for the mechanic, once for whoever took the booking.

**10 The end must be after the start**

Line 24

A booking that ends before it starts is refused by the table, whatever any program does.

**11 The list of statuses**

Lines 25 to 28

Status must be Booked, Done or Cancelled. The list runs over two lines because of its length; the server reads it as one. In stage 6 you will try to add a booking with the status Maybe, and this rule will turn it away. The closing bracket and the semicolon end the table. GO sends everything above it to the server before the next table starts.

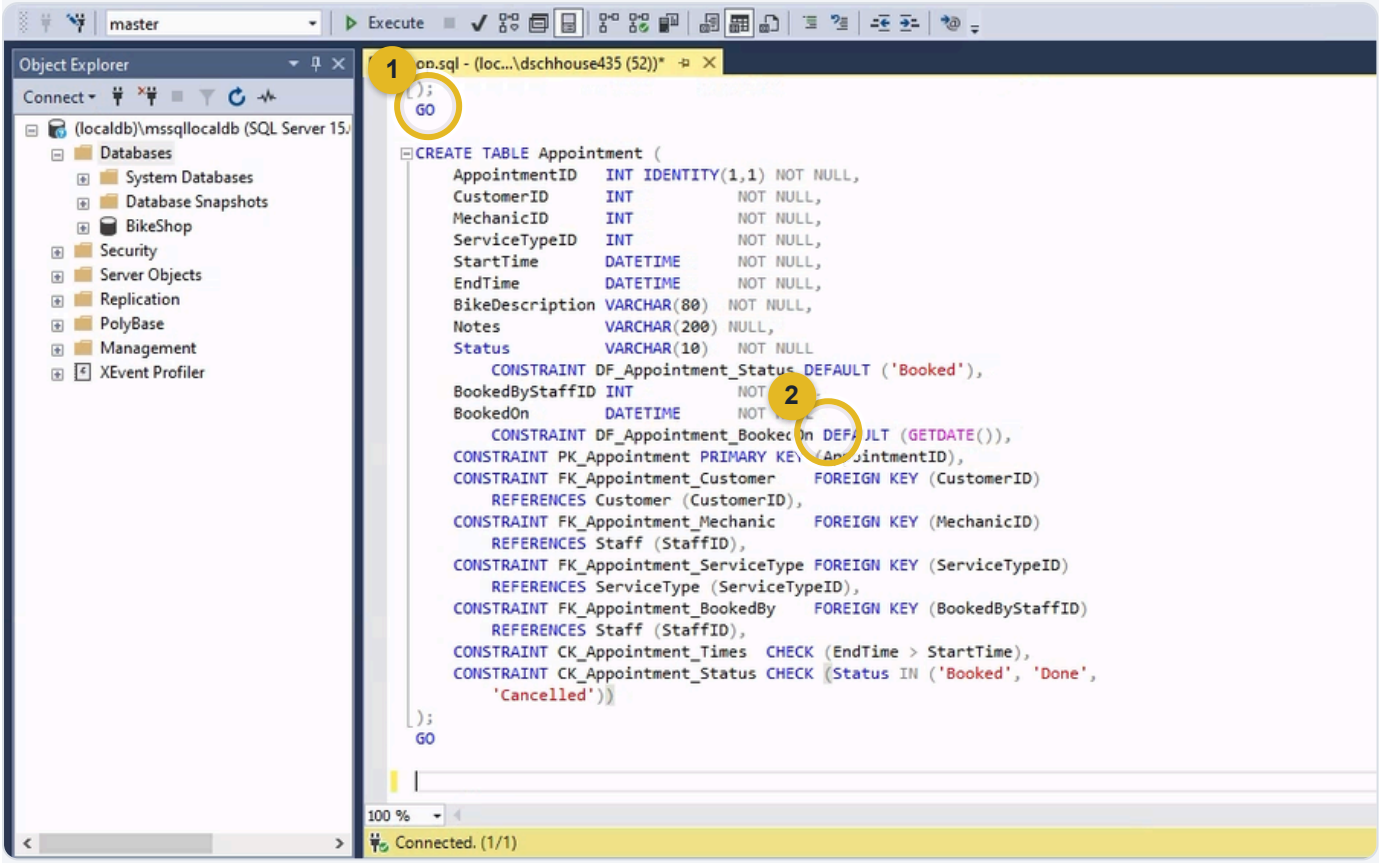
Press the Copy button under this step.

> **You should see** The button reads Copied for a moment.

Click on the empty line under the last GO, at the very end of the script.

> **You should see** The cursor blinks on an empty line under the last GO, with nothing after it.

Press Ctrl and V together.




The Appointment table pasted at the end of the script, under the GO that ends the ServiceType table. The next table goes on the empty line under its own GO.

1. The GO that ends the ServiceType table.

2. The pasted Appointment table, from CREATE TABLE Appointment down to GO.

> **You should see** The new table is at the end of your script, from CREATE TABLE down to its own GO.

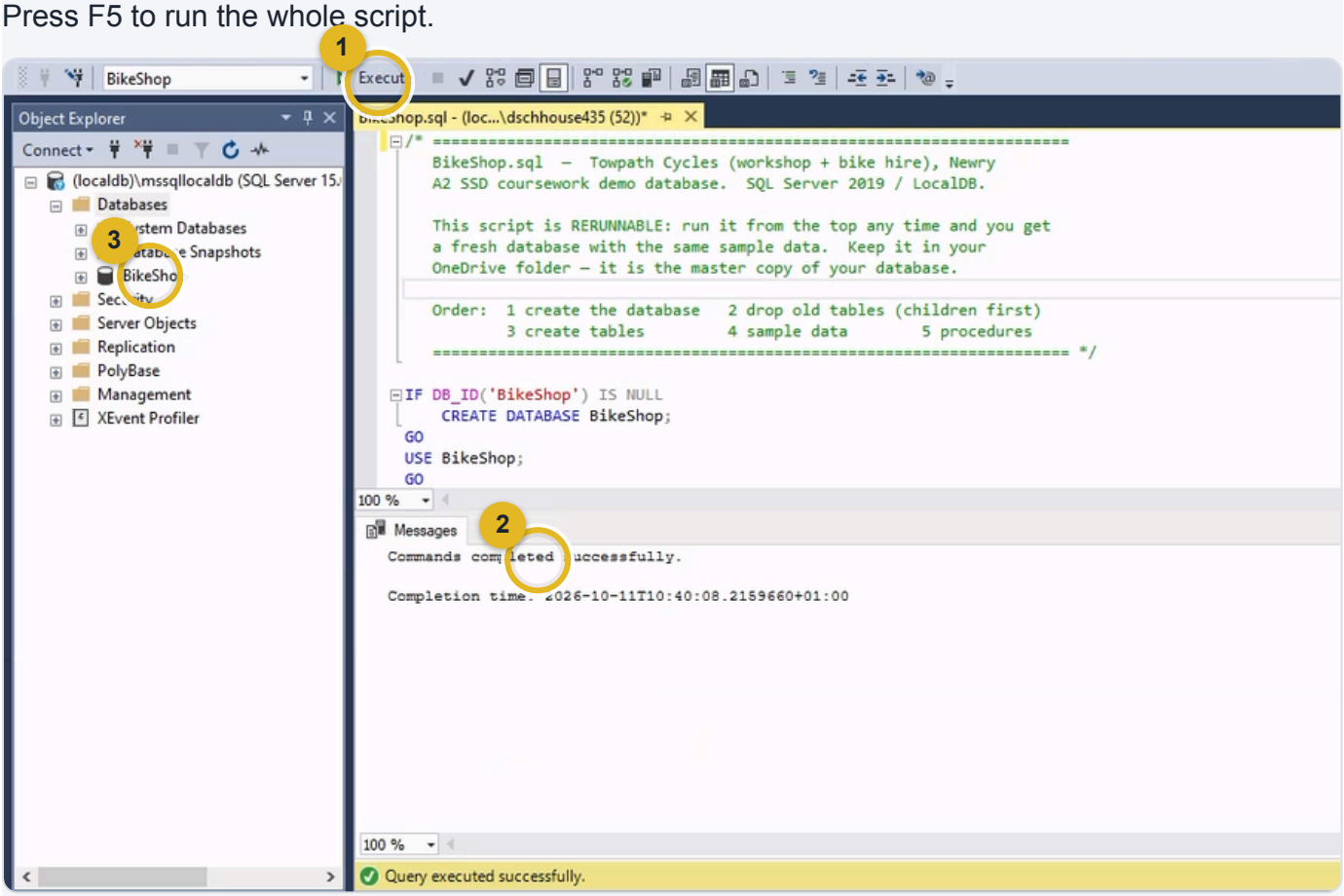
Press Ctrl and S together to save the script.

> **You should see** The star after BikeShop.sql on the tab disappears, because the file is saved.

Click anywhere in the script so that nothing is highlighted.

> **You should see** The cursor blinks in the script, and no line is highlighted in blue.

Press F5 to run the whole script.




Running the whole script with nothing highlighted. Commands completed successfully means every line ran, and the database appears in Object Explorer.

1. Click Execute, or press F5, with nothing highlighted.

2. The Messages pane says: Commands completed successfully.

3. BikeShop appears under Databases.

**You should see** The Messages pane says: Commands completed successfully. Every table so far was dropped and made again.

**IF IT GOES WRONG** Red text that says 'Incorrect syntax near …'

A bracket, a comma or a quote mark is missing or doubled. The line number in the red text is the line to look at. Compare it with the block above, character by character.

**IF IT GOES WRONG** Red text that says 'There is already an object named … in the database'

The DROP TABLE line for that table is missing from the top of the script, so the old table was still there. Check the ten DROP TABLE lines against the block in stage 1.

**IF IT GOES WRONG** Red text that says 'Could not drop object … because it is referenced by a FOREIGN KEY constraint'

A table is being dropped before a table that points at it. Put the DROP TABLE lines back in the order shown in stage 1: PartSale first, Customer last.

**IF IT GOES WRONG** Red text that says 'Foreign key … references invalid table …'

A table is being made before the table it points at. The tables must be in the order of this chapter: Customer, Staff, ServiceType, Appointment, Bike, Hire, Supplier, Part, PartOrder, PartSale.

STAGE 3 OF 6 · APPOINTMENT, BIKE AND HIRE · SCREEN 2 OF 3

# Add the Bike table, which has a rule that reads two columns at once.

Bike is every bike the shop owns, for hire or for sale. It is the longest table, because a bike has many facts, and two of its checks read more than one column at once. Read what is new in each part, then add it.

Copy this one **Bike: every bike the shop owns** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
CREATE TABLE Bike (
    BikeID      INT IDENTITY(1,1) NOT NULL,
    FrameNumber VARCHAR(20)  NOT NULL,
    Make        VARCHAR(30) NOT NULL,
    Model       VARCHAR(40) NOT NULL,
    Size        VARCHAR(2)   NOT NULL,
    Colour      VARCHAR(20) NOT NULL,
    Purpose     VARCHAR(4)   NOT NULL,
    DailyRate   DECIMAL(6,2) NULL,
    SalePrice   DECIMAL(8,2) NULL,
    Status      VARCHAR(10)  NOT NULL
        CONSTRAINT DF_Bike_Status DEFAULT ('Available'),
    CustomerID  INT          NULL,      -- reserved for / sold to
    SoldOn      DATE         NULL,
    CONSTRAINT PK_Bike PRIMARY KEY (BikeID),
    CONSTRAINT UQ_Bike_FrameNumber UNIQUE (FrameNumber),
    CONSTRAINT FK_Bike_Customer FOREIGN KEY (CustomerID)
        REFERENCES Customer (CustomerID),
    CONSTRAINT CK_Bike_Size    CHECK (Size IN ('XS', 'S', 'M', 'L', 'XL')),
    CONSTRAINT CK_Bike_Purpose CHECK (Purpose IN ('Hire', 'Sale')),
    CONSTRAINT CK_Bike_Status  CHECK (Status IN ('Available', 'Reserved',
        'Sold', 'In repair')),
    CONSTRAINT CK_Bike_DailyRate CHECK (DailyRate IS NULL OR DailyRate ⩾ 0),
    CONSTRAINT CK_Bike_SalePrice CHECK (SalePrice IS NULL OR SalePrice ⩾ 0),
    CONSTRAINT CK_Bike_PriceForPurpose CHECK (
        (Purpose = 'Hire' AND DailyRate IS NOT NULL)
            OR (Purpose = 'Sale' AND SalePrice IS NOT NULL)),
    CONSTRAINT CK_Bike_SoldNeedsDetails CHECK (
        (Status = 'Sold' AND SoldOn IS NOT NULL AND CustomerID IS NOT NULL)
            OR (Status <> 'Sold' AND SoldOn IS NULL))
);
GO
```

**1 The table name and its number column**

Lines 1 to 2

BikeID is a number that the table gives out itself.

**2 What the bike is**

Lines 3 to 7

FrameNumber is the code stamped on the frame, such as TK-2201. Make, Model, Size and Colour are text. Size is VARCHAR(2) because the longest size, XS or XL, is two letters.

**3 Hire or sale, and the two prices**

Lines 8 to 10

Purpose is the word Hire or the word Sale. DailyRate and SalePrice both allow NULL, because a hire bike has no sale price and a sale bike has no daily rate.

**4 The status, and who the bike is for**

Lines 11 to 14

Status starts as Available. CustomerID may be empty, and holds the customer a bike is reserved for or sold to. SoldOn is the day it was sold, empty until then.

**5 The key, the unique frame number and the foreign key**

Lines 15 to 18

No two bikes can share a frame number, and CustomerID, when it is filled, must be a real customer.

**6 The lists and the prices**

Lines 19 to 24

Three lists: a size must be XS, S, M, L or XL; a purpose must be Hire or Sale; a status must be Available, Reserved, Sold or In repair. The two price checks say IS NULL OR, which means: an empty price is fine, and a filled price must not be below zero.

**7 A hire bike needs a daily rate, a sale bike needs a price**

Lines 25 to 27

This CHECK reads two columns at once. Either the purpose is Hire and the daily rate is filled, or the purpose is Sale and the sale price is filled. A sale bike with no price is refused.

**8 A sold bike needs a date and a customer**

Lines 28 to 32

Either the status is Sold and both the day and the customer are filled, or the status is anything else and the day is empty. The two-character sign <> means is not equal to. The closing bracket and the semicolon end the table. GO sends everything above it to the server before the next table starts.

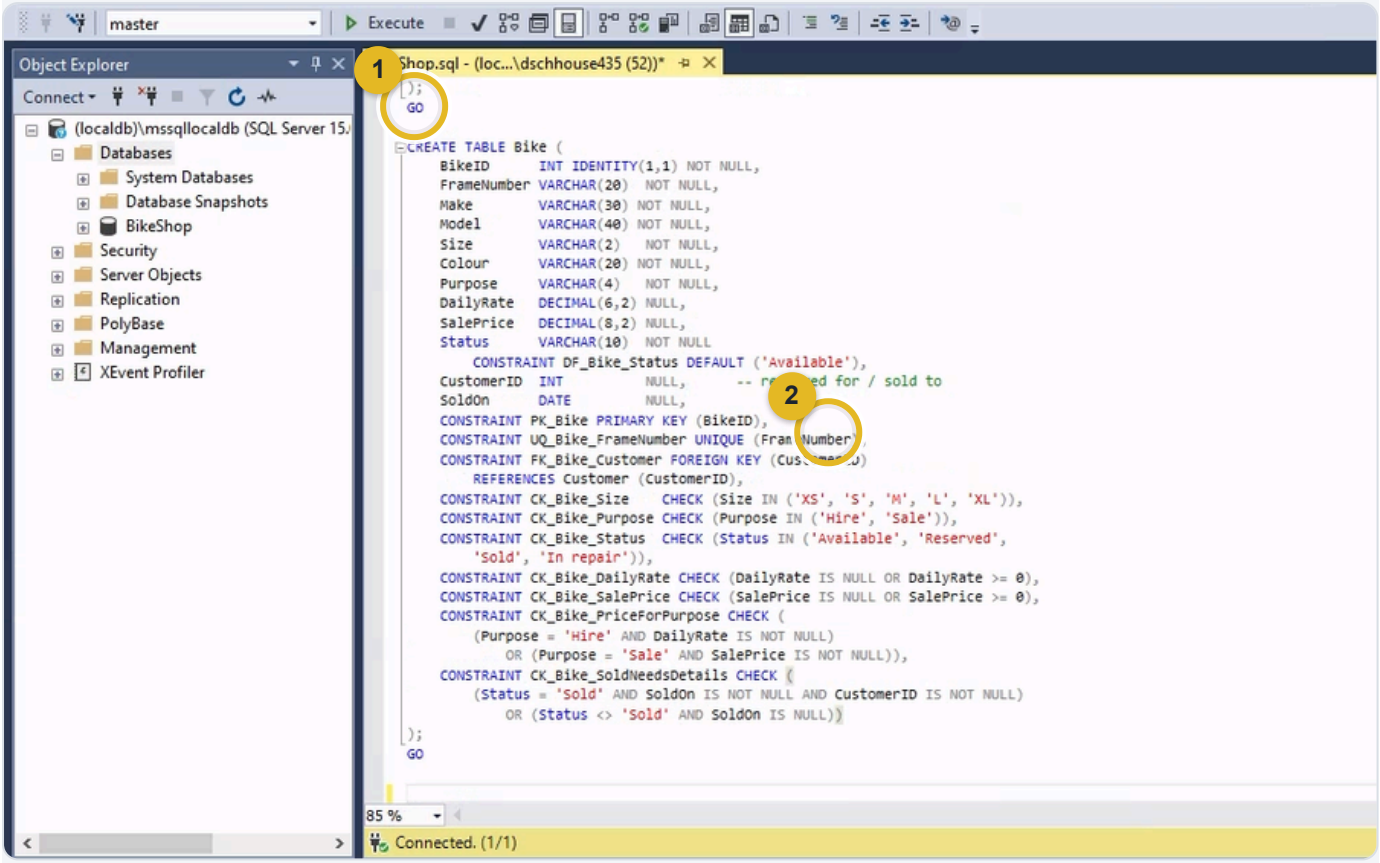
Press the Copy button under this step.

**You should see** The button reads Copied for a moment.

Click on the empty line under the last GO, at the very end of the script.

**You should see** The cursor blinks on an empty line under the last GO, with nothing after it.

Press Ctrl and V together.




The Bike table pasted at the end of the script, under the GO that ends the Appointment table. The next table goes on the empty line under its own GO.

1. The GO that ends the Appointment table.

2. The pasted Bike table, from CREATE TABLE Bike down to GO.

> **You should see** The new table is at the end of your script, from CREATE TABLE down to its own GO.

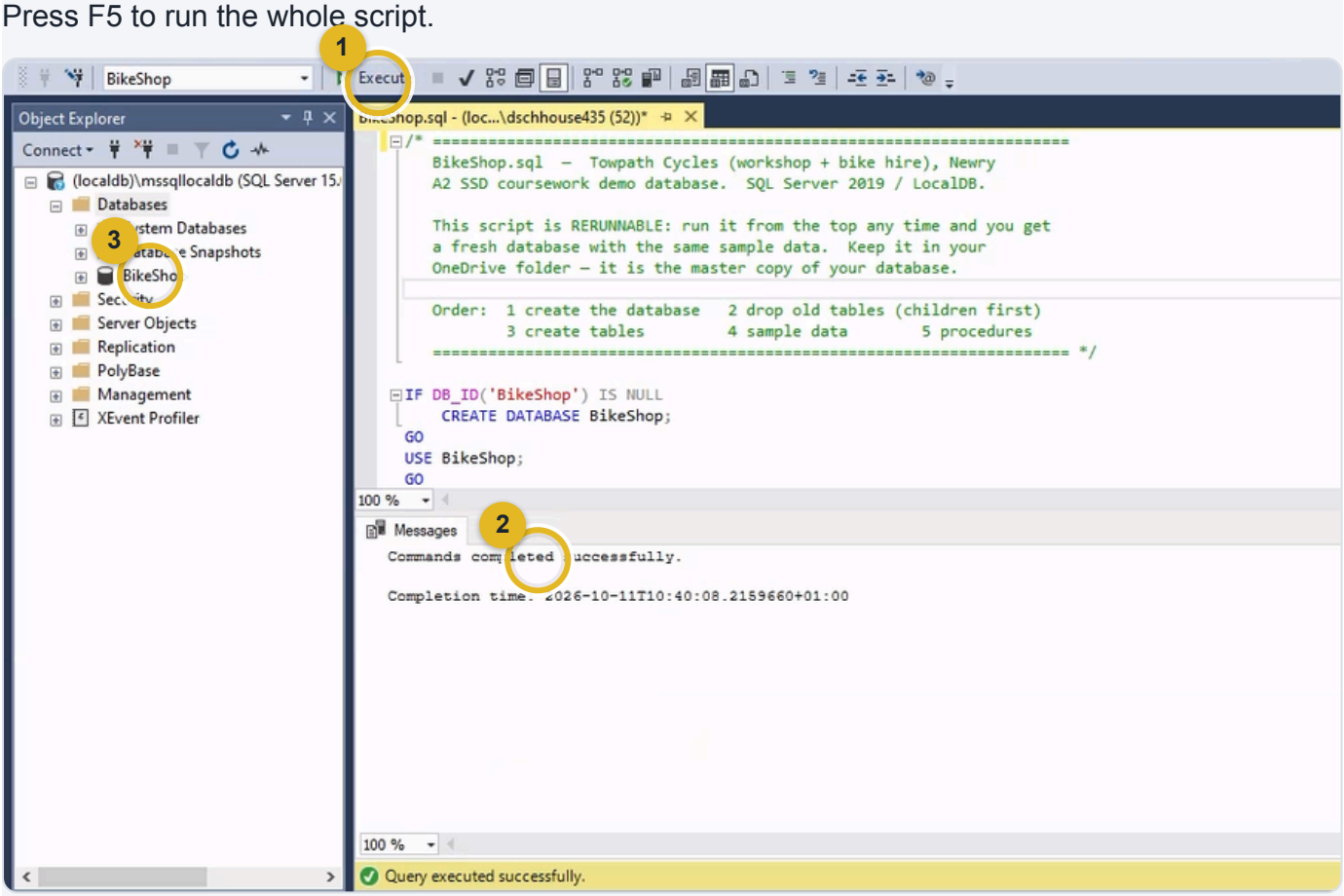
Press Ctrl and S together to save the script.

> **You should see** The star after BikeShop.sql on the tab disappears, because the file is saved.

Click anywhere in the script so that nothing is highlighted.

> **You should see** The cursor blinks in the script, and no line is highlighted in blue.

Press F5 to run the whole script.




Running the whole script with nothing highlighted. Commands completed successfully means every line ran, and the database appears in Object Explorer.

1. Click Execute, or press F5, with nothing highlighted.
2. The Messages pane says: Commands completed successfully.
3. BikeShop appears under Databases.

> **You should see** The Messages pane says: Commands completed successfully. Every table so far was dropped and made again.

**IF IT GOES WRONG** Red text that says 'Incorrect syntax near …'

A bracket, a comma or a quote mark is missing or doubled. The line number in the red text is the line to look at. Compare it with the block above, character by character.

**IF IT GOES WRONG** Red text that says 'There is already an object named … in the database'

The DROP TABLE line for that table is missing from the top of the script, so the old table was still there. Check the ten DROP TABLE lines against the block in stage 1.

**IF IT GOES WRONG** Red text that says 'Could not drop object … because it is referenced by a FOREIGN KEY constraint'

A table is being dropped before a table that points at it. Put the DROP TABLE lines back in the order shown in stage 1: PartSale first, Customer last.

**IF IT GOES WRONG** Red text that says 'Foreign key … references invalid table …'

A table is being made before the table it points at. The tables must be in the order of this chapter: Customer, Staff, ServiceType, Appointment, Bike, Hire, Supplier, Part, PartOrder, PartSale.

# Add the Hire table, and read the deposit check.

Hire is one row per bike hire: which bike, which customer, the day it goes out and the day it comes back. Most of it you have seen before. Read the deposit check with care, because it is the shape of every money rule in your own scenario.

Copy this one **Hire: one row per bike hire** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
CREATE TABLE Hire (
    HireID          INT IDENTITY(1,1) NOT NULL,
    BikeID          INT          NOT NULL,
    CustomerID      INT          NOT NULL,
    StartDate       DATE         NOT NULL,
    EndDate         DATE         NOT NULL,    -- the day the bike comes back
    TotalCost       DECIMAL(8,2) NOT NULL,
    DepositPaid     DECIMAL(8,2) NOT NULL
        CONSTRAINT DF_Hire_DepositPaid DEFAULT (0),
    Status          VARCHAR(10)  NOT NULL
        CONSTRAINT DF_Hire_Status DEFAULT ('Booked'),
    BookedByStaffID INT          NOT NULL,
    BookedOn        DATETIME     NOT NULL
        CONSTRAINT DF_Hire_BookedOn DEFAULT (GETDATE()),
    CONSTRAINT PK_Hire PRIMARY KEY (HireID),
    CONSTRAINT FK_Hire_Bike     FOREIGN KEY (BikeID)
        REFERENCES Bike (BikeID),
    CONSTRAINT FK_Hire_Customer FOREIGN KEY (CustomerID)
        REFERENCES Customer (CustomerID),
    CONSTRAINT FK_Hire_BookedBy FOREIGN KEY (BookedByStaffID)
        REFERENCES Staff (StaffID),
    CONSTRAINT CK_Hire_Dates   CHECK (EndDate > StartDate),
    CONSTRAINT CK_Hire_Total   CHECK (TotalCost ⩾ 0),
    CONSTRAINT CK_Hire_Deposit CHECK (DepositPaid ⩾ 0),
    CONSTRAINT CK_Hire_Status  CHECK (Status IN ('Booked', 'Out', 'Returned',
        'Cancelled'))
);
GO
```

**1 The number column and two pointers**

Lines 1 to 4

HireID is given out by the table. BikeID and CustomerID are the numbers of a real bike and a real customer.

**2 The two days**

Lines 5 to 6

A hire is by the day, not the hour, so both are DATE with no clock part. The comment says what EndDate means: the day the bike comes back.

**3 The money**

Lines 7 to 9

TotalCost is what the hire costs. DepositPaid is what was paid up front, and it starts at 0 unless a value is given.

**4 The status, who booked it and when**

Lines 10 to 14

Status is Booked until told otherwise. BookedByStaffID is the staff member who took the booking, and BookedOn is the date and time it was made, stamped by GETDATE().

**5 The key and three foreign keys**

Lines 15 to 21

HireID names the row. The three FOREIGN KEY rules say the bike, the customer and the staff member must each be a real row in their own table.

**6 The dates and the total**

Lines 22 to 23

The bike must come back after it goes out, and the total cost cannot be below zero.

**7 The deposit check, walked**

Line 24

CONSTRAINT names the rule: CK for check, Hire for the table, Deposit for what it checks. CHECK opens the test. Inside the brackets, DepositPaid >= 0 is true for a deposit of 0 or more and false for a deposit of minus 5. When the test is false, the row is refused and the red text names CK_Hire_Deposit. A program never gets the chance to store a negative deposit.

**8 The list of statuses**

Lines 25 to 28

Status must be Booked, Out, Returned or Cancelled, and nothing else. The closing bracket and the semicolon end the table. GO sends everything above it to the server before the next table starts.

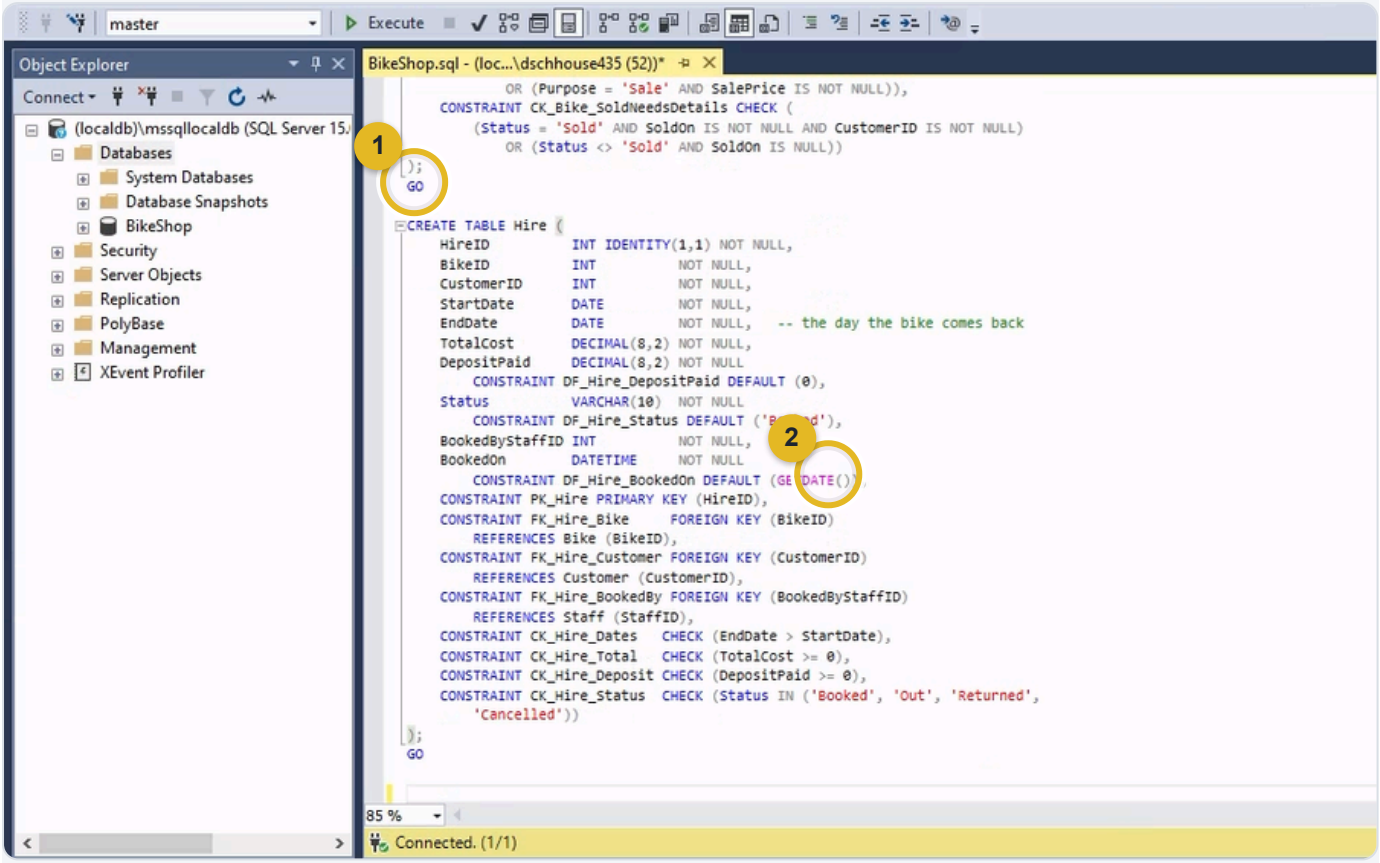
Press the Copy button under this step.

> **You should see** The button reads Copied for a moment.

Click on the empty line under the last GO, at the very end of the script.

> **You should see** The cursor blinks on an empty line under the last GO, with nothing after it.

Press Ctrl and V together.




The Hire table pasted at the end of the script, under the GO that ends the Bike table. The next table goes on the empty line under its own GO.

1. The GO that ends the Bike table.

2. The pasted Hire table, from CREATE TABLE Hire down to GO.

> **You should see** The new table is at the end of your script, from CREATE TABLE down to its own GO.

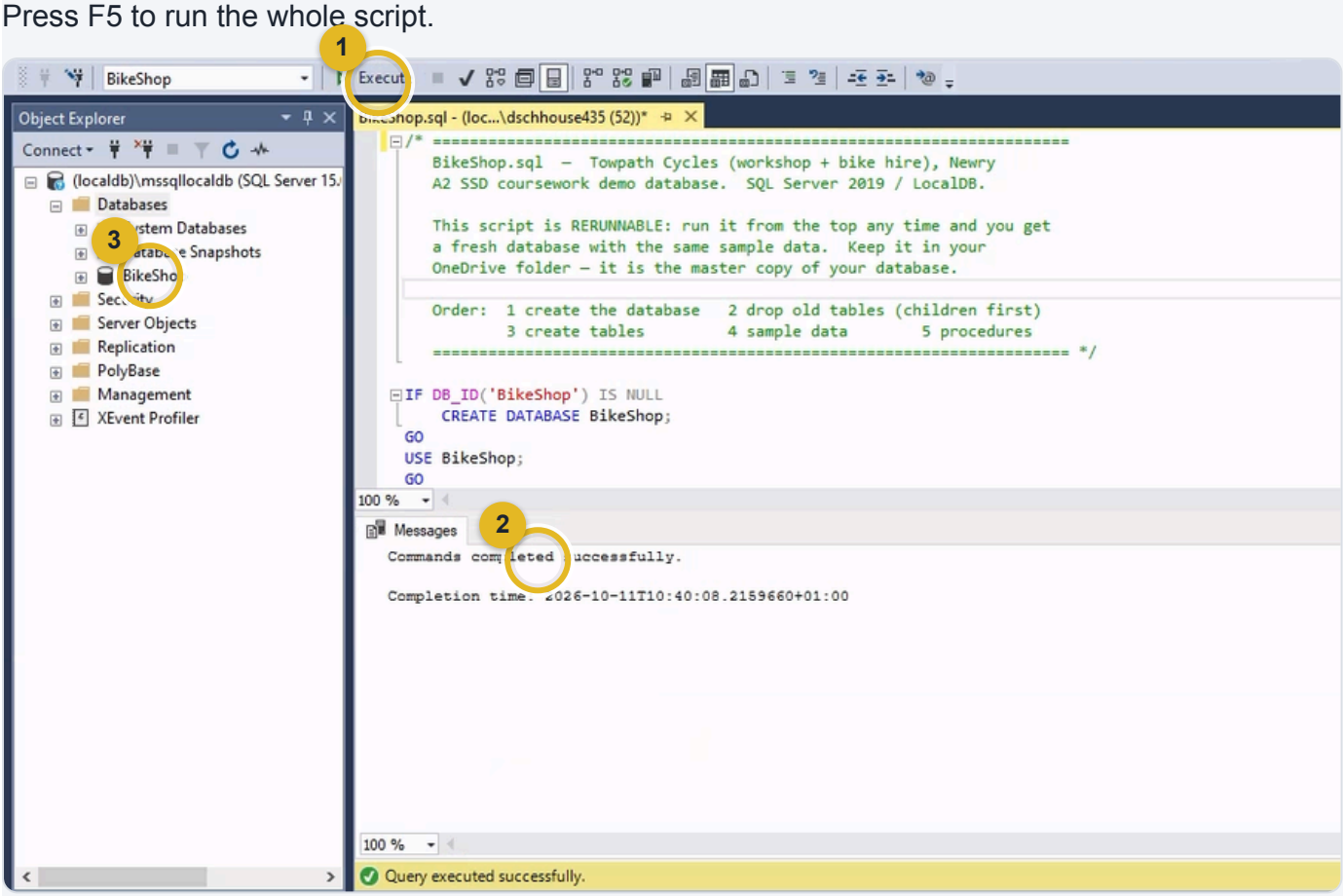
Press Ctrl and S together to save the script.

> **You should see** The star after BikeShop.sql on the tab disappears, because the file is saved.

Click anywhere in the script so that nothing is highlighted.

> **You should see** The cursor blinks in the script, and no line is highlighted in blue.

Press F5 to run the whole script.




Running the whole script with nothing highlighted. Commands completed successfully means every line ran, and the database appears in Object Explorer.

1. Click Execute, or press F5, with nothing highlighted.

2. The Messages pane says: Commands completed successfully.

3. BikeShop appears under Databases.

> **You should see** The Messages pane says: Commands completed successfully. Every table so far was dropped and made again.

**IF IT GOES WRONG** Red text that says 'Incorrect syntax near …'

A bracket, a comma or a quote mark is missing or doubled. The line number in the red text is the line to look at. Compare it with the block above, character by character.

**IF IT GOES WRONG** Red text that says 'There is already an object named … in the database'

The DROP TABLE line for that table is missing from the top of the script, so the old table was still there. Check the ten DROP TABLE lines against the block in stage 1.

**IF IT GOES WRONG** Red text that says 'Could not drop object … because it is referenced by a FOREIGN KEY constraint'

A table is being dropped before a table that points at it. Put the DROP TABLE lines back in the order shown in stage 1: PartSale first, Customer last.

**IF IT GOES WRONG** Red text that says 'Foreign key … references invalid table …'

A table is being made before the table it points at. The tables must be in the order of this chapter: Customer, Staff, ServiceType, Appointment, Bike, Hire, Supplier, Part, PartOrder, PartSale.

STAGE 4 OF 6 · SUPPLIER AND PART, THEN PARTORDER AND PARTSALE · SCREEN 1 OF 4

# Write the Supplier table yourself.

Supplier is the smallest table in the database: who the shop buys parts from. It has four columns, a primary key and one UNIQUE rule, and you have seen every piece of it in Customer and Staff. Write it from the plan below, run it, and only then open the answer to compare.

Your turn **Supplier: who the shop buys from** SQL

1. In SQL Server Management Studio (SSMS), click on the empty line under the last GO at the end of your script and type CREATE TABLE Supplier, a space and an opening bracket.
2. Type four columns, one per line with a comma after each: SupplierID as an INT that the table gives out itself and can never be empty; Name as text of up to 60 letters that must be filled; Phone as text of up to 15 characters that must be filled; Email as text of up to 100 characters that may be empty.
3. Type a CONSTRAINT line that makes SupplierID the primary key, named PK_Supplier, with a comma after it.
4. Type a CONSTRAINT line named UQ_Supplier_Name that makes Name UNIQUE, with no comma after it.
5. Close the bracket with a semicolon after it, and type GO on the next line.

**THE TEST** Running the whole script with nothing highlighted is the test: the Messages pane must say Commands completed successfully, and after a Refresh, Supplier must be in the Tables folder in Object Explorer.

**YOU SHOULD SEE** Commands completed successfully, and seven tables in the Tables folder once you right-click it and click Refresh. Red text means a line to fix: the line number in it is the one to look at.

**HINT** Customer is the pattern. Its first line is CustomerID INT IDENTITY(1,1) NOT NULL, and its UNIQUE rule would be written CONSTRAINT UQ_Supplier_Name UNIQUE (Name). A column that may be empty says NULL where the others say NOT NULL.

▾ **Stuck? Show me the finished table**

**Read it, then write yours** **Supplier** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  CREATE TABLE Supplier (
2      SupplierID INT IDENTITY(1,1) NOT NULL,
3      Name       VARCHAR(60)  NOT NULL,
4      Phone      VARCHAR(15)   NOT NULL,
5      Email      VARCHAR(100) NULL,
6      CONSTRAINT PK_Supplier PRIMARY KEY (SupplierID),
7      CONSTRAINT UQ_Supplier_Name UNIQUE (Name)
8  );
9  GO
```

**1 The table name and its number column**

Lines 1 to 2

SupplierID is a number that the table gives out itself.

**2 The name, the phone number and the email address**

Lines 3 to 5

Name holds up to 60 letters, because a supplier's name is longer than a person's. Phone is text, as in Customer. Email may be empty.

**3 The key and the UNIQUE rule**

Lines 6 to 9

SupplierID names the row, and no two suppliers can share a name. The closing bracket and the semicolon end the table. GO sends everything above it to the server before the next table starts.

Do the five numbered tasks in the Supplier box above, in order.

> **You should see** Your Supplier table is under the last GO, ending with a closing bracket, a semicolon and GO on its own line.

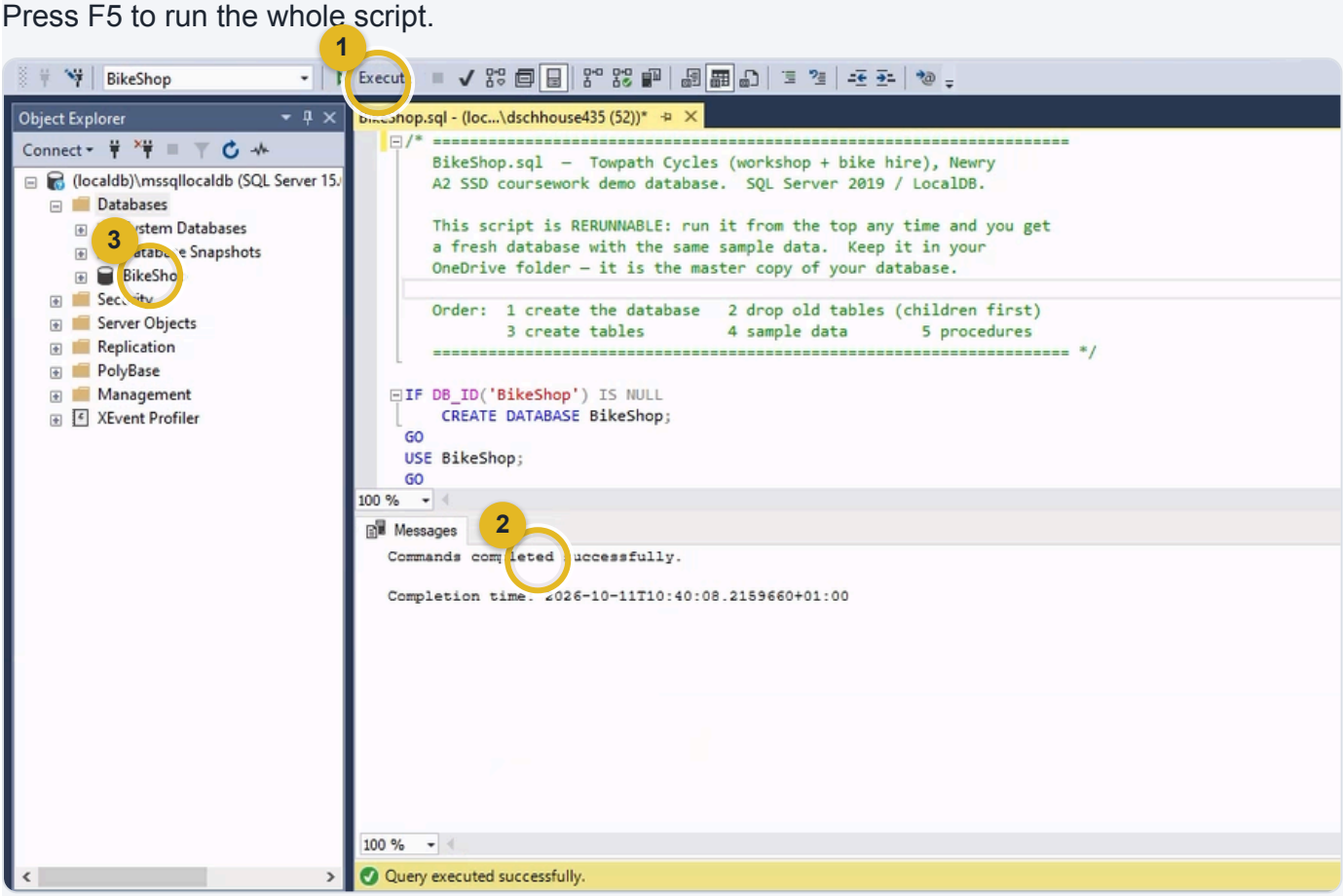
Press Ctrl and S together to save the script.

> **You should see** The star after BikeShop.sql on the tab disappears, because the file is saved.

Click anywhere in the script so that nothing is highlighted.

> **You should see** The cursor blinks in the script, and no line is highlighted in blue.

Press F5 to run the whole script.




Running the whole script with nothing highlighted. Commands completed successfully means every line ran, and the database appears in Object Explorer.

1. Click Execute, or press F5, with nothing highlighted.
2. The Messages pane says: Commands completed successfully.
3. BikeShop appears under Databases.

**You should see** The Messages pane says: Commands completed successfully.

**IF IT GOES WRONG** Red text that says 'Incorrect syntax near …'

A bracket, a comma or a quote mark is missing or doubled. The line number in the red text is the line to look at. Compare it with the block above, character by character.

**IF IT GOES WRONG** Red text that says 'There is already an object named … in the database'

The DROP TABLE line for that table is missing from the top of the script, so the old table was still there. Check the ten DROP TABLE lines against the block in stage 1.

**IF IT GOES WRONG** Red text that says 'Could not drop object … because it is referenced by a FOREIGN KEY constraint'

A table is being dropped before a table that points at it. Put the DROP TABLE lines back in the order shown in stage 1: PartSale first, Customer last.

**IF IT GOES WRONG** Red text that says 'Foreign key … references invalid table …'

A table is being made before the table it points at. The tables must be in the order of this chapter: Customer, Staff, ServiceType, Appointment, Bike, Hire, Supplier, Part, PartOrder, PartSale.

**IF IT GOES WRONG** Red text that says 'Column, parameter, or variable … Cannot find data type …'

A type is misspelled, such as VARCHAR without the R or DECIMAL with the letters swapped. The types in this chapter are INT, VARCHAR, DECIMAL, DATE, DATETIME and BIT.

In Object Explorer, right-click the Tables folder under BikeShop and click Refresh.

**You should see** Supplier is in the list of tables.

Click Stuck? Show me the finished table under the tasks.

**You should see** The finished Supplier table opens under the tasks.

Read the finished table beside your own, line by line.

**You should see** Your table has the same four columns, the same two rules and the same GO. Spaces and the order of NOT NULL and the type do not matter; names and brackets do.

# Write the Part table yourself, with the rule that stock can never go below zero.

Part is every part on the shelf: what it is called, who supplies it, what the shop pays, what the customer pays, and how many are in stock. It has one foreign key, to Supplier, and five CHECK rules. One of those rules is the most important line in this whole script for chapter 8, where parts are sold: stock can never go below zero. Write the table from the plan, run it, then compare.

Your turn **Part: every part on the shelf** SQL

1. Click on the empty line under the last GO and type CREATE TABLE Part, a space and an opening bracket.
2. Type the columns, one per line with a comma after each: PartID, an INT the table gives out itself; PartCode, text of up to 12 characters; Name, text of up to 60; SupplierID, an INT; UnitCost and SellPrice, each DECIMAL(7,2); QtyInStock, an INT with a DEFAULT of 0 named DF_Part_QtyInStock; ReorderLevel and ReorderQty, each an INT. Every column is NOT NULL.
3. Type the primary key on PartID, named PK_Part, and a UNIQUE rule on PartCode, named UQ_Part_PartCode.
4. Type a FOREIGN KEY named FK_Part_Supplier that says SupplierID must be a real row in Supplier, in the shape of FK_Bike_Customer.
5. Type five CHECK rules, each named CK_Part_ and the column: UnitCost and SellPrice are each 0 or more; QtyInStock is 0 or more; ReorderLevel is 0 or more; ReorderQty is more than 0. The last one has no comma after it.
6. Close the bracket with a semicolon after it, and type GO on the next line.

**THE TEST** The whole script is the test again. Commands completed successfully, and Part in the Tables folder after a Refresh.

**YOU SHOULD SEE** Commands completed successfully, and eight tables in the Tables folder after a Refresh. The one rule the examiner will look for is CHECK (QtyInStock >= 0), and the answer shows exactly where it goes.

**HINT** A DEFAULT sits on the column's own line, after NOT NULL: CONSTRAINT DF_Part_QtyInStock DEFAULT (0). A foreign key is two lines: CONSTRAINT FK_Part_Supplier FOREIGN KEY (SupplierID), then REFERENCES Supplier (SupplierID). The checks are the same shape as CK_Hire_Deposit.

**Stuck? Show me the finished table**

Read it, then write yours **Part** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
CREATE TABLE Part (
    PartID       INT IDENTITY(1,1) NOT NULL,
    PartCode     VARCHAR(12)  NOT NULL,
    Name         VARCHAR(60) NOT NULL,
    SupplierID   INT          NOT NULL,
    UnitCost     DECIMAL(7,2) NOT NULL,    -- what we pay
    SellPrice    DECIMAL(7,2) NOT NULL,    -- what the customer pays
    QtyInStock   INT          NOT NULL
        CONSTRAINT DF_Part_QtyInStock DEFAULT (0),
    ReorderLevel INT          NOT NULL,
    ReorderQty   INT          NOT NULL,
    CONSTRAINT PK_Part PRIMARY KEY (PartID),
    CONSTRAINT UQ_Part_PartCode UNIQUE (PartCode),
    CONSTRAINT FK_Part_Supplier FOREIGN KEY (SupplierID)
        REFERENCES Supplier (SupplierID),
    CONSTRAINT CK_Part_UnitCost     CHECK (UnitCost ≥ 0),
    CONSTRAINT CK_Part_SellPrice    CHECK (SellPrice ≥ 0),
    -- stock can never go below zero
    CONSTRAINT CK_Part_QtyInStock   CHECK (QtyInStock ≥ 0),
    CONSTRAINT CK_Part_ReorderLevel CHECK (ReorderLevel ≥ 0),
    CONSTRAINT CK_Part_ReorderQty   CHECK (ReorderQty > 0)
);
GO
```

**1 The table name and its number column**

Lines 1 to 2

PartID is a number that the table gives out itself.

**2 The code, the name and the supplier**

Lines 3 to 5

PartCode is the short code on the shelf label, such as TUBE-700. Name is what a person reads. SupplierID points at Supplier.

**3 The two prices**

Lines 6 to 7

UnitCost is what the shop pays the supplier, and SellPrice is what the customer pays. Both are DECIMAL(7,2), so the pennies are exact. The comments say which is which.

**4 How many, and when to order more**

Lines 8 to 11

QtyInStock starts at 0 by DEFAULT. ReorderLevel is the count at which more should be ordered, and ReorderQty is how many to order then. Chapter 8 uses both.

**5 The key, the unique code and the foreign key**

Lines 12 to 15

PartID names the row, no two parts share a code, and the supplier must be a real row in Supplier.

**6 Neither price below zero**

Lines 16 to 17

These are two checks of the same shape as the deposit check in Hire.

**7 Stock can never go below zero**

Line 19

CHECK (QtyInStock >= 0) is the rule the whole stock screen in chapter 8 relies on. If a program ever tries to sell three of a part when two are left, this one line refuses it, and the count stays right.

**8 The reorder rules**

Lines 20 to 23

The reorder level may be 0, meaning never reorder. The reorder quantity must be more than 0, because ordering nothing makes no sense. The closing bracket and the semicolon end the table. GO sends everything above it to the server before the next table starts.

Do the five numbered tasks in the Part box above, in order.

> **You should see** Your Part table is under the last GO, ending with a closing bracket, a semicolon and GO on its own line.

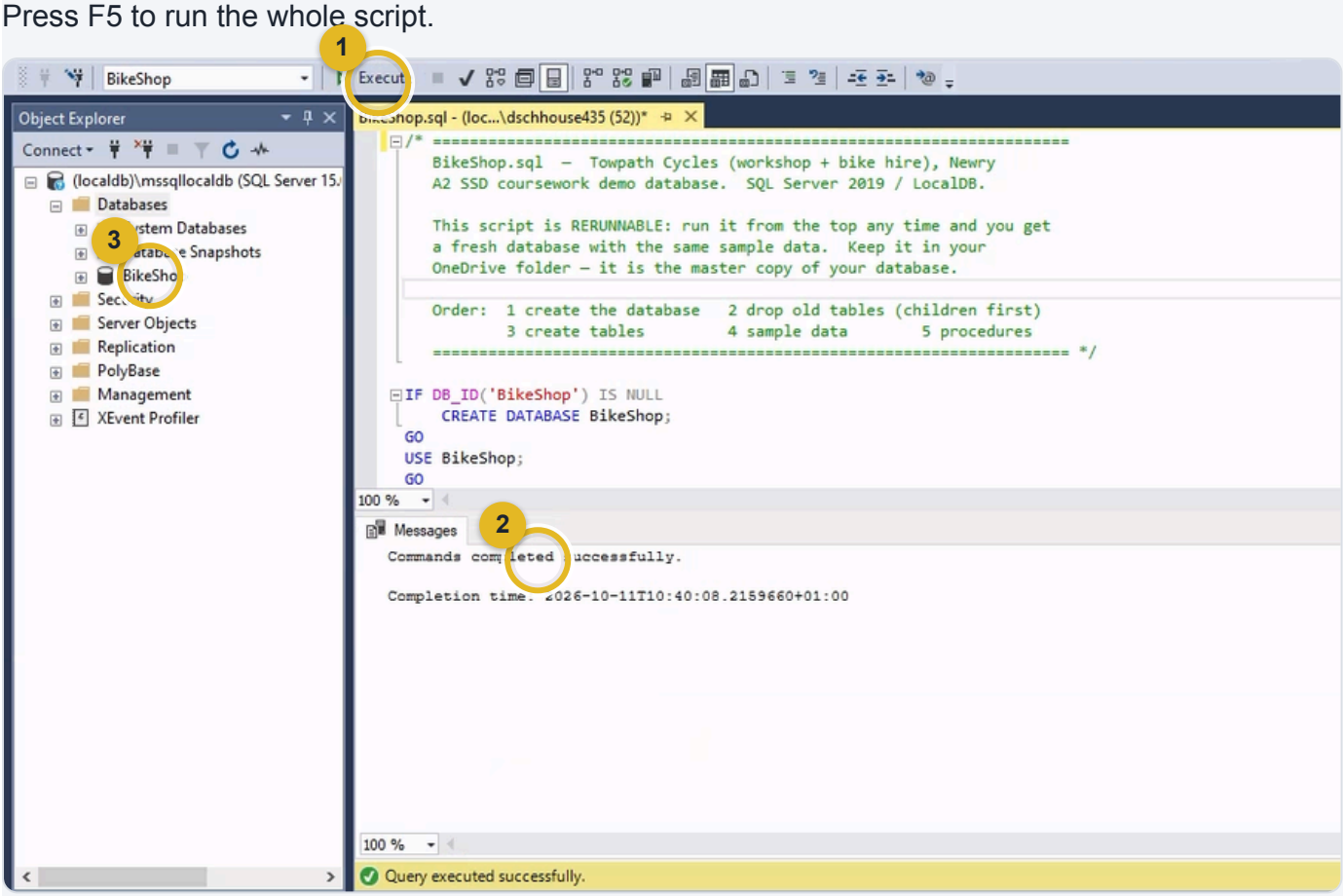
Press Ctrl and S together to save the script.

> **You should see** The star after BikeShop.sql on the tab disappears, because the file is saved.

Click anywhere in the script so that nothing is highlighted.

> **You should see** The cursor blinks in the script, and no line is highlighted in blue.

Press F5 to run the whole script.




Running the whole script with nothing highlighted. Commands completed successfully means every line ran, and the database appears in Object Explorer.

1. Click Execute, or press F5, with nothing highlighted.
2. The Messages pane says: Commands completed successfully.
3. BikeShop appears under Databases.

**You should see** The Messages pane says: Commands completed successfully.

**IF IT GOES WRONG** Red text that says 'Incorrect syntax near …'

A bracket, a comma or a quote mark is missing or doubled. The line number in the red text is the line to look at. Compare it with the block above, character by character.

**IF IT GOES WRONG** Red text that says 'There is already an object named … in the database'

The DROP TABLE line for that table is missing from the top of the script, so the old table was still there. Check the ten DROP TABLE lines against the block in stage 1.

**IF IT GOES WRONG** Red text that says 'Could not drop object … because it is referenced by a FOREIGN KEY constraint'

A table is being dropped before a table that points at it. Put the DROP TABLE lines back in the order shown in stage 1: PartSale first, Customer last.

**IF IT GOES WRONG** Red text that says 'Foreign key … references invalid table …'

A table is being made before the table it points at. The tables must be in the order of this chapter: Customer, Staff, ServiceType, Appointment, Bike, Hire, Supplier, Part, PartOrder, PartSale.

**IF IT GOES WRONG** Red text that says 'Column, parameter, or variable … Cannot find data type …'

A type is misspelled, such as VARCHAR without the R or DECIMAL with the letters swapped. The types in this chapter are INT, VARCHAR, DECIMAL, DATE, DATETIME and BIT.

In Object Explorer, right-click the Tables folder under BikeShop and click Refresh.

**You should see** Part is in the list of tables.

Click Stuck? Show me the finished table under the tasks.

**You should see** The finished Part table opens under the tasks.

Read the finished table beside your own, line by line.

**You should see** Your table has the same nine columns and the same eight rules, and the line CHECK (QtyInStock >= 0) is in it with the name CK_Part_QtyInStock.

STAGE 4 OF 6 · SUPPLIER AND PART, THEN PARTORDER AND PARTSALE · SCREEN 3 OF 4

# Add the PartOrder table, which records orders to suppliers.

PartOrder is one row per order the shop places with a supplier for more of a part. Nothing in it is new except that three of its columns stay empty until the order arrives. Read what each part does, then add it.

Copy this one **PartOrder: one row per order to a supplier** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  CREATE TABLE PartOrder (
2      PartOrderID     INT IDENTITY(1,1) NOT NULL,
3      PartID          INT         NOT NULL,
4      QtyOrdered      INT         NOT NULL,
5      OrderedOn       DATE        NOT NULL
6          CONSTRAINT DF_PartOrder_OrderedOn DEFAULT (CAST(GETDATE() AS DATE)),
7      ExpectedOn      DATE        NULL,
8      ReceivedOn      DATE        NULL,
9      QtyReceived     INT         NULL,
10     Status          VARCHAR(10) NOT NULL
11         CONSTRAINT DF_PartOrder_Status DEFAULT ('Ordered'),
12     PlacedByStaffID INT         NOT NULL,
13     CONSTRAINT PK_PartOrder PRIMARY KEY (PartOrderID),
14     CONSTRAINT FK_PartOrder_Part     FOREIGN KEY (PartID)
15         REFERENCES Part (PartID),
16     CONSTRAINT FK_PartOrder_PlacedBy FOREIGN KEY (PlacedByStaffID)
17         REFERENCES Staff (StaffID),
18     CONSTRAINT CK_PartOrder_Qty         CHECK (QtyOrdered > 0),
19     CONSTRAINT CK_PartOrder_QtyReceived CHECK (QtyReceived IS NULL
20         OR QtyReceived ⩾ 0),
21     CONSTRAINT CK_PartOrder_Status      CHECK (Status IN ('Ordered', 'Received',
22         'Cancelled'))
23 );
24 GO
```

**1 The number, the part and how many**

Lines 1 to 4

PartOrderID is given out by the table. PartID points at the part. QtyOrdered is how many were asked for.

**2 The day it was ordered**

Lines 5 to 6

OrderedOn is a DATE that its DEFAULT stamps with today, using the same CAST of GETDATE() as DateJoined in Customer.

**3 Three columns that wait for the delivery**

Lines 7 to 9

ExpectedOn, ReceivedOn and QtyReceived all allow NULL, because an order that has not arrived yet has no delivery day and no count received.

**4 The status, and who placed it**

Lines 10 to 12

Status starts as Ordered until something changes it, and PlacedByStaffID is the number of the member of staff who placed the order.

**5 The key and two foreign keys**

Lines 13 to 17

The part must be a real row in Part, and the staff member a real row in Staff.

**6 The two quantity checks**

Lines 18 to 20

QtyOrdered must be more than 0. QtyReceived may be empty, and when it is filled it must be 0 or more.

**7 The list of statuses**

Lines 21 to 24

Status must be Ordered, Received or Cancelled. The closing bracket and the semicolon end the table. GO sends everything above it to the server before the next table starts.

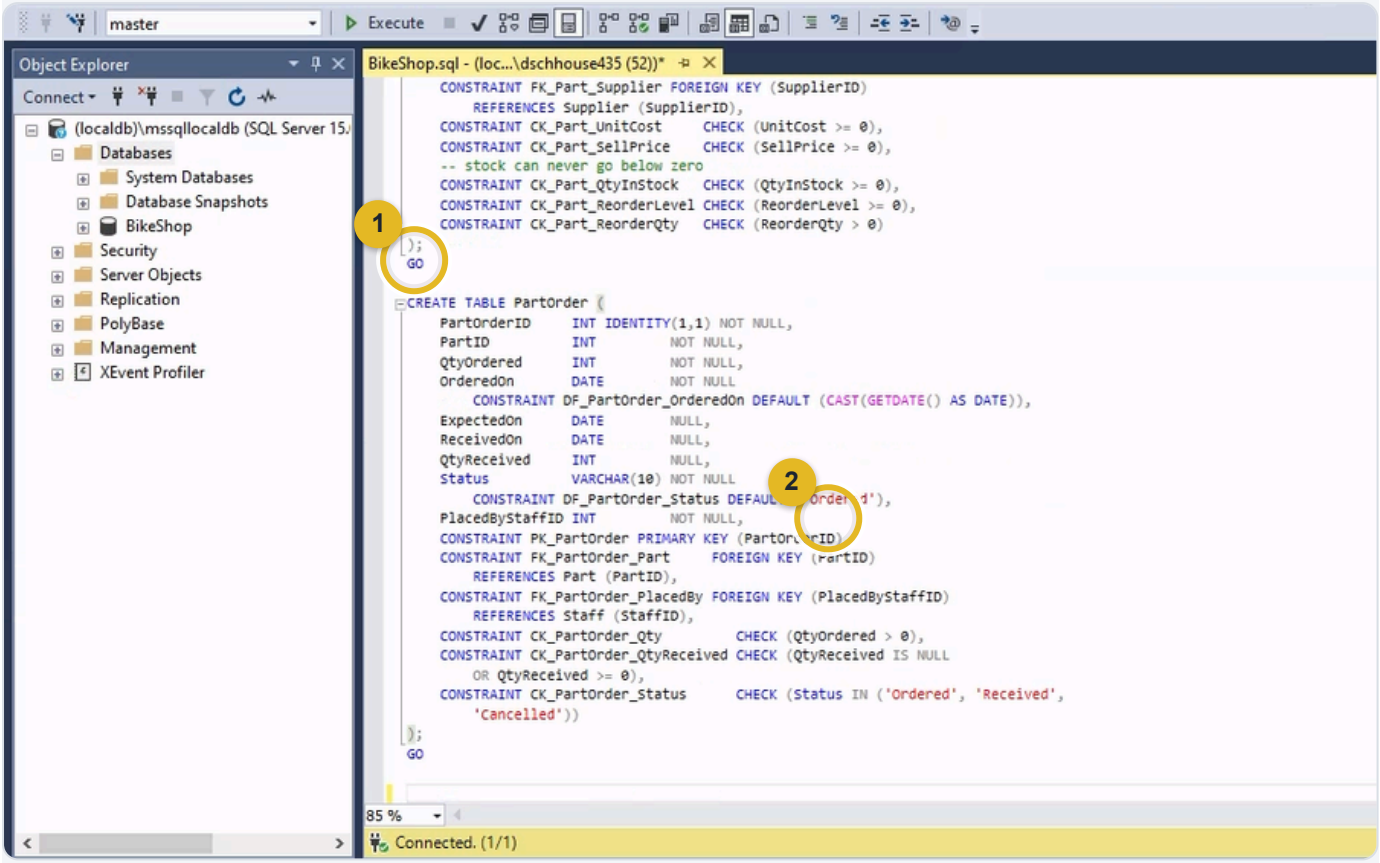
Press the Copy button under this step.

> **You should see** The button reads Copied for a moment.

Click on the empty line under the last GO, at the very end of the script.

> **You should see** The cursor blinks on an empty line under the last GO, with nothing after it.

Press Ctrl and V together.




The PartOrder table pasted at the end of the script, under the GO that ends the Part table. The next table goes on the empty line under its own GO.

1. The GO that ends the Part table.

2. The pasted PartOrder table, from CREATE TABLE PartOrder down to GO.

> **You should see** The new table is at the end of your script, from CREATE TABLE down to its own GO.

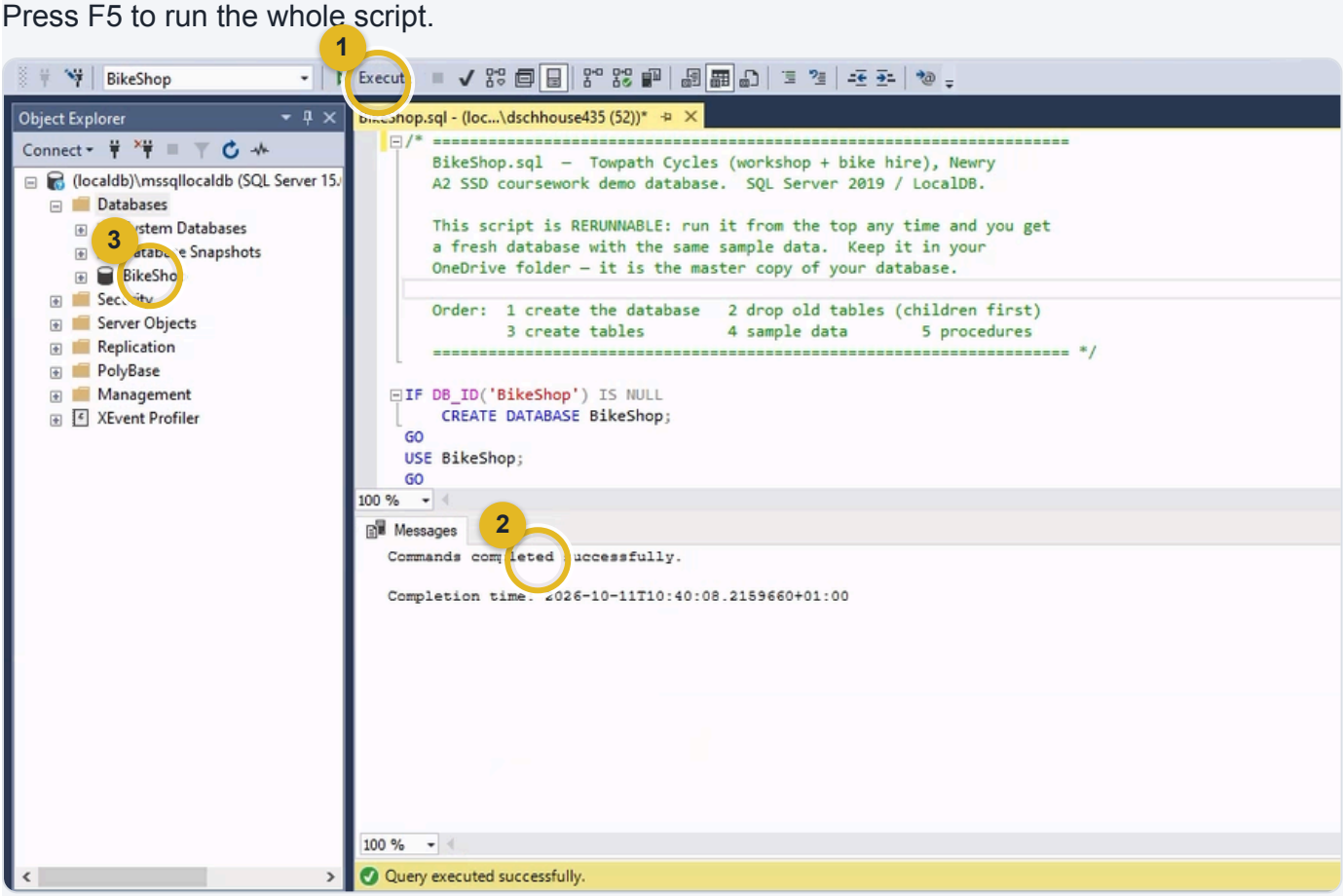
Press Ctrl and S together to save the script.

> **You should see** The star after BikeShop.sql on the tab disappears, because the file is saved.

Click anywhere in the script so that nothing is highlighted.

> **You should see** The cursor blinks in the script, and no line is highlighted in blue.

Press F5 to run the whole script.




Running the whole script with nothing highlighted. Commands completed successfully means every line ran, and the database appears in Object Explorer.

1. Click Execute, or press F5, with nothing highlighted.

2. The Messages pane says: Commands completed successfully.

3. BikeShop appears under Databases.

> **You should see** The Messages pane says: Commands completed successfully. Every table so far was dropped and made again.

**IF IT GOES WRONG** Red text that says 'Incorrect syntax near …'

A bracket, a comma or a quote mark is missing or doubled. The line number in the red text is the line to look at. Compare it with the block above, character by character.

**IF IT GOES WRONG** Red text that says 'There is already an object named … in the database'

The DROP TABLE line for that table is missing from the top of the script, so the old table was still there. Check the ten DROP TABLE lines against the block in stage 1.

**IF IT GOES WRONG** Red text that says 'Could not drop object … because it is referenced by a FOREIGN KEY constraint'

A table is being dropped before a table that points at it. Put the DROP TABLE lines back in the order shown in stage 1: PartSale first, Customer last.

**IF IT GOES WRONG** Red text that says 'Foreign key … references invalid table …'

A table is being made before the table it points at. The tables must be in the order of this chapter: Customer, Staff, ServiceType, Appointment, Bike, Hire, Supplier, Part, PartOrder, PartSale.

STAGE 4 OF 6 · SUPPLIER AND PART, THEN PARTORDER AND PARTSALE · SCREEN 4 OF 4

## Add the PartSale table, the last of the ten.

PartSale is one row per sale of a part over the counter. Two things are new: a customer column that may be empty, because many people who buy an inner tube are not on the books, and a price column that keeps the price on the day, so that a later price change does not rewrite history.

Copy this one **PartSale: one row per sale over the counter** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
CREATE TABLE PartSale (
    PartSaleID    INT IDENTITY(1,1) NOT NULL,
    PartID        INT          NOT NULL,
    CustomerID    INT          NULL,       -- NULL = walk-in customer
    Quantity      INT          NOT NULL,
    -- the price on the day: history keeps it
    UnitPrice     DECIMAL(7,2) NOT NULL,
    SoldOn        DATETIME     NOT NULL
        CONSTRAINT DF_PartSale_SoldOn DEFAULT (GETDATE()),
    SoldByStaffID INT          NOT NULL,
    Status        VARCHAR(10)  NOT NULL
        CONSTRAINT DF_PartSale_Status DEFAULT ('Sold'),
    CONSTRAINT PK_PartSale PRIMARY KEY (PartSaleID),
    CONSTRAINT FK_PartSale_Part     FOREIGN KEY (PartID)
        REFERENCES Part (PartID),
    CONSTRAINT FK_PartSale_Customer FOREIGN KEY (CustomerID)
        REFERENCES Customer (CustomerID),
    CONSTRAINT FK_PartSale_SoldBy   FOREIGN KEY (SoldByStaffID)
        REFERENCES Staff (StaffID),
    CONSTRAINT CK_PartSale_Quantity CHECK (Quantity > 0),
    CONSTRAINT CK_PartSale_Status   CHECK (Status IN ('Sold', 'Refunded'))
);
GO
```

**1 The number and the part**

Lines 1 to 3

PartSaleID is given out by the table. PartID points at the part that was sold.

**2 The customer, who may be a walk-in, and how many**

Lines 4 to 5

CustomerID allows NULL. The comment says why: an empty box means a walk-in customer, someone not on the books. Quantity is how many were sold.

**3 The price on the day, and when**

Lines 7 to 9

UnitPrice is copied from the part's SellPrice at the moment of the sale, so a price rise next month does not change what was paid today. SoldOn is stamped with the date and time by GETDATE().

**4 Who sold it, and the status**

Lines 10 to 12

SoldByStaffID is the number of the member of staff at the counter, and Status starts as Sold.

**5 The key and three foreign keys**

Lines 13 to 19

The part, the customer when there is one, and the staff member must each be a real row in their own table.

**6 The two checks**

Lines 20 to 23

A sale of 0 or fewer parts is refused, and the status must be Sold or Refunded. The closing bracket and the semicolon end the table. GO sends everything above it to the server before the next table starts.

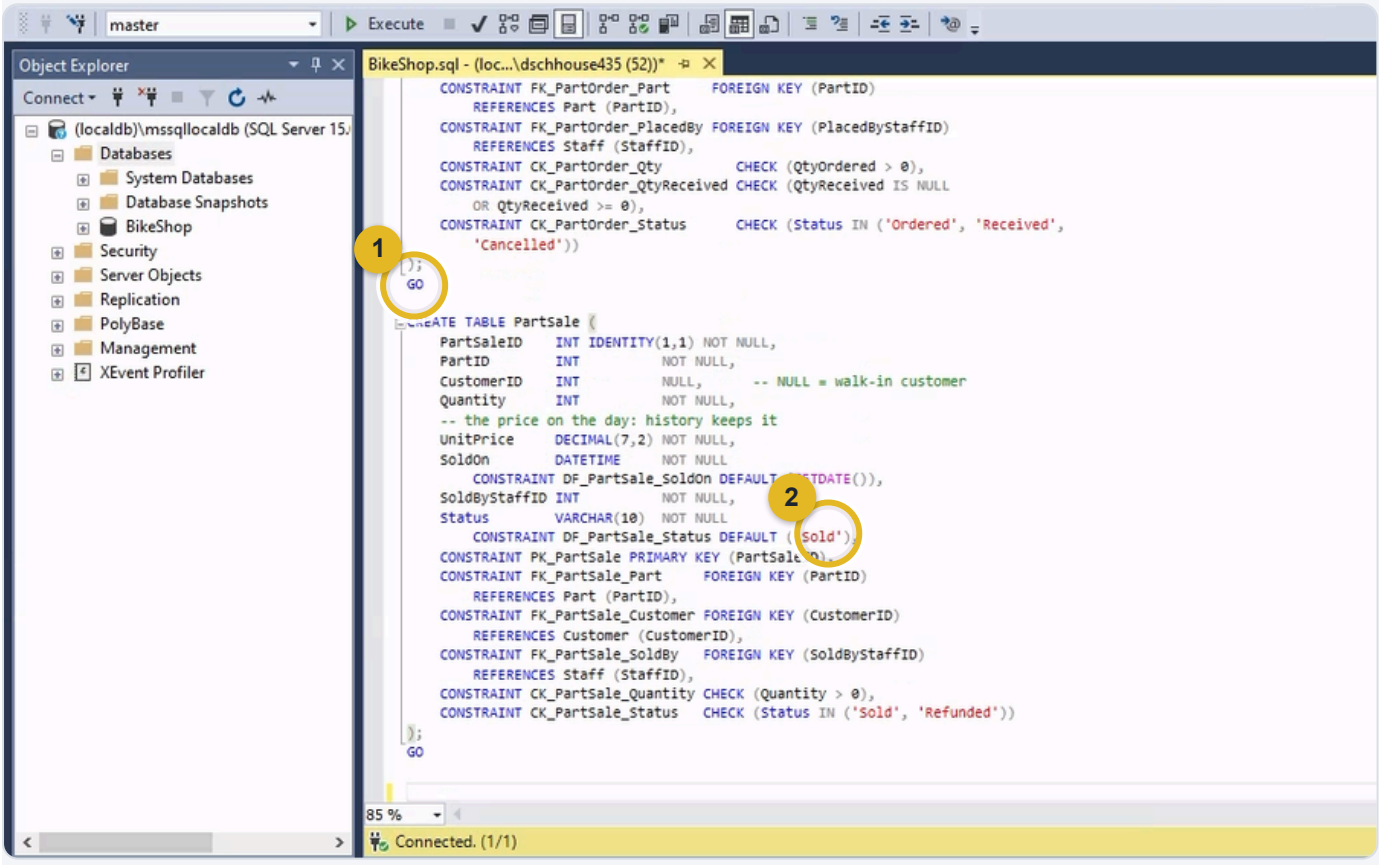
Press the Copy button under this step.

> **You should see** The button reads Copied for a moment.

Click on the empty line under the last GO, at the very end of the script.

> **You should see** The cursor blinks on an empty line under the last GO, with nothing after it.

Press Ctrl and V together.




The PartSale table pasted at the end of the script, under the GO that ends the PartOrder table. The next table goes on the empty line under its own GO.

1. The GO that ends the PartOrder table.

2. The pasted PartSale table, from CREATE TABLE PartSale down to GO.

> **You should see** The new table is at the end of your script, from CREATE TABLE down to its own GO.

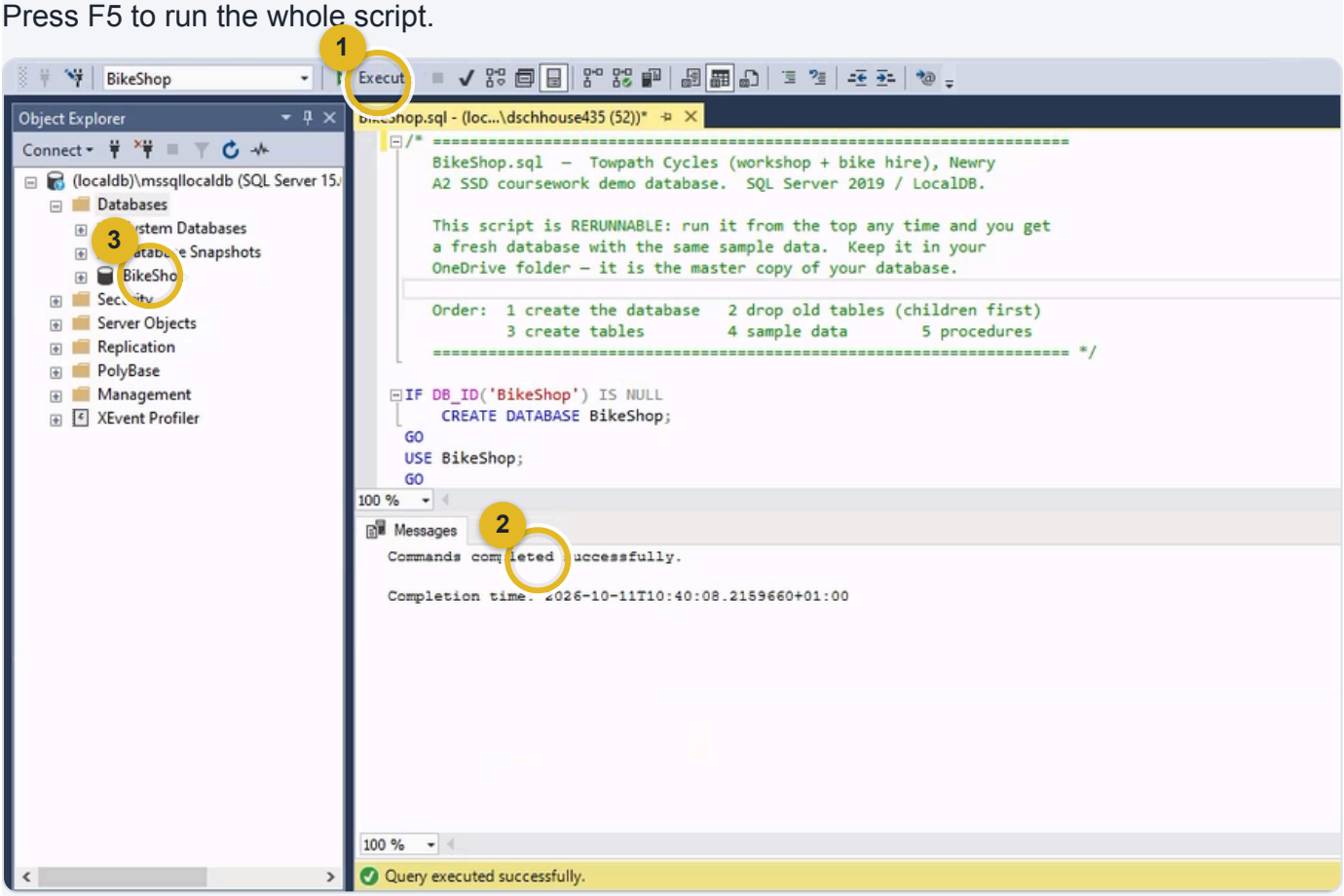
Press Ctrl and S together to save the script.

> **You should see** The star after BikeShop.sql on the tab disappears, because the file is saved.

Click anywhere in the script so that nothing is highlighted.

> **You should see** The cursor blinks in the script, and no line is highlighted in blue.

Press F5 to run the whole script.




Running the whole script with nothing highlighted. Commands completed successfully means every line ran, and the database appears in Object Explorer.

1. Click Execute, or press F5, with nothing highlighted.
2. The Messages pane says: Commands completed successfully.
3. BikeShop appears under Databases.

> **You should see** The Messages pane says: Commands completed successfully. All ten tables were dropped and made again.

**IF IT GOES WRONG** Red text that says 'Incorrect syntax near …'

A bracket, a comma or a quote mark is missing or doubled. The line number in the red text is the line to look at. Compare it with the block above, character by character.

**IF IT GOES WRONG** Red text that says 'There is already an object named … in the database'

The DROP TABLE line for that table is missing from the top of the script, so the old table was still there. Check the ten DROP TABLE lines against the block in stage 1.

**IF IT GOES WRONG** Red text that says 'Could not drop object … because it is referenced by a FOREIGN KEY constraint'

A table is being dropped before a table that points at it. Put the DROP TABLE lines back in the order shown in stage 1: PartSale first, Customer last.

**IF IT GOES WRONG** Red text that says 'Foreign key … references invalid table …'

A table is being made before the table it points at. The tables must be in the order of this chapter: Customer, Staff, ServiceType, Appointment, Bike, Hire, Supplier, Part, PartOrder, PartSale.

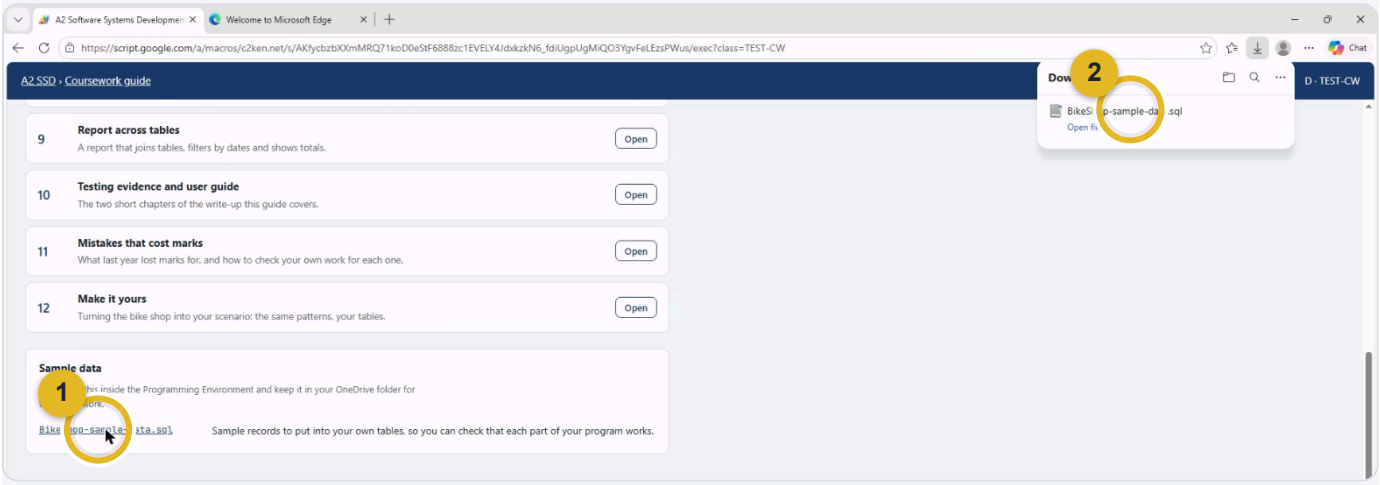
# Download the sample data file.

Ten empty tables are hard to test with. The sample data is a file of INSERT lines that fills every table with made-up rows: twelve customers, six members of staff, seven jobs, three suppliers, twelve parts, twelve bikes, fourteen bookings, six hires, four orders and eight sales. It is a download from the front page of this guide, and its lines go into your script under the last table, so that every run of the script puts the same rows back.

In the Edge browser, click Coursework guide at the top left to go back to the front page.

> **You should see** The list of chapters is showing, and under it is a card called Sample data.

Click BikeShop-sample-data.sql in the Sample data card.




The Sample data card on the front page of this guide. One click saves BikeShop-sample-data.sql into your Downloads folder.

1. Click BikeShop-sample-data.sql in the Sample data card.
2. Edge shows the file in its Downloads pop-up.

> **You should see** Edge shows the file in its Downloads pop-up at the top right. It is now in your Downloads folder.

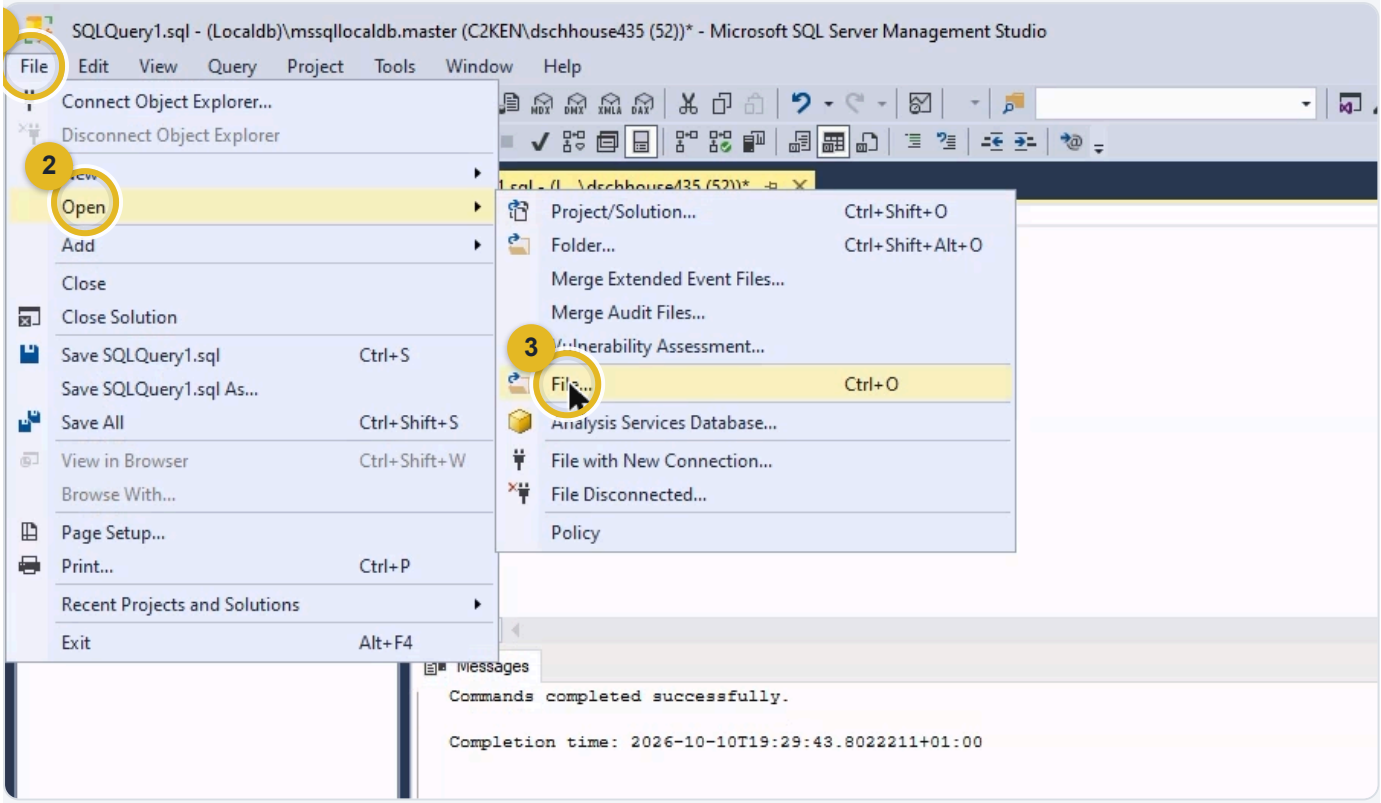
**IF IT GOES WRONG** Edge asks whether to keep the file

Click Keep. The file is a text file of SQL lines from this guide, and nothing in it runs by itself.

In SQL Server Management Studio (SSMS), click File at the top left.

> **You should see** The File menu opens.

Point at Open in the File menu.




The File menu in SQL Server Management Studio (SSMS). Files open from here.

1. Click File at the top left.

2. Point at Open.

3. File… opens the box where you choose the file.

> **You should see** A small menu opens beside Open, and File… is in it.

Click File… in the small menu.

> **You should see** The Open File box opens.

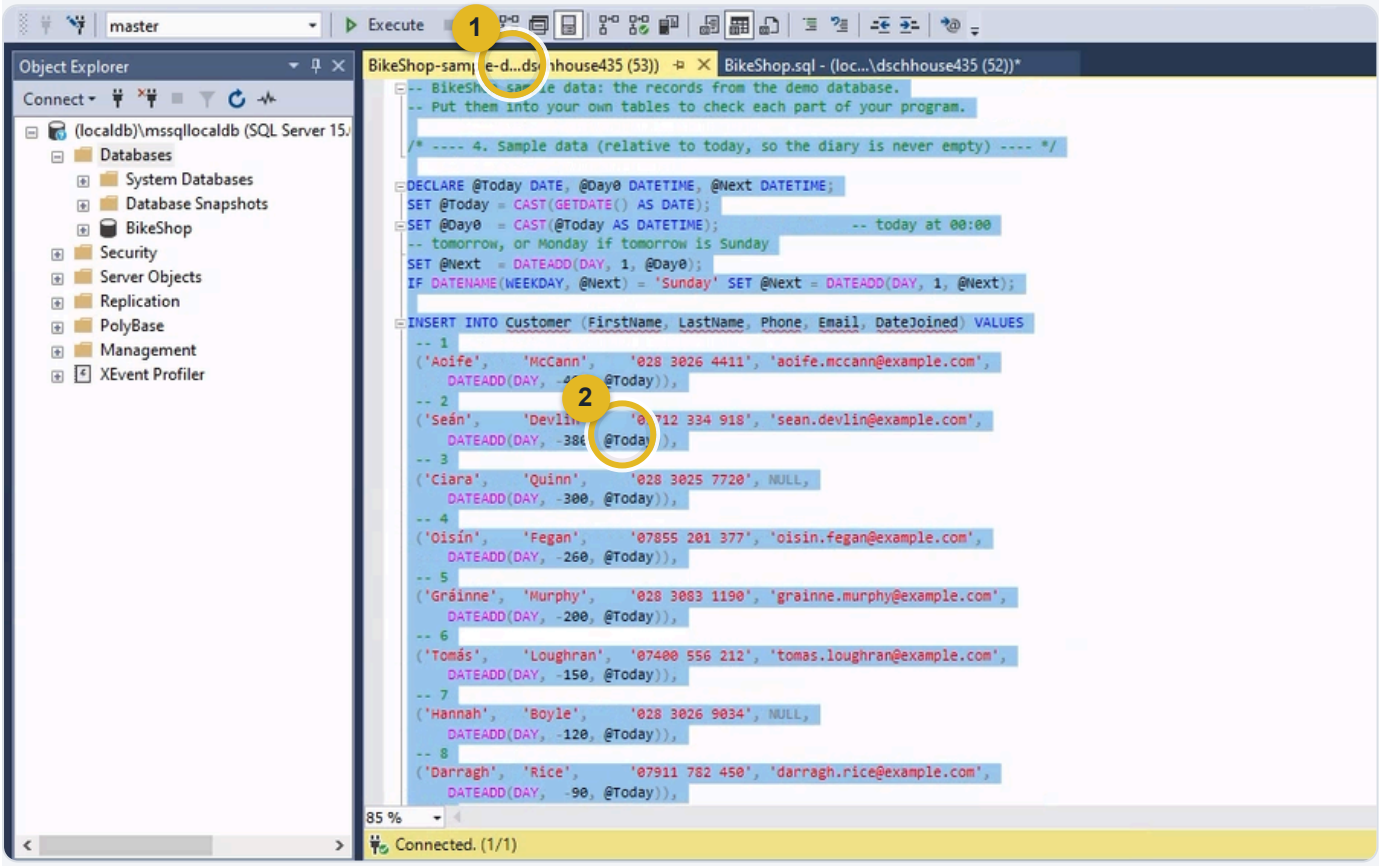
In the Open File box, double-click BikeShop-sample-data.sql in your Downloads folder.

> **You should see** A second tab opens, named BikeShop-sample-data.sql. Its first line is a comment that says BikeShop sample data, and INSERT INTO Customer is a few lines down.

# Copy every line of the sample data into your script, under the last table.

The sample data goes into BikeShop.sql in one piece, under the GO that ends the PartSale table. Copy the whole file, paste it, and the second tab can be closed. It does not matter that the pasted part is long: it is the same INSERT shape over and over, and the next screen reads the first one.

Press Ctrl and A together in the BikeShop-sample-data.sql tab.




The sample data file open in its own tab, with every line highlighted. It stays as it is: its lines are about to be copied into BikeShop.sql.

1. The second tab, BikeShop-sample-data.sql.

2. Every line highlighted, from the first comment line to the GO at the end.

> **You should see** Every line in the tab is highlighted, from the first comment line down to the GO at the end.

Press Ctrl and C together.

> **You should see** Nothing changes on the screen, and the lines are copied.

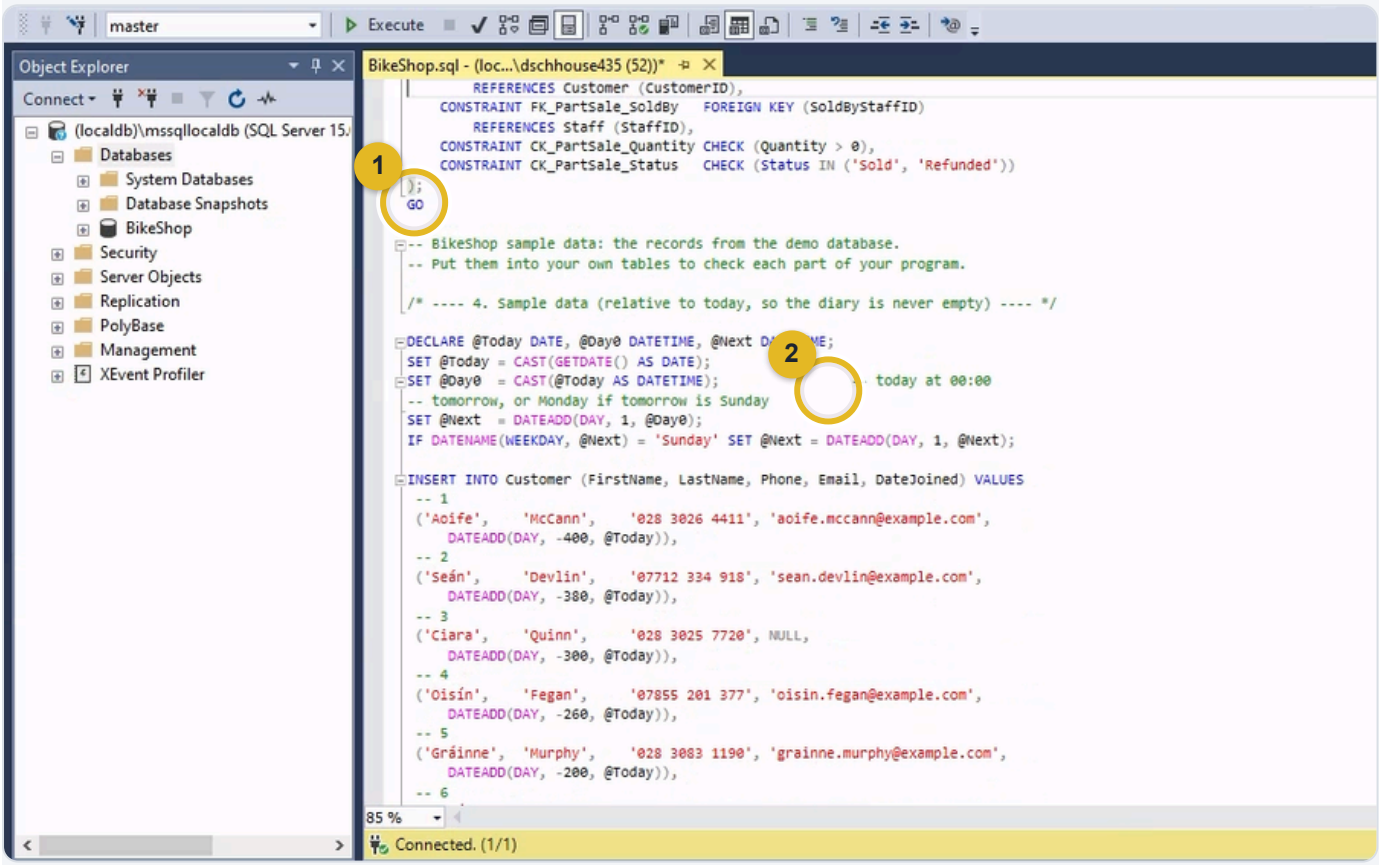
Click the BikeShop.sql tab at the top of the window.

> **You should see** The BikeShop.sql tab is in front, with your ten tables in it.

Click on the empty line under the GO that ends the PartSale table, at the very end of the script.

> **You should see** The cursor blinks on the empty line under the last GO.

Press Ctrl and V together.




The sample data pasted under the last table. The ten CREATE TABLE blocks are above it, and its GO is now the last line of the script.

1. The GO that ends the PartSale table.

2. The pasted sample data, starting with its comment line and DECLARE @Today.

> **You should see** The sample data is in your script, starting with its comment line and DECLARE @Today, and ending with GO. That GO is now the last line of the script.

Press Ctrl and S together to save the script.

> **You should see** The star after BikeShop.sql on the tab disappears, because the file is saved.

Click the small cross on the BikeShop-sample-data.sql tab to close it.

> **You should see** Only the BikeShop.sql tab is left. The downloaded file is still in your Downloads folder, and you will not need it again.

# Read the Customer INSERT, and what @Today does.

Every table is filled the same way: INSERT INTO, the table name, the list of columns in brackets, VALUES, then one row per bracket. Read the Customer one. The one thing that is not a plain value is @Today. A name beginning with @ is a variable: a named box that holds one value while the script runs. The lines above the INSERT in your script declare @Today and set it to today's date with CAST(GETDATE() AS DATE). Every DateJoined is counted back from that box, so the sample customers always joined between 400 and 5 days ago, whatever day you run the script. Two more boxes, @Day0 and @Next, do the same for the bookings, so that the diary always has bookings tomorrow.

Read this one **INSERT INTO Customer: the twelve sample customers** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
INSERT INTO Customer (FirstName, LastName, Phone, Email, DateJoined) VALUES
 -- 1
 ('Aoife',    'McCann',    '028 3026 4411', 'aoife.mccann@example.com',
     DATEADD(DAY, -400, @Today)),
 -- 2
 ('Seán',     'Devlin',    '07712 334 918', 'sean.devlin@example.com',
     DATEADD(DAY, -380, @Today)),
 -- 3
 ('Ciara',    'Quinn',     '028 3025 7720', NULL,
     DATEADD(DAY, -300, @Today)),
 -- 4
 ('Oisín',    'Fegan',     '07855 201 377', 'oisin.fegan@example.com',
     DATEADD(DAY, -260, @Today)),
 -- 5
 ('Gráinne',  'Murphy',    '028 3083 1190', 'grainne.murphy@example.com',
     DATEADD(DAY, -200, @Today)),
 -- 6
 ('Tomás',    'Loughran',  '07400 556 212', 'tomas.loughran@example.com',
     DATEADD(DAY, -150, @Today)),
 -- 7
 ('Hannah',   'Boyle',     '028 3026 9034', NULL,
     DATEADD(DAY, -120, @Today)),
 -- 8
 ('Darragh',  'Rice',      '07911 782 450', 'darragh.rice@example.com',
     DATEADD(DAY,  -90, @Today)),
 -- 9
 ('Méabh',    'Cunningham','028 3083 2278', 'meabh.c@example.com',
     DATEADD(DAY,  -60, @Today)),
 -- 10
 ('Jack',     'O''Hare',   '07525 118 906', 'jack.ohare@example.com',
     DATEADD(DAY,  -40, @Today)),
 -- 11
 ('Róisín',   'Campbell',  '028 3026 1552',
     'roisin.campbell@example.com',DATEADD(DAY,  -21, @Today)),
 -- 12
 ('Ben',      'Toner',     '07388 640 271', 'ben.toner@example.com',
     DATEADD(DAY,   -5, @Today));
```

**1 Which table, and which columns**

Line 1

The five columns named in the brackets are the ones given a value, in that order. CustomerID is not in the list, because the table gives it out itself. VALUES says the rows follow.

**2 The first customer**

Lines 3 to 4

One row is one bracket with five values in the column order: Aoife, McCann, a phone number in quotes because it is text, an email address, and a date. DATEADD(DAY, -400, @Today) means: take the date in the @Today box and go back 400 days. The comment line with the number 1 is a note for a person, so that you can count the rows.

**3** **Eleven more**

Lines 6 to 37

Each row ends with a comma, and the last one ends with a semicolon, which ends the whole INSERT. Ciara Quinn and Hannah Boyle have NULL where the email address goes: an empty box, which the Email column allows. Jack O'Hare is written with two apostrophes, because one apostrophe would end the text early. Names with a fada, such as Seán and Gráinne, are stored as they are.

You can change the names to suit your own scenario, as long as every row keeps five values in the same order, and every phone number keeps at least ten characters, or the CHECK in Customer will refuse it.

# Type the last line of the script.

The script ends with one line that prints the words BikeShop is ready, and a GO. Those two lines are typed, not copied, because from now on they are the landmark of the script: every stored procedure in chapters 4 to 9 goes on the empty line above the PRINT line, so that the PRINT line is always last and the last thing the Messages pane says is always BikeShop is ready.

**Type this one** **The last two lines of BikeShop.sql** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  PRINT 'BikeShop is ready.';
2  GO
```

**1 The words that mean the script got to the end**

Lines 1 to 2

PRINT writes a line in the Messages pane. The words are in single quotes because they are text, and the semicolon ends the statement. GO on its own line sends it. Nothing goes after this GO, ever.

**Why you type it** Two lines, typed so that the semicolon after the quote, the apostrophes round the words and the GO on its own line are in your fingers. Every chapter after this one finds its place by this line.

Click at the end of the last GO, at the very end of the script.

**You should see** The cursor blinks after the O of the last GO.

Press the Enter key twice.

**You should see** The cursor blinks on a new empty line, with one empty line between it and the GO that ends the sample data.

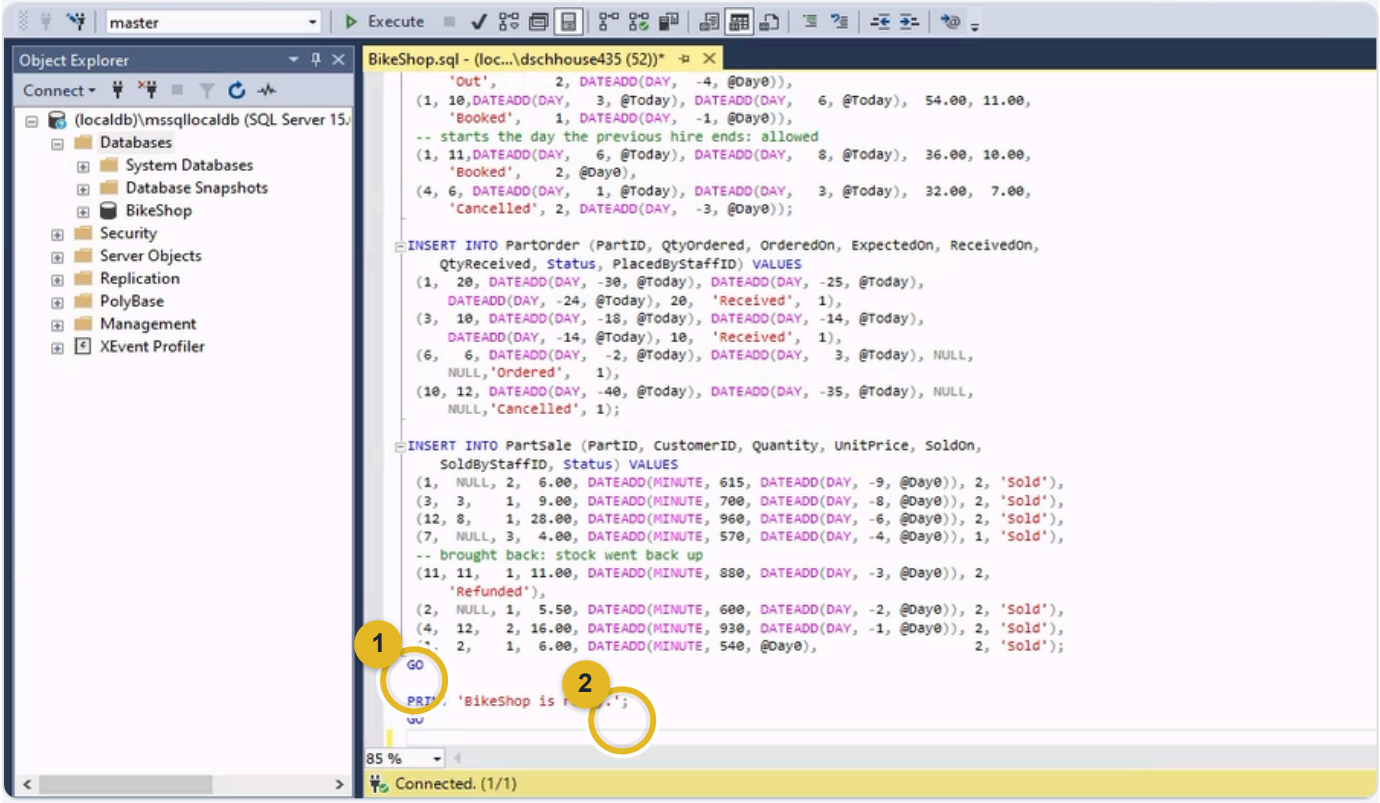
Type PRINT 'BikeShop is ready.'; on that line, with the single quotes and the semicolon, and press Enter.

**You should see** The PRINT line is typed, and the cursor is on the line under it.

**IF IT GOES WRONG** A red wavy line under a word

A word is misspelled. Compare your line with the PRINT line in the code block above, letter by letter, and check that it ends with a single quote and a semicolon.

Type GO and press Enter.




The end of the script. From now on, every new line of SQL goes on the empty line above the line that prints BikeShop is ready, so that line stays last.

1. The empty line above the PRINT line, where every later chapter adds its lines.

2. The two typed lines: PRINT BikeShop is ready, then GO.

> **You should see** GO is the last line of the script, with an empty line above the PRINT line.

Press Ctrl and S together to save the script.

> **You should see** The star after BikeShop.sql on the tab disappears, because the file is saved.

STAGE 6 OF 6 · PROVE IT: RUN IT, LOOK AT IT, AND TICK THE TABLE · SCREEN 1 OF 5

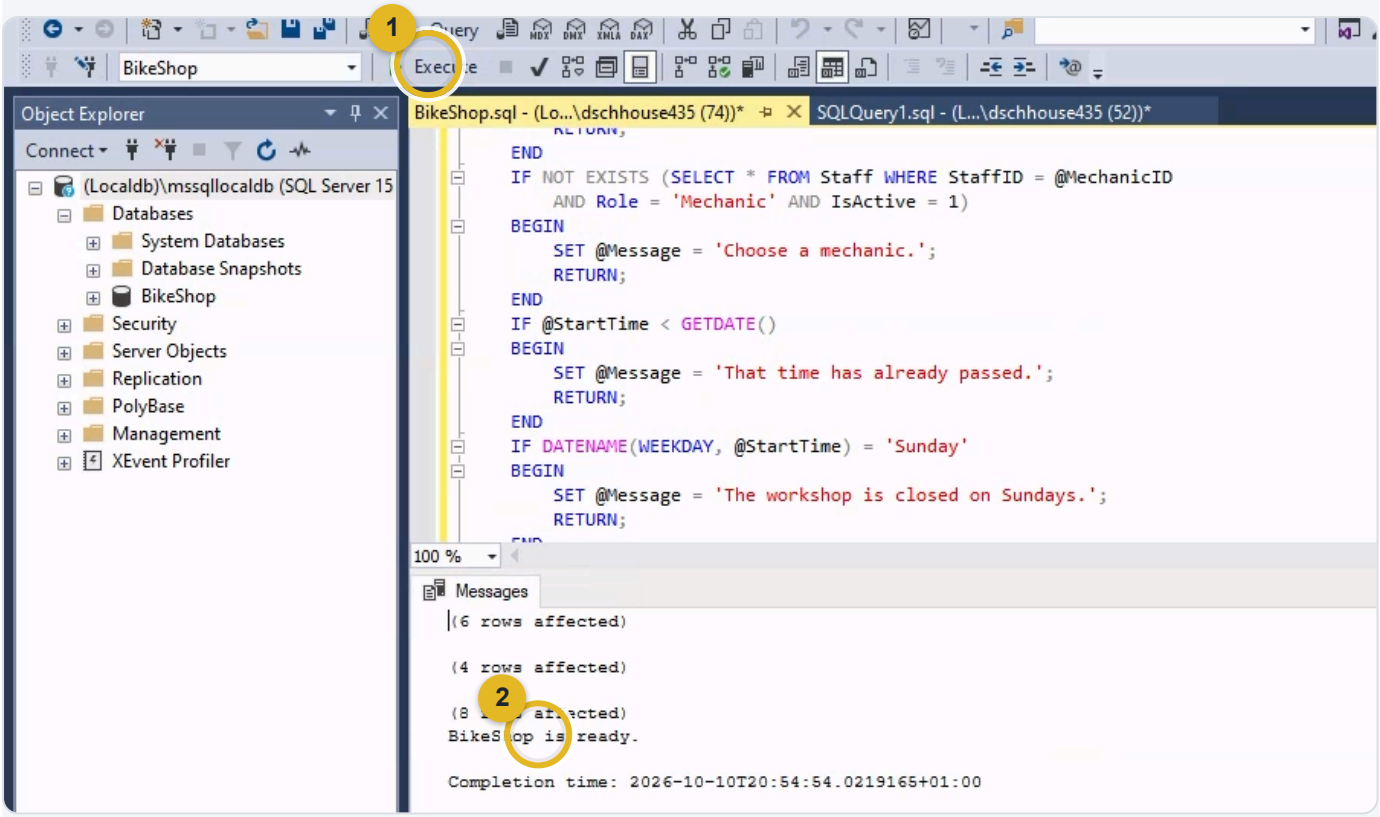
# Run the whole script and see the sample rows go in.

The script is complete. Run it with nothing highlighted and read the Messages pane from the top: one line per INSERT counting the rows that went in, then BikeShop is ready. Run it a second time and the counts are the same, because the tables were dropped and made again before the rows went in. That second run is the proof that the script can be run again.

Click anywhere in the script so that nothing is highlighted.

> **You should see** The cursor blinks in the script, and no line is highlighted in blue.

Press F5 to run the whole script.




Execute with nothing highlighted runs the whole script. The Messages pane counts the sample rows it put in, one line per table, then says BikeShop is ready.

1. Click Execute, or press F5.

2. The last lines say BikeShop is ready, then a Completion time. Red text here is an error to read.

> **You should see** The Messages pane shows a line for each table, such as (12 rows affected) and (6 rows affected), then BikeShop is ready, then a Completion time.

**IF IT GOES WRONG** Red text that says 'Incorrect syntax near …'

A bracket, a comma or a quote mark is missing or doubled. The line number in the red text is the line to look at. Compare that line with the same line in the code block on the screen for its table, or in the sample data download, character by character.

**IF IT GOES WRONG** Red text that says 'There is already an object named … in the database'

The DROP TABLE line for that table is missing from the top of the script, so the old table was still there. Check the ten DROP TABLE lines against the block in stage 1.

**IF IT GOES WRONG** Red text that says 'Could not drop object … because it is referenced by a FOREIGN KEY constraint'

A table is being dropped before a table that points at it. Put the DROP TABLE lines back in the order shown in stage 1: PartSale first, Customer last.

**IF IT GOES WRONG** Red text that says 'Foreign key … references invalid table …'

A table is being made before the table it points at. The tables must be in the order of this chapter: Customer, Staff, ServiceType, Appointment, Bike, Hire, Supplier, Part, PartOrder, PartSale.

**IF IT GOES WRONG** Red text that says 'The INSERT statement conflicted with the FOREIGN KEY constraint …'

A table is being filled before the table it points at. The INSERT lines must stay in the order of the download: Customer, Staff, ServiceType, Supplier, Part, Bike, Appointment, Hire, PartOrder, PartSale.

**IF IT GOES WRONG** Red text that says 'Must declare the scalar variable @Today'

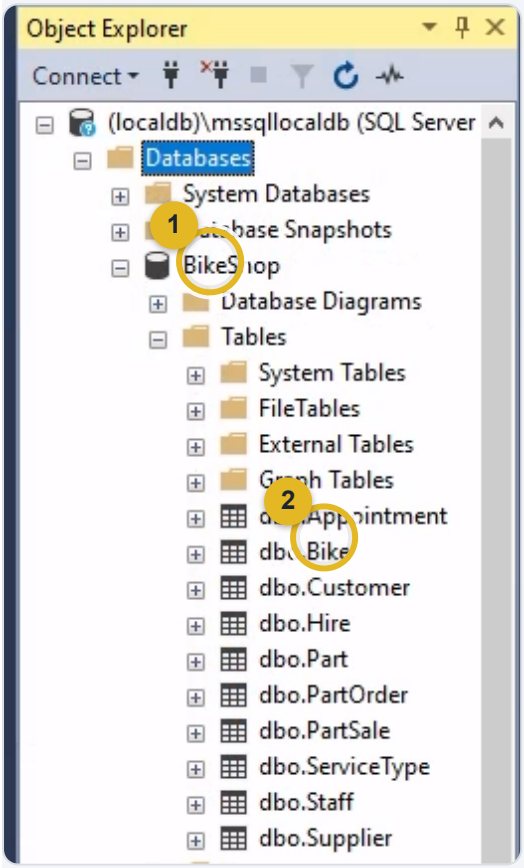
The DECLARE line and the SET lines above the Customer INSERT are missing, or a GO has got between them and the INSERTs. The sample data runs as one piece, from DECLARE down to its GO.

Press F5 again, with nothing highlighted.

**You should see** The same lines appear again, ending with BikeShop is ready. Nothing is doubled, because the old tables were dropped first.

# See the ten tables in Object Explorer.

Object Explorer, on the left, lists what the server holds. It does not look again by itself, so after a script makes tables you refresh it. The ten tables appear in alphabetical order, each with dbo. in front, which the server adds to every table and which you never type.




Object Explorer, on the left of SQL Server Management Studio (SSMS). Right-click Tables and click Refresh to see the tables your script has made. The dbo. in front of each name is added by the server, and you never type it.

1. Open Databases, then BikeShop.
2. Tables: the ten tables, in alphabetical order.

In Object Explorer, click the plus sign beside Databases.

> **You should see** Databases opens out, and BikeShop is in the list under it.

Click the plus sign beside BikeShop.

> **You should see** BikeShop opens out, and one of the folders under it is called Tables.

Right-click the Tables folder and click Refresh.

> **You should see** The Tables folder is read again from the server.

Click the plus sign beside the Tables folder.

> **You should see** Ten tables are listed: Appointment, Bike, Customer, Hire, Part, PartOrder, PartSale, ServiceType, Staff and Supplier.

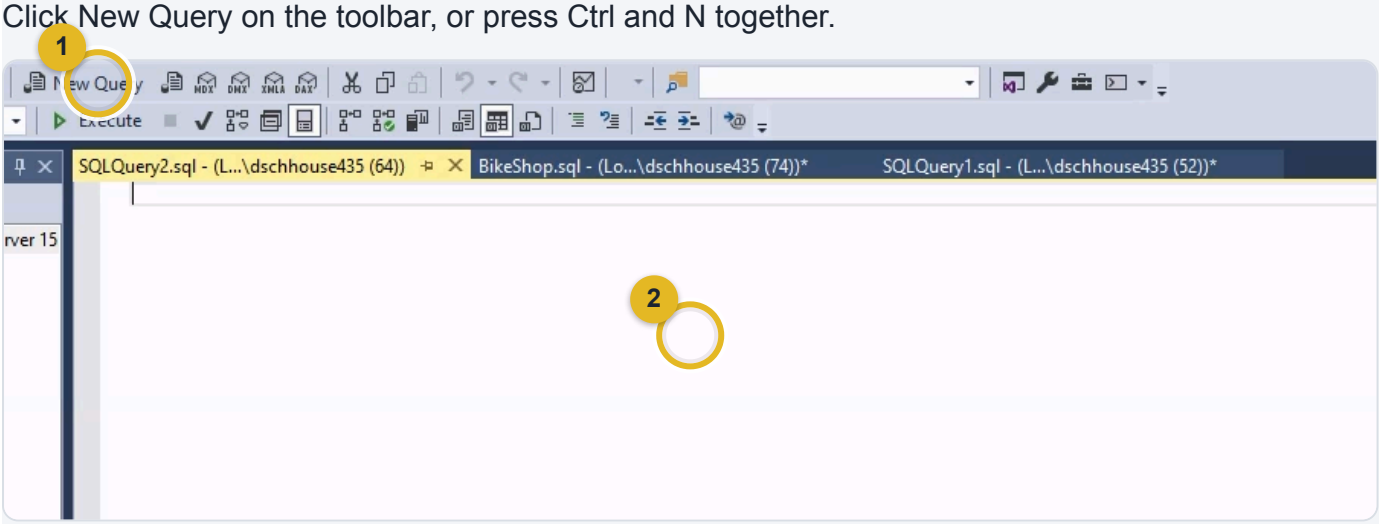
**IF IT GOES WRONG** Fewer than ten tables

One CREATE TABLE block is missing from the script, or the script stopped at a red error before it reached that table. Scroll the Messages pane up to the first red line and fix that.

# Ask the Customer table for its rows.

A question to the database is written in a New Query window, not in the script, so that the script stays the clean master copy. SELECT * FROM Customer; means: show me every column of every row of Customer. The star means every column. The twelve names that come back are the first proof that the data is in.

Click New Query on the toolbar, or press Ctrl and N together.




New Query opens a second, empty window. Questions you ask the database go in here, so your script stays clean.

1. Click New Query, or press Ctrl and N together.

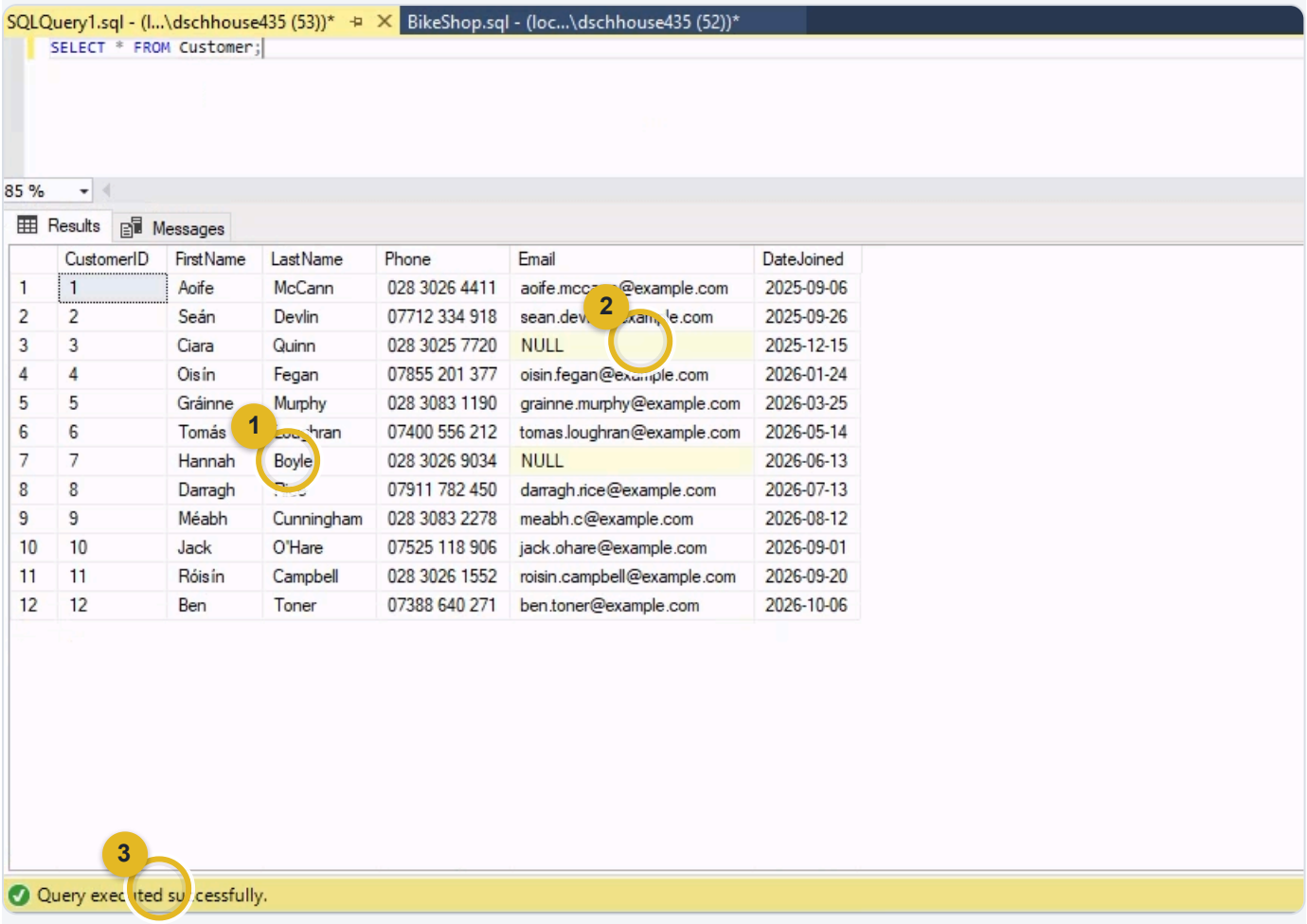
2. Type the question into the empty window.

> **You should see** A new tab opens with an empty query window, and the cursor blinks in it.

Type SELECT * FROM Customer; in the new window.

> **You should see** The line is typed, with the star between SELECT and FROM, and a semicolon at the end.

Press F5 to run the question.



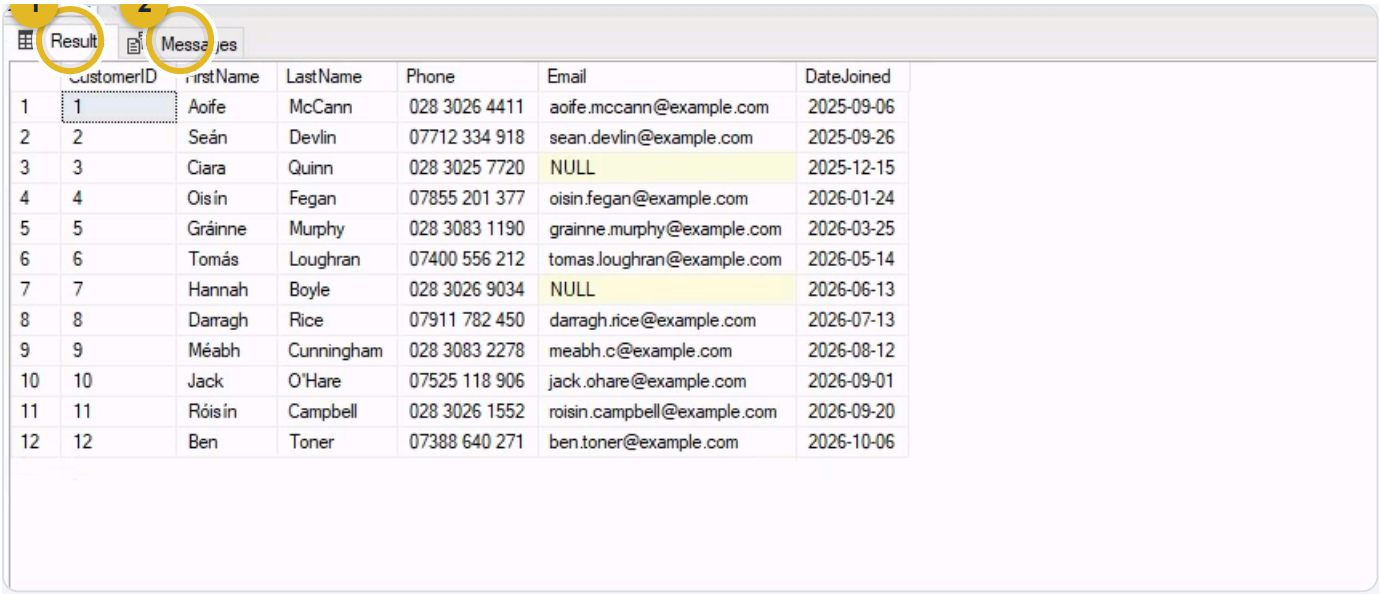

Twelve customers, numbered 1 to 12 by the table itself. NULL in the Email column is an empty box, which the Email column allows.

1. Twelve rows, Aoife McCann first and Ben Toner last.
2. NULL means the box is empty.
3. Query executed successfully: the question worked.

> **You should see** A Results tab appears under the window with twelve rows, numbered 1 to 12: Aoife McCann first and Ben Toner last. Two Email boxes say NULL. The bottom bar says 12 rows.

**IF IT GOES WRONG** Red text that says 'Invalid object name Customer'

The new window is looking at the wrong database. In the box on the toolbar that says master, choose BikeShop, and press F5 again.




| | CustomerID | FirstName | LastName | Phone | Email | DateJoined |
|---|---|---|---|---|---|---|
| 1 | 1 | Aoife | McCann | 028 3026 4411 | aoife.mccann@example.com | 2025-09-06 |
| 2 | 2 | Seán | Devlin | 07712 334 918 | sean.devlin@example.com | 2025-09-26 |
| 3 | 3 | Ciara | Quinn | 028 3025 7720 | NULL | 2025-12-15 |
| 4 | 4 | Oisín | Fegan | 07855 201 377 | oisin.fegan@example.com | 2026-01-24 |
| 5 | 5 | Gráinne | Murphy | 028 3083 1190 | grainne.murphy@example.com | 2026-03-25 |
| 6 | 6 | Tomás | Loughran | 07400 556 212 | tomas.loughran@example.com | 2026-05-14 |
| 7 | 7 | Hannah | Boyle | 028 3026 9034 | NULL | 2026-06-13 |
| 8 | 8 | Darragh | Rice | 07911 782 450 | darragh.rice@example.com | 2026-07-13 |
| 9 | 9 | Méabh | Cunningham | 028 3083 2278 | meabh.c@example.com | 2026-08-12 |
| 10 | 10 | Jack | O'Hare | 07525 118 906 | jack.ohare@example.com | 2026-09-01 |
| 11 | 11 | Róisín | Campbell | 028 3026 1552 | roisin.campbell@example.com | 2026-09-20 |
| 12 | 12 | Ben | Toner | 07388 640 271 | ben.toner@example.com | 2026-10-06 |

Results shows the rows a question returns. Messages shows what it printed and any error, in red.

1. Results: the rows, as a grid.
2. Messages: printed lines and errors, in red.

STAGE 6 OF 6 · PROVE IT: RUN IT, LOOK AT IT, AND TICK THE TABLE · SCREEN 4 OF 5

# Tick the table: the tables, the rows, and two rows the tables must refuse.

Every row below is a test the examiner would run, and the last two are the ones that matter most. A table with a CHECK rule refuses a bad row by itself, with red text that names the rule, before any program exists. You type each refused INSERT into your New Query window, over the SELECT, and the red text is the evidence. Nothing is added, and SELECT * FROM Customer; afterwards still shows twelve rows.

| | THE TEST | WHAT MUST HAPPEN | HOW TO DO IT, AND WHAT YOU SHOULD SEE |
|---|---|---|---|
| | Ten tables listed | **Must be allowed** | Right-click Tables in Object Explorer and click Refresh. You see ten tables: Appointment, Bike, Customer, Hire, Part, PartOrder, PartSale, ServiceType, Staff and Supplier. |
| | Twelve customers | **Must be allowed** | Type SELECT * FROM Customer; in the New Query window and press F5. You see twelve rows, Aoife McCann first and Ben Toner last, and the bottom bar says 12 rows. |
| | The script runs again and the rows are not doubled | **Must be allowed** | Click in the BikeShop.sql tab and press F5 with nothing highlighted. Then click the New Query tab and press F5 to run SELECT * FROM Customer; again. You still see twelve rows, not twenty-four. |
| | A booking with the status Maybe | **Must be refused** | Click in the New Query window and press Ctrl and A to highlight the SELECT line, then type this on one line: INSERT INTO Appointment (CustomerID, MechanicID, ServiceTypeID, StartTime, EndTime, BikeDescription, BookedByStaffID, Status) VALUES (1, 3, 1, '2026-10-12 09:00', '2026-10-12 09:30', 'Test bike', 2, 'Maybe'); Then press F5. You see red text in the Messages tab: The INSERT statement conflicted with the CHECK constraint "CK_Appointment_Status". Under it: The statement has been terminated. Nothing was added. |
| | A customer with a five-digit phone number | **Must be refused** | Click in the New Query window and press Ctrl and A to highlight the INSERT line, then type this on one line: INSERT INTO Customer (FirstName, LastName, Phone) VALUES ('Test', 'Person', '12345'); Then press F5. You see red text: The INSERT statement conflicted with the CHECK constraint "CK_Customer_Phone". Press Ctrl and A, type SELECT * FROM Customer; and press F5. It still shows twelve rows. |

A refusal, in red. The table itself turned the row away, because Maybe is not in the list the CHECK allows. No program was needed.

1. The red text names the rule that refused the row: CK_Appointment_Status.
2. The statement has been terminated: nothing was added.

STAGE 6 OF 6 · PROVE IT: RUN IT, LOOK AT IT, AND TICK THE TABLE · SCREEN 5 OF 5

# Finish the chapter.

Five ticks are your evidence that one script makes the whole database, that it can be run again, and that the rules in the tables refuse bad rows before any program is written. Keep a screenshot of the red refusal and of the twelve customers for your testing evidence in chapter 10. Close the New Query window without saving it: the script is the only file that matters, and it is saved in your OneDrive folder.

**Make it yours**

Your scenario has its own ten tables from chapter 2. Build them in this order: every table that points at nothing first, then the tables that point at those, then the rest, and drop them in the opposite order at the top. Give every rule a name in the same shape, PK, FK, UQ, CK or DF, then the table, then what it is about, so that every red refusal tells you which rule spoke. Write at least one CHECK with a list of allowed words, one that compares two columns, and one that says a count can never go below zero. Then make your own sample data, with at least ten rows in your busiest table, and dates counted back from @Today.

CHAPTER 4 OF 12

# Stored procedures

One procedure per action: the only way your program touches the data.

STAGE 1 OF 7 · WHAT A STORED PROCEDURE IS · SCREEN 1 OF 3

## Read what a stored procedure is, and why every action gets one.

In chapter 3 you typed SELECT, INSERT and DELETE lines into a query window and ran them yourself. Your program cannot do that. A program needs each action it takes on the data to be written down once, given a name, and kept inside the database, so that the program only has to say the name. That is a stored procedure: a recipe with a name, saved in the database. The program says the name and hands over the ingredients, such as a customer's name and phone number, and the database does the rest.

Every action your program will ever take on the data gets its own stored procedure: search for customers, fetch one customer, add one, change one, delete one, and so on. The program never sends a SELECT or an INSERT of its own. That is the rule. The top mark band asks for a program that adds, deletes, changes and displays related data from more than one table, and marking last year's work showed that doing every one of those through a stored procedure is what the examiner expects to see.

There is one more reason. A stored procedure can check before it acts. If the shop tries to delete a customer who has bookings, the stored procedure says no, in a sentence, and the program shows that sentence. The rule lives in the database, so no screen can forget it. By the end of this chapter you will have fifteen stored procedures in your script, and the first refusal.

**Words used in this chapter**

| | |
|---|---|
| `Stored procedure` | A recipe with a name, saved inside the database, that the program runs by saying the name. |
| `Parameter` | One ingredient a stored procedure needs, such as a customer's first name, with a name that starts with @. |
| `EXEC` | The word that runs a stored procedure by name, handing over the parameters it needs. |
| `OUTPUT` | A parameter that carries an answer back out of the stored procedure, instead of carrying a value in. |
| `@Message` | The sentence a stored procedure hands back when it refuses to act. When it is blank, the action worked. |
| `RETURN` | Stop here: nothing below this line runs. |
| `SCOPE_IDENTITY()` | The number the database gave to the row that was added a moment ago. |

| | |
|---|---|
| `IF EXISTS` | A test that is true when the SELECT inside the brackets finds at least one row. |
| `LIKE` | A match on part of a piece of text, where % stands for any letters at all. |
| `JOIN` | Lining up the rows of two tables where a number in one matches a number in the other, so names can be shown instead of numbers. |
| `GO` | A word for SQL Server Management Studio (SSMS), not for the database: run everything above this line as one batch, then start a fresh one. |
| `Variable` | A named box that holds one value while a script runs, made with DECLARE and filled with SET. |
| `Batch` | A group of lines that SQL Server Management Studio (SSMS) sends to the database together. |

# Check that you have these ready, and learn how the stored procedures are named.

**Before you start**

- Chapter 3 is finished: your BikeShop.sql builds the ten tables and fills them with the sample data when you press F5.
- SQL Server Management Studio (SSMS) is open, connected to your BikeShop database.
- This guide is open in the Edge browser beside you. Tick each step as you do it, and the chapter remembers where you are.

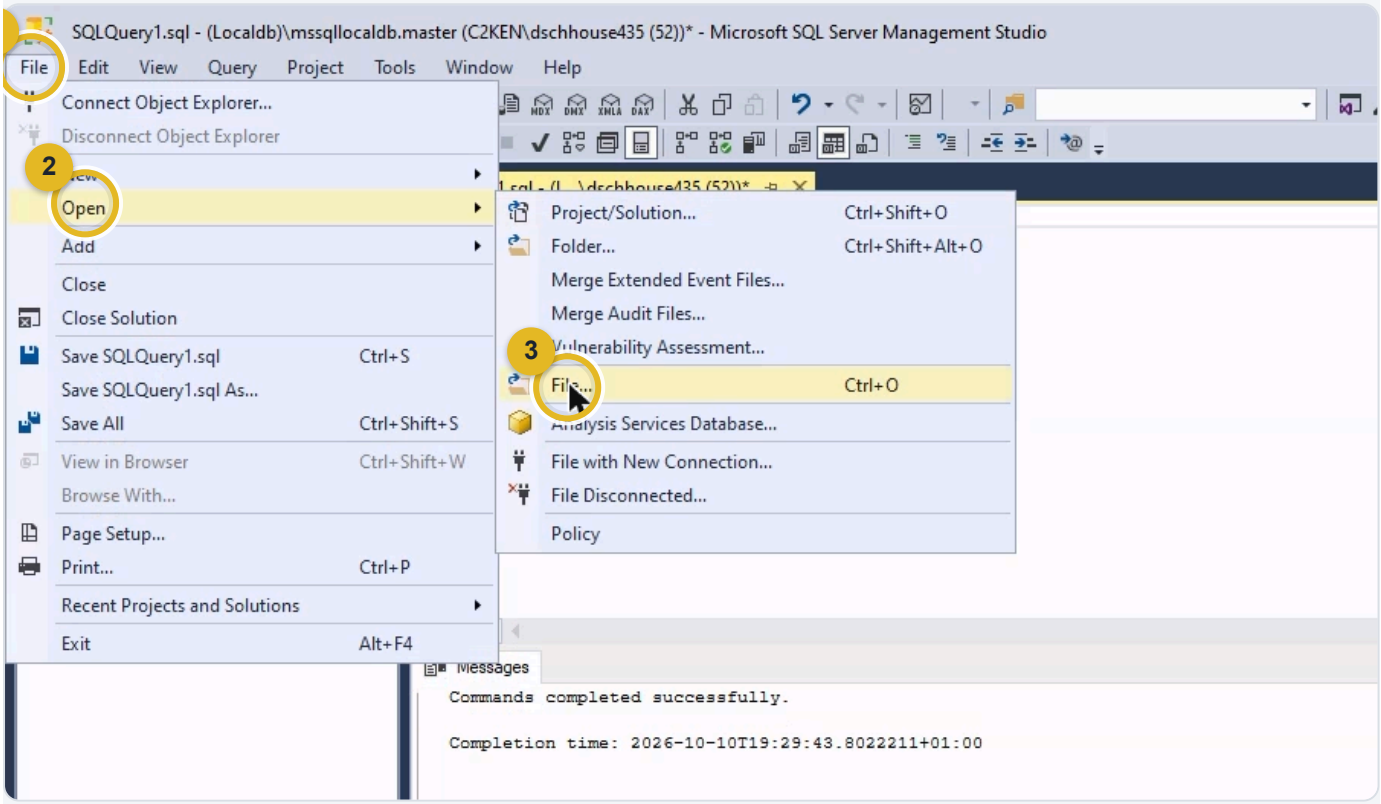
Every stored procedure in this guide is named the same way: usp, an underscore, the table it works on, an underscore, and what it does. usp_Customer_Search searches the Customer table. usp_Customer_Delete deletes from it. The usp at the front stands for user stored procedure, and it keeps your own ones apart from the ones the database comes with. When SQL Server Management Studio (SSMS) lists them, every Customer one sits beside the others, which makes them easy to find.

There are four shapes, and you will meet each one in turn. A Get fetches rows and changes nothing. An Add puts one row in and hands back its new number. An Update changes one row. A Delete takes one row out, and checks first. Each shape is the same from table to table, so once you have written one Add, you can write every Add.

# Open your script, and find the line where every stored procedure goes.

Your stored procedures go into BikeShop.sql, under the sample data and above the line that prints BikeShop is ready. That PRINT line always stays last, so that when the whole script runs, the last thing you see is the message that it worked. The picture shows the exact spot.

Click File at the top left of SQL Server Management Studio (SSMS).




The File menu in SQL Server Management Studio (SSMS). Your script opens from here.

1. Click File at the top left.

2. Point at Open.

3. Click File… and choose BikeShop.sql from your OneDrive folder.

> **You should see** The File menu drops down.

Point at Open in the menu.

> **You should see** A small menu opens beside Open, with File… in it.

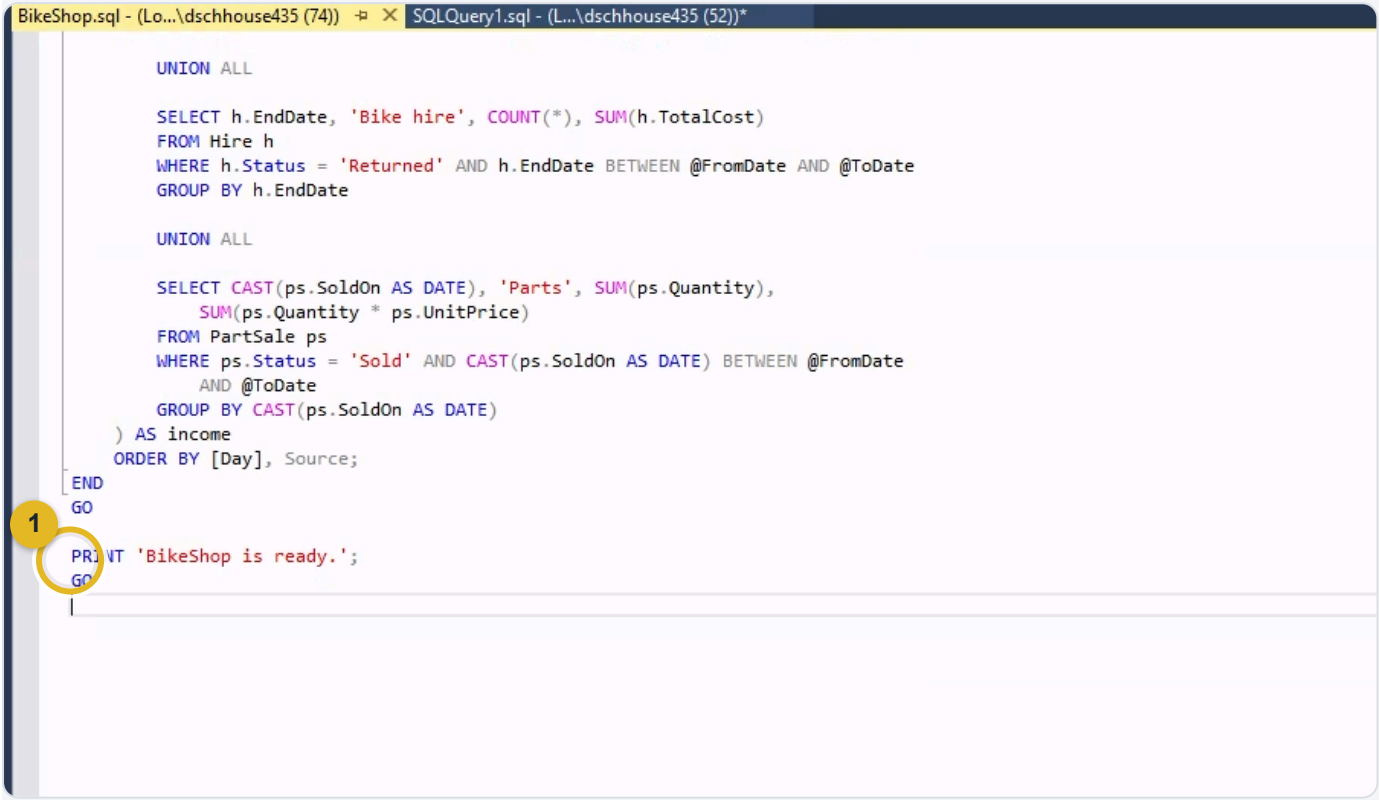
Click File… in the small menu.

> **You should see** A window opens for you to choose a file.

Choose BikeShop.sql from your OneDrive folder and press Open.

> **You should see** Your script is in a tab, and the first lines are the ones you built in chapter 3.

Scroll to the very end of the script and click on the empty line above the line that prints BikeShop is ready. That line always stays last.

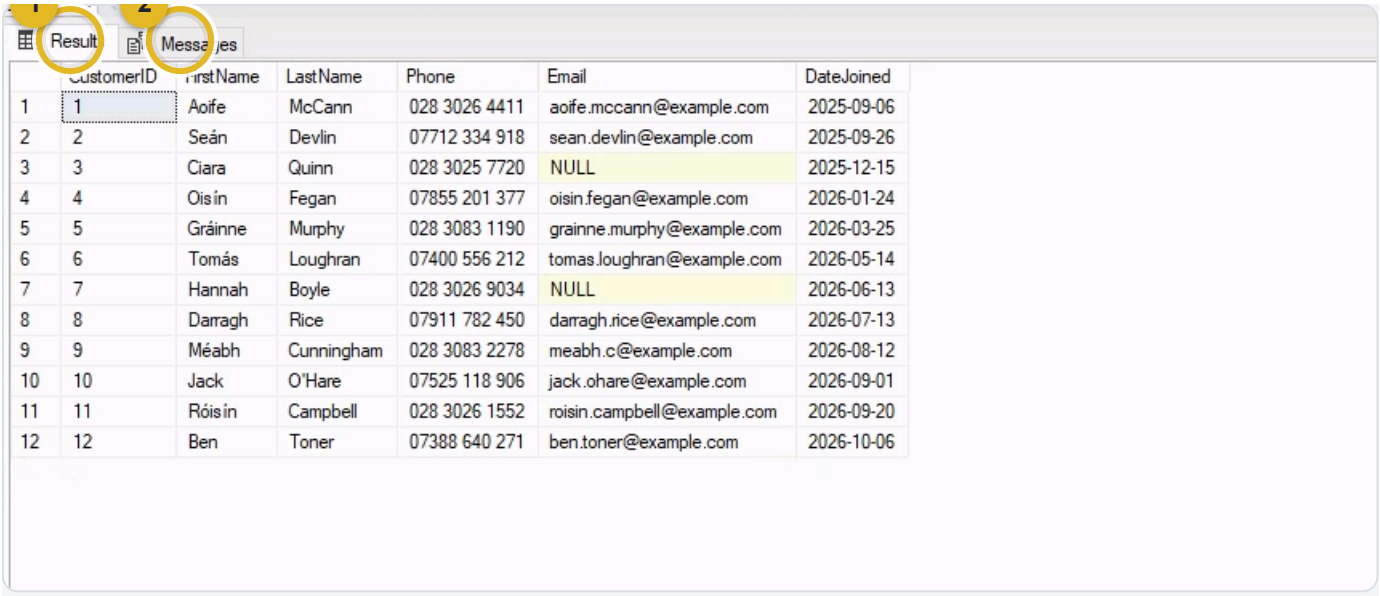



The end of your script. Every stored procedure in this chapter goes on the empty line above the line that prints BikeShop is ready, one after another, each ending with GO.

1. Click on the empty line above the line that prints BikeShop is ready, so the cursor blinks there.

> **You should see** The cursor blinks on the empty line, with the sample data above it.

The Messages pane and the Results pane sit under the query window. Rows that a SELECT finds appear under Results. Lines that PRINT writes, and any error, appear under Messages. You will look at both in this chapter.




| | CustomerID | FirstName | LastName | Phone | Email | DateJoined |
|---|---|---|---|---|---|---|
| 1 | 1 | Aoife | McCann | 028 3026 4411 | aoife.mccann@example.com | 2025-09-06 |
| 2 | 2 | Seán | Devlin | 07712 334 918 | sean.devlin@example.com | 2025-09-26 |
| 3 | 3 | Ciara | Quinn | 028 3025 7720 | NULL | 2025-12-15 |
| 4 | 4 | Oisín | Fegan | 07855 201 377 | oisin.fegan@example.com | 2026-01-24 |
| 5 | 5 | Gráinne | Murphy | 028 3083 1190 | grainne.murphy@example.com | 2026-03-25 |
| 6 | 6 | Tomás | Loughran | 07400 556 212 | tomas.loughran@example.com | 2026-05-14 |
| 7 | 7 | Hannah | Boyle | 028 3026 9034 | NULL | 2026-06-13 |
| 8 | 8 | Darragh | Rice | 07911 782 450 | darragh.rice@example.com | 2026-07-13 |
| 9 | 9 | Méabh | Cunningham | 028 3083 2278 | meabh.c@example.com | 2026-08-12 |
| 10 | 10 | Jack | O'Hare | 07525 118 906 | jack.ohare@example.com | 2026-09-01 |
| 11 | 11 | Róisín | Campbell | 028 3026 1552 | roisin.campbell@example.com | 2026-09-20 |
| 12 | 12 | Ben | Toner | 07388 640 271 | ben.toner@example.com | 2026-10-06 |

Rows that a SELECT finds appear under Results. Lines that PRINT writes, and any error, appear under Messages.

1. Results: the rows a SELECT found.
2. Messages: the lines that PRINT wrote, and any error.

# Copy the customer search into your script, and run the whole script once.

The first stored procedure searches for customers. The program's customers screen in chapter 6 will have a search box, and whatever is typed in it goes into this stored procedure as @SearchText. Any customer whose first name, last name or phone number contains that text comes back, in surname order. Read every part, then paste it in.

Copy this one **usp_Customer_Search: find customers by part of a name or phone number** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
CREATE OR ALTER PROCEDURE usp_Customer_Search
    @SearchText VARCHAR(50)
AS
BEGIN
    SELECT CustomerID,
           FirstName + ' ' + LastName AS FullName,
           Phone, Email, DateJoined
    FROM Customer
    WHERE FirstName LIKE '%' + @SearchText + '%'
       OR LastName  LIKE '%' + @SearchText + '%'
       OR Phone     LIKE '%' + @SearchText + '%'
    ORDER BY LastName, FirstName;
END
GO
```

**1 The name, and what goes in**

Lines 1 to 2

CREATE OR ALTER PROCEDURE makes a stored procedure with this name, or replaces the one already there, so the script can run again and again. The one parameter is @SearchText, up to 50 characters of text. A parameter is an ingredient the stored procedure needs, and its name always starts with @.

**2 AS and BEGIN**

Lines 3 to 4

This stored procedure only reads, so it has no @Message. AS and BEGIN mark where the work starts.

**3 What comes back**

Lines 5 to 7

Each row that comes back has the customer's number, their first and last names joined with a space into one column called FullName, their phone, their email and the day they joined. AS gives the joined column its name.

**4 Which table**

Line 8

The rows come from the Customer table, and from nowhere else.

**5 The search**

Lines 9 to 11

LIKE matches part of a piece of text. The % signs stand for any letters at all, so '%' + @SearchText + '%' means the search text anywhere inside. There are three tests joined by OR, so a customer comes back when the text is in their first name, or in their last name, or in their phone number.

**6 In surname order**

Lines 12 to 14

The rows come back in order of surname, and two customers with the same surname are put in first-name order. END closes the work that BEGIN opened. GO is not SQL. It is a word for SQL Server Management

> Studio (SSMS): run everything above this line as one batch, then start a fresh one. Every stored procedure ends with its own GO.

Press the Copy button under this step.

> **You should see** The button reads Copied for a moment.

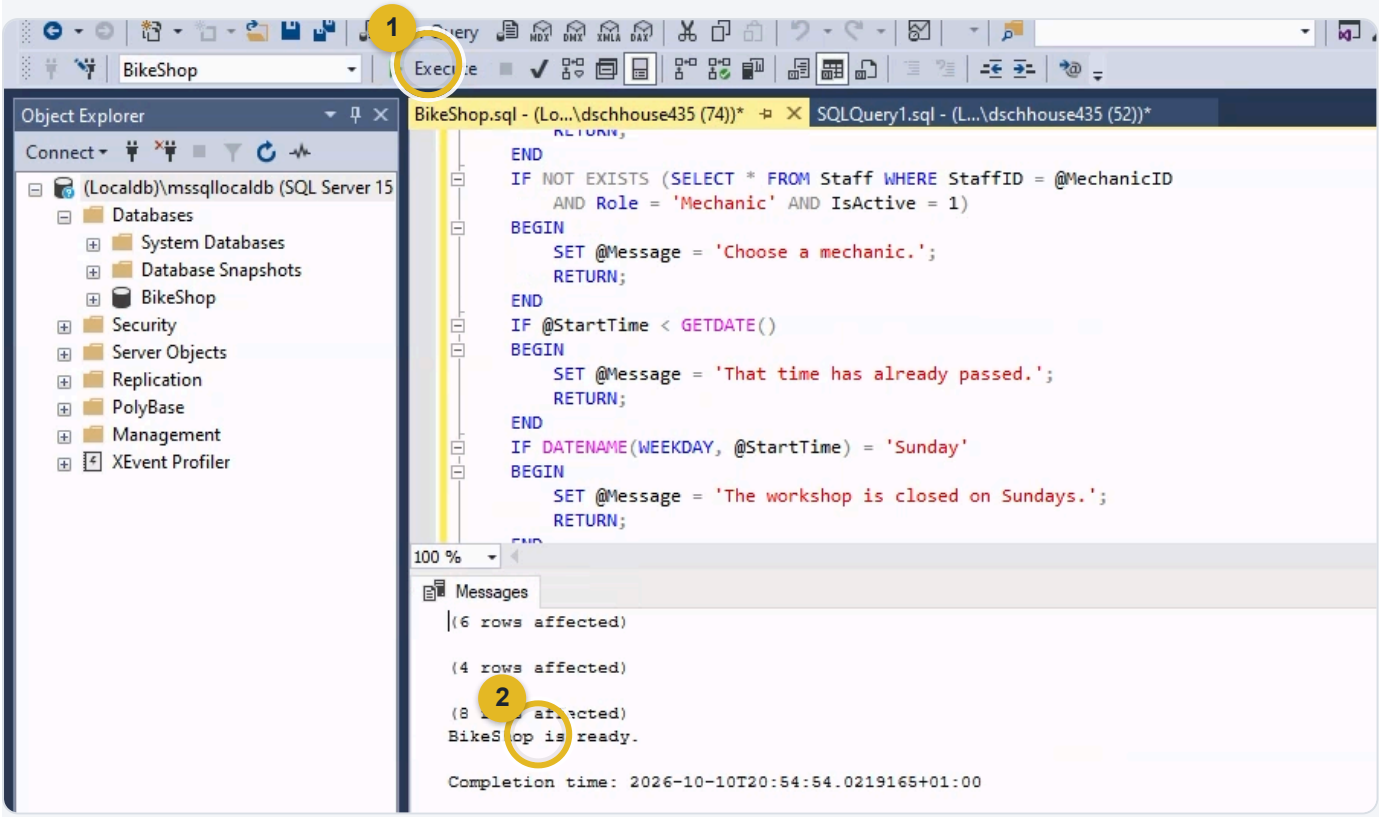
Click on the empty line above the line that prints BikeShop is ready, in your BikeShop.sql tab.

> **You should see** The cursor blinks on the empty line, and the line that prints BikeShop is ready is under it.

Press Ctrl and V together to paste usp_Customer_Search.

> **You should see** usp_Customer_Search is in your script, ending with GO, and the line that prints BikeShop is ready is still last.

Press F5, or click Execute on the toolbar, with nothing selected. This runs the whole script, so your database is built again from nothing with the new stored procedure in it.




Execute with nothing selected runs the whole script. The Messages pane counts the sample rows it put back, then says BikeShop is ready.

1. Click Execute, or press F5.

2. The last lines say BikeShop is ready, then a Completion time. Red text here is an error to read.

> **You should see** The Messages pane ends with BikeShop is ready and a Completion time, with no red text above it.

**IF IT GOES WRONG** A red error in the Messages pane

Double-click the error. SQL Server Management Studio (SSMS) jumps to the line it is about. Compare that line with the block above, letter by letter.

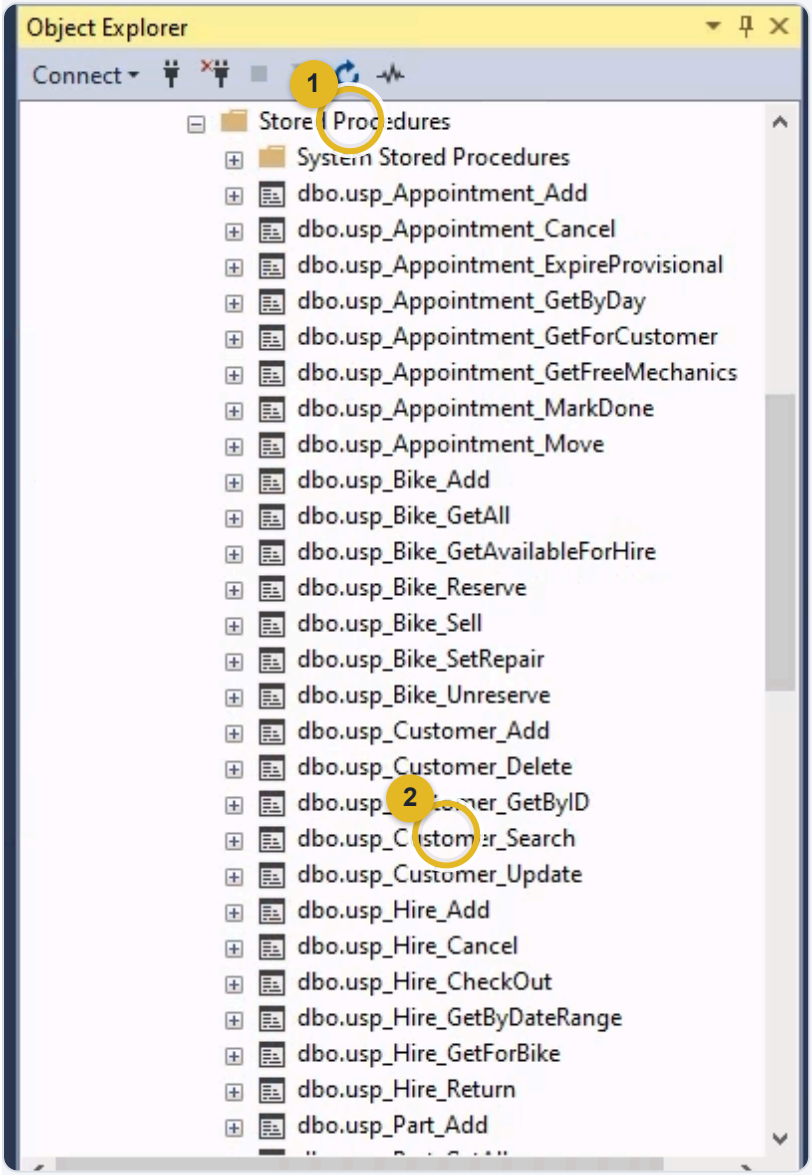
In the Object Explorer pane on the left, click the plus sign beside BikeShop.

> **You should see** Folders such as Tables and Programmability appear under BikeShop.

Click the plus sign beside Programmability.

> **You should see** More folders appear under Programmability, including Stored Procedures.

Click the plus sign beside Stored Procedures.



Where a stored procedure lives once it has run: inside the database, under Programmability, then Stored Procedures.

1. Expand Programmability, then Stored Procedures.

2. dbo.usp_Customer_Search. If a new one does not show, right-click Stored Procedures and click Refresh.

**You should see** dbo.usp_Customer_Search is listed. The dbo in front is the owner the database gives everything, and you never type it.

**IF IT GOES WRONG** Stored Procedures is empty

The list was drawn before the script ran. Right-click Stored Procedures and click Refresh.

# Test the search in a new query window.

Tests never go in your script. They go in a second window, so that the script stays clean and can be run again whenever you want the sample data back. The test below runs the stored procedure with the letters ar as its search text.

**Copy this one** **A test that searches for ar** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

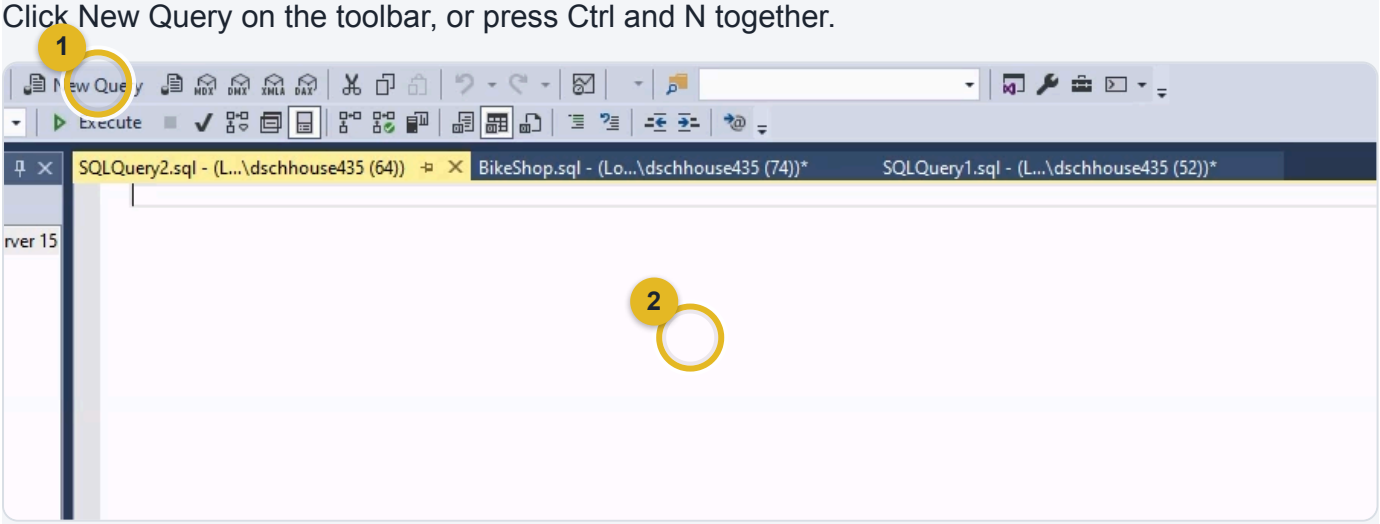
```sql
-- Every customer with the letters ar in their first name, last name or
-- phone number: three rows, in surname order.
EXEC usp_Customer_Search @SearchText = 'ar';
```

**1 Run the stored procedure**

Line 3

EXEC runs the stored procedure by name. The parameter is named, with an equals sign and the value it should hold. The rows it finds appear under Results, as if you had typed the SELECT yourself.

Click New Query on the toolbar, or press Ctrl and N together.




New Query opens a second, empty window. Tests go in here, so your script stays clean.

1. Click New Query, or press Ctrl and N together.
2. Paste the test into the empty window.

**You should see** A new tab opens with an empty query window, and the cursor blinks in it.

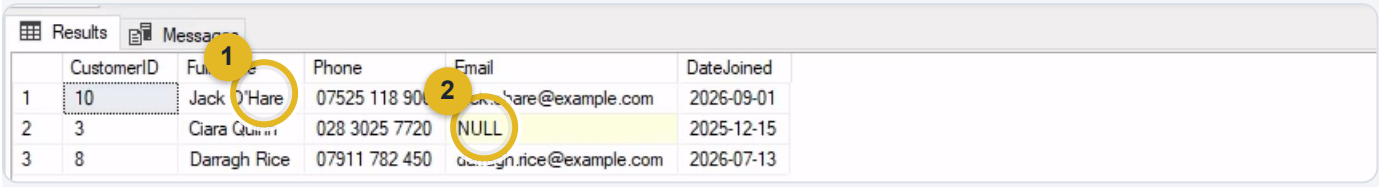
Press the Copy button under this step.

> **You should see** The button reads Copied for a moment.

Click in the empty query window and press Ctrl and V together to paste the test.

> **You should see** The test is in the window, and it runs usp_Customer_Search.

Press F5 to run the test.




The search for ar found three customers: the letters are in Jack's surname, Ciara's first name and Darragh's first name. Surnames are in order.

1. FullName is one column made from FirstName and LastName joined together. The word AS gave the column that name.

2. Ciara Quinn has NULL for Email, which means no email at all, because the shop never took one.

> **You should see** The Results tab shows three rows: Jack O'Hare, Ciara Quinn and Darragh Rice, in that order.

**IF IT GOES WRONG** 'Could not find stored procedure'

The database has not got the stored procedure yet, or the test window is talking to a different database. Check that the lines you added ran without a red error. Then look at the box on the toolbar that names the database: if it says master, choose BikeShop, and press F5 again.

In the EXEC line, change ar inside the quotes to 028.

> **You should see** The EXEC line now ends with @SearchText = '028', followed by a semicolon.

Press F5 to run the search again.

> **You should see** The Results tab shows six rows: Hannah Boyle, Róisín Campbell, Méabh Cunningham, Aoife McCann, Gráinne Murphy and Ciara Quinn. Every one of them has a phone number that starts with 028, so the third LIKE found them.

**IF IT GOES WRONG** No rows at all, with the column headings still there

The search text is spelled differently from any name or phone number. Check that the quotes are the plain

single ones, and that there is no space inside them.

**IF IT GOES WRONG** More rows than the picture shows

The search text is shorter than ar, or the quotes are empty, so more customers match. Put ar back, inside single quotes.

# Read the stored procedure that fetches one customer by number, and add it.

The second Get is shorter. The customers screen will pick one row in its grid and need that customer's details, so it hands over the number and gets back the one row. There is nothing new in it, so read it, then paste it under the search.

**Read this one** **usp_Customer_GetByID: one customer, by number** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
CREATE OR ALTER PROCEDURE usp_Customer_GetByID
    @CustomerID INT
AS
BEGIN
    SELECT CustomerID, FirstName, LastName, Phone, Email, DateJoined
    FROM Customer
    WHERE CustomerID = @CustomerID;
END
GO
```

**1 The name, and what goes in**

Lines 1 to 2

The one parameter is the customer's number, a whole number, which is what INT means.

**2 AS and BEGIN**

Lines 3 to 4

This stored procedure only reads, so it has no @Message. AS and BEGIN mark where the work starts.

**3 What comes back, and from where**

Lines 5 to 6

The SELECT takes six columns from the Customer table. The names stay in two columns this time, because the edit form in chapter 6 has a box for each.

**4 Only the one customer**

Lines 7 to 9

The WHERE line keeps only the row whose number matches the parameter. A number is given to one row and never reused, so this is one row or none. GO ends the batch, ready for the next stored procedure.

Press the Copy button under this step.

**You should see** The button reads Copied for a moment.

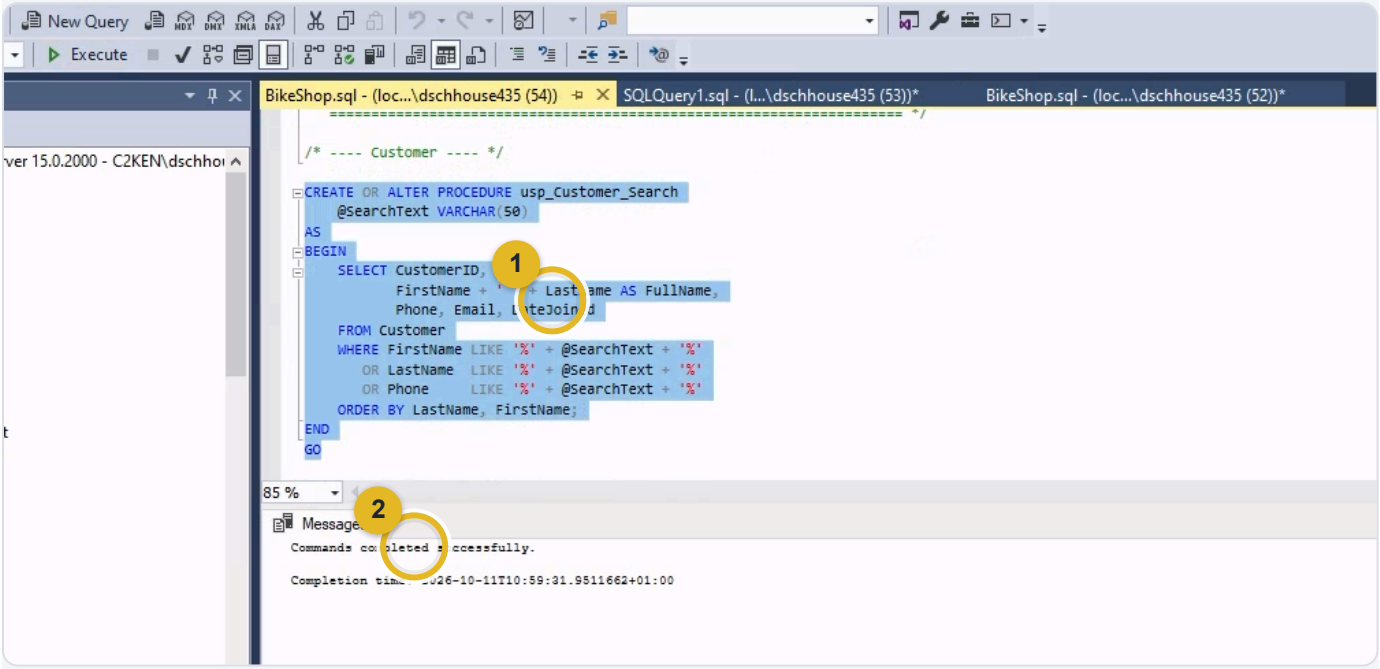
Click on the empty line above the line that prints BikeShop is ready, in your BikeShop.sql tab.

> **You should see** The cursor blinks on the empty line, and the line that prints BikeShop is ready is under it.

Press Ctrl and V together to paste usp_Customer_GetByID.

> **You should see** usp_Customer_GetByID is in your script, ending with GO, and the line that prints BikeShop is ready is still last.

Select the lines you added, by dragging from CREATE OR ALTER PROCEDURE down to GO.




Select the lines you added, then press F5. Only the highlighted lines run.

1. Drag from CREATE OR ALTER PROCEDURE down to GO, so the whole block is highlighted.

2. The Messages pane says: Commands completed successfully.

> **You should see** Every line from CREATE OR ALTER PROCEDURE down to GO is highlighted.

Press F5 to run the highlighted lines.

> **You should see** The Messages pane says: Commands completed successfully. Only the highlighted lines ran.

**IF IT GOES WRONG** The Messages pane shows (12 rows affected) and BikeShop is ready

Nothing was selected, so the whole script ran and the sample data was put back. Nothing is broken. Carry on.

**IF IT GOES WRONG** A red error in the Messages pane

Double-click the error. SQL Server Management Studio (SSMS) jumps to the line it is about. Compare that line with the block above, letter by letter.

From now on, every stored procedure you add runs this way: paste it in, select it from CREATE OR ALTER PROCEDURE down to GO, and press F5. The whole script only needs to run again when you want the sample data put back.

# Test the fetch by number.

**Copy this one** **A test that fetches customer 3** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  -- Customer 3, Ciara Quinn, on her own: one row.
2  EXEC usp_Customer_GetByID @CustomerID = 3;
```

**1 Run the stored procedure**

Line 2

EXEC runs the stored procedure with the customer number 3. Customer 3 in the sample data is Ciara Quinn.

Press the Copy button under this step.

**You should see** The button reads Copied for a moment.

Click in your test window and press Ctrl and A together to select the old test.

**You should see** The whole of the old test is highlighted.

Press Ctrl and V together to paste the new test over the old one.

**You should see** The new test has replaced the old one, and it runs usp_Customer_GetByID.

Press F5 to run the test.

**You should see** The Results tab shows one row: CustomerID 3, Ciara, Quinn, 028 3025 7720, NULL for Email, and the day she joined.

**IF IT GOES WRONG** 'Could not find stored procedure'

The database has not got the stored procedure yet, or the test window is talking to a different database. Check that the lines you added ran without a red error. Then look at the box on the toolbar that names the database: if it says master, choose BikeShop, and press F5 again.

In the EXEC line, change the 3 to 99.

**You should see** The EXEC line now ends with @CustomerID = 99, followed by a semicolon.

Press F5 to run the test again.

> **You should see** The Results tab shows the column headings with no row under them. There is no customer 99, and the stored procedure found nothing, which is right, not an error.

# Copy the stored procedure that adds a customer, and read the two things that come back out.

An Add puts one row into the table. Two things come back out of it. The first is the number the database chose for the new row, because the program will need it. The second is @Message, the one way a stored procedure says no. Every stored procedure that changes data has @Message as its last parameter. This one has no rule to check, so its message is always blank, and that blank is the sign that it worked.

Copy this one **usp_Customer_Add: put one customer in, and hand back the new number** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
CREATE OR ALTER PROCEDURE usp_Customer_Add
    @FirstName     VARCHAR(40),
    @LastName      VARCHAR(40),
    @Phone         VARCHAR(15),
    @Email         VARCHAR(100),
    @NewCustomerID INT OUTPUT,
    @Message VARCHAR(200) OUTPUT
AS
BEGIN
    SET @Message = '';
    INSERT INTO Customer (FirstName, LastName, Phone, Email)
    VALUES (@FirstName, @LastName, @Phone, @Email);
    SET @NewCustomerID = SCOPE_IDENTITY();
END
GO
```

**1 The name, and what goes in and out**

Lines 1 to 7

Four parameters carry the customer's details in: first name, last name, phone and email, each with the same size as its column in the table. The last two have the word OUTPUT after them, which means they carry an answer back out: @NewCustomerID will hold the new number, and @Message the refusal, if there is one.

**2 Start with a blank message**

Lines 8 to 10

AS and BEGIN mark where the work starts. The first line of work makes @Message blank. A blank message is the sign that everything worked, so it must be blank before any check can change it.

**3 Add the row**

Lines 11 to 12

INSERT INTO names the table and the four columns to fill. VALUES gives the four parameters, in the same order. CustomerID and DateJoined are not listed, because the table fills them itself: the next number, and today's date.

**4 Hand back the new number**

Lines 13 to 15

SCOPE_IDENTITY() is the number the database gave to the row that was added a moment ago. SET puts it into the OUTPUT parameter, so whoever ran the stored procedure gets it back. GO ends the batch, ready for the next stored procedure.

Press the Copy button under this step.

**You should see** The button reads Copied for a moment.

Click on the empty line above the line that prints BikeShop is ready, in your BikeShop.sql tab.

> **You should see** The cursor blinks on the empty line, and the line that prints BikeShop is ready is under it.

Press Ctrl and V together to paste usp_Customer_Add.

> **You should see** usp_Customer_Add is in your script, ending with GO, and the line that prints BikeShop is ready is still last.

Select the lines you added, by dragging from CREATE OR ALTER PROCEDURE down to GO.

> **You should see** Every line from CREATE OR ALTER PROCEDURE down to GO is highlighted.

Press F5 to run the highlighted lines.

> **You should see** The Messages pane says: Commands completed successfully. Only the highlighted lines ran.

**IF IT GOES WRONG** The Messages pane shows (12 rows affected) and BikeShop is ready

Nothing was selected, so the whole script ran and the sample data was put back. Nothing is broken. Carry on.

**IF IT GOES WRONG** A red error in the Messages pane

Double-click the error. SQL Server Management Studio (SSMS) jumps to the line it is about. Compare that line with the block above, letter by letter.

## Test the add, and read how a test collects the answers.

A stored procedure with OUTPUT parameters needs somewhere to put its answers. The test makes two variables, runs the stored procedure, and then looks at what came back. If the message is blank, it prints the new number. Then it fetches the new row with the stored procedure from the last stage, using that number.

Copy this one **A test that adds Eimear Hughes and prints her new number** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
-- Add a new customer, print the number the database gave her, then fetch
-- her row back by that number.
DECLARE @NewID INT, @Message VARCHAR(200);

EXEC usp_Customer_Add
     @FirstName = 'Eimear', @LastName = 'Hughes', @Phone = '07700 900 123',
     @Email = 'eimear.hughes@example.com',
     @NewCustomerID = @NewID OUTPUT, @Message = @Message OUTPUT;

IF @Message = ''
BEGIN
    PRINT 'Added. New customer number ' + CAST(@NewID AS VARCHAR) + '.';
END
ELSE
BEGIN
    PRINT @Message;
END

EXEC usp_Customer_GetByID @CustomerID = @NewID;
```

**1 Two boxes to hold the answers**

Line 3

DECLARE makes a variable. A variable is a named box that holds one value while the script runs. @NewID will hold the number the database gives the new row, and @Message the refusal, if there is one.

**2 Run the stored procedure**

Lines 5 to 8

EXEC runs the stored procedure with every parameter named, so the order does not matter. The two OUTPUT ones are given a variable to fill, and the word OUTPUT is written again here, in the test that runs the stored procedure.

**3 If the message is blank, it worked**

Lines 10 to 13

A blank message means the row was saved. PRINT writes one line in the Messages pane. CAST turns the number into text so that it can be joined onto the sentence with the plus signs.

**4 Otherwise, print the refusal**

Lines 14 to 17

If the stored procedure set @Message, something was refused, and the sentence it wrote is printed as it is.

**5 Fetch the new row by its number**

Line 19

The number that came back is handed straight to the fetch from stage 2, so the new row appears under Results.

Press the Copy button under this step.

> **You should see** The button reads Copied for a moment.

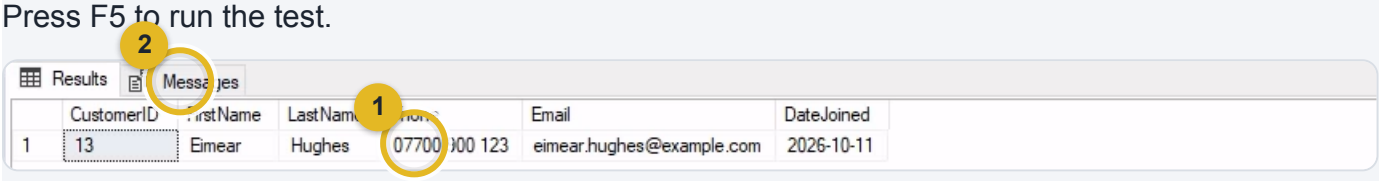
Click in your test window and press Ctrl and A together to select the old test.

> **You should see** The whole of the old test is highlighted.

Press Ctrl and V together to paste the new test over the old one.

> **You should see** The new test has replaced the old one, and it runs usp_Customer_Add.

Press F5 to run the test.




The database chose the number 13 for Eimear Hughes, handed it back through @NewCustomerID, and the second EXEC fetched her row with it.

1. The new row, fetched by the number the database chose.

2. Click Messages to see the printed line: Added. New customer number 13.

> **You should see** The Results tab shows one row for Eimear Hughes with CustomerID 13. Click Messages: Added. New customer number 13.

**IF IT GOES WRONG** 'Could not find stored procedure'

The database has not got the stored procedure yet, or the test window is talking to a different database. Check that the lines you added ran without a red error. Then look at the box on the toolbar that names the database: if it says master, choose BikeShop, and press F5 again.

**IF IT GOES WRONG** Must declare the scalar variable @NewID

The DECLARE line at the top of the test did not run with the rest. Select nothing, so that the whole test window runs, and press F5 again.

Press F5 again, changing nothing.

> **You should see** A second Eimear Hughes, with CustomerID 14. The database never gives a number twice, and this stored procedure has no rule against the same name twice. The whole script will put the sample data back later.

# Type the lines that make an Update different from an Add.

An Update changes one row that is already there. It is the Add with two things changed: the customer's number comes in as well, so the stored procedure knows which row, and the INSERT becomes an UPDATE with a WHERE. Everything else is the same, so this one is pasted with a gap, and you type the four lines that differ.

**Type this one** **usp_Customer_Update: change one customer** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
CREATE OR ALTER PROCEDURE usp_Customer_Update
    @CustomerID INT,
    @FirstName  VARCHAR(40),
    @LastName   VARCHAR(40),
    @Phone      VARCHAR(15),
    @Email      VARCHAR(100),
    @Message VARCHAR(200) OUTPUT
AS
BEGIN
    SET @Message = '';
    UPDATE Customer
    SET FirstName = @FirstName, LastName = @LastName, Phone = @Phone,
        Email = @Email
    WHERE CustomerID = @CustomerID;
END
GO
```

**1 The name, and what goes in and out**

Lines 1 to 7

The customer's number comes first, then the same four details as the Add. @Message comes back out. There is no new number to hand back, because the row already has one.

**2 Start with a blank message**

Lines 8 to 10

AS and BEGIN mark where the work starts. The first line of work makes @Message blank. A blank message is the sign that everything worked, so it must be blank before any check can change it.

**3 Change the one row**

Lines 11 to 16

UPDATE names the table. SET gives each column its new value from the matching parameter, with commas between. WHERE picks the one row whose number matches, and without it every customer would get these details. GO ends the batch, ready for the next stored procedure.

**Why you type it** UPDATE without a WHERE changes every row in the table. Typing the WHERE yourself, once, is the best way to never forget it.

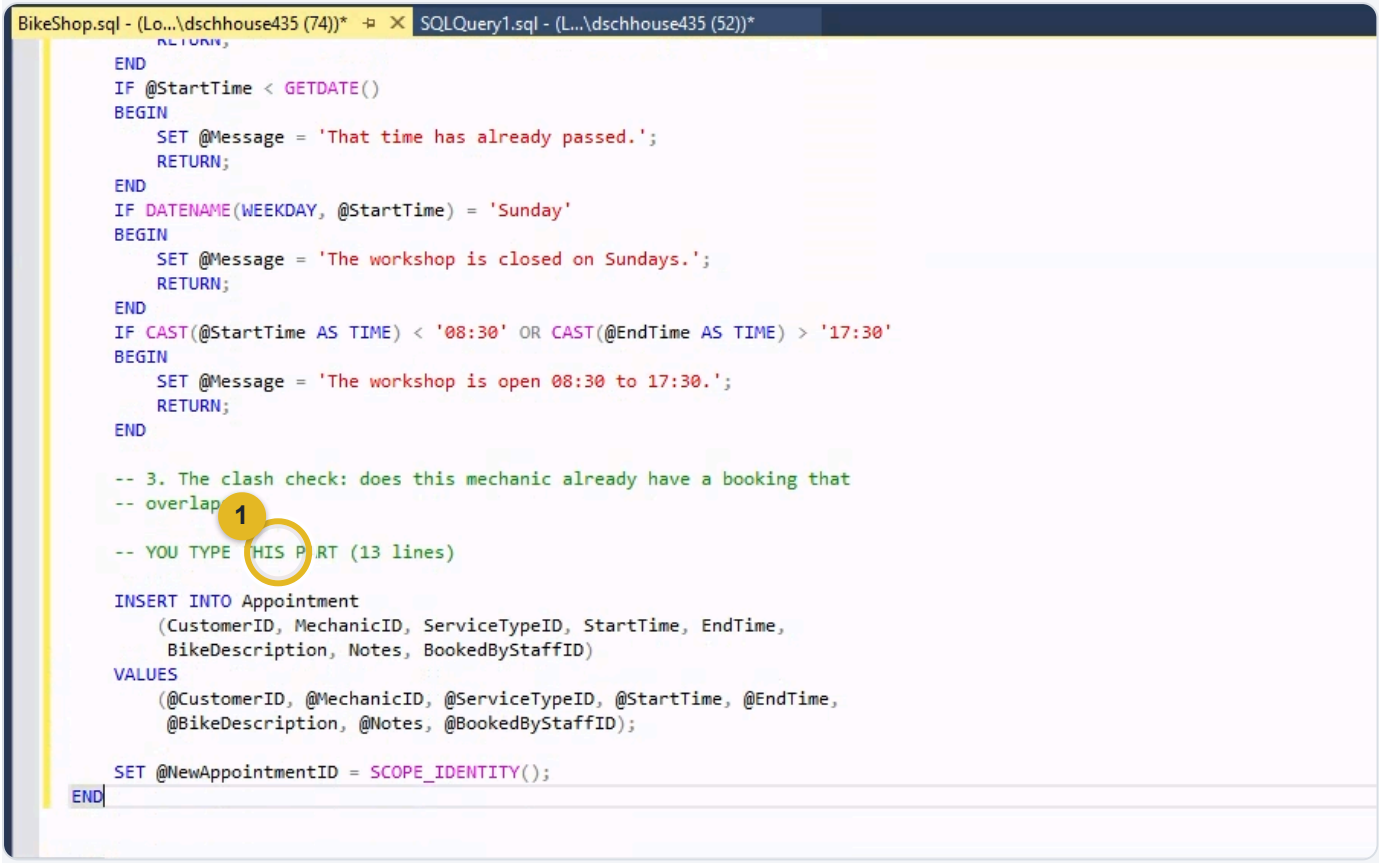
Press the Copy button under this step.

**You should see** The button reads Copied for a moment.

Click on the empty line above the line that prints BikeShop is ready, in your BikeShop.sql tab.

**You should see** The cursor blinks on the empty line, and the line that prints BikeShop is ready is under it.

Press Ctrl and V together to paste usp_Customer_Update.




After the paste. Everything is in except the yellow lines. Each marker line shows where a yellow block goes.

1 Click on the marker line, type the yellow lines in its place, and delete the marker.

> **You should see** The stored procedure is in your script, with a line that says YOU TYPE THIS PART where each yellow block goes.

Type the yellow lines in place of each line that says YOU TYPE THIS PART, so that no marker line is left.

> **You should see** Four lines of your own, from UPDATE Customer to the WHERE line, and no marker line.

**IF IT GOES WRONG** A red wavy line under a word

A spelling slip. Compare your line with the yellow line, letter by letter, and check that each line ends the way the yellow line ends.

**IF IT GOES WRONG** 'Must declare the scalar variable …'

A name beginning with @ is spelled differently from the line that declares it. Make the two match.

Select the lines you added, by dragging from CREATE OR ALTER PROCEDURE down to GO.

> **You should see** Every line from CREATE OR ALTER PROCEDURE down to GO is highlighted.

Press F5 to run the highlighted lines.

> **You should see** The Messages pane says: Commands completed successfully. Only the highlighted lines ran.

**IF IT GOES WRONG** The Messages pane shows (12 rows affected) and BikeShop is ready

Nothing was selected, so the whole script ran and the sample data was put back. Nothing is broken. Carry on.

**IF IT GOES WRONG** A red error in the Messages pane

Double-click the error. SQL Server Management Studio (SSMS) jumps to the line it is about. Compare that line with the block above, letter by letter.

## Test the update on Ben Toner.

Copy this one **A test that changes Ben Toner's phone number** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
-- Change Ben Toner's phone number (customer 12), then fetch his row back.
DECLARE @Message VARCHAR(200);

EXEC usp_Customer_Update
     @CustomerID = 12, @FirstName = 'Ben', @LastName = 'Toner',
     @Phone = '07388 640 999', @Email = 'ben.toner@example.com',
     @Message = @Message OUTPUT;

IF @Message = ''
BEGIN
    PRINT 'Updated customer number 12.';
END
ELSE
BEGIN
    PRINT @Message;
END

EXEC usp_Customer_GetByID @CustomerID = 12;
```

**1 One box for the message**

Line 2

DECLARE makes a variable, a named box that holds one value while the script runs. @Message will hold the refusal, if there is one.

**2 Run the stored procedure**

Lines 4 to 7

Customer 12 is Ben Toner. His name and email are given again unchanged, and the phone is new. An Update always sends every column, and the screen in chapter 6 will do the same.

**3 If the message is blank, it worked**

Lines 9 to 12

A blank message means the row was changed, and one line says so.

**4 Otherwise, print the refusal**

Lines 13 to 16

If the stored procedure set @Message, something was refused, and the sentence it wrote is printed as it is.

**5 Fetch the row to see the change**

Line 18

The fetch from stage 2 shows Ben's row with the new phone number in it.

Press the Copy button under this step.

> **You should see** The button reads Copied for a moment.

Click in your test window and press Ctrl and A together to select the old test.

> **You should see** The whole of the old test is highlighted.

Press Ctrl and V together to paste the new test over the old one.

> **You should see** The new test has replaced the old one, and it runs usp_Customer_Update.

Press F5 to run the test.

> **You should see** The Results tab shows Ben Toner with the phone 07388 640 999. Click Messages: Updated customer number 12.

**IF IT GOES WRONG** 'Could not find stored procedure'

The database has not got the stored procedure yet, or the test window is talking to a different database. Check that the lines you added ran without a red error. Then look at the box on the toolbar that names the database: if it says master, choose BikeShop, and press F5 again.

**IF IT GOES WRONG** Every customer now has the phone 07388 640 999

The WHERE line is missing from your UPDATE, so every row was changed. Put the WHERE line in, select the block, press F5, then click in BikeShop.sql and press F5 with nothing selected to put the sample data back.

STAGE 4 OF 7 · THE DELETE THAT REFUSES · SCREEN 1 OF 3

# Copy the delete, and read how a stored procedure says no.

A customer with bookings on record cannot be deleted, because the bookings would be left pointing at nobody. The database would refuse the DELETE anyway, with a long red error, but the shop should get a sentence, not an error. So the stored procedure checks first. If any booking, hire, sale or bike belongs to this customer, it writes a sentence into @Message and stops. Nothing below the stop runs, so the DELETE never happens.

Copy this one **usp_Customer_Delete: take one customer out, unless they have history** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
CREATE OR ALTER PROCEDURE usp_Customer_Delete
    @CustomerID INT,
    @Message VARCHAR(200) OUTPUT
AS
BEGIN
    SET @Message = '';
    IF EXISTS (SELECT * FROM Appointment WHERE CustomerID = @CustomerID)
       OR EXISTS (SELECT * FROM Hire     WHERE CustomerID = @CustomerID)
       OR EXISTS (SELECT * FROM PartSale WHERE CustomerID = @CustomerID)
       OR EXISTS (SELECT * FROM Bike     WHERE CustomerID = @CustomerID)
    BEGIN
        SET @Message =
            'This customer has bookings or sales on record, so they cannot ' +
            'be deleted.';
        RETURN;
    END

    DELETE FROM Customer WHERE CustomerID = @CustomerID;
END
GO
```

**1 The name, and what goes in and out**

Lines 1 to 3

The customer's number goes in, and @Message comes back out.

**2 Start with a blank message**

Lines 4 to 6

AS and BEGIN mark where the work starts. The first line of work makes @Message blank. A blank message is the sign that everything worked, so it must be blank before any check can change it.

**3 Look in four tables**

Lines 7 to 10

IF EXISTS is true when the SELECT inside the brackets finds at least one row. Four tables are looked in, joined by OR: a booking, a hire, a part sale or a bike that belongs to this customer. One match is enough.

**4 Say no and stop**

Lines 11 to 16

When the test is true, the lines between BEGIN and END run. SET puts a plain sentence into @Message. The sentence is split over two lines with a plus sign, which joins the two halves into one. RETURN means stop here: nothing below this line runs.

**5 Delete the row**

Lines 18 to 20

This line only runs when the test above was false. DELETE FROM names the table, and WHERE picks the one row. Without the WHERE, every customer would go. GO ends the batch, ready for the next stored procedure.

Press the Copy button under this step.

> **You should see** The button reads Copied for a moment.

Click on the empty line above the line that prints BikeShop is ready, in your BikeShop.sql tab.

> **You should see** The cursor blinks on the empty line, and the line that prints BikeShop is ready is under it.

Press Ctrl and V together to paste usp_Customer_Delete.

> **You should see** usp_Customer_Delete is in your script, ending with GO, and the line that prints BikeShop is ready is still last.

Select the lines you added, by dragging from CREATE OR ALTER PROCEDURE down to GO.

> **You should see** Every line from CREATE OR ALTER PROCEDURE down to GO is highlighted.

Press F5 to run the highlighted lines.

**You should see** The Messages pane says: Commands completed successfully. Only the highlighted lines ran.

**IF IT GOES WRONG** The Messages pane shows (12 rows affected) and BikeShop is ready

Nothing was selected, so the whole script ran and the sample data was put back. Nothing is broken. Carry on.

**IF IT GOES WRONG** A red error in the Messages pane

Double-click the error. SQL Server Management Studio (SSMS) jumps to the line it is about. Compare that line with the block above, letter by letter.

# Test the delete on a customer who has bookings.

Aoife McCann is customer 1, and she has two bookings. The test tries to delete her, then prints whichever came back: the refusal, or a line that says the rule is broken.

**Copy this one** **A test that tries to delete customer 1** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
-- Try to delete Aoife McCann (customer 1). She has bookings on record, so
-- the stored procedure refuses and hands back its sentence.
DECLARE @Message VARCHAR(200);

EXEC usp_Customer_Delete @CustomerID = 1, @Message = @Message OUTPUT;

IF @Message = ''
BEGIN
    PRINT 'Deleted customer number 1. The rule is broken: she had bookings.';
END
ELSE
BEGIN
    PRINT 'Refused: ' + @Message;
END
```

**1 One box for the message**

Line 3

DECLARE makes a variable, a named box that holds one value while the script runs. @Message will hold the refusal, if there is one.

**2 Run the stored procedure**

Line 5

EXEC runs the stored procedure with customer 1 and a variable for the message to come back in.

**3 If the message is blank, the rule is broken**

Lines 7 to 10

A blank message would mean the row was deleted, and for customer 1 that must never happen. The printed line says so, so that you cannot miss it.

**4 Otherwise, print the refusal**

Lines 11 to 14

The sentence from inside the stored procedure is printed after the word Refused.

Press the Copy button under this step.

**You should see** The button reads Copied for a moment.

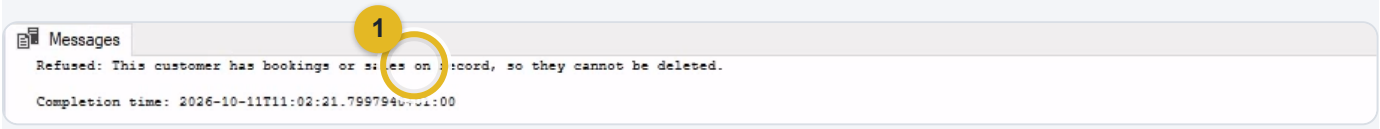
Click in your test window and press Ctrl and A together to select the old test.

> **You should see** The whole of the old test is highlighted.

Press Ctrl and V together to paste the new test over the old one.

> **You should see** The new test has replaced the old one, and it runs usp_Customer_Delete.

Press F5 to run the test.




Aoife McCann has two bookings, so the stored procedure set @Message and stopped. The DELETE line never ran.

1 The sentence came from inside the stored procedure, word for word.

> **You should see** The Messages tab says: Refused: This customer has bookings or sales on record, so they cannot be deleted.

**IF IT GOES WRONG** 'Could not find stored procedure'

The database has not got the stored procedure yet, or the test window is talking to a different database. Check that the lines you added ran without a red error. Then look at the box on the toolbar that names the database: if it says master, choose BikeShop, and press F5 again.

**IF IT GOES WRONG** The Messages tab says Deleted customer number 1. The rule is broken: she had bookings.

The IF EXISTS test did not find her bookings. Check that the four table names inside the brackets are spelled the way the tables are, and that each test reads CustomerID = @CustomerID.

**IF IT GOES WRONG** A long red error that mentions FK_Appointment_Customer

The stored procedure ran the DELETE without checking first, and the table itself refused. The IF EXISTS lines are missing or the RETURN is missing. Compare your block with the one above.

# Test the delete on a customer who has nothing on record.

Every customer in the sample data has at least one booking, so the test makes its own customer first. It adds Niall Keenan, deletes him by the number that came back, and then looks for him. No row should come back, because he is gone.

**Copy this one** **A test that adds a customer, deletes them, and looks for them** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
-- Add a customer who has no bookings, delete them straight away, then
-- look for them by number: no row comes back.
DECLARE @NewID INT, @Message VARCHAR(200);

EXEC usp_Customer_Add
     @FirstName = 'Niall', @LastName = 'Keenan', @Phone = '07700 900 000',
     @Email = NULL,
     @NewCustomerID = @NewID OUTPUT, @Message = @Message OUTPUT;
PRINT 'Added customer number ' + CAST(@NewID AS VARCHAR) + '.';

EXEC usp_Customer_Delete @CustomerID = @NewID, @Message = @Message OUTPUT;

IF @Message = ''
BEGIN
    PRINT 'Deleted customer number ' + CAST(@NewID AS VARCHAR) + '.';
END
ELSE
BEGIN
    PRINT 'Refused: ' + @Message;
END

EXEC usp_Customer_GetByID @CustomerID = @NewID;
```

**1 Two boxes to hold the answers**

Line 3

DECLARE makes a variable. A variable is a named box that holds one value while the script runs. @NewID will hold the number the database gives the new row, and @Message the refusal, if there is one.

**2 Add a customer with nothing on record**

Lines 5 to 9

This is the Add from stage 3, with NULL for the email, which means no email at all. The new number comes back in @NewID, and one line prints it.

**3 Delete by that number**

Line 11

The Delete is given the number that the Add handed back a moment ago.

**4 If the message is blank, it worked**

Lines 13 to 16

A blank message means the row was deleted, and one line says so, with the number.

**5 Otherwise, print the refusal**

Lines 17 to 20

If the stored procedure set @Message, something was refused, and the sentence it wrote is printed as it is.

**6 Look for the deleted customer**

Line 22

The fetch by number finds nothing, so the Results tab shows the headings with no row under them.

Press the Copy button under this step.

> **You should see** The button reads Copied for a moment.

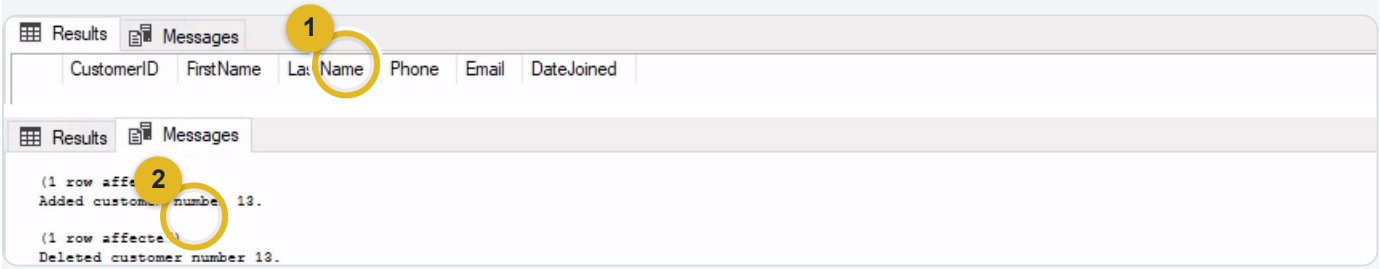
Click in your test window and press Ctrl and A together to select the old test.

> **You should see** The whole of the old test is highlighted.

Press Ctrl and V together to paste the new test over the old one.

> **You should see** The new test has replaced the old one, and it runs usp_Customer_Add, then usp_Customer_Delete.

Press F5 to run the test.




Niall Keenan was added, had no bookings, and was deleted. The last EXEC looks for him by number and finds no row.

1 Headings with nothing under them: the row is gone.

2 The Messages tab shows the two printed lines.

> **You should see** The Results tab shows column headings and no row. Click the Messages tab. It says Added customer number 15, then Deleted customer number 15. The number is 15 if you ran the add test twice in stage 3, and 14 if you ran it once. The picture was taken on fresh sample data, so it shows 13. The two numbers must match each other.

**IF IT GOES WRONG** 'Could not find stored procedure'

The database has not got the stored procedure yet, or the test window is talking to a different database. Check that the lines you added ran without a red error. Then look at the box on the toolbar that names the database: if it says master, choose BikeShop, and press F5 again.

Click anywhere in the script in your BikeShop.sql tab, so that no line is highlighted.

> **You should see** Your script is at the front, and nothing in it is highlighted.

Press F5 with nothing selected, so that the sample data is put back and the customers you added are gone.

> **You should see** The Messages pane ends with BikeShop is ready. The two Eimear Hughes rows and the changed phone number are gone, and Aoife McCann is still customer 1.

STAGE 5 OF 7 · NAMES, NOT NUMBERS · SCREEN 1 OF 5

# Copy the stored procedure that lists a customer's bookings with names in it.

A booking row holds numbers: the mechanic is 4, the service is 4. Nobody in the shop knows who 4 is. A Get that shows bookings has to swap each number for the name it stands for, and JOIN is how. JOIN lines up the rows of two tables where a number in one matches a number in the other. This stored procedure joins the bookings to Staff for the mechanic's name and to ServiceType for the service's name. The customers screen in chapter 6 will show its rows in a grid.

Copy this one **usp_Appointment_GetForCustomer: one customer's bookings, with names** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
CREATE OR ALTER PROCEDURE usp_Appointment_GetForCustomer
    @CustomerID INT
AS
BEGIN
    SELECT a.AppointmentID, a.StartTime, a.EndTime,
           m.FirstName + ' ' + m.LastName AS Mechanic,
           st.Name AS Service, st.Price, a.BikeDescription, a.Status
    FROM Appointment a
    JOIN Staff       m  ON m.StaffID        = a.MechanicID
    JOIN ServiceType st ON st.ServiceTypeID = a.ServiceTypeID
    WHERE a.CustomerID = @CustomerID
    ORDER BY a.StartTime DESC;
END
GO
```

**1 The name, and what goes in**

Lines 1 to 2

The one parameter is the customer's number.

**2 AS and BEGIN**

Lines 3 to 4

This stored procedure only reads, so it has no @Message. AS and BEGIN mark where the work starts.

**3 What comes back**

Lines 5 to 7

Each column has a short letter and a dot in front, which says which table it comes from: a for Appointment, m for the mechanic's row in Staff, st for ServiceType. The mechanic's two names are joined into one column called Mechanic, and the service's name is called Service. The booking's own columns come as they are.

**4 Start from the bookings**

Line 8

The rows start from the Appointment table. The a after the table name is the short letter the SELECT used, and it saves typing the whole word each time.

**5 Two joins: the mechanic and the service**

Lines 9 to 10

The first JOIN lines up each booking with the Staff row whose StaffID matches the booking's MechanicID, and calls it m. The second lines up each booking with the ServiceType row whose number matches, and calls it st. ON says which numbers must match. After the joins, every name is within reach.

**6 Only this customer**

Line 11

The WHERE line keeps only the bookings whose customer number matches the parameter.

**7 Newest first**

Lines 12 to 14

DESC means descending, so the latest booking is at the top of the list. GO ends the batch, ready for the next stored procedure.

Press the Copy button under this step.

> **You should see** The button reads Copied for a moment.

Click on the empty line above the line that prints BikeShop is ready, in your BikeShop.sql tab.

> **You should see** The cursor blinks on the empty line, and the line that prints BikeShop is ready is under it.

Press Ctrl and V together to paste usp_Appointment_GetForCustomer.

> **You should see** usp_Appointment_GetForCustomer is in your script, ending with GO, and the line that prints BikeShop is ready is still last.

Select the lines you added, by dragging from CREATE OR ALTER PROCEDURE down to GO.

> **You should see** Every line from CREATE OR ALTER PROCEDURE down to GO is highlighted.

Press F5 to run the highlighted lines.

> **You should see** The Messages pane says: Commands completed successfully. Only the highlighted lines ran.

**IF IT GOES WRONG** The Messages pane shows (12 rows affected) and BikeShop is ready

Nothing was selected, so the whole script ran and the sample data was put back. Nothing is broken. Carry on.

**IF IT GOES WRONG** A red error in the Messages pane

Double-click the error. SQL Server Management Studio (SSMS) jumps to the line it is about. Compare that line with the block above, letter by letter.

# Test the bookings list for Aoife McCann.

**Copy this one** **A test that lists the bookings of customer 1** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  -- Every booking Aoife McCann (customer 1) has, newest first, with the
2  -- mechanic and the service as names.
3  EXEC usp_Appointment_GetForCustomer @CustomerID = 1;
```

**1 Run the stored procedure**

Line 3

EXEC runs the stored procedure with customer 1, who is Aoife McCann. Her bookings come back under Results, newest first.

Press the Copy button under this step.

**You should see** The button reads Copied for a moment.

Click in your test window and press Ctrl and A together to select the old test.

**You should see** The whole of the old test is highlighted.

Press Ctrl and V together to paste the new test over the old one.

**You should see** The new test has replaced the old one, and it runs usp_Appointment_GetForCustomer.

Press F5 to run the test.

Aoife McCann's two bookings, newest first. The Mechanic and Service columns hold names, not numbers, because of the two JOINs.

1. Mechanic reads Conor Walsh, not 4. Service reads Wheel true, not 4.

2. Last week's Basic service, marked Done.

> **You should see** The Results tab shows two rows. The first is a Wheel true with Conor Walsh, 20.00, marked Booked. The second is a Basic service with Aoife Byrne, 40.00, marked Done. Both are for a Blue Trek hybrid.

**IF IT GOES WRONG** 'Could not find stored procedure'

The database has not got the stored procedure yet, or the test window is talking to a different database. Check that the lines you added ran without a red error. Then look at the box on the toolbar that names the database: if it says master, choose BikeShop, and press F5 again.

In the EXEC line, change the 1 to 2.

> **You should see** The EXEC line now ends with @CustomerID = 2, followed by a semicolon.

Press F5 to run the test again.

> **You should see** The Results tab shows two rows for Seán Devlin, both a Full service on a Black Giant road bike: one with Niamh Hughes marked Booked, and one with Conor Walsh marked Done.

**IF IT GOES WRONG** 'The multi-part identifier could not be bound'

A short letter in front of a column does not match the one given after a table name. Check that Appointment is called a, Staff is called m, and ServiceType is called st, in the FROM and JOIN lines.

**IF IT GOES WRONG** Numbers in the Mechanic column instead of names

The SELECT is reading a.MechanicID instead of the two name columns from m. Compare the SELECT with the block above.

# Copy the sign-in stored procedure, and read how one row means yes.

The program's first screen in chapter 5 is the sign-in. The staff member types a username and a password, and the program hands both to this stored procedure. If one row comes back, the sign-in is right, and the row says who it is and whether they are the manager. If no row comes back, the sign-in is wrong. There is no @Message here, because no row is the whole answer.

**Copy this one** **usp_Staff_Login: one row for the right username and password, or none** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
CREATE OR ALTER PROCEDURE usp_Staff_Login
    @Username VARCHAR(30),
    @Password VARCHAR(30)
AS
BEGIN
    SELECT StaffID, FirstName + ' ' + LastName AS FullName, Role
    FROM Staff
    WHERE Username = @Username AND Password = @Password AND IsActive = 1;
END
GO
```

**1 The name, and what goes in**

Lines 1 to 3

Two parameters go in: the username and the password, each up to 30 characters.

**2 AS and BEGIN**

Lines 4 to 5

This stored procedure only reads, so it has no @Message. AS and BEGIN mark where the work starts.

**3 What comes back, and from where**

Lines 6 to 7

The SELECT brings back three things from the Staff table: the number, the two names joined into FullName, and the Role, which is Manager, Counter or Mechanic. The password never comes back out.

**4 Three tests, all of which must pass**

Lines 8 to 10

The username must match, AND the password must match, AND IsActive must be 1. A staff member who has left is switched off with IsActive set to 0, not deleted, so their history stays and their sign-in stops working. One row comes back when all three pass, and none otherwise. GO ends the batch, ready for the next stored procedure.

Press the Copy button under this step.

> **You should see** The button reads Copied for a moment.

Click on the empty line above the line that prints BikeShop is ready, in your BikeShop.sql tab.

> **You should see** The cursor blinks on the empty line, and the line that prints BikeShop is ready is under it.

Press Ctrl and V together to paste usp_Staff_Login.

> **You should see** usp_Staff_Login is in your script, ending with GO, and the line that prints BikeShop is ready is still last.

Select the lines you added, by dragging from CREATE OR ALTER PROCEDURE down to GO.

> **You should see** Every line from CREATE OR ALTER PROCEDURE down to GO is highlighted.

Press F5 to run the highlighted lines.

> **You should see** The Messages pane says: Commands completed successfully. Only the highlighted lines ran.

**IF IT GOES WRONG** The Messages pane shows (12 rows affected) and BikeShop is ready

Nothing was selected, so the whole script ran and the sample data was put back. Nothing is broken. Carry on.

**IF IT GOES WRONG** A red error in the Messages pane

Double-click the error. SQL Server Management Studio (SSMS) jumps to the line it is about. Compare that line with the block above, letter by letter.

# Test the sign-in three times: right, left the shop, and wrong password.

Three tests, one after another. Siobhán Rooney is the manager, with the username siobhan and the password canal01. Liam Doran left in the summer, so IsActive is 0 for him. The third test is the right username with a made-up password.

**Copy this one** **A test that signs in as the manager** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  -- Sign in as Siobhán Rooney, the manager: one row comes back.
2  EXEC usp_Staff_Login @Username = 'siobhan', @Password = 'canal01';
```

**1 Run the stored procedure**

Line 2

EXEC runs the stored procedure with the username siobhan and the password canal01, both inside single quotes because they are text.

**Copy this one** **A test that signs in as someone who left** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  -- Liam Doran left the shop, so IsActive is 0: no row comes back.
2  EXEC usp_Staff_Login @Username = 'liam', @Password = 'canal06';
```

**1 Run the stored procedure**

Line 2

Liam's username and password are right, but his IsActive is 0, so the IsActive test in the WHERE line fails and no row comes back.

**Copy this one** **A test that signs in with the wrong password** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  -- The right username with the wrong password: no row comes back.
2  EXEC usp_Staff_Login @Username = 'siobhan', @Password = 'wrong';
```

**1 Run the stored procedure**

Line 2

This sign-in uses the right username with the wrong password, so the password test in the WHERE line fails and no row comes back.

Press the Copy button under this step.

**You should see** The button reads Copied for a moment.

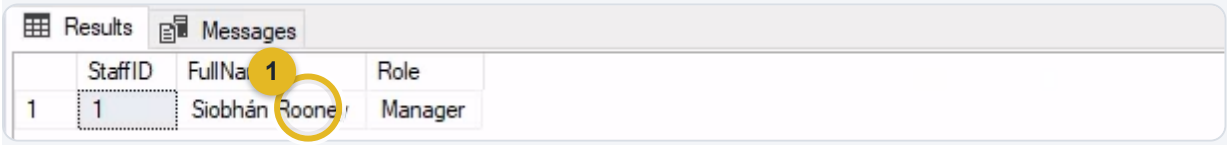
Click in your test window and press Ctrl and A together to select the old test.

**You should see** The whole of the old test is highlighted.

Press Ctrl and V together to paste the new test over the old one.

**You should see** The new test has replaced the old one, and it runs usp_Staff_Login.

Press F5 to run the test.




The right username and password find exactly one row. The program in chapter 5 will use this row to know who signed in.

1 One row: StaffID 1, Siobhán Rooney, Manager.

**You should see** The Results tab shows one row: StaffID 1, Siobhán Rooney, Manager.

**IF IT GOES WRONG** 'Could not find stored procedure'

The database has not got the stored procedure yet, or the test window is talking to a different database. Check that the lines you added ran without a red error. Then look at the box on the toolbar that names the database: if it says master, choose BikeShop, and press F5 again.

Press the Copy button under this step.

> **You should see** The button reads Copied for a moment.

Click in your test window and press Ctrl and A together to select the old test.

> **You should see** The whole of the old test is highlighted.

Press Ctrl and V together to paste the new test over the old one.

> **You should see** The new test has replaced the old one, and it runs usp_Staff_Login.

Press F5 to run the test.

> **You should see** The Results tab shows the three column headings with no row under them. Liam left, so the sign-in is refused.

Press the Copy button under this step.

> **You should see** The button reads Copied for a moment.

Click in your test window and press Ctrl and A together to select the old test.

> **You should see** The whole of the old test is highlighted.

Press Ctrl and V together to paste the new test over the old one.

> **You should see** The new test has replaced the old one, and it runs usp_Staff_Login.

Press F5 to run the test.

> **You should see** The Results tab shows the three column headings with no row under them again. A wrong password gets nothing back.

**IF IT GOES WRONG** Liam Doran comes back as a row

The IsActive = 1 test is missing from the WHERE line, or it reads IsActive = 0. Compare your WHERE line with the block above, select the block, and press F5.

# Read the three Staff stored procedures, and add them.

Three more for the Staff table, and nothing in them is new. Two are Gets: the mechanics who are still working, for the diary in chapter 7, and everyone, for the manager. The third is an Add with a check in it: a username that is already taken is refused, the way the delete refused, with a sentence and a RETURN.

Read this one **usp_Staff_GetMechanics: the mechanics still working** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  CREATE OR ALTER PROCEDURE usp_Staff_GetMechanics
2  AS
3  BEGIN
4      SELECT StaffID, FirstName + ' ' + LastName AS FullName
5      FROM Staff
6      WHERE Role = 'Mechanic' AND IsActive = 1
7      ORDER BY FirstName, LastName;
8  END
9  GO
```

**1 AS and BEGIN**

Lines 1 to 3

There are no parameters at all, because the stored procedure needs nothing to do its work. AS and BEGIN mark where the work starts.

**2 What comes back, and from where**

Lines 4 to 5

This stored procedure sends back the number and the full name from the Staff table.

**3 Only working mechanics, in first-name order**

Lines 6 to 9

The role must be Mechanic and IsActive must be 1, so Liam Doran is left out. First names are put in order, because that is how the diary will show them. GO ends the batch, ready for the next stored procedure.

Read this one **usp_Staff_GetAll: everyone, working or not** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
CREATE OR ALTER PROCEDURE usp_Staff_GetAll
AS
BEGIN
    SELECT StaffID, FirstName + ' ' + LastName AS FullName, Role, Username,
        IsActive
    FROM Staff
    ORDER BY LastName, FirstName;
END
GO
```

**1 AS and BEGIN**

Lines 1 to 3

This stored procedure only reads, so it has no @Message. AS and BEGIN mark where the work starts.

**2 What comes back**

Lines 4 to 5

The SELECT brings back the number, the full name, the role, the username and the IsActive flag. The password is never sent out of the database.

**3 Everyone, in surname order**

Lines 6 to 9

There is no WHERE, so every row comes back, including Liam Doran with IsActive 0. GO ends the batch, ready for the next stored procedure.

Read this one **usp_Staff_Add: add a staff member, unless the username is taken** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
CREATE OR ALTER PROCEDURE usp_Staff_Add
    @FirstName  VARCHAR(40),
    @LastName   VARCHAR(40),
    @Role       VARCHAR(10),
    @Username   VARCHAR(30),
    @Password   VARCHAR(30),
    @NewStaffID INT OUTPUT,
    @Message VARCHAR(200) OUTPUT
AS
BEGIN
    SET @Message = '';
    IF EXISTS (SELECT * FROM Staff WHERE Username = @Username)
    BEGIN
        SET @Message = 'That username is already taken.';
        RETURN;
    END

    INSERT INTO Staff (FirstName, LastName, Role, Username, Password)
    VALUES (@FirstName, @LastName, @Role, @Username, @Password);
    SET @NewStaffID = SCOPE_IDENTITY();
END
GO
```

**1 The name, and what goes in and out**

Lines 1 to 8

Five details go in: the two names, the role, the username and the password. The new number and @Message come back out.

**2 Start with a blank message**

Lines 9 to 11

AS and BEGIN mark where the work starts. The first line of work makes @Message blank. A blank message is the sign that everything worked, so it must be blank before any check can change it.

**3 Refuse a username that is taken**

Lines 12 to 16

If any row already has this username, @Message gets a sentence and RETURN stops the stored procedure, so the INSERT below never runs.

**4 Add the row and hand back the number**

Lines 18 to 22

The same INSERT and SCOPE_IDENTITY() as the customer Add, with five columns instead of four. GO ends the batch, ready for the next stored procedure.

Press the Copy button under this step.

> **You should see** The button reads Copied for a moment.

Click on the empty line above the line that prints BikeShop is ready, in your BikeShop.sql tab.

> **You should see** The cursor blinks on the empty line, and the line that prints BikeShop is ready is under it.

Press Ctrl and V together to paste usp_Staff_GetMechanics.

> **You should see** usp_Staff_GetMechanics is in your script, ending with GO, and the line that prints BikeShop is ready is still last.

Press the Copy button under this step.

> **You should see** The button reads Copied for a moment.

Click on the empty line above the line that prints BikeShop is ready, in your BikeShop.sql tab.

> **You should see** The cursor blinks on the empty line, and the line that prints BikeShop is ready is under it.

Press Ctrl and V together to paste usp_Staff_GetAll.

> **You should see** usp_Staff_GetAll is in your script, under usp_Staff_GetMechanics.

Press the Copy button under this step.

> **You should see** The button reads Copied for a moment.

Click on the empty line above the line that prints BikeShop is ready, in your BikeShop.sql tab.

> **You should see** The cursor blinks on the empty line, and the line that prints BikeShop is ready is under it.

Press Ctrl and V together to paste usp_Staff_Add.

> **You should see** All three are in your script, each ending with GO, and the line that prints BikeShop is ready is still last.

Select the three stored procedures you added, by dragging from the first CREATE OR ALTER PROCEDURE down to the last GO.

> **You should see** Every line from CREATE OR ALTER PROCEDURE down to GO is highlighted.

Press F5 to run the highlighted lines.

**You should see** The Messages pane says: Commands completed successfully. Only the highlighted lines ran.

**IF IT GOES WRONG** The Messages pane shows (12 rows affected) and BikeShop is ready

Nothing was selected, so the whole script ran and the sample data was put back. Nothing is broken. Carry on.

**IF IT GOES WRONG** A red error in the Messages pane

Double-click the error. SQL Server Management Studio (SSMS) jumps to the line it is about. Compare that line with the block above, letter by letter.

**Copy this one** **A test that lists the mechanics, then everyone** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
-- The mechanics still working (three rows), then everyone (six rows).
EXEC usp_Staff_GetMechanics;
EXEC usp_Staff_GetAll;
```

**1 The working mechanics**

Line 2

There are no parameters, so nothing follows the name. Three rows come back.

**2 Everyone**

Line 3

Two EXECs in one test give two grids under Results, one above the other. Six rows come back in the second.

Copy this one **A test that adds Eoin Daly, then tries Aoife's username** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
-- Add a new counter worker, then try to add him again with Aoife's
-- username: the second one is refused.
DECLARE @NewID INT, @Message VARCHAR(200);

EXEC usp_Staff_Add
     @FirstName = 'Eoin', @LastName = 'Daly', @Role = 'Counter',
     @Username = 'eoin', @Password = 'canal07',
     @NewStaffID = @NewID OUTPUT, @Message = @Message OUTPUT;

IF @Message = ''
BEGIN
    PRINT 'Added staff number ' + CAST(@NewID AS VARCHAR) + '.';
END
ELSE
BEGIN
    PRINT 'Refused: ' + @Message;
END

EXEC usp_Staff_Add
     @FirstName = 'Eoin', @LastName = 'Daly', @Role = 'Counter',
     @Username = 'aoife', @Password = 'canal07',
     @NewStaffID = @NewID OUTPUT, @Message = @Message OUTPUT;

IF @Message = ''
BEGIN
    PRINT 'Added staff number ' + CAST(@NewID AS VARCHAR) + '.';
END
ELSE
BEGIN
    PRINT 'Refused: ' + @Message;
END
```

**1 Two boxes to hold the answers**

Line 3

DECLARE makes a variable. A variable is a named box that holds one value while the script runs. @NewID will hold the number the database gives the new row, and @Message the refusal, if there is one.

**2 Add Eoin Daly at the counter**

Lines 5 to 8

The first EXEC adds a new counter worker with the username eoin. His new number comes back in @NewID.

**3 If the message is blank, it worked**

Lines 10 to 13

A blank message means he was added, and one line prints his number.

**4 Otherwise, print the refusal**

Lines 14 to 17

If the stored procedure set @Message, something was refused, and the sentence it wrote is printed as it is.

**5 The same person with a taken username**

Lines 19 to 22

The second EXEC tries Eoin again, this time with the username aoife, which Aoife Byrne already has. The IF EXISTS inside the stored procedure finds her row.

**6 If the message is blank, the rule is broken**

Lines 24 to 27

A blank message here would mean two staff members with the same username, which must never happen.

**7 Otherwise, print the refusal**

Lines 28 to 31

The refusal is printed: That username is already taken.

Press the Copy button under this step.

**You should see** The button reads Copied for a moment.

Click in your test window and press Ctrl and A together to select the old test.

**You should see** The whole of the old test is highlighted.

Press Ctrl and V together to paste the new test over the old one.

**You should see** The new test has replaced the old one, and it runs usp_Staff_GetMechanics, then usp_Staff_GetAll.

Press F5 to run the test.

**You should see** The Results tab shows two grids. The first has three rows: Aoife Byrne, Conor Walsh and Niamh Hughes. The second has six, from Aoife Byrne to Conor Walsh in surname order, with Liam Doran showing IsActive 0.

**IF IT GOES WRONG** 'Could not find stored procedure'

The database has not got the stored procedure yet, or the test window is talking to a different database. Check that the lines you added ran without a red error. Then look at the box on the toolbar that names the database: if it says master, choose BikeShop, and press F5 again.

Press the Copy button under this step.

> **You should see** The button reads Copied for a moment.

Click in your test window and press Ctrl and A together to select the old test.

> **You should see** The whole of the old test is highlighted.

Press Ctrl and V together to paste the new test over the old one.

> **You should see** The new test has replaced the old one, and it runs usp_Staff_Add twice.

Press F5 to run the test.

> **You should see** The Messages tab says: Added staff number 7. Then: Refused: That username is already taken.

**IF IT GOES WRONG** Both lines say Refused: That username is already taken

The test ran twice, so eoin was taken by the first run. That is the rule working. Click in BikeShop.sql and press F5 with nothing selected to put the sample data back.

# Type the whole of the stored procedure that lists the services.

Every stored procedure so far was pasted. This one you type, every line, because it is short and because the whole shape should be in your fingers before you write your own. It lists the services the shop offers, shortest first, and makes one extra column called Display: the name, the minutes and the price in one piece of text, which the diary in chapter 7 will show in a drop-down list.

**Type this one** **usp_ServiceType_GetAll: every service, with a Display column** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  CREATE OR ALTER PROCEDURE usp_ServiceType_GetAll
2  AS
3  BEGIN
4      SELECT ServiceTypeID, Name, DurationMinutes, Price,
5             Name + ' (' + CAST(DurationMinutes AS VARCHAR(5)) + ' min, £'
6                 + CAST(Price AS VARCHAR) + ')' AS Display
7      FROM ServiceType
8      ORDER BY DurationMinutes, Name;
9  END
10 GO
```

**1 AS and BEGIN**

Lines 1 to 3

There are no parameters, because every service is wanted. AS and BEGIN mark where the work starts.

**2 What comes back**

Lines 4 to 6

The SELECT brings back the four columns of the table, then Display, which is built from three of them. CAST turns a number into text so that it can be joined on with plus signs. VARCHAR(5) is room for up to five characters of minutes. The pound sign is typed as it is, inside the quotes.

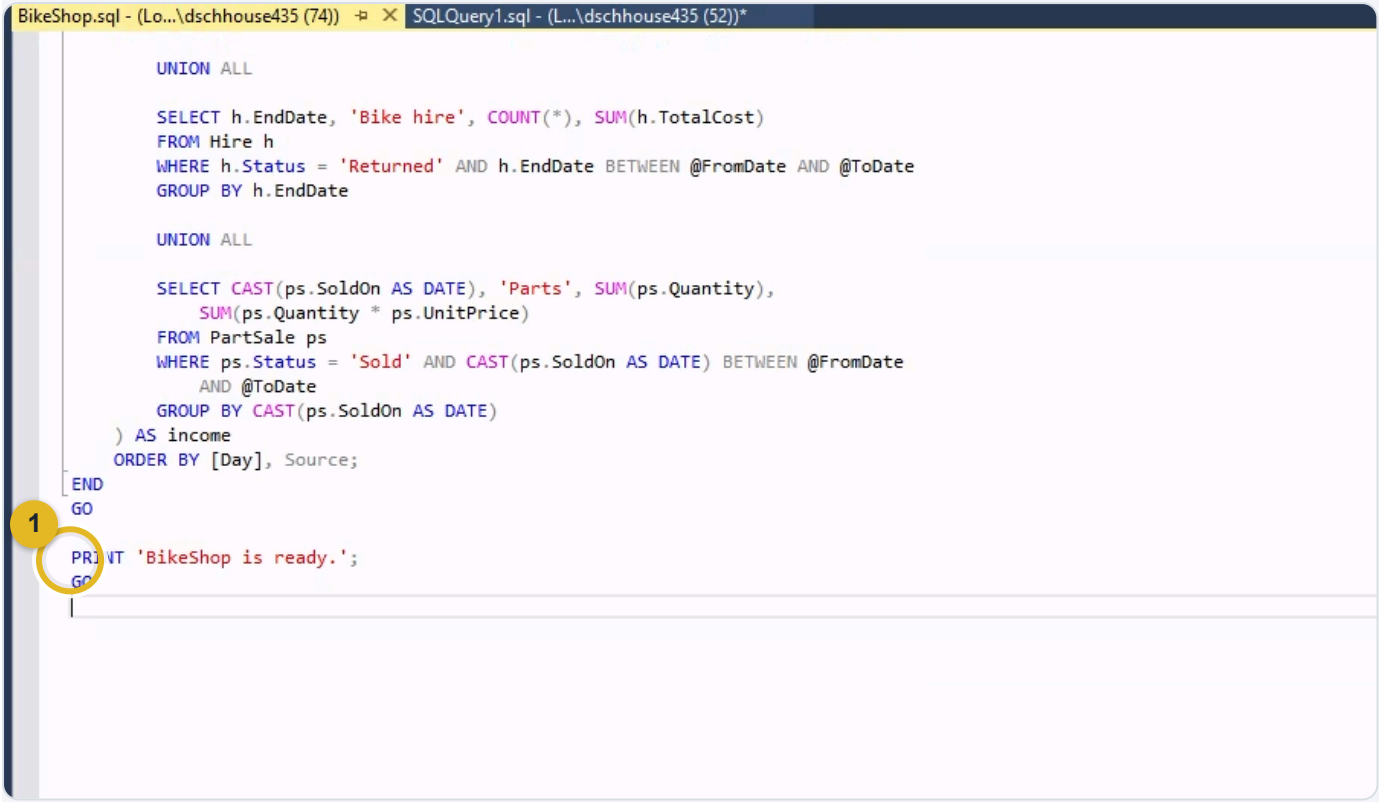
**3 Every service, shortest first**

Lines 7 to 10

There is no WHERE, so every row comes back. ORDER BY puts the shortest services first, and two with the same minutes in name order. GO ends the batch, ready for the next stored procedure.

**Why you type it** This is the first stored procedure that is wholly yours. Typing every line, from the name to the GO, puts the shape of a Get into your fingers for the two you write next.

Click on the empty line above the line that prints BikeShop is ready, in your BikeShop.sql tab.




The end of your script. Every stored procedure in this chapter goes on the empty line above the line that prints BikeShop is ready, one after another, each ending with GO.

1. Click on the empty line above the line that prints BikeShop is ready, so the cursor blinks there.

> **You should see** The cursor blinks on the empty line, and the line that prints BikeShop is ready is under it.

Type the whole block above, every line from CREATE OR ALTER PROCEDURE down to GO.

> **You should see** Your own usp_ServiceType_GetAll is in your script, ending with GO, and the line that prints BikeShop is ready is still last.

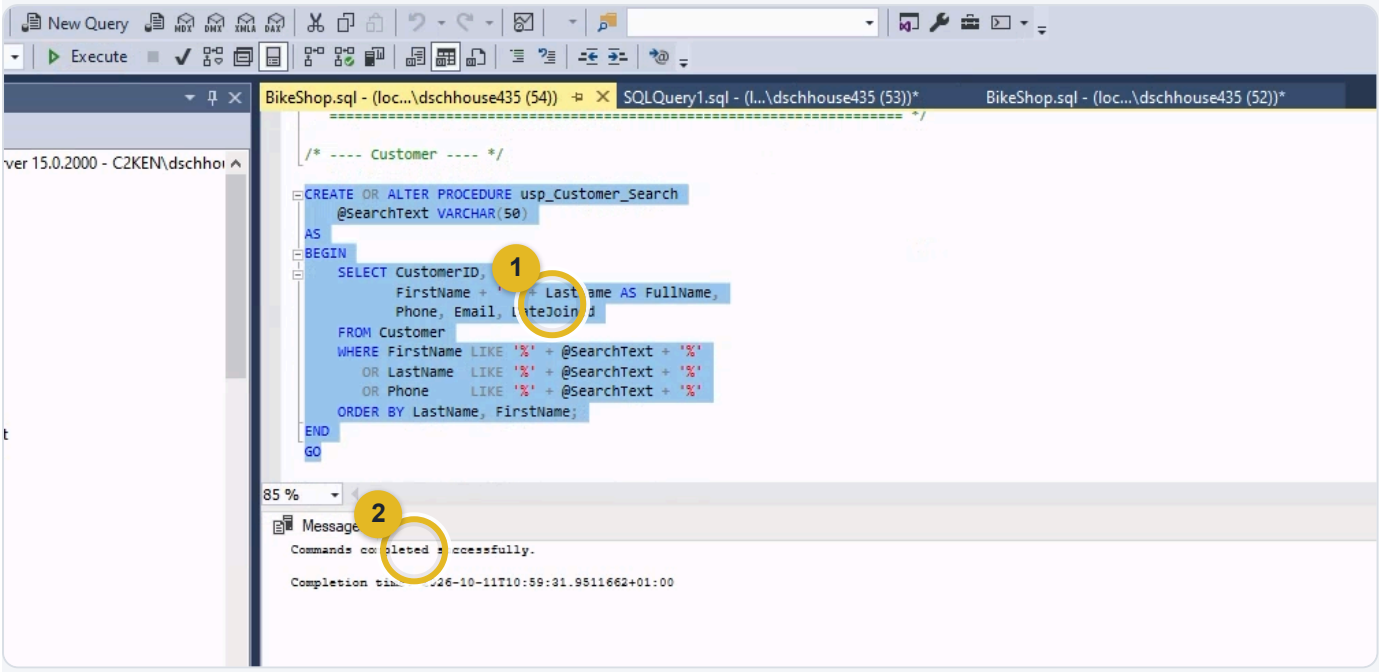
**IF IT GOES WRONG** A red wavy line under a word

A spelling slip. Compare your line with the yellow line, letter by letter, and check that each line ends the way the yellow line ends.

**IF IT GOES WRONG** 'Must declare the scalar variable …'

A name beginning with @ is spelled differently from the line that declares it. Make the two match.

Select the lines you added, by dragging from CREATE OR ALTER PROCEDURE down to GO.




Select the lines you added, then press F5. Only the highlighted lines run.

1. Drag from CREATE OR ALTER PROCEDURE down to GO, so the whole block is highlighted.

2. The Messages pane says: Commands completed successfully.

> **You should see** Every line from CREATE OR ALTER PROCEDURE down to GO is highlighted.

Press F5 to run the highlighted lines.

> **You should see** The Messages pane says: Commands completed successfully. Only the highlighted lines ran.

**IF IT GOES WRONG** The Messages pane shows (12 rows affected) and BikeShop is ready

Nothing was selected, so the whole script ran and the sample data was put back. Nothing is broken. Carry on.

**IF IT GOES WRONG** A red error in the Messages pane

Double-click the error. SQL Server Management Studio (SSMS) jumps to the line it is about. Compare that line with the block above, letter by letter.

Copy this one **A test that lists every service** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  -- Every service the shop offers, shortest first: seven rows.
2  EXEC usp_ServiceType_GetAll;
```

**1 Run the stored procedure**

Line 2

There are no parameters, so nothing follows the name. Seven rows come back.

Press the Copy button under this step.

> **You should see** The button reads Copied for a moment.

Click in your test window and press Ctrl and A together to select the old test.

> **You should see** The whole of the old test is highlighted.

Press Ctrl and V together to paste the new test over the old one.

> **You should see** The new test has replaced the old one, and it runs usp_ServiceType_GetAll.

Press F5 to run the test.

Results | Messages

| | ServiceTypeID | Na | DurationMinutes | Price | Display |
|---|---|---|---|---|---|
| 1 | 1 | Punc ure rep air | 30 | 15.00 | Puncture repair (30 mi , £15.00) |
| 2 | 2 | Safety | 30 | 0.00 | Safety check (3 , £0.00) |
| 3 | 5 | Brake pad replacement | 45 | 28.00 | Brake pad replacement (45 min, £28.00) |
| 4 | 3 | Gear tune-up | 45 | 25.00 | Gear tune-up (45 min, £25.00) |
| 5 | 4 | Wheel true | 45 | 20.00 | Wheel true (45 min, £20.00) |
| 6 | 6 | Basic service | 60 | 40.00 | Basic service (60 min, £40.00) |
| 7 | 7 | Full service | 120 | 85.00 | Full service (120 min, £85.00) |

Every service, shortest first. The Display column is the text the program will show in a drop-down list in chapter 7.

1. Puncture repair and Safety check both take 30 minutes, so they are put in name order.

2. Display: the name, the minutes and the price in one piece of text.

**You should see** The Results tab shows seven rows, Puncture repair first and Full service last, with a Display column such as Basic service (60 min, £40.00).

**IF IT GOES WRONG** 'Could not find stored procedure'

The database has not got the stored procedure yet, or the test window is talking to a different database. Check that the lines you added ran without a red error. Then look at the box on the toolbar that names the database: if it says master, choose BikeShop, and press F5 again.

**IF IT GOES WRONG** 'Incorrect syntax near'

A bracket, a quote or a plus sign is missing in the Display line. Count the opening and closing brackets in your line against the block above, and check that each piece of text is inside single quotes.

# Type the lines that make the service Add different from the customer Add.

The Add for a service is the customer Add with different ingredients. The parameters change, and the INSERT names a different table and different columns. The middle is the same: a blank message, an INSERT, and SCOPE_IDENTITY() handed back. You type the lines that differ.

**Type this one** **usp_ServiceType_Add: put one service in** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  CREATE OR ALTER PROCEDURE usp_ServiceType_Add
2      @Name             VARCHAR(40),
3      @DurationMinutes  INT,
4      @Price            DECIMAL(8,2),
5      @NewServiceTypeID INT OUTPUT,
6      @Message VARCHAR(200) OUTPUT
7  AS
8  BEGIN
9      SET @Message = '';
10     INSERT INTO ServiceType (Name, DurationMinutes, Price)
11     VALUES (@Name, @DurationMinutes, @Price);
12     SET @NewServiceTypeID = SCOPE_IDENTITY();
13 END
14 GO
```

**1 The name, and what goes in and out**

Lines 1 to 6

Three details go in: the name, the minutes it takes, and the price. DECIMAL(8,2) is a number with two places after the point, which is how the table keeps money. The new number and @Message come back out.

**2 Start with a blank message**

Lines 7 to 9

AS and BEGIN mark where the work starts. The first line of work makes @Message blank. A blank message is the sign that everything worked, so it must be blank before any check can change it.

**3 Add the row and hand back the number**

Lines 10 to 14

INSERT INTO names the ServiceType table and its three columns, VALUES gives the three parameters in the same order, and SCOPE_IDENTITY() hands back the number the database chose. GO ends the batch, ready for the next stored procedure.

**Why you type it** The parameters and the INSERT are the two places an Add changes from table to table. Typing them here is the practice for writing the supplier Add on your own.

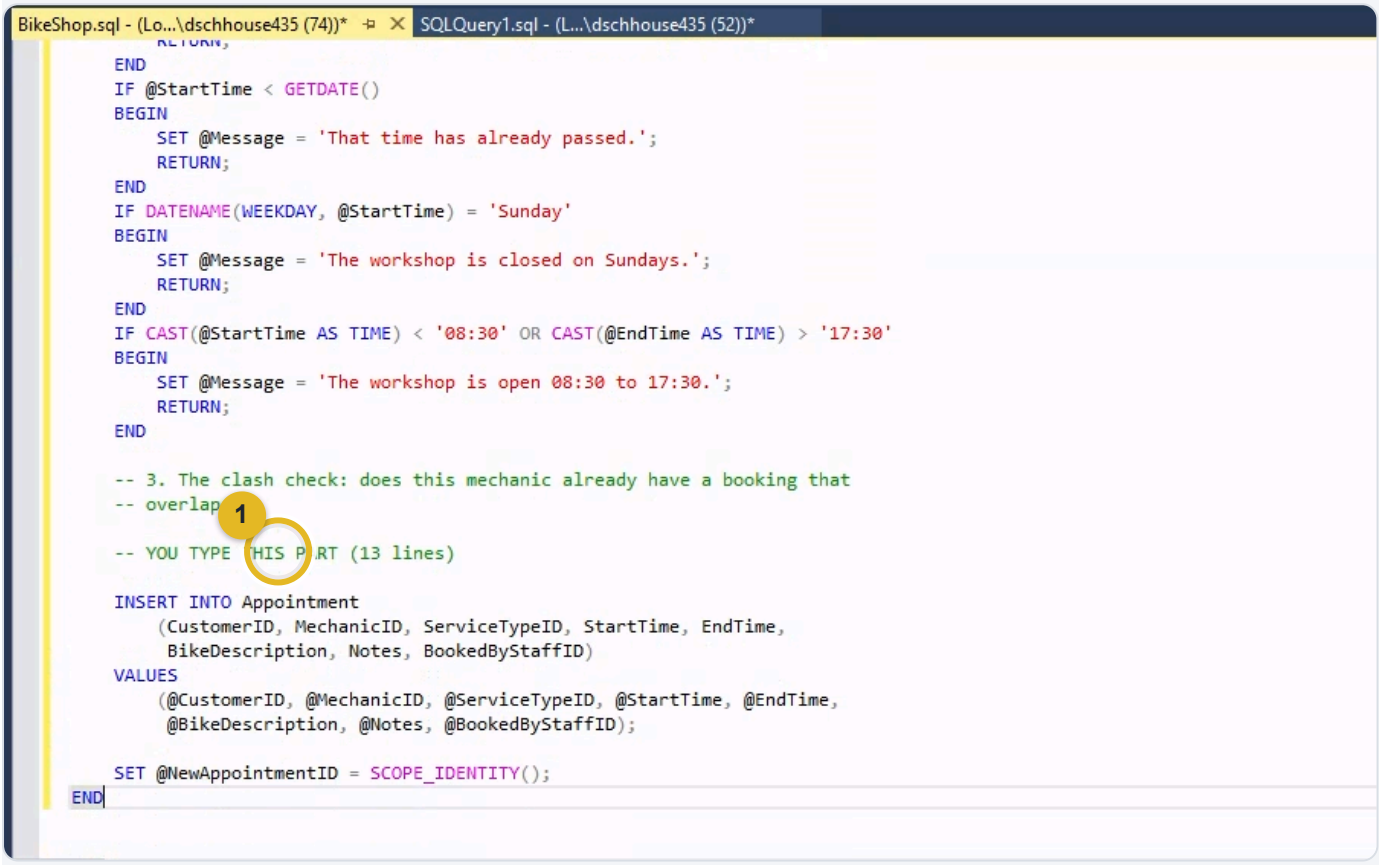
Press the Copy button under this step.

> **You should see** The button reads Copied for a moment.

Click on the empty line above the line that prints BikeShop is ready, in your BikeShop.sql tab.

> **You should see** The cursor blinks on the empty line, and the line that prints BikeShop is ready is under it.

Press Ctrl and V together to paste usp_ServiceType_Add.




After the paste. Everything is in except the yellow lines. Each marker line shows where a yellow block goes.

1. Click on the marker line, type the yellow lines in its place, and delete the marker.

> **You should see** The stored procedure is in your script, with a line that says YOU TYPE THIS PART where each yellow block goes.

Type the yellow lines in place of each line that says YOU TYPE THIS PART, so that no marker line is left.

> **You should see** The first gap now holds four parameter lines, the second holds three lines from INSERT to SCOPE_IDENTITY(), and no marker line is left.

**IF IT GOES WRONG** A red wavy line under a word

A spelling slip. Compare your line with the yellow line, letter by letter, and check that each line ends the way the yellow line ends.

**IF IT GOES WRONG** 'Must declare the scalar variable …'

A name beginning with @ is spelled differently from the line that declares it. Make the two match.

Select the lines you added, by dragging from CREATE OR ALTER PROCEDURE down to GO.

> **You should see** Every line from CREATE OR ALTER PROCEDURE down to GO is highlighted.

Press F5 to run the highlighted lines.

> **You should see** The Messages pane says: Commands completed successfully. Only the highlighted lines ran.

**IF IT GOES WRONG** The Messages pane shows (12 rows affected) and BikeShop is ready

Nothing was selected, so the whole script ran and the sample data was put back. Nothing is broken. Carry on.

**IF IT GOES WRONG** A red error in the Messages pane

Double-click the error. SQL Server Management Studio (SSMS) jumps to the line it is about. Compare that line with the block above, letter by letter.

Copy this one **A test that adds a Chain replacement service** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
-- Add a new service, print its number, then list every service again.
DECLARE @NewID INT, @Message VARCHAR(200);

EXEC usp_ServiceType_Add
     @Name = 'Chain replacement', @DurationMinutes = 30, @Price = 18.00,
     @NewServiceTypeID = @NewID OUTPUT, @Message = @Message OUTPUT;

IF @Message = ''
BEGIN
    PRINT 'Added. New service number ' + CAST(@NewID AS VARCHAR) + '.';
END
ELSE
BEGIN
    PRINT @Message;
END

EXEC usp_ServiceType_GetAll;
```

**1 Two boxes to hold the answers**

Line 2

DECLARE makes a variable. A variable is a named box that holds one value while the script runs. @NewID will hold the number the database gives the new row, and @Message the refusal, if there is one.

**2 Run the stored procedure**

Lines 4 to 6

The EXEC adds a 30-minute Chain replacement at 18.00. The price is typed as a plain number, with no pound sign and no quotes.

**3 If the message is blank, it worked**

Lines 8 to 11

A blank message means the row was saved. PRINT writes one line in the Messages pane. CAST turns the number into text so that it can be joined onto the sentence with the plus signs.

**4 Otherwise, print the refusal**

Lines 12 to 15

If the stored procedure set @Message, something was refused, and the sentence it wrote is printed as it is.

**5 List every service again**

Line 17

The stored procedure you typed on the last screen shows the new row among the others.

Press the Copy button under this step.

> **You should see** The button reads Copied for a moment.

Click in your test window and press Ctrl and A together to select the old test.

> **You should see** The whole of the old test is highlighted.

Press Ctrl and V together to paste the new test over the old one.

> **You should see** The new test has replaced the old one, and it runs usp_ServiceType_Add.

Press F5 to run the test.

> **You should see** The Results tab shows eight rows, with Chain replacement first, because it takes 30 minutes and C comes before P. Click the Messages tab. It says: Added. New service number 8.

**IF IT GOES WRONG** 'Could not find stored procedure'

The database has not got the stored procedure yet, or the test window is talking to a different database. Check that the lines you added ran without a red error. Then look at the box on the toolbar that names the database: if it says master, choose BikeShop, and press F5 again.

# Your turn: write the stored procedure that changes a service.

You have the customer Update from stage 3 and the service Add from the last screen. The service Update is the two put together: the service's number and its three details go in, @Message comes back out, and an UPDATE with a WHERE changes the one row. Write it in your script, on the empty line above the line that prints BikeShop is ready, select it and press F5 as before, then test it on the next screen.

Your turn **usp_ServiceType_Update: change one service** SQL

1. In SQL Server Management Studio (SSMS), click on the empty line above the line that prints BikeShop is ready and type the first line: CREATE OR ALTER PROCEDURE usp_ServiceType_Update.
2. Type five parameters, one per line with a comma after each but the last: @ServiceTypeID INT, then @Name VARCHAR(40), @DurationMinutes INT and @Price DECIMAL(8,2) as in the Add, then @Message VARCHAR(200) OUTPUT.
3. Type AS, BEGIN, and the line that makes the message blank: SET @Message = '';
4. Type UPDATE ServiceType, then SET with the three columns given their parameters, separated by commas, then WHERE ServiceTypeID = @ServiceTypeID; and then END and GO.
5. Select the lines you typed, from CREATE OR ALTER PROCEDURE down to GO, and press F5.

**THE TEST** The ready-made test on the next screen changes the price of Wheel true, which is service 4, to 22.00.

**YOU SHOULD SEE** The Messages tab says Updated service number 4, and the list shows Wheel true at 22.00 with every other price unchanged. If every price has become 22.00, the WHERE line is missing.

**HINT** Open usp_Customer_Update in your script beside you and change one name at a time: the table, the columns and the parameters. The shape stays the same.

**Stuck? Show me the finished procedure**

Read it, then write yours **usp_ServiceType_Update** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  CREATE OR ALTER PROCEDURE usp_ServiceType_Update
2      @ServiceTypeID   INT,
3      @Name            VARCHAR(40),
4      @DurationMinutes INT,
5      @Price           DECIMAL(8,2),
6      @Message VARCHAR(200) OUTPUT
7  AS
8  BEGIN
9      SET @Message = '';
10     UPDATE ServiceType
11     SET Name = @Name, DurationMinutes = @DurationMinutes, Price = @Price
12     WHERE ServiceTypeID = @ServiceTypeID;
13 END
14 GO
```

**1 The name, and what goes in and out**

Lines 1 to 6

The service's number goes in first, then the same three details as the Add, and @Message comes back out.

**2 Start with a blank message**

Lines 7 to 9

AS and BEGIN mark where the work starts. The first line of work makes @Message blank. A blank message is the sign that everything worked, so it must be blank before any check can change it.

**3 Change the one row**

Lines 10 to 14

SET gives each of the three columns its new value from the matching parameter, and WHERE picks the one row by number. GO ends the batch, ready for the next stored procedure.

# Test your service Update, and compare it with the finished one.

Copy this one **A test that changes the price of Wheel true** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
-- Put the price of Wheel true (service 4) up to 22.00, then list every
-- service again.
DECLARE @Message VARCHAR(200);

EXEC usp_ServiceType_Update
     @ServiceTypeID = 4, @Name = 'Wheel true', @DurationMinutes = 45,
     @Price = 22.00, @Message = @Message OUTPUT;

IF @Message = ''
BEGIN
    PRINT 'Updated service number 4.';
END
ELSE
BEGIN
    PRINT @Message;
END

EXEC usp_ServiceType_GetAll;
```

**1 One box for the message**

Line 3

DECLARE makes a variable, a named box that holds one value while the script runs. @Message will hold the refusal, if there is one.

**2 Run the stored procedure**

Lines 5 to 7

Service 4 is Wheel true. Its name and minutes are sent again unchanged, and the price is new.

**3 If the message is blank, it worked**

Lines 9 to 12

A blank message means the row was changed, and one line says so.

**4 Otherwise, print the refusal**

Lines 13 to 16

If the stored procedure set @Message, something was refused, and the sentence it wrote is printed as it is.

**5 List every service again**

Line 18

Wheel true shows its new price among the others.

Press the Copy button under this step.

> **You should see** The button reads Copied for a moment.

Click in your test window and press Ctrl and A together to select the old test.

> **You should see** The whole of the old test is highlighted.

Press Ctrl and V together to paste the new test over the old one.

> **You should see** The new test has replaced the old one, and it runs usp_ServiceType_Update.

Press F5 to run the test.

> **You should see** The list shows Wheel true at 22.00, with Display reading Wheel true (45 min, £22.00), and every other price as it was. Click the Messages tab. It says: Updated service number 4.

**IF IT GOES WRONG** 'Could not find stored procedure'

The database has not got the stored procedure yet, or the test window is talking to a different database. Check that the lines you added ran without a red error. Then look at the box on the toolbar that names the database: if it says master, choose BikeShop, and press F5 again.

This is the UPDATE part of the finished stored procedure, to hold beside yours. Yours may have different spacing, and that is fine, as long as the test passes.

**Read this one** **The finished usp_ServiceType_Update, from UPDATE to the WHERE line** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  UPDATE ServiceType
2  SET Name = @Name, DurationMinutes = @DurationMinutes, Price = @Price
3  WHERE ServiceTypeID = @ServiceTypeID;
```

**1 The UPDATE with its WHERE**

Lines 1 to 3

The UPDATE sets three columns from three parameters, and the WHERE picks one row.

**IF IT GOES WRONG** Every service now costs 22.00

The WHERE line is missing from your UPDATE. Put it in, select the block, press F5, then click in BikeShop.sql and press F5 with nothing selected to put the sample data back.

**IF IT GOES WRONG** 'Procedure or function usp_ServiceType_Update has too many arguments specified'

The test sends five parameters and yours takes fewer. Check that all five are in the list at the top, each with a comma after it except the last.

## Your turn: write the two Supplier stored procedures.

The Supplier table has three columns besides its number: Name, Phone and Email. Write usp_Supplier_GetAll, which lists every supplier in name order, and usp_Supplier_Add, which puts one in and hands back the new number. You have written both kinds already, for other tables. Name the parameters @Name, @Phone and @Email, with @NewSupplierID and @Message as the two OUTPUT ones. Write both in your script, one under the other, on the empty line above the line that prints BikeShop is ready. Then select both and press F5. You will test them on the next screen.

Your turn **usp_Supplier_GetAll: every supplier, in name order** SQL

1. In SQL Server Management Studio (SSMS), click on the empty line above the line that prints BikeShop is ready and type the first line: CREATE OR ALTER PROCEDURE usp_Supplier_GetAll, with no parameters.
2. Type AS and BEGIN.
3. Type a SELECT of the four columns: SupplierID, Name, Phone, Email, then FROM Supplier, then ORDER BY Name; and then END and GO.
4. Select the lines you typed, from CREATE OR ALTER PROCEDURE down to GO, and press F5.

THE TEST The ready-made test on the next screen lists every supplier.

YOU SHOULD SEE Three rows: Lagan Bike Parts, Mourne Cycle Supplies and Northern Components Ltd, in that order.

HINT Open usp_Staff_GetAll in your script beside you. The shape is the same with no WHERE, four plain columns, and one column to order by.

▾ **Stuck? Show me the finished procedure**

Read it, then write yours **usp_Supplier_GetAll** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
CREATE OR ALTER PROCEDURE usp_Supplier_GetAll
AS
BEGIN
    SELECT SupplierID, Name, Phone, Email
    FROM Supplier
    ORDER BY Name;
END
GO
```

**1 AS and BEGIN**

Lines 1 to 3

There are no parameters, because every supplier is wanted. AS and BEGIN mark where the work starts.

**2 Every supplier, in name order**

Lines 4 to 8

The SELECT brings back the four columns from the Supplier table, with no WHERE, in order of name. GO ends the batch, ready for the next stored procedure.

Your turn **usp_Supplier_Add: put one supplier in** SQL

1. Under the GO of usp_Supplier_GetAll, type the first line: CREATE OR ALTER PROCEDURE usp_Supplier_Add.
2. Type five parameters, one per line with a comma after each but the last: @Name VARCHAR(60), @Phone VARCHAR(15), @Email VARCHAR(100), @NewSupplierID INT OUTPUT, and @Message VARCHAR(200) OUTPUT.
3. Type AS, BEGIN, and the line that makes the message blank: SET @Message = '';
4. Type INSERT INTO Supplier (Name, Phone, Email) and then VALUES (@Name, @Phone, @Email); on the next line.
5. Type SET @NewSupplierID = SCOPE_IDENTITY(); and then END and GO.
6. Select the lines you typed, from CREATE OR ALTER PROCEDURE down to GO, and press F5.

**THE TEST** The ready-made test on the next screen adds Carlingford Cycles and lists every supplier again.

**YOU SHOULD SEE** The Messages tab says Added. New supplier number 4, and the list has four rows with Carlingford Cycles first.

**HINT** Open usp_ServiceType_Add in your script beside you and change one name at a time: the table, the three columns, the three parameters, and the name of the OUTPUT number.

**Stuck? Show me the finished procedure**

Read it, then write yours **usp_Supplier_Add** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
CREATE OR ALTER PROCEDURE usp_Supplier_Add
    @Name          VARCHAR(60),
    @Phone         VARCHAR(15),
    @Email         VARCHAR(100),
    @NewSupplierID INT OUTPUT,
    @Message VARCHAR(200) OUTPUT
AS
BEGIN
    SET @Message = '';
    INSERT INTO Supplier (Name, Phone, Email)
    VALUES (@Name, @Phone, @Email);
    SET @NewSupplierID = SCOPE_IDENTITY();
END
GO
```

### 1 The name, and what goes in and out

Lines 1 to 6

The three details go in, with the same sizes as their columns in the table. The new number and @Message come back out.

### 2 Start with a blank message

Lines 7 to 9

AS and BEGIN mark where the work starts. The first line of work makes @Message blank. A blank message is the sign that everything worked, so it must be blank before any check can change it.

### 3 Add the row and hand back the number

Lines 10 to 14

INSERT INTO names the table and its three columns, VALUES gives the three parameters, and SCOPE_IDENTITY() hands back the number the database chose. GO ends the batch, ready for the next stored procedure.

# Test your Supplier stored procedures, and compare them with the finished ones.

**Copy this one** **A test that lists every supplier** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  -- Every supplier, in name order: three rows.
2  EXEC usp_Supplier_GetAll;
```

**1 Run the stored procedure**

Line 2

There are no parameters, so nothing follows the name. Three rows come back.

Copy this one **A test that adds Carlingford Cycles and lists every supplier again** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
-- Add a new supplier, print its number, then list every supplier again.
DECLARE @NewID INT, @Message VARCHAR(200);

EXEC usp_Supplier_Add
     @Name = 'Carlingford Cycles', @Phone = '028 4175 3030',
     @Email = 'hello@carlingfordcycles.example',
     @NewSupplierID = @NewID OUTPUT, @Message = @Message OUTPUT;

IF @Message = ''
BEGIN
    PRINT 'Added. New supplier number ' + CAST(@NewID AS VARCHAR) + '.';
END
ELSE
BEGIN
    PRINT @Message;
END

EXEC usp_Supplier_GetAll;
```

**1 Two boxes to hold the answers**

Line 2

DECLARE makes a variable. A variable is a named box that holds one value while the script runs. @NewID will hold the number the database gives the new row, and @Message the refusal, if there is one.

**2 Run the stored procedure**

Lines 4 to 7

The EXEC adds a new supplier with a name, a phone and an email. The new number comes back in @NewID.

**3 If the message is blank, it worked**

Lines 9 to 12

A blank message means the row was saved. PRINT writes one line in the Messages pane. CAST turns the number into text so that it can be joined onto the sentence with the plus signs.

**4 Otherwise, print the refusal**

Lines 13 to 16

If the stored procedure set @Message, something was refused, and the sentence it wrote is printed as it is.

**5 List every supplier again**

Line 18

Your own Get shows the row that your own Add put in.

Press the Copy button under this step.

> **You should see** The button reads Copied for a moment.

Click in your test window and press Ctrl and A together to select the old test.

> **You should see** The whole of the old test is highlighted.

Press Ctrl and V together to paste the new test over the old one.

> **You should see** The new test has replaced the old one, and it runs usp_Supplier_GetAll.

Press F5 to run the test.

> **You should see** The Results tab shows three rows: Lagan Bike Parts, Mourne Cycle Supplies and Northern Components Ltd.

**IF IT GOES WRONG** 'Could not find stored procedure'

The database has not got the stored procedure yet, or the test window is talking to a different database. Check that the lines you added ran without a red error. Then look at the box on the toolbar that names the database: if it says master, choose BikeShop, and press F5 again.

Press the Copy button under this step.

> **You should see** The button reads Copied for a moment.

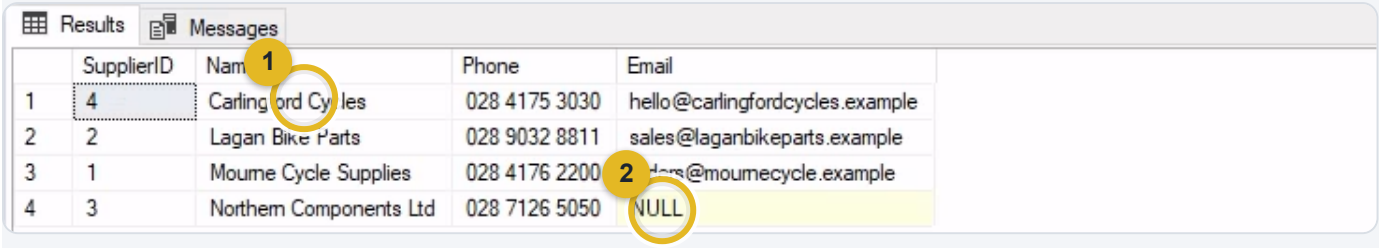
Click in your test window and press Ctrl and A together to select the old test.

> **You should see** The whole of the old test is highlighted.

Press Ctrl and V together to paste the new test over the old one.

> **You should see** The new test has replaced the old one, and it runs usp_Supplier_Add.

Press F5 to run the test.




Your own stored procedures at work: the supplier you added is in the list, in name order, with the number 4.

1 Carlingford Cycles, number 4, first because C comes before L.

2 Northern Components Ltd has NULL for Email.

**You should see** The Results tab shows four rows, with Carlingford Cycles first and the number 4. Click the Messages tab. It says: Added. New supplier number 4.

These are the key parts of the two finished stored procedures, to hold beside yours.

**Read this one** **The finished usp_Supplier_GetAll, from SELECT to ORDER BY** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
SELECT SupplierID, Name, Phone, Email
FROM Supplier
ORDER BY Name;
```

**1 The SELECT**

Lines 1 to 3

The SELECT lists four columns from the Supplier table, in order of name.

**Read this one** **The finished usp_Supplier_Add, from INSERT to SCOPE_IDENTITY()** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
INSERT INTO Supplier (Name, Phone, Email)
VALUES (@Name, @Phone, @Email);
SET @NewSupplierID = SCOPE_IDENTITY();
```

**1 The INSERT and the number handed back**

Lines 1 to 3

The INSERT puts three parameters into three columns, and the new number is put into the OUTPUT parameter.

**IF IT GOES WRONG** 'Procedure or function usp_Supplier_Add has too many arguments specified'

The test sends five parameters and yours takes fewer. Check that all five are in the list at the top, each with a comma after it except the last.

**IF IT GOES WRONG** The Messages tab says Added. New supplier number 4, but the list has three rows

The Results tab is still showing the list from an earlier test, or the EXEC usp_Supplier_GetAll line at the end of the add test is missing. Click in BikeShop.sql and press F5 with nothing selected to put the sample data back. Then paste the add test again, check that it ends with EXEC usp_Supplier_GetAll;, and press F5. Running the add test a second time without putting the data back would try to add Carlingford Cycles twice, and the table refuses a name it already has.

**IF IT GOES WRONG** Carlingford Cycles is last instead of first

The ORDER BY Name line is missing from your Get, so the rows come in the order they were added. Put it in, select the block, and press F5. Then run the suppliers test, the one that only lists, not the add test.

STAGE 7 OF 7 · PROVE IT · SCREEN 1 OF 2

# Tick the table: every stored procedure, and the three refusals.

Every row below is a test the examiner would run. Three of them are refusals that must happen: deleting customer 1, signing in as Liam, and signing in with the wrong password. Start by clicking in BikeShop.sql and pressing F5 with nothing selected, so that the sample data is back. Then paste each test from the chapter into your test window, press F5, and tick the row when you have seen the result.

| | THE TEST | WHAT MUST HAPPEN | HOW TO DO IT, AND WHAT YOU SHOULD SEE |
|---|---|---|---|
| | The whole script runs clean | **Must be allowed** | Click in BikeShop.sql and press F5 with nothing selected. The Messages pane ends with BikeShop is ready, with no red text above it. |
| | Fifteen stored procedures are listed | **Must be allowed** | In Object Explorer, right-click Stored Procedures under Programmability and click Refresh. Fifteen are listed, from dbo.usp_Appointment_GetForCustomer to dbo.usp_Supplier_GetAll. |
| | Search for ar | **Must be allowed** | Run the search test from stage 2 with ar as the search text. The Results tab shows three rows: Jack O'Hare, Ciara Quinn and Darragh Rice. |
| | Search for 028 | **Must be allowed** | Run the search test from stage 2 again with 028 as the search text. The Results tab shows six rows, and every phone number starts with 028. |
| | Fetch customer 3 | **Must be allowed** | Run the fetch test from stage 2 for customer 3. The Results tab shows one row: Ciara Quinn, with NULL under Email. |
| | Add Eimear Hughes | **Must be allowed** | Run the add test from stage 3. The Messages tab says Added. New customer number 13, and her row is under Results. |
| | Change Ben Toner's phone | **Must be allowed** | Run the update test from stage 3. The Messages tab says Updated customer number 12, and his row shows 07388 640 999. |
| | Delete customer 1 is refused | **Must be refused** | Run the first delete test from stage 4. The Messages tab says: Refused: This customer has bookings or sales on record, so they cannot be deleted. Then run the search test from stage 2 with McCann as the search text, and check that Aoife McCann is still there. |

| THE TEST | WHAT MUST HAPPEN | HOW TO DO IT, AND WHAT YOU SHOULD SEE |
|---|---|---|
| Delete a customer with nothing on record | Must be allowed | Run the second delete test from stage 4. The Messages tab says Added customer number 14, then Deleted customer number 14, and the Results tab shows column headings with no row under them. |
| Bookings for customer 1 show names | Must be allowed | Run the bookings test from stage 5 for customer 1. The Results tab shows two rows, with Conor Walsh and Aoife Byrne in the Mechanic column and Wheel true and Basic service in the Service column, and no numbers in either column. |
| Sign in as siobhan | Must be allowed | Run the first sign-in test from stage 5. The Results tab shows one row: StaffID 1, Siobhán Rooney, Manager. |
| Sign in as liam is refused | Must be refused | Run the second sign-in test from stage 5, with the username liam and the password canal06. Only the column headings appear, with no row under them, because his IsActive is 0. |
| Sign in with a wrong password is refused | Must be refused | Run the third sign-in test from stage 5, with the username siobhan and the password wrong. Only the column headings appear, with no row under them. |
| The mechanics and everyone | Must be allowed | Run the staff test from stage 5. The first grid has three rows and Liam Doran is not in it. The second grid has six rows and Liam Doran is in it with IsActive 0. |
| A taken username is refused | Must be refused | Run the staff add test from stage 5. The Messages tab says Added staff number 7, then Refused: That username is already taken. |
| Every service, with Display | Must be allowed | Run the services test from stage 6. The Results tab shows seven rows, with Puncture repair first and Full service last, and a Display column such as Basic service (60 min, £40.00). |
| Add a service | Must be allowed | Run the service add test from stage 6. The Results tab shows eight rows with Chain replacement first, and the Messages tab says Added. New service number 8. |
| Change a service | Must be allowed | Run the service update test from stage 6. The price for Wheel true shows 22.00, and every other price is unchanged. |
| Every supplier | Must be allowed | Run the suppliers test from stage 6. The Results tab shows three rows in name order: Lagan Bike Parts, Mourne Cycle Supplies, Northern Components Ltd. |

| THE TEST | WHAT MUST HAPPEN | HOW TO DO IT, AND WHAT YOU SHOULD SEE |
|---|---|---|
| Add a supplier | Must be allowed | Run the supplier add test from stage 6. The Results tab shows four rows with Carlingford Cycles first, and the Messages tab says Added. New supplier number 4. |

The numbers in the add rows are the ones you see when the sample data has been put back first. Each extra run of an add test makes the next number one higher, and that is the database doing its job.

# Finish the chapter.

Twenty ticks are your evidence that every action on customers, staff, services and suppliers goes through a stored procedure, and that the database says no in a sentence when it should. Keep screenshots of the three refusals for your testing evidence in chapter 10. When you want the sample data back, click in BikeShop.sql and press F5 with nothing selected. Save your script with Ctrl and S together before you close SQL Server Management Studio (SSMS).

**Make it yours**

Your scenario has its own tables, and most tables need four stored procedures: a Get, an Add, an Update and a Delete. A table that other tables point at, such as a customer with bookings, needs a Delete that refuses. For each table, write down which of the four it needs. Then write one plain sentence for the rule each Add or Delete checks before it acts, because that sentence is what goes into @Message. Name each stored procedure with usp, then the table, then the action, for example usp_Customer_Add. In every Add, Update and Delete, set @Message to a blank message first and use RETURN after a refusal. In every Add, hand back SCOPE_IDENTITY().

CHAPTER 5 OF 12

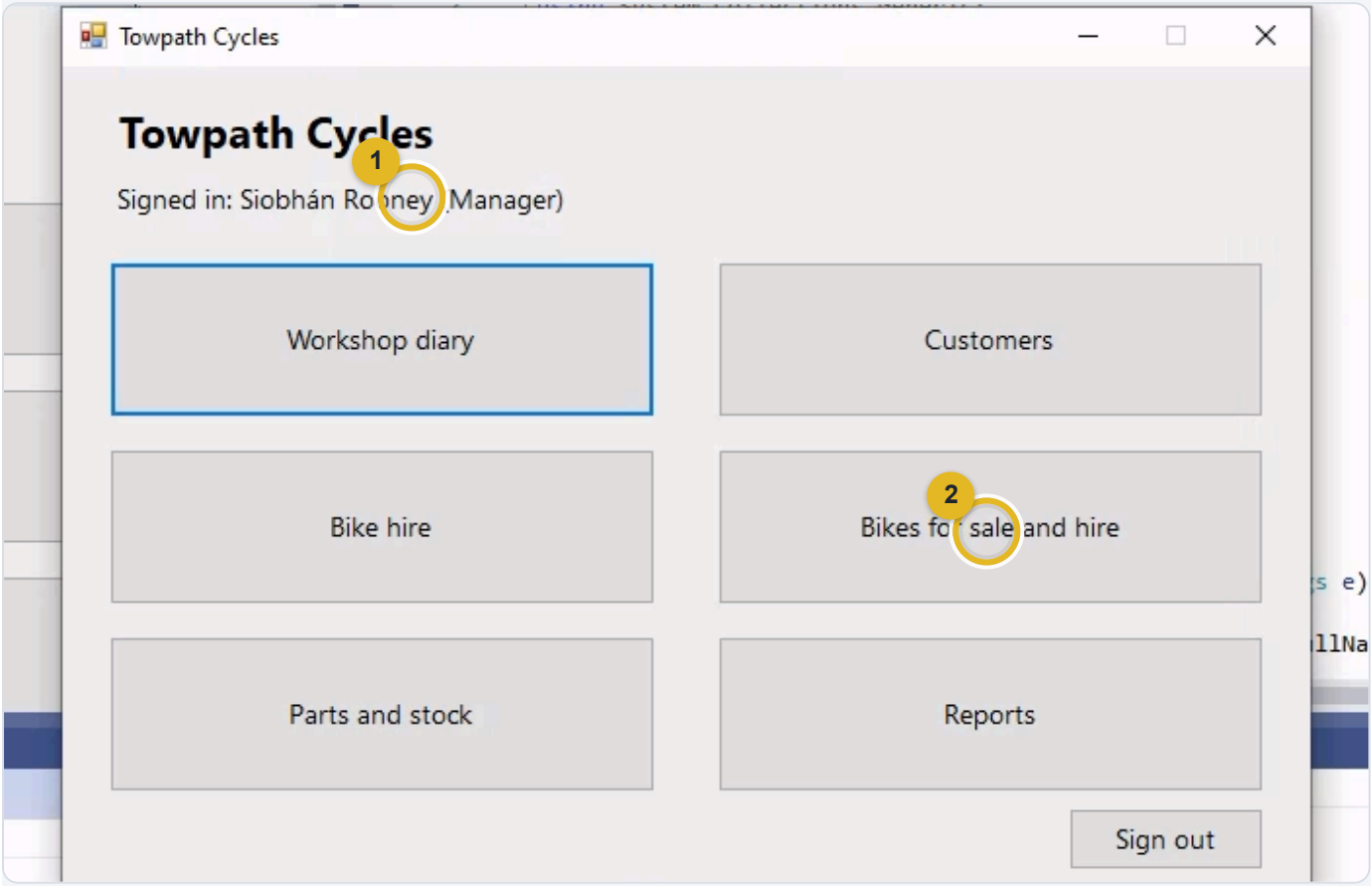
# Connect C#

The project, the connection string, the helper class and the first grid.

STAGE 1 OF 7 · MAKE THE PROJECT IN VISUAL STUDIO 2019 · SCREEN 1 OF 4

## See what you will build in this chapter.

By the end of this chapter your program starts, asks for a username and a password, checks them against the Staff table in your BikeShop database, and opens a main menu with a button for every part of the shop. The menu knows who is signed in, and it switches the Reports button off for anyone who is not a manager.




The main menu. A manager sees every button; a mechanic sees fewer.

1. Who is signed in.
2. One button for each screen.

Up to now you have worked in SQL Server Management Studio (SSMS), where you typed SQL and ran it yourself. In this chapter you open Visual Studio 2019 for the first time and write C#, the language your program is written in. The program will run your stored procedures for you, the same ones you wrote in chapter 4.

**What a C# program is made of**

A C# program is a set of files, each ending in .cs. Each file holds a class: a named box of code that does one job. Inside a class are methods: named chunks of code that run when something calls them, the way a stored procedure runs when something executes it. Visual Studio 2019 keeps all the files together in a project, and the project is what you build and run.

# Check that you have these ready.

**Before you start**

- Chapter 4 finished: BikeShop.sql runs clean in SQL Server Management Studio (SSMS) and ends with the line BikeShop is ready.
- The stored procedure usp_Staff_Login in your script, from chapter 4.
- Visual Studio 2019 open, inside the Programming Environment, as you did in chapter 1.
- Your OneDrive folder for this course, the one that holds BikeShop.sql.

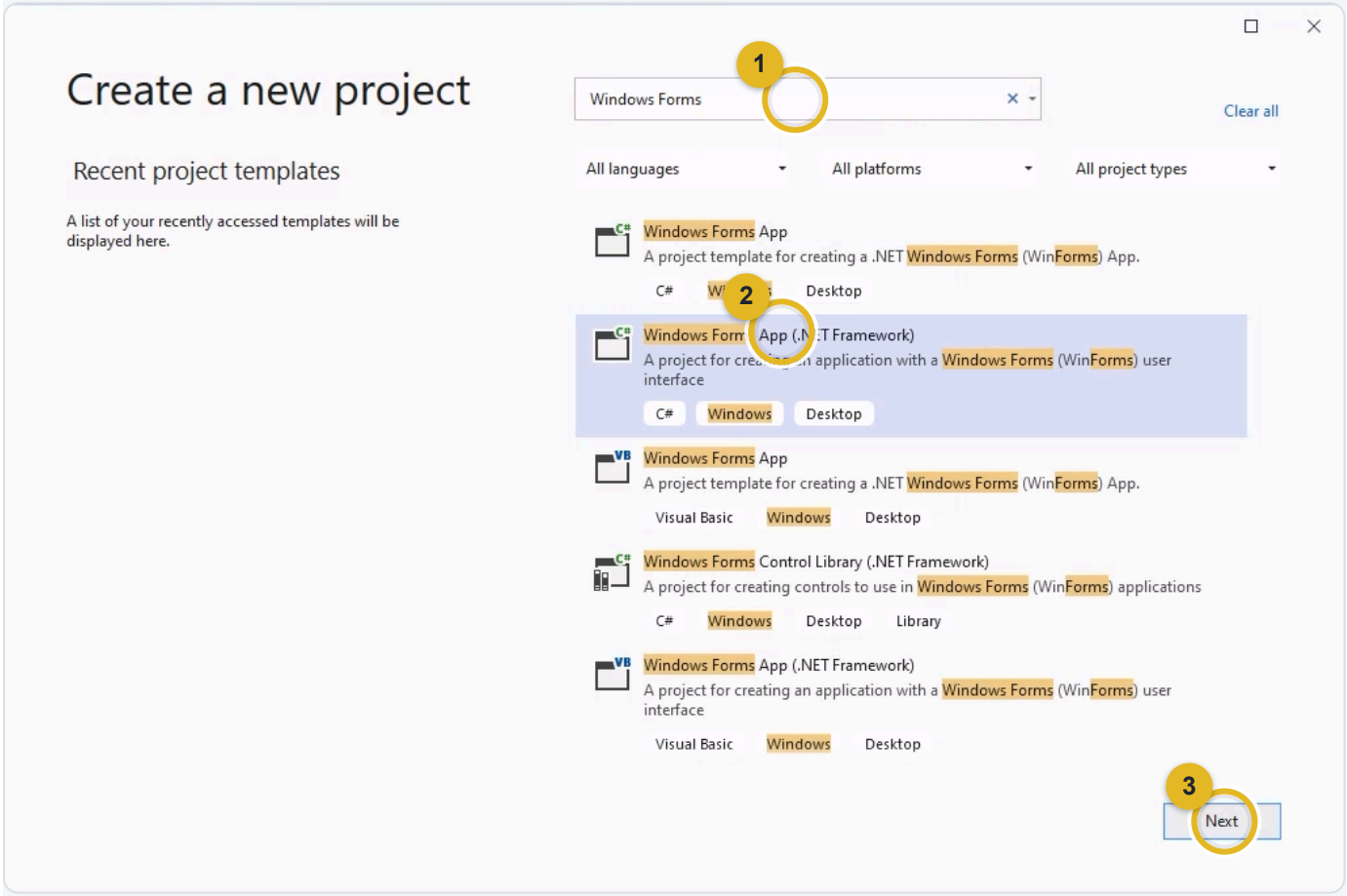
**Words used in this chapter**

| | |
|---|---|
| `Project` | The set of files that make one program. Visual Studio 2019 builds and runs the whole project at once. |
| `Solution Explorer` | The panel on the right of Visual Studio 2019 that lists every folder and file in your project. |
| `Class` | A named box of code in a .cs file that does one job, such as talking to the database or holding who is signed in. |
| `Method` | A named chunk of code inside a class. Code elsewhere calls it by name, and it runs. Methods are the C# version of stored procedures. |
| `Namespace` | The family name of a class, written at the top of its file. Classes in different folders have different family names, such as BikeShop.Data and BikeShop.Forms. |
| `Connection string` | One line of text that tells the program where SQL Server is, which database to open and how to sign in to it. |
| `Reference` | A ready-made set of code from Microsoft that your project is allowed to use. Most are there from the start, and one has to be added by hand. |
| `Form` | A window. Each form is a class with a designer, where you drag buttons and boxes on, and a code tab, where you write what the buttons do. |
| `Control` | One thing on a form: a button, a text box, a label. Each control has a Name that the code uses. |
| `Event` | Something that happens to a control, such as a click. The code that runs when it happens is an event method, and the designer makes it for you. |
| `Build` | Turning your .cs files into a program that can run. Press F6. It either says Build succeeded or lists the errors. |
| `Static` | A class marked static has one copy of everything, shared by the whole program, so you never make a new one. Db and Session are static. |

# Make a new Windows Forms project called BikeShop.

The first window of Visual Studio 2019 asks what you want to do. You want a new project of the Windows Forms kind, which is the kind with windows, buttons and boxes. Make sure to pick the one marked C# and .NET Framework, not the one marked Visual Basic and not the one marked .NET Core.

On the start window of Visual Studio 2019, click Create a new project.




Choose Windows Forms App (.NET Framework), the one marked C#. The one without (.NET Framework) is a different kind of project.

1. Type Windows Forms in the search box.
2. Click Windows Forms App (.NET Framework) with C# under it.
3. Click Next.

> **You should see** A list of project kinds opens, with a search box at the top.

Type Windows Forms in the search box at the top.

> **You should see** The list shrinks to the Windows Forms kinds, including Windows Forms App (.NET Framework).

Click Windows Forms App (.NET Framework) with C# under it, then press Next.

> **You should see** The Configure your new project window opens, with a Project name box at the top.

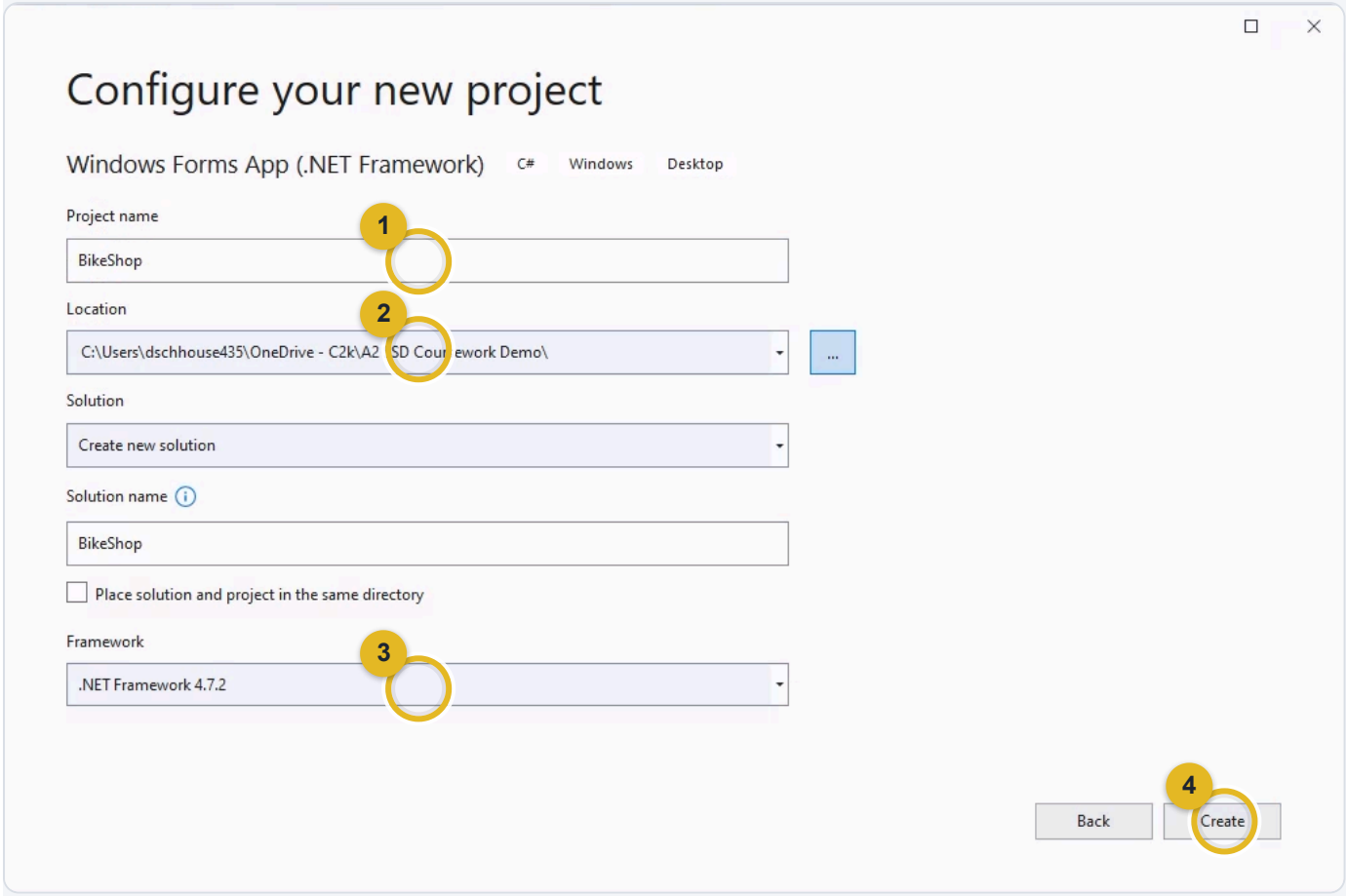
**IF IT GOES WRONG** The list shows only Visual Basic

You clicked the Visual Basic one. Look for the same name with C# in the small grey words under it, and click that one instead.

**IF IT GOES WRONG** The list shows Windows Forms App with no (.NET Framework)

That one is the .NET Core kind and the stored procedures you call will still work, but the pictures in this guide will not match. Pick the one with (.NET Framework) in its name.

Type BikeShop in the Project name box, with a capital B and a capital S and no space.




Name the project BikeShop and put it in your OneDrive coursework folder.

1. Type BikeShop as the Project name.
2. Choose your OneDrive coursework folder as the Location.
3. Choose .NET Framework 4.7.2.
4. Click Create.

> **You should see** The Solution name box under it fills in with BikeShop too.

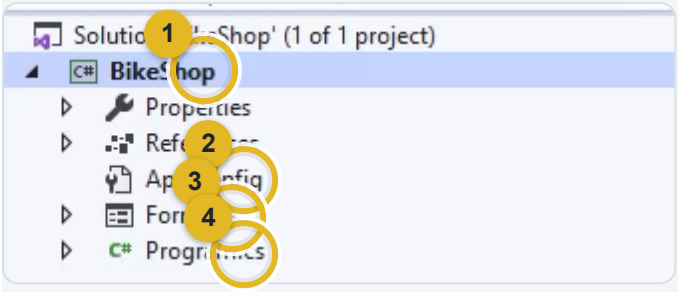
Click the three dots beside the Location box.

> **You should see** A Project Location window opens, showing your folders.

Click your OneDrive folder for this course, the one that holds BikeShop.sql, and press Select Folder.

> **You should see** The Location box shows a path that ends in your OneDrive folder.

In the Framework box, make sure it says .NET Framework 4.7.2, and press Create.




Your new project in Solution Explorer. Five things are there before you have done anything: Properties, References, App.config, Form1.cs and Program.cs.

1. The project, BikeShop, in bold.
2. App.config, which holds the connection string in the next stage.
3. Form1.cs, a window you will delete later in this chapter.
4. Program.cs, which says which window opens first.

**You should see** Visual Studio 2019 opens with an empty window called Form1 in the middle and Solution Explorer on the right.

**IF IT GOES WRONG** The Framework box offers nothing called 4.7.2

Pick the highest 4.x number it offers. Write the number down: you will match it in App.config in the next stage.

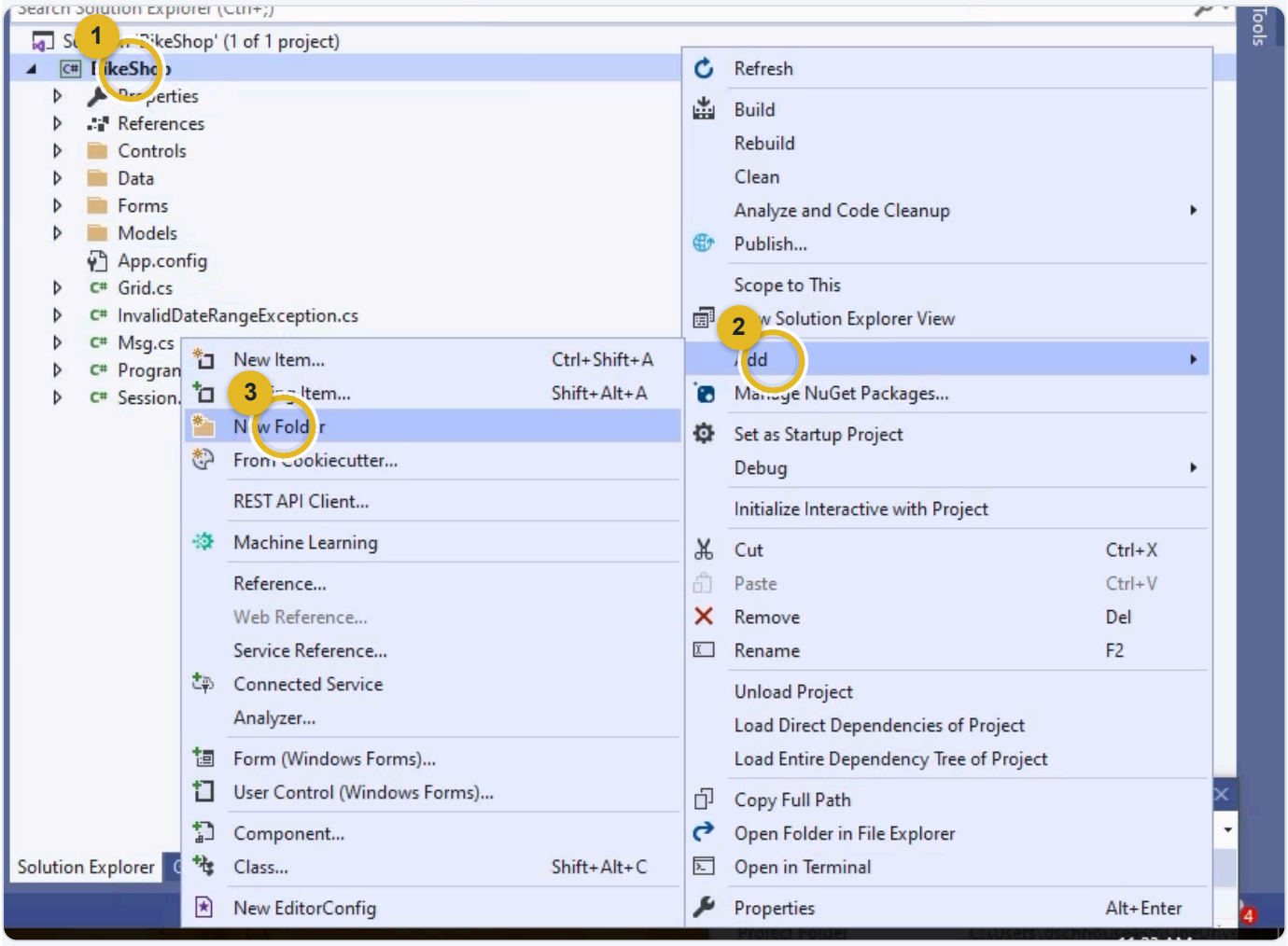
### Where the project lives

The project is a folder called BikeShop inside your OneDrive folder, next to BikeShop.sql. Every file you add in this guide goes inside it. Because it is in OneDrive, it follows you to any machine you sign in on, the same way your script does.

# Add the four folders that keep the files tidy.

A project with thirty files in one long list is hard to read, so you sort them into folders before you add any. Data holds the classes that talk to the database. Models holds classes that describe one thing, such as one member of staff. Forms holds the windows. Controls holds pieces that later chapters put on more than one form. Each folder also gives its classes a namespace, which is a name that groups classes together: a class in the Data folder is in the namespace BikeShop.Data.

In Solution Explorer, right-click the project name BikeShop, point at Add, and click New Folder.




Adding a folder to the project. Type its name straight away and press Enter.

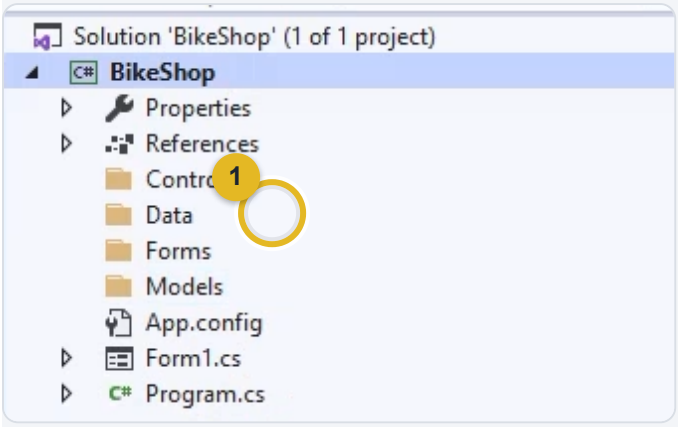
1. Right-click BikeShop, the project, in Solution Explorer.
2. Point at Add.
3. Click New Folder, type the name and press Enter.

> **You should see** A new folder appears under the project with its name highlighted, ready to be typed over.

Type Data and press Enter.

> **You should see** The folder is named Data.

Repeat steps 5.1.6 and 5.1.7 three more times, typing Models, then Forms, then Controls as the names.




The four folders in place. Every file you add from now on goes into one of these, except two that stay at the top level.

1 Controls, Data, Forms and Models, in alphabetical order.

**You should see** Four folders sit under the project: Controls, Data, Forms and Models, in alphabetical order.

**IF IT GOES WRONG** A folder is called NewFolder1

You clicked away before typing the name. Right-click the folder, click Rename, type the name and press Enter.

Leave Form1.cs where it is for now. It keeps the project able to build while you add the files in the next three stages, and you delete it in stage 5, when your own sign-in window is ready to take its place.

# Read what a connection string is.

In SQL Server Management Studio (SSMS) you connected to the server once, by picking a server name in the Connect window. Your program has to do the same thing every time it runs, and it has nobody to pick a name for it. So it reads one line of text that says where the server is, which database to open and how to sign in. That line is the connection string.

The connection string does not live in C# code. It lives in a small settings file called App.config, which Visual Studio 2019 made for you when it made the project. A settings file can be changed without touching any code, which matters when the program moves from the school server to your own machine at home.

**RULE THAT NEVER BREAKS ·** The connection string is written once, in App.config, and read by one class, Db. No other file in the program ever mentions the server name.

# Put the connection string into App.config.

App.config is already in your project. You replace all of it with the version below, which adds the connectionStrings part. The file is written in XML, a language of tags in angle brackets, not C#. You do not need to write any XML yourself: copy it, and read the four parts under it.

In Solution Explorer, double-click App.config.

> **You should see** The file opens in a tab, starting with a line that begins with a question mark and the word xml.

Press the Copy button under this step.

> **You should see** The button reads Copied for a moment.

Click in the code tab and press Ctrl and A together to select everything.

> **You should see** Everything in the code tab is highlighted.

Press Ctrl and V together to paste the copied code over it.

App.config after the paste. The connection string is the long quoted text on the line that starts with connectionString.

1. The name, BikeShop, that the helper class asks for.

2. The connection string: the server, the database, and how to sign in.

> **You should see** The file now has a connectionStrings part in the middle, with a line that names BikeShop.

Press Ctrl and S together to save the file.

**You should see** The star after App.config on the tab disappears.

**Copy this one** **App.config: the whole file** C#

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  <?xml version="1.0" encoding="utf-8" ?>
2  <configuration>
3    <startup>
4      <supportedRuntime version="v4.0" sku=".NETFramework,Version=v4.7.2" />
5    </startup>
6    <connectionStrings>
7      <!— School: LocalDB inside the Programming Environment.
8           At home, change Server= to your own SQL Server name. —>
9      <add name="BikeShop"
10          connectionString="Server=(localdb)\mssqllocaldb;Database=BikeShop;
11            Integrated Security=True;"
12          providerName="System.Data.SqlClient" />
13   </connectionStrings>
14 </configuration>
```

**1 The top of the file**

Lines 1 to 5

The first line says that this file is written in XML. The startup part names the version of .NET Framework the program runs on, 4.7.2, which is the version you picked when you made the project. If you had to pick a different number in stage 1, change 4.7.2 here to match it.

**2 The name of the connection string**

Lines 6 to 9

The connectionStrings part can hold several connection strings, each with a name. Yours has one, named BikeShop. The helper class in the next stage asks for a connection string by this name, so the name here and the name in Db.cs must match exactly.

**3 The connection string itself**

Lines 10 to 11

Three settings, separated by semicolons. Server= names the SQL Server to talk to: (localdb)\mssqllocaldb is the small server inside the Programming Environment, the same name you pick in the Connect window of SQL Server Management Studio (SSMS). Database= names the database to open, BikeShop. Integrated Security=True means sign in as the person who is running the program, so no password is stored in this file.

**4 The provider and the closing tags**

Lines 12 to 14

The provider is the set of code that knows how to talk to SQL Server, called System.Data.SqlClient. The last two lines close the connectionStrings part and the file. Every tag that opens is closed again, which is how XML works.

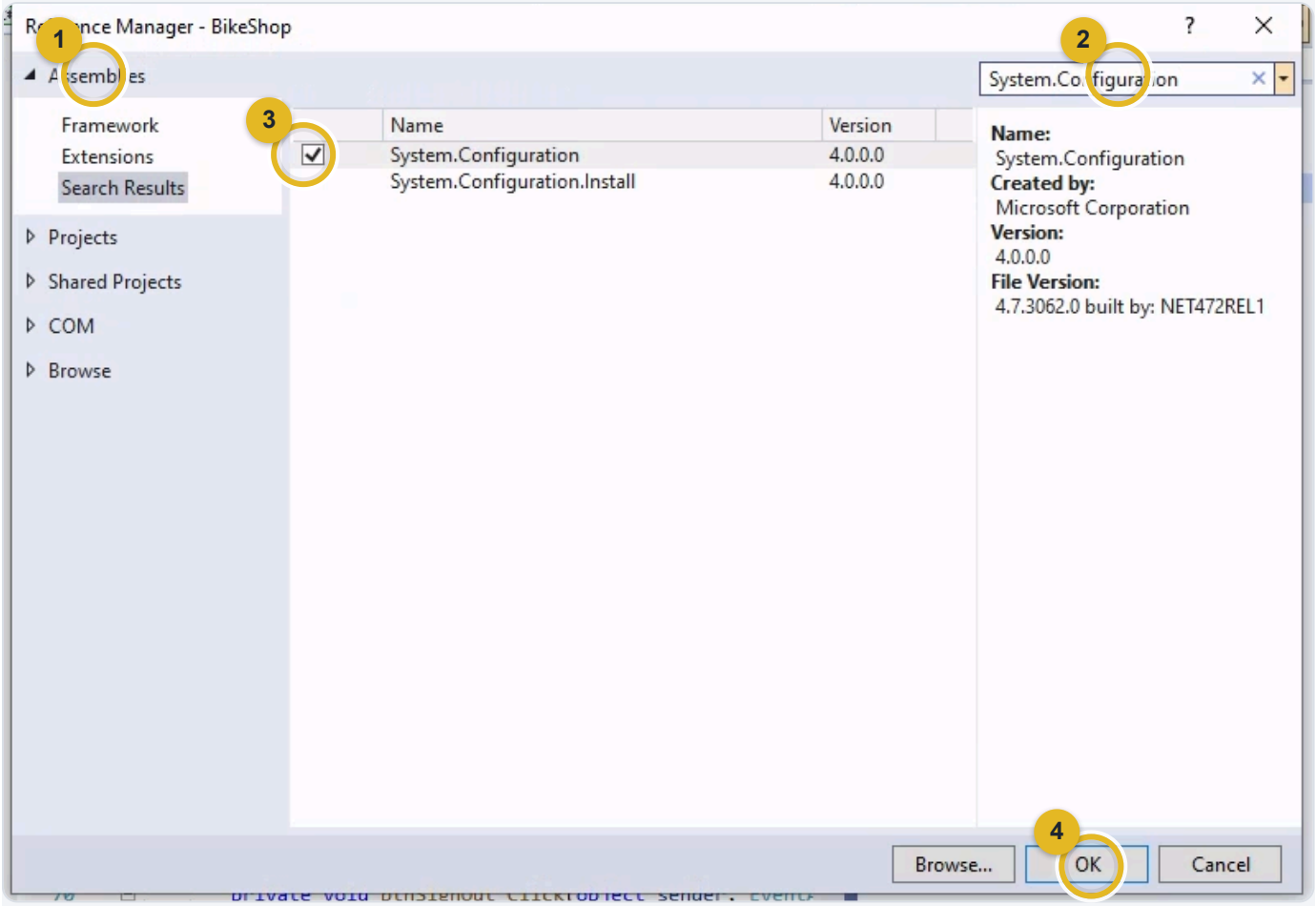
**If you work at home**

The comment line above the add line says what to change. At home, your SQL Server has a different name: it is the name you type in the Server name box when SQL Server Management Studio (SSMS) opens on your own machine, often a dot, or your computer name followed by a backslash and SQLEXPRESS. Change only the part after Server= and before the first semicolon. Nothing in the C# code changes.

# Add the reference that reads App.config.

Your program can read App.config only if it is allowed to use a set of Microsoft code called System.Configuration. Most sets like it are allowed from the start, and they appear under References in Solution Explorer. This one is not, so you add it by hand. If you forget, the helper class in the next stage will not build, and the error will say that the name ConfigurationManager could not be found.

In Solution Explorer, right-click References, and click Add Reference…




Adding System.Configuration, so your program can read the connection string from App.config.

1. Click Assemblies on the left.
2. Type System.Configuration in the search box.
3. Tick System.Configuration.
4. Click OK.

> **You should see** The Reference Manager window opens with a long list of names and a search box at the top right.

In the search box at the top right, type System.Configuration.

> **You should see** The list shrinks to one or two lines that start with System.Configuration.

Tick the box beside System.Configuration, and press OK.

**You should see** The window closes. When you open References in Solution Explorer, System.Configuration is in the list.

**IF IT GOES WRONG** Two lines both say System.Configuration

Tick the one whose version number begins with 4, which matches .NET Framework 4.7.2.

STAGE 3 OF 7 · THE ONE HELPER THE WHOLE PROGRAM TALKS THROUGH · SCREEN 1 OF 3

# Read what the helper class does.

In Visual Studio 2019 you now add the most important file in the program: Db.cs. Db is one class that every other class goes through to reach the database. Nothing else in the program opens a connection or names a stored procedure. When a screen needs rows, it asks Db. When a screen needs to change data, it asks Db. That is the second rule from chapter 0: every table is touched through a stored procedure, and in C# every stored procedure is run through Db.

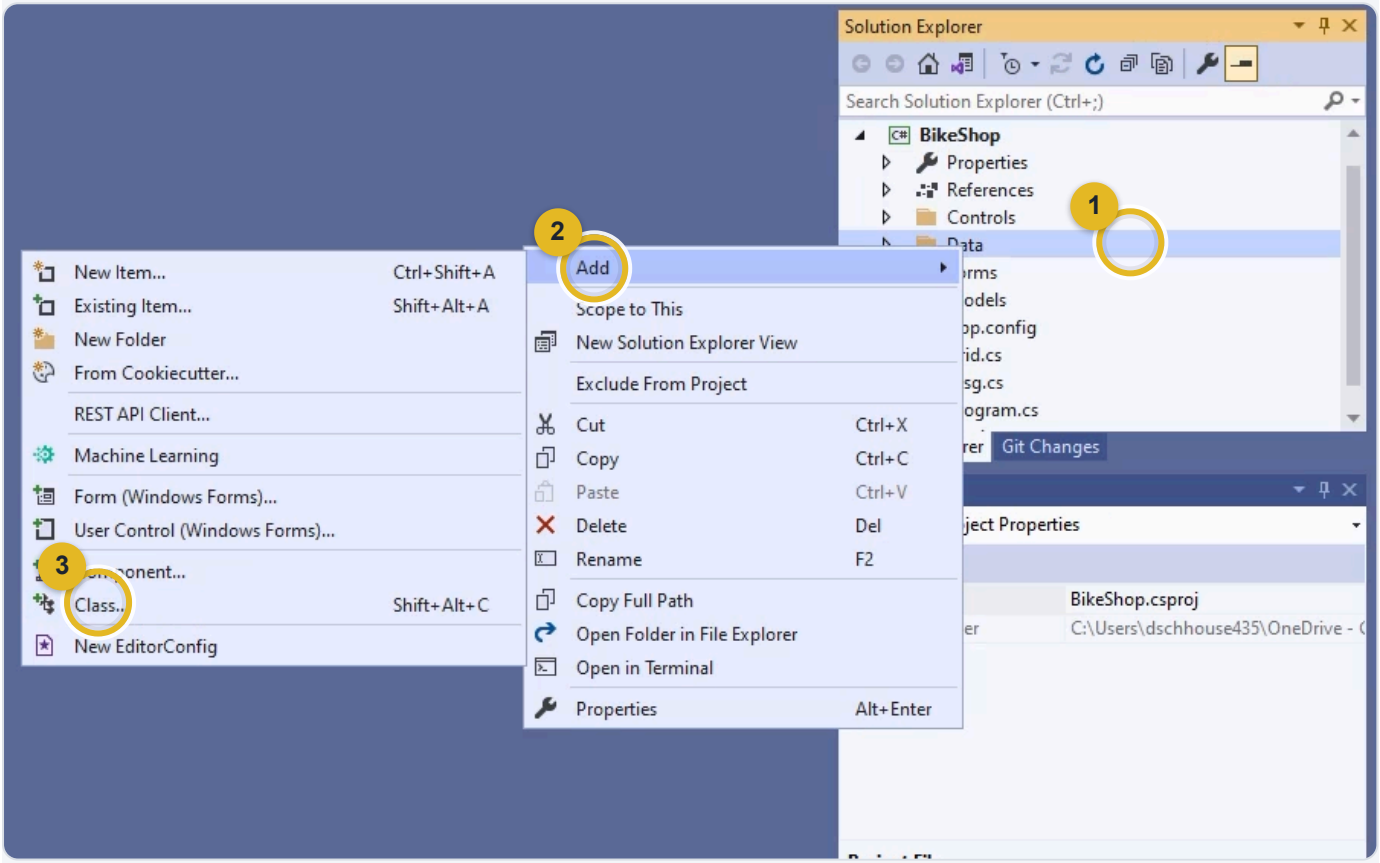
Db has four methods. P makes a parameter, which is one value to send to a stored procedure, such as a username. Out makes an output parameter, a slot the stored procedure fills in and the program reads afterwards, such as the new ID of a row that was just added. GetTable runs a stored procedure that selects rows and hands back a table. Run runs a stored procedure that changes data and hands back its message: an empty message means it worked.

**RULE THAT NEVER BREAKS ·** Every stored procedure is run through Db.GetTable or Db.Run. No form and no other class ever opens its own connection to SQL Server.

# Add the Db class to the Data folder.

A class is added to a folder from Solution Explorer. Visual Studio 2019 makes the file and writes an empty class inside it, with the right family name for the folder, BikeShop.Data. You then paste the finished class over the top of it.

In Solution Explorer, right-click the Data folder, point at Add, and click Class…

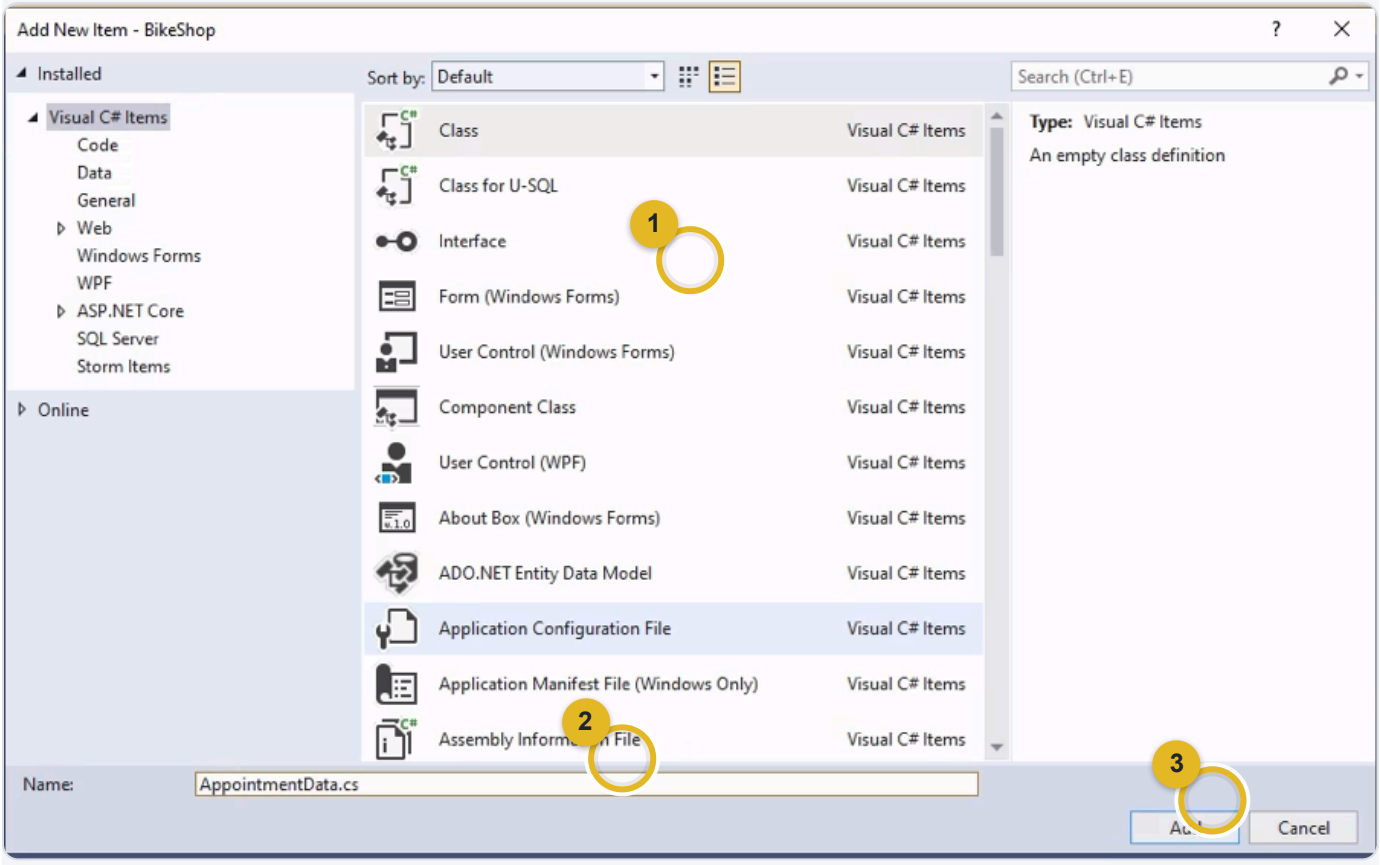



Adding a class to the Data folder. This picture was taken in the finished project, so Solution Explorer lists files you have not made yet.

1. Right-click the Data folder in Solution Explorer.
2. Point at Add.
3. Click Class…

**You should see** The Add New Item window opens with Class selected.

In the Name box at the bottom, type Db.cs and press Add.




The Add New Item window. The Name box is at the bottom: type the file name there, ending in .cs, and press Add.

1. Class is already selected in the middle list.

2. Type the file name in the Name box, ending in .cs.

3. Press Add.

> **You should see** A new file opens with an empty class in it.

**IF IT GOES WRONG** The file is called Class1.cs

You pressed Add before typing the name. Right-click the file in Solution Explorer, click Rename, type Db.cs and press Enter. Then say Yes when it offers to rename the class as well.

Press the Copy button under this step.

> **You should see** The button reads Copied for a moment.

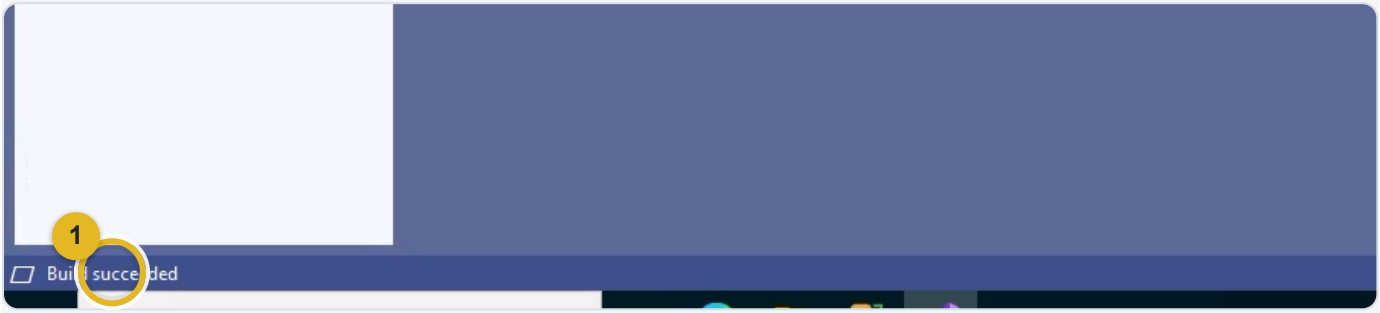
Click in the code tab and press Ctrl and A together to select everything.

> **You should see** Everything in the code tab is highlighted.

Press Ctrl and V together to paste the copied code over it.

> **You should see** The code tab holds the whole file, starting with the using lines.

Press F6 to build.




Build succeeded, at the bottom left. Anything else means an error list to read.

1. Build succeeded. Press F6 to build at any time.

> **You should see** Build succeeded, at the bottom left.

**IF IT GOES WRONG** 'The name ConfigurationManager does not exist'

The System.Configuration reference from stage 2 is missing. Go back to that screen and add it, then press F6 again.

**IF IT GOES WRONG** 'The type or namespace name … could not be found'

A file is missing, or a class name is spelled differently from the one in this chapter. Double-click the error: the line it points at names the thing it cannot find. Check that the file exists in Solution Explorer and that its class is spelled the same way.

# Read Db.cs: the using lines and the connection string.

The whole file is below. It is long, so it is read in pieces, one region at a time. A region is a named fold in the code, marked with #region at its top and #endregion at its bottom. Visual Studio 2019 can fold a region shut with the small minus sign beside it, so that a long file reads like a list of names.

Copy this one **Data/Db.cs: the whole file** C#

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
using System;
using System.Configuration;
using System.Data;
using System.Data.SqlClient;

namespace BikeShop.Data
{
    // The ONE place the program talks to SQL Server.  Every method runs a
    // stored procedure.
    // Four things to know:  P makes a parameter.  Out makes an output
    // parameter.
    // GetTable runs a procedure that SELECTs rows.  Run runs a procedure that
    // changes data.
    public static class Db
    {
        #region ConnectionString
        // The connection string lives in App.config, so it can change without
        // touching the code.
        private static string ConnectionString
        {
            get
            {
                ConnectionStringSettingsCollection all =
                    ConfigurationManager.ConnectionStrings;
                return all["BikeShop"].ConnectionString;
            }
        }
        #endregion

        #region P
        // Makes a parameter to send to a procedure.  A C# null becomes a SQL
        // NULL.
        public static SqlParameter P(string name, object value)
        {
            if (value == null)
            {
                value = DBNull.Value;
            }
            return new SqlParameter(name, value);
        }
        #endregion

        #region Out
        // Makes an output parameter: the procedure fills it in, and you read
        // its .Value afterwards.
        public static SqlParameter Out(string name)
        {
            SqlParameter parameter = new SqlParameter(name, SqlDbType.Int);
            parameter.Direction = ParameterDirection.Output;
            return parameter;
        }
        #endregion

```

```
        #region GetTable
        // For procedures that SELECT rows.  Fills a DataTable you can put
        // straight into a grid or a combo box.
        public static DataTable GetTable(string procedureName,
            params SqlParameter[] parameters)
        {
            DataTable table = new DataTable();
            using (SqlConnection connection =
                new SqlConnection(ConnectionString))
            using (SqlCommand command = new SqlCommand(procedureName,
                connection))
            {
                command.CommandType = CommandType.StoredProcedure;
                command.Parameters.AddRange(parameters);
                using (SqlDataAdapter adapter = new SqlDataAdapter(command))
                {
                    adapter.Fill(table);
                }
            }
            return table;
        }
        #endregion

        #region Run
        // For procedures that change data.  Adds the @Message output parameter
        // itself and returns
        // whatever the procedure put in it.  "" means it worked.  Anything else
        // is a sentence to
        // show the user.  If SQL Server itself says no (a CHECK or a FOREIGN
        // KEY), that message
        // comes back the same way, so a screen never needs try and catch round
        // a database call.
        public static string Run(string procedureName,
            params SqlParameter[] parameters)
        {
            SqlParameter message = new SqlParameter("@Message",
                SqlDbType.VarChar, 200);
            message.Direction = ParameterDirection.Output;
            try
            {
                using (SqlConnection connection =
                    new SqlConnection(ConnectionString))
                using (SqlCommand command = new SqlCommand(procedureName,
                    connection))
                {
                    command.CommandType = CommandType.StoredProcedure;
                    command.Parameters.AddRange(parameters);
                    command.Parameters.Add(message);
                    connection.Open();
                    command.ExecuteNonQuery();
                }
            }
            catch (SqlException ex)
            {
                return ex.Message;
            }
            return message.Value.ToString();
        }
```

```
112          #endregion
113      }
114  }
```

**1 The using lines**

Lines 1 to 4

Each using line names a set of ready-made code this file needs. System.Data.SqlClient is the one that knows how to talk to SQL Server. System.Configuration is the one that reads App.config, and it works because of the reference you added in stage 2.

**2 The namespace and the class**

Lines 6 to 15

The namespace line gives the class its family name: this class is BikeShop.Data.Db. The class is static, which means there is one shared copy of it for the whole program, so a form calls Db.GetTable directly without making a Db first. The comment lines above it are the four things to know, in one sentence each.

**3 ConnectionString: reading App.config**

Lines 16 to 28

This is a property, a named value with code behind it. When code asks for ConnectionString, the get part runs: it asks ConfigurationManager for every connection string in App.config, picks the one named BikeShop, and hands back its text. It is private, so only Db can read it, which keeps the server name in one place.

**4 P: one parameter to send**

Lines 30 to 41

P takes a name, such as @Username, and a value, and wraps them as a SqlParameter, which is what SQL Server expects to receive. The if part handles an empty value: C# calls an empty value null, and SQL Server calls it NULL, and they are not the same thing, so null is swapped for DBNull.Value before it is sent.

**5 Out: a slot the procedure fills in**

Lines 43 to 52

Out makes a parameter that goes the other way. The stored procedure writes a whole number into it, such as the ID of the row it just added, and the program reads parameter.Value afterwards. Direction set to Output is what tells SQL Server that this one is for writing into.

**6 GetTable: run a procedure that selects rows**

Lines 54 to 67

GetTable takes the name of a stored procedure and any number of parameters, which is what params means. It makes an empty DataTable, then opens a connection using the connection string and makes a command that names the stored procedure. A using line with a block after it means close this when the block ends, so the connection is always closed, even if something goes wrong. CommandType set to StoredProcedure tells SQL Server that the name is a stored procedure and not a query to read.

**7 GetTable: fill the table and hand it back**

Lines 68 to 75

A SqlDataAdapter is the piece that runs the command and copies every row it selects into the table. Fill does that in one go. The table is then handed back, and whoever asked can put it straight into a grid on a form.

**8 Run: a procedure that changes data**

Lines 77 to 91

Run is for stored procedures that insert, update or delete. Every one of those in chapter 4 has an @Message output parameter, so Run makes that parameter itself, 200 characters long, before it starts. The comment above says the rule: an empty message means it worked, and anything else is a sentence to show the user.

**9 Run: open, run, close**

Lines 92 to 105

The same connection and command as GetTable, with the message parameter added to the list. Open opens the connection, and ExecuteNonQuery runs the stored procedure without expecting rows back. Both using lines close again when the block ends.

**10 Run: when SQL Server itself says no**

Lines 106 to 114

try and catch mean: try this, and if SQL Server throws an error, run the catch part instead of crashing. A CHECK rule or a foreign key that refuses a change arrives here as a SqlException, and its message is handed back the same way as a message from the stored procedure. So a form only ever sees a sentence, and it never needs try and catch of its own round a database call. The last line hands back whatever the stored procedure put into @Message.

### Why one helper

If every form opened its own connection, a change to the server name would mean a change on every form, and one form would be bound to get it wrong. With Db, the connection string is read in one place, the closing of connections happens in one place, and the rule about messages is kept in one place. The examiner can see all of it on one page.

STAGE 4 OF 7 · WHO IS SIGNED IN, AND HOW THE PROGRAM ASKS · SCREEN 1 OF 5

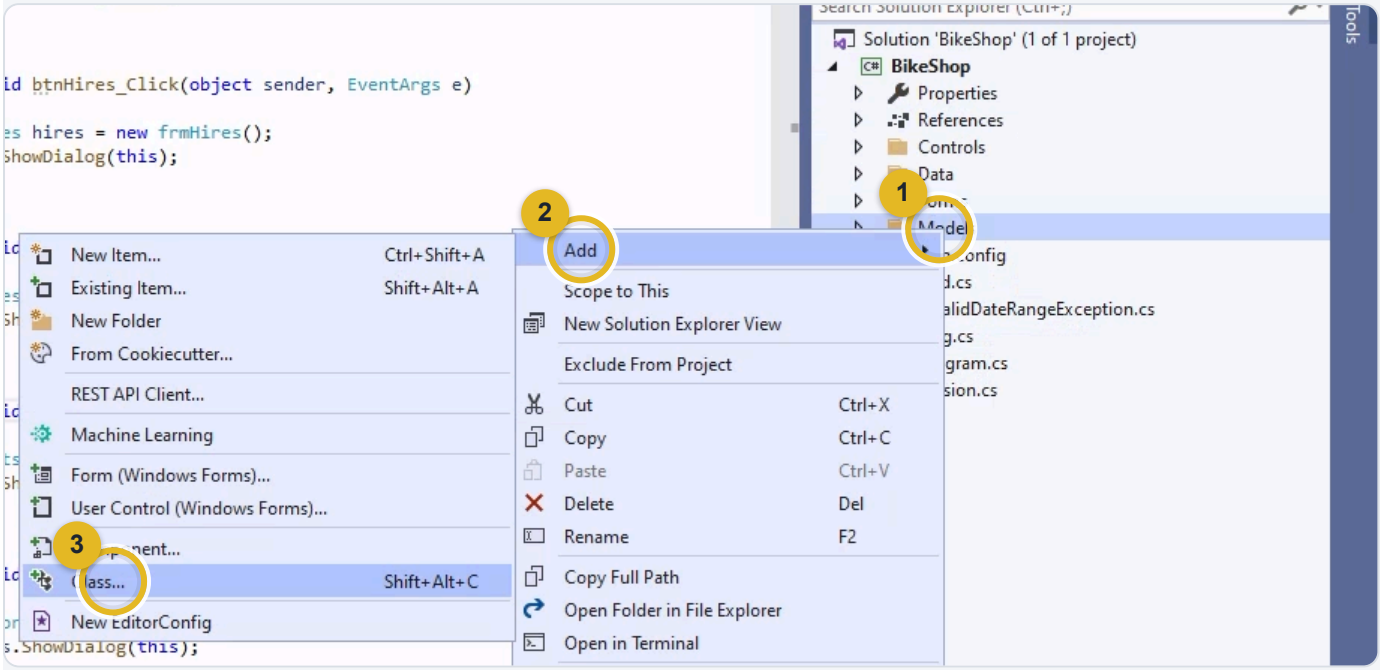
## Read what the four small classes do.

In this stage you add four short files to Visual Studio 2019. StaffMember describes one member of staff: it is a class with six properties and no methods, one property per column of the Staff table. Session remembers who is signed in, for the whole program, from sign-in to sign-out. StaffData is the class that asks Db to run the Staff stored procedures. Msg shows message boxes, so that every form says things the same way.

The pattern of StaffData is the one every later chapter repeats: a class in the Data folder, one method per stored procedure, each method one call to Db. Read it closely here, because chapters 6 to 9 add CustomerData, PartData and the rest in exactly this shape.

# Add the StaffMember class to the Models folder.

In Solution Explorer, right-click the Models folder, point at Add, and click Class…




Right-click the Models folder, point at Add, and click Class…

1. Right-click the Models folder.
2. Point at Add.
3. Click Class…

> **You should see** The Add New Item window opens with Class selected.

In the Name box at the bottom, type StaffMember.cs and press Add.

> **You should see** A new file opens with an empty class in it.

Press the Copy button under this step.

> **You should see** The button reads Copied for a moment.

Click in the code tab and press Ctrl and A together to select everything.

> **You should see** Everything in the code tab is highlighted.

Press Ctrl and V together to paste the copied code over it.

**You should see** The code tab holds the whole file, starting with the namespace line.

**Copy this one** **Models/StaffMember.cs: the whole file** C#

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  namespace BikeShop.Models
2  {
3      // One member of staff.  Plain properties only.
4      public class StaffMember
5      {
6          public int StaffID { get; set; }
7          public string FirstName { get; set; }
8          public string LastName { get; set; }
9          public string Role { get; set; }
10         public string Username { get; set; }
11         public string Password { get; set; }
12     }
13 }
```

**1 The namespace and the class**

Lines 1 to 5

This class lives in the Models folder, so its family name is BikeShop.Models. It is not static: the program makes a new StaffMember each time it needs to hold one person, the way the Staff table holds one row per person.

**2 Six properties, one per column**

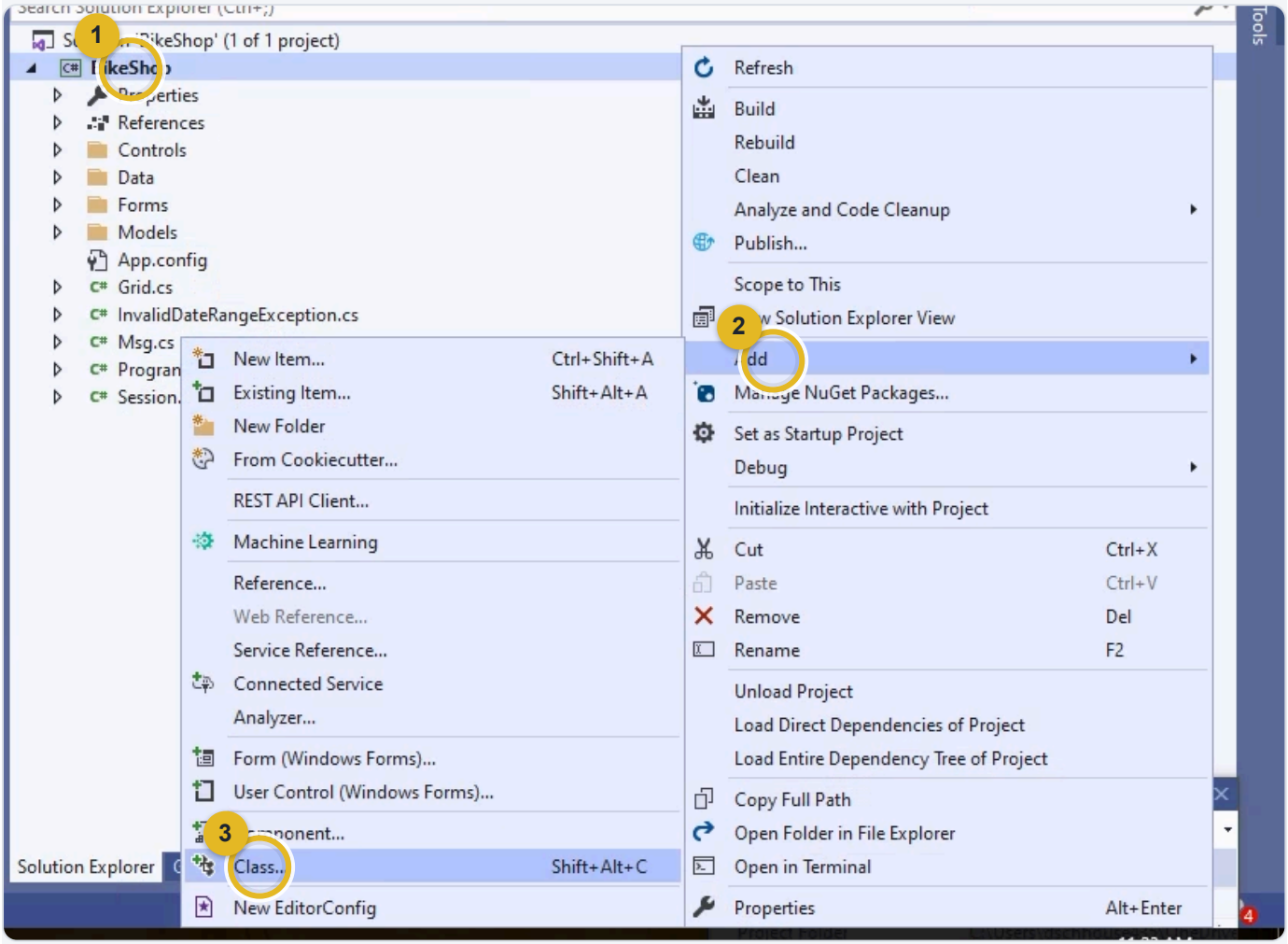
Lines 6 to 13

Each line is a property: a named value the class holds. get; set; means that other code can read the value and change it. The names and the kinds match the Staff table: StaffID is a whole number, and the other five are text.

# Add the Session class at the top level, and read it.

Session does not go in a folder. It is used by every form, so it sits at the top level of the project, beside Program.cs, with the plain family name BikeShop. To add a class at the top level you right-click the project name instead of a folder.

In Solution Explorer, right-click the project name BikeShop, point at Add, and click Class…




Right-click the project name BikeShop, not a folder, point at Add, and click Class… The new file lands at the top level of the project.

1. Right-click the project name, BikeShop, in bold.
2. Point at Add.
3. Click Class…

> **You should see** The Add New Item window opens with Class selected.

In the Name box at the bottom, type Session.cs and press Add.

> **You should see** A new file opens with an empty class in it.

Press the Copy button under this step.

> **You should see** The button reads Copied for a moment.

Click in the code tab and press Ctrl and A together to select everything.

> **You should see** Everything in the code tab is highlighted.

Press Ctrl and V together to paste the copied code over it.

> **You should see** The code tab holds the whole file, starting with the namespace line.

Copy this one **Session.cs: the whole file** C#

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  namespace BikeShop
2  {
3      // Who is signed in.  Set once by frmLogin, read by any form that needs to
4      // record who did something.
5      public static class Session
6      {
7          public static int StaffID { get; private set; }
8          public static string FullName { get; private set; }
9          public static string Role { get; private set; }
10
11         public static bool IsSignedIn
12         {
13             get { return StaffID > 0; }
14         }
15
16         public static bool IsManager
17         {
18             get { return Role == "Manager"; }
19         }
20
21         public static void SignIn(int staffID, string fullName, string role)
22         {
23             StaffID = staffID;
24             FullName = fullName;
25             Role = role;
26         }
27
28         public static void SignOut()
29         {
30             StaffID = 0;
31             FullName = null;
32             Role = null;
33         }
34     }
35 }
```

**1 The namespace and the class**

Lines 1 to 6

The family name is plain BikeShop, because the file is at the top level. The class is static: there is one Session for the whole program, which is the point, because only one person is signed in at a time.

**2 Three things about the person**

Lines 7 to 9

The ID, the full name and the role of whoever is signed in. get; private set; means any form can read them, but only code inside Session can change them, so a form cannot sign someone in by accident.

**3 IsSignedIn: is anyone there**

Lines 11 to 14

A property that works out its answer each time it is read. bool means the answer is true or false. Nobody has a StaffID of zero, so a StaffID above zero means someone is signed in.

**4 IsManager: is it a manager**

Lines 16 to 19

True when the role of the signed-in person is exactly the word Manager. The main menu reads this to decide whether the Reports button is switched on.

**5 SignIn: remember the person**

Lines 21 to 26

The sign-in form calls this once, after the database has said yes, passing the three values from the row it got back. void means the method hands nothing back. The three lines store the three values.

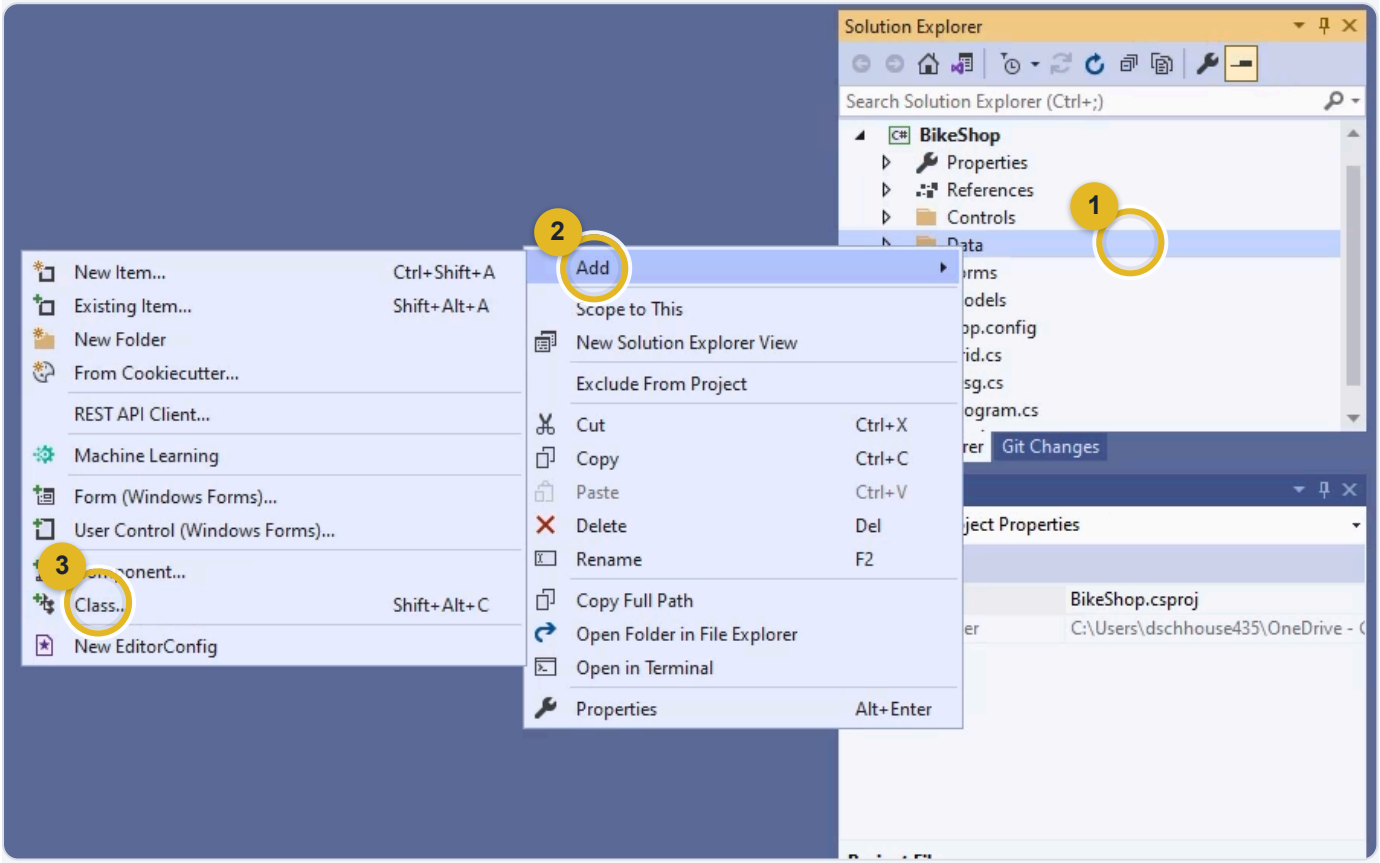
**6 SignOut: forget the person**

Lines 28 to 35

The main menu calls this when Sign out is pressed. StaffID goes back to zero, so IsSignedIn becomes false, and the name and role are cleared. null means no value at all.

# Add the StaffData class to the Data folder, and read its Login method.

In Solution Explorer, right-click the Data folder, point at Add, and click Class…




Adding a class to the Data folder. This picture was taken in the finished project, so Solution Explorer lists files you have not made yet.

1. Right-click the Data folder in Solution Explorer.
2. Point at Add.
3. Click Class…

> **You should see** The Add New Item window opens with Class selected.

In the Name box at the bottom, type StaffData.cs and press Add.

> **You should see** A new file opens with an empty class in it.

Press the Copy button under this step.

> **You should see** The button reads Copied for a moment.

Click in the code tab and press Ctrl and A together to select everything.

> **You should see** Everything in the code tab is highlighted.

Press Ctrl and V together to paste the copied code over it.

> **You should see** The code tab holds the whole file, starting with the using lines.

Copy this one **Data/StaffData.cs: the whole file** C#

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
using System.Data;
using System.Data.SqlClient;
using BikeShop.Models;

namespace BikeShop.Data
{
    public static class StaffData
    {
        #region Login
        // Returns one row if the details match an active member of staff,
        // otherwise no rows.
        public static DataTable Login(string username, string password)
        {
            return Db.GetTable("usp_Staff_Login", Db.P("@Username", username),
                Db.P("@Password", password));
        }
        #endregion

        // The staff who are mechanics.
        public static DataTable GetMechanics()
        {
            return Db.GetTable("usp_Staff_GetMechanics");
        }

        // Every member of staff.
        public static DataTable GetAll()
        {
            return Db.GetTable("usp_Staff_GetAll");
        }

        // Saves a new member of staff.  Returns "" if it worked, or the
        // sentence to show the user.
        public static string Add(StaffMember staff)
        {
            SqlParameter newID = Db.Out("@NewStaffID");
            string message = Db.Run("usp_Staff_Add",
                Db.P("@FirstName", staff.FirstName),
                Db.P("@LastName", staff.LastName),
                Db.P("@Role", staff.Role),
                Db.P("@Username", staff.Username),
                Db.P("@Password", staff.Password),
                newID);
            if (message == "")
            {
                staff.StaffID = (int)newID.Value;
            }
            return message;
        }
    }
}
```

**1 The using lines, the namespace and the class**

Lines 1 to 8

System.Data is for the DataTable that comes back. BikeShop.Models is where StaffMember lives, so this file can use it. The class is static, like Db: a form calls StaffData.Login directly.

### 2 Login: ask the database if the details match

Lines 9 to 17

This is the method the sign-in form calls. It hands Db.GetTable the name of your stored procedure, usp_Staff_Login, and two parameters made with Db.P: the username and the password, with the same @ names the stored procedure declares. The stored procedure sends back one row when the details match an active member of staff and no rows when they do not, and the DataTable it comes back in is handed straight on. Nothing here decides whether the sign-in is allowed: the database decides.

### 3 GetMechanics and GetAll: two lists

Lines 19 to 29

Two more methods in the same shape, each one line long. GetMechanics runs usp_Staff_GetMechanics, which the diary in chapter 7 uses to fill its list of mechanics. GetAll runs usp_Staff_GetAll. Neither needs a parameter.

### 4 Add: save a new person and learn their ID

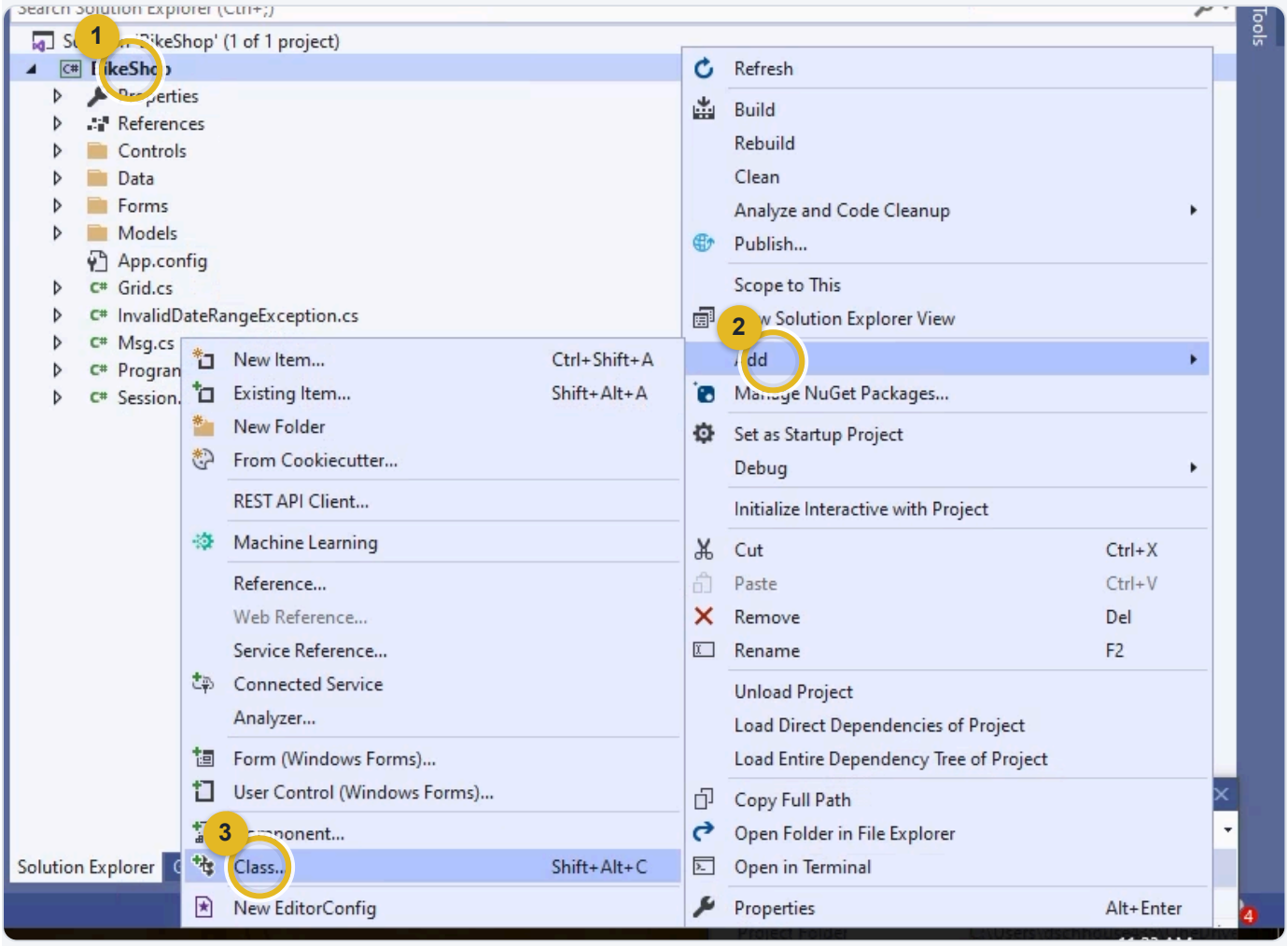
Lines 31 to 50

Add takes a filled-in StaffMember and runs usp_Staff_Add through Db.Run, with one Db.P for each column and one Db.Out for the new ID. Db.Run hands back the message. If the message is empty, the stored procedure worked, so the new ID is read out of the output parameter and stored in the StaffMember. Either way the message is handed back for the form to show. You do not call Add in this chapter: it is here so the pattern is complete.

# Add the Msg class at the top level, and read it.

A message box is a small window with a sentence and a button, the kind that pops up to say something worked or ask Are you sure. Msg wraps the three kinds the program uses, so every form shows them with the same title and the same icons. Like Session, it is used everywhere, so it goes at the top level.

In Solution Explorer, right-click the project name BikeShop, point at Add, and click Class…




Right-click the project name BikeShop, not a folder, point at Add, and click Class… The new file lands at the top level of the project.

1. Right-click the project name, BikeShop, in bold.
2. Point at Add.
3. Click Class…

> **You should see** The Add New Item window opens with Class selected.

In the Name box at the bottom, type Msg.cs and press Add.

> **You should see** A new file opens with an empty class in it.

Press the Copy button under this step.

> **You should see** The button reads Copied for a moment.

Click in the code tab and press Ctrl and A together to select everything.

> **You should see** Everything in the code tab is highlighted.

Press Ctrl and V together to paste the copied code over it.

> **You should see** The code tab holds the whole file, starting with the using lines.

Press F6 to build.

> **You should see** Build succeeded, at the bottom left.

**IF IT GOES WRONG** 'The type or namespace name … could not be found'

A file is missing, or a class name is spelled differently from the one in this chapter. Double-click the error: the line it points at names the thing it cannot find. Check that the file exists in Solution Explorer and that its class is spelled the same way.

Copy this one **Msg.cs: the whole file** C#

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  using System.Windows.Forms;
2
3  namespace BikeShop
4  {
5      // Three kinds of message box, so every form says things the same way.
6      public static class Msg
7      {
8          private const string Title = "Towpath Cycles";
9
10         public static void Info(string text)
11         {
12             MessageBox.Show(text, Title, MessageBoxButtons.OK,
13                 MessageBoxIcon.Information);
14         }
15
16         public static void Problem(string text)
17         {
18             MessageBox.Show(text, Title, MessageBoxButtons.OK,
19                 MessageBoxIcon.Warning);
20         }
21
22         // Returns true when the user clicks Yes.
23         public static bool Ask(string question)
24         {
25             return MessageBox.Show(question, Title, MessageBoxButtons.YesNo,
26                 MessageBoxIcon.Question) == DialogResult.Yes;
27         }
28     }
29 }
```

### 1 The using line, the class and the title

Lines 1 to 8

System.Windows.Forms is the set of code that makes windows and message boxes. The class is static. Title is a const, a value that never changes: the words in the title bar of every message box, Towpath Cycles.

### 2 Info: something worked

Lines 10 to 14

MessageBox.Show opens the box. It is given the sentence to show, the title, which buttons to offer, OK only, and which icon to show, the blue Information one.

### 3 Problem: something was refused

Lines 16 to 20

The same box with the yellow Warning icon. Forms use this to show a message that came back from a stored procedure.

### 4 Ask: a question with Yes and No

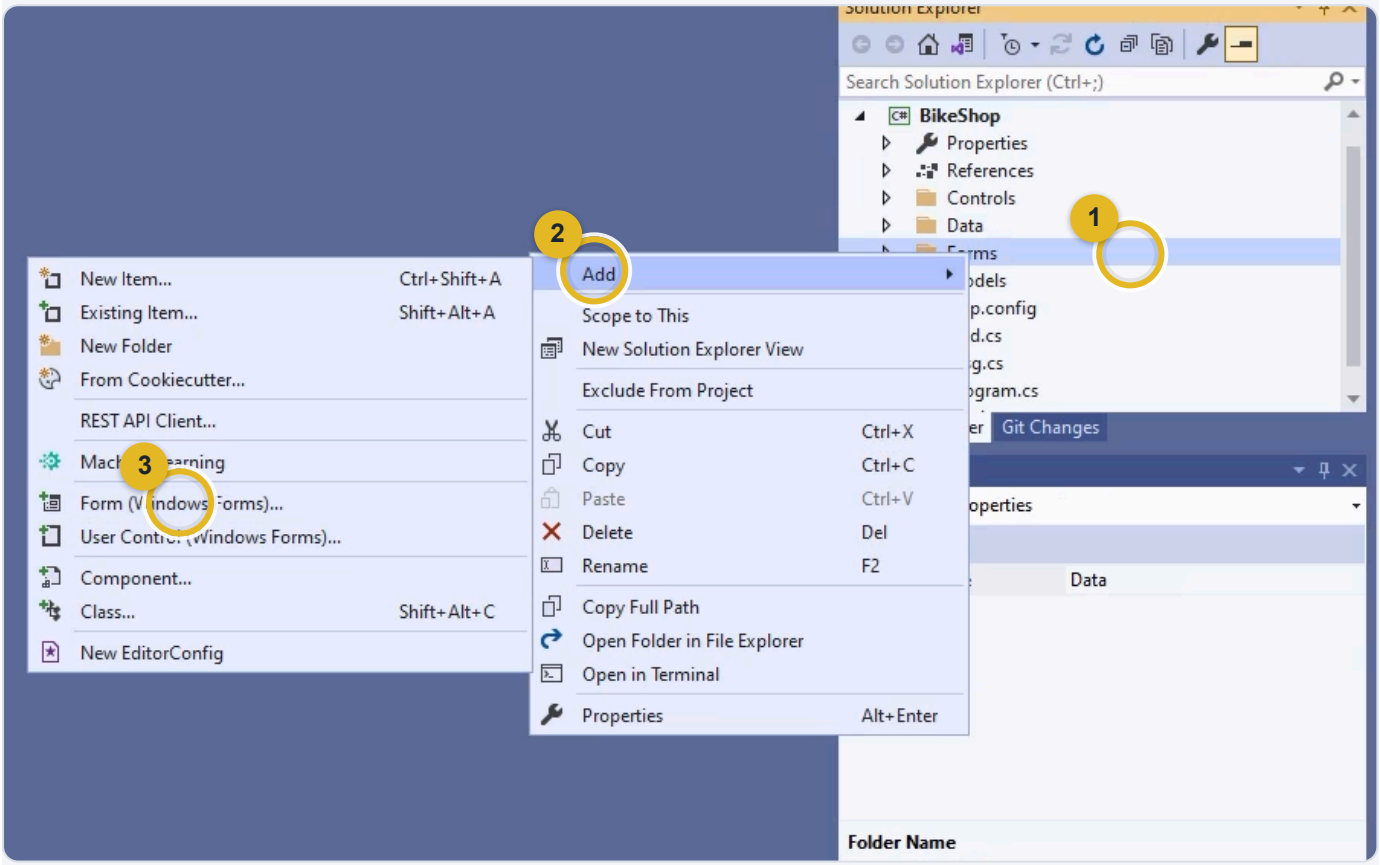
Lines 22 to 29

This box offers Yes and No. MessageBox.Show hands back which button was pressed, as a DialogResult, and the method compares it with DialogResult.Yes. So the method hands back true when Yes was pressed, and a form can write if (Msg.Ask("Delete this customer?")) and act only on a Yes.

# Add the sign-in form to the Forms folder.

A form is a window. Visual Studio 2019 gives every form two tabs: the designer, where you drag controls onto the window, and the code tab, where you write what the controls do. You add the form, build its controls in the designer, and then paste the code. This is the order every form in this guide follows.

In Solution Explorer, right-click the Forms folder, point at Add, and click Form (Windows Forms)…




Adding a form to the Forms folder. Taken in the finished project, so Solution Explorer lists more files than yours.

1. Right-click the Forms folder.
2. Point at Add.
3. Click Form (Windows Forms)…, type the name, press Add.

> **You should see** The Add New Item window opens with Form (Windows Forms) selected.

In the Name box at the bottom, type frmLogin.cs and press Add.

> **You should see** An empty form opens in the designer, with frmLogin in its title bar.

Open the Toolbox (View, then Toolbox) and the Properties window (View, then Properties Window), if they are not already open.

> **You should see** The Toolbox is on the left, the form in the middle, the Properties window on the right.

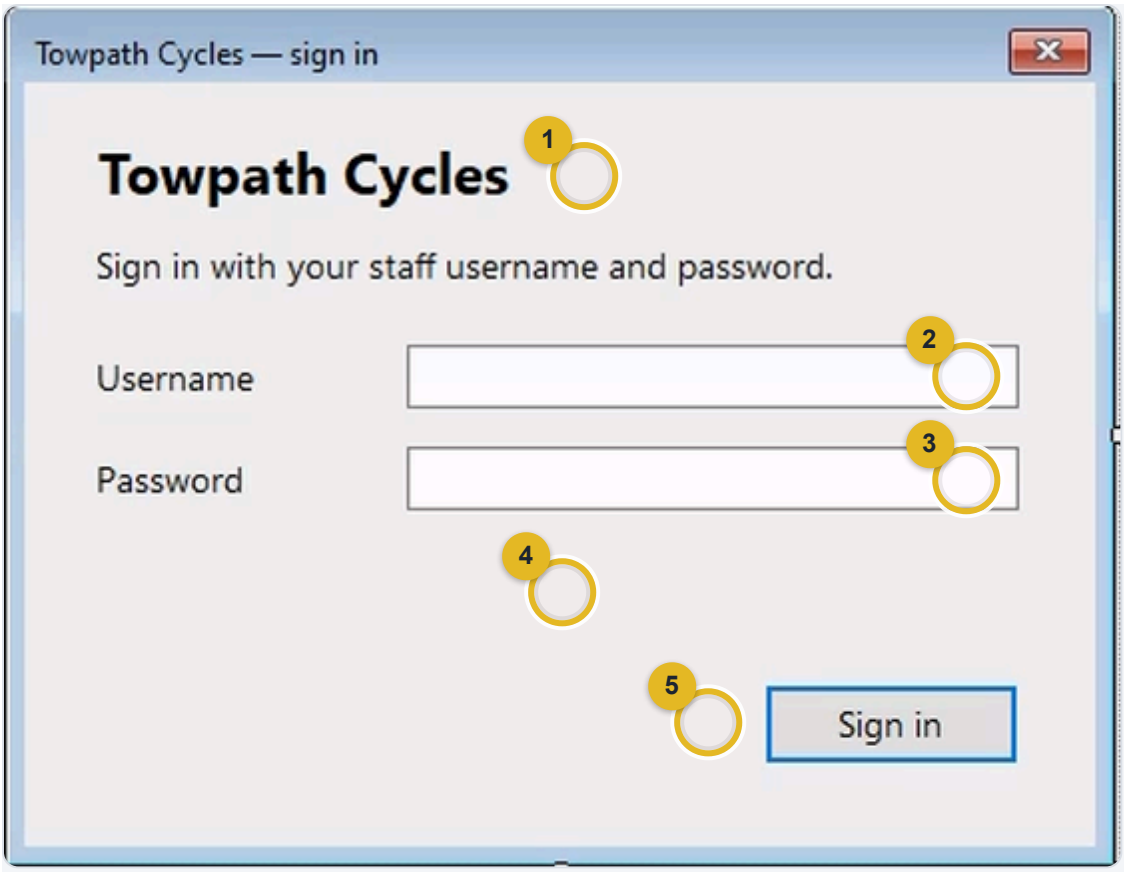
# Set the form and its nine controls.

Click the form itself first and set the form's own properties. Then work down the table: drag the control from the Toolbox, drop it on the form, set its Name in the Properties window, and set the other settings in the row. Where the Then column says double-click, double-clicking the control makes its event method for you.

The first window. A member of staff types their username and password; the database says whether they match.

**Click the form itself, then set these in the Properties window**

| | | | |
|---|---|---|---|
| Text | `Towpath Cycles — sign in` | Size | `420, 300` |
| FormBorderStyle | `FixedDialog` | StartPosition | `CenterParent` |
| MaximizeBox | `False` | MinimizeBox | `False` |
| ShowInTaskbar | `False` | AcceptButton | `btnSignIn (after you add it)` |

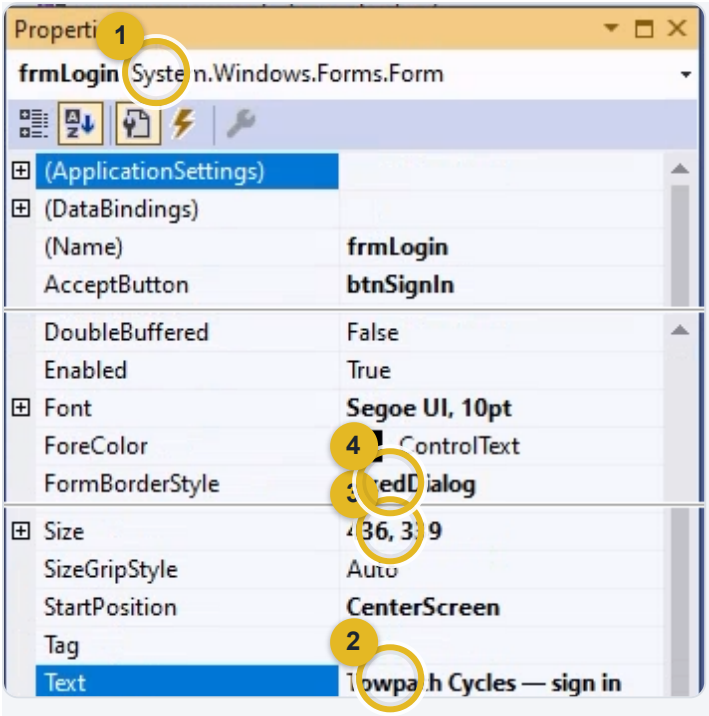



The sign-in form with every control in place. Match this layout: the two boxes one above the other, the red message label under them, the button at the bottom.

1. lblTitle, in 16 point bold.
2. txtUsername, with lblUsername to its left.
3. txtPassword, with lblPassword to its left. It shows dots instead of letters.
4. lblMessage, empty for now, in Firebrick red.
5. btnSignIn, which you double-click last.

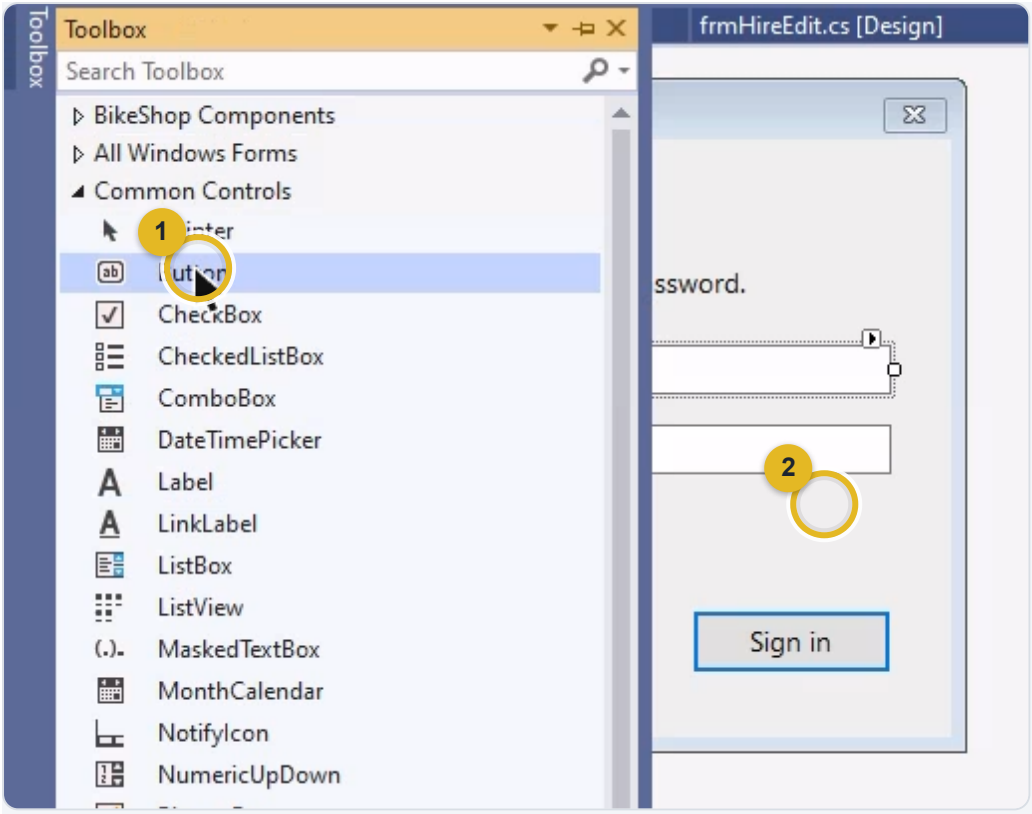
| DRAG THIS FROM THE TOOLBOX | NAME IT (THE NAME PROPERTY) | SET THESE IN THE PROPERTIES WINDOW | THEN |
|---|---|---|---|
| Label | `lblTitle` | `Text = Towpath Cycles`<br>`Font = Segoe UI, 16pt, Bold`<br>The shop name, big, so the window looks like it belongs to the shop. | |
| Label | `lblSub` | `Text = Sign in with your staff username and password.`<br>Tells the user what to do. | |
| Label | `lblUsername` | `Text = Username` | |
| TextBox | `txtUsername` | nothing else<br>Where the username is typed. | |
| Label | `lblPassword` | `Text = Password` | |
| TextBox | `txtPassword` | `UseSystemPasswordChar = True`<br>Shows dots instead of letters. | |
| Label | `lblMessage` | `ForeColor = Firebrick`<br>Starts empty. Shows "Those details are not recognised" when sign-in fails. | |
| Button | `btnSignIn` | `Text = Sign in`<br>Checks the details against the Staff table. | Double-click it. Visual Studio makes `btnSignIn_Click` for you; the code goes inside it. |

The form's own properties come first. Click the title bar of the form in the designer, and the Properties window shows frmLogin at its top. Text is the words in the title bar. Size is the width and the height, with a comma between. FixedDialog means the window cannot be dragged bigger. AcceptButton is set last, after btnSignIn exists: it makes the Enter key press the Sign in button, so nobody has to reach for the mouse.



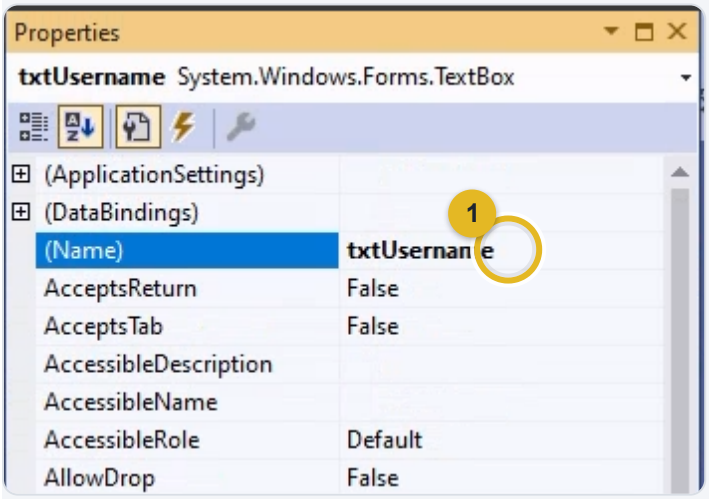

The form's own properties. Click the form's title bar first, so that the Properties window shows the form and not a control.

1. The box at the top names what is selected: frmLogin, the form.
2. Text is the words in the title bar.
3. Size is width then height, in pixels, with a comma between.
4. FormBorderStyle decides whether the window can be resized.




Drag a control from the Toolbox on the left and drop it on the form in the designer.

1. Find the control in the Toolbox.
2. Drag it on to the form and let go.




Set (Name) first, before anything else. It is at the top of the list, in brackets.

1. Click the box beside (Name), type the name from the table and press Enter.

**Name first, every time**

Set the Name of each control before you do anything else to it, and before you double-click it. The code in this guide calls the controls by these exact names. If a button is double-clicked while it is still called button1, the designer makes a method called button1_Click, and the pasted code will not match it.

**IF IT GOES WRONG** The Properties window shows a control, not the form

Click on the title bar of the form in the designer, or pick frmLogin from the drop-down box at the top of the Properties window.

**IF IT GOES WRONG** The password box shows letters, not dots

Click txtPassword, find UseSystemPasswordChar in the Properties window, and set it to True.

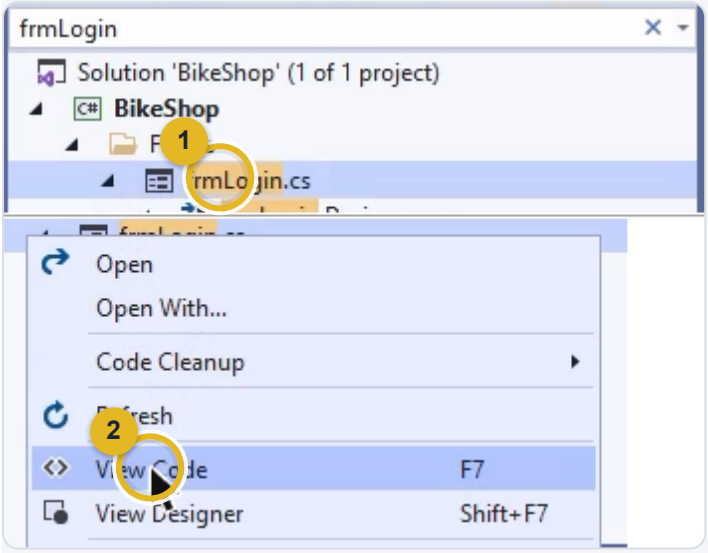
**IF IT GOES WRONG** lblMessage is too narrow for a sentence

Click it, find AutoSize in the Properties window, and set it to False. Then drag its right edge out to the width of the form.

# Put the code behind the sign-in form.

Double-clicking the Sign in button in the last screen opened the code tab on an empty method called btnSignIn_Click. The whole file is below. You paste it over everything in the code tab, including that empty method, and then read the SignIn region piece by piece.

Click the frmLogin.cs tab, or press F7 while the designer is in front.




View Code opens the form's code. Pressing F7 on the designer does the same.

1. Right-click the form in Solution Explorer.

2. Click View Code, or press F7.

> **You should see** The code tab for the form comes to the front.

Press the Copy button under this step.

> **You should see** The button reads Copied for a moment.

Click in the code tab and press Ctrl and A together to select everything.

> **You should see** Everything in the code tab is highlighted.

Press Ctrl and V together to paste the copied code over it.

> **You should see** The code tab holds the whole file, starting with the using lines.

Press F6 to build.

> **You should see** The Error List at the bottom shows one error: "The type or namespace name 'frmMain' could not be found". That is expected, and you fix it in stage 6.

**IF IT GOES WRONG** 'frmLogin' does not contain a definition for 'button1_Click' (or another old name)

The event was made before the control was given its Name, so the designer still calls the method by its old name. Double-click the error: Visual Studio 2019 opens frmLogin.Designer.cs on the line that hooks up the event. At the end of that line, change the old name (such as button1_Click) to the control's Name followed by _Click: btnSignIn_Click. Then build again.

**IF IT GOES WRONG** 'The name lblMessage does not exist' (or txtUsername, txtPassword)

A control on the form has a different Name from the table. Go back to the designer, click the control, and set its Name to match the table exactly, including capital letters.

**IF IT GOES WRONG** 'The type or namespace name frmMain could not be found'

That is expected until stage 6, where you add frmMain. Carry on to the Program.cs screen, and build again at the end of stage 6.

Copy this one **Forms/frmLogin.cs: the whole file** C#

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
using System;
using System.Collections.Generic;
using System.Data;
using System.Data.SqlClient;
using System.Drawing;
using System.Windows.Forms;
using BikeShop.Data;
using BikeShop.Models;

namespace BikeShop.Forms
{
    public partial class frmLogin : Form
    {
        public frmLogin()
        {
            InitializeComponent();
        }

        #region SignIn
        private void btnSignIn_Click(object sender, EventArgs e)
        {
            lblMessage.Text = "";
            string username = txtUsername.Text.Trim();
            string password = txtPassword.Text;

            if (username == "" || password == "")
            {
                lblMessage.Text = "Type your username and your password.";
                return;
            }

            // One row comes back when the details match an active member of
            // staff; no rows when they do not.
            // If SQL Server itself cannot be reached, say so instead of
            // crashing.
            DataTable match;
            try
            {
                match = StaffData.Login(username, password);
            }
            catch (SqlException)
            {
                lblMessage.Text =
                    "Could not reach the database. Is SQL Server running?";
                return;
            }
            if (match.Rows.Count == 0)
            {
                lblMessage.Text =
                    "Those details are not recognised. Check them and try " +
                    "again.";
                txtPassword.Clear();
                txtPassword.Focus();
```

```
54                  return;
55              }
56
57              DataRow staff = match.Rows[0];
58              Session.SignIn((int)staff["StaffID"], (string)staff["FullName"],
59                  (string)staff["Role"]);
60
61              frmMain main = new frmMain();
62              // so we know when the main window closes
63              main.FormClosed += Main_FormClosed;
64              Hide();
65              main.Show();
66          }
67          #endregion
68
69          // The main window has closed.  After Sign out we come back here; after
70          // the X the program ends.
71          private void Main_FormClosed(object sender, FormClosedEventArgs e)
72          {
73              if (Session.IsSignedIn)
74              {
75                  Close();
76              }
77              else
78              {
79                  txtPassword.Clear();
80                  Show();
81                  txtUsername.Focus();
82              }
83          }
84      }
85  }
```

**1 The using lines**

Lines 1 to 8

The using lines a form needs: System.Windows.Forms for the window itself, System.Data for the table that comes back from the database, System.Data.SqlClient for the error that means SQL Server could not be reached, and your own Data and Models folders.

**2 The class and the constructor**

Lines 10 to 17

The form is a class called frmLogin, inside the namespace BikeShop.Forms. partial means the class is split over two files: this one, and the Designer file that Visual Studio 2019 writes when you drag controls on. The method with the same name as the class runs when the window is made, and InitializeComponent is the designer's own method that builds the controls.

**3 SignIn: read the two boxes**

Lines 19 to 24

This is the method the designer made when you double-clicked Sign in, so it runs on every click. The red label is cleared first. Then the two boxes are read into two strings, which is what C# calls text. Trim takes spaces off either end of the username, because a space typed by mistake should not stop a sign-in.

**4 Refuse two empty boxes**

Lines 26 to 30

The two vertical bars mean or. If either box is empty, a sentence goes into the red label and return stops the method there, so the database is never asked. This is the one check the form does itself, because an empty box is something the form can see.

### 5 Ask the database, or say it cannot be reached

Lines 32 to 46

StaffData.Login runs your stored procedure and the table that comes back goes into match. It is wrapped in try and catch because this is the first time the program touches SQL Server: if the server is not running, the program should say so in the red label instead of crashing. This is the only try and catch any form in this guide has, because Db handles every other error for you.

### 6 Refuse when the database sent no rows

Lines 47 to 55

Rows.Count is how many rows came back. Zero rows means the stored procedure found no active member of staff with those details, so the form shows the sentence, empties the password box, puts the cursor back in it, and stops. The form does not know why the database said no, and it does not need to: the deciding was done in usp_Staff_Login.

### 7 Remember who signed in

Lines 57 to 59

One row came back, and Rows[0] is the first row, because C# counts from zero. Its three columns are read by name and passed to Session.SignIn. The (int) and (string) in brackets say what kind of value each column holds, because a DataRow does not know until it is told.

### 8 Open the main menu

Lines 61 to 67

A new main menu window is made. FormClosed is an event: the line with += says that when the main menu closes, run Main_FormClosed below. Then this window hides itself and the main menu is shown. The sign-in window is not closed, because closing the first window of a program ends the program.

### 9 When the main menu closes

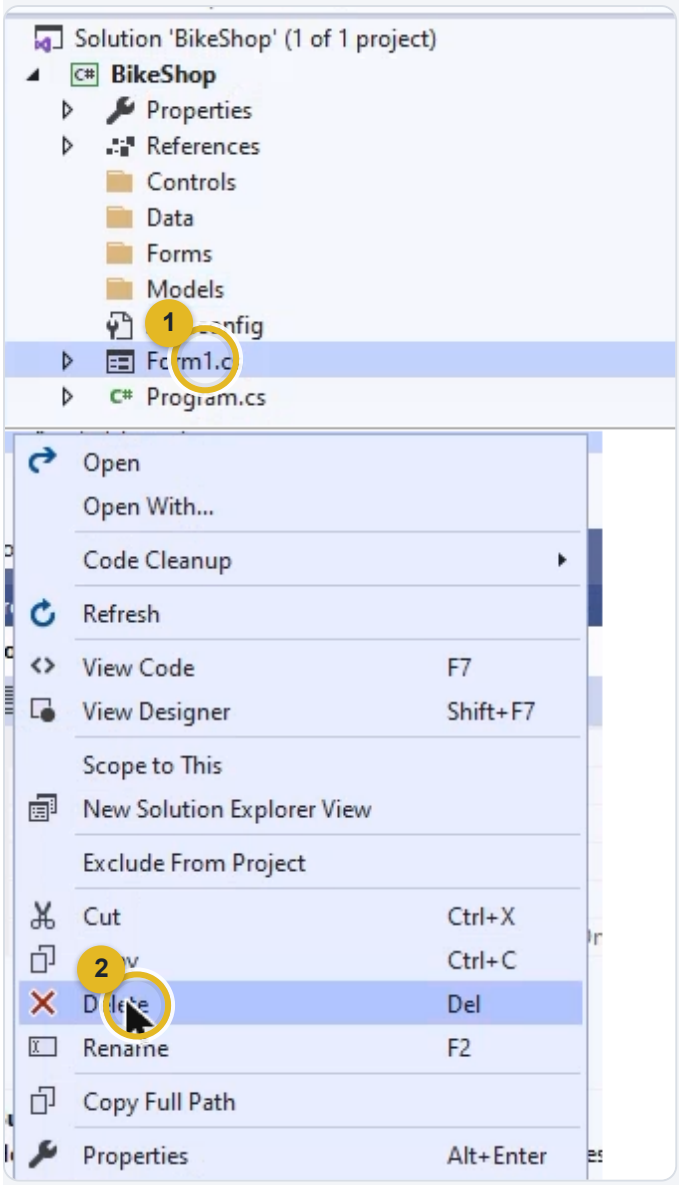
Lines 69 to 85

Two ways the main menu can close. After Sign out, Session is empty, so IsSignedIn is false, and the sign-in window empties the password box and shows itself again, ready for the next person. After the X in the corner, Session still holds the person, so the sign-in window closes too, and the program ends.

# Delete Form1 and tell Program.cs to open your sign-in window.

The empty window called Form1 that came with the project has done its job. Program.cs, the file the program starts in, still says to open Form1 first. In this screen you delete Form1 and change that one line so that frmLogin opens first instead. The whole of Program.cs is below, and the line that opens the window is the one you type yourself.

In Solution Explorer, right-click Form1.cs and click Delete, then press OK.




Right-click Form1.cs and click Delete. Your sign-in window takes its place.

1. Right-click Form1.cs.

2. Click Delete, then OK.

> **You should see** Form1.cs is gone from Solution Explorer.

In Solution Explorer, double-click Program.cs.

> **You should see** Program.cs opens in a tab, with a method called Main in the middle of it.

Press the Copy button under this step. It copies Program.cs with a gap where the yellow line goes.

> **You should see** The button reads Copied for a moment.

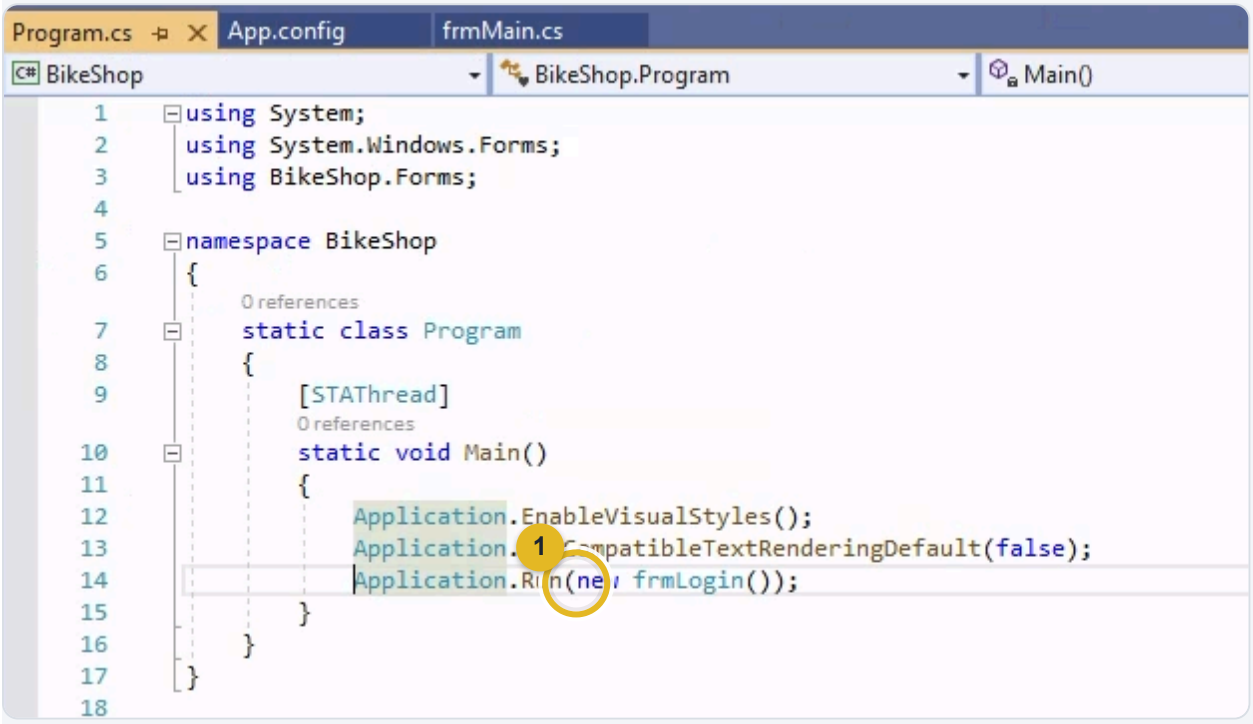
Click in the code tab and press Ctrl and A together to select everything.

> **You should see** Everything in the code tab is highlighted.

Press Ctrl and V together to paste the copied code over it.

> **You should see** The code tab holds the whole file, with one line that says YOU TYPE THIS PART.

Select the whole line that says YOU TYPE THIS PART and type the yellow line over it.




Program.cs after your typed line. The window named inside Application.Run is the first window the program opens.

1. The line you typed, which names frmLogin inside Application.Run.

> **You should see** The yellow line is in your file, and no line says YOU TYPE THIS PART any more.

**IF IT GOES WRONG** A red wavy line under a word

A spelling slip. Compare your line with the yellow line, letter by letter, and check that it ends with a semicolon.

**IF IT GOES WRONG** 'The type or namespace name Form1 could not be found'

The old line is still there. Find the line with Form1 in it and replace it with the yellow line.

**Type this one** **Program.cs: the whole file** C#

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  using System;
2  using System.Windows.Forms;
3  using BikeShop.Forms;
4
5  namespace BikeShop
6  {
7      static class Program
8      {
9          [STAThread]
10         static void Main()
11         {
12             Application.EnableVisualStyles();
13             Application.SetCompatibleTextRenderingDefault(false);
14             Application.Run(new frmLogin());
15         }
16     }
17 }
```

**1 The using lines**

Lines 1 to 3

The third using line is new: BikeShop.Forms is where frmLogin lives, and without this line Program.cs would not know the name.

**2 Main: where the program starts**

Lines 5 to 11

Every C# program has one method called Main, and the program starts by running it. STAThread above it is a setting every Windows Forms program carries, and you never change it.

**3 Two lines that came with the project**

Lines 12 to 13

These two lines make the buttons and boxes look the way Windows draws them today. They were in the file when Visual Studio 2019 made it, and they stay.

**4 The window that opens first**

Lines 14 to 17

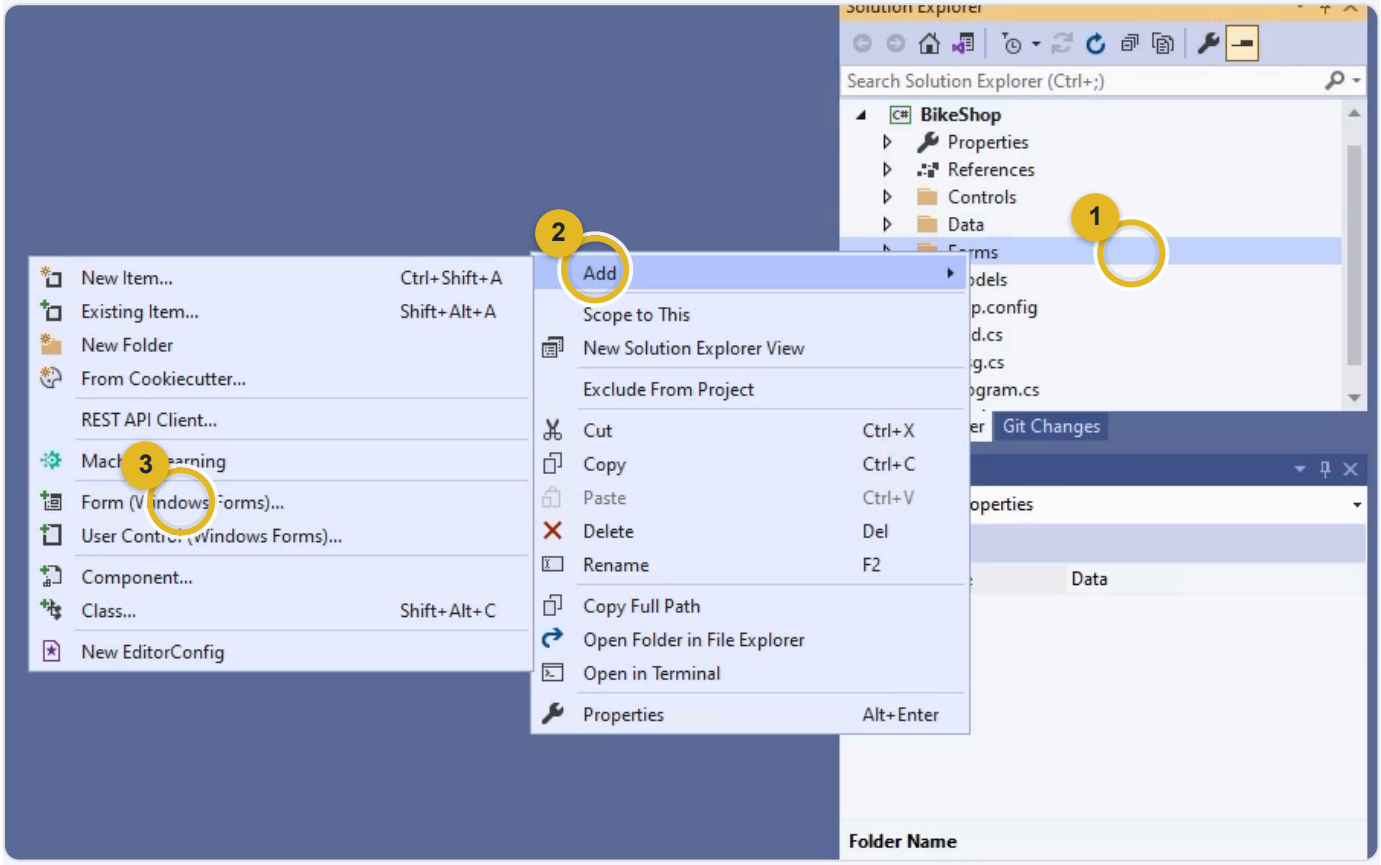
Application.Run opens one window and keeps the program running until that window closes. The window named inside the brackets is the first window the program shows. It was Form1. Now it is a new frmLogin, your sign-in window, and when the sign-in window closes, the program ends.

**Why you type it** This is the one line that decides which window a program opens first, so it is worth typing with your own hands: every project you ever make will have it.

# Add six empty forms, so that everything builds.

The main menu in the next stage opens six other windows, one per button. Those windows are built in later chapters, but the main menu will not build unless they exist by name. So you add six empty forms now, each with its exact name, and leave them empty. Chapters 6 to 9 fill them in.

In Solution Explorer, right-click the Forms folder, point at Add, and click Form (Windows Forms)… Type frmCustomers as the name and press Add.




Adding a form to the Forms folder. Taken in the finished project, so Solution Explorer lists more files than yours.

1. Right-click the Forms folder.
2. Point at Add.
3. Click Form (Windows Forms)…, type the name, press Add.

**You should see** An empty form opens in the designer, with frmCustomers in its title bar.

Do the same five more times, naming the forms frmDiary, frmHires, frmBikes, frmParts and frmReports.

> **You should see** The Forms folder in Solution Explorer lists seven forms: frmBikes, frmCustomers, frmDiary, frmHires, frmLogin, frmParts and frmReports.

**IF IT GOES WRONG** A form is called Form2

You pressed Add before typing the name. Right-click the file in Solution Explorer, click Rename, type the right name ending in .cs, press Enter, and say Yes when it offers to rename the class as well.

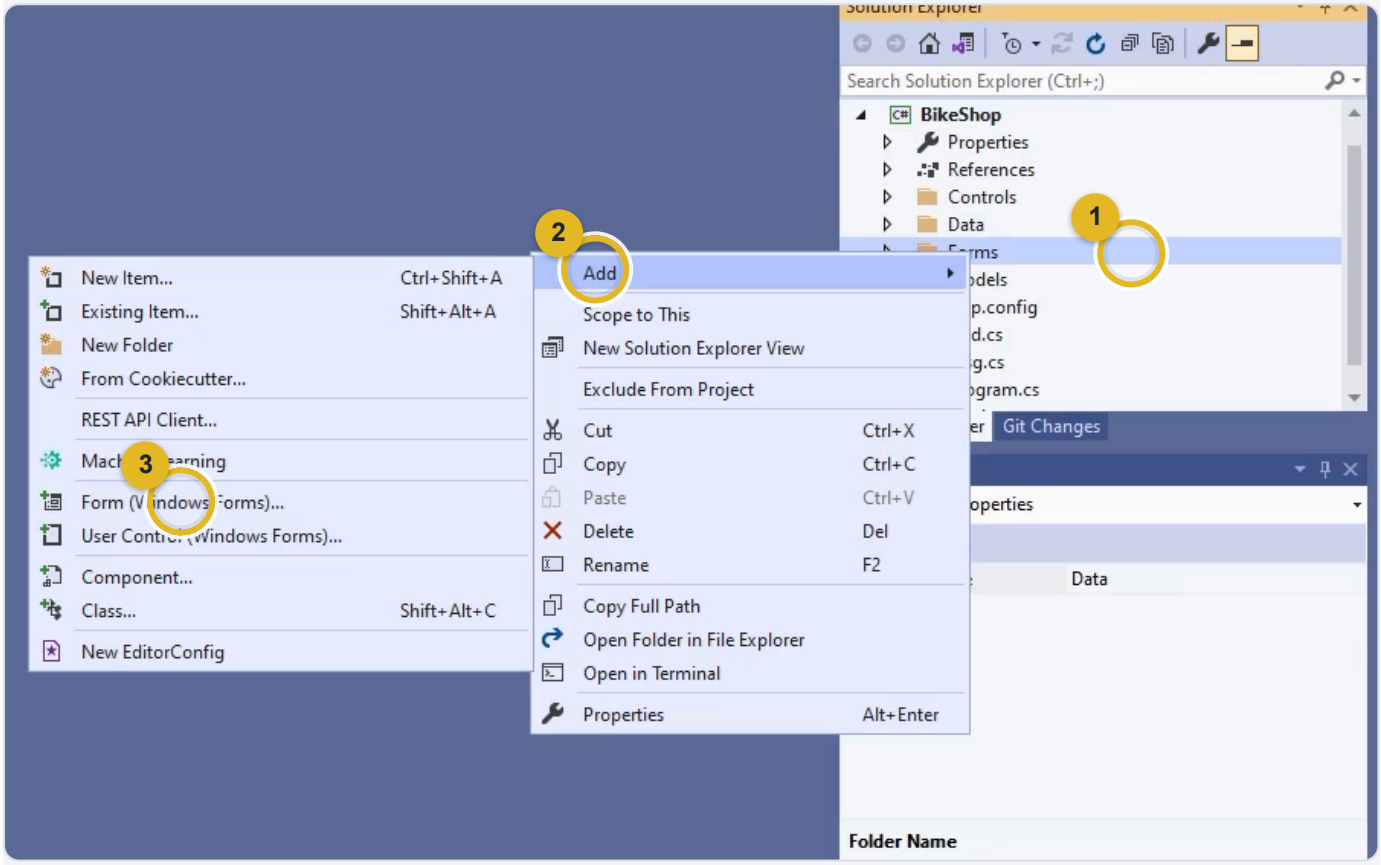
Close the six designer tabs you opened, by clicking the X on each tab.

> **You should see** Only the frmLogin tabs and Program.cs are left open.

# Add the main menu form and set its controls.

The main menu is the window that opens after a sign-in. It is a form in Visual Studio 2019 like frmLogin: add it, drag the controls on, name them, and paste the code. It has one new thing: a Load event, which runs when the window opens, before anyone clicks anything. That is where the menu reads Session and sets itself up for the person signed in.

In Solution Explorer, right-click the Forms folder, point at Add, and click Form (Windows Forms)… Type frmMain as the name and press Add.




Adding a form to the Forms folder. Taken in the finished project, so Solution Explorer lists more files than yours.

1. Right-click the Forms folder.
2. Point at Add.
3. Click Form (Windows Forms)…, type the name, press Add.

> **You should see** An empty form opens in the designer, with frmMain in its title bar.

Open the Toolbox (View, then Toolbox) and the Properties window (View, then Properties Window), if they are not already open.

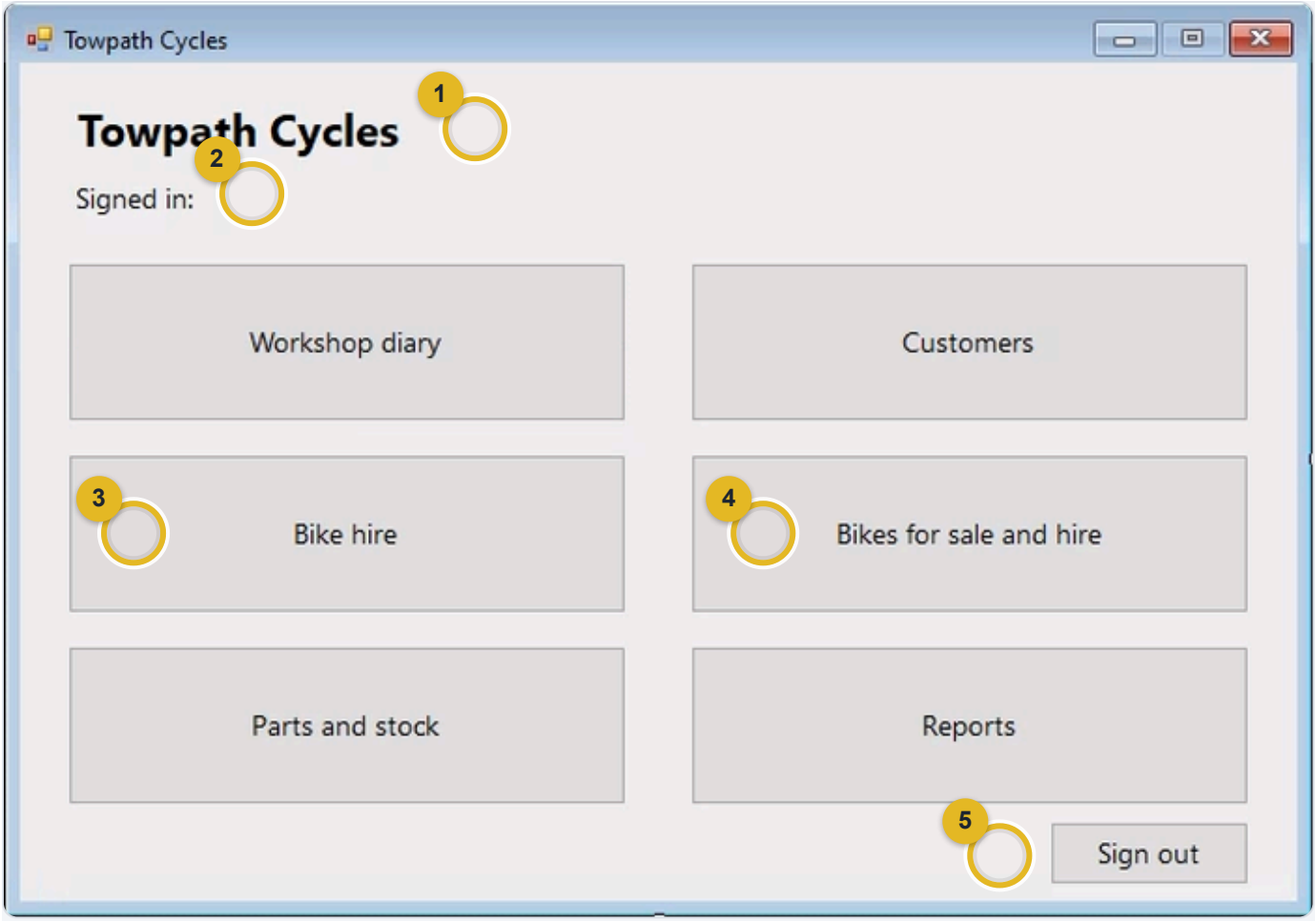
> **You should see** The Toolbox is on the left, the form in the middle, the Properties window on the right.

Click the form itself first and set the form's own properties. Then work down the table: drag the control from the Toolbox, drop it on the form, set its Name in the Properties window, and set the other settings in the row. Where the Then column says double-click, double-clicking the control makes its event method for you.

The menu. One big button for each part of the system; the Reports button is switched off unless the manager is signed in.

**Click the form itself, then set these in the Properties window**

| | | | |
|---|---|---|---|
| Text | `Towpath Cycles` | Size | `640, 420` |
| FormBorderStyle | `FixedSingle` | StartPosition | `CenterParent` |
| MaximizeBox | `False` | | |

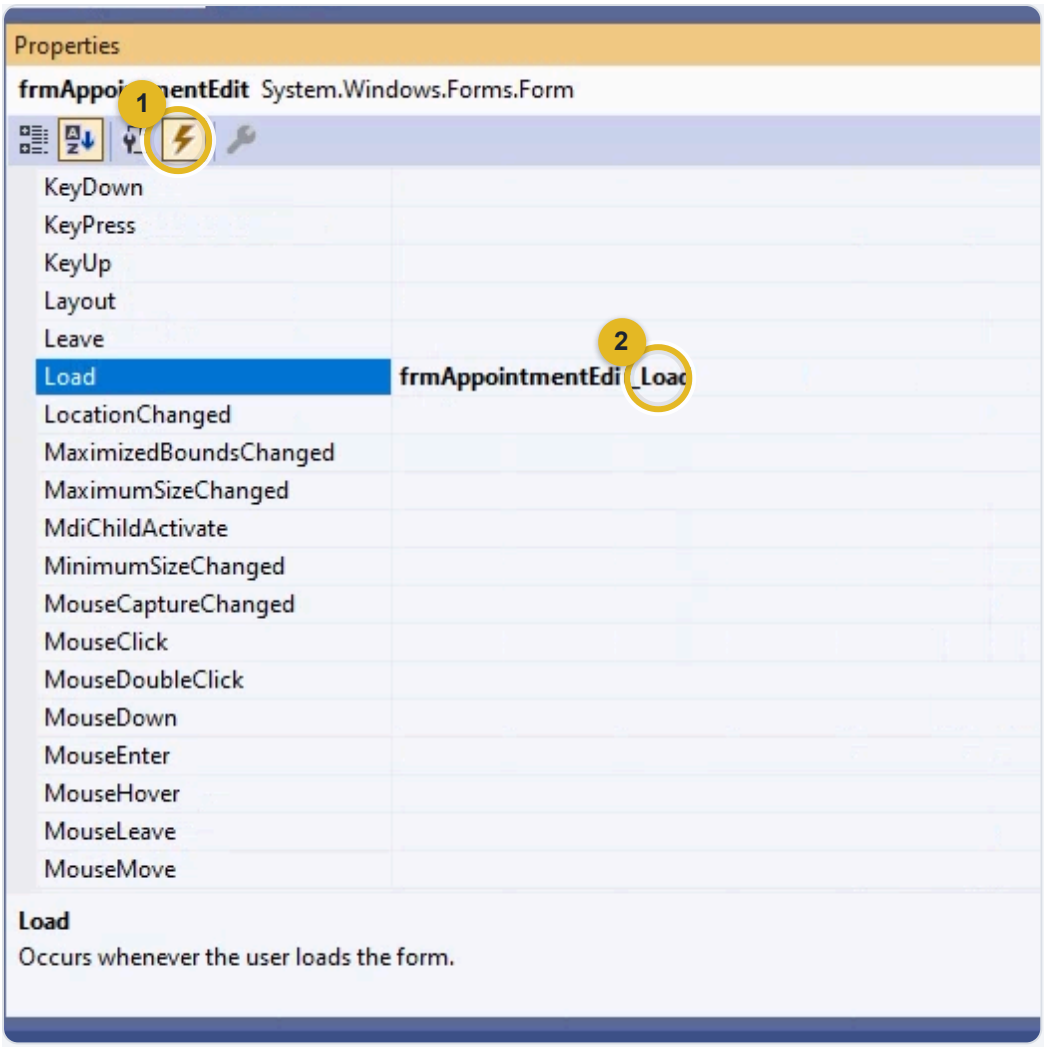



The main menu with every control in place. Match this layout: six buttons in two columns, Reports last, and Sign out on its own at the bottom.

1. lblWelcome, in 16 point bold.
2. lblSignedIn, which the code fills in when the window opens.
3. btnDiary, btnHires and btnParts down the left.
4. btnCustomers, btnBikes and btnReports down the right.
5. btnSignOut at the bottom right.

| DRAG THIS FROM THE TOOLBOX | NAME IT (THE NAME PROPERTY) | SET THESE IN THE PROPERTIES WINDOW | THEN |
|---|---|---|---|
| Label | `lblWelcome` | `Text = Towpath Cycles`<br>`Font = Segoe UI, 16pt, Bold` | |
| Label | `lblSignedIn` | `Text = Signed in:`<br>Filled in when the window opens: name and role. | |
| Button | `btnDiary` | `Text = Workshop diary`<br>Opens the day-by-day diary of repair bookings. | Double-click it. Visual Studio makes `btnDiary_Click` for you; the code goes inside it. |
| Button | `btnCustomers` | `Text = Customers` | Double-click it. Visual Studio makes `btnCustomers_Click` for you; the code goes inside it. |
| Button | `btnHires` | `Text = Bike hire` | Double-click it. Visual Studio makes `btnHires_Click` for you; the code goes inside it. |
| Button | `btnBikes` | `Text = Bikes for sale and hire` | Double-click it. Visual Studio makes `btnBikes_Click` for you; the code goes inside it. |
| Button | `btnParts` | `Text = Parts and stock` | Double-click it. Visual Studio makes `btnParts_Click` for you; the code goes inside it. |
| Button | `btnReports` | `Text = Reports`<br>Manager only. The code switches it off for everyone else. | Double-click it. Visual Studio makes `btnReports_Click` for you; the code goes inside it. |
| Button | `btnSignOut` | `Text = Sign out`<br>Closes this window and shows the sign-in window again. | Double-click it. Visual Studio makes `btnSignOut_Click` for you; the code goes inside it. |

The Load event is made from the form itself, not from a button. Click the title bar of the form, and in the Properties window press the lightning bolt at the top, which switches the window from properties to events. The events are listed in alphabetical order, so Load comes after Leave. Double-click the empty box beside Load, and the designer makes frmMain_Load for you.




The lightning bolt shows the events, in alphabetical order. Double-click the empty box beside one to make its method.

1. Press the lightning bolt.
2. Double-click the empty box beside the event named in the table. Load is part of the way down, just after Leave.

**IF IT GOES WRONG** Double-clicking the form made a method called frmMain_Load already

Double-clicking an empty part of a form makes its Load event, which is the one you want. Nothing to undo.

**IF IT GOES WRONG** The Reports button text is too long for the button

Make the button wider: click it and drag its right-hand edge out. The code changes its text to Reports (manager only) for non-managers, so leave room for that.

# Put the code behind the main menu.

Double-clicking the last button opened the code tab. Paste the whole file over everything there, build, and then read the Load method and the Open region below.

Click the frmMain.cs tab, or press F7 while the designer is in front.

> **You should see** The code tab for the form comes to the front.

Press the Copy button under this step.

> **You should see** The button reads Copied for a moment.

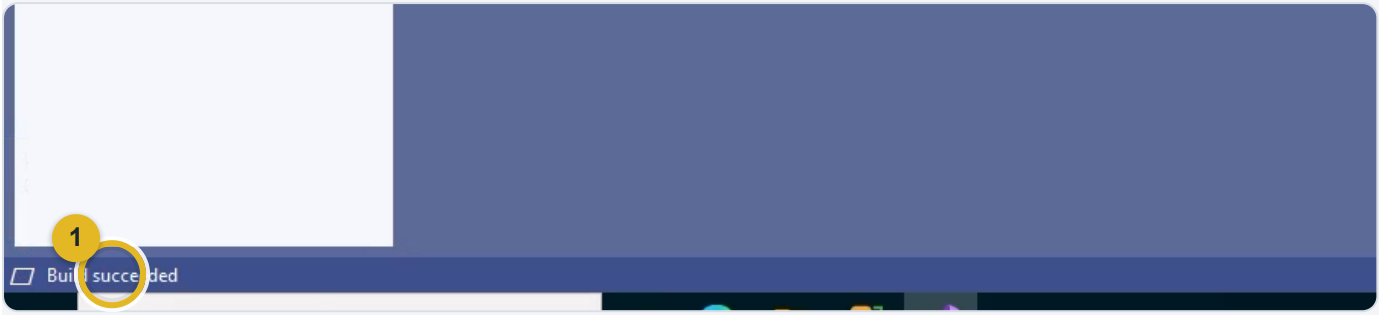
Click in the code tab and press Ctrl and A together to select everything.

> **You should see** Everything in the code tab is highlighted.

Press Ctrl and V together to paste the copied code over it.

> **You should see** The code tab holds the whole file, starting with the using lines.

Press F6 to build.




Build succeeded, at the bottom left. Anything else means an error list to read.

1 Build succeeded. Press F6 to build at any time.

**You should see** Build succeeded, at the bottom left.

**IF IT GOES WRONG** 'frmMain' does not contain a definition for 'button1_Click' (or another old name)

The event was made before the control was given its Name, so the designer still calls the method by its old name. Double-click the error: Visual Studio 2019 opens frmMain.Designer.cs on the line that hooks up the event. At the end of that line, change the old name (such as button1_Click) to the control's Name followed by _Click: btnDiary_Click. Then build again.

**IF IT GOES WRONG** 'The type or namespace name frmCustomers could not be found' (or another form name)

One of the six empty forms from the last stage is missing or spelled differently. Check the Forms folder in Solution Explorer against the list of seven names, and add or rename the one that is wrong.

**IF IT GOES WRONG** 'The name btnReports does not exist' (or another control name)

A control on the form has a different Name from the table. Go back to the designer, click the control, and set its Name to match the table exactly.

Copy this one **Forms/frmMain.cs: the whole file** C#

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
using System;
using System.Collections.Generic;
using System.Data;
using System.Drawing;
using System.Windows.Forms;
using BikeShop.Data;
using BikeShop.Models;

namespace BikeShop.Forms
{
    public partial class frmMain : Form
    {
        public frmMain()
        {
            InitializeComponent();
        }

        private void frmMain_Load(object sender, EventArgs e)
        {
            lblSignedIn.Text = "Signed in: " + Session.FullName + " ("
                + Session.Role + ")";

            // Only a manager sees the money.  The button stays visible so
            // everyone knows the report exists.
            btnReports.Enabled = Session.IsManager;
            if (!Session.IsManager)
            {
                btnReports.Text = "Reports (manager only)";
            }
        }

        #region Open
        private void btnCustomers_Click(object sender, EventArgs e)
        {
            frmCustomers customers = new frmCustomers();
            customers.ShowDialog(this);
        }

        private void btnDiary_Click(object sender, EventArgs e)
        {
            frmDiary diary = new frmDiary();
            diary.ShowDialog(this);
        }

        private void btnHires_Click(object sender, EventArgs e)
        {
            frmHires hires = new frmHires();
            hires.ShowDialog(this);
        }

        private void btnBikes_Click(object sender, EventArgs e)
        {
            frmBikes bikes = new frmBikes();
```

```
54              bikes.ShowDialog(this);
55          }
56
57          private void btnParts_Click(object sender, EventArgs e)
58          {
59              frmParts parts = new frmParts();
60              parts.ShowDialog(this);
61          }
62
63          private void btnReports_Click(object sender, EventArgs e)
64          {
65              frmReports reports = new frmReports();
66              reports.ShowDialog(this);
67          }
68          #endregion
69
70          private void btnSignOut_Click(object sender, EventArgs e)
71          {
72              Session.SignOut();
73              Close();        // frmLogin notices the close and shows itself again
74          }
75      }
76  }
```

**1 The using lines**

Lines 1 to 7

The same using lines as the sign-in form, without System.Data.SqlClient, because this form never touches the database itself.

**2 The class and the constructor**

Lines 9 to 16

The main menu is a partial class called frmMain, inside the namespace BikeShop.Forms. Like frmLogin, it has a method with the same name as the class that runs when the window is made, and InitializeComponent builds the controls you dragged on.

**3 Load: say who is signed in**

Lines 18 to 21

This is the Load event method the designer made, so it runs once when the window opens. It builds a sentence out of pieces: the words Signed in, the full name from Session, and the role in brackets. The plus signs join text together. The sentence goes into the label under the title.

**4 Load: switch Reports off for a non-manager**

Lines 23 to 30

Enabled is whether a button can be pressed. It is set to Session.IsManager, which is true for a manager and false for everyone else, so the button is greyed out for a counter worker or a mechanic. The exclamation mark means not: if the person is not a manager, the text of the button changes to say why it is off. The button stays visible, so everyone knows the reports exist.

**5 Open: the Customers button**

Lines 32 to 37

When Customers is clicked, a new frmCustomers window is made and shown with ShowDialog. ShowDialog means the main menu waits, and cannot be clicked, until that window is closed again. In C#, the word this means the main menu itself, and passing it in puts the new window in front of the menu.

### 6 Open: the other five buttons

Lines 39 to 68

Five more methods in exactly the same shape, one per button, each opening its own form. frmDiary, frmHires, frmBikes, frmParts and frmReports are the empty forms from stage 5, and the later chapters fill them in without changing a line here.

### 7 Sign out

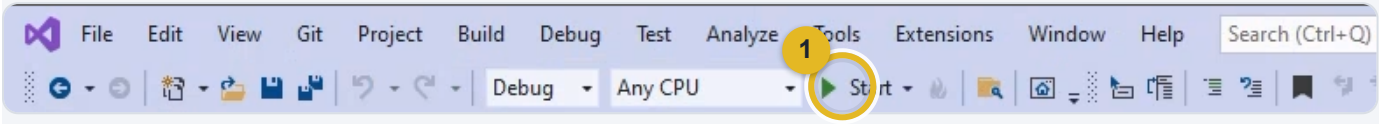
Lines 70 to 76

Session.SignOut clears who is signed in, and then the main menu closes itself. The sign-in window notices the close, sees that nobody is signed in any more, and shows itself again.

# Run the program and sign in as the manager.

This is the first time your program runs. F5 builds it and then starts it, which is why there was no need to build first. The sign-in window appears on its own, outside Visual Studio 2019. While the program is running, Visual Studio 2019 shows an orange bar at the bottom and will not let you edit code, so close the program before you change anything.

Press F5 to run your program.




Start runs your program. Close the program window to come back to Visual Studio.

1 Click Start, or press F5.

**You should see** The sign-in window opens, with Towpath Cycles — sign in in its title bar and the cursor in the Username box.

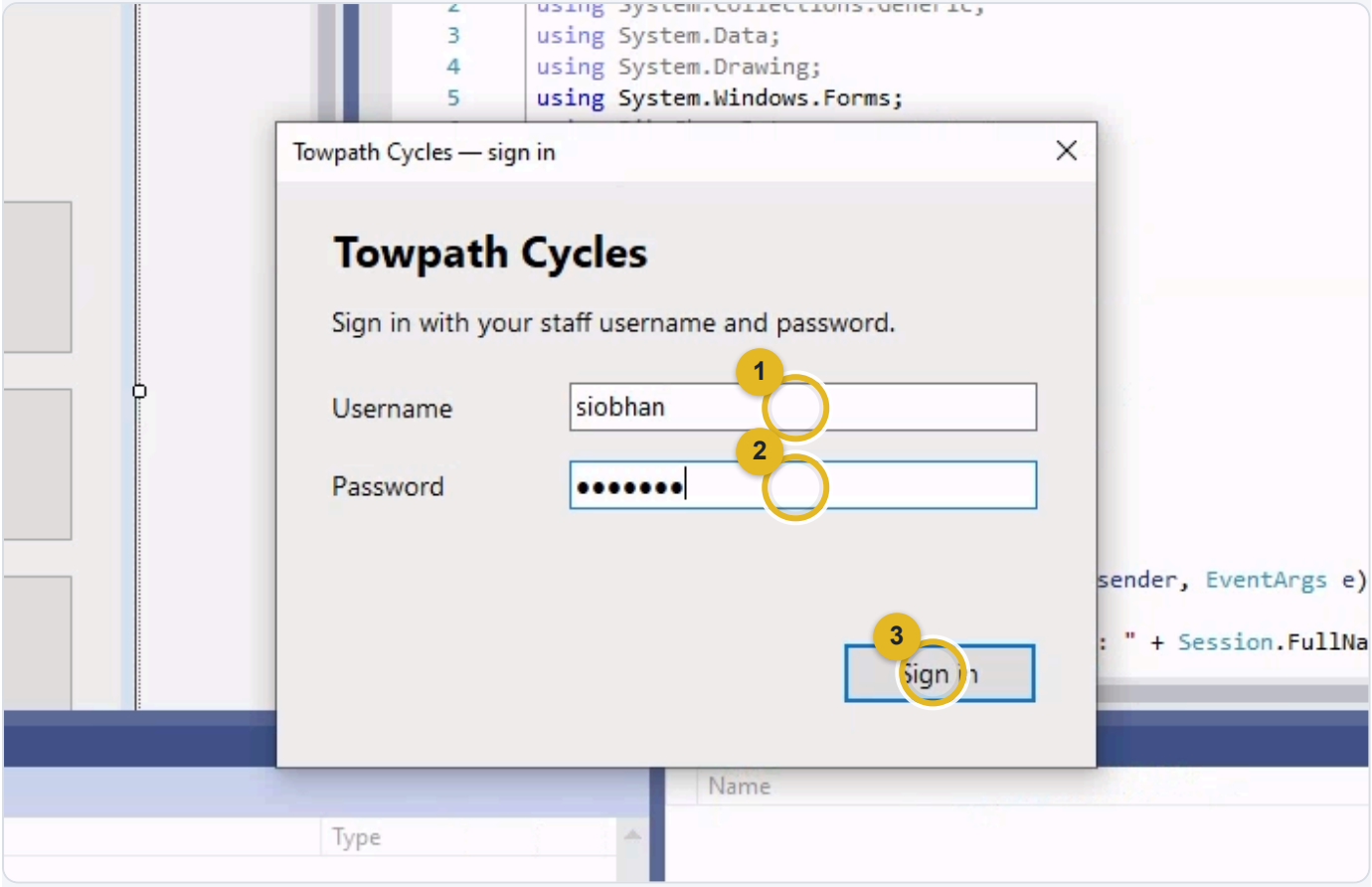
**IF IT GOES WRONG** A window says There were build errors. Would you like to continue

Press No. The Error List at the bottom names the problem. Double-click the first error to go to its line, fix it, and press F5 again.

**IF IT GOES WRONG** The window opens but the cursor is not in the Username box

Click in the box. Later, if you like, click the form in the designer and set its ActiveControl to txtUsername.

Type siobhan in the Username box, type canal01 in the Password box, and press Enter.




The sign-in window of the finished program.

1. Type the username.
2. Type the password.
3. Click Sign in.

> **You should see** The main menu opens, with Signed in: Siobhán Rooney (Manager) under the title, and the Reports button is switched on.

**IF IT GOES WRONG** The red label says Could not reach the database. Is SQL Server running?

The connection string in App.config does not match a running server. In SQL Server Management Studio (SSMS), check the server name you connect with, and make it the same as the part after Server= in App.config.

**IF IT GOES WRONG** The red label says Those details are not recognised

The username and password are both lower case with no spaces: siobhan and canal01. If they are right, open SQL Server Management Studio (SSMS) and run SELECT * FROM Staff to check that Siobhán Rooney is in the table with IsActive set to 1.

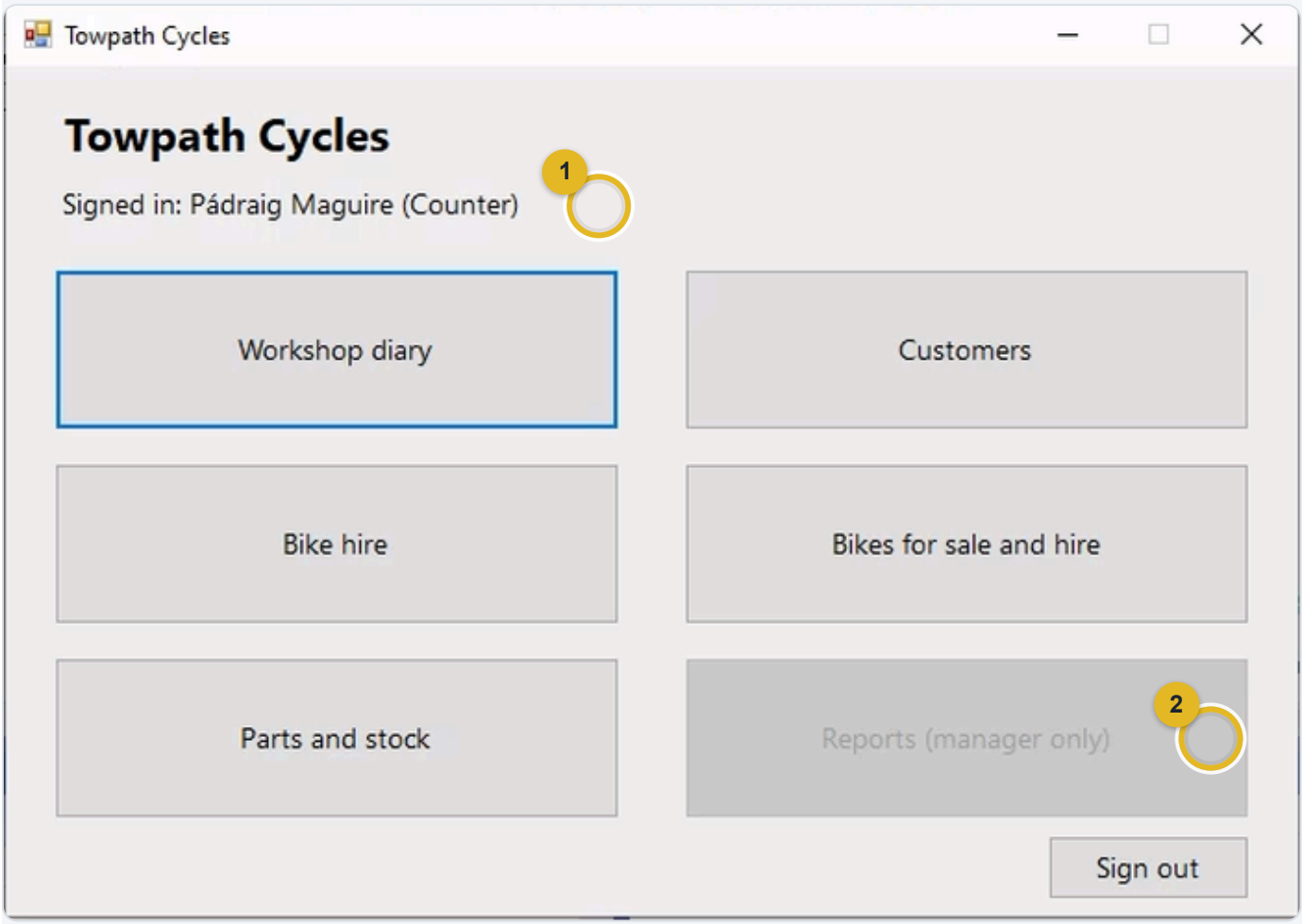
Press the Sign out button.

> **You should see** The main menu closes and the sign-in window comes back, with the password box empty.

# Sign in as a counter worker, and see fewer buttons.

Pádraig Maguire works the counter. His sign-in is allowed, because he is an active member of staff, but he is not a manager, so the main menu should switch the Reports button off for him and say why.

Type padraig in the Username box, type canal02 in the Password box, and press Enter.




The main menu for a counter worker. The Reports button is still there, so everyone knows the reports exist, but it is switched off and says why.

1. Signed in: Pádraig Maguire (Counter).
2. Reports (manager only), greyed out.

> **You should see** The main menu opens, with Signed in: Pádraig Maguire (Counter) under the title. The Reports button is greyed out and reads Reports (manager only).

Click the greyed-out Reports button.

> **You should see** Nothing happens. A switched-off button does not click.

Press Sign out, then close the sign-in window with the X in its corner.

> **You should see** Both windows are gone, and the orange bar at the bottom of Visual Studio 2019 disappears.

**Why the button stays visible**

A button that disappears leaves a gap, and a counter worker who has never seen it does not know the report exists. A button that is greyed out and says manager only tells the truth: the report is there, and it is not for you. The examiner looks for exactly this kind of difference between what two roles can do.

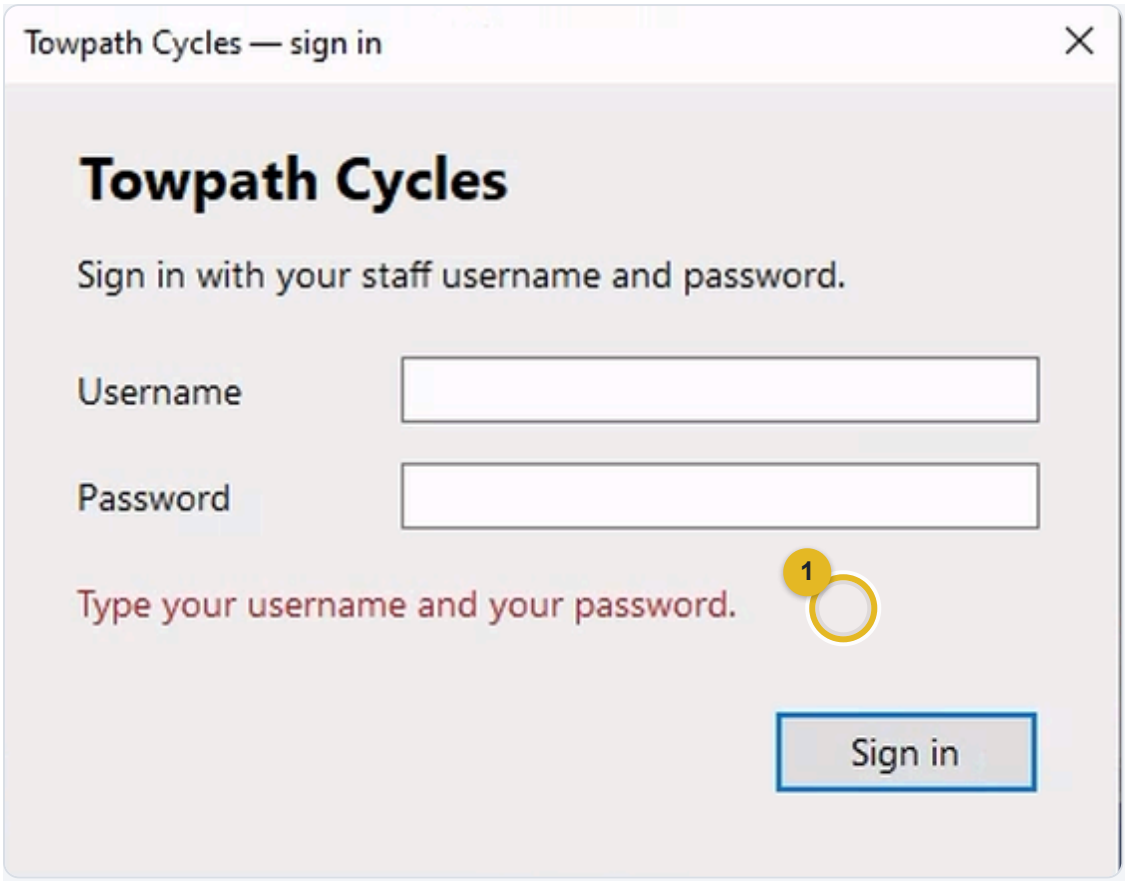
# Run each sign-in once.

The table on the next screen is evidence for your write-up: each row is one sign-in you tried and what the program did. Run your program from Visual Studio 2019 with F5, and work down the rows. Three rows are sign-ins that must be refused, and a refusal that happens is a pass, not a fail. Take a screenshot of each refusal, because chapter 10 asks for them.

The sixth member of staff, Liam Doran, left in the summer. His row in the Staff table is still there, with IsActive set to 0, so that the appointments he did still have a mechanic's name on them. Your stored procedure checks IsActive, so his old username and password should no longer get in. That is the database saying no, inside your program.

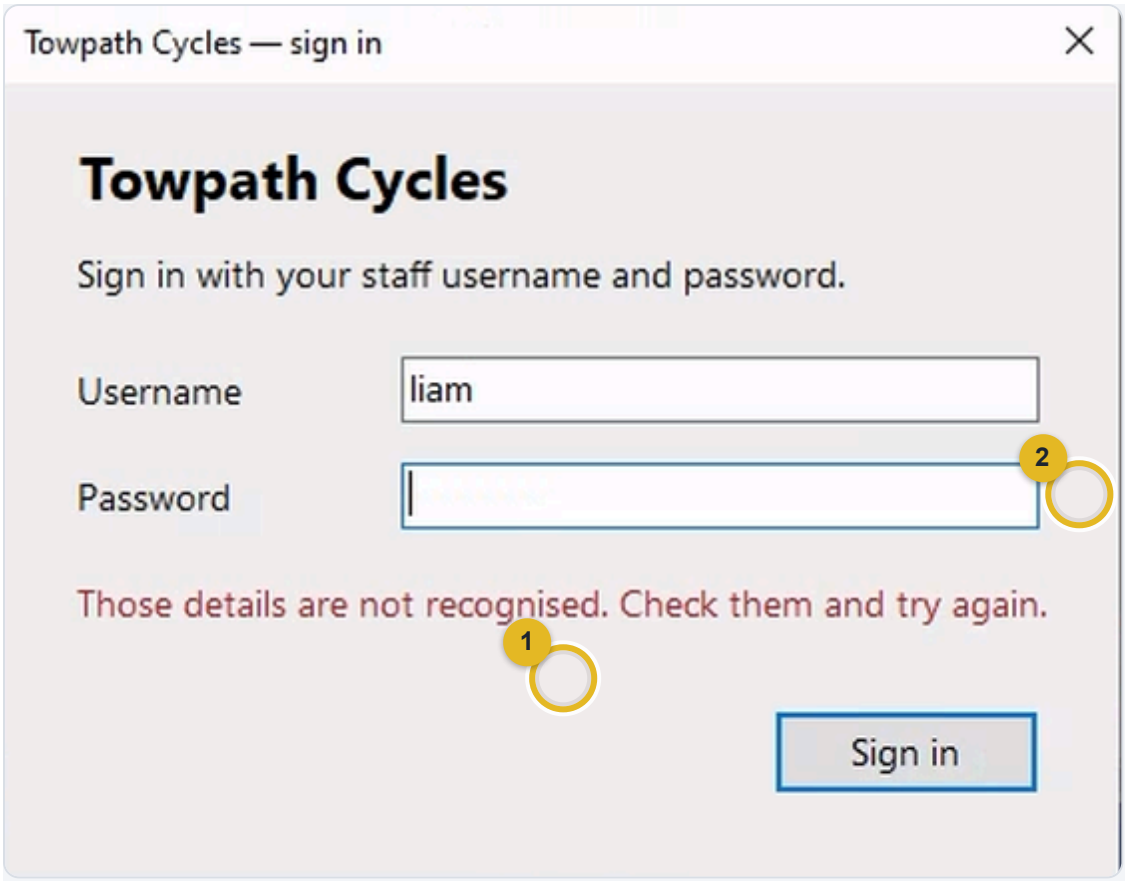
# Tick the table: four sign-ins, three refusals.

| | THE TEST | WHAT MUST HAPPEN | HOW TO DO IT, AND WHAT YOU SHOULD SEE |
|---|---|---|---|
| | The manager signs in and sees every button on | Must be allowed | Run the program, sign in as siobhan with canal01 and press Enter. The main menu opens with Signed in: Siobhán Rooney (Manager), and the Reports button is switched on. |
| | A counter worker signs in and sees Reports switched off | Must be allowed | Press Sign out, sign in as padraig with canal02 and press Enter. The main menu opens with Signed in: Pádraig Maguire (Counter), and the Reports button is greyed out and reads Reports (manager only). |
| | A mechanic signs in and sees Reports switched off | Must be allowed | Press Sign out, sign in as aoife with canal03 and press Enter. The main menu opens with Signed in: Aoife Byrne (Mechanic), and the Reports button is greyed out. |
| | Sign out brings the sign-in window back with the password box empty | Must be allowed | Press Sign out. The main menu closes and the sign-in window comes back with the password box empty and the cursor in the Username box. |
| | Two empty boxes are refused before the database is asked | Must be refused | Leave both boxes empty and press Enter. The red label says "Type your username and your password." and the window stays open. |
| | A wrong password is refused | Must be refused | Type siobhan in the Username box, type canal99 in the Password box, and press Enter. The red label says "Those details are not recognised. Check them and try again." and the password box is emptied. |
| | A member of staff who has left is refused | Must be refused | Type liam in the Username box, type canal06 in the Password box, and press Enter. The red label says Those details are not recognised, even though both are the right words, because Liam Doran has IsActive set to 0. |




Sign in with both boxes empty. The program refuses before it asks the database anything.

1 The red message appears here and the window stays open.




Sign in as liam with canal06. The stored procedure sends back no rows, because Liam Doran is marked as left, so the program refuses and empties the password box.

1. The database said no, and the program turned that into a sentence.
2. The password box is emptied and the cursor is put back in it.

**IF IT GOES WRONG** liam with canal06 gets in

Your usp_Staff_Login is missing the line AND IsActive = 1. Open BikeShop.sql in SQL Server Management Studio (SSMS), find the stored procedure, add the line to its WHERE part, select the stored procedure and press F5. Then run your program again.

**IF IT GOES WRONG** The wrong password gets in

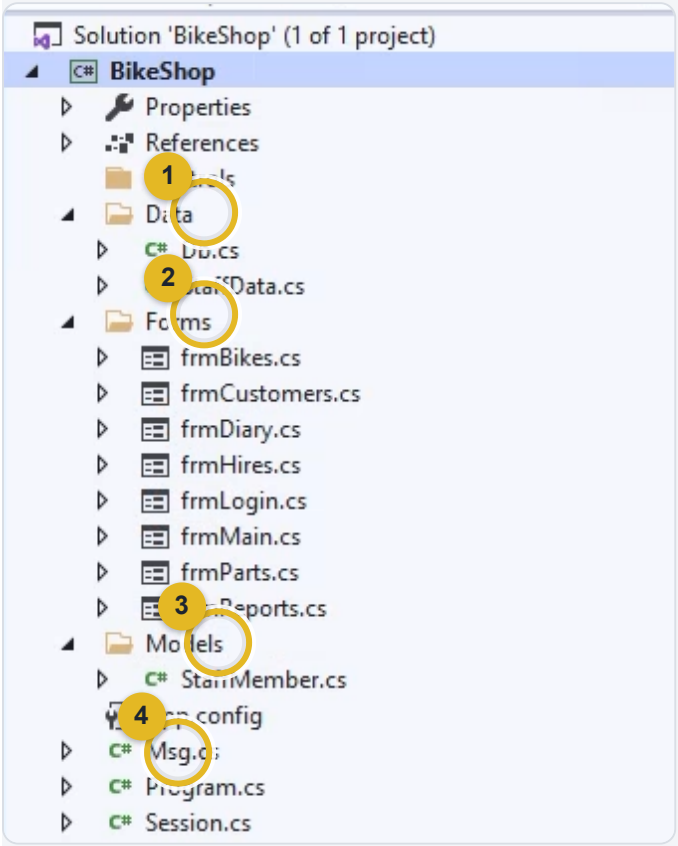
Your usp_Staff_Login is not checking the password. Open it in SQL Server Management Studio (SSMS) and check that its WHERE part has Username = @Username AND Password = @Password.

**IF IT GOES WRONG** Every sign-in says Could not reach the database

The program cannot find SQL Server. Check that SQL Server Management Studio (SSMS) can connect to the same server name that App.config has after Server=, and that the database there is called BikeShop.

# Finish the chapter.

Press Ctrl and Shift and S together in Visual Studio 2019 to save every file. Your project in Solution Explorer should now look like the picture below: two files in Data, eight forms in Forms, one file in Models, and Msg.cs and Session.cs at the top level beside App.config and Program.cs.




Your project at the end of this chapter, taken in the finished project. Your Solution Explorer should list the same files.

1. Data: Db.cs and StaffData.cs.
2. Forms: eight forms.
3. Models: StaffMember.cs.
4. Msg.cs and Session.cs at the top level, beside App.config and Program.cs.

**Before you start**

- App.config holds a connection string named BikeShop, and System.Configuration is under References.
- Db.cs builds, and nothing else in the project opens a connection.
- siobhan with canal01 gets in and sees Reports switched on; padraig with canal02 sees it switched off.
- liam with canal06 is refused, and two empty boxes are refused.

- Screenshots of the three refusals are saved for chapter 10.

**Make it yours**

In your own project, the sign-in table is whatever your scenario calls its staff, and the roles are whatever your scenario needs: a manager and a receptionist, or an owner and a groomer. Keep the shape exactly: one stored procedure that checks the username, the password and an active flag; one Session class that remembers who is signed in; and one button on the main menu that only one role can press. Db.cs and Msg.cs do not change at all, and App.config changes only the database name.

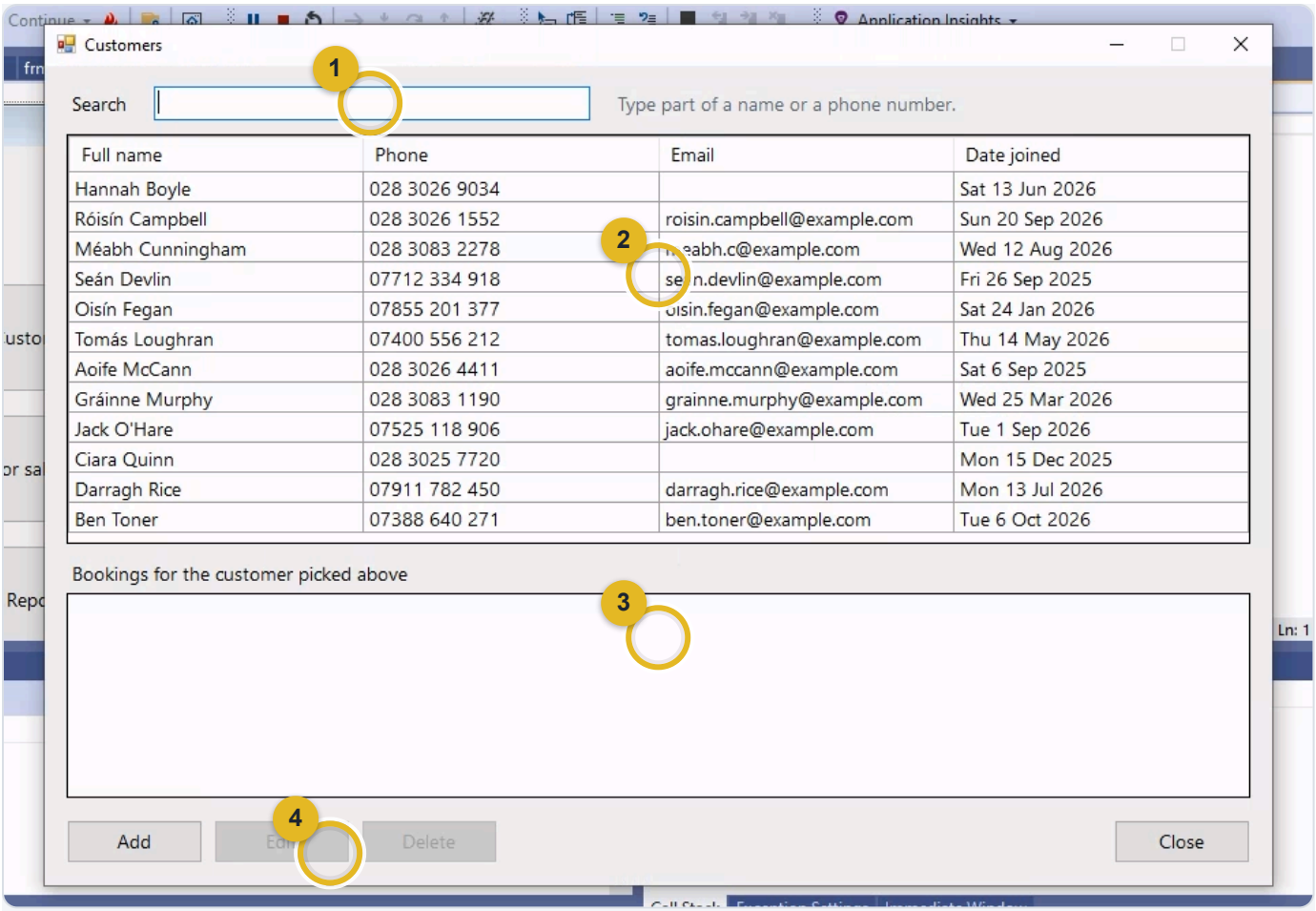
# Add, edit, delete, display

The customers screen: a grid of names and an edit form that checks its boxes.

STAGE 1 OF 7 · ONE CUSTOMER IN YOUR HANDS · SCREEN 1 OF 6

## See what you will build in this chapter.

Chapter 5 gave your program a front door: a sign-in window and a main menu with six buttons that open six empty forms. In this chapter the Customers button opens a real screen. A grid lists every customer. A Search box narrows the grid as you type. Add opens a small form, Edit opens the same form with the customer already in it, and Delete asks before it acts. The database still makes the final decisions: it refuses to delete a customer who has bookings, and the refusal appears on the screen in its own words.




The customers screen as it opens. Nothing is picked yet, so Edit and Delete are switched off and the bookings grid is empty.

1. The Search box. The grid narrows as you type.
2. Twelve customers, sorted by last name. The CustomerID column is hidden.
3. The bookings grid: empty until a customer is picked.
4. Edit and Delete are off until a row is picked.

Five files do the work, and you meet them in order. Customer.cs holds one customer. CustomerData.cs calls the five customer stored procedures from chapter 4. Grid.cs tidies any grid. frmCustomers.cs is the list. frmCustomerEdit.cs is the add and edit form. At the end, two small classes for the services and the suppliers follow the same pattern, and you write the Supplier pair yourself.

### Words used in this chapter

| | |
|---|---|
| `Class` | A description of one kind of thing, written as code. The Customer class describes what one customer has: a number, two names, a phone, an email and a date. |
| `Object` | One real thing made from a class. The class is the plan; the object is one customer, with real names in it. |
| `Property` | One piece of information inside a class, such as FirstName. The program can read a property and, usually, change it. |

| | |
|---|---|
| `Method` | A named piece of code that does one job, such as Search or Tidy. Writing the name with brackets after it runs the job. |
| `Constructor` | The method that runs when an object is made with the word new. A form's constructor can take a value, and this chapter's edit form takes the customer's number. |
| `Static` | A class marked static has no objects made from it. You call its methods by the class name, such as CustomerData.Search, the way you called Db in chapter 5. |
| `Grid` | The DataGridView control: a table on a form, with a heading row and one row per record. In the code it is called dgvCustomers. |
| `DataTable` | A table of rows held in memory after the database answered. Db.GetTable gives one back, and a grid can show it. |
| `Event` | Something that happens to a control: a click, a key typed, a row picked. An event method is the code that runs when that happens. |
| `null` | The value that means nothing is there at all. An empty email box becomes null, so the database stores NULL and not an empty piece of text. |
| `bool` | A value that is either true or false. The form uses one called picked to switch two buttons on or off. |
| `DialogResult` | The answer a small window gives when it closes: OK or Cancel. The list form waits for it, and loads the grid again on OK. |

# Check that you have these ready.

**Before you start**

- Chapter 5 is finished: your program signs in as siobhan with the password canal01 and shows the main menu, and the six empty forms are in the Forms folder with the exact names frmCustomers, frmDiary, frmHires, frmBikes, frmParts and frmReports.
- Chapter 4 is finished: usp_Customer_Search, usp_Customer_GetByID, usp_Customer_Add, usp_Customer_Update, usp_Customer_Delete and usp_Appointment_GetForCustomer are in your database. If you are not sure, run the whole script once in SQL Server Management Studio (SSMS) and look for the line that says BikeShop is ready.
- The BikeShop project is open in Visual Studio 2019, with Solution Explorer showing the Data, Models, Forms and Controls folders.

Every file in this chapter goes into the Models folder, the Data folder, the Forms folder or the project itself, and the table at the start of each stage says which. Press F6 after each file, so a slip is found while it is still one file old.

# A class holds one customer in the program.

In chapter 4, a customer was one row of the Customer table: a number, two names, a phone, an email and a date. Inside the program a customer needs a shape too, so that the form can hand one whole customer to the data class in a single go, instead of five separate values that could be passed in the wrong order. That shape is a class.

A class is a description of one kind of thing. It lists the pieces of information the thing has, and each piece is called a property. One real customer made from the class is called an object. The class below has one property for each column of the Customer table, and one extra property, FullName, which works out its own answer from the two names.

Copy this one **Models/Customer.cs: one customer** C#

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  using System;
2
3  namespace BikeShop.Models
4  {
5      // One customer.  Plain properties only: the form checks what the user typed
6      // before it fills these in.
7      public class Customer
8      {
9          public int CustomerID { get; set; }
10         public string FirstName { get; set; }
11         public string LastName { get; set; }
12         public string Phone { get; set; }
13         public string Email { get; set; }
14         public DateTime DateJoined { get; set; }
15
16         public string FullName
17         {
18             get { return FirstName + " " + LastName; }
19         }
20     }
21 }
```

**1 The class**

Lines 1 to 8

using System gives the file the DateTime type, which holds a date and a time together. namespace BikeShop.Models is the folder the class lives in, and the other files reach it with using BikeShop.Models. public class Customer starts the description.

**2 One property for each column**

Lines 9 to 14

Each line is a property with the same name and the same kind of value as its column in the Customer table: int for a whole number, string for text, DateTime for a date. { get; set; } means the program can read the property and change it.

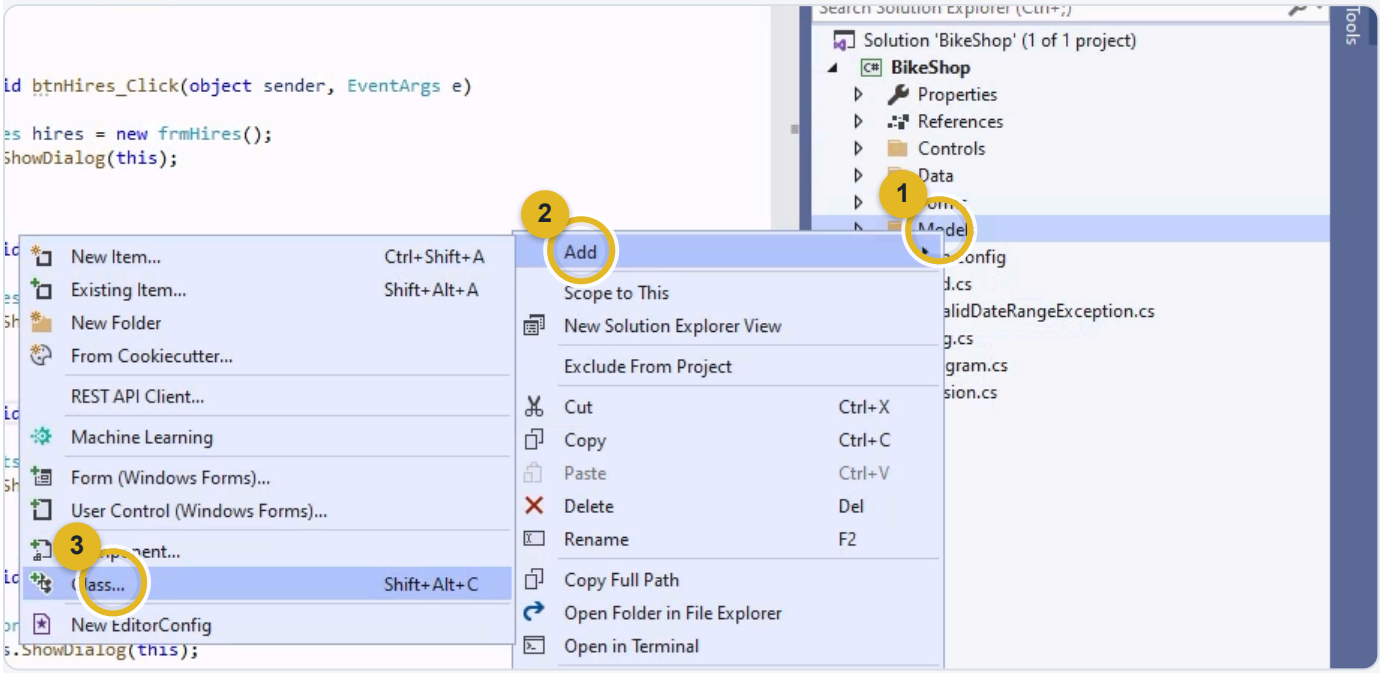
**3 A property that works out its own answer**

Lines 16 to 21

FullName has only a get, so the program can read it but never set it. Every time it is read, it joins the first name, a space and the last name. The two closing braces end the class and the namespace.

# Add the Customer class to the Models folder.

In Solution Explorer in Visual Studio 2019, right-click the Models folder, point at Add, and click Class…

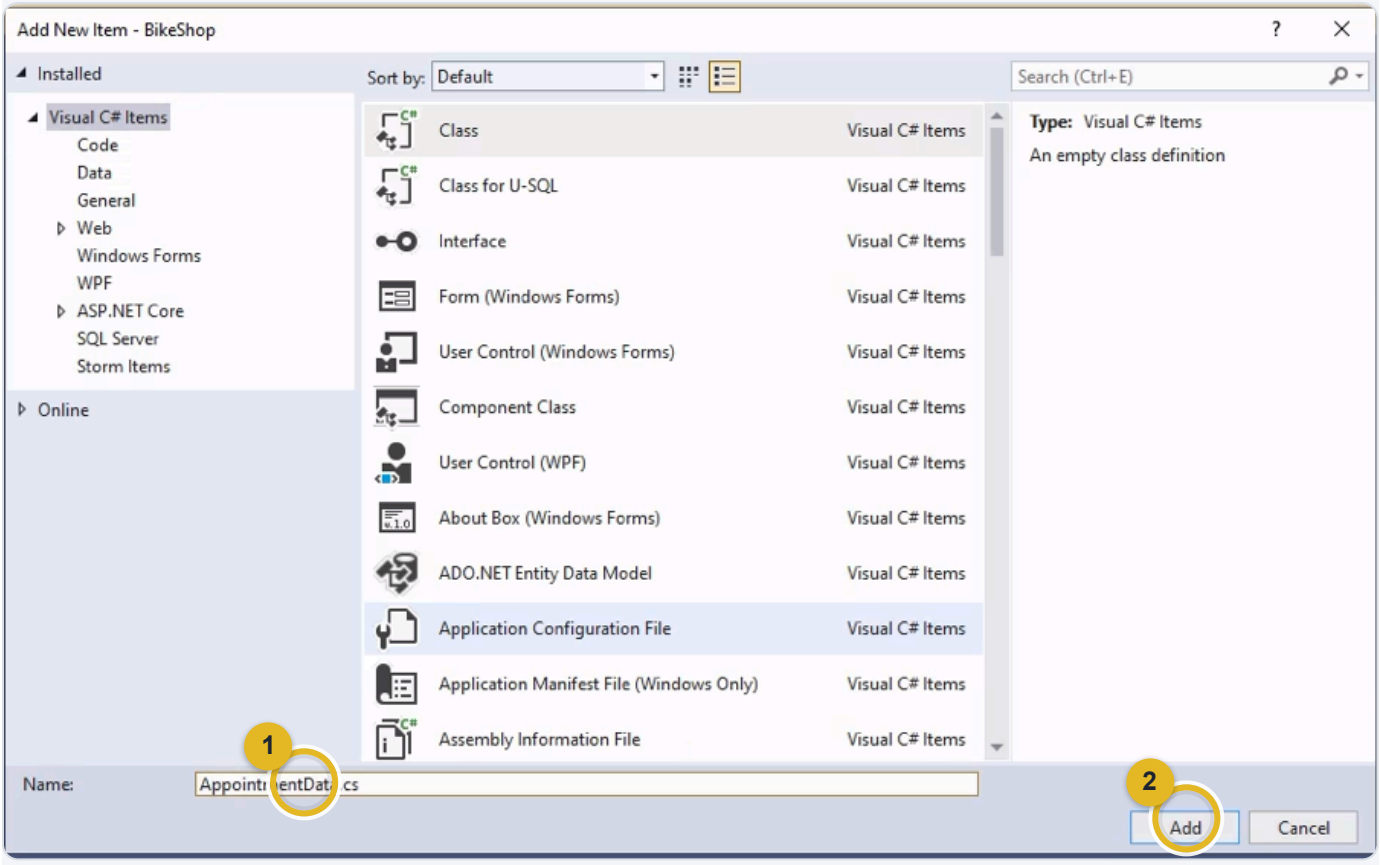



Adding a class to the Models folder. This picture was taken in the finished project, so Solution Explorer lists files you have not made yet.

1. Right-click the Models folder in Solution Explorer.
2. Point at Add.
3. Click Class…

> **You should see** The Add New Item window opens with Class selected.

In the Name box at the bottom, type Customer.cs and press Add.




Name the new class before you press Add.

1. Type the name in the Name box, with .cs on the end.

2. Press Add.

> **You should see** A new file opens with an empty class in it.

Press the Copy button under this step.

> **You should see** The button reads Copied for a moment.

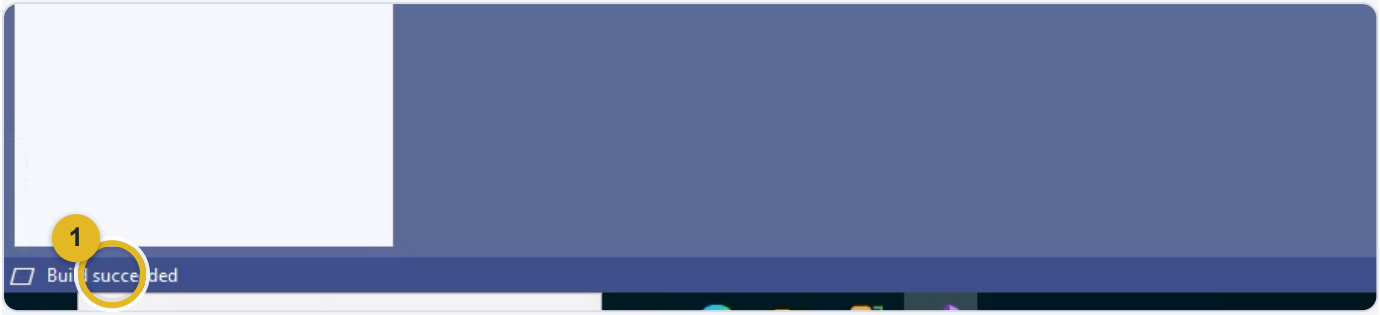
Click anywhere in the new file and press Ctrl and A together.

> **You should see** Everything in the file is highlighted.

Press Ctrl and V together to paste over the highlighted text.

> **You should see** Customer.cs holds the Customer class, ending with the FullName property, and no red wavy line anywhere.

Press F6 to build the project.




Build succeeded, at the bottom left. Anything else means an error list to read.

1 Build succeeded. Press F6 to build at any time.

> **You should see** Build succeeded, at the bottom left.

# Read how CustomerData talks to the five customer stored procedures.

A data class is the only place in the program that names a stored procedure. CustomerData has one method for each customer stored procedure from chapter 4, and every method is the same shape: give Db the procedure's name and its parameters, and hand back what comes out. The class is static, so you never make a CustomerData object: you write CustomerData.Search, the way you wrote Db.GetTable in chapter 5.

Read it region by region. A region is a fold in the editor, marked #region and #endregion, that groups the lines for one job. Search and Delete are two lines each. GetByID turns one row into one Customer object. Add is the interesting one: the stored procedure hands back the new customer's number through an output parameter, and Db.Out makes that parameter.

Copy this one **Data/CustomerData.cs: one method for each stored procedure** C#

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
using System;
using System.Data;
using System.Data.SqlClient;
using BikeShop.Models;

namespace BikeShop.Data
{
    public static class CustomerData
    {
        #region Search
        // Customers whose name or phone contains the search text.
        public static DataTable Search(string searchText)
        {
            return Db.GetTable("usp_Customer_Search",
                Db.P("@SearchText", searchText));
        }
        #endregion

        #region GetByID
        // One customer as an object, or null if there is no customer with that
        // ID.
        public static Customer GetByID(int customerID)
        {
            DataTable table = Db.GetTable("usp_Customer_GetByID",
                Db.P("@CustomerID", customerID));
            if (table.Rows.Count == 0)
            {
                return null;
            }

            DataRow row = table.Rows[0];
            Customer customer = new Customer();
            customer.CustomerID = (int)row["CustomerID"];
            customer.FirstName = (string)row["FirstName"];
            customer.LastName = (string)row["LastName"];
            customer.Phone = (string)row["Phone"];
            if (row["Email"] == DBNull.Value)
            {
                customer.Email = null;
            }
            else
            {
                customer.Email = (string)row["Email"];
            }
            customer.DateJoined = (DateTime)row["DateJoined"];
            return customer;
        }
        #endregion

        #region Add
        // Saves a new customer.  Returns "" if it worked, or the sentence to
        // show the user.
        public static string Add(Customer customer)
```

```
54          {
55              SqlParameter newID = Db.Out("@NewCustomerID");
56              string message = Db.Run("usp_Customer_Add",
57                  Db.P("@FirstName", customer.FirstName),
58                  Db.P("@LastName", customer.LastName),
59                  Db.P("@Phone", customer.Phone),
60                  Db.P("@Email", customer.Email),
61                  newID);
62              if (message == "")
63              {
64                  customer.CustomerID = (int)newID.Value;
65              }
66              return message;
67          }
68          #endregion
69
70          #region Update
71          // Saves changes to an existing customer.  Returns "" if it worked.
72          public static string Update(Customer customer)
73          {
74              return Db.Run("usp_Customer_Update",
75                  Db.P("@CustomerID", customer.CustomerID),
76                  Db.P("@FirstName", customer.FirstName),
77                  Db.P("@LastName", customer.LastName),
78                  Db.P("@Phone", customer.Phone),
79                  Db.P("@Email", customer.Email));
80          }
81          #endregion
82
83          #region Delete
84          // The procedure refuses if the customer has bookings.  Returns "" if it
85          // worked.
86          public static string Delete(int customerID)
87          {
88              return Db.Run("usp_Customer_Delete",
89                  Db.P("@CustomerID", customerID));
90          }
91          #endregion
92      }
93  }
```

**1 The usings and the class**

Lines 1 to 9

System.Data gives DataTable and DataRow. System.Data.SqlClient gives SqlParameter, the type Db.P and Db.Out make. using BikeShop.Models lets the file say Customer. The class is static: no objects, only methods called by the class name.

**2 Search: the rows whose name or phone contains the text**

Lines 10 to 17

Search hands Db.GetTable the name of the stored procedure and one parameter, made by Db.P. Whatever usp_Customer_Search selects comes back as a DataTable, and the form gives that table to the grid. An empty search text matches every customer.

**3 GetByID: one customer as an object**

Lines 19 to 48

The stored procedure answers with a table of one row, or no rows. No rows means there is no such customer, and the method returns null: nothing at all. Otherwise row is that one row, and each column is copied into a new Customer object. (int) and (string) in front of a column say what kind of value is inside. Email can be NULL in the database, which arrives as DBNull.Value, so the method checks for that first and stores null in the object.

**4 Add: save a new customer, and get the new number back**

Lines 50 to 68

usp_Customer_Add has two output parameters: @NewCustomerID and @Message. Db.Run makes @Message itself, so this method makes the other one with Db.Out, and keeps hold of it in newID. Db.Run returns the message: an empty string when the save worked, or a sentence to show the user. When the message is empty, the number the database chose is read from newID.Value and put into the customer object, so the caller knows it.

**5 Update: save changes to a customer who already has a number**

Lines 70 to 81

Five parameters go in, one for each box on the edit form plus the number that says which row to change. Nothing comes back except the message.

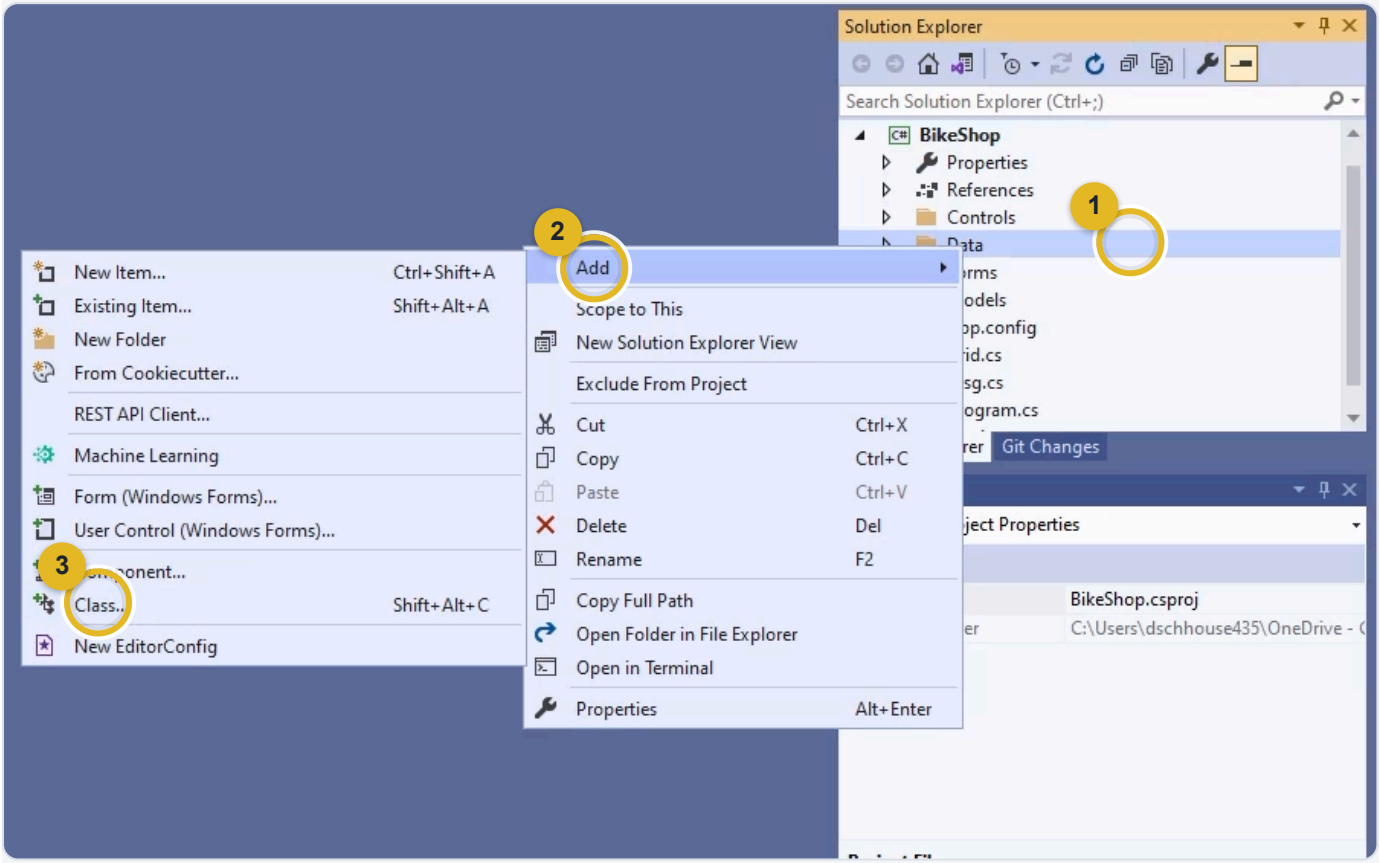
**6 Delete: ask the database to take one customer out**

Lines 83 to 93

Only the number goes in. The stored procedure from chapter 4 refuses when the customer has bookings or sales, and its sentence comes back as the message. The last two braces end the class and the namespace.

# Add the CustomerData class to the Data folder.

In Solution Explorer in Visual Studio 2019, right-click the Data folder, point at Add, and click Class…

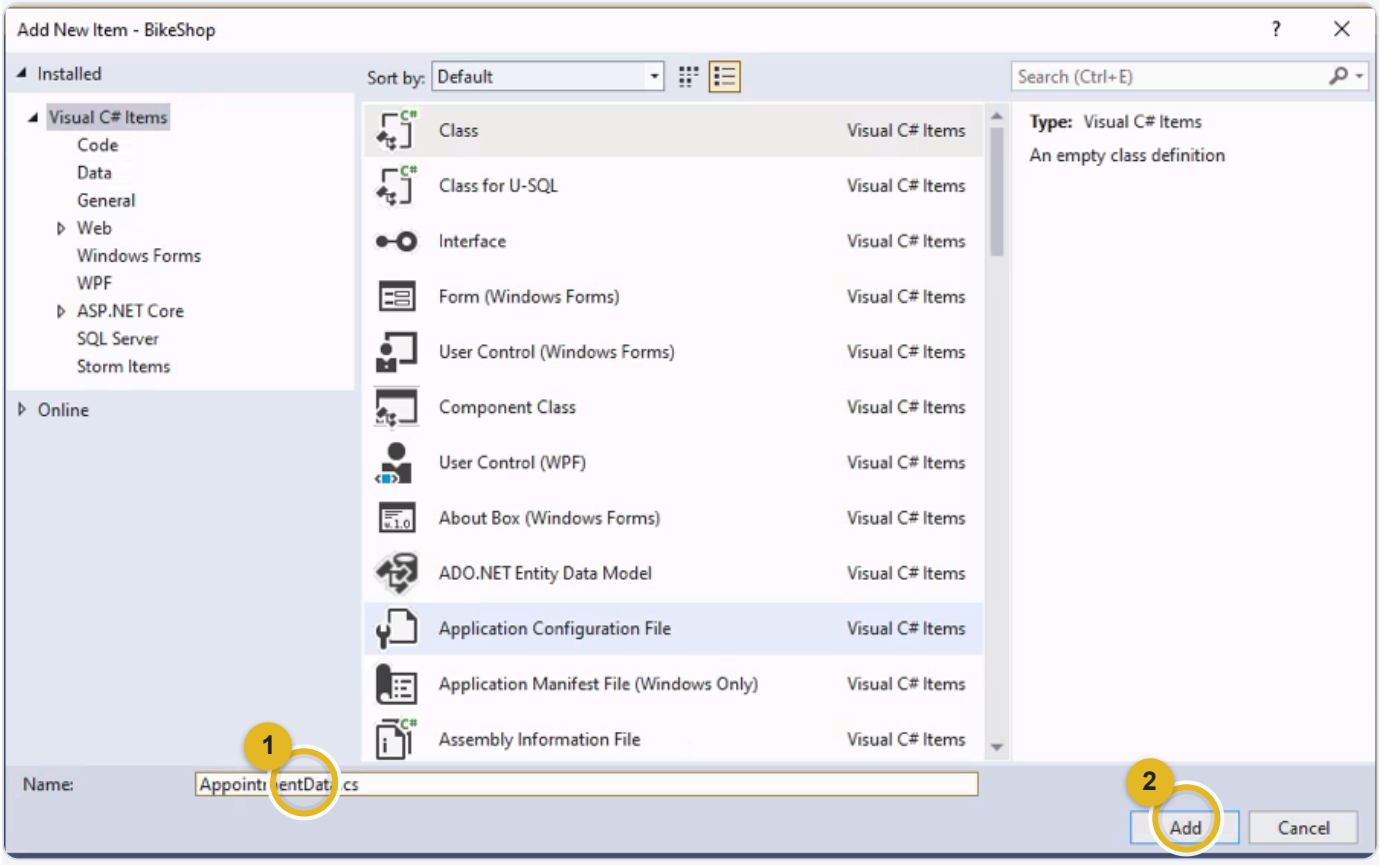



Adding a class to the Data folder. Taken in the finished project, so Solution Explorer lists more files than yours.

1. Right-click the Data folder in Solution Explorer.
2. Point at Add.
3. Click Class…

**You should see** The Add New Item window opens with Class selected.

In the Name box at the bottom, type CustomerData.cs and press Add.




Name the new class before you press Add.

1. Type the name in the Name box, with .cs on the end.

2. Press Add.

> **You should see** A new file opens with an empty class in it.

Press the Copy button under this step.

> **You should see** The button reads Copied for a moment.

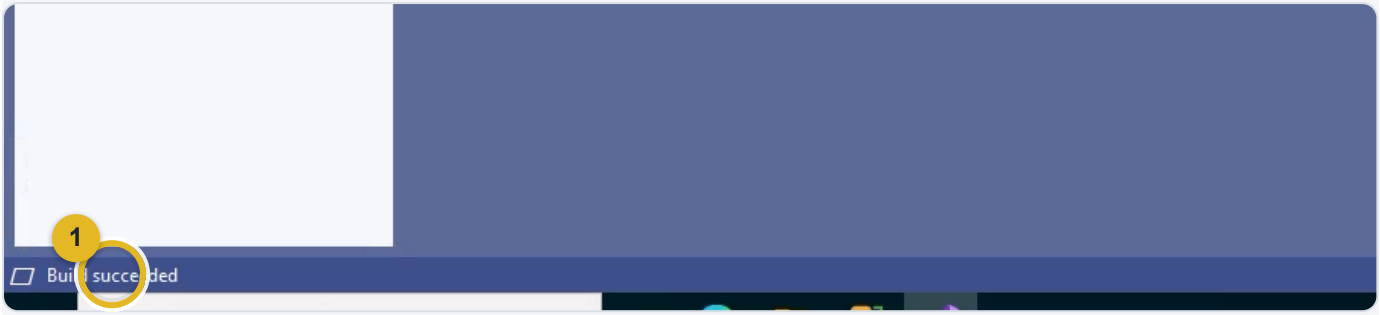
Click anywhere in the new file and press Ctrl and A together.

> **You should see** Everything in the file is highlighted.

Press Ctrl and V together to paste over the highlighted text.

> **You should see** CustomerData.cs holds five regions: Search, GetByID, Add, Update and Delete.

Press F6 to build the project.




Build succeeded, at the bottom left. Anything else means an error list to read.

1 Build succeeded. Press F6 to build at any time.

**You should see** Build succeeded, at the bottom left.

**IF IT GOES WRONG** 'The name Db does not exist' or 'The name Msg does not exist'

Your helper classes from chapter 5 are called something else, or sit in a different namespace. Match the names you used there.

**IF IT GOES WRONG** 'The type or namespace name Customer could not be found'

The Customer class is missing from the Models folder, or its file was saved under another name. Go back to step 6.1.1 and check Solution Explorer.

# What a grid shows, and why it needs tidying.

A grid is the DataGridView control: a table on a form with a heading row and one row per record. Give a grid a DataTable and it shows every column, with the column's name as the heading. Left alone, that means a heading like CustomerID, which is a number the user never needs, and a heading like DateJoined, which runs two words together.

Grid.cs is a small helper, written once and used by every grid in the program. Its method Tidy hides the columns you name, turns every heading into plain words, shows money with a pound sign and two decimal places, and shows dates as a day name and a date. The rule for the user is simple to state: names, never numbers. Two more methods, SelectedID and SelectedText, read one value from the row the user has picked.

Grid.cs goes at the top of the project, beside Program.cs, Session.cs and Msg.cs, because every form uses it. In Visual Studio 2019 that means adding the class to the project line itself, not to a folder.

# Read the Grid helper.

Copy this one **Grid.cs: the helpers every grid uses** C#

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
using System;
using System.Text;
using System.Windows.Forms;

namespace BikeShop
{
    // Small helpers every grid uses.  Call Tidy straight after you set a grid's
    // DataSource.
    public static class Grid
    {
        #region Tidy
        // Hides the ID columns (the user never needs a number) and turns column
        // names into plain headers.
        public static void Tidy(DataGridView grid,
            params string[] hiddenColumns)
        {
            foreach (string name in hiddenColumns)
                if (grid.Columns.Contains(name)) grid.Columns[name].Visible =
                    false;

            foreach (DataGridViewColumn column in grid.Columns)
            {
                column.HeaderText = Words(column.Name);
                if (column.ValueType == typeof(decimal) && IsMoney(column.Name))
                {
                    column.DefaultCellStyle.Format = "£#,##0.00";
                    column.DefaultCellStyle.Alignment =
                        DataGridViewContentAlignment.MiddleRight;
                }
                else if (column.ValueType == typeof(DateTime))
                {
                    column.DefaultCellStyle.Format = "ddd d MMM yyyy";
                    if (column.Name.Contains("Time"))
                    {
                        column.DefaultCellStyle.Format = "HH:mm";
                    }
                }
            }
            grid.ClearSelection();
        }

        private static bool IsMoney(string name)
        {
            return name.Contains("Price") || name.Contains("Rate")
                || name.Contains("Cost") || name.Contains("Income")
                || name.Contains("Deposit") || name.Contains("Total")
                    || name.Contains("Paid");
        }
        #endregion

        #region Words
        // "BikeDescription" → "Bike description", "QtyInStock" → "Qty in
        // stock", "Mechanic" → "Mechanic"
```

```
54          public static string Words(string columnName)
55          {
56              StringBuilder words = new StringBuilder();
57              for (int i = 0; i < columnName.Length; i++)
58              {
59                  char c = columnName[i];
60                  if (i > 0 && char.IsUpper(c) && char.IsLower(columnName[i - 1]))
61                  {
62                      words.Append(' ');
63                      words.Append(char.ToLower(c));
64                  }
65                  else
66                  {
67                      words.Append(c);
68                  }
69              }
70              return words.ToString();
71          }
72          #endregion
73
74          #region Selected
75          // The hidden ID of the picked row (0 when nothing is picked).
76          public static int SelectedID(DataGridView grid, string idColumn)
77          {
78              if (grid.SelectedRows.Count == 0) return 0;
79              return Convert.ToInt32(grid.SelectedRows[0].Cells[idColumn].Value);
80          }
81
82          // A text value from the picked row ("" when nothing is picked).
83          public static string SelectedText(DataGridView grid, string column)
84          {
85              if (grid.SelectedRows.Count == 0) return "";
86              return Convert.ToString(grid.SelectedRows[0].Cells[column].Value);
87          }
88          #endregion
89      }
90  }
```

**1 The usings and the class**

Lines 1 to 10

System.Text gives StringBuilder, which the Words method uses to build a heading one letter at a time. System.Windows.Forms gives DataGridView. The class is static and sits in the BikeShop namespace itself, so every form can say Grid without a using line.

**2 Tidy: the grid, and the columns to hide**

Lines 11 to 16

Tidy takes the grid and then any number of column names, which params gathers into a list called hiddenColumns. A form writes Grid.Tidy(dgvCustomers, "CustomerID") with one name, and a form with two number columns writes two.

**3 Hide the ID columns**

Lines 17 to 19

foreach looks at each name in the list in turn. If the grid has a column with that name, its Visible setting is switched to false. The column is still there, so SelectedID can read the hidden number later.

### 4 Plain words in every heading

Lines 21 to 23

This foreach looks at every column of the grid. HeaderText is the words the user sees at the top of the column, and Words turns the column's name into them: DateJoined becomes Date joined.

### 5 Money columns

Lines 24 to 29

ValueType is the kind of value the column holds. decimal is the kind for money. When the column holds decimals and its name says money, IsMoney answers true, and the column is shown with a pound sign and two decimal places, pushed to the right the way money is written.

### 6 Date and time columns

Lines 30 to 38

A DateTime column is shown as a day name, a day, a month and a year, such as Mon 12 Oct 2026. If the column's name contains the word Time, the hours and minutes are shown instead, because the diary grids in chapter 7 show one day at a time. The closing braces end the if, the else if and the foreach.

### 7 Nothing picked to begin with

Lines 39 to 40

A grid picks its first row by itself as soon as it is filled. ClearSelection undoes that, so the form starts with no row picked and Edit and Delete switched off.

### 8 Which columns hold money

Lines 42 to 49

IsMoney answers true when the column's name contains any of the seven money words. The two vertical bars mean or: any one match is enough. private means only Grid itself can call this method.

### 9 Words: BikeDescription becomes Bike description

Lines 51 to 72

Words walks along the column's name one letter at a time. char is one letter. When a capital letter follows a small one, a space goes in and the capital becomes small. StringBuilder collects the letters, and ToString hands back the finished heading.

### 10 SelectedID: the hidden number of the picked row

Lines 74 to 80

SelectedRows is the list of rows the user has picked. When the list is empty the method returns 0, which every form treats as nothing picked. Otherwise it reads the named cell of the first picked row and turns the value into a whole number.

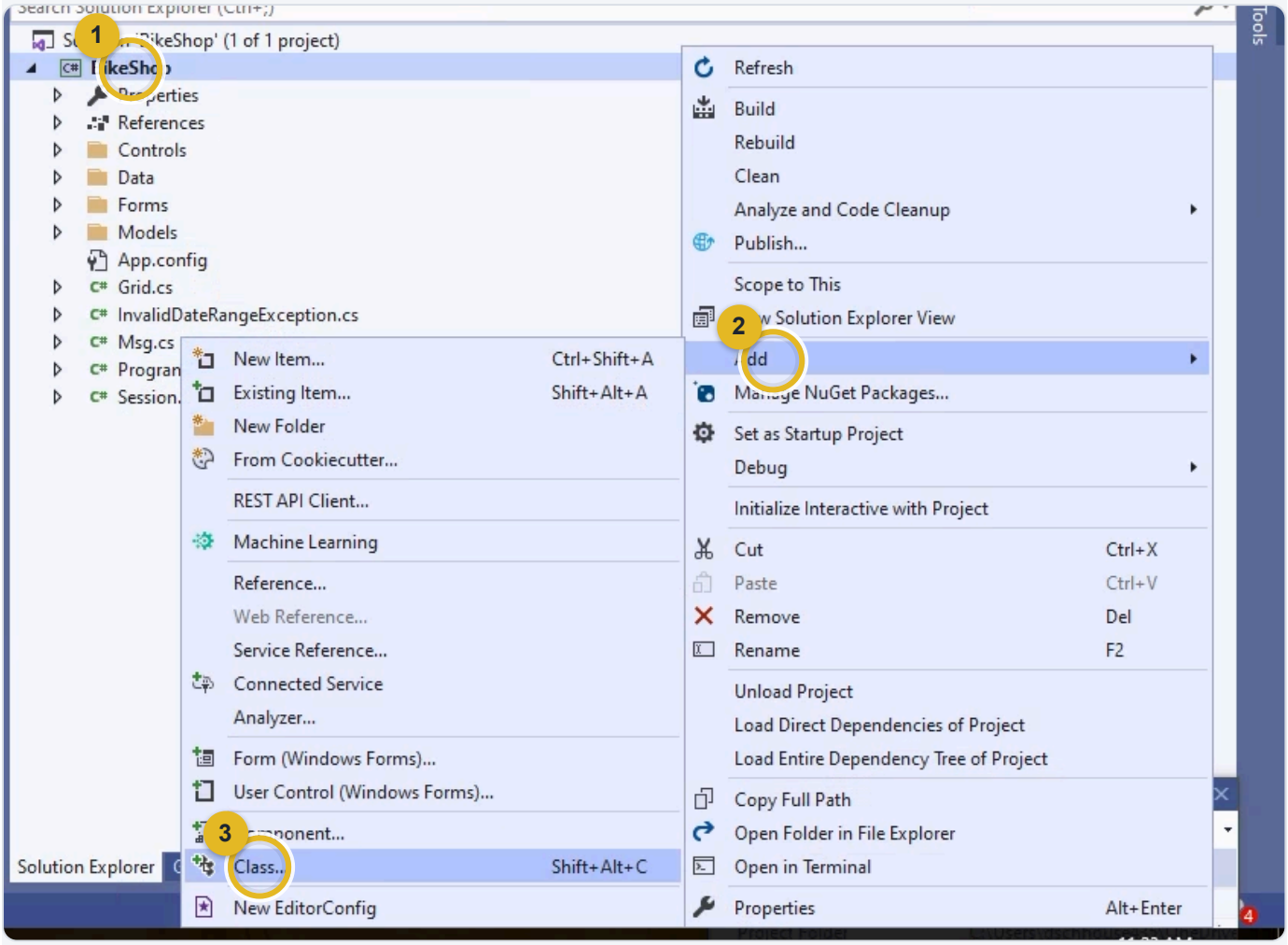
### 11 SelectedText: a text value from the picked row

Lines 83 to 90

The same shape, for text: an empty string when nothing is picked, otherwise the named cell as text. The delete button uses this to put the customer's name into its question.

# Add Grid.cs to the project itself.

In Solution Explorer in Visual Studio 2019, right-click the BikeShop project line, the one above the Data, Forms and Models folders, point at Add, and click Class…




Adding a class to the project itself, not to a folder. Grid.cs sits at the top, beside Program.cs, Session.cs and Msg.cs.

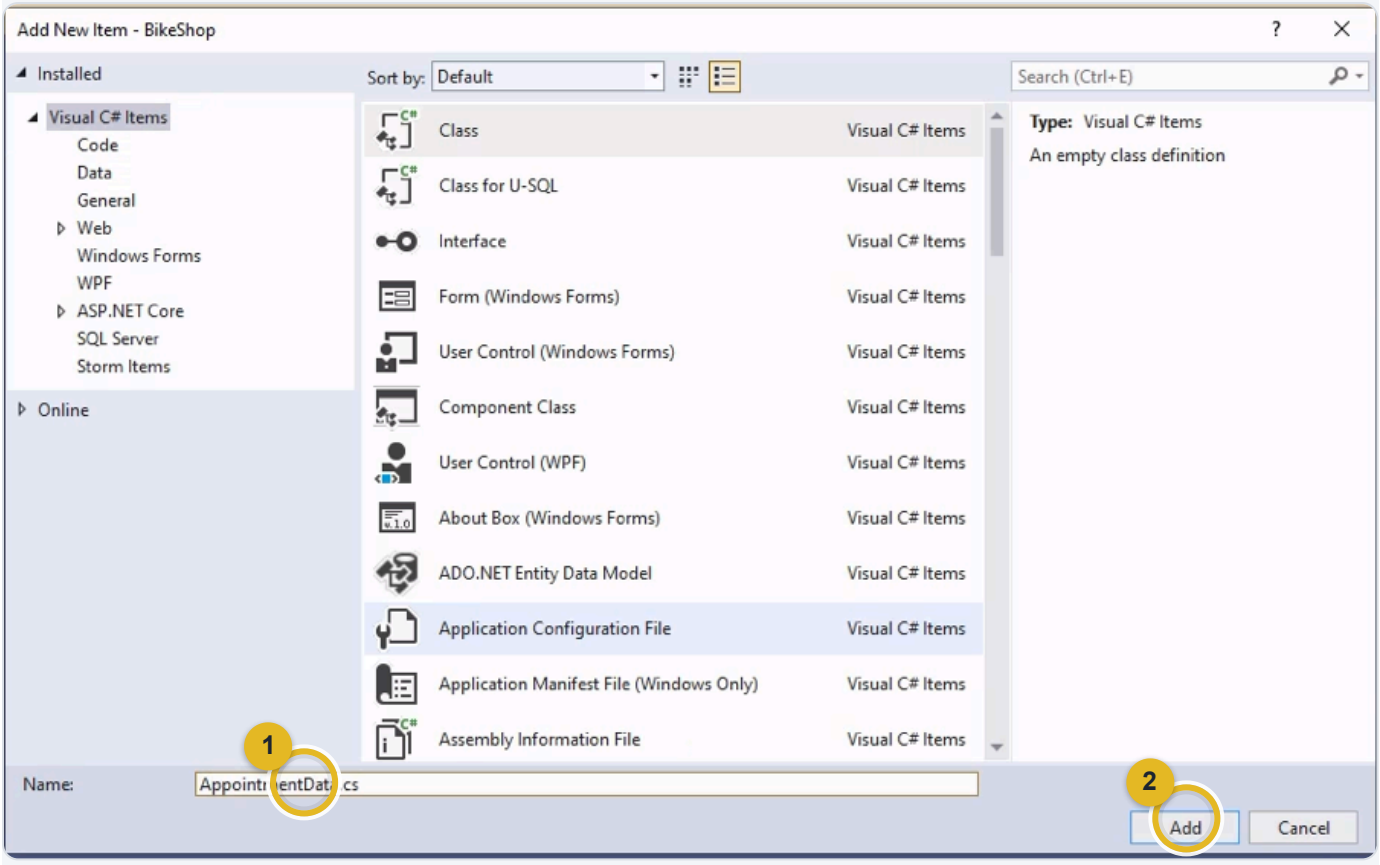
1. Right-click the BikeShop project line, the one above the Data, Forms and Models folders.

2. Point at Add.

3. Click Class…

**You should see** The Add New Item window opens with Class selected.

In the Name box at the bottom, type Grid.cs and press Add.




Name the new class before you press Add.

1. Type the name in the Name box, with .cs on the end.

2. Press Add.

> **You should see** A new file opens with an empty class in it.

Press the Copy button under this step.

> **You should see** The button reads Copied for a moment.

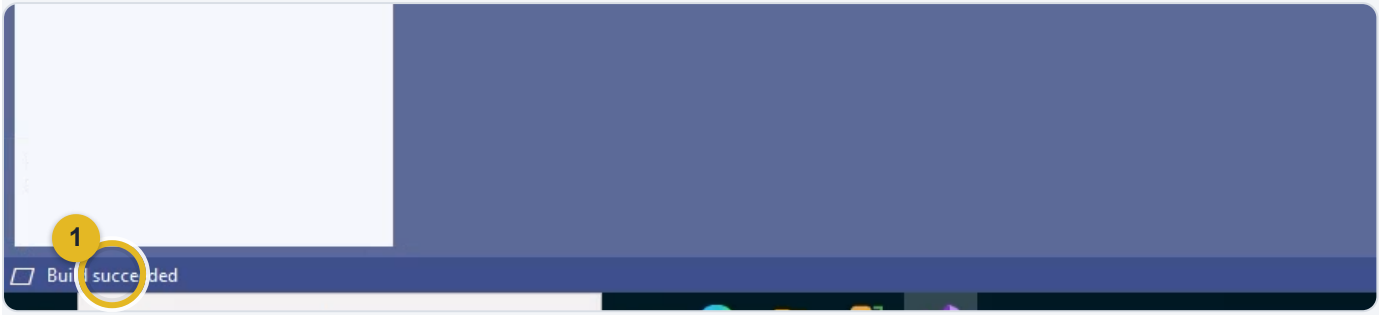
Click anywhere in the new file and press Ctrl and A together.

> **You should see** Everything in the file is highlighted.

Press Ctrl and V together to paste over the highlighted text.

> **You should see** Grid.cs holds three regions: Tidy, Words and Selected.

Press F6 to build the project.




Build succeeded, at the bottom left. Anything else means an error list to read.

1 Build succeeded. Press F6 to build at any time.

**You should see** Build succeeded, at the bottom left.

**IF IT GOES WRONG** Grid.cs appears inside the Data or Forms folder in Solution Explorer

The right-click landed on a folder instead of the project line. Drag Grid.cs up on to the BikeShop project line in Solution Explorer, and it moves. The program builds either way.

STAGE 3 OF 7 · THE CUSTOMERS SCREEN · SCREEN 1 OF 7

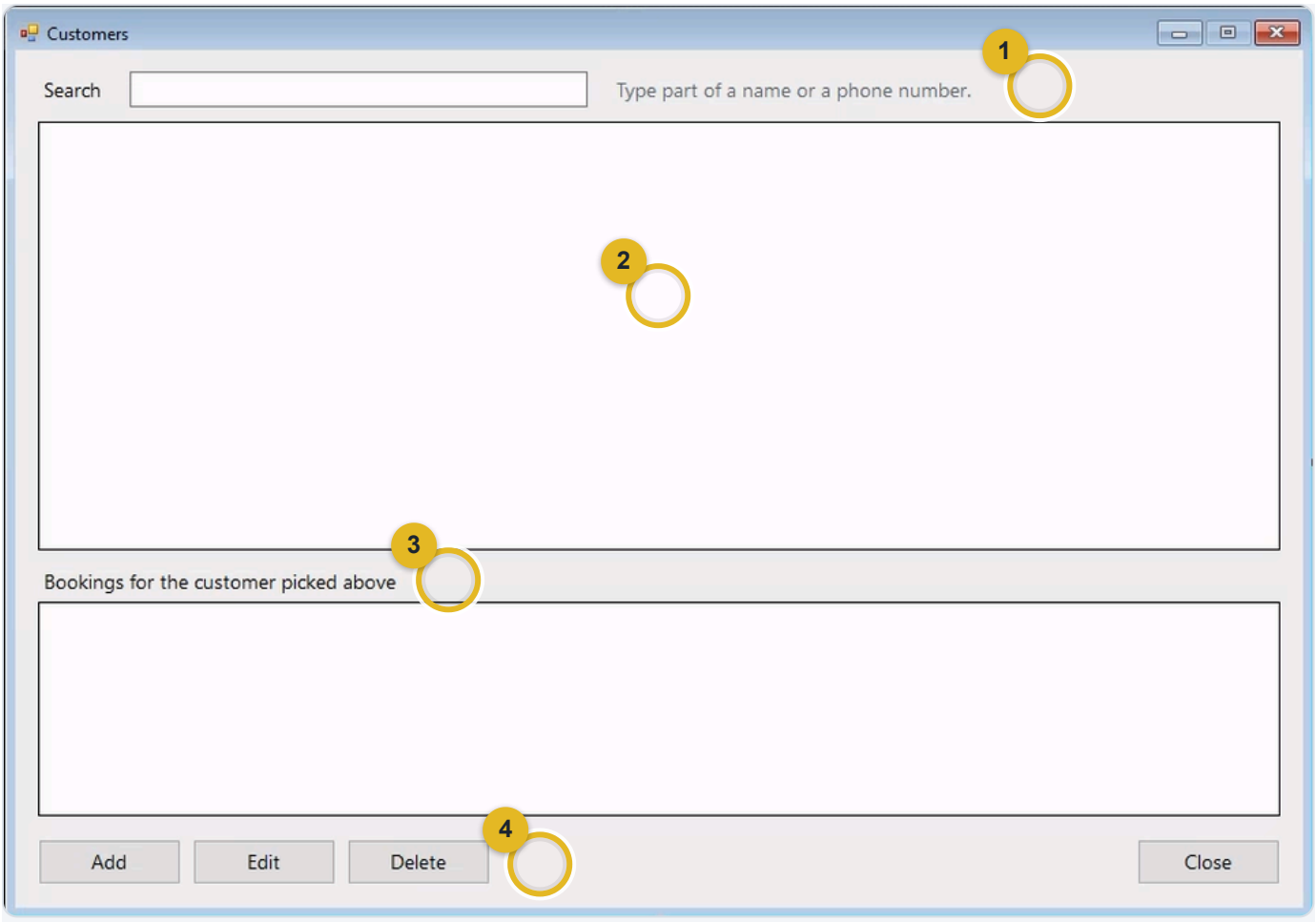
# Set out the controls on frmCustomers in Visual Studio 2019.

Chapter 5 added frmCustomers as an empty form. Now it gets its controls: a Search box with a label and a grey hint, the customers grid, a label and a second grid for the picked customer's bookings, and four buttons. Click the form itself first and set the form's own properties. Then work down the table: drag the control from the Toolbox, drop it on the form, set its Name in the Properties window, and set the other settings in the row. Where the Then column says double-click, double-clicking the control makes its event method for you.

Search, add, edit and delete customers. The lower grid shows the bookings of whichever customer is picked.

**Click the form itself, then set these in the Properties window**

| | | | |
|---|---|---|---|
| Text | `Customers` | Size | `900, 600` |
| FormBorderStyle | `FixedSingle` | StartPosition | `CenterParent` |
| MaximizeBox | `False` | | |




frmCustomers in the designer. Yours should look like this before you add any code.

1. lblSearch, txtSearch and the grey hint lblHint.
2. The customers grid, dgvCustomers.
3. lblBookings, then the bookings grid, dgvBookings.
4. btnAdd, btnEdit, btnDelete, and btnClose at the right.

| DRAG THIS FROM THE TOOLBOX | NAME IT (THE NAME PROPERTY) | SET THESE IN THE PROPERTIES WINDOW | THEN |
|---|---|---|---|
| Label | `lblSearch` | `Text = Search` | |
| TextBox | `txtSearch` | nothing else<br>The list narrows as you type. | Double-click it. Visual Studio makes `txtSearch_TextChanged` for you; the code goes inside it. |
| Label | `lblHint` | `Text = Type part of a name or a phone number.`<br>`ForeColor = grey (92, 103, 116)` | |
| DataGridView | `dgvCustomers` | `the grid settings (chapter 6)`<br>The customers. Double-click a row to edit it. | Events tab (the lightning bolt) → SelectionChanged: type `dgvCustomers_SelectionChanged` beside it and press Enter.,Events tab (the lightning bolt) → CellDoubleClick: type `dgvCustomers_CellDoubleClick` beside it and press Enter. |
| Label | `lblBookings` | `Text = Bookings for the customer picked above` | |
| DataGridView | `dgvBookings` | `the grid settings (chapter 6)`<br>Read-only: that customer's bookings, newest first. | |
| Button | `btnAdd` | `Text = Add`<br>Opens the edit window empty. | Double-click it. Visual Studio makes `btnAdd_Click` for you; the code goes inside it. |
| Button | `btnEdit` | `Text = Edit`<br>`Enabled = False`<br>Switched off until a row is picked. | Double-click it. Visual Studio makes `btnEdit_Click` for you; the code goes inside it. |
| Button | `btnDelete` | `Text = Delete`<br>`Enabled = False`<br>Asks first. The database refuses | Double-click it. Visual Studio makes `btnDelete_Click` for you; the code goes inside it. |

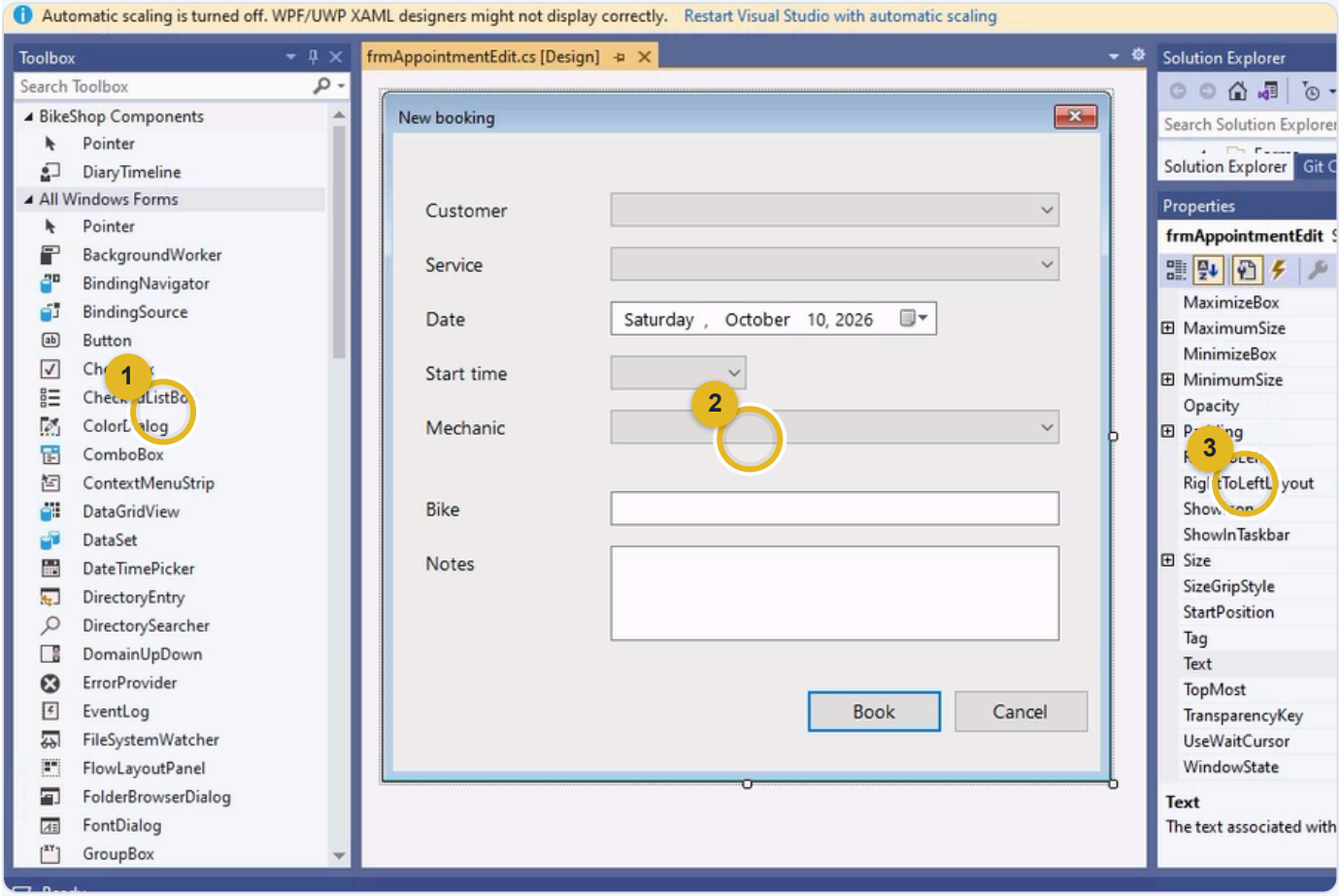
| DRAG THIS FROM THE TOOLBOX | NAME IT (THE NAME PROPERTY) | SET THESE IN THE PROPERTIES WINDOW | THEN |
|---|---|---|---|
| | | if the customer has bookings or sales. | |
| Button | `btnClose` | `Text = Close` | Double-click it. Visual Studio makes `btnClose_Click` for you; the code goes inside it. |

**The grid settings**

This is your first grid, so here are the settings the table calls the grid settings. Set them on dgvCustomers, and again on dgvBookings, in the Properties window: AllowUserToAddRows = False, AllowUserToDeleteRows = False, AllowUserToResizeRows = False, AutoSizeColumnsMode = Fill, BackgroundColor = White, MultiSelect = False, ReadOnly = True, RowHeadersVisible = False, SelectionMode = FullRowSelect. Together they make a grid the user can only read and pick from: one whole row at a time, no empty row at the bottom, no grey margin at the left, and columns that fill the width.

# Lay out the controls, one at a time.

In Solution Explorer in Visual Studio 2019, double-click frmCustomers.cs in the Forms folder.




The three parts of the designer: the Toolbox you drag from, the form you drop on, the Properties window where you name and set each control.

1. The Toolbox. Drag a control from here on to the form.
2. The form. Drop the control where the picture shows it.
3. The Properties window. Set Name first, then the rest of the row.

> **You should see** The empty form opens in the designer.

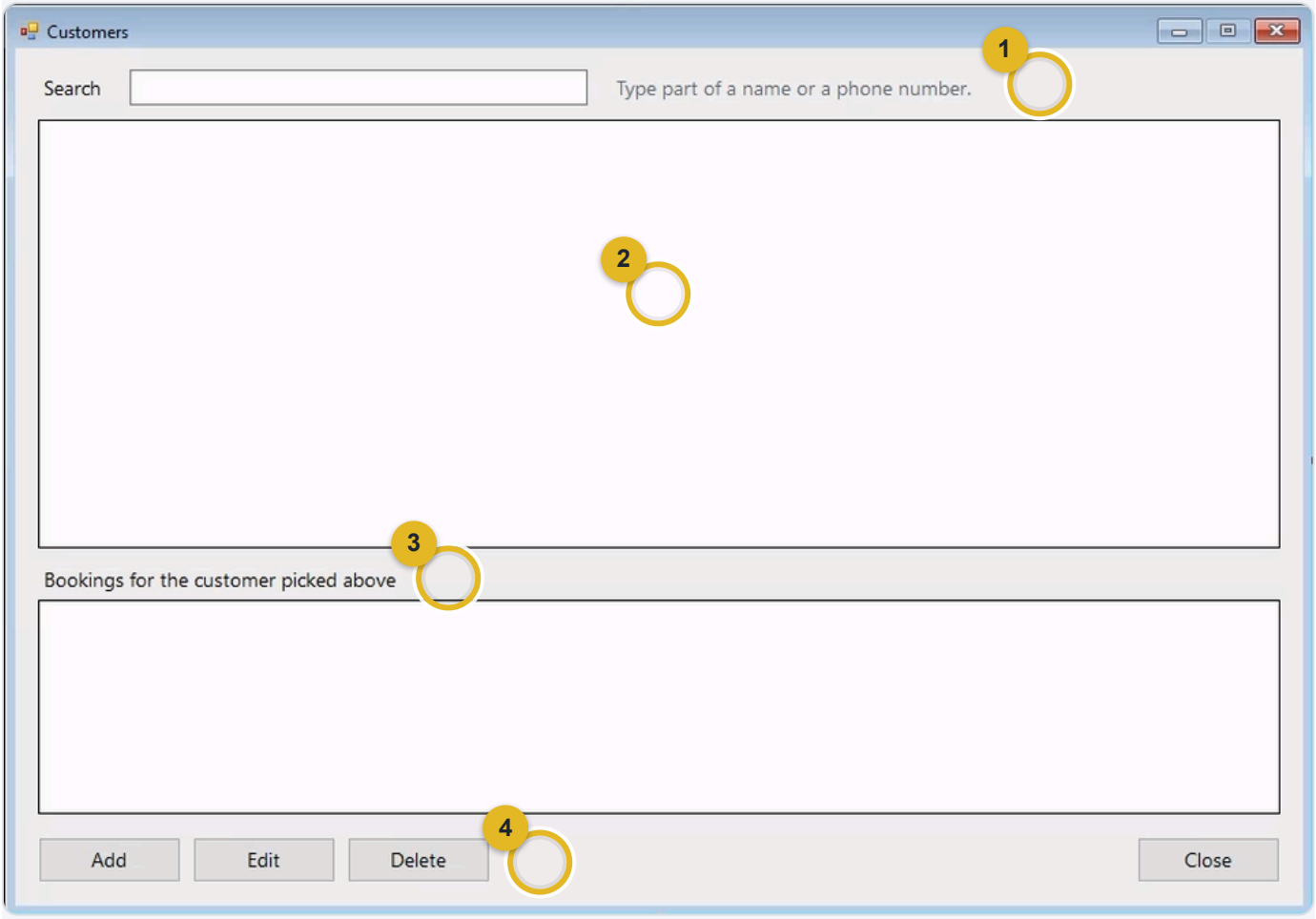
Open the Toolbox (View, then Toolbox) and the Properties window (View, then Properties Window), if they are not already open.

> **You should see** The Toolbox is on the left, the form in the middle, the Properties window on the right.

Click the form itself and set Text to Customers, Size to 900, 600 and FormBorderStyle to FixedSingle.

> **You should see** The form grows, its title bar reads Customers, and its edges can no longer be dragged.

Work down the table: drag each control from the Toolbox on to the form, set its Name, then set the rest of its row.




frmCustomers in the designer. Yours should look like this before you add any code.

1. lblSearch, txtSearch and the grey hint lblHint.
2. The customers grid, dgvCustomers.
3. lblBookings, then the bookings grid, dgvBookings.
4. btnAdd, btnEdit, btnDelete, and btnClose at the right.

> **You should see** The form looks like the picture, with every control named as the table says.

For txtSearch, double-click the box on the form.

> **You should see** The code tab opens with an empty method called txtSearch_TextChanged. Press Shift and F7 together to go back to the designer.

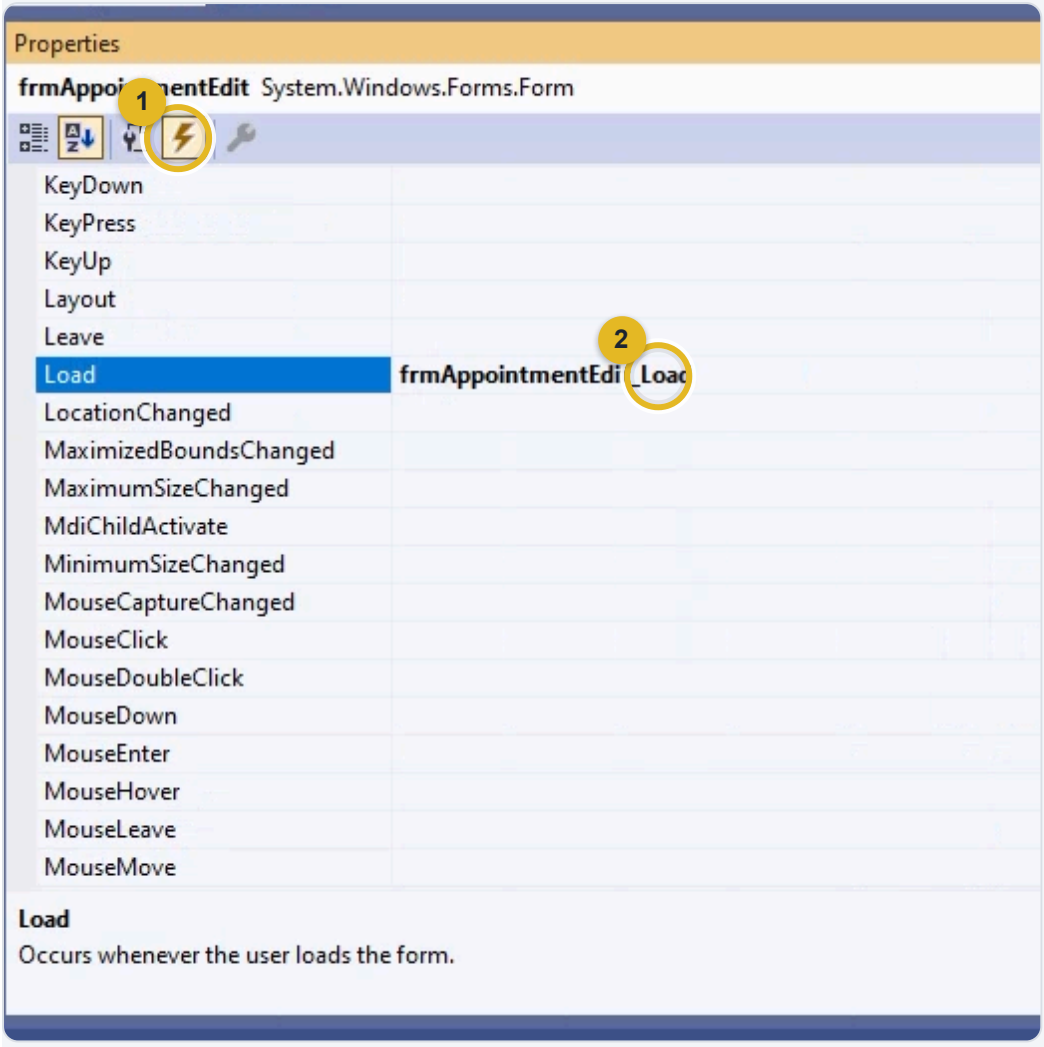
Click the dgvCustomers grid on the form.

> **You should see** The Properties window shows dgvCustomers at the top.

Press the lightning bolt at the top of the Properties window.

> **You should see** The Properties window shows the list of events for dgvCustomers, in alphabetical order.

Double-click the empty box beside SelectionChanged.




The lightning bolt shows the events, in alphabetical order. Double-click the empty box beside one to make its method.

1. Press the lightning bolt.

2. Double-click the empty box beside the event your step names. The events are in alphabetical order.

> **You should see** The code tab opens with an empty method called dgvCustomers_SelectionChanged. Press Shift and F7 together to go back.

With dgvCustomers still picked and the lightning bolt still pressed, double-click the empty box beside CellDoubleClick.

> **You should see** The code tab opens with an empty method called dgvCustomers_CellDoubleClick. Press Shift and F7 together to go back.

Double-click each of the four buttons in turn: Add, Edit, Delete, then Close.

> **You should see** Four empty Click methods are made, named btnAdd_Click, btnEdit_Click, btnDelete_Click and btnClose_Click.

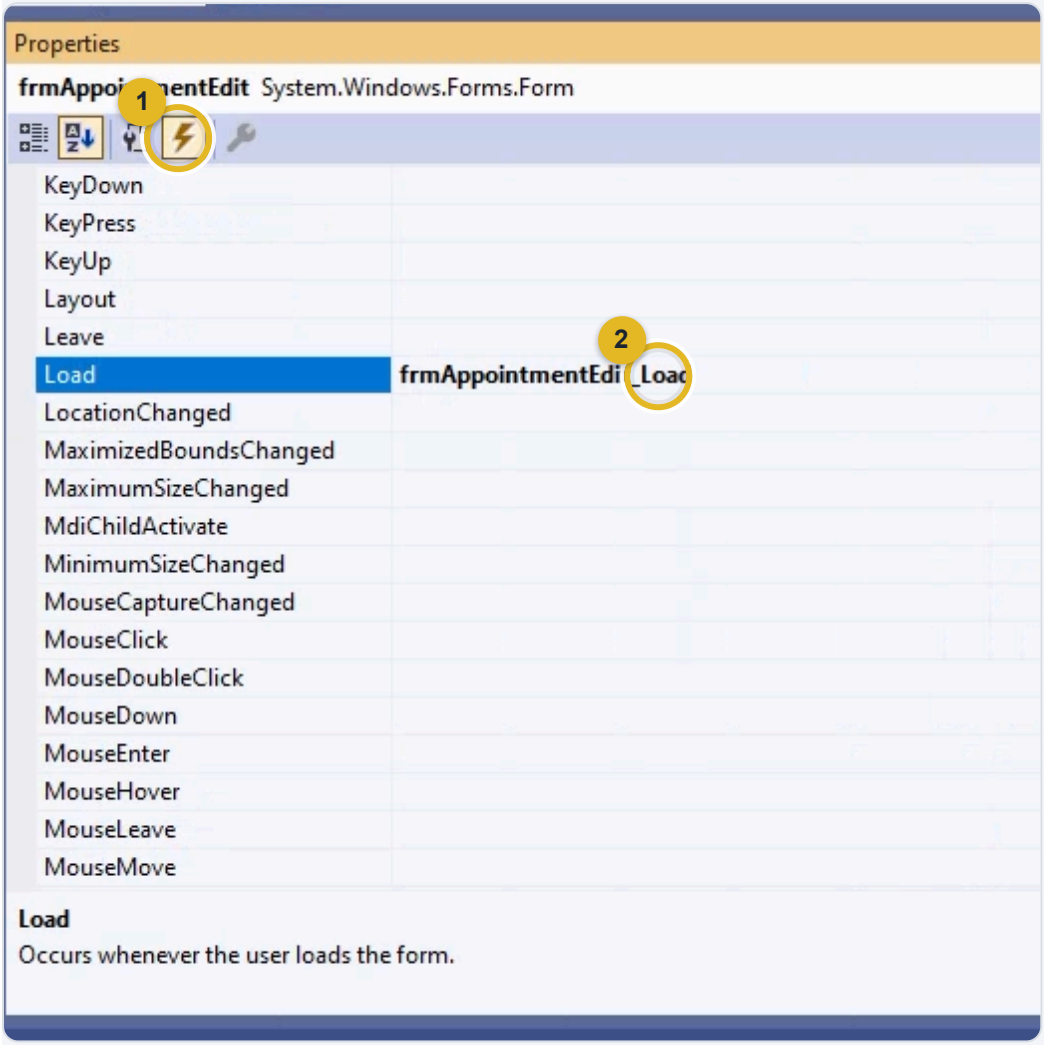
Click the form itself, on its grey background.

> **You should see** The Properties window shows frmCustomers in the box at its top.

Press the lightning bolt at the top of the Properties window.

> **You should see** The Properties window shows the list of events for frmCustomers.

Double-click the empty box beside Load, which is part of the way down, after Leave.




The lightning bolt shows the events, in alphabetical order. Double-click the empty box beside one to make its method.

1. Press the lightning bolt.

2. Double-click the empty box beside the event your step names. The events are in alphabetical order.

> **You should see** The code tab opens with an empty method called frmCustomers_Load.

# Read how the grid loads, and searches as you type.

Three pieces of code go behind this form. Read them one at a time, then put the whole file in at once. The first piece is one method, LoadCustomers. It is called when the form opens, every time the search text changes, and after every add, edit and delete, so the grid always shows what the database holds right now.

**Read this one** **Forms/frmCustomers.cs: loading the grid** C#

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  // Fills the grid from the database.  Called when the form opens, when
2  // the search text changes,
3  // and after every add, edit or delete, so the grid always shows what
4  // the database holds.
5  private void LoadCustomers()
6  {
7      dgvCustomers.DataSource =
8          CustomerData.Search(txtSearch.Text.Trim());
9      // hide the ID, give the headers plain words
10     Grid.Tidy(dgvCustomers, "CustomerID");
11 }
```

**1 Ask the database, and give the grid the answer**

Lines 5 to 8

DataSource is the table a grid shows. Trim takes spaces off both ends of the search text. CustomerData.Search sends the text to usp_Customer_Search and hands back the DataTable, and the grid shows it. An empty box brings every customer.

**2 Tidy the grid**

Lines 10 to 11

One line, straight after the DataSource line, every time. CustomerID is hidden, the headings become plain words, and no row is picked.

Searching as you type is one more method, txtSearch_TextChanged, which does nothing but call LoadCustomers. Every key the user presses changes the text, the event fires, and the grid is loaded again from the database with the new text.

# Read how the buttons wake up when a row is picked.

Edit and Delete only make sense once a customer is picked, so they start switched off. The grid's SelectionChanged event fires whenever the picked row changes, and this piece switches the two buttons on or off and fills the lower grid.

Read this one **Forms/frmCustomers.cs: when a row is picked** C#

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  // Edit and Delete only make sense once a row is picked.
2  private void dgvCustomers_SelectionChanged(object sender, EventArgs e)
3  {
4      bool picked = dgvCustomers.SelectedRows.Count > 0;
5      btnEdit.Enabled = picked;
6      btnDelete.Enabled = picked;
7      ShowBookings();
8  }
9
10 // The lower grid shows the picked customer's bookings: two tables on
11 // one screen.
12 private void ShowBookings()
13 {
14     if (dgvCustomers.SelectedRows.Count == 0)
15     {
16         dgvBookings.DataSource = null;
17         return;
18     }
19
20     dgvBookings.DataSource =
21         AppointmentData.GetForCustomer(Grid.SelectedID(dgvCustomers,
22         "CustomerID"));
23     Grid.Tidy(dgvBookings, "AppointmentID");
24 }
```

**1 Switch the two buttons on or off**

Lines 2 to 8

picked is a bool: true when at least one row is picked, false when none is. Enabled is the setting that switches a button on or off, so both buttons follow picked. Then ShowBookings fills the lower grid.

**2 Nothing picked: an empty lower grid**

Lines 12 to 18

When no row is picked, the lower grid is given null, which empties it, and the method stops there with return.

**3 Fetch that customer's bookings**

Lines 20 to 24

Grid.SelectedID reads the hidden CustomerID of the picked row. AppointmentData.GetForCustomer sends that number to usp_Appointment_GetForCustomer from chapter 4, the stored procedure that joins three tables so the grid shows names and not numbers. Grid.Tidy hides the AppointmentID column. AppointmentData is a class you make in chapter 7, so the screen called Park the bookings line tells you what to do about it until then.

# Read Add, Edit and Delete.

The three buttons. Add and Edit open the edit form, which you build in the next stage. Delete asks a question, then tells the database and shows its answer. The word new makes an object from a class: new frmCustomerEdit(0) makes the edit form, and the number in the brackets goes to the form's constructor.

Read this one **Forms/frmCustomers.cs: the three buttons** C#

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  private void btnAdd_Click(object sender, EventArgs e)
2  {
3      // 0 means "a new customer"
4      frmCustomerEdit edit = new frmCustomerEdit(0);
5      if (edit.ShowDialog(this) == DialogResult.OK)
6      {
7          LoadCustomers();
8      }
9  }
10
11 private void btnEdit_Click(object sender, EventArgs e)
12 {
13     frmCustomerEdit edit =
14         new frmCustomerEdit(Grid.SelectedID(dgvCustomers,
15         "CustomerID"));
16     if (edit.ShowDialog(this) == DialogResult.OK)
17     {
18         LoadCustomers();
19     }
20 }
21
22 private void dgvCustomers_CellDoubleClick(object sender,
23     DataGridViewCellEventArgs e)
24 {
25     if (e.RowIndex ⩾ 0)
26     {
27         btnEdit_Click(sender, e);
28     }
29 }
30
31 private void btnDelete_Click(object sender, EventArgs e)
32 {
33     string name = Grid.SelectedText(dgvCustomers, "FullName");
34     if (!Msg.Ask("Delete " + name + "? This cannot be undone."))
35     {
36         return;
37     }
38
39     string message =
40         CustomerData.Delete(Grid.SelectedID(dgvCustomers,
41         "CustomerID"));
42     if (message ≠ "")
43     {
44         // the database refuses when the customer has bookings or sales,
45         // and says so
46         Msg.Problem(message);
47         return;
48     }
49     LoadCustomers();
50 }
```

**1** **Add opens the form with the number 0**

Lines 1 to 9

0 means a new customer. ShowDialog opens the edit form and waits until it closes. The form answers with a DialogResult, and the answer is OK only when the save worked. On OK the grid is loaded again, so the new customer appears in it.

**2 Edit opens the same form with the picked customer's number**

Lines 11 to 20

The same three lines, with the hidden CustomerID of the picked row in place of 0. The form reads that number and fills its own boxes.

**3 A double-click on a row is an Edit**

Lines 22 to 29

e.RowIndex is the row that was double-clicked. A double-click on the heading row gives -1, so the check keeps that out. Otherwise the Edit button's own method is called, so the two ways of editing share one piece of code.

**4 Delete asks first**

Lines 31 to 37

Grid.SelectedText reads the picked customer's full name, and Msg.Ask puts it into a yes-or-no question. The exclamation mark means not: if the user did not say yes, the method stops at return and nothing happens.

**5 Tell the database, and show its answer**

Lines 39 to 50

CustomerData.Delete sends the number to usp_Customer_Delete and hands back the message. A sentence means the database refused, and Msg.Problem shows that sentence. An empty message means the row is gone, and the grid is loaded again.

# Put the whole file behind the customers screen.

Here is the whole file: the three pieces you read, with the constructor, the Load method, the search box's event and the Close button around them. You replace everything in the code tab with it. Do not press F6 yet: the file names frmCustomerEdit, which you make in the next stage, and one line names AppointmentData, which the next screen deals with.

Copy this one **Forms/frmCustomers.cs: the whole file** C#

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
using System;
using System.Collections.Generic;
using System.Data;
using System.Drawing;
using System.Windows.Forms;
using BikeShop.Data;
using BikeShop.Models;

namespace BikeShop.Forms
{
    public partial class frmCustomers : Form
    {
        public frmCustomers()
        {
            InitializeComponent();
        }

        private void frmCustomers_Load(object sender, EventArgs e)
        {
            LoadCustomers();
        }

        #region LoadCustomers
        // Fills the grid from the database.  Called when the form opens, when
        // the search text changes,
        // and after every add, edit or delete, so the grid always shows what
        // the database holds.
        private void LoadCustomers()
        {
            dgvCustomers.DataSource =
                CustomerData.Search(txtSearch.Text.Trim());
            // hide the ID, give the headers plain words
            Grid.Tidy(dgvCustomers, "CustomerID");
        }
        #endregion

        private void txtSearch_TextChanged(object sender, EventArgs e)
        {
            LoadCustomers();
        }

        #region Selection
        // Edit and Delete only make sense once a row is picked.
        private void dgvCustomers_SelectionChanged(object sender, EventArgs e)
        {
            bool picked = dgvCustomers.SelectedRows.Count > 0;
            btnEdit.Enabled = picked;
            btnDelete.Enabled = picked;
            ShowBookings();
        }

        // The lower grid shows the picked customer's bookings: two tables on
        // one screen.
```

```
54          private void ShowBookings()
55          {
56              if (dgvCustomers.SelectedRows.Count == 0)
57              {
58                  dgvBookings.DataSource = null;
59                  return;
60              }
61
62              dgvBookings.DataSource =
63                  AppointmentData.GetForCustomer(Grid.SelectedID(dgvCustomers,
64                  "CustomerID"));
65              Grid.Tidy(dgvBookings, "AppointmentID");
66          }
67          #endregion
68
69          #region AddEditDelete
70          private void btnAdd_Click(object sender, EventArgs e)
71          {
72              // 0 means "a new customer"
73              frmCustomerEdit edit = new frmCustomerEdit(0);
74              if (edit.ShowDialog(this) == DialogResult.OK)
75              {
76                  LoadCustomers();
77              }
78          }
79
80          private void btnEdit_Click(object sender, EventArgs e)
81          {
82              frmCustomerEdit edit =
83                  new frmCustomerEdit(Grid.SelectedID(dgvCustomers,
84                  "CustomerID"));
85              if (edit.ShowDialog(this) == DialogResult.OK)
86              {
87                  LoadCustomers();
88              }
89          }
90
91          private void dgvCustomers_CellDoubleClick(object sender,
92              DataGridViewCellEventArgs e)
93          {
94              if (e.RowIndex ≥ 0)
95              {
96                  btnEdit_Click(sender, e);
97              }
98          }
99
100         private void btnDelete_Click(object sender, EventArgs e)
101         {
102             string name = Grid.SelectedText(dgvCustomers, "FullName");
103             if (!Msg.Ask("Delete " + name + "? This cannot be undone."))
104             {
105                 return;
106             }
107
108             string message =
109                 CustomerData.Delete(Grid.SelectedID(dgvCustomers,
110                 "CustomerID"));
111             if (message ≠ "")
```

```
112             {
113                 // the database refuses when the customer has bookings or sales,
114                 // and says so
115                 Msg.Problem(message);
116                 return;
117             }
118             LoadCustomers();
119         }
120         #endregion
121
122         private void btnClose_Click(object sender, EventArgs e)
123         {
124             Close();
125         }
126     }
127 }
```

**1 The usings**

Lines 1 to 7

These lines bring in the tools the form needs: Windows Forms for the window itself, System.Data for the table that fills a grid, System.Drawing for colours, and your own Data and Models folders.

**2 The class behind the form**

Lines 9 to 12

partial means this class is in two files. The designer keeps the controls you dragged in frmCustomers.Designer.cs, and this file holds what the controls do. : Form means the class is a window.

**3 Made, then loaded**

Lines 13 to 21

The constructor runs when the main menu makes the form with new. InitializeComponent builds the controls you dragged. Then, when the form loads, the grid is filled for the first time.

**4 Loading the grid**

Lines 23 to 35

One trip to the database and one call to Grid.Tidy. This is the piece you read first.

**5 Search as you type**

Lines 37 to 40

Every change to the search box loads the grid again with the new text.

**6 When a row is picked**

Lines 42 to 67

Edit and Delete follow the picked row, and the lower grid shows that customer's bookings.

**7 The three buttons**

Lines 69 to 120

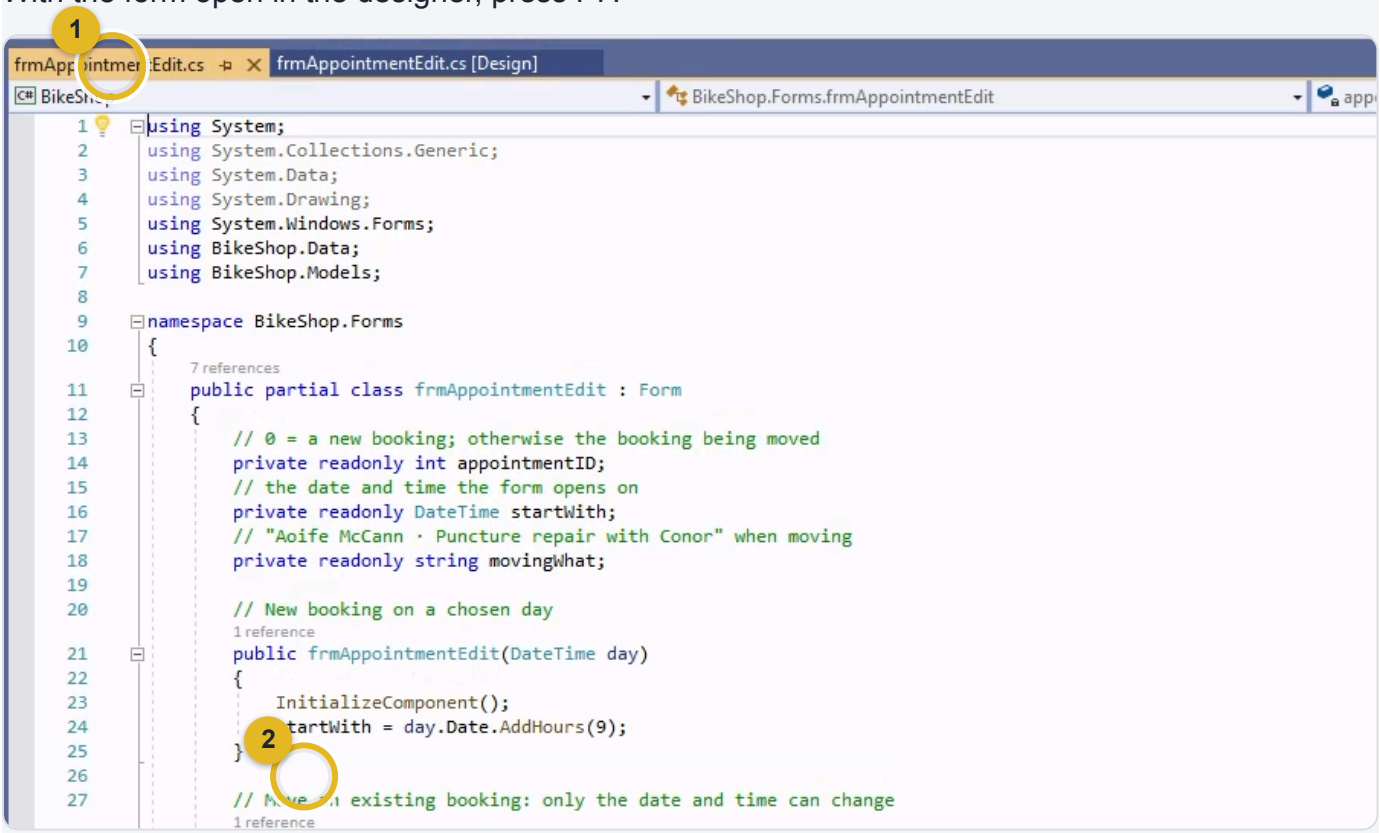
Add and Edit open the edit form and wait for OK. Delete asks, tells the database, and shows its answer.

**8 Close**

Lines 122 to 127

Close shuts the window and the main menu is back. The two closing braces end the class and the namespace.

With the form open in the designer, press F7.




F7 shows the code behind the form that is open in the designer. Shift and F7 go back to the designer.

1. The code tab is in front.

2. Select everything here before you paste over it.

> **You should see** The code tab for the form comes to the front.

Press the Copy button under this step.

> **You should see** The button reads Copied for a moment.

Click anywhere in the code tab and press Ctrl and A together.

> **You should see** Everything in the code tab is highlighted.

Press Ctrl and V together to paste over the highlighted text.

> **You should see** The file ends with btnClose_Click. Red wavy lines sit under frmCustomerEdit and under AppointmentData, because neither exists yet. That is expected.

# Park the bookings line until chapter 7.

One line in ShowBookings calls AppointmentData.GetForCustomer. AppointmentData is the data class for bookings, and you build it in chapter 7, because it belongs with the diary. Until then the program cannot build with that line in it. So you park the line: two slashes at the front turn a line into a comment, which the program ignores and the editor shows in green. In its place you type two lines that call the same stored procedure straight through Db, which you have. The bookings grid works from today, and in chapter 7, straight after step 7.5.10 adds AppointmentData, you swap the lines back.

In the code tab, find ShowBookings, and select the three lines from dgvBookings.DataSource = down to the line that reads "CustomerID"));, by clicking at the start of the first line and dragging to the end of the third.

> **You should see** The three lines are highlighted.

Press Ctrl and K together, then Ctrl and C together.

> **You should see** The three lines turn green, each with two slashes at the front. That is the comment shortcut in Visual Studio 2019.

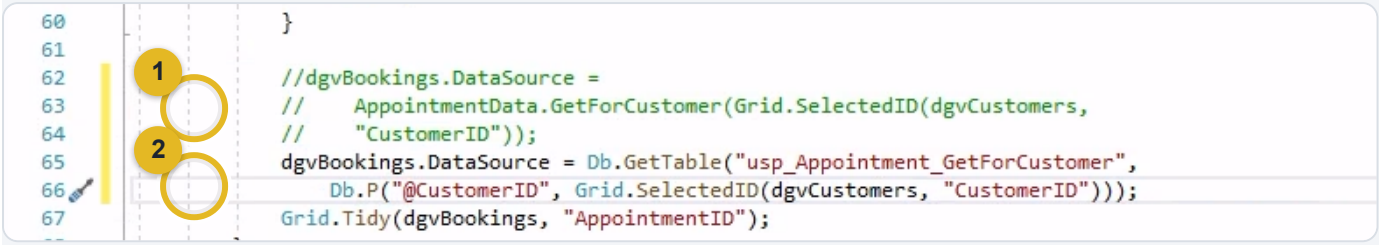
Click at the end of the third green line and press Enter.

> **You should see** The cursor blinks on a new empty line under the green lines.

Type the first new line exactly as it reads here, with the comma on the end: dgvBookings.DataSource = Db.GetTable("usp_Appointment_GetForCustomer", and nothing after the comma.

> **You should see** The line ends with a comma, so the editor waits for more. If the editor adds a closing bracket after the comma by itself, leave it: it moves down as you type the next line, and your own brackets replace it.

Press Enter and type the second new line exactly as it reads here, with the semicolon on the end: Db.P("@CustomerID", Grid.SelectedID(dgvCustomers, "CustomerID"))); and nothing after the semicolon.




The parked lines in green, and the two lines that stand in for them until chapter 7.

1. Three green lines: parked, not deleted. Chapter 7 brings them back.

2. The two lines you type. They call the same stored procedure through Db.

> **You should see** The two typed lines sit under the three green ones, and the only red wavy lines left are under frmCustomerEdit.

**IF IT GOES WRONG** A red wavy line under Db

The form cannot see your Db class. Check that the file still begins with using BikeShop.Data, and that Db is in the Data folder from chapter 5.

**IF IT GOES WRONG** A red wavy line at the end of the typed line

Count the closing brackets: three, then the semicolon. One bracket closes SelectedID, one closes Db.P, and one closes Db.GetTable.

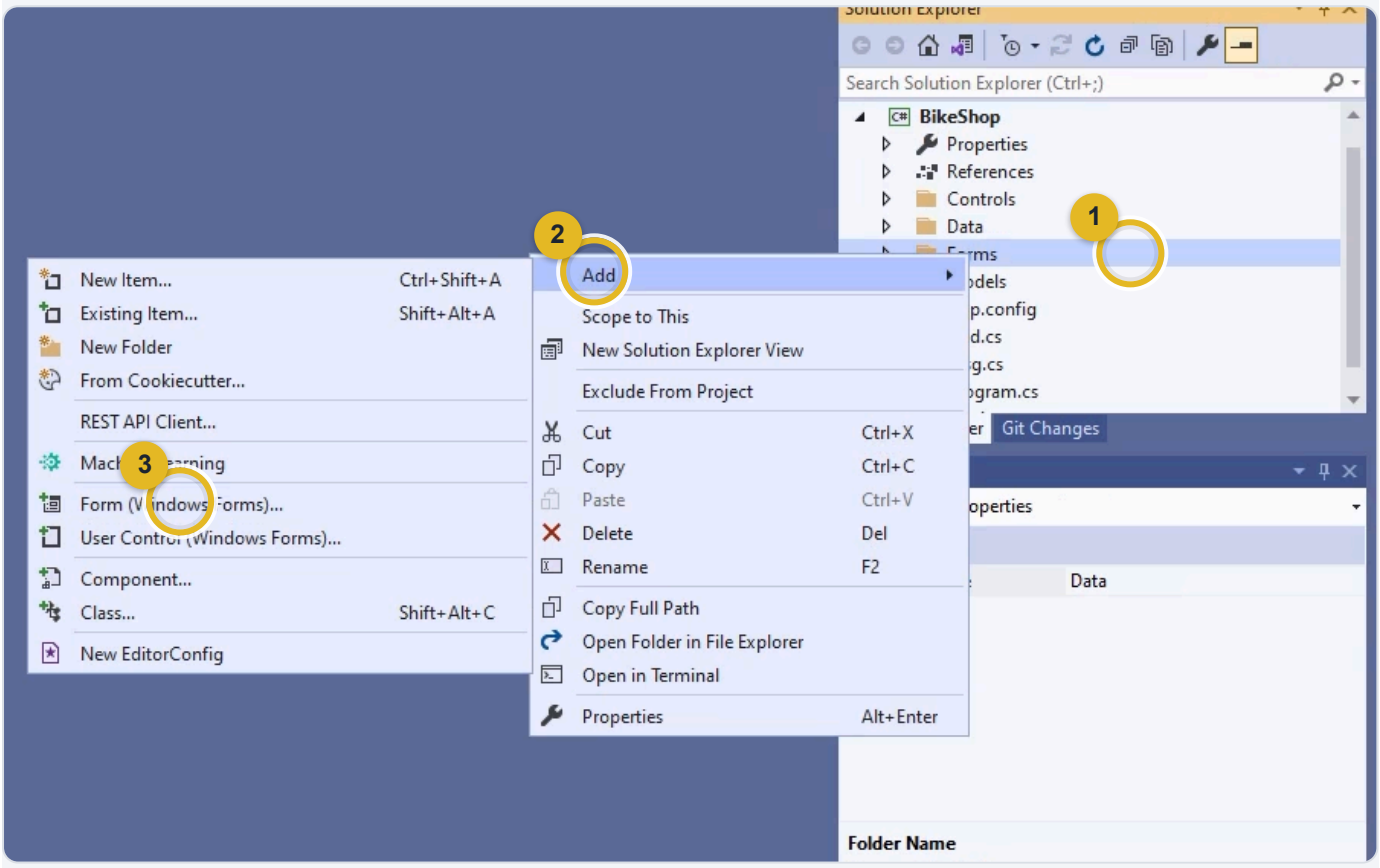
### In chapter 7

Straight after step 7.5.10 adds AppointmentData, come back to frmCustomers.cs. Delete the two lines you typed, select the three green lines, and press Ctrl and K together, then Ctrl and U together: the slashes come off, and ShowBookings calls AppointmentData the way the finished program does. The program works either way, because both versions run the same stored procedure, so nothing is lost if you do it later.

# Add the form frmCustomerEdit in Visual Studio 2019.

Add and Edit open the same small window. Opened with the number 0 it is titled Add a customer and starts empty. Opened with a customer's number it is titled Edit customer and fills its boxes from the database first. One form, two jobs, and the number decides which.

In Solution Explorer in Visual Studio 2019, right-click the Forms folder, point at Add, and click Form (Windows Forms)…




Adding a form to the Forms folder. Taken in the finished project, so Solution Explorer lists more files than yours.

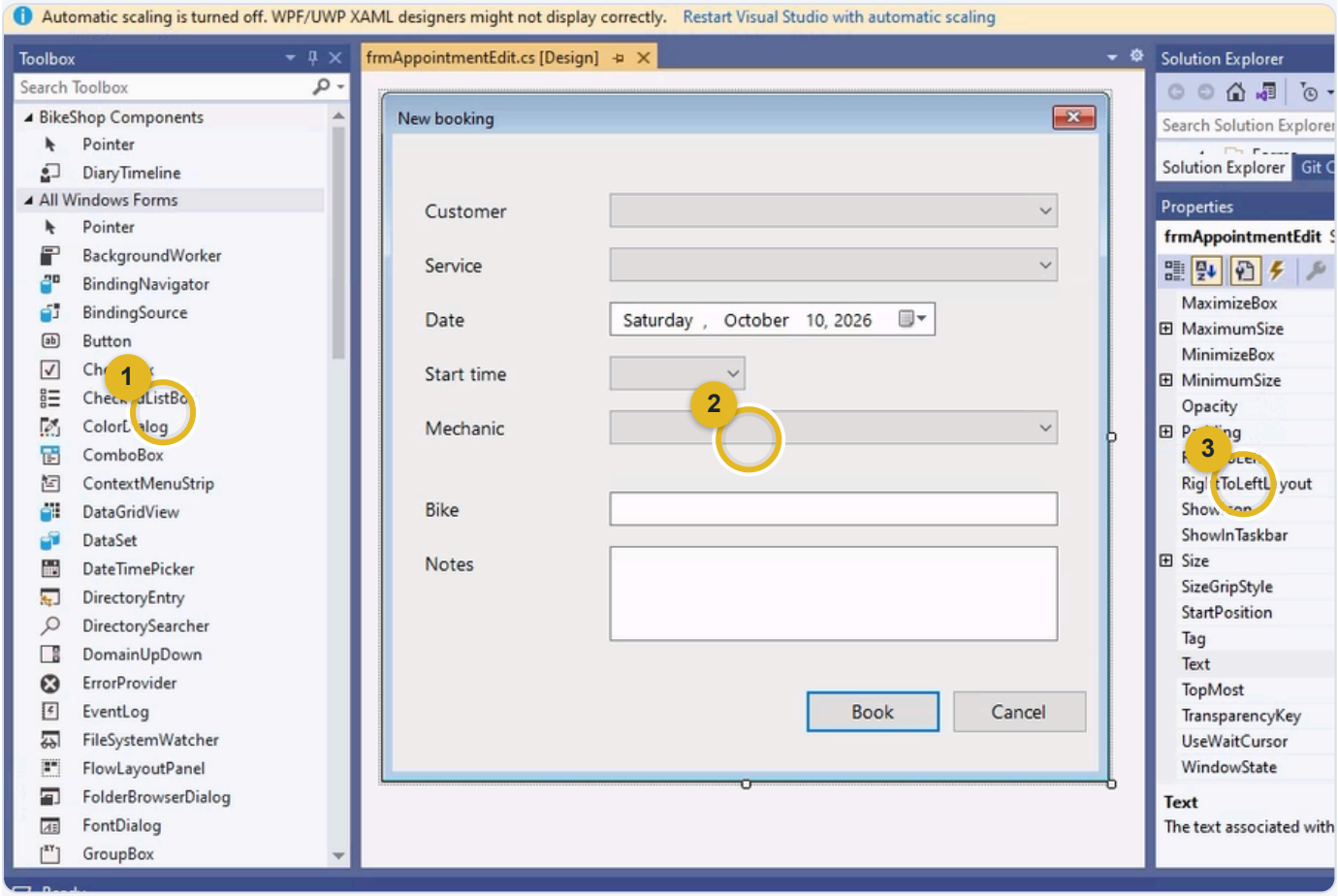
1. Right-click the Forms folder.
2. Point at Add.
3. Click Form (Windows Forms)…, type the name, press Add.

> **You should see** The Add New Item window opens with Form (Windows Forms) selected.

In the Name box at the bottom, type frmCustomerEdit and press Add.

> **You should see** An empty form opens in the designer.

Open the Toolbox (View, then Toolbox) and the Properties window (View, then Properties Window), if they are not already open.




The three parts of the designer: the Toolbox you drag from, the form you drop on, the Properties window where you name and set each control.

1. The Toolbox. Drag a control from here on to the form.

2. The form. Drop the control where the picture shows it.

3. The Properties window. Set Name first, then the rest of the row.

> **You should see** The Toolbox is on the left, the form in the middle, the Properties window on the right.

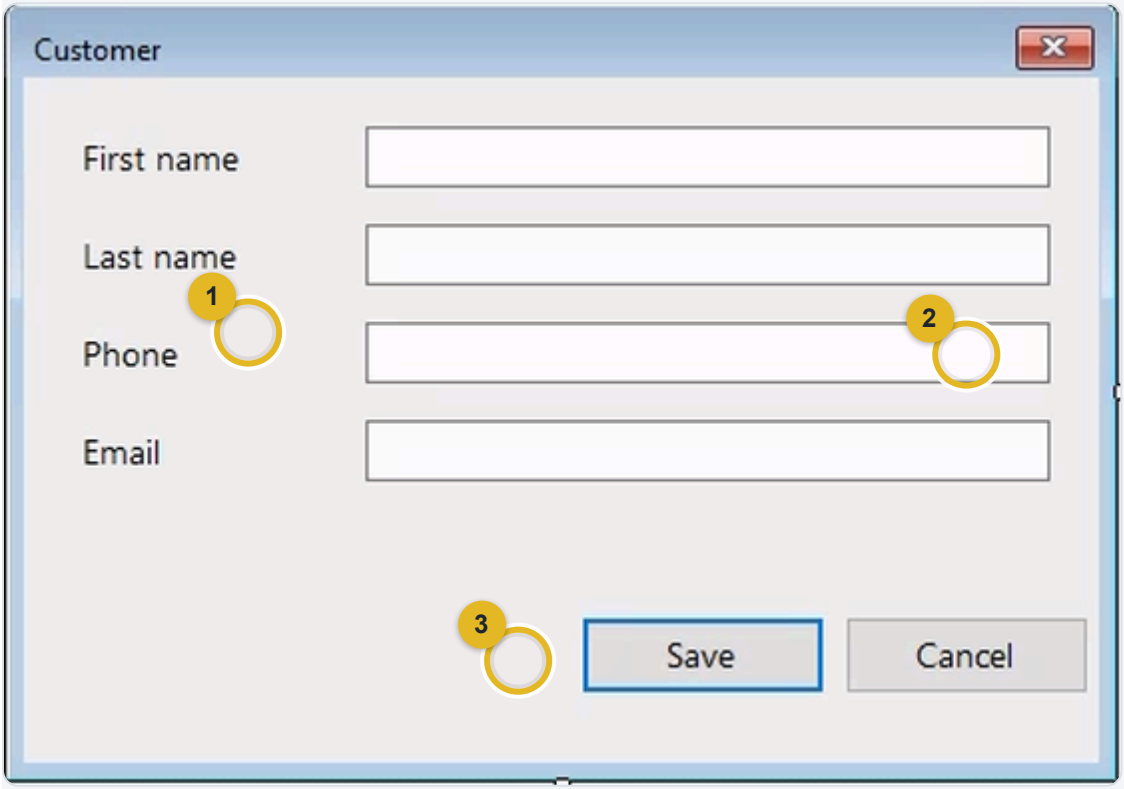
# Set out the controls on frmCustomerEdit.

Four labels, four boxes and two buttons. Click the form itself first and set the form's own properties. Then work down the table: drag the control from the Toolbox, drop it on the form, set its Name in the Properties window, and set the other settings in the row. Where the Then column says double-click, double-clicking the control makes its event method for you. Two settings on the form itself matter here: AcceptButton = btnSave means the Enter key presses Save, and CancelButton = btnCancel means the Escape key presses Cancel.

One window does both Add and Edit. It opens empty for a new customer, or filled in for an existing one.

**Click the form itself, then set these in the Properties window**

| | | | |
|---|---|---|---|
| Text | `Customer` | Size | `440, 280` |
| FormBorderStyle | `FixedDialog` | StartPosition | `CenterParent` |
| MaximizeBox | `False` | MinimizeBox | `False` |
| ShowInTaskbar | `False` | AcceptButton | `btnSave (after you add it)` |
| CancelButton | `btnCancel (after you add it)` | | |




frmCustomerEdit in the designer. A small fixed window: four boxes and two buttons.

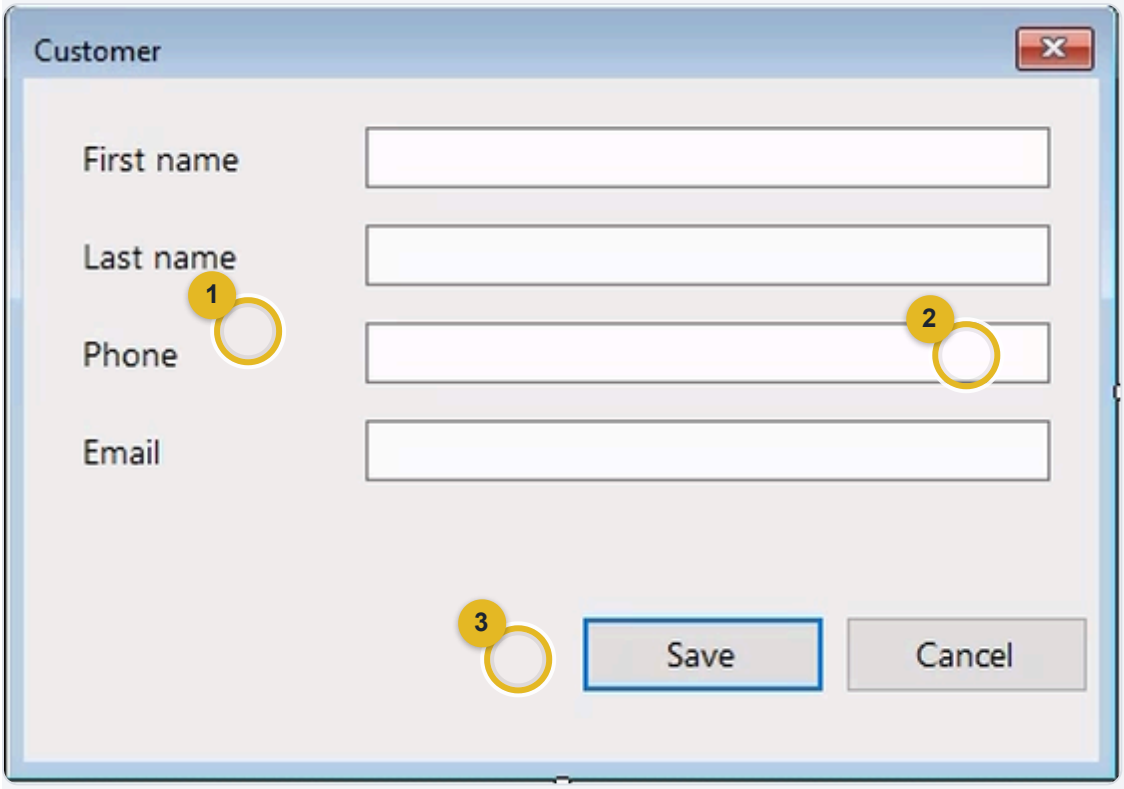
1. The four labels: lblFirstName, lblLastName, lblPhone and lblEmail.
2. The four boxes: txtFirstName, txtLastName, txtPhone and txtEmail.
3. btnSave, then btnCancel.

| DRAG THIS FROM THE TOOLBOX | NAME IT (THE NAME PROPERTY) | SET THESE IN THE PROPERTIES WINDOW | THEN |
|---|---|---|---|
| Label | `lblFirstName` | `Text = First name` | |
| TextBox | `txtFirstName` | nothing else | |
| Label | `lblLastName` | `Text = Last name` | |
| TextBox | `txtLastName` | nothing else | |
| Label | `lblPhone` | `Text = Phone` | |
| TextBox | `txtPhone` | nothing else | |
| Label | `lblEmail` | `Text = Email` | |
| TextBox | `txtEmail` | nothing else | |
| Button | `btnSave` | `Text = Save`<br>Checks every box, stops at the first problem and shows it in a message, then saves. | Double-click it. Visual Studio makes `btnSave_Click` for you; the code goes inside it. |
| Button | `btnCancel` | `Text = Cancel`<br>`DialogResult = Cancel`<br>Closes without saving. No code needed: set DialogResult to Cancel. | |

Click the form itself and set Text to Customer, Size to 440, 280 and FormBorderStyle to FixedDialog.

> **You should see** A small window with a title bar reading Customer and no minimise or maximise buttons.

Work down the table: drag each control from the Toolbox on to the form, set its Name, then set the rest of its row.




frmCustomerEdit in the designer. A small fixed window: four boxes and two buttons.

1. The four labels: lblFirstName, lblLastName, lblPhone and lblEmail.
2. The four boxes: txtFirstName, txtLastName, txtPhone and txtEmail.
3. btnSave, then btnCancel.

> **You should see** The form looks like the picture, with every control named as the table says.

Double-click the Save button.

> **You should see** The code tab opens with an empty method called btnSave_Click. Press Shift and F7 together to go back to the designer.

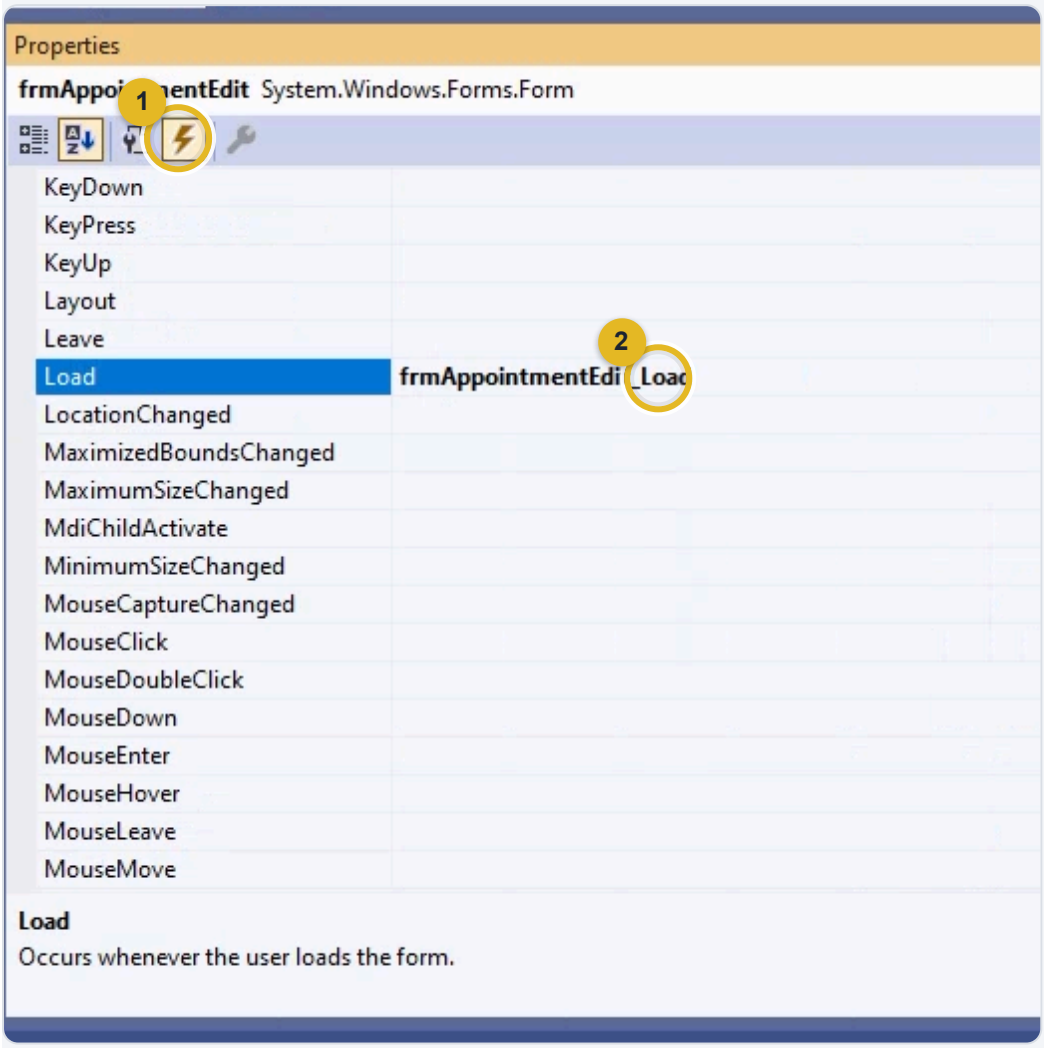
Click the Cancel button and, in the Properties window, set DialogResult to Cancel.

> **You should see** The Properties window shows DialogResult set to Cancel. Cancel needs no code: a button with DialogResult set closes the form by itself and tells the form that opened it that Cancel was pressed.

Click the form itself, set AcceptButton to btnSave and CancelButton to btnCancel.

> **You should see** Both settings show the button names in the Properties window.

With the form still picked, press the lightning bolt and double-click the empty box beside Load, which is part of the way down, after Leave.




The lightning bolt shows the events, in alphabetical order. Double-click the empty box beside one to make its method.

1. Press the lightning bolt.

2. Double-click the empty box beside the event your step names. The events are in alphabetical order.

> **You should see** The code tab opens with an empty method called frmCustomerEdit_Load.

# Read how one form does two jobs: the constructor.

A constructor is the method that runs when an object is made with new. This form's constructor takes one number, and keeps it in a field called customerID. A field is a variable that belongs to the whole form, so every method in the form can read it. readonly means it is set once, in the constructor, and never changes.

Read this one **Forms/frmCustomerEdit.cs: the constructor and Load** C#

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  private readonly int customerID;
2
3  public frmCustomerEdit(int id)
4  {
5      InitializeComponent();
6      customerID = id;
7  }
8
9  private void frmCustomerEdit_Load(object sender, EventArgs e)
10 {
11     // The boxes stop typing at the length the database column holds.
12     txtFirstName.MaxLength = 40;
13     txtLastName.MaxLength = 40;
14     txtPhone.MaxLength = 15;
15     txtEmail.MaxLength = 100;
16
17     if (customerID == 0)
18     {
19         Text = "Add a customer";
20         return;
21     }
22
23     Text = "Edit customer";
24     Customer customer = CustomerData.GetByID(customerID);
25     txtFirstName.Text = customer.FirstName;
26     txtLastName.Text = customer.LastName;
27     txtPhone.Text = customer.Phone;
28     txtEmail.Text = customer.Email;
```

**1 The constructor remembers which customer**

Lines 1 to 7

frmCustomerEdit(int id) is the constructor, and id is the number the list form passed in the brackets. InitializeComponent builds the controls, then the number is kept in customerID.

**2 Four boxes, four limits**

Lines 9 to 15

MaxLength is the most letters a box will take. Each limit matches the width of its column in the Customer table from chapter 3, so the database can never be given text that is too long.

**3 Adding: an empty form**

Lines 17 to 21

Text is the words in the title bar. When the number is 0 the title says Add a customer and the method stops at return, leaving the boxes empty.

**4 Editing: fill the boxes from the database**

Lines 23 to 28

Otherwise the title says Edit customer, CustomerData.GetByID fetches that customer as an object, and each property is copied into its box.

# Read Save: plain ifs, and the first problem wins.

Save checks the boxes one at a time with plain ifs. The first problem found is shown in a message, the cursor is put in that box, and the method stops. Nothing is sent to the database until every check has passed. Then one object is built, Add or Update is called depending on the number, and the database gets the last word.

Read this one **Forms/frmCustomerEdit.cs: the Save button and the two checks** C#

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
private void btnSave_Click(object sender, EventArgs e)
{
    // Check what was typed, one problem at a time: show the first and
    // put the cursor in that box.
    if (txtFirstName.Text.Trim() == "")
    {
        Msg.Problem("Type the customer's first name.");
        txtFirstName.Focus();
        return;
    }
    if (txtLastName.Text.Trim() == "")
    {
        Msg.Problem("Type the customer's last name.");
        txtLastName.Focus();
        return;
    }
    string phoneProblem = PhoneProblem(txtPhone.Text.Trim());
    if (phoneProblem ≠ "")
    {
        Msg.Problem(phoneProblem);
        txtPhone.Focus();
        return;
    }
    string emailProblem = EmailProblem(txtEmail.Text.Trim());
    if (emailProblem ≠ "")
    {
        Msg.Problem(emailProblem);
        txtEmail.Focus();
        return;
    }

    Customer customer = new Customer();
    customer.CustomerID = customerID;
    customer.FirstName = txtFirstName.Text.Trim();
    customer.LastName = txtLastName.Text.Trim();
    customer.Phone = txtPhone.Text.Trim();
    customer.Email = txtEmail.Text.Trim();
    if (customer.Email == "")
    {
        // nothing typed: store a NULL, not an empty piece of text
        customer.Email = null;
    }

    string message;
    if (customerID == 0)
    {
        message = CustomerData.Add(customer);
    }
    else
    {
        message = CustomerData.Update(customer);
    }
    if (message ≠ "")
```

```
    {
        Msg.Problem(message);
        return;
    }
    // closes this form and tells the list to refresh
    DialogResult = DialogResult.OK;
}

// Returns "" when the phone number is fine; otherwise a sentence saying
// what is wrong with it.
private string PhoneProblem(string phone)
{
    if (phone == "")
    {
        return "Type the customer's phone number.";
    }

    int digits = 0;
    foreach (char letter in phone)
    {
        if (char.IsDigit(letter))
        {
            digits = digits + 1;
        }
        else if (letter != ' ' && letter != '+' && letter != '-')
        {
            return "A phone number can only have digits, spaces, + " +
                "and -.";
        }
    }
    if (digits < 10)
    {
        return "A phone number needs at least 10 digits.";
    }
    return "";
}

// Returns "" when the email is fine.  An email is optional, so a blank
// box is fine too.
private string EmailProblem(string email)
{
    if (email == "")
    {
        return "";
    }

    int at = email.IndexOf('@');
    if (at < 1 || email.IndexOf('.', at) < at + 2
        || email.EndsWith("."))
    {
        return "An email address must look like name@example.com.";
    }
    return "";
}
```

**1** **First name: must not be empty**

Lines 1 to 10

Trim takes the spaces off both ends, so a box with only spaces in it counts as empty. Msg.Problem shows the sentence, Focus puts the cursor in the box, and return stops the method.

**2 Last name: the same check**

Lines 11 to 16

The same three lines for the last name. This is the check you prove at the end of the chapter.

**3 Phone: ask PhoneProblem**

Lines 17 to 23

The phone check is longer, so it lives in its own method below. PhoneProblem answers with an empty string when the number is fine, or a sentence saying what is wrong. A sentence means stop, show it, and put the cursor in the phone box.

**4 Email: ask EmailProblem**

Lines 24 to 30

The email check has the same shape as the phone check: ask EmailProblem, and if it answers with a sentence, show it, put the cursor in the Email box and stop.

**5 Build one Customer object from the boxes**

Lines 32 to 42

new Customer() makes an empty object, and each box's text goes into its property. An empty email box becomes null, so the database stores NULL and not an empty piece of text.

**6 Add or Update, decided by the number**

Lines 44 to 52

The same number the constructor kept. 0 means Add; anything else means Update. Both hand back the database's message.

**7 The database's answer**

Lines 53 to 57

A sentence means the database refused, and the user reads that sentence. The form stays open so the boxes can be changed.

**8 Tell the list it worked**

Lines 59 to 60

Setting DialogResult closes the form and hands OK back to ShowDialog in the list form, which loads the grid again.

**9 PhoneProblem: an empty box**

Lines 64 to 69

The method takes the phone text and answers with a sentence or an empty string. An empty box gets the first sentence.

**10 Count the digits, refuse anything else**

Lines 71 to 83

foreach looks at each letter in turn. char.IsDigit asks whether the letter is 0 to 9, and digits counts them. A space, a plus or a dash is allowed and ignored. Any other letter means the method answers at once with

the sentence about what a phone number may contain.

**11 At least ten digits**

Lines 84 to 89

Fewer than ten digits is refused with its own sentence. Otherwise the method answers with an empty string: the number is fine. The database checks the same rule with CK_Customer_Phone from chapter 3, so a number that slipped past the form would still be refused.

**12 EmailProblem: a blank box is fine**

Lines 93 to 98

An email is optional, so an empty box answers with an empty string straight away.

**13 The shape of an email**

Lines 100 to 107

IndexOf finds where the @ sign is, counting from 0. The if refuses when the @ is missing or first, when there is no dot at least two letters after it, or when the email ends with a dot. Anything else is accepted, because a perfect check would be pages long and the shop only needs a sensible one.

# Put the whole file behind the edit form, and build.

Here is the whole file: the constructor, Load and the Save region you read, in one piece. After the paste you press F6, and this time the build must succeed, because frmCustomerEdit now exists for frmCustomers to open.

Copy this one **Forms/frmCustomerEdit.cs: the whole file** C#

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
using System;
using System.Collections.Generic;
using System.Data;
using System.Drawing;
using System.Windows.Forms;
using BikeShop.Data;
using BikeShop.Models;

namespace BikeShop.Forms
{
    public partial class frmCustomerEdit : Form
    {
        // 0 = adding; otherwise the customer being edited
        private readonly int customerID;

        public frmCustomerEdit(int id)
        {
            InitializeComponent();
            customerID = id;
        }

        private void frmCustomerEdit_Load(object sender, EventArgs e)
        {
            // The boxes stop typing at the length the database column holds.
            txtFirstName.MaxLength = 40;
            txtLastName.MaxLength = 40;
            txtPhone.MaxLength = 15;
            txtEmail.MaxLength = 100;

            if (customerID == 0)
            {
                Text = "Add a customer";
                return;
            }

            Text = "Edit customer";
            Customer customer = CustomerData.GetByID(customerID);
            txtFirstName.Text = customer.FirstName;
            txtLastName.Text = customer.LastName;
            txtPhone.Text = customer.Phone;
            txtEmail.Text = customer.Email;
        }

        #region Save
        private void btnSave_Click(object sender, EventArgs e)
        {
            // Check what was typed, one problem at a time: show the first and
            // put the cursor in that box.
            if (txtFirstName.Text.Trim() == "")
            {
                Msg.Problem("Type the customer's first name.");
                txtFirstName.Focus();
                return;
```

```
            }
            if (txtLastName.Text.Trim() == "")
            {
                Msg.Problem("Type the customer's last name.");
                txtLastName.Focus();
                return;
            }
            string phoneProblem = PhoneProblem(txtPhone.Text.Trim());
            if (phoneProblem != "")
            {
                Msg.Problem(phoneProblem);
                txtPhone.Focus();
                return;
            }
            string emailProblem = EmailProblem(txtEmail.Text.Trim());
            if (emailProblem != "")
            {
                Msg.Problem(emailProblem);
                txtEmail.Focus();
                return;
            }

            Customer customer = new Customer();
            customer.CustomerID = customerID;
            customer.FirstName = txtFirstName.Text.Trim();
            customer.LastName = txtLastName.Text.Trim();
            customer.Phone = txtPhone.Text.Trim();
            customer.Email = txtEmail.Text.Trim();
            if (customer.Email == "")
            {
                // nothing typed: store a NULL, not an empty piece of text
                customer.Email = null;
            }

            string message;
            if (customerID == 0)
            {
                message = CustomerData.Add(customer);
            }
            else
            {
                message = CustomerData.Update(customer);
            }
            if (message != "")
            {
                Msg.Problem(message);
                return;
            }
            // closes this form and tells the list to refresh
            DialogResult = DialogResult.OK;
        }

        // Returns "" when the phone number is fine; otherwise a sentence saying
        // what is wrong with it.
        private string PhoneProblem(string phone)
        {
            if (phone == "")
            {
```

```
112                 return "Type the customer's phone number.";
113             }
114
115             int digits = 0;
116             foreach (char letter in phone)
117             {
118                 if (char.IsDigit(letter))
119                 {
120                     digits = digits + 1;
121                 }
122                 else if (letter ≠ ' ' && letter ≠ '+' && letter ≠ '-')
123                 {
124                     return "A phone number can only have digits, spaces, + " +
125                         "and -.";
126                 }
127             }
128             if (digits < 10)
129             {
130                 return "A phone number needs at least 10 digits.";
131             }
132             return "";
133         }
134
135         // Returns "" when the email is fine.  An email is optional, so a blank
136         // box is fine too.
137         private string EmailProblem(string email)
138         {
139             if (email == "")
140             {
141                 return "";
142             }
143
144             int at = email.IndexOf('@');
145             if (at < 1 || email.IndexOf('.', at) < at + 2
146                 || email.EndsWith("."))
147             {
148                 return "An email address must look like name@example.com.";
149             }
150             return "";
151         }
152         #endregion
153     }
154 }
```

**1 The usings**

Lines 1 to 7

These lines bring in the tools the form needs: Windows Forms for the window itself, System.Data for the table that fills a grid, System.Drawing for colours, and your own Data and Models folders.

**2 The class behind the form**

Lines 9 to 12

partial again: the designer keeps the four boxes and two buttons in frmCustomerEdit.Designer.cs, and this file holds what they do.

**3 The constructor remembers which customer**

Lines 14 to 20

The number from the brackets is kept in customerID for the whole life of the form.

**4 Load: Add or Edit**

Lines 22 to 42

Load sets the four length limits, then makes either an empty form titled Add a customer or a filled one titled Edit customer.

**5 Save: the first problem wins**

Lines 44 to 104

Four plain ifs, one object, Add or Update, and the database's answer. This is the piece you read on the last screen.

**6 PhoneProblem**

Lines 108 to 133

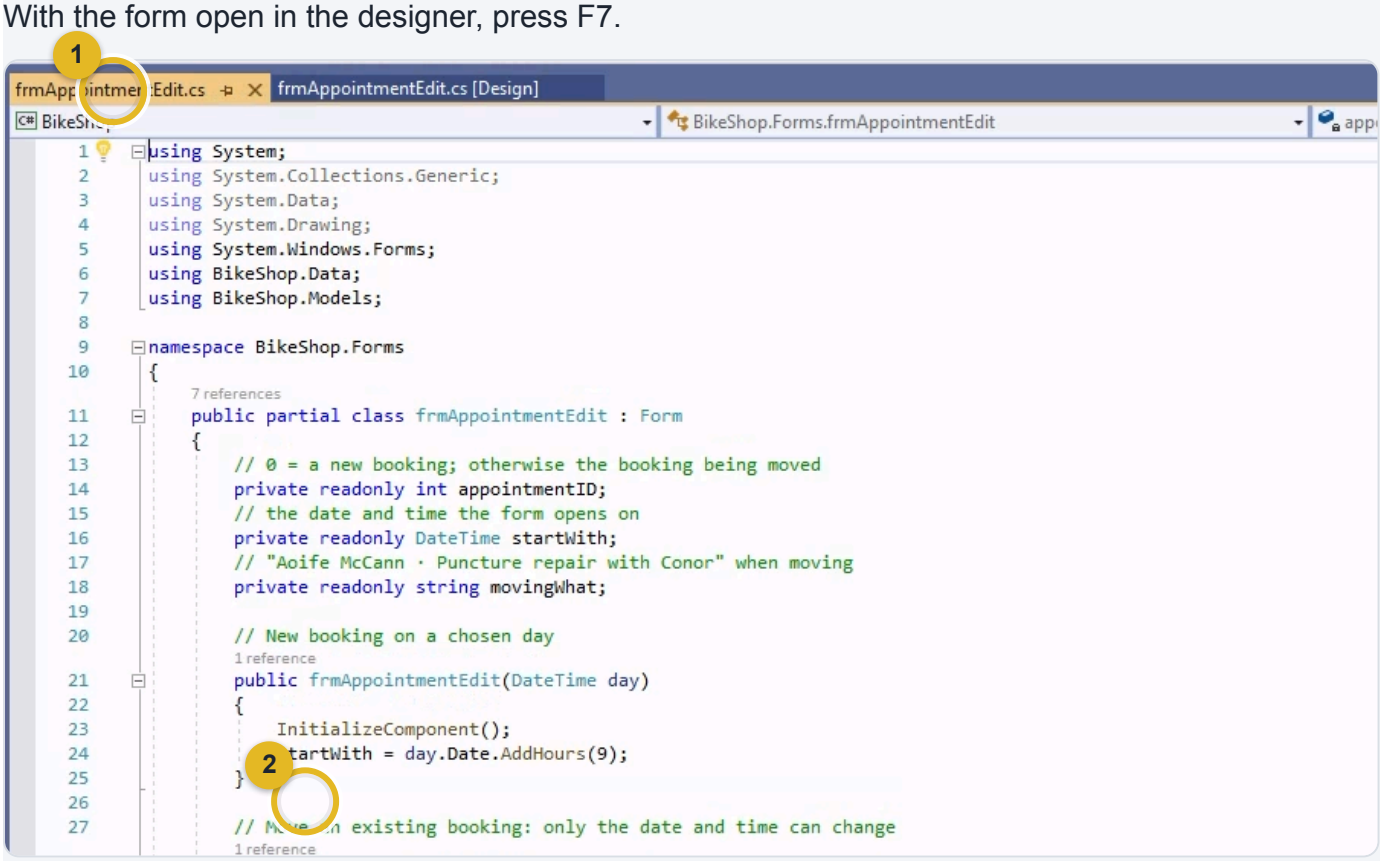
PhoneProblem answers with an empty string when the phone number is fine, and otherwise with the sentence to show.

**7 EmailProblem**

Lines 137 to 154

A blank box is fine, and anything else must look like name@example.com. The closing braces end the region, the class and the namespace.

With the form open in the designer, press F7.




F7 shows the code behind the form that is open in the designer. Shift and F7 go back to the designer.

1 The code tab is in front.

2 Select everything here before you paste over it.

> **You should see** The code tab for the form comes to the front.

Press the Copy button under this step.

> **You should see** The button reads Copied for a moment.

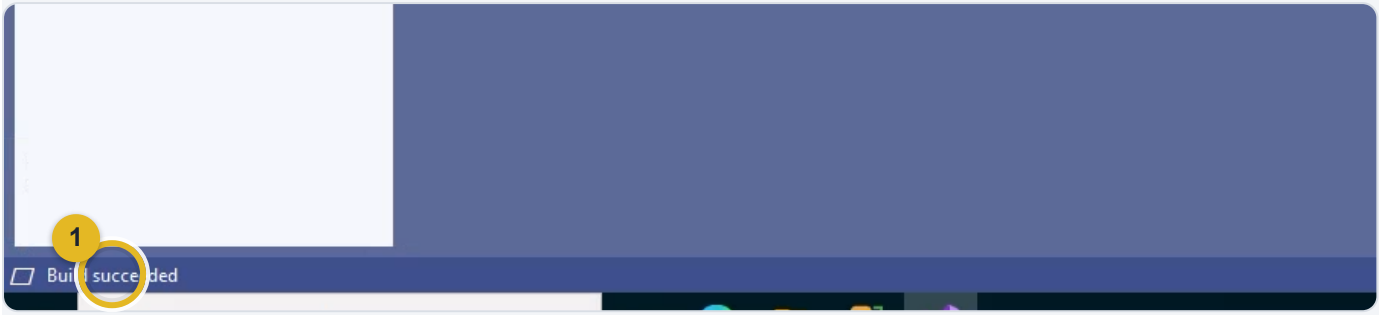
Click anywhere in the code tab and press Ctrl and A together.

> **You should see** Everything in the code tab is highlighted.

Press Ctrl and V together to paste over the highlighted text.

> **You should see** The file ends with EmailProblem, and no red wavy line anywhere.

Press F6 to build the project.




Build succeeded, at the bottom left. Anything else means an error list to read.

1 Build succeeded. Press F6 to build at any time.

**You should see** Build succeeded, at the bottom left.

**IF IT GOES WRONG** 'frmCustomerEdit' does not contain a definition for 'button1_Click' (or another old name)

The event was made before the control was given its Name, so the designer still calls the method by its old name. Double-click the error: Visual Studio 2019 opens frmCustomerEdit.Designer.cs on the line that hooks up the event. At the end of that line, change the old name to btnSave_Click and press F6 again.

**IF IT GOES WRONG** 'frmCustomers' does not contain a definition for 'button1_Click' (or another old name)

The event was made before the control was given its Name, so the designer still calls the method by its old name. Double-click the error: Visual Studio 2019 opens frmCustomers.Designer.cs on the line that hooks up the event. At the end of that line, change the old name to btnAdd_Click and press F6 again.

**IF IT GOES WRONG** 'frmCustomerEdit' does not contain a definition for 'txtFirstName'

A control on the form has a different Name from the one in the table. Open the form in the designer, click the control, and check its Name in the Properties window, letter by letter, against the table.

**IF IT GOES WRONG** 'frmCustomers' does not contain a definition for 'dgvBookings'

A control on the form has a different Name from the one in the table. Open the form in the designer, click the control, and check its Name in the Properties window, letter by letter, against the table.

**IF IT GOES WRONG** 'The name Db does not exist' or 'The name Msg does not exist'

Your helper classes from chapter 5 are called something else, or sit in a different namespace. Match the names you used there.

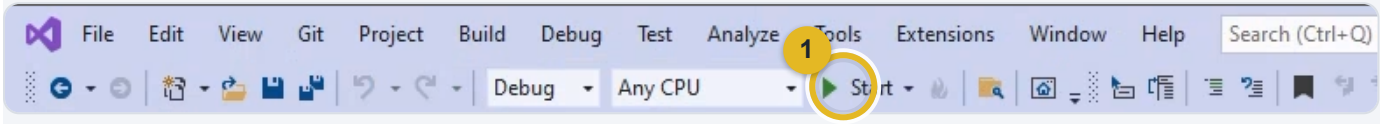
**IF IT GOES WRONG** 'The name AppointmentData does not exist in the current context'

The three AppointmentData lines in frmCustomers.cs do not start with two slashes. Go back to step 6.3.14 and type two slashes (//) at the start of each of the three lines.

# Open the customers screen from the main menu.

The Customers button on the main menu already opens this form: chapter 5 pasted the two lines behind it, frmCustomers customers = new frmCustomers(); and customers.ShowDialog(this);. So the program is ready to run. Start it in Visual Studio 2019, sign in, and open the screen.

Press F5 to start the program.

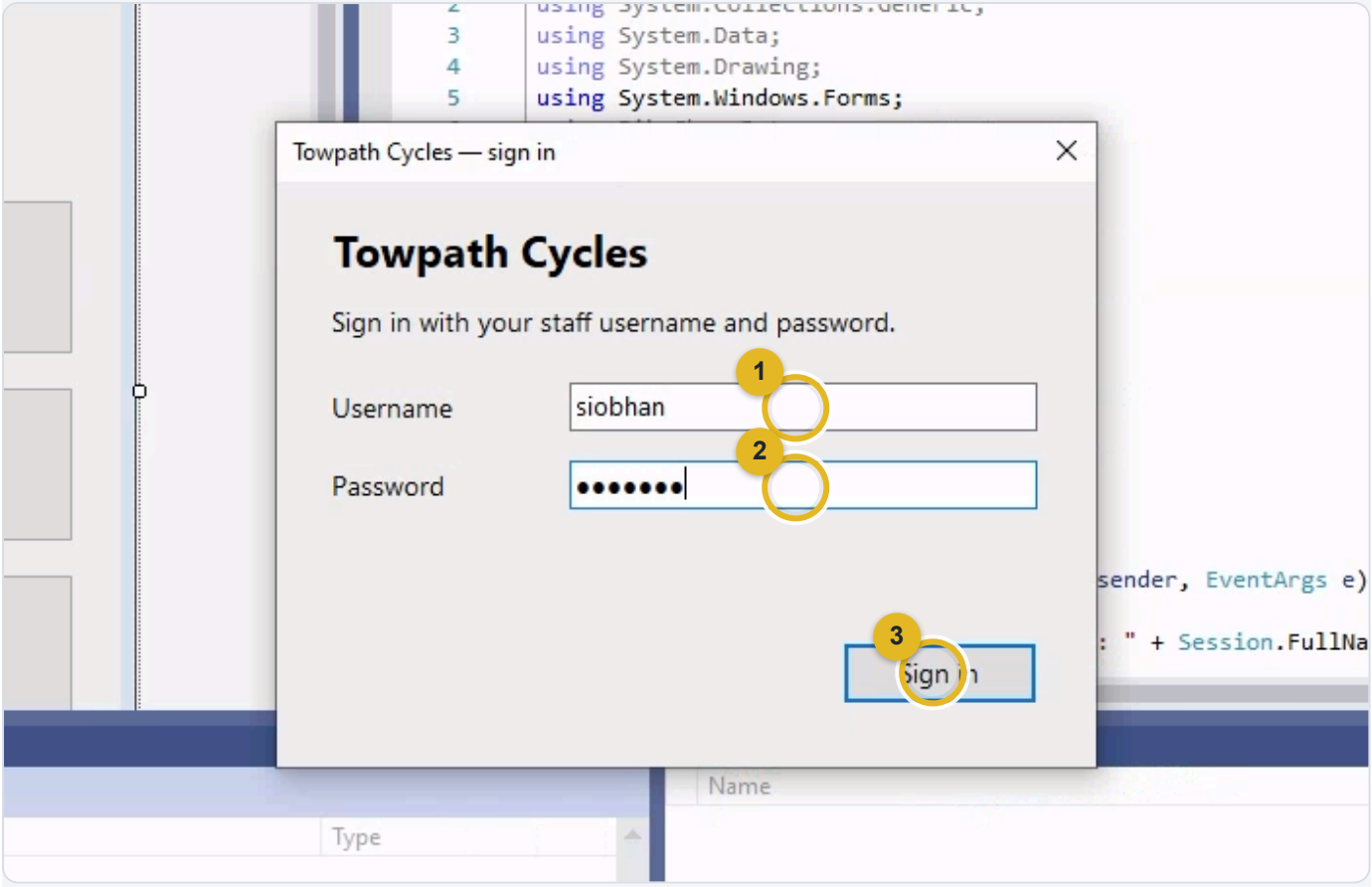



Start runs your program. Close the program window to come back to Visual Studio 2019.

1. Click Start, or press F5.

> **You should see** The sign-in window opens.

Type siobhan as the username, canal01 as the password, and click Sign in.

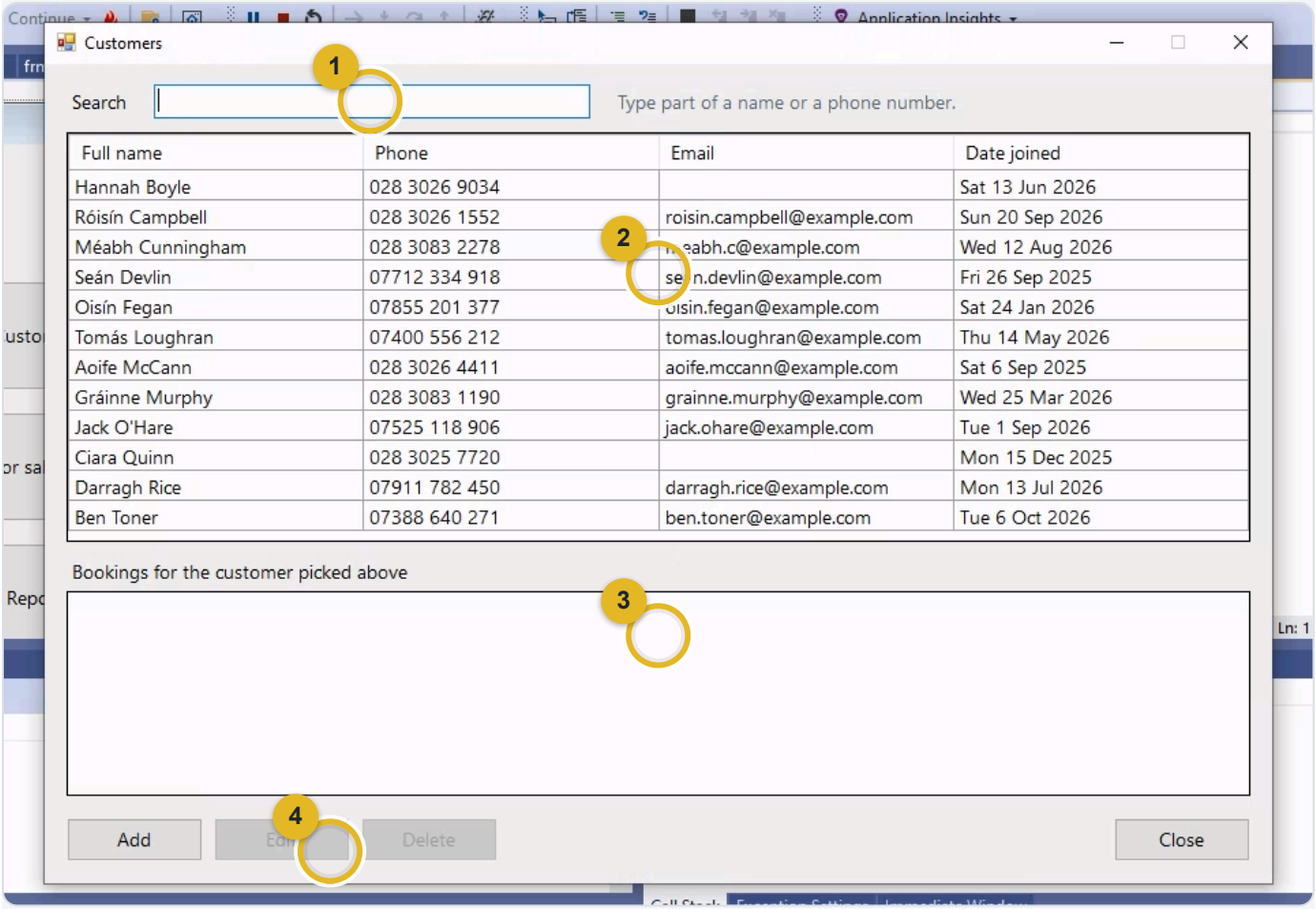



The sign-in window. Siobhán Rooney is the manager, and either role can open the customers screen.

1. Type the username siobhan.
2. Type the password canal01.
3. Click Sign in.

> **You should see** The main menu opens.

Click the Customers button.




The customers screen as it opens. Nothing is picked yet, so Edit and Delete are switched off and the bookings grid is empty.

1. The Search box. The grid narrows as you type.
2. Twelve customers, sorted by last name. The CustomerID column is hidden.
3. The bookings grid: empty until a customer is picked.
4. Edit and Delete are off until a row is picked.

> **You should see** The Customers screen opens with twelve customers in the grid, Hannah Boyle first and Ben Toner last, and Edit and Delete greyed out.

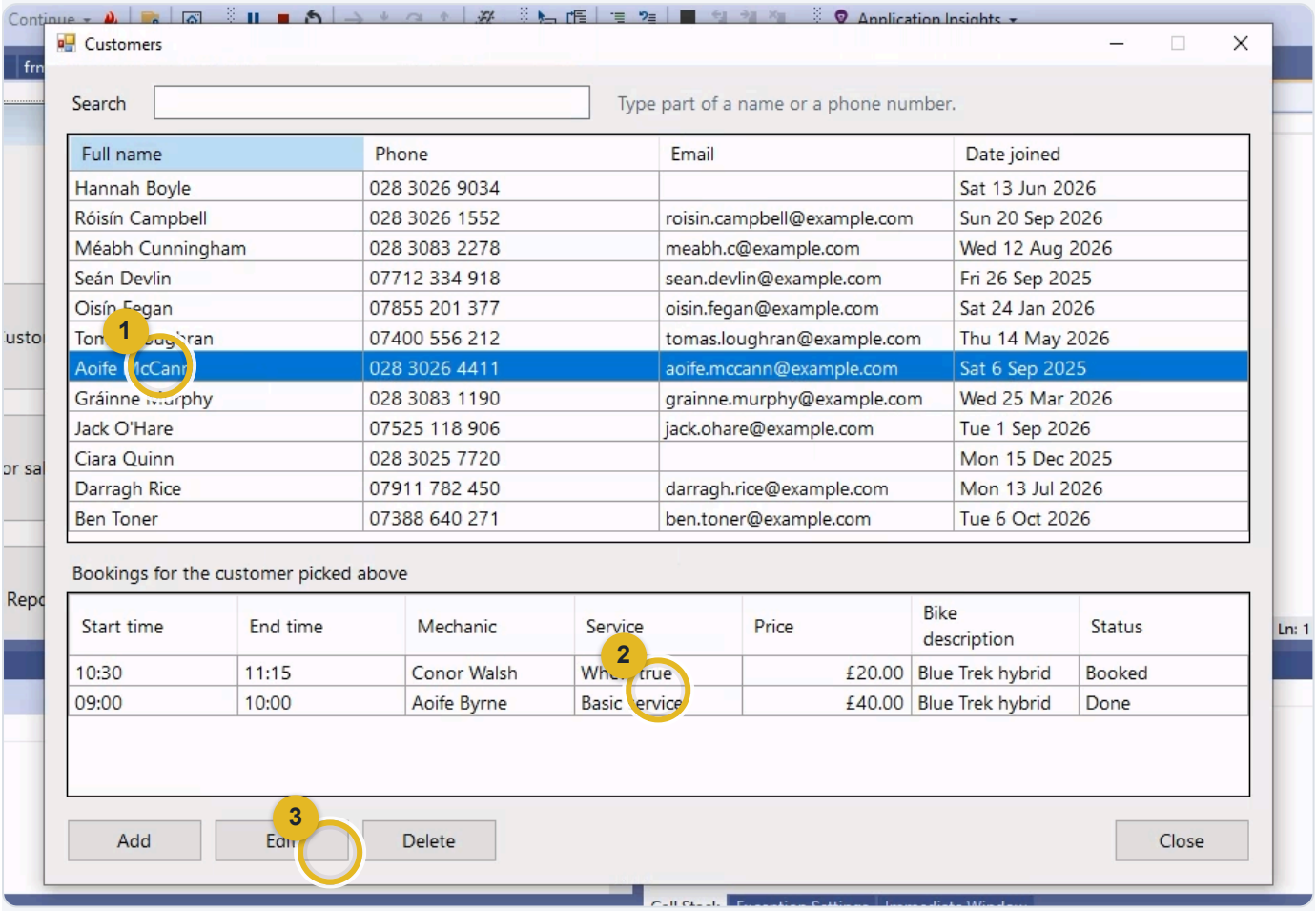
**IF IT GOES WRONG** The grid is empty

The search text is not empty, or the connection string in App.config points at a database without the sample data. Clear the Search box first. If the grid is still empty, run the whole script again in SQL Server Management Studio (SSMS).

**IF IT GOES WRONG** The grid shows a column headed CustomerID

Grid.Tidy is not being called, or is called before the DataSource line. In LoadCustomers the Tidy line comes straight after the DataSource line.

Click the row Aoife McCann.



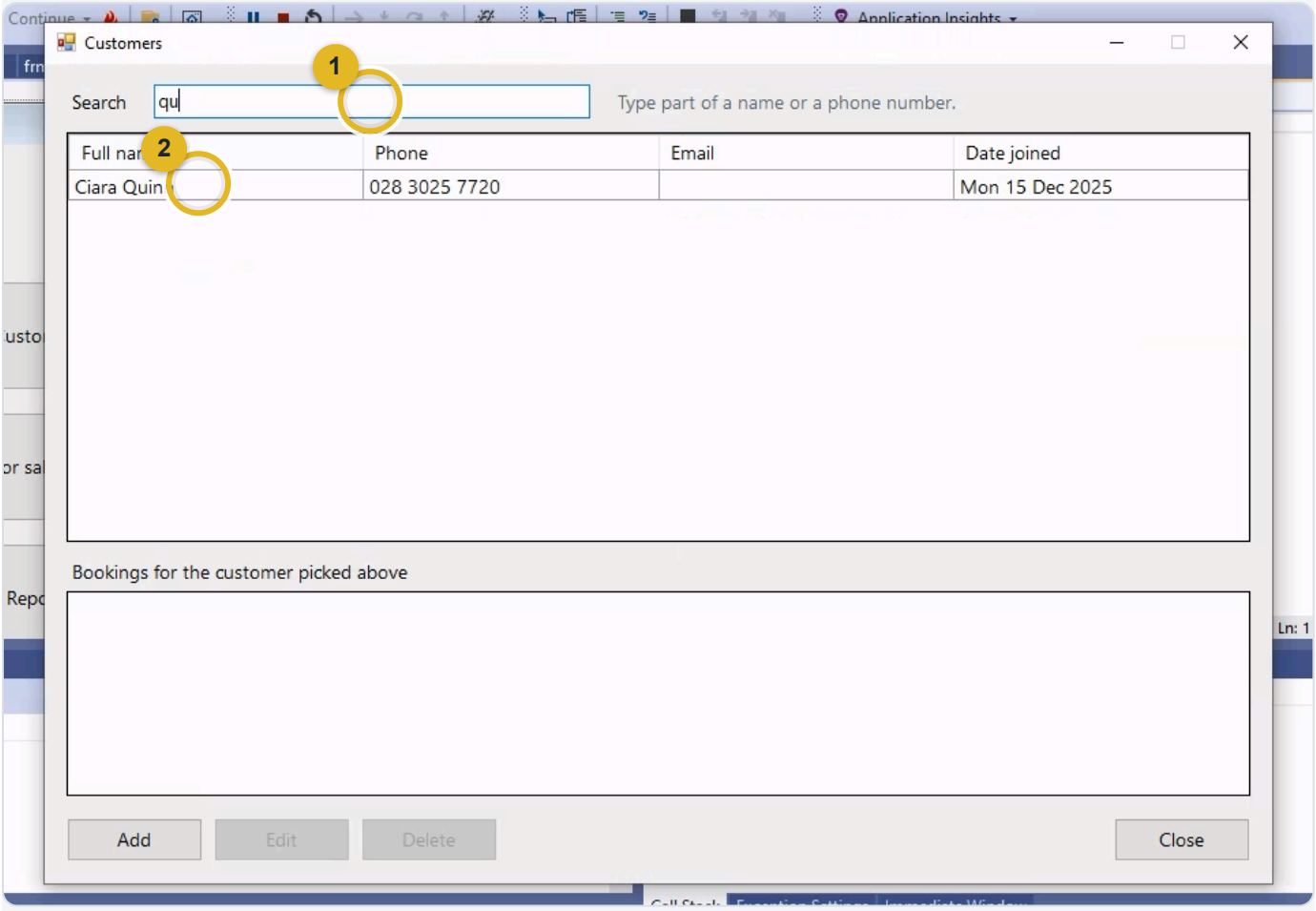

One click on a row switches the buttons on and fills the lower grid with that customer's bookings, newest first.

1. Click the row Aoife McCann.
2. Her bookings, newest first, with the status in words and the mechanic's name in each row.
3. Edit and Delete are on.

> **You should see** The row is highlighted, Edit and Delete switch on, and the bookings grid shows her two bookings, newest first, with the mechanic's name in each row.

# Search as you type.

Click in the Search box and type qu.




Two letters typed, one row left. The grid is loaded again from the database every time the search text changes.

1. Type qu in the Search box.

2. One row left: Ciara Quinn.

> **You should see** The grid narrows to one row, Ciara Quinn, and the bookings grid empties because nothing is picked.

Press Backspace twice to clear the Search box.

> **You should see** All twelve customers are back.

Type 028 in the Search box.

> **You should see** Six rows: every customer whose phone number starts with 028. The search looks at the phone number as well as the names.

Press Backspace three times to clear the Search box.

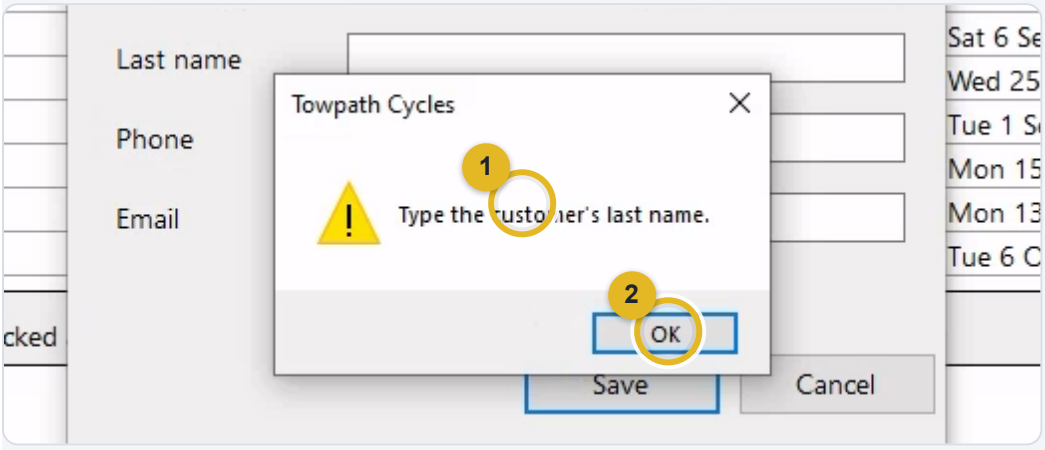
> **You should see** All twelve customers are back.

# Add a customer, and watch the form refuse first.

Click the Add button.

> **You should see** A small window titled Add a customer opens with four empty boxes.

Type Nuala in the First name box and click Save.




The form refuses before the database is asked. The sentence comes from btnSave_Click.

1. The first problem found, and only that one.

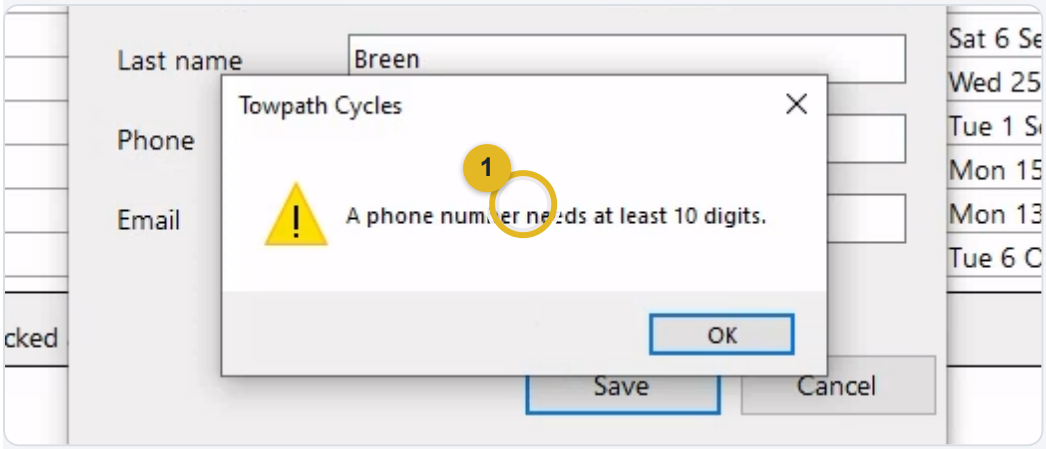
2. Press OK. The cursor is put in the Last name box for you.

> **You should see** A message reads: Type the customer's last name. Press OK, and the cursor is in the Last name box.

Type Breen in the Last name box and 077009001 in the Phone box.

> **You should see** Breen is in the Last name box and 077009001 is in the Phone box.

Click the Save button.

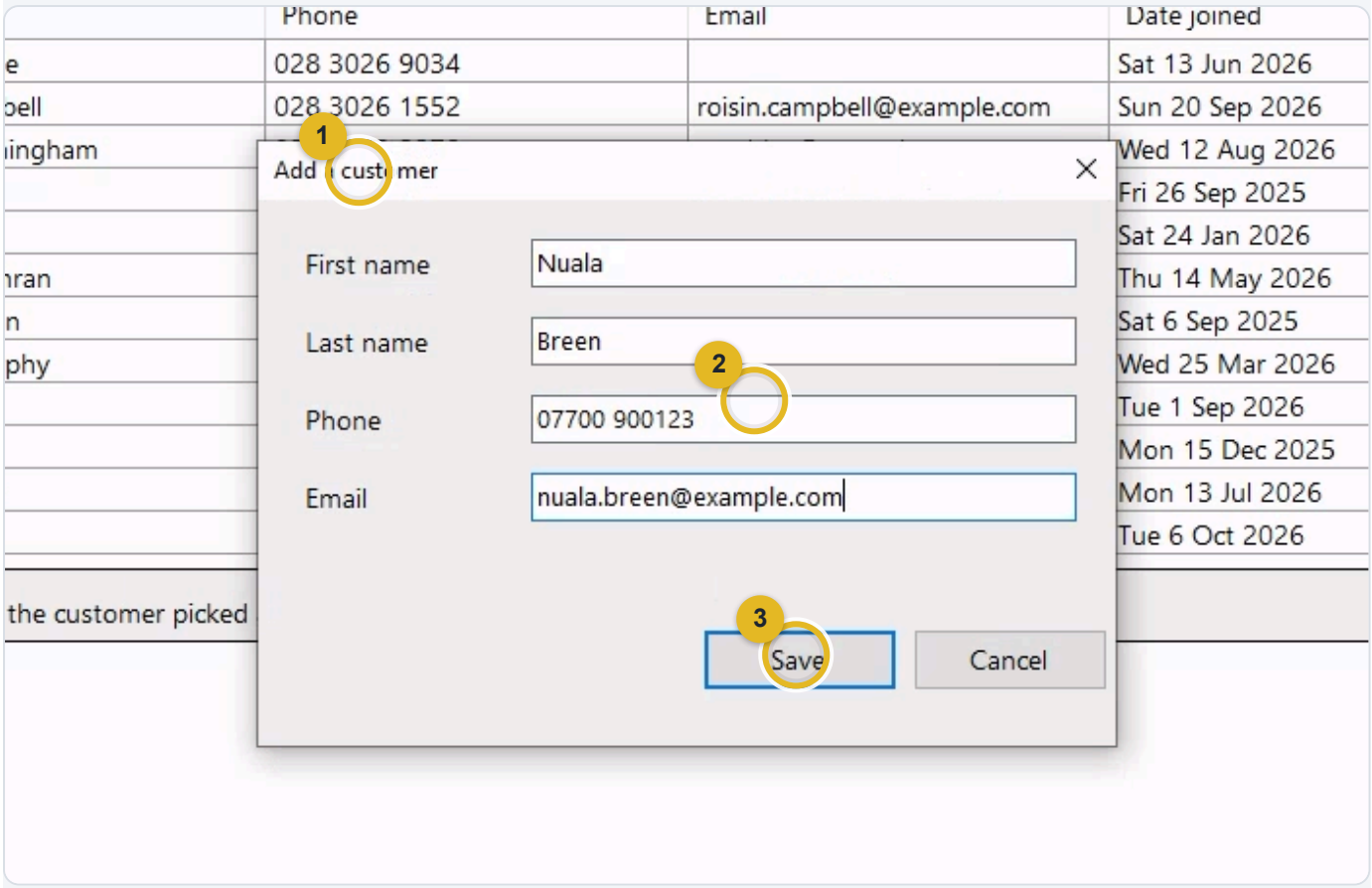



The phone check counted the digits and found too few. The sentence comes from PhoneProblem.

1 Nine digits typed, ten needed.

**You should see** A message reads: A phone number needs at least 10 digits. Nine digits were typed. Press OK, and the cursor is in the Phone box.

Change the Phone box to 07700 900123, type nuala.breen@example.com in the Email box, and click Save.



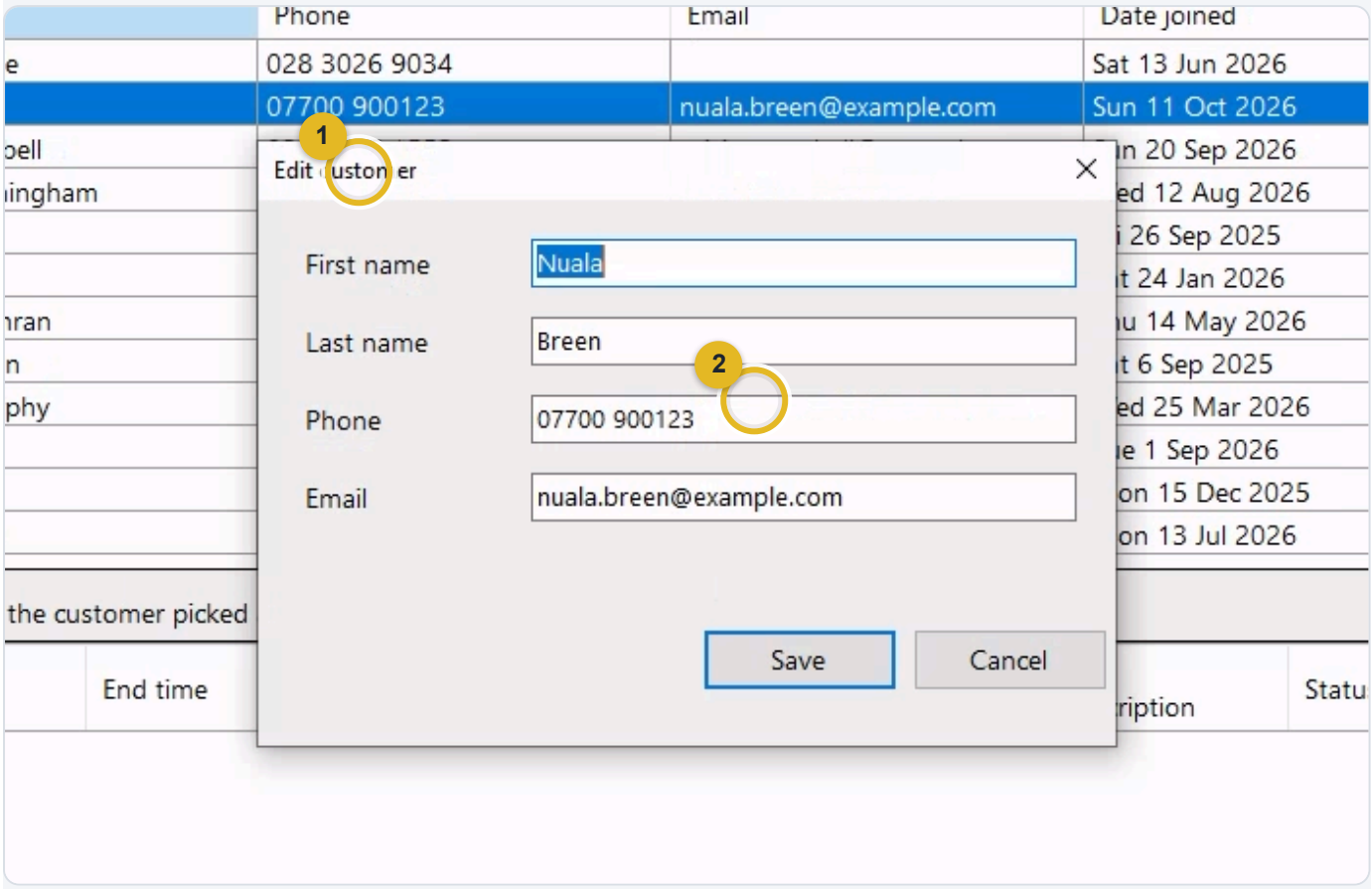

The same form does the adding and the editing. Its title says which job it is doing.

1. The title reads Add a customer, because the form was opened with the number 0.
2. The four boxes, filled in.
3. Save checks every box before it talks to the database.

> **You should see** The window closes and Nuala Breen is the second row of the grid, between Hannah Boyle and Róisín Campbell, with today's date in the Date joined column.

# Edit the customer, two ways.

Click the row Nuala Breen, then click Edit.




Edit opens the same form with the customer already in it, fetched by number from the database.

1. The title reads Edit customer, because the form was opened with a customer number.

2. The boxes are filled from usp_Customer_GetByID before you see them.

> **You should see** The window opens titled Edit customer, with her four details already in the boxes.

Change the last digit of the phone number to 4, so the box reads 07700 900124, and click Save.

> **You should see** The window closes and the Phone column of her row reads 07700 900124.

Double-click the row Nuala Breen.

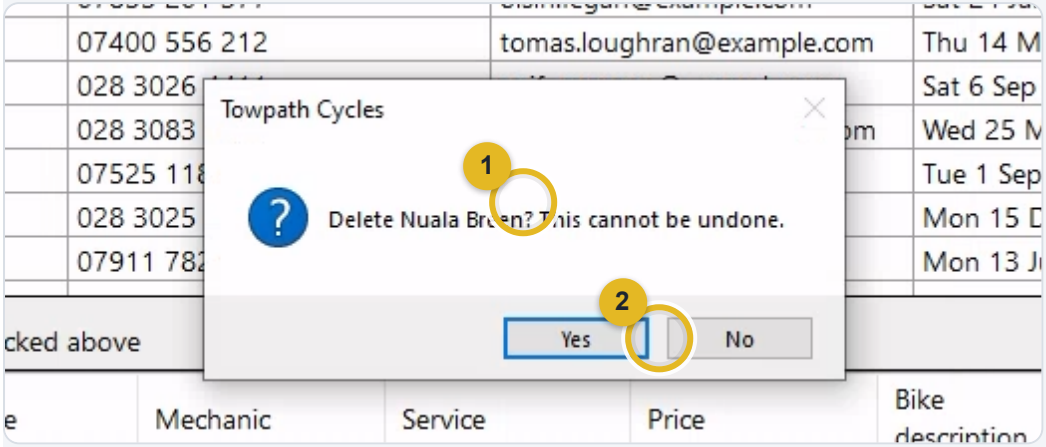
> **You should see** The same Edit customer window opens: a double-click on a row is an Edit.

Change the Phone box to 07700 900999 and press Escape.

> **You should see** The window closes and her row still reads 07700 900124. Escape is the Cancel button, and Cancel saves nothing.

# Delete: the form asks first.

With Nuala Breen still picked, click Delete.




Msg.Ask names the customer and says what will happen. No means nothing happens at all.

1. The name comes from the picked row of the grid.

2. Yes deletes. No goes back with nothing changed.

> **You should see** A question reads: Delete Nuala Breen? This cannot be undone.

Click No in the question window.

> **You should see** Nothing changes, and Nuala Breen is still in the grid.

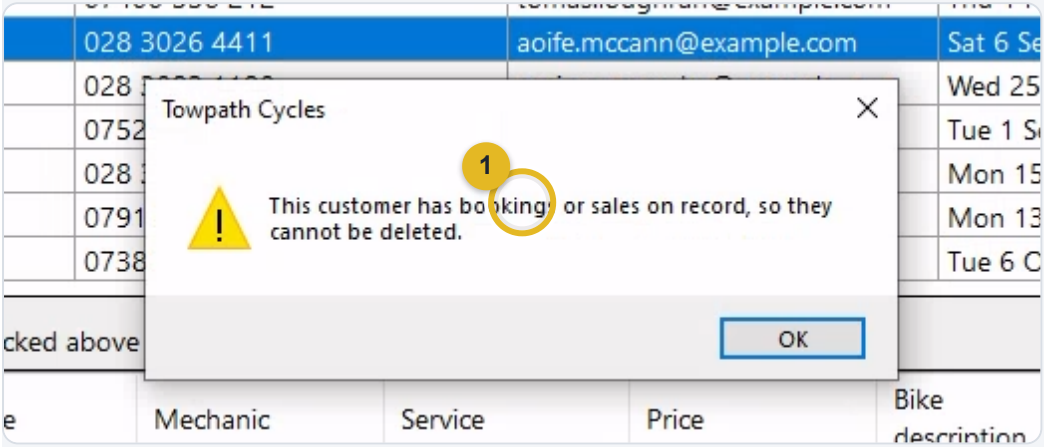
Click Delete again, and this time click Yes.

> **You should see** Her row is gone, and the grid is back to twelve customers.

# Delete: the database says no.

Every one of the twelve sample customers has at least one booking, and usp_Customer_Delete from chapter 4 refuses to delete a customer with history. The form does not check that, because it cannot know: only the database can. Watch the refusal arrive on the screen.

Click the row Aoife McCann, click Delete, and click Yes.




The database said no, and its sentence came all the way up to the screen: usp_Customer_Delete, then Db.Run, then CustomerData.Delete, then Msg.Problem.

1 This sentence was typed in chapter 4, inside the stored procedure.

> **You should see** A message reads: This customer has bookings or sales on record, so they cannot be deleted. Aoife McCann is still in the grid.

Click OK, then click Close.

> **You should see** The Customers screen closes and the main menu is back.

Close the main menu window.

> **You should see** The program ends and Visual Studio 2019 is back.

That sentence travelled a long way. It was typed in chapter 4 inside usp_Customer_Delete, set into @Message, read by Db.Run, handed back by CustomerData.Delete, and shown by Msg.Problem. No form in this program decides whether a delete is allowed. The database decides, and the form passes on what it said.

# ServiceType: the same shape, shorter.

Chapter 7 needs a list of the kinds of service the workshop does, and chapter 8 needs a list of suppliers. Both are the Customer pattern again with fewer columns: a class in Models with one property per column, and a static class in Data with one method per stored procedure. The first pair is given to you. The second pair you write.

**Copy this one** **Models/ServiceType.cs: one kind of service** C#

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  namespace BikeShop.Models
2  {
3      // One kind of service, such as a puncture repair.  Plain properties only.
4      public class ServiceType
5      {
6          public int ServiceTypeID { get; set; }
7          public string Name { get; set; }
8          public int DurationMinutes { get; set; }
9          public decimal Price { get; set; }
10     }
11 }
```

**1 The class**

Lines 1 to 5

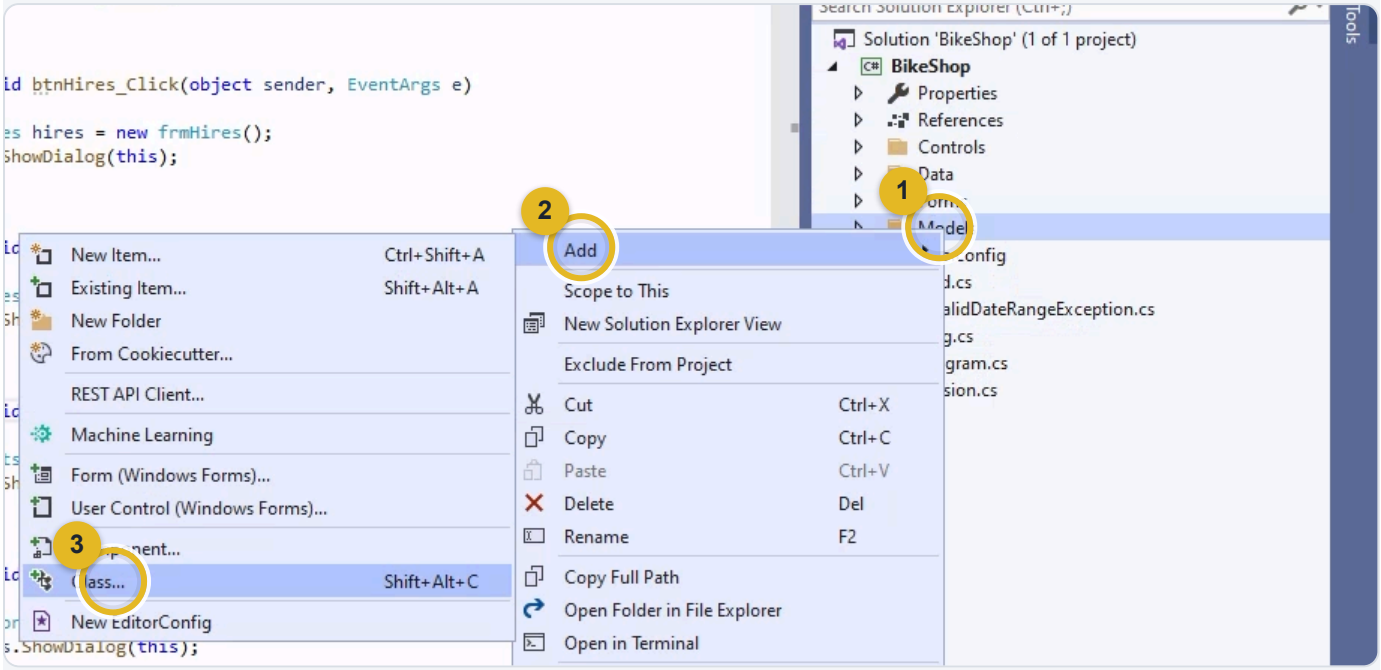
No using line, because none of the four properties needs a type from outside. The class sits in Models with Customer.

**2 One property for each column**

Lines 6 to 11

A number, a name, how many minutes the service takes, and its price. decimal is the kind of value for money. The closing braces end the class and the namespace.

In Solution Explorer in Visual Studio 2019, right-click the Models folder, point at Add, click Class…, type ServiceType.cs in the Name box and press Add.




Adding a class to the Models folder. This picture was taken in the finished project, so Solution Explorer lists files you have not made yet.

1. Right-click the Models folder in Solution Explorer.
2. Point at Add.
3. Click Class…

> **You should see** A new file opens with an empty class in it.

Press the Copy button under this step.

> **You should see** The button reads Copied for a moment.

Click anywhere in the new file and press Ctrl and A together.

> **You should see** Everything in the file is highlighted.

Press Ctrl and V together to paste over the highlighted text.

> **You should see** ServiceType.cs holds the four properties and no red wavy line.

# ServiceTypeData: three methods, three stored procedures.

Copy this one **Data/ServiceTypeData.cs: GetAll, Add and Update** C#

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  using System.Data;
2  using System.Data.SqlClient;
3  using BikeShop.Models;
4
5  namespace BikeShop.Data
6  {
7      public static class ServiceTypeData
8      {
9          // Every kind of service.
10         public static DataTable GetAll()
11         {
12             return Db.GetTable("usp_ServiceType_GetAll");
13         }
14
15         // Saves a new service.  Returns "" if it worked, or the sentence to
16         // show the user.
17         public static string Add(ServiceType service)
18         {
19             SqlParameter newID = Db.Out("@NewServiceTypeID");
20             string message = Db.Run("usp_ServiceType_Add",
21                 Db.P("@Name", service.Name),
22                 Db.P("@DurationMinutes", service.DurationMinutes),
23                 Db.P("@Price", service.Price),
24                 newID);
25             if (message == "")
26             {
27                 service.ServiceTypeID = (int)newID.Value;
28             }
29             return message;
30         }
31
32         // Saves changes to an existing service.
33         public static string Update(ServiceType service)
34         {
35             return Db.Run("usp_ServiceType_Update",
36                 Db.P("@ServiceTypeID", service.ServiceTypeID),
37                 Db.P("@Name", service.Name),
38                 Db.P("@DurationMinutes", service.DurationMinutes),
39                 Db.P("@Price", service.Price));
40         }
41     }
42 }
```

**1 The usings and the class**

Lines 1 to 8

The same three usings as CustomerData, less System, because nothing here needs it. Static again: ServiceTypeData.GetAll, by the class name.

**2 GetAll: every service**

Lines 10 to 13

usp_ServiceType_GetAll takes no parameters, so Db.GetTable is given the name alone. The whole table comes back, ready for a grid or a list.

**3 Add: the same shape as the customer Add**

Lines 17 to 30

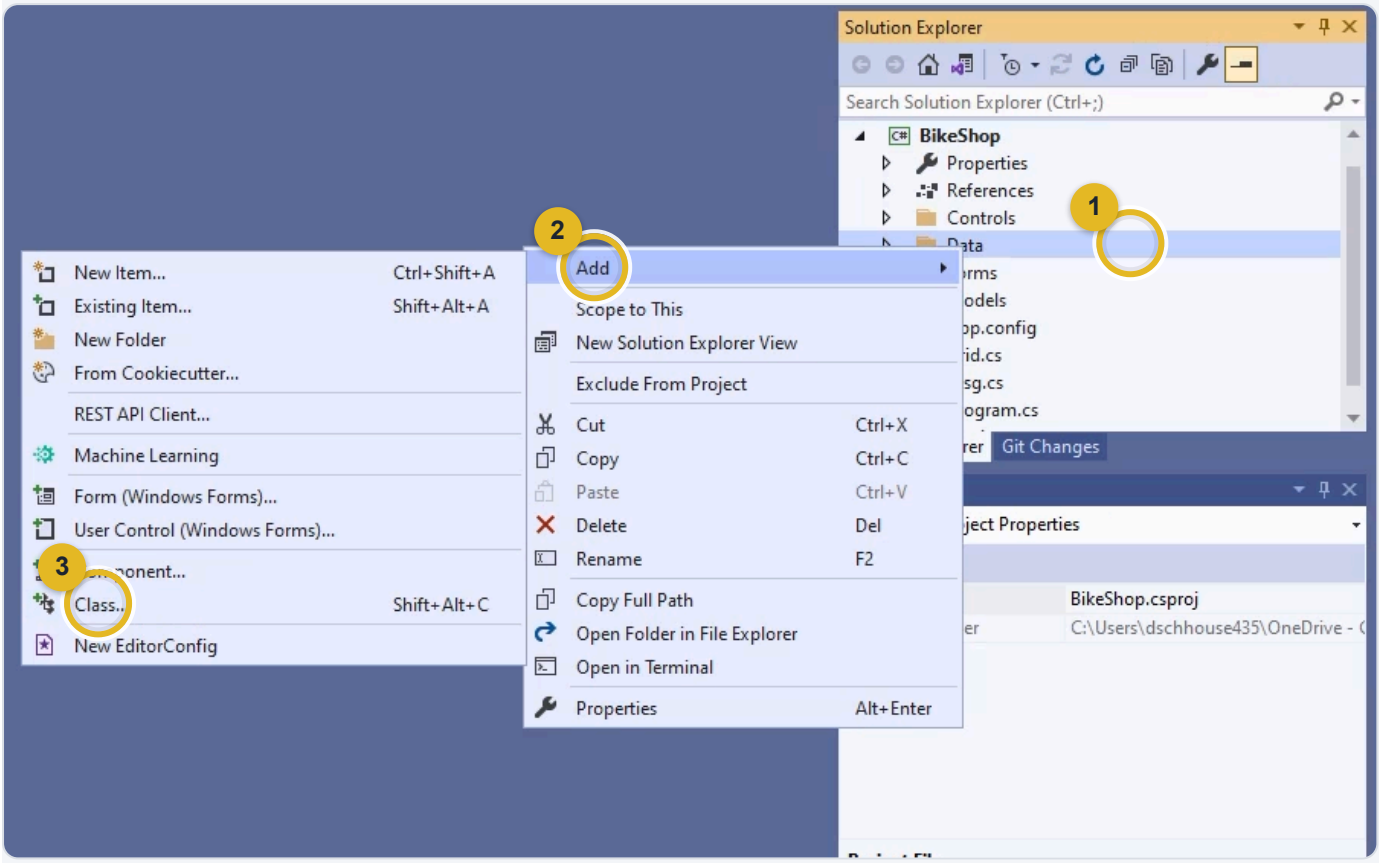
Db.Out makes the output parameter for the new number, Db.Run calls the stored procedure with three values and that parameter, and when the message is empty the new number goes into the object.

**4 Update: four values in, the message back**

Lines 33 to 42

The number says which row, and the other three are the new values. The closing braces end the class and the namespace.

In Solution Explorer in Visual Studio 2019, right-click the Data folder, point at Add, click Class…, type ServiceTypeData.cs in the Name box and press Add.




Adding a class to the Data folder. Taken in the finished project, so Solution Explorer lists more files than yours.

1. Right-click the Data folder in Solution Explorer.
2. Point at Add.
3. Click Class…

> **You should see** A new file opens with an empty class in it.

Press the Copy button under this step.

> **You should see** The button reads Copied for a moment.

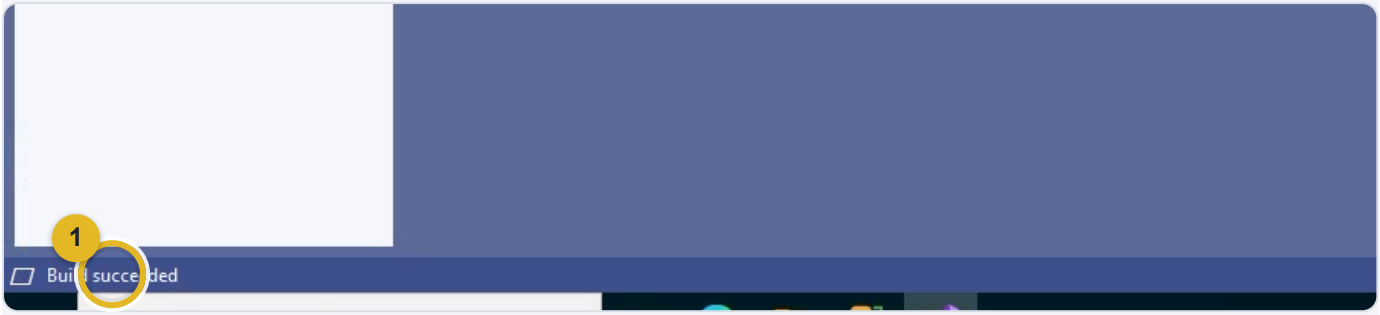
Click anywhere in the new file and press Ctrl and A together.

> **You should see** Everything in the file is highlighted.

Press Ctrl and V together to paste over the highlighted text.

> **You should see** ServiceTypeData.cs holds GetAll, Add and Update.

Press F6 to build the project.




Build succeeded, at the bottom left. Anything else means an error list to read.

1 Build succeeded. Press F6 to build at any time.

> **You should see** Build succeeded, at the bottom left.

# Your turn: the Supplier class.

Chapter 8 needs suppliers. The Supplier table from chapter 3 has four columns: SupplierID, Name, Phone and Email. Write the class yourself, on the ServiceType pattern, in a new file in the Models folder. If you get stuck, the finished code is under Stuck? Show me, and no one minds.

Your turn **Models/Supplier.cs: one supplier** C#

1. In Solution Explorer in Visual Studio 2019, right-click the Models folder, point at Add, click Class…, type Supplier.cs in the Name box and press Add.
2. Change the first line to namespace BikeShop.Models, the way ServiceType.cs starts, and change the class line to public class Supplier.
3. Inside the class, type one property for each of the four columns: an int called SupplierID, then three strings called Name, Phone and Email, each ending with { get; set; }.
4. Delete any using lines Visual Studio 2019 put at the top: nothing in this class needs them.

**THE TEST** Press F6 to build the project.

**YOU SHOULD SEE** Build succeeded, at the bottom left. Supplier.cs is four properties long, and Solution Explorer lists it in the Models folder beside Customer.cs and ServiceType.cs.

**HINT** Open ServiceType.cs beside you. Change the class name, swap the four property lines for the four columns of the Supplier table, and change nothing else. Every property line ends with { get; set; }.

**Stuck? Show me the finished code**

Read it, then write yours **Models/Supplier.cs** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  namespace BikeShop.Models
2  {
3      // One supplier of parts.  Plain properties only.
4      public class Supplier
5      {
6          public int SupplierID { get; set; }
7          public string Name { get; set; }
8          public string Phone { get; set; }
9          public string Email { get; set; }
10     }
11 }
```

**1 The class**

Lines 1 to 5

The same start as ServiceType: no using line, the Models namespace, and the class.

**2 One property for each column**

Lines 6 to 11

The class has one number and three pieces of text, matching the four columns of the Supplier table.

# Your turn: the SupplierData class.

Now the data class, on the ServiceTypeData pattern, with two methods: GetAll, which calls usp_Supplier_GetAll, and Add, which calls usp_Supplier_Add with the parameters @Name, @Phone and @Email and the output parameter @NewSupplierID. There is no Update, because chapter 8 never changes a supplier.

Your turn **Data/SupplierData.cs: GetAll and Add** C#

1. In Solution Explorer in Visual Studio 2019, right-click the Data folder, point at Add, click Class…, type SupplierData.cs in the Name box and press Add.
2. Make the top of the file match ServiceTypeData.cs: the three usings, namespace BikeShop.Data, and public static class SupplierData.
3. Type GetAll: a public static method that returns a DataTable, with one line inside it that returns Db.GetTable with the name usp_Supplier_GetAll.
4. Type Add: a public static method that takes a Supplier called supplier and returns a string. Inside it, make the output parameter with Db.Out("@NewSupplierID"), call Db.Run with the name usp_Supplier_Add, three Db.P lines for @Name, @Phone and @Email, and the output parameter last, keep the message, and when the message is empty put (int)newID.Value into supplier.SupplierID. Return the message.

**THE TEST** Press F6 to build the project.

**YOU SHOULD SEE** Build succeeded, at the bottom left. SupplierData.cs has two methods, GetAll and Add, and nothing else.

**HINT** Copy ServiceTypeData.cs into the new file, delete the Update method, and change the names one at a time: the class, the two stored procedures, the output parameter, and the three Db.P lines. Every line that said service now says supplier.

**Stuck? Show me the finished code**

Read it, then write yours **Data/SupplierData.cs** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  using System.Data;
2  using System.Data.SqlClient;
3  using BikeShop.Models;
4
5  namespace BikeShop.Data
6  {
7      public static class SupplierData
8      {
9          // Every supplier.
10         public static DataTable GetAll()
11         {
12             return Db.GetTable("usp_Supplier_GetAll");
13         }
14
15         // Saves a new supplier.  Returns "" if it worked, or the sentence to
16         // show the user.
17         public static string Add(Supplier supplier)
18         {
19             SqlParameter newID = Db.Out("@NewSupplierID");
20             string message = Db.Run("usp_Supplier_Add",
21                 Db.P("@Name", supplier.Name),
22                 Db.P("@Phone", supplier.Phone),
23                 Db.P("@Email", supplier.Email),
24                 newID);
25             if (message == "")
26             {
27                 supplier.SupplierID = (int)newID.Value;
28             }
29             return message;
30         }
31     }
32 }
```

**1 The usings and the class**

Lines 1 to 8

The same three usings and the Data namespace. The class is static, like every data class.

**2 GetAll: every supplier**

Lines 10 to 13

No parameters, so Db.GetTable is given the name alone.

**3 Add: three values in, the new number back**

Lines 17 to 32

Db.Out for the new number, Db.Run with three Db.P lines and the output parameter, and the number copied into the object when the message is empty. The closing braces end the method, the class and the namespace.

STAGE 7 OF 7 · PROVE IT · SCREEN 1 OF 2

# Prove that the customers screen does what this chapter says.

Run the program, sign in as siobhan with the password canal01, open Customers, and work down the table. The rows marked Must be refused are the proof: a check that never fails is not a check. Tick each row when you have seen it with your own eyes.

| | THE TEST | WHAT MUST HAPPEN | HOW TO DO IT, AND WHAT YOU SHOULD SEE |
|---|---|---|---|
| | The grid opens with twelve customers sorted by last name and no CustomerID column | Must be allowed | Click Customers on the main menu and read the first and last rows: Hannah Boyle, then Ben Toner. |
| | Edit and Delete are off until a row is picked | Must be allowed | Look at the two buttons as the screen opens, then click any row and look again. |
| | The bookings grid shows the picked customer's bookings, newest first | Must be allowed | Click Aoife McCann and count the rows in the lower grid: two, with a mechanic's name in each. |
| | The grid narrows as you type | Must be allowed | Type qu in the Search box and see one row, Ciara Quinn. Clear the box and see twelve. |
| | A blank last name is refused by the form | Must be refused | Click Add, type Nuala in First name, leave Last name empty and click Save. The message must say: Type the customer's last name. |
| | A phone number with letters in it is refused by the form | Must be refused | Type Breen in Last name, type abc in Phone and click Save. The message must say that a phone number can only have digits, spaces, + and -. |
| | A phone number with nine digits is refused by the form | Must be refused | Change Phone to 077009001 and click Save. The message must say that a phone number needs at least 10 digits. |
| | An email without an @ is refused by the form | Must be refused | Change Phone to 07700 900123, type nuala.breen in Email and click Save. The message must say that an email address must look like name@example.com. |
| | A good customer is saved and appears in the grid | Must be allowed | Change Email to nuala.breen@example.com and click Save. Nuala Breen is the second row, with today's date joined. |

| THE TEST | WHAT MUST HAPPEN | HOW TO DO IT, AND WHAT YOU SHOULD SEE |
|---|---|---|
| Edit changes the row, and Cancel changes nothing | **Must be allowed** | Double-click Nuala Breen, change the phone number to 07700 900124 and click Save: the row shows the new number. Open her again, change the phone number and press Escape: the row keeps 07700 900124. |
| Delete asks first, and No does nothing | **Must be allowed** | Click Nuala Breen, click Delete and click No. She is still in the grid. |
| A customer with no history can be deleted | **Must be allowed** | Click Delete again and click Yes. Nuala Breen is gone, and the grid is back to twelve. |
| A customer with bookings is refused by the database | **Must be refused** | Click Aoife McCann, click Delete and click Yes. The message must say that this customer has bookings or sales on record, and she must still be in the grid. |
| The Supplier pair you wrote builds | **Must be allowed** | Press F6 in Visual Studio 2019 and read Build succeeded, with Supplier.cs in Models and SupplierData.cs in Data. |

# Finished: what you have, and what comes next.

Your program now adds, edits, deletes and displays. The pattern is the same for every table that follows: a class in Models, a data class in Data, a list form with a grid, and an edit form that checks its boxes and lets the database have the last word. Chapter 7 uses the Customer and ServiceType classes to book a job in the diary, and chapter 8 uses Supplier to order parts.

**Make it yours**

Choose one sentence in frmCustomers.cs, such as the question Delete asks, and change the words between its quotation marks. Then press F5, open Customers, click a row, click Delete and read your own words on the screen.

**Still to do in chapter 7**

In chapter 7, step 7.5.10 adds AppointmentData. After that step, delete the two lines you typed under the three green lines in ShowBookings. Then select the three green lines and press Ctrl and K together, then Ctrl and U together. Press F6, and the bookings grid works as before, through the data class this time.

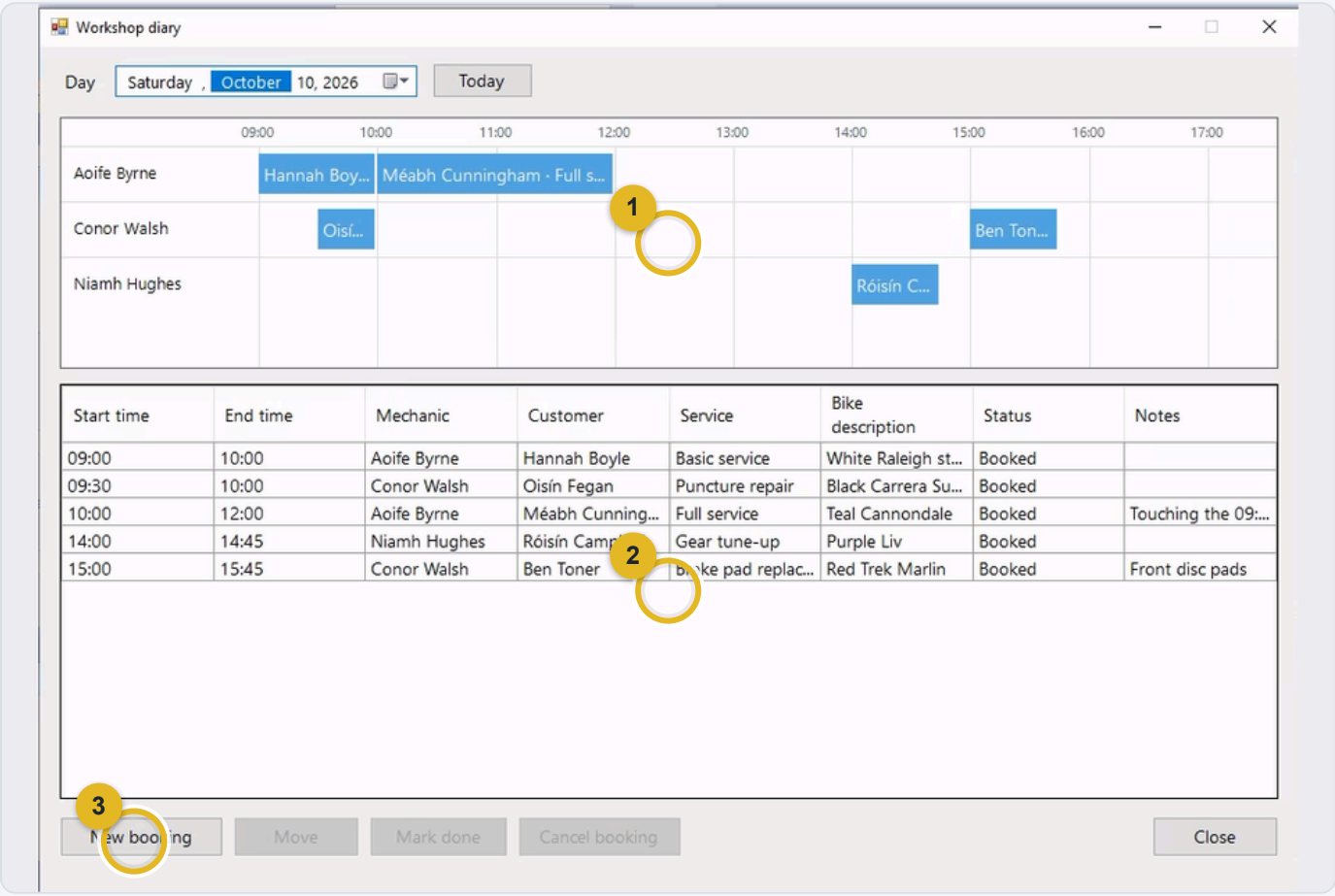
# Bookings

Make it impossible to book one mechanic twice at the same time, and show the day in a diary that the counter staff can read at a glance.

STAGE 1 OF 8 · THE RULE: NO DOUBLE BOOKING · SCREEN 1 OF 6

## See what you will build in this chapter.

By the end of this chapter your program will have a diary. The counter staff pick a day and see every booking for that day, with the mechanic's name, the customer's name and the job. They can add a booking, move one, mark one done or cancel one.




What you will build in this chapter: the diary for one day, with a New booking button that can never double-book a mechanic.

1. The drawn time line: one row for each mechanic, 08:30 to 17:30 across.
2. The same bookings as a grid, with names.
3. New booking. The database checks every booking before it is saved.

The important part is not the picture. It is a rule that lives in the database: one mechanic can never have two bookings at the same time. You will write that rule as a stored procedure, test it, and then use the same rule again for bike hires.

# Check that you have these ready.

**Before you start**

- Chapter 6 is finished: your Customers screen adds, edits and deletes a customer.
- SQL Server Management Studio (SSMS) is open, connected to your BikeShop database.
- Your BikeShop solution is open in Visual Studio 2019.
- This guide is open in the Edge browser beside you. Tick each step as you do it. Your ticks are saved, so you can close the guide and come back.

**Words used in this chapter**

| | |
|---|---|
| `SSMS` | SQL Server Management Studio, the program you write and run SQL in |
| `stored procedure` | a named piece of SQL that is saved in the database and run by name |
| `clash` | two bookings for the same mechanic whose times overlap |
| `slot` | a start time and an end time together |
| `status` | the state a booking is in: Booked, Done or Cancelled |
| `@Message` | the sentence a stored procedure hands back when it refuses; empty means it worked |
| `RETURN` | stops the stored procedure at that line, so nothing after it runs |
| `OUTPUT parameter` | a value the stored procedure hands back to the program that called it, such as @Message or the new booking number |

# Read what goes wrong without the rule.

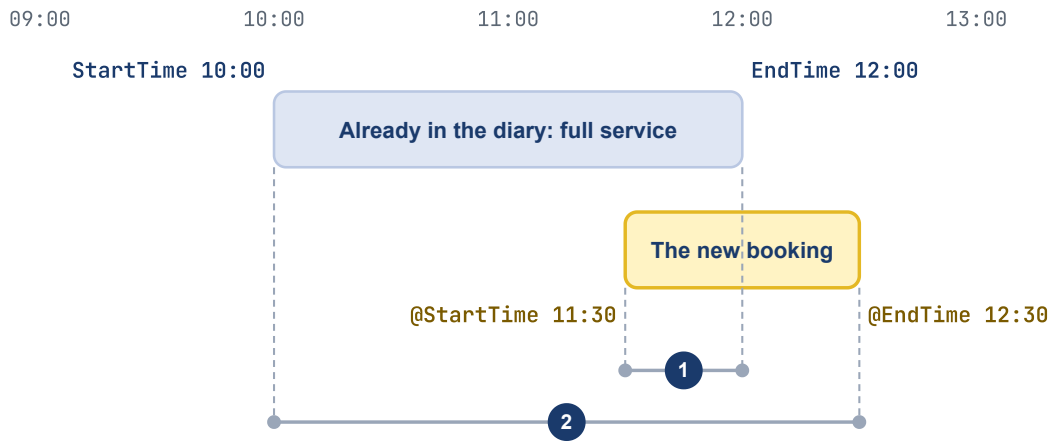
Aoife is one of the shop's mechanics. She can only work on one bike at a time. Pádraig, at the counter, books a customer's bike in with Aoife for a full service at ten o'clock. A full service takes two hours, so Aoife is busy until twelve.

Ten minutes later Siobhán, on the other computer at the counter, books a different customer in with Aoife at eleven o'clock. Siobhán's screen does not know about Pádraig's booking. Now Aoife has two bikes for the same hour, and one customer is going to be let down.

The booking form cannot stop this on its own. Each form only knows about the bookings it loaded when it opened. The database sees every booking from every computer in the shop. So the check belongs in the database, inside the stored procedure that saves a booking.

# Look at two bookings on a time line.

Here are the two bookings drawn on one time line. The blue one is already in the diary. The gold one is the new one that is being asked for. Each booking has a start time and an end time, so there are four times in the picture.




1. `@StartTime < EndTime`
   Does the new booking start before the old one ends? 11:30 is before 12:00, so yes.

2. `@EndTime > StartTime`
   Does the new booking end after the old one starts? 12:30 is after 10:00, so yes.

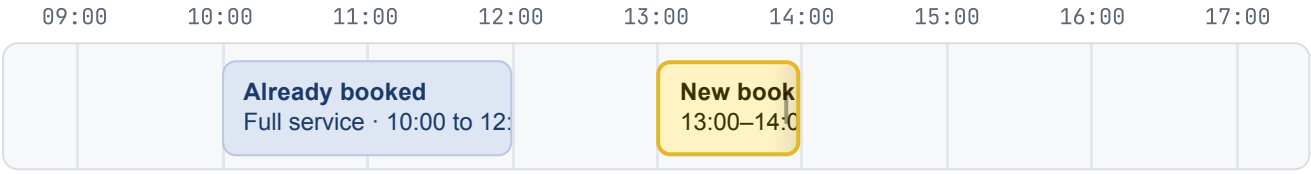
Two bookings overlap when the answer to both questions is yes. If either answer is no, the two bookings do not touch each other. In the SQL, the names that start with @ belong to the new booking, and the plain names belong to the booking already in the table.

# Try the rule yourself.

Drag the gold booking along Aoife's day, or press one of the buttons. The two questions are answered underneath each time. Watch when both answers are yes.

Aoife's day, 08:30 to 17:30

Drag the gold block marked New booking to a different time. Drag its right edge to make it longer or shorter.




The new booking for Aoife would run from **13:00 to 14:00** (60 minutes). The booking already in the Appointment table runs from **10:00 to 12:00**.

**Allowed**

One answer is No, so the two bookings do not overlap and the booking is saved.

**Question 1. Does the new booking start before the old one ends?**
Is 13:00 before 12:00?
`@StartTime < EndTime`
**No**

**Question 2. Does the new booking end after the old one starts?**
Is 14:00 after 10:00?
`@EndTime > StartTime`
**Yes**

Press each of the five buttons under the time line and read the two answers each time.

> **You should see** Before, Touching and After each have one answer that is no, so they are allowed. Overlaps the start and Inside have two yes answers, so they are refused.

Drag the gold booking so that it ends exactly at 10:00.

> **You should see** Allowed. Ending at 10:00 is not ending after 10:00, so the second answer is no.

Drag the right-hand edge to make the gold booking three hours long, then drag it over the whole of the 10:00 booking.

> **You should see** Refused. A long booking that covers the old one from end to end still gets two yes answers.

# Remember the rule.

Why ask two questions instead of one? Asking only whether the new start time falls inside the old booking is not enough. A booking from 09:30 to 10:30 starts outside the 10:00 booking, but it still overlaps it. The two questions together catch every case: the same start, overlapping the start, inside, overlapping the end, and a long booking that covers the old one whole.

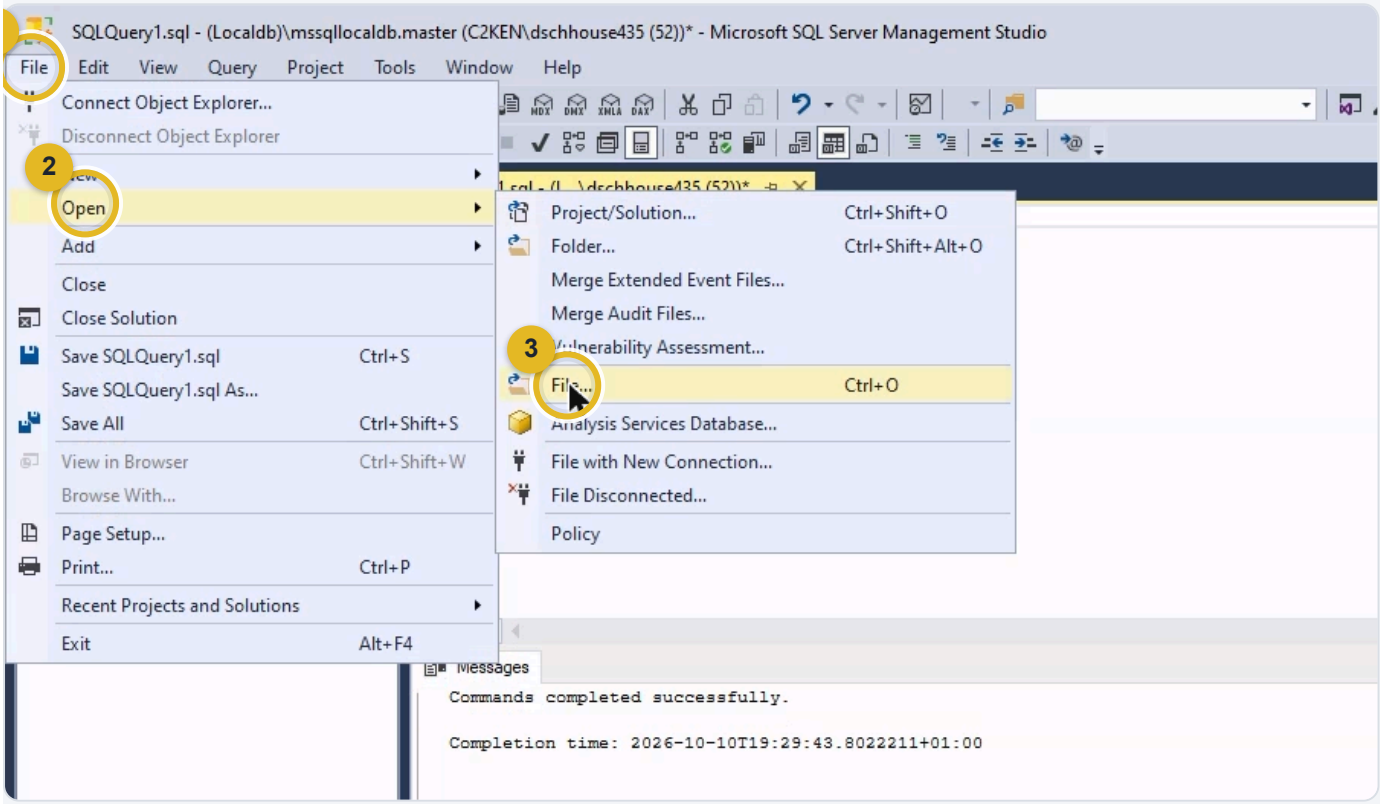
Touching is allowed. A booking that ends at 12:00 and another that starts at 12:00 do not overlap, because 12:00 is not before 12:00.

| | |
|---|---|
| **RULE THAT NEVER BREAKS · NO DOUBLE BOOKING** | The database refuses a double booking. The check is in one stored procedure, so every form on every computer that uses the database gets the same answer. You will type the two questions three times in this chapter, so that you can write them for your own tables without looking. |

# Open your script in SQL Server Management Studio (SSMS).

Your BikeShop.sql script is the file that makes your tables and your stored procedures. Every new stored procedure in this chapter goes near the end of that file, just above the line that prints BikeShop is ready, so that one run of the script builds the whole database again from nothing.

In SQL Server Management Studio (SSMS), click File, then Open, then File…, and open BikeShop.sql from your OneDrive folder.

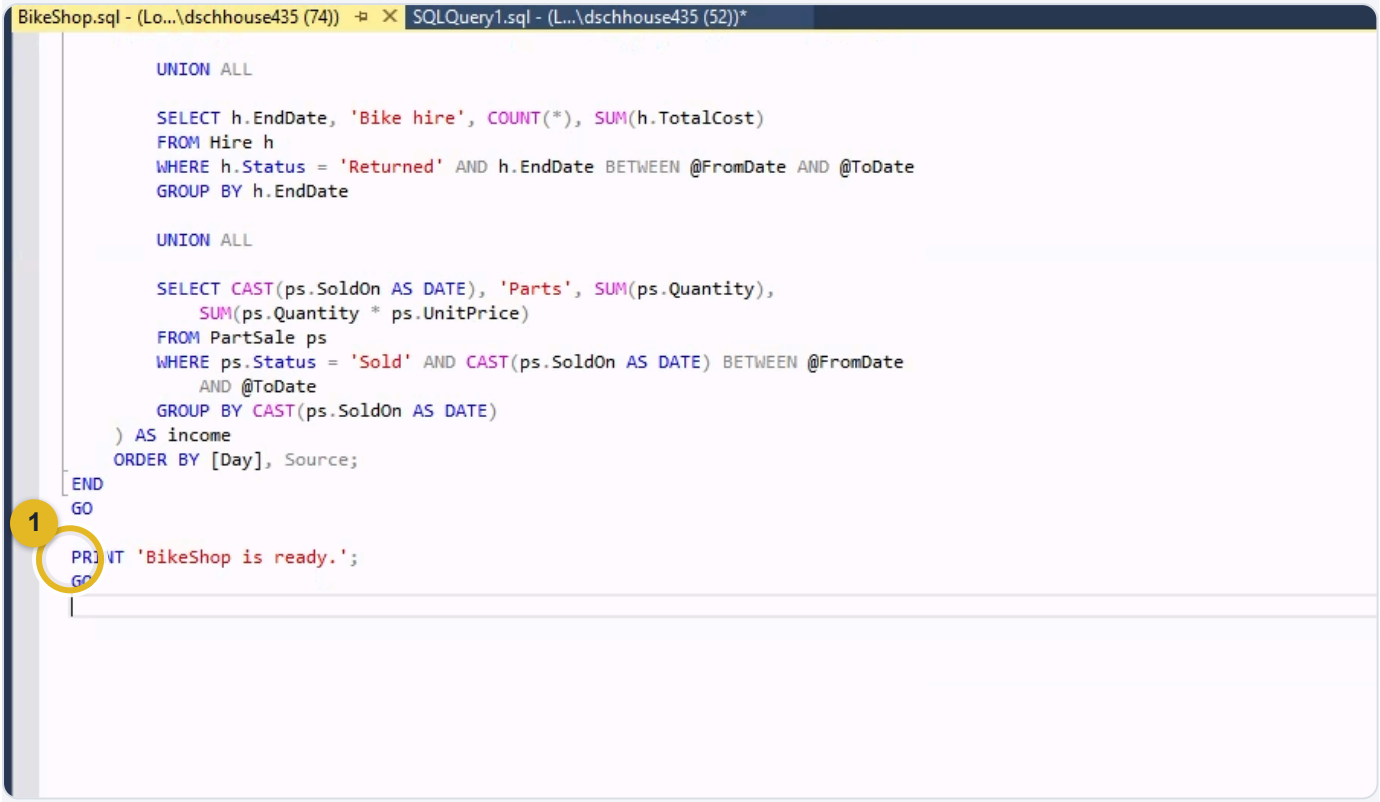



The File menu in SQL Server Management Studio (SSMS). Your script opens from here.

1. Click File at the top left.
2. Point at Open.
3. Click File… and choose BikeShop.sql from your OneDrive folder.

> **You should see** Your script is open in a tab, with its name at the top of the tab.

Scroll to the very end of the script and click on the empty line above the line that prints BikeShop is ready. That line always stays last.




The end of your script. Everything new in this chapter goes on the empty line above the line that prints BikeShop is ready, one stored procedure after another, each ending with GO.

1. Click on the empty line above the line that prints BikeShop is ready, so the cursor blinks there.

> **You should see** The text cursor blinks on an empty line, with the PRINT line and its GO underneath it.

## Read the stored procedure, then add it to your script.

This is the stored procedure that the booking form will call. Most of it is checks of the kind you wrote in chapter 4, so that part is copied. The yellow lines are the clash test, and you type those yourself. The Copy button under the first step leaves a gap for them.

Type this one **usp_Appointment_Add: save a booking, or refuse it** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
CREATE OR ALTER PROCEDURE usp_Appointment_Add
    @CustomerID       INT,
    @MechanicID       INT,
    @ServiceTypeID    INT,
    @StartTime        DATETIME,
    @BikeDescription  VARCHAR(80),
    @Notes            VARCHAR(200),
    @BookedByStaffID  INT,
    @NewAppointmentID INT OUTPUT,
    @Message VARCHAR(200) OUTPUT
AS
BEGIN
    SET @Message = '';

    -- 1. Work out when the job ends: the service type knows how long it takes.
    DECLARE @EndTime DATETIME;
    SELECT @EndTime = DATEADD(MINUTE, DurationMinutes, @StartTime)
    FROM ServiceType
    WHERE ServiceTypeID = @ServiceTypeID;

    -- 2. The simple checks.
    IF @EndTime IS NULL
    BEGIN
        SET @Message = 'Choose a service.';
        RETURN;
    END
    IF NOT EXISTS (SELECT * FROM Staff WHERE StaffID = @MechanicID
        AND Role = 'Mechanic' AND IsActive = 1)
    BEGIN
        SET @Message = 'Choose a mechanic.';
        RETURN;
    END
    IF @StartTime < GETDATE()
    BEGIN
        SET @Message = 'That time has already passed.';
        RETURN;
    END
    IF DATENAME(WEEKDAY, @StartTime) = 'Sunday'
    BEGIN
        SET @Message = 'The workshop is closed on Sundays.';
        RETURN;
    END
    IF CAST(@StartTime AS TIME) < '08:30' OR CAST(@EndTime AS TIME) > '17:30'
    BEGIN
        SET @Message = 'The workshop is open 08:30 to 17:30.';
        RETURN;
    END

    -- 3. The clash check: does this mechanic already have a booking that
    -- overlaps?

    IF EXISTS (SELECT *
               FROM Appointment
```

```
54                  WHERE MechanicID = @MechanicID
55                    -- a cancelled booking does not block the slot
56                    AND Status <> 'Cancelled'
57                    -- the new one starts before the old one ends
58                    AND @StartTime < EndTime
59                    -- and ends after the old one starts
60                    AND @EndTime   > StartTime)
61      BEGIN
62          SET @Message = 'That mechanic already has a booking in that time.';
63          RETURN;
64      END
65
66      INSERT INTO Appointment
67          (CustomerID, MechanicID, ServiceTypeID, StartTime, EndTime,
68           BikeDescription, Notes, BookedByStaffID)
69      VALUES
70          (@CustomerID, @MechanicID, @ServiceTypeID, @StartTime, @EndTime,
71           @BikeDescription, @Notes, @BookedByStaffID);
72
73      SET @NewAppointmentID = SCOPE_IDENTITY();
74  END
75  GO
```

**1 The name, and what goes in and out**

Lines 1 to 10

CREATE OR ALTER makes the stored procedure, or replaces it if it is already there, so you can run your script again and again. Each line that starts with @ is a parameter: one piece of information the form sends in. The two marked OUTPUT travel the other way. The stored procedure fills them in and hands them back.

**2 Start with a blank message**

Lines 11 to 13

AS and BEGIN mark where the work starts. The first job is to make @Message empty. An empty message means it worked. Every refusal below puts a sentence into @Message instead, so the program always knows which it was.

**3 Work out when the job ends**

Lines 16 to 19

The form only sends the start time. The ServiceType table knows how many minutes each service takes. DATEADD adds that many minutes to the start time, and the answer is kept in @EndTime.

**4 Refuse if no service was chosen**

Lines 22 to 26

If no row matched the service number, @EndTime is still empty, which SQL calls NULL. The stored procedure puts a sentence into @Message, and RETURN stops it right there. Nothing after RETURN runs, so nothing is saved.

**5 Refuse if that is not a working mechanic**

Lines 27 to 32

NOT EXISTS asks: is there no row in the Staff table with this number who is a mechanic and still active? If there is none, refuse.

### 6 Refuse a time in the past

Lines 33 to 37

GETDATE() is the clock right now. A start time earlier than now is refused.

### 7 Refuse a Sunday

Lines 38 to 42

DATENAME(WEEKDAY, …) gives the name of the day, such as Sunday. The workshop is shut on Sundays, so a Sunday is refused.

### 8 Refuse a time outside opening hours

Lines 43 to 47

CAST(… AS TIME) keeps only the clock part of a date and time. A job that starts before 08:30, or ends after 17:30, is refused.

### 9 The clash test

Lines 52 to 64

EXISTS asks: is there any booking in the table for this mechanic, not cancelled, whose time overlaps the new one? The two overlap lines are the two questions from stage 1. The new one starts before the old one ends, and the new one ends after the old one starts. If any such booking exists, refuse.

### 10 Save the booking

Lines 66 to 71

Only a booking that passed every check gets this far. INSERT adds one row to the Appointment table, with the values the form sent and the end time the stored procedure worked out.

### 11 Hand back the new number

Lines 73 to 74

SCOPE_IDENTITY() is the number SQL Server gave to the row it just added. It goes into the OUTPUT parameter, so the program knows which booking it made. END closes the stored procedure.

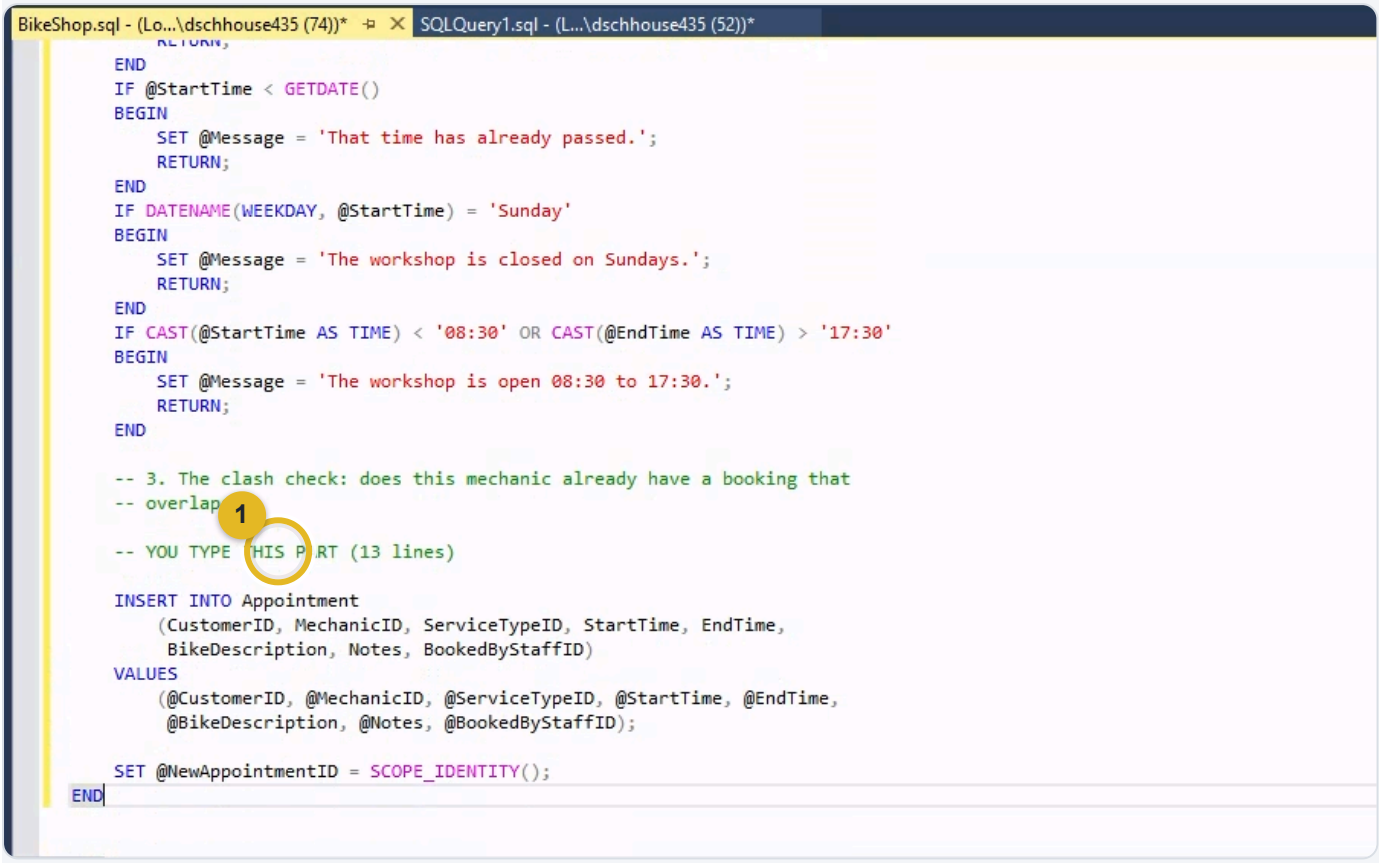
### 12 GO ends the batch

Line 75

GO is not SQL. It is a word for SSMS: run everything above this line as one batch, then start fresh. A CREATE OR ALTER PROCEDURE must be the first thing in its batch, so every stored procedure ends with GO before the next one begins. Leave GO out and the next one you paste fails with: CREATE PROCEDURE must be the first statement.

**Why you type it** This test is the only thing between your diary and a double booking. Type it, so that you can write it again on your own tables without looking.

Press the Copy button under this step, click on the empty line above the line that prints BikeShop is ready, and paste.




After the paste. Everything is in except the yellow lines. The marker line shows where they go.

1 Click on the marker line, type the yellow lines in its place, and delete the marker.

> **You should see** The stored procedure is in your script, with one line that says YOU TYPE THIS PART.

Type the yellow lines in place of the line that says YOU TYPE THIS PART, so that the marker line is gone.

> **You should see** The whole stored procedure is in your script, with no red wavy underline anywhere in it.

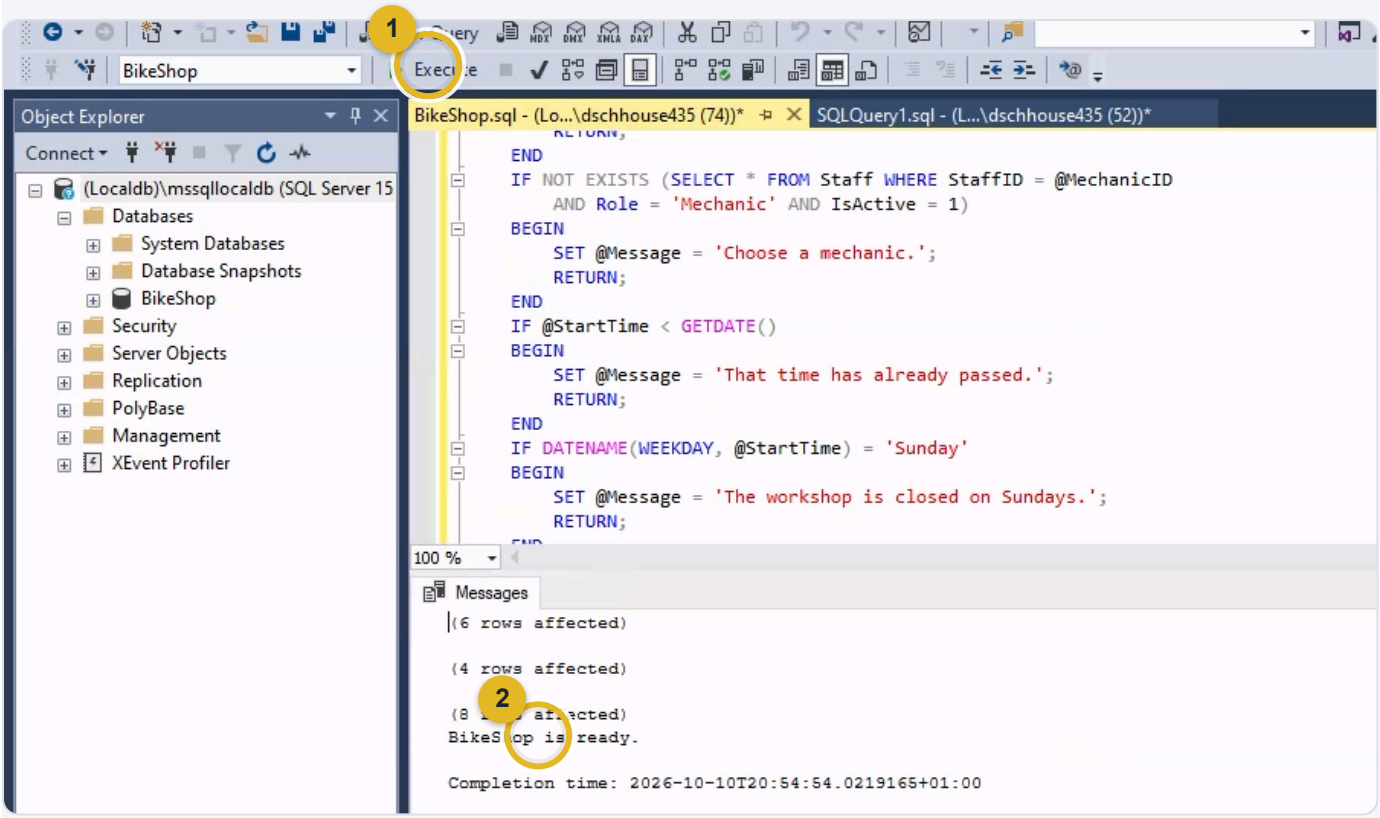
**IF IT GOES WRONG** A red wavy line under a word

Rest the mouse on it to read the message. 'Incorrect syntax near AND' usually means a missing bracket or comma on the line before. Compare your typed lines with the block, character by character.

**IF IT GOES WRONG** 'Must declare the scalar variable @StartTime'

A parameter name in your typed lines is spelled differently from the list at the top of the stored procedure. Check every letter.

Press F5, or click Execute on the toolbar, with nothing selected. This runs the whole script, so your database is built again from nothing with the new stored procedure in it.




Execute with nothing selected runs the whole script. The Messages pane counts the sample rows it put back, then says BikeShop is ready.

1. Click Execute, or press F5.

2. The last lines say BikeShop is ready, then a Completion time. Red text here is an error to read.

**You should see** The Messages pane at the bottom fills with lines such as (6 rows affected), which is SQL Server counting the sample rows it put back, and ends with BikeShop is ready and a Completion time. No red text.

**IF IT GOES WRONG** A red error in the Messages pane

Double-click the error. SSMS jumps to the line it is about. The line number counts from the top of the script, so the problem is usually in the part you just typed.

# Read the test that makes one good booking, and run it twice.

A stored procedure does nothing until something calls it. This short test calls it once. It books Aoife for a basic service at 13:00 tomorrow, and prints what came back. You run it in a second window, so that your script stays clean.

Copy this one **A test that books Aoife at 13:00 tomorrow** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
-- Book Aoife (mechanic 3) for a basic service (service 6) at 13:00 on the test
-- day.
-- Run it once: it books.  Run it again: the same slot is refused.
DECLARE @Day DATE, @Start DATETIME, @NewID INT, @Message VARCHAR(200);
-- tomorrow
SET @Day = DATEADD(DAY, 1, CAST(GETDATE() AS DATE));
-- the workshop is shut on Sundays
IF DATENAME(WEEKDAY, @Day) = 'Sunday' SET @Day = DATEADD(DAY, 1, @Day);
-- 13:00 on the test day
SET @Start = DATEADD(HOUR, 13, CAST(@Day AS DATETIME));

EXEC usp_Appointment_Add
     @CustomerID = 1, @MechanicID = 3, @ServiceTypeID = 6, @StartTime = @Start,
     @BikeDescription = 'Blue Trek hybrid', @Notes = NULL, @BookedByStaffID = 2,
     @NewAppointmentID = @NewID OUTPUT, @Message = @Message OUTPUT;

IF @Message = ''
BEGIN
    PRINT 'Booked. New appointment number ' + CAST(@NewID AS VARCHAR) + ' at '
        + CAST(CAST(@Start AS TIME) AS VARCHAR(5)) + ' on '
        + CAST(@Day AS VARCHAR) + '.';
END
ELSE
BEGIN
    PRINT @Message;
END
```

**1 Four boxes to hold values**

Line 4

A variable is a named box that holds one value while the script runs. @Day will hold the test day, @Start the start time, @NewID the number of the new booking, and @Message the stored procedure's answer.

**2 Pick tomorrow at 13:00**

Lines 6 to 10

GETDATE() is today. Adding one day gives tomorrow. If tomorrow is a Sunday, the test moves on one more day, because the workshop is shut. Then 13 hours are added to the start of that day, which gives 13:00.

**3 Run the stored procedure**

Lines 12 to 15

EXEC runs the stored procedure. Each parameter is named, so the order does not matter. Customer 1, mechanic 3 (Aoife) and service 6 (a basic service) are rows from the sample data. The two boxes marked OUTPUT are filled in by the stored procedure.

**4 If the message is empty, it worked**

Lines 17 to 22

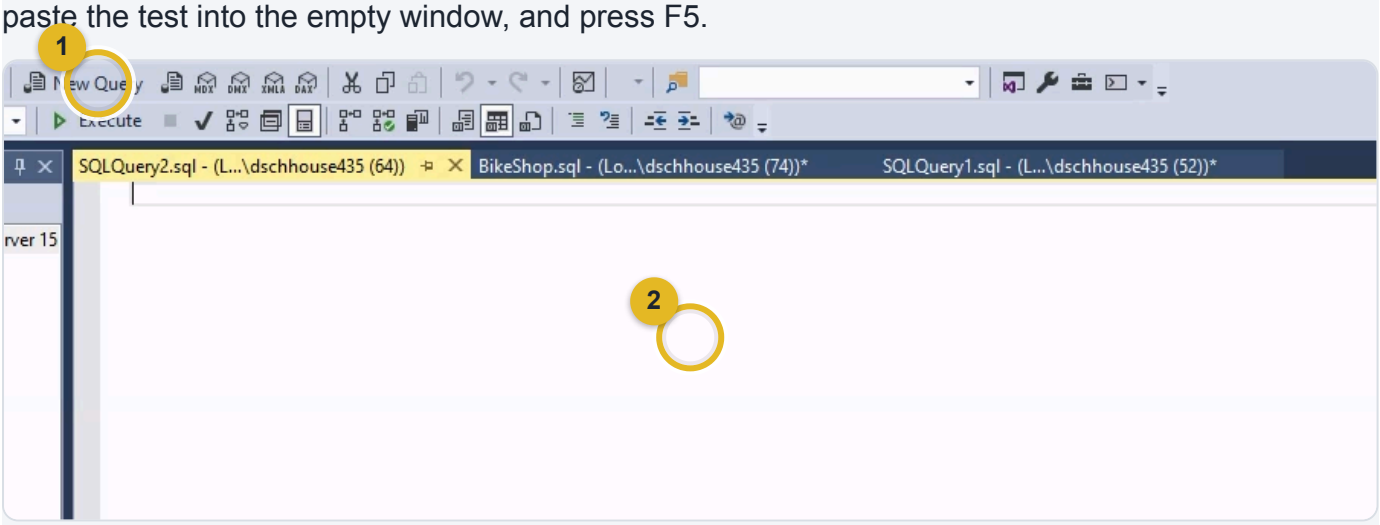
An empty message means the booking was saved. PRINT writes one line in the Messages pane, with the new number, the time and the day. CAST turns a number or a date into text so that it can be joined to the words.

**5 Otherwise, print the refusal**

Lines 23 to 26

If @Message holds a sentence, the stored procedure refused, and the sentence is printed as it is.

Click New Query on the toolbar, or press Ctrl and N together. Press the Copy button under this step, paste the test into the empty window, and press F5.



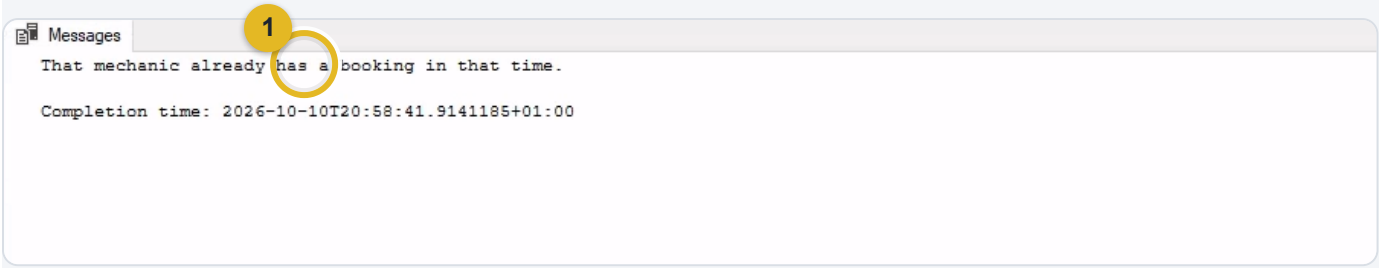

New Query opens a second, empty window. Tests go in here, so your script stays clean.

1 Click New Query, or press Ctrl and N together.

2 Paste the test into the empty window.

**You should see** The Messages pane says (1 row affected), and under it: Booked. New appointment number 15 at 13:00 on 2026-10-12. The first line is SQL Server counting the row it saved. Your number and date will be different. Write the number down, because stage 7 uses it.

Press F5 again, changing nothing.




The second run: the same slot, refused by the stored procedure. Nothing was saved.

1 The sentence came from @Message. This is the refusal.

> **You should see** The Messages pane says only: That mechanic already has a booking in that time. The stored procedure refused the second booking, and nothing was saved.

STAGE 2 OF 8 · THE STORED PROCEDURE THAT REFUSES A DOUBLE BOOKING · SCREEN 4 OF 4

# Try a refusal of your own, then put the test back.

On the line that works out @Start, change the 13 in DATEADD(HOUR, 13, …) to 7, and press F5.

> **You should see** The Messages pane says: The workshop is open 08:30 to 17:30. Seven in the morning is before the workshop opens, so the stored procedure refused, and nothing was saved.

Change the 7 back to 13. Leave this test window open: stage 6 uses it again.

> **You should see** The test reads exactly as it did when you pasted it. If you press F5 by mistake, it is refused again, because Aoife already has 13:00, and nothing changes.

**IF IT GOES WRONG** 'Choose a mechanic.' instead of a booking

Staff number 3 is not a mechanic in your Staff table, or is not active. Look at your Staff rows and use the number of a mechanic who is.

**IF IT GOES WRONG** 'That time has already passed.'

Your test day is being worked out as today. Check that the two SET lines near the top of the test are exactly as shown.

**IF IT GOES WRONG** 'The workshop is open 08:30 to 17:30.' when you expected a booking

A basic service is 60 minutes, so 17:00 is the last start that fits. Check that the hour in DATEADD(HOUR, 13, …) is 13 again.

# Read the stored procedure that moves a booking, then add it to your script.

Moving a booking means giving a booking that already exists a new start time. The clash test is the same two questions, with one extra line: a booking must not be tested against itself. Without that line, moving a booking fifteen minutes later would be refused, because its new time overlaps where it already is.

**Type this one** **usp_Appointment_Move: give a booking a new start time** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
CREATE OR ALTER PROCEDURE usp_Appointment_Move
    @AppointmentID INT,
    @NewStartTime  DATETIME,
    @Message VARCHAR(200) OUTPUT
AS
BEGIN
    SET @Message = '';

    DECLARE @MechanicID INT, @Minutes INT, @Status VARCHAR(10);
    SELECT @MechanicID = MechanicID,
       @Minutes    = DATEDIFF(MINUTE, StartTime, EndTime),
       @Status     = Status
    FROM Appointment
    WHERE AppointmentID = @AppointmentID;

    IF @MechanicID IS NULL
    BEGIN
        SET @Message = 'That booking does not exist.';
        RETURN;
    END
    IF @Status <> 'Booked'
    BEGIN
        SET @Message = 'Only a booking that is still open can be moved.';
        RETURN;
    END

    DECLARE @NewEndTime DATETIME;
    SET @NewEndTime = DATEADD(MINUTE, @Minutes, @NewStartTime);

    IF @NewStartTime < GETDATE()
    BEGIN
        SET @Message = 'That time has already passed.';
        RETURN;
    END
    IF DATENAME(WEEKDAY, @NewStartTime) = 'Sunday'
    BEGIN
        SET @Message = 'The workshop is closed on Sundays.';
        RETURN;
    END
    IF CAST(@NewStartTime AS TIME) < '08:30'
        OR CAST(@NewEndTime AS TIME) > '17:30'
    BEGIN
        SET @Message = 'The workshop is open 08:30 to 17:30.';
        RETURN;
    END

    IF EXISTS (SELECT *
               FROM Appointment
               WHERE MechanicID = @MechanicID
                 -- a booking cannot clash with itself
                 AND AppointmentID <> @AppointmentID
                 AND Status <> 'Cancelled'
                 AND @NewStartTime < EndTime
```

```
54                  AND @NewEndTime   > StartTime)
55      BEGIN
56          SET @Message = 'That mechanic already has a booking in that time.';
57          RETURN;
58      END
59
60      UPDATE Appointment
61      SET StartTime = @NewStartTime, EndTime = @NewEndTime
62      WHERE AppointmentID = @AppointmentID;
63  END
64  GO
```

**1 The name, and what goes in and out**

Lines 1 to 4

A move needs only two things: which booking, and its new start time. @Message comes back empty, or with a sentence that says why the move was refused.

**2 Start with a blank message**

Lines 5 to 7

AS and BEGIN mark where the work starts. The first job is to make @Message empty. An empty message means it worked. Every refusal below puts a sentence into @Message instead, so the program always knows which it was.

**3 Look up the booking**

Lines 9 to 14

One SELECT fills three boxes from the booking's row: which mechanic it is for, how long it is, and its status. DATEDIFF counts the minutes between its start and its end.

**4 Refuse a number that is not a booking**

Lines 16 to 20

If no row matched, @MechanicID is still empty, so there is no such booking. Refuse.

**5 Refuse a booking that is finished or cancelled**

Lines 21 to 25

The sign <> means is not. Only a booking whose status is still Booked can be moved.

**6 Work out the new end time**

Lines 27 to 28

The job keeps its length. The same number of minutes is added to the new start time.

**7 Refuse a time in the past**

Lines 30 to 34

The same check as when a booking is made.

**8 Refuse a Sunday**

Lines 35 to 39

The same check as when a booking is made.

**9 Refuse a time outside opening hours**

Lines 40 to 45

The same check as when a booking is made, using the new start and the new end.

**10 The clash test, with one new line**

Lines 47 to 58

The same two overlap questions as before, plus AppointmentID <> @AppointmentID: leave this booking itself out of the search. Without that line, a booking would clash with where it already is, and it could never move by a few minutes.

**11 Change the two times**

Lines 60 to 64

UPDATE changes a row that is already in the table. Only StartTime and EndTime change. The customer, the mechanic and the service stay as they were. GO ends the batch, ready for the next stored procedure.

**Why you type it** The clash test again, from memory this time, with the one line that stops a booking clashing with itself.

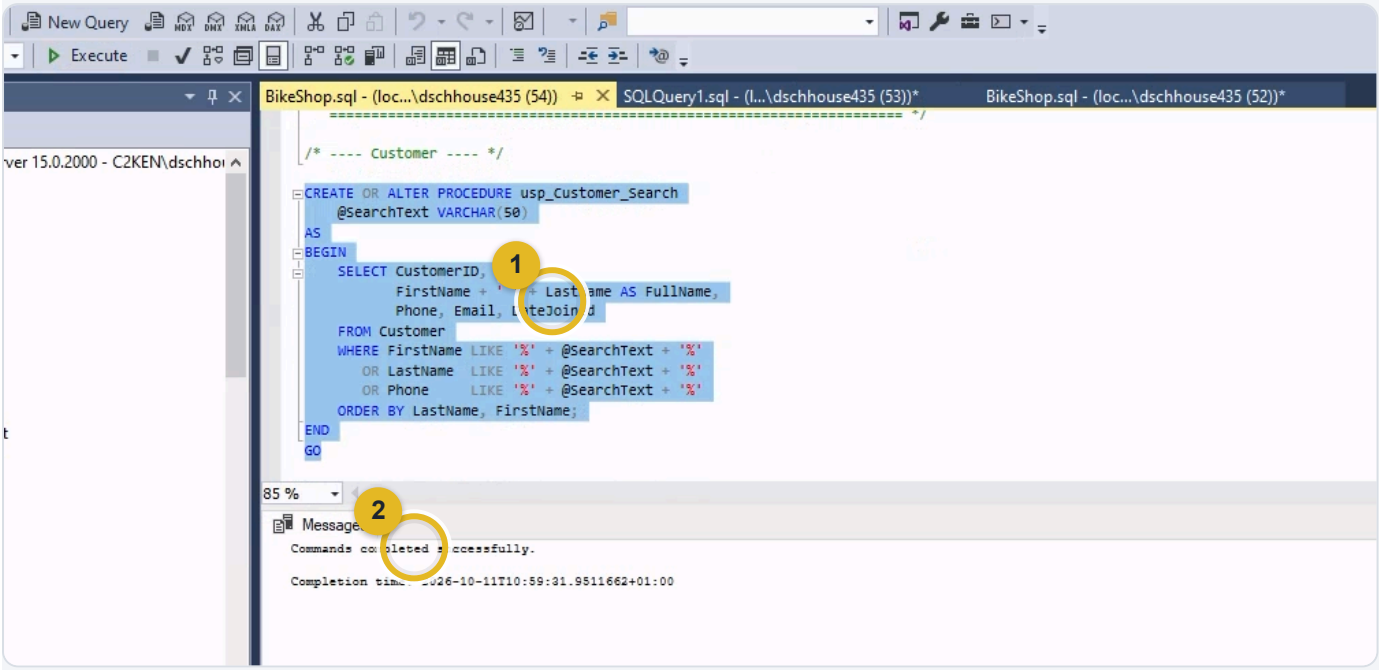
Press the Copy button under this step and paste it on the empty line above the line that prints BikeShop is ready.

**You should see** The stored procedure is in your script, with one line that says YOU TYPE THIS PART.

Type the yellow lines in place of the line that says YOU TYPE THIS PART.

**You should see** No line says YOU TYPE THIS PART any more, and there is no red wavy underline anywhere in the stored procedure.

Select the lines you just added, from CREATE OR ALTER PROCEDURE down to GO, and press F5.




Select the lines you just added, then press F5. Only the highlighted lines run, so the test bookings you have made stay in the database.

1. Drag from CREATE OR ALTER PROCEDURE down to GO, so the whole block is highlighted.

2. The Messages pane says: Commands completed successfully.

**You should see** The Messages pane says: Commands completed successfully. Only the selected lines ran, so your test booking from stage 2 is still there.

**IF IT GOES WRONG** The Messages pane shows (6 rows affected) and BikeShop is ready

Nothing was selected, so the whole script ran and the sample data was put back. Your test bookings are gone. Run the stage 2 test again to make a new one, and write down its new number.

# Read the test that moves a booking three times, and run it.

This test makes a fresh booking for Aoife at 15:00, then moves it three times. The first move is onto its own time. The second is fifteen minutes later. The third is onto her 13:00 booking from stage 2.

Copy this one **A test that books Aoife at 15:00 and moves the booking three times** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
-- Book Aoife at 15:00, then move that booking three times: onto itself, to
-- 15:15, and onto her 13:00 booking.
DECLARE @Day DATE, @Start DATETIME, @B INT, @Message VARCHAR(200);
SET @Day = DATEADD(DAY, 1, CAST(GETDATE() AS DATE));
IF DATENAME(WEEKDAY, @Day) = 'Sunday' SET @Day = DATEADD(DAY, 1, @Day);
-- 15:00 on the test day
SET @Start = DATEADD(HOUR, 15, CAST(@Day AS DATETIME));

EXEC usp_Appointment_Add
     @CustomerID = 2, @MechanicID = 3, @ServiceTypeID = 6, @StartTime = @Start,
     @BikeDescription = 'Black Giant road bike', @Notes = NULL,
         @BookedByStaffID = 2,
     @NewAppointmentID = @B OUTPUT, @Message = @Message OUTPUT;
IF @Message <> ''
BEGIN
    PRINT 'Could not make the 15:00 booking: ' + @Message;
    -- stop here
    RETURN;
END
PRINT 'Booked number ' + CAST(@B AS VARCHAR) + ' at 15:00.';

-- 1. Onto its own time: a booking never clashes with itself.
EXEC usp_Appointment_Move @AppointmentID = @B, @NewStartTime = @Start,
    @Message = @Message OUTPUT;
IF @Message = ''
BEGIN
    PRINT 'Moved onto itself: allowed.';
END
ELSE
BEGIN
    PRINT 'Moved onto itself: ' + @Message;
END

-- 2. To 15:15: the only booking it overlaps is itself, so still allowed.
SET @Start = DATEADD(MINUTE, 15, @Start);
EXEC usp_Appointment_Move @AppointmentID = @B, @NewStartTime = @Start,
    @Message = @Message OUTPUT;
IF @Message = ''
BEGIN
    PRINT 'Moved to 15:15: allowed.';
END
ELSE
BEGIN
    PRINT 'Moved to 15:15: ' + @Message;
END

-- 3. To 13:30: inside the 13:00 booking from the first section, so refused.
SET @Start = DATEADD(HOUR, 13, CAST(@Day AS DATETIME));
SET @Start = DATEADD(MINUTE, 30, @Start);
EXEC usp_Appointment_Move @AppointmentID = @B, @NewStartTime = @Start,
    @Message = @Message OUTPUT;
IF @Message = ''
BEGIN
```

```
54      PRINT 'Moved to 13:30: allowed. (Run the first section first and this ' +
55          'move is refused.)';
56  END
57  ELSE BEGIN PRINT 'Move to 13:30 refused: ' + @Message; END
```

**1 Set up tomorrow at 15:00**

Lines 3 to 7

The same four boxes as the first test. @B will hold the number of the booking that gets moved.

**2 Make a booking at 15:00 to move**

Lines 9 to 20

EXEC books Aoife at 15:00. If the stored procedure refused, the message is not empty: the test prints why, and RETURN stops the whole test. Otherwise it prints the new number.

**3 Move 1: onto its own time**

Lines 23 to 32

The new start time is the same as the old one. The extra line in the stored procedure leaves the booking itself out of the clash test, so this is allowed.

**4 Move 2: fifteen minutes later**

Lines 35 to 45

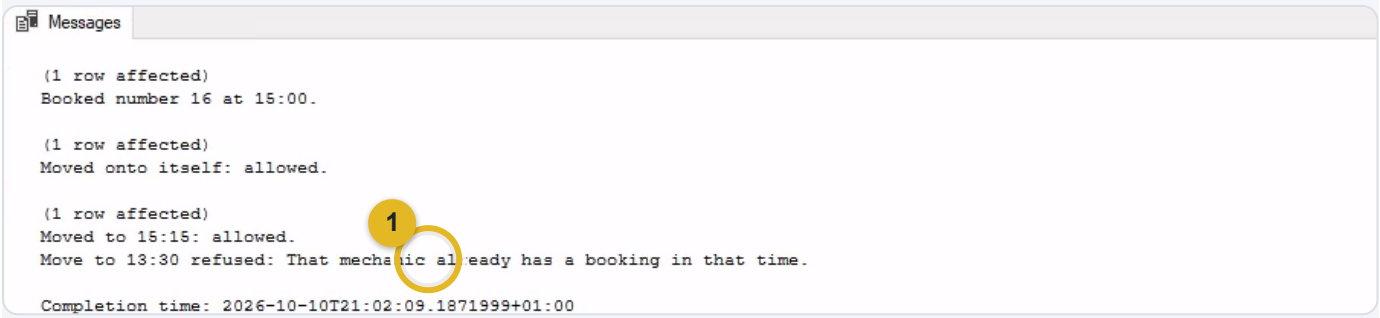
From 15:15 to 16:15 overlaps only the place where this booking already is, so it is allowed.

**5 Move 3: onto the 13:00 booking**

Lines 48 to 57

13:30 is inside Aoife's 13:00 to 14:00 booking from stage 2. Both answers are yes, so the move is refused, and the booking stays at 15:15.

Click New Query, press the Copy button under this step, paste the test, and press F5.




Three moves: two allowed, the third refused. Each (1 row affected) is SQL Server counting the row it saved or changed.

1 The last line is the refusal. The booking stayed at 15:15.

**You should see** Four lines in the Messages pane, with a (1 row affected) count above each of the first three: Booked number 16 at 15:00. Moved onto itself: allowed. Moved to 15:15: allowed. Move to 13:30 refused: That mechanic already has a booking in that time. Write this number down too. Stage 7 cancels it.

**IF IT GOES WRONG** 'Could not make the 15:00 booking: …' and nothing else

You ran this test twice, and Aoife already has your 15:00 booking from the first run. Change the 15 in DATEADD(HOUR, 15, …) to 16 and run it again.

**IF IT GOES WRONG** 'Only a booking that is still open can be moved.'

The booking number belongs to one that is done or cancelled. Book a fresh one and move that.

# Read how a bike hire is like a booking.

The hire shop has the same problem, with dates instead of times. A hire has a StartDate and an EndDate, which are whole days. A bike that is already out cannot be hired to someone else for days that overlap.

A live hire has a Status of Booked or Out. A hire that is Returned or Cancelled does not block the bike. So the clash test for hires looks at the Hire table, for the same BikeID, with a status of Booked or Out, and asks the same two questions with the dates.

Your usp_Hire_Add stored procedure from chapter 4 already looks up the bike, works out the cost and checks the deposit. It is missing only the clash test. You write that part yourself.

# Write the clash test for bike hires yourself.

Your turn **usp_Hire_Add: add the clash test before the INSERT** SQL

1. In SQL Server Management Studio (SSMS), find your usp_Hire_Add stored procedure in your script. It already works out the total cost and checks the deposit.
2. Just before its INSERT line, add an IF EXISTS test on the Hire table: the same BikeID, a Status of Booked or Out, and the two overlap questions with @StartDate and @EndDate in place of the times.
3. Refuse the way the booking stored procedure does: SET @Message = 'That bike is already out on those dates.'; and then RETURN;
4. Press F5 to run your script.

**THE TEST** The ready-made test that follows hires bike 1 for three days, starting ten days from now. Then it tries the same bike for three days starting two days into that hire.

**YOU SHOULD SEE** 'Hired. New hire number …' and then 'Overlapping hire refused: That bike is already out on those dates.' If you see 'The overlapping hire got through: the rule is broken.', your test is letting it through. Check both questions and the status list.

**HINT** The two questions are @StartDate < EndDate and @EndDate > StartDate. A hire that ends on a day and another that starts on that same day do not overlap, the same as two bookings that touch.

**Stuck? Show me the finished procedure**

Read it, then write yours **usp_Hire_Add** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
CREATE OR ALTER PROCEDURE usp_Hire_Add
    @BikeID          INT,
    @CustomerID      INT,
    @StartDate       DATE,
    @EndDate         DATE,
    @DepositPaid     DECIMAL(8,2),
    @BookedByStaffID INT,
    @NewHireID       INT OUTPUT,
    @Message VARCHAR(200) OUTPUT
AS
BEGIN
    SET @Message = '';

    DECLARE @DailyRate DECIMAL(6,2), @Purpose VARCHAR(4), @Status VARCHAR(10);
    SELECT @DailyRate = DailyRate, @Purpose = Purpose, @Status = Status
    FROM Bike
    WHERE BikeID = @BikeID;

    IF @Purpose IS NULL OR @Purpose <> 'Hire'
    BEGIN
        SET @Message = 'Choose a hire bike.';
        RETURN;
    END
    IF @Status <> 'Available'
    BEGIN
        SET @Message = 'That bike is not available for hire.';
        RETURN;
    END
    IF @EndDate ⩽ @StartDate
    BEGIN
        SET @Message = 'The return date must be after the start date.';
        RETURN;
    END
    IF @StartDate < CAST(GETDATE() AS DATE)
    BEGIN
        SET @Message = 'The start date has already passed.';
        RETURN;
    END

    DECLARE @Days INT;
    SET @Days = DATEDIFF(DAY, @StartDate, @EndDate);
    DECLARE @TotalCost DECIMAL(8,2);
    SET @TotalCost = @Days * @DailyRate;
    DECLARE @MinDeposit DECIMAL(8,2);
    SET @MinDeposit = ROUND(@TotalCost * 0.20, 2);

    IF @DepositPaid < @MinDeposit
    BEGIN
        SET @Message = 'The deposit must be at least £'
            + CAST(@MinDeposit AS VARCHAR)
            + ' (20% of £' + CAST(@TotalCost AS VARCHAR) + ').';
        RETURN;
    END
```

```
54
55      IF EXISTS (SELECT *
56                 FROM Hire
57                 WHERE BikeID = @BikeID
58                   AND Status IN ('Booked', 'Out')
59                   -- same rule as appointments, with dates
60                   AND @StartDate < EndDate
61                   AND @EndDate   > StartDate)
62      BEGIN
63          SET @Message = 'That bike is already out on those dates.';
64          RETURN;
65      END
66
67      INSERT INTO Hire (BikeID, CustomerID, StartDate, EndDate, TotalCost,
68          DepositPaid, BookedByStaffID)
69      VALUES (@BikeID, @CustomerID, @StartDate, @EndDate, @TotalCost,
70          @DepositPaid, @BookedByStaffID);
71
72      SET @NewHireID = SCOPE_IDENTITY();
73  END
74  GO
```

**1 The name, and what goes in and out**

Lines 1 to 9

A hire needs the bike, the customer, two dates, the deposit paid and who booked it. The new hire number and @Message come back out.

**2 Start with a blank message**

Lines 10 to 12

AS and BEGIN mark where the work starts. The first job is to make @Message empty. An empty message means it worked. Every refusal below puts a sentence into @Message instead, so the program always knows which it was.

**3 Look up the bike**

Lines 14 to 17

One SELECT fills three boxes from the bike's row: its price per day, what it is for (Hire or Sale), and its status.

**4 Refuse a bike that is not a hire bike**

Lines 19 to 23

NULL means no bike matched the number. A bike that is for sale cannot be hired.

**5 Refuse a bike that is not available**

Lines 24 to 28

A bike that is being repaired, or has been sold, is not available.

**6 Refuse a return date that is not after the start**

Lines 29 to 33

A hire must be at least one day long.

**7 Refuse a start date that has passed**

Lines 34 to 38

CAST(GETDATE() AS DATE) is today's date without the clock part.

**8 Work out the cost and the smallest deposit**

Lines 40 to 45

DATEDIFF counts the days. The cost is the days times the daily rate. The deposit must be at least 20 per cent of that, and ROUND keeps it to pennies.

**9 Refuse a deposit that is too small**

Lines 47 to 53

The sentence names the amounts. CAST turns each number into text so that it can be joined to the words.

**10 The clash test, with dates: this is the part you write**

Lines 55 to 65

Is there a hire of this bike, Booked or Out, whose dates overlap the new ones? The two questions are the same as for bookings, with the dates in place of the times. If there is, refuse.

**11 Save the hire**

Lines 67 to 70

One new row in the Hire table, with the cost the stored procedure worked out.

**12 Hand back the new number**

Lines 72 to 74

The number of the new row goes back through the OUTPUT parameter. GO ends the batch, ready for the next stored procedure.

# Test your hire stored procedure with two overlapping hires.

Copy this one **A test that hires bike 1, then tries to hire it again for overlapping days** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
-- Hire bike 1 to customer 1 for three days, starting ten days from now.  Then
-- try the same bike for dates that overlap.
DECLARE @From DATE, @To DATE, @H INT, @Message VARCHAR(200);
SET @From = DATEADD(DAY, 10, CAST(GETDATE() AS DATE));
SET @To = DATEADD(DAY, 3, @From);

EXEC usp_Hire_Add @BikeID = 1, @CustomerID = 1, @StartDate = @From,
    @EndDate = @To,
     @DepositPaid = 50, @BookedByStaffID = 2, @NewHireID = @H OUTPUT,
         @Message = @Message OUTPUT;
IF @Message = ''
BEGIN
    PRINT 'Hired. New hire number ' + CAST(@H AS VARCHAR) + ' from '
        + CAST(@From AS VARCHAR) + ' for 3 days.';
END
ELSE BEGIN PRINT 'Hire refused: ' + @Message; END

-- The same bike again, starting the day before that hire ends: the dates
-- overlap, so it must be refused.
SET @From = DATEADD(DAY, 2, @From);
SET @To = DATEADD(DAY, 3, @From);
EXEC usp_Hire_Add @BikeID = 1, @CustomerID = 2, @StartDate = @From,
    @EndDate = @To,
     @DepositPaid = 50, @BookedByStaffID = 2, @NewHireID = @H OUTPUT,
         @Message = @Message OUTPUT;
IF @Message = ''
BEGIN
    PRINT 'The overlapping hire got through: the rule is broken.';
END
ELSE BEGIN PRINT 'Overlapping hire refused: ' + @Message; END
```

**1 Pick three days, ten days from now**

Lines 3 to 5

@From is ten days from today. @To is three days after that. @H will hold the new hire number.

**2 Hire bike 1 to customer 1**

Lines 7 to 16

The first hire should work. The test prints the new number and the dates, or the reason it was refused.

**3 Move the dates two days on**

Lines 20 to 21

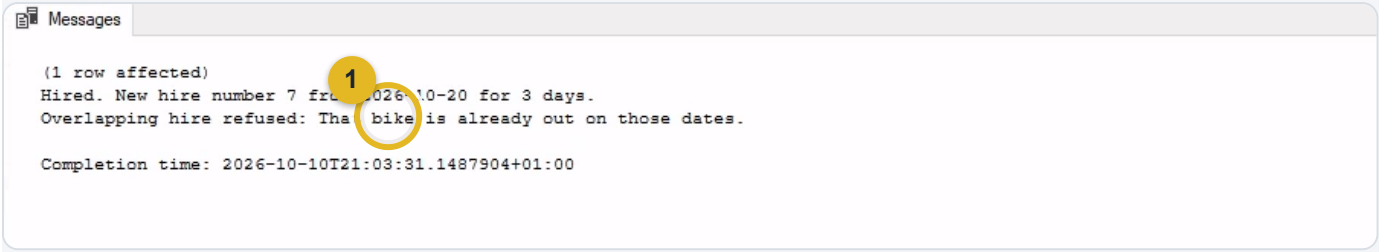
The second hire starts two days into the first one, so the two overlap by one day.

**4 Try the same bike again**

Lines 22 to 30

This one must be refused. If the line that says the rule is broken is printed, your clash test let it through.

Click New Query, press the Copy button under this step, paste the test, and press F5.




The first hire is made; the overlapping one is refused. Your hire number may be different.

1 If the second line says the rule is broken, go back to your clash test.

> **You should see** Two lines, with a (1 row affected) count above the first: Hired. New hire number 7 from 2026-10-20 for 3 days. Overlapping hire refused: That bike is already out on those dates. Your hire number may be different.

# See the four pieces of the booking form.

Now the C# side. The booking form has four pieces, and you make them in this order.

One more stored procedure, which tells the form which mechanics are free at the chosen time.

A data class called AppointmentData, which calls the stored procedures. It makes no decisions, so you copy it.

The form itself, which you build by dragging controls from the Toolbox and setting their properties.

The code behind the form. You copy the parts that fill the lists, and you type the two parts that make a decision.

# Read the stored procedure that lists the free mechanics, then add it to your script.

The form should never offer a mechanic who is already busy, so it asks the database. This is the same rule turned round: list each mechanic for whom no booking clashes with the slot.

Type this one **usp_Appointment_GetFreeMechanics: who is free at this time, for this service** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
CREATE OR ALTER PROCEDURE usp_Appointment_GetFreeMechanics
    @StartTime     DATETIME,
    @ServiceTypeID INT
AS
BEGIN
    DECLARE @EndTime DATETIME;
    SELECT @EndTime = DATEADD(MINUTE, DurationMinutes, @StartTime)
    FROM ServiceType
    WHERE ServiceTypeID = @ServiceTypeID;

    SELECT s.StaffID, s.FirstName + ' ' + s.LastName AS FullName
    FROM Staff s
    WHERE s.Role = 'Mechanic' AND s.IsActive = 1
      AND NOT EXISTS (SELECT *
                      FROM Appointment a
                      WHERE a.MechanicID = s.StaffID
                        AND a.Status <> 'Cancelled'
                        AND @StartTime < a.EndTime
                        AND @EndTime   > a.StartTime)
    ORDER BY s.FirstName, s.LastName;
END
GO
```

**1 The name, and what goes in and out**

Lines 1 to 3

This one takes a start time and a service, and gives back a list. There is no @Message, because a list is never refused.

**2 The start**

Lines 4 to 5

AS and BEGIN mark where the work starts.

**3 Work out when the job would end**

Lines 6 to 9

The same DATEADD as in the booking stored procedure: the service's length is added to the start time.

**4 Start with every working mechanic**

Lines 11 to 13

s is a short name for the Staff table. FirstName + ' ' + LastName joins the two names with a space, and AS FullName names that column. Only active mechanics are kept.

**5 Keep only the ones with no clash**

Lines 14 to 19

NOT EXISTS keeps a mechanic only when there is no booking of theirs that overlaps the slot. a is a short name for the Appointment table. The two overlap questions point at a.StartTime and a.EndTime, the times already in the table.

**6 Sort by name**

Lines 20 to 22

The list comes out in name order, which is how the form shows it. GO ends the batch, ready for the next stored procedure.

**Why you type it** The clash test a third time, inside NOT EXISTS. Each mechanic is kept only when no booking of theirs overlaps the slot.

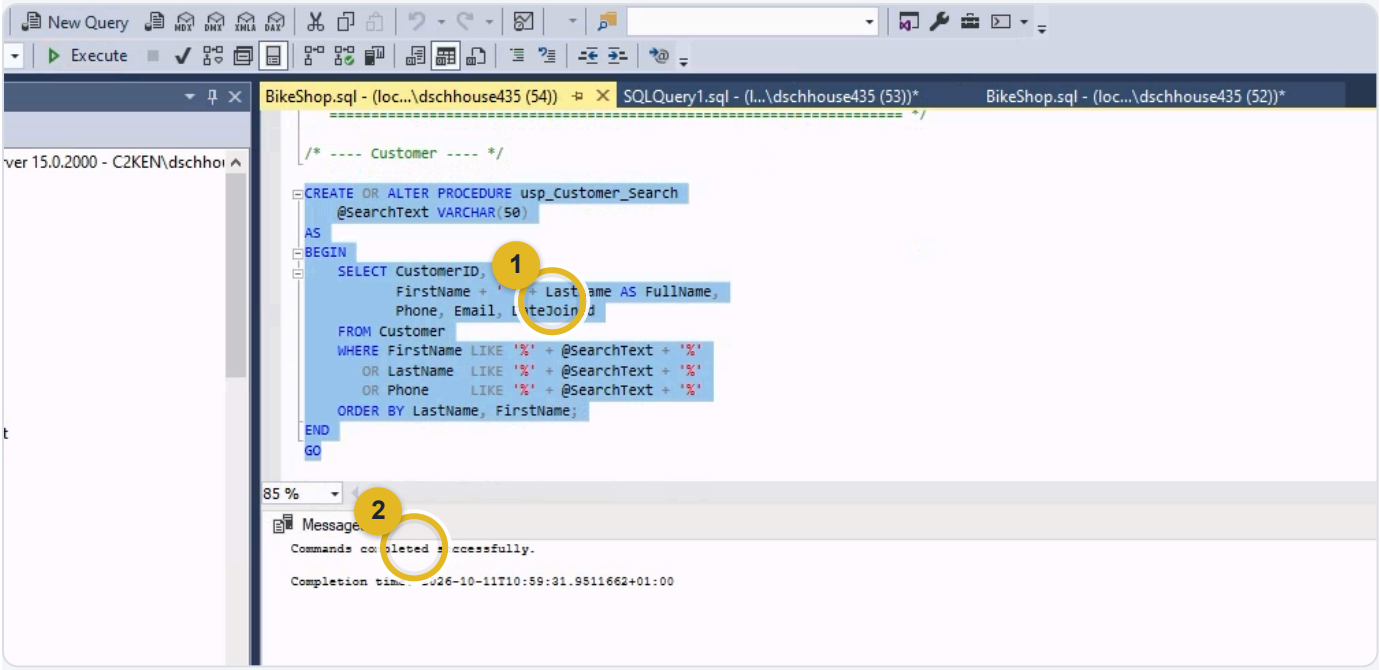
Press the Copy button under this step and paste it on the empty line above the line that prints BikeShop is ready.

**You should see** The stored procedure is in your script, with one line that says YOU TYPE THIS PART.

Type the yellow lines in place of the line that says YOU TYPE THIS PART.

**You should see** No line says YOU TYPE THIS PART any more, and there is no red wavy underline anywhere in the stored procedure.

Select the lines you just added, from CREATE OR ALTER PROCEDURE down to GO, and press F5.




Select the lines you just added, then press F5. Only the highlighted lines run, so the test bookings you have made stay in the database.

1. Drag from CREATE OR ALTER PROCEDURE down to GO, so the whole block is highlighted.

2. The Messages pane says: Commands completed successfully.

**You should see** The Messages pane says: Commands completed successfully. Your test bookings are still there.

**IF IT GOES WRONG** The Messages pane shows (6 rows affected) and BikeShop is ready

Nothing was selected, so the whole script ran and the sample data was put back. Your test bookings are gone. Run the stage 2 test again to make a new one, and write down its new number.

# Ask the database who is free at 13:00.

**Copy this one** **A test that asks who is free at 13:00 tomorrow for a basic service** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  -- Who is free at 13:00 on the test day for a basic service?  After the first
2  -- section, Aoife is missing from the list.
3  DECLARE @Day DATE, @Start DATETIME;
4  SET @Day = DATEADD(DAY, 1, CAST(GETDATE() AS DATE));
5  IF DATENAME(WEEKDAY, @Day) = 'Sunday' SET @Day = DATEADD(DAY, 1, @Day);
6  SET @Start = DATEADD(HOUR, 13, CAST(@Day AS DATETIME));
7
8  EXEC usp_Appointment_GetFreeMechanics @StartTime = @Start, @ServiceTypeID = 6;
```

**1 Tomorrow at 13:00 again**

Lines 3 to 6

The same two SET lines as the first test, so the time matches your 13:00 booking.

**2 Ask who is free**

Line 8

Service 6 is a basic service. The answer is a list, so it appears in the Results pane, not the Messages pane.

Click New Query, press the Copy button under this step, paste the test, and press F5.

Who is free at 13:00 for a basic service. Aoife Byrne is missing because of your booking.

1 Two columns: the number the program keeps, and the name the form shows.

**You should see** A small table in the Results pane with two columns, StaffID and FullName. Aoife is not in it, because of your 13:00 booking. Conor and Niamh are.

# Add the Appointment and Hire classes, then read the AppointmentData class and add it in Visual Studio 2019.

First, two small classes for the Models folder, built the same way as the Customer class from chapter 6. Appointment holds one booking. Hire holds one bike hire. You need Hire in chapter 8, and you add it now because it is the same job. Neither one makes a decision, so you copy them.

**Copy this one** **Models/Appointment.cs: one booking** C#

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  using System;
2
3  namespace BikeShop.Models
4  {
5      // One service booking.  Plain properties only: the form checks the values,
6      // and the procedure checks the clash.
7      public class Appointment
8      {
9          public int AppointmentID { get; set; }
10         public int CustomerID { get; set; }
11         public int MechanicID { get; set; }
12         public int ServiceTypeID { get; set; }
13         public DateTime StartTime { get; set; }
14         public string BikeDescription { get; set; }
15         public string Notes { get; set; }
16         public int BookedByStaffID { get; set; }
17     }
18 }
```

**1 The class**

Lines 1 to 8

using System gives the file the DateTime type, which holds a date and a time together. The class sits in the Models folder with Customer.

**2 One property for each column**

Lines 9 to 18

Each line is a property: one piece of information about the booking, with the same name as its column in the Appointment table. { get; set; } means the program can read it and change it. The two closing braces end the class and the namespace.

Copy this one **Models/Hire.cs: one bike hire** C#

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  using System;
2
3  namespace BikeShop.Models
4  {
5      // One bike hire.  Plain properties only: the form checks the dates and the
6      // deposit.
7      public class Hire
8      {
9          public int HireID { get; set; }
10         public int BikeID { get; set; }
11         public int CustomerID { get; set; }
12         public DateTime StartDate { get; set; }
13         public DateTime EndDate { get; set; }
14         public decimal DepositPaid { get; set; }
15         public int BookedByStaffID { get; set; }
16
17         public int Days
18         {
19             get { return (EndDate - StartDate).Days; }
20         }
21     }
22 }
```

**1 The class**

Lines 1 to 8

The same start as Appointment. A hire has a start date and an end date, so it needs DateTime too.

**2 One property for each column**

Lines 9 to 15

One property for each column in the Hire table. decimal is the type for money, so DepositPaid is a decimal.

**3 A property that works out its own answer**

Lines 17 to 22

Days has only a get, so the program can read it but never set it. It takes the start date away from the end date and gives the number of days in between. The hire form in chapter 8 uses it to work out the cost.

Now the data class. It is the same shape as CustomerData from chapter 6: one small method for each stored procedure, and nothing else. Read it, then add it.

Copy this one **Data/AppointmentData.cs: one method for each stored procedure** C#

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  using System;
2  using System.Data;
3  using System.Data.SqlClient;
4  using BikeShop.Models;
5
6  namespace BikeShop.Data
7  {
8      public static class AppointmentData
9      {
10         #region GetByDay
11         // Every booking on one day.
12         public static DataTable GetByDay(DateTime day)
13         {
14             return Db.GetTable("usp_Appointment_GetByDay",
15                 Db.P("@Day", day.Date));
16         }
17         #endregion
18
19         #region GetFreeMechanics
20         // The mechanics who are free for the whole length of this service,
21         // starting at this time.
22         public static DataTable GetFreeMechanics(DateTime startTime,
23             int serviceTypeID)
24         {
25             return Db.GetTable("usp_Appointment_GetFreeMechanics",
26                 Db.P("@StartTime", startTime),
27                 Db.P("@ServiceTypeID", serviceTypeID));
28         }
29         #endregion
30
31         // All the bookings for one customer.
32         public static DataTable GetForCustomer(int customerID)
33         {
34             return Db.GetTable("usp_Appointment_GetForCustomer",
35                 Db.P("@CustomerID", customerID));
36         }
37
38         #region Add
39         // Saves a new booking.  The procedure works out the end time and
40         // refuses a clash.
41         // Returns "" if it worked, or the sentence to show the user.
42         public static string Add(Appointment appointment)
43         {
44             SqlParameter newID = Db.Out("@NewAppointmentID");
45             string message = Db.Run("usp_Appointment_Add",
46                 Db.P("@CustomerID", appointment.CustomerID),
47                 Db.P("@MechanicID", appointment.MechanicID),
48                 Db.P("@ServiceTypeID", appointment.ServiceTypeID),
49                 Db.P("@StartTime", appointment.StartTime),
50                 Db.P("@BikeDescription", appointment.BikeDescription),
51                 Db.P("@Notes", appointment.Notes),
52                 Db.P("@BookedByStaffID", appointment.BookedByStaffID),
53                 newID);
```

```
54              if (message == "")
55              {
56                  appointment.AppointmentID = (int)newID.Value;
57              }
58              return message;
59          }
60          #endregion
61
62          #region Move
63          // Moves a booking to a new start time.  The procedure checks the new
64          // time is free.
65          public static string Move(int appointmentID, DateTime newStartTime)
66          {
67              return Db.Run("usp_Appointment_Move",
68                  Db.P("@AppointmentID", appointmentID),
69                  Db.P("@NewStartTime", newStartTime));
70          }
71          #endregion
72
73          #region Cancel
74          // Cancels a booking.  Returns "" if it worked.
75          public static string Cancel(int appointmentID)
76          {
77              return Db.Run("usp_Appointment_Cancel",
78                  Db.P("@AppointmentID", appointmentID));
79          }
80          #endregion
81
82          // Marks a booking as done.
83          public static string MarkDone(int appointmentID)
84          {
85              return Db.Run("usp_Appointment_MarkDone",
86                  Db.P("@AppointmentID", appointmentID));
87          }
88      }
89  }
```

**1 The usings**

Lines 1 to 4

Each using line names a set of ready-made tools this file needs. System.Data.SqlClient is the one that talks to SQL Server. BikeShop.Models is where your Appointment class lives.

**2 The class**

Lines 6 to 9

static means you never make a copy of this class. You just write AppointmentData.Add(…). It sits in the Data folder with CustomerData.

**3 GetByDay: one day’s bookings**

Lines 10 to 17

Db.GetTable runs a stored procedure and gives back a table of rows. Db.P makes one named parameter. The #region and #endregion lines only fold the code up in Visual Studio. They do nothing when the program runs.

**4**

**GetFreeMechanics: who is free**

Lines 19 to 29

Two parameters this time, the start time and the service. Back comes the list of free mechanics for the combo box.

**5** **GetForCustomer: used in chapter 9**

Lines 32 to 36

All the bookings for one customer. The report in chapter 9 uses this one.

**6** **Add: send the booking, read the answer**

Lines 38 to 60

Db.Out makes an OUTPUT parameter. Db.Run runs the stored procedure and gives back @Message as a string. If the string is empty, the new number is read out of newID and kept in the appointment object. The string goes back to the form, which shows it if it is a sentence.

**7** **Move: send the new time**

Lines 62 to 71

Only the booking number and the new start time go to the database. Back comes the empty string, or the reason.

**8** **Cancel**

Lines 73 to 80

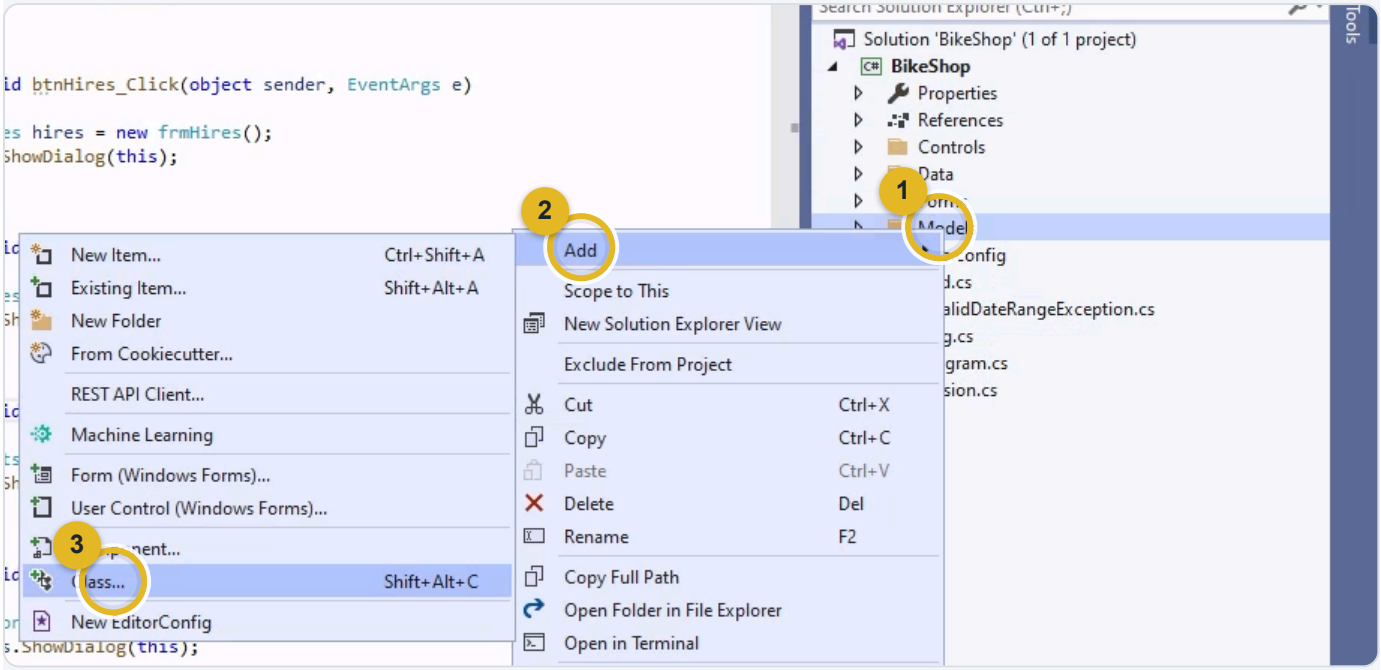
One number in, the answer out.

**9** **MarkDone**

Lines 83 to 89

The twin of Cancel. The two closing braces end the class and the namespace.

In Visual Studio, in Solution Explorer, right-click the Models folder, point at Add, and click Class…. Type Appointment.cs in the Name box and press Add.




Adding a class to the Models folder. Solution Explorer lists the classes you made in chapters 5 and 6.

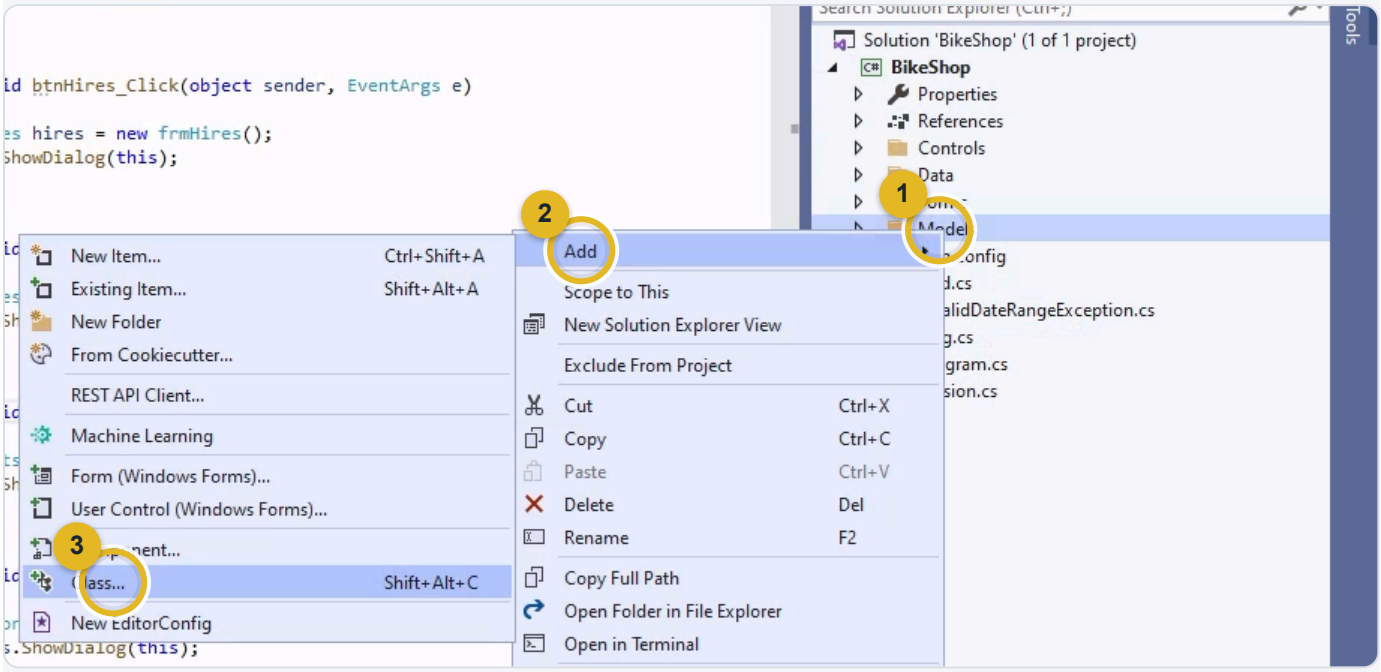
1. Right-click the Models folder in Solution Explorer.
2. Point at Add.
3. Click Class…

> **You should see** A new file called Appointment.cs opens with an empty class in it.

Click in the new file, press Ctrl and A together to select everything, press the Copy button under this step, and paste over it.

> **You should see** Appointment.cs holds the Appointment class with its eight properties, and no red wavy line anywhere.

Do the same for Hire. Right-click the Models folder, point at Add, click Class…, type Hire.cs in the Name box and press Add. Then select everything in the new file, press the Copy button under this step, and paste over it.




Adding a class to the Models folder. Solution Explorer lists the classes you made in chapters 5 and 6.

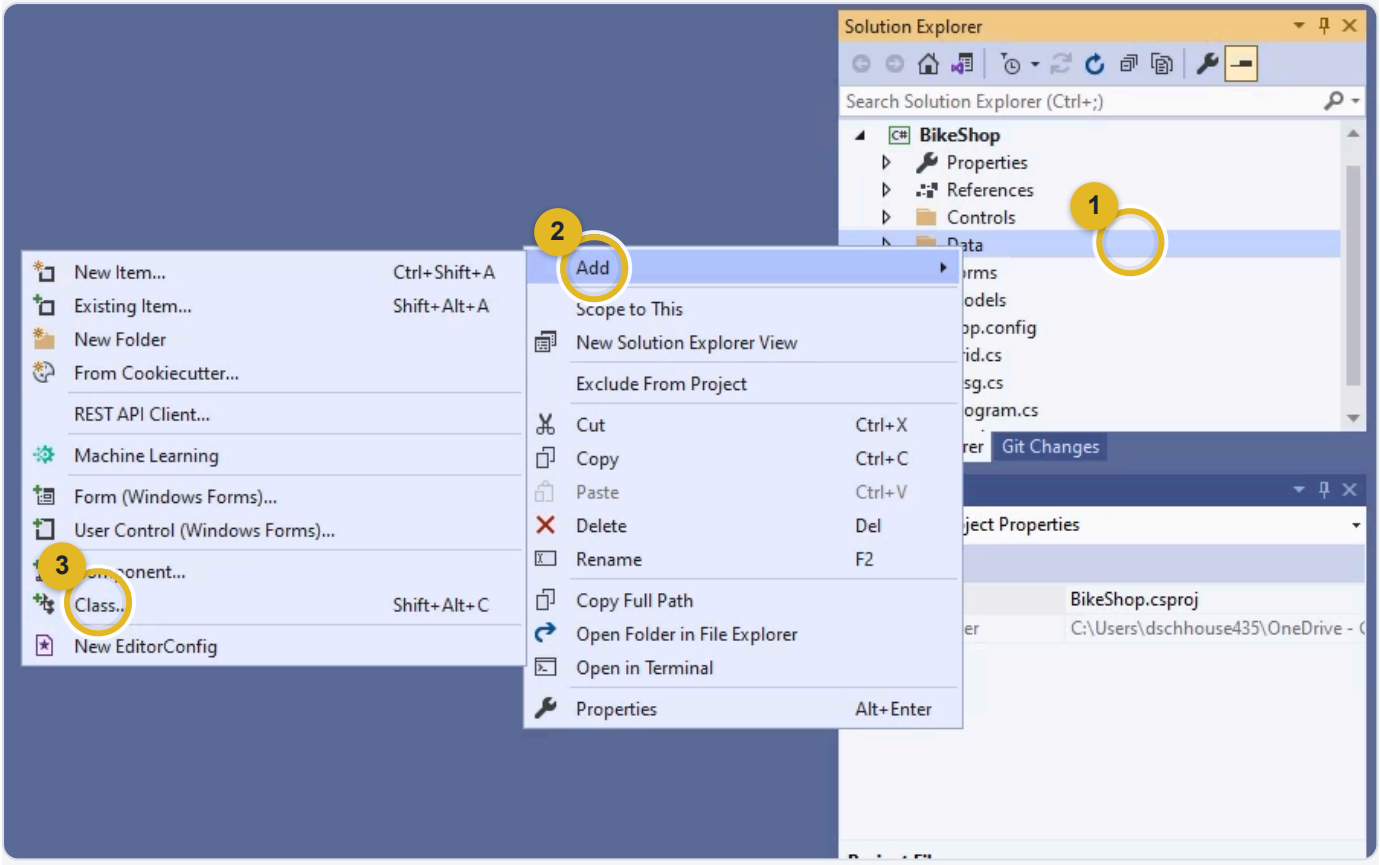
1. Right-click the Models folder in Solution Explorer.

2. Point at Add.

3. Click Class…

> **You should see** Hire.cs holds the Hire class, ending with the Days property, and no red wavy line anywhere.

In Visual Studio, in Solution Explorer, right-click the Data folder, point at Add, and click Class…




Adding a class to the Data folder. This picture was taken in the finished project, so Solution Explorer lists files you have not made yet.

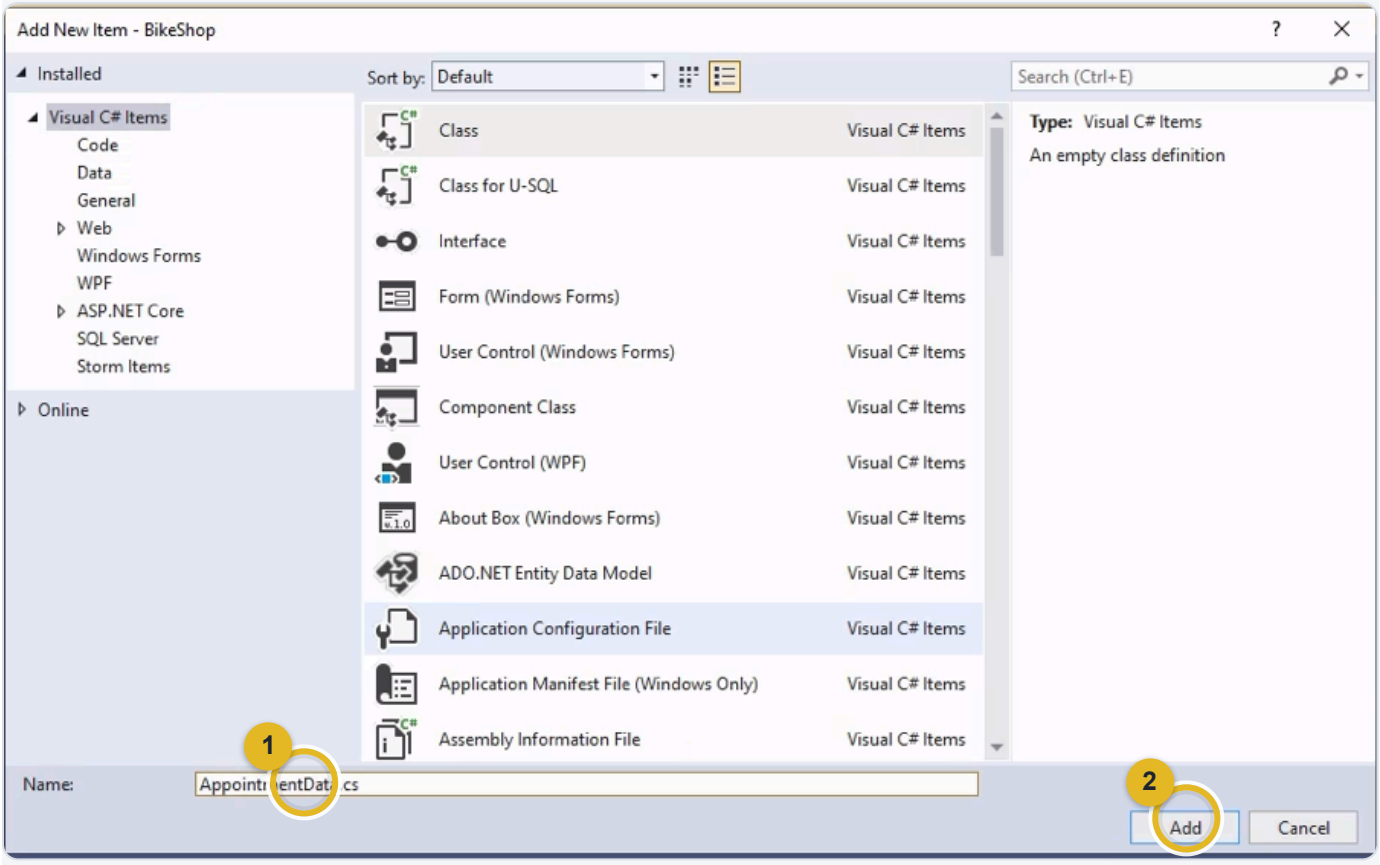
1. Right-click the Data folder in Solution Explorer.

2. Point at Add.

3. Click Class…

> **You should see** The Add New Item window opens with Class selected.

In the Name box at the bottom, type AppointmentData.cs and press Add.

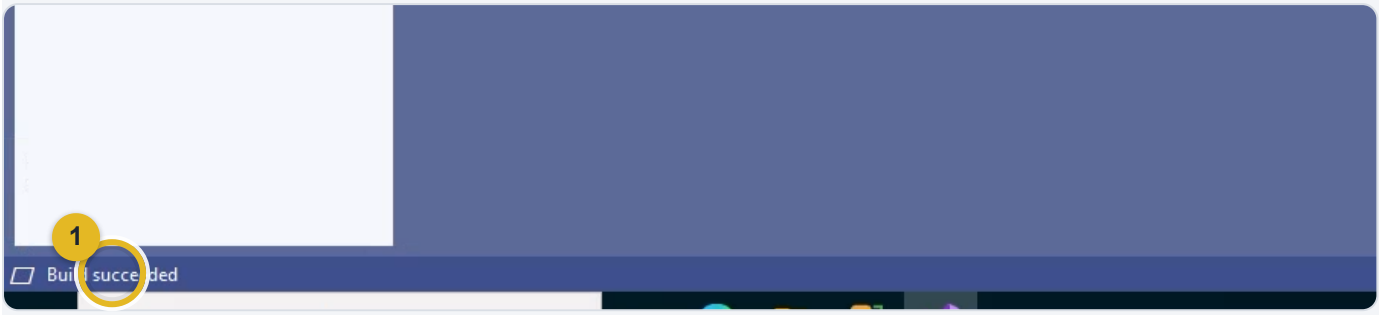



Name the new class before you press Add.

1. Type AppointmentData.cs in the Name box.
2. Press Add.

**You should see** A new file opens with an empty class in it.

Click in the new file, press Ctrl and A together to select everything, press the Copy button under this step, and paste over it. Then press F6 to build.




Build succeeded, at the bottom left. Anything else means an error list to read.

1 Build succeeded. Press F6 to build at any time.

**You should see** Build succeeded, at the bottom left.

**IF IT GOES WRONG** 'The name Db does not exist'

Your helper class from chapter 5 is called something else, or is in a different namespace. Match the name you used.

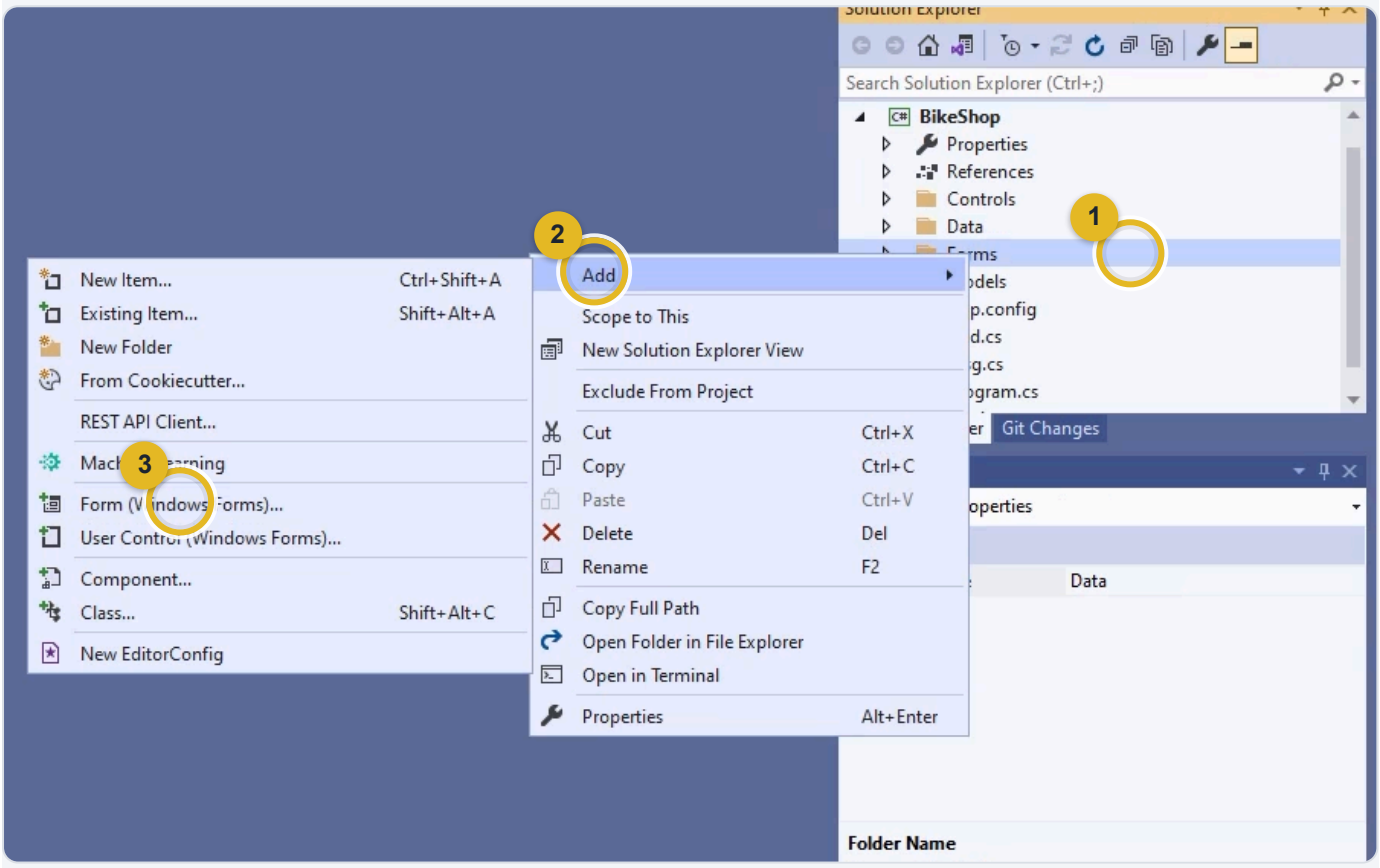
**IF IT GOES WRONG** 'The type or namespace name Appointment could not be found'

The Appointment class is missing from the Models folder. Go back to step 7.5.5 and add it.

# Add the booking form.

Now the form. It is a small window with six boxes down it, a grey note that only shows once the program is running, and two buttons. You add an empty form first, then drag each control on to it from the Toolbox.

In Solution Explorer, right-click the Forms folder, point at Add, and click Form (Windows Forms)…. Type frmAppointmentEdit as the name and press Add.

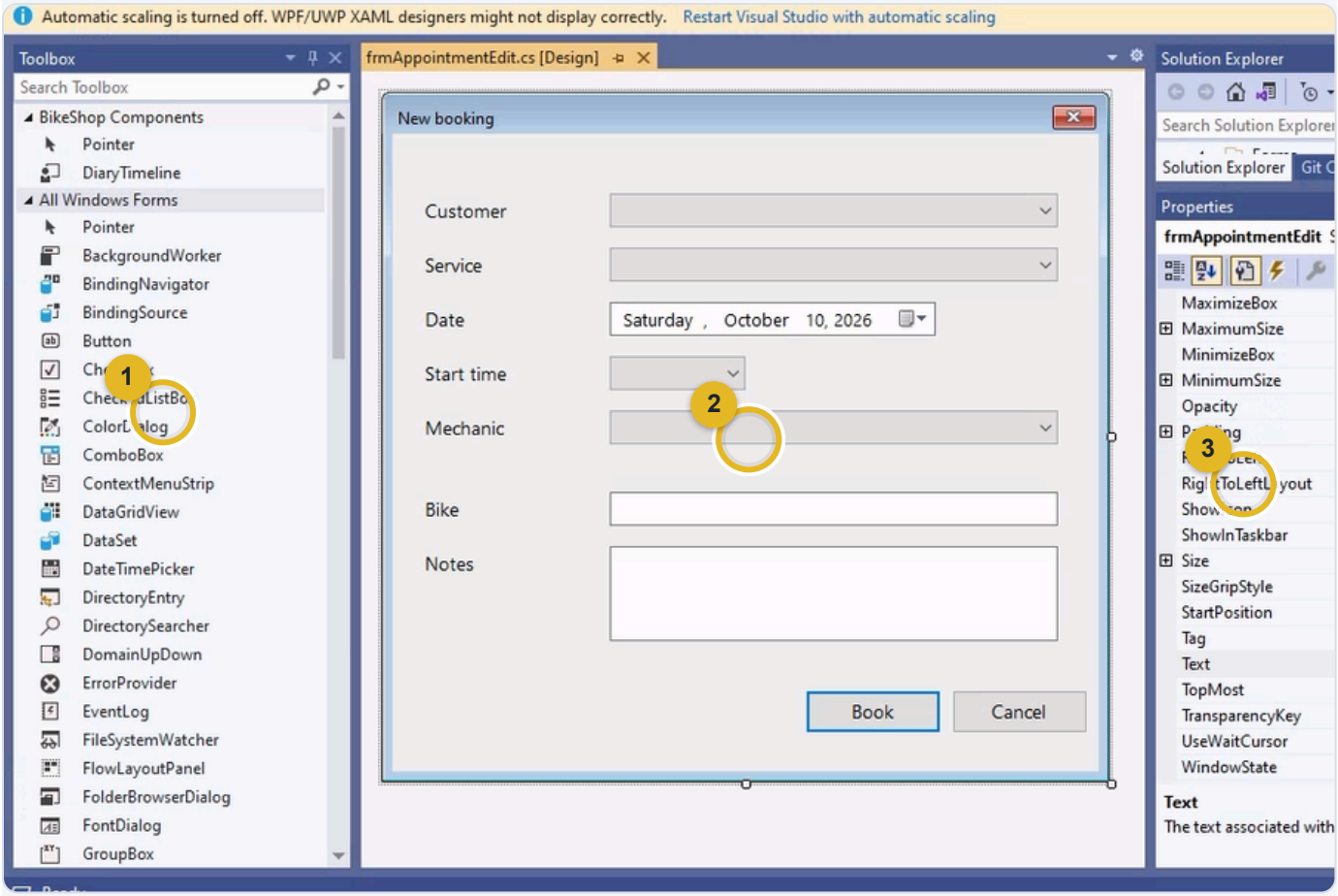



Adding a form to the Forms folder. Taken in the finished project, so Solution Explorer lists more files than yours.

1. Right-click the Forms folder.
2. Point at Add.
3. Click Form (Windows Forms)…, type the name, press Add.

**You should see** An empty form opens in the designer.

Open the Toolbox (View, then Toolbox) and the Properties window (View, then Properties Window), if they are not already open.




The three parts of the designer: the Toolbox you drag from, the form you drop on, the Properties window where you name and set each control.

1. The Toolbox. Drag a control from here on to the form.

2. The form. Drop the control where the picture shows it.

3. The Properties window. Set Name first, then the other settings in the table.

> **You should see** The Toolbox is on the left, the form in the middle, the Properties window on the right.

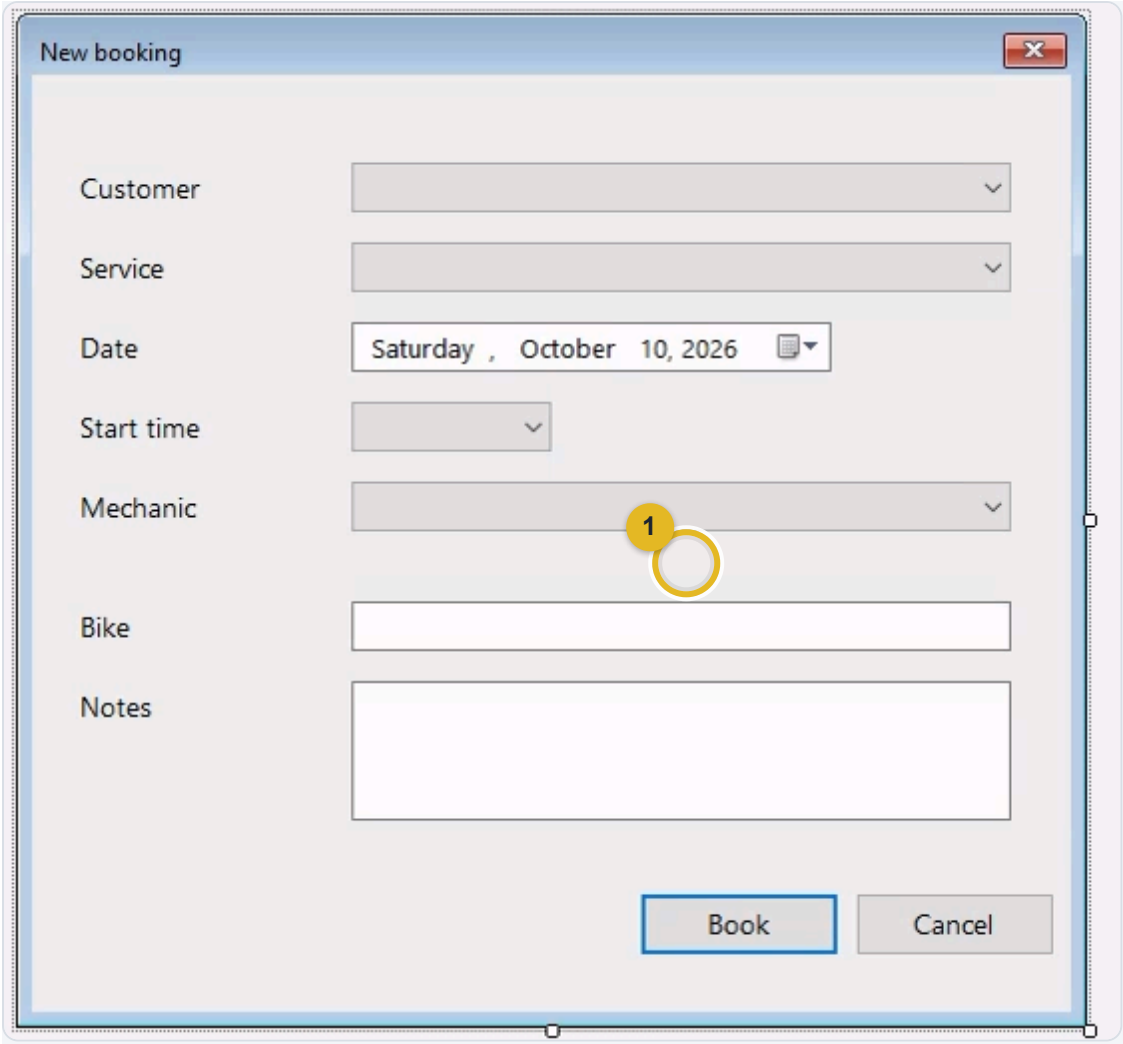
# Drag each control from the Toolbox and set its properties.

Click the form itself first and set the form's own properties. Then work down the table: drag the control from the Toolbox, drop it on the form, set its Name in the Properties window, and set the other settings in the row. Where the Then column says double-click, double-clicking the control makes its event method for you.

Books a repair. The mechanic list only ever shows mechanics who are free for that service at that time; the database checks again when you click Book.

**Click the form itself, then set these in the Properties window**

| | | | |
|---|---|---|---|
| Text | `New booking` | Size | `520, 470` |
| FormBorderStyle | `FixedDialog` | StartPosition | `CenterParent` |
| MaximizeBox | `False` | MinimizeBox | `False` |
| ShowInTaskbar | `False` | AcceptButton | `btnBook (after you add it)` |
| CancelButton | `btnCancel (after you add it)` | | |




frmAppointmentEdit in the designer. Yours should look like this before you add any code. The grey note has no words yet, so there is nothing to see where it sits.

1. Each box lines up with its label. The grey note sits under the Mechanic list, invisible until the program fills it in.

| DRAG THIS FROM THE TOOLBOX | NAME IT (THE NAME PROPERTY) | SET THESE IN THE PROPERTIES WINDOW | THEN |
|---|---|---|---|
| Label | `lblMoving` | nothing else<br>Empty for a new booking. When moving, says which booking is being moved. | |
| Label | `lblCustomer` | `Text = Customer` | |
| ComboBox | `cboCustomer` | `DropDownStyle = DropDownList`<br>Every customer, by name. | |
| Label | `lblService` | `Text = Service` | |
| ComboBox | `cboService` | `DropDownStyle = DropDownList`<br>Each service with its minutes and price. | Double-click it. Visual Studio makes `cboService_SelectedIndexChanged` for you; the code goes inside it. |
| Label | `lblDate` | `Text = Date` | |
| DateTimePicker | `dtpDate` | `Format = Long` | Double-click it. Visual Studio makes `dtpDate_ValueChanged` for you; the code goes inside it. |
| Label | `lblStart` | `Text = Start time` | |
| ComboBox | `cboStart` | `DropDownStyle = DropDownList`<br>Every quarter hour from 08:30 to 17:15. Filled by code when the window opens. | Double-click it. Visual Studio makes `cboStart_SelectedIndexChanged` for you; the code goes inside it. |
| Label | `lblMechanic` | `Text = Mechanic` | |
| ComboBox | `cboMechanic` | `DropDownStyle = DropDownList`<br>Only the mechanics who are free then. | |

| DRAG THIS FROM THE TOOLBOX | NAME IT (THE NAME PROPERTY) | SET THESE IN THE PROPERTIES WINDOW | THEN |
|---|---|---|---|
| Label | `lblMechanicNote` | `ForeColor = grey (92, 103, 116)`<br>"2 mechanics are free." or "No mechanic is free then." | |
| Label | `lblBike` | `Text = Bike` | |
| TextBox | `txtBike` | nothing else<br>Which bike: "Red Giant hybrid, rear wheel". | |
| Label | `lblNotes` | `Text = Notes` | |
| TextBox | `txtNotes` | `Multiline = True` | |
| Button | `btnBook` | `Text = Book`<br>Says "Move" when moving. | Double-click it. Visual Studio makes `btnBook_Click` for you; the code goes inside it. |
| Button | `btnCancel` | `Text = Cancel`<br>`DialogResult = Cancel` | |

# Read the top of the form’s code.

Four pieces of code go behind this form. Read them one at a time. Then you put the whole file in at once, and type the yellow lines as you go. This first piece is the top of the file.

Read this one **Forms/frmAppointmentEdit.cs: the top of the file** C#

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  using System;
2  using System.Collections.Generic;
3  using System.Data;
4  using System.Drawing;
5  using System.Windows.Forms;
6  using BikeShop.Data;
7  using BikeShop.Models;
8
9  namespace BikeShop.Forms
10 {
11     public partial class frmAppointmentEdit : Form
12     {
13         // 0 = a new booking; otherwise the booking being moved
14         private readonly int appointmentID;
15         // the date and time the form opens on
16         private readonly DateTime startWith;
17         // "Aoife McCann · Puncture repair with Conor" when moving
18         private readonly string movingWhat;
19
20         // New booking on a chosen day
21         public frmAppointmentEdit(DateTime day)
22         {
23             InitializeComponent();
24             startWith = day.Date.AddHours(9);
25         }
26
27         // Move an existing booking: only the date and time can change
28         public frmAppointmentEdit(int id, DateTime currentStart, string summary)
29         {
30             InitializeComponent();
31             appointmentID = id;
32             startWith = currentStart;
33             movingWhat = summary;
34         }
```

**1 The usings**

Lines 1 to 7

The tools this form needs: Windows Forms for the window itself, System.Data for the tables that fill the lists, and your own Data and Models folders.

**2 The form’s class**

Lines 9 to 12

partial means the designer keeps the other half of this class, the controls you dragged, in its own file. The words : Form mean it is a window.

**3 Three facts the form remembers**

Lines 14 to 18

readonly means set once, when the form is made, and never changed. appointmentID is 0 for a new booking. Any other number means a booking that is being moved.

**4 Opening it for a new booking**

Lines 21 to 25

A constructor runs when the form is made. InitializeComponent builds the controls from the designer's file. The diary passes in the day, and the form starts at 09:00 on that day.

**5 Opening it to move a booking**

Lines 28 to 34

The same form, made a second way. The booking number, its current start time and a one-line summary are remembered for later.

# Read what happens when the form loads.

This piece runs once, just before the form appears. It fills the three lists and, when a booking is being moved, locks everything except the date and the time.

Read this one **Forms/frmAppointmentEdit.cs: when the form loads** C#

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  private void frmAppointmentEdit_Load(object sender, EventArgs e)
2  {
3      // The two text boxes stop typing at the length the database column
4      // holds.
5      txtBike.MaxLength = 80;
6      txtNotes.MaxLength = 200;
7
8      FillTimes();
9      dtpDate.Value = startWith.Date;
10     cboStart.SelectedItem = startWith.ToString("HH:mm");
11
12     // Set the two names first, then the DataSource, so the combo box
13     // knows what to show and what to keep.
14     cboCustomer.DisplayMember = "FullName";
15     cboCustomer.ValueMember = "CustomerID";
16     cboCustomer.DataSource = CustomerData.Search("");
17
18     // "Puncture repair · 30 min · £15.00"
19     cboService.DisplayMember = "Display";
20     cboService.ValueMember = "ServiceTypeID";
21     cboService.DataSource = ServiceTypeData.GetAll();
22
23     if (appointmentID ≠ 0)
24     {
25         Text = "Move a booking";
26         lblMoving.Text = "Moving: " + movingWhat;
27         btnBook.Text = "Move";
28         cboCustomer.Enabled = false;
29         cboService.Enabled = false;
30         cboMechanic.Enabled = false;
31         txtBike.Enabled = false;
32         txtNotes.Enabled = false;
33         lblMechanicNote.Text = "The same mechanic keeps the job.";
34     }
35     else
36     {
37         lblMoving.Text = "";
38         RefreshMechanics();
39     }
40 }
41
42 // The shop opens 08:30 and the last slot starts 17:15: a time every 15
43 // minutes.
44 private void FillTimes()
45 {
46     cboStart.Items.Clear();
47     TimeSpan time = new TimeSpan(8, 30, 0);
48     while (time ⩽ new TimeSpan(17, 15, 0))
49     {
50         cboStart.Items.Add(time.ToString(@"hh\:mm"));
51         time = time.Add(TimeSpan.FromMinutes(15));
52     }
```

```
53  }
```

**1 When the form loads**

Lines 1 to 2

This method runs once, just before the form appears. The designer made its first line when you set the Load event.

**2 Stop typing at the column’s length**

Lines 5 to 6

The database column for the bike holds 80 characters and the notes column holds 200. MaxLength stops the user typing more than will fit.

**3 Fill the time list and pick the start**

Lines 8 to 10

FillTimes, lower down, fills the start-time list. Then the date picker and the time list are set to the time the form was opened with.

**4 Fill the customer list**

Lines 14 to 16

DisplayMember is the column to show, and ValueMember is the column to keep. Set both before DataSource, or the box shows System.Data.DataRowView instead of names.

**5 Fill the service list**

Lines 19 to 21

The Display column reads like Puncture repair · 30 min · £15.00, so the user sees the length and the price.

**6 When moving, lock everything but the date and time**

Lines 23 to 34

A number other than 0 means a booking is being moved. The title, the label and the button change their words, and every box except the date and the time is switched off.

**7 When new, fill the mechanic list**

Lines 35 to 40

A new booking: the moving label is cleared, and RefreshMechanics asks the database who is free.

**8 Times every 15 minutes from 08:30 to 17:15**

Lines 44 to 53

TimeSpan is a length of time. The while loop adds 15 minutes each time round and puts the text, such as 09:45, into the list, until it passes 17:15.

# Read the part that asks which mechanics are free.

Whenever the date, the time or the service changes, the form asks the database again. The list is never out of date, and the Book button is switched off when nobody is free. The yellow lines are the ones you type.

Read this one **Forms/frmAppointmentEdit.cs: which mechanics are free** C#

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  // Asks the database which mechanics are free for this service at this
2  // time.
3  // Runs again whenever the date, the time or the service changes.
4  private void RefreshMechanics()
5  {
6      if (appointmentID ≠ 0 || cboService.SelectedValue == null
7          || cboStart.SelectedItem == null)
8      {
9          return;
10     }
11
12     cboMechanic.DisplayMember = "FullName";
13     cboMechanic.ValueMember = "StaffID";
14     cboMechanic.DataSource = AppointmentData.GetFreeMechanics(
15         ChosenStart(), (int)cboService.SelectedValue);
16
17     int free = cboMechanic.Items.Count;
18     if (free == 0)
19     {
20         lblMechanicNote.Text =
21             "No mechanic is free then. Try another time.";
22     }
23     else if (free == 1)
24     {
25         lblMechanicNote.Text = "1 mechanic is free.";
26     }
27     else
28     {
29         lblMechanicNote.Text = free + " mechanics are free.";
30     }
31     btnBook.Enabled = free > 0;
32 }
33
34 private void cboService_SelectedIndexChanged(object sender, EventArgs e)
35 {
36     RefreshMechanics();
37 }
38
39 private void dtpDate_ValueChanged(object sender, EventArgs e)
40 {
41     RefreshMechanics();
42 }
43
44 private void cboStart_SelectedIndexChanged(object sender, EventArgs e)
45 {
46     RefreshMechanics();
47 }
```

**1 Do nothing when there is nothing to ask**

Lines 4 to 10

If the form is moving a booking, or no service or time has been picked yet, there is nothing to ask. return leaves the method.

**2 Ask the database who is free**

Lines 12 to 15

Show the name, keep the number. The DataSource is the list that usp_Appointment_GetFreeMechanics gives back for the chosen start time and service.

**3 Count them: nobody free**

Lines 17 to 22

free is how many mechanics came back. If it is 0, the grey note says so in a full sentence.

**4 One mechanic free**

Lines 23 to 26

One is a special case, because the sentence reads differently.

**5 More than one free, and the Book button**

Lines 27 to 32

Otherwise the note gives the number. The last line switches the Book button on only when free is more than 0.

**6 The three events that call it**

Lines 34 to 47

Changing the service, the date or the start time each runs RefreshMechanics again. The designer made these three method headings when you double-clicked the controls.

**Why you type it** Whenever the date, the time or the service changes, the form asks the database again. The list is never out of date, and the Book button is off when nobody is free.

# Read the part behind the Book button.

The message check is the whole point. The database decides. The form only shows what the database said, and refreshes the list. The yellow lines are the ones you type.

Read this one **Forms/frmAppointmentEdit.cs: the Book button** C#

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
private void btnBook_Click(object sender, EventArgs e)
{
    if (appointmentID ≠ 0)
    {
        MoveBooking();
        return;
    }

    // Check what was chosen and typed, one problem at a time: show the
    // first and put the cursor there.
    if (ChosenID(cboCustomer) == 0)
    {
        Msg.Problem("Choose the customer.");
        cboCustomer.Focus();
        return;
    }
    if (ChosenID(cboService) == 0)
    {
        Msg.Problem("Choose the service.");
        cboService.Focus();
        return;
    }
    if (ChosenID(cboMechanic) == 0)
    {
        Msg.Problem("Choose a mechanic.");
        cboMechanic.Focus();
        return;
    }
    // The start time needs no check: the list only offers times in
    // 15-minute steps.
    if (txtBike.Text.Trim() == "")
    {
        Msg.Problem("Type a description of the bike.");
        txtBike.Focus();
        return;
    }

    Appointment booking = new Appointment();
    booking.CustomerID = ChosenID(cboCustomer);
    booking.ServiceTypeID = ChosenID(cboService);
    booking.MechanicID = ChosenID(cboMechanic);
    booking.StartTime = ChosenStart();
    booking.BikeDescription = txtBike.Text.Trim();
    booking.Notes = txtNotes.Text.Trim();
    if (booking.Notes == "")
    {
        // nothing typed: store a NULL, not an empty piece of text
        booking.Notes = null;
    }
    booking.BookedByStaffID = Session.StaffID;

    // The database checks the clash, the hours and the day, and answers
    // in words.
```

```
54      string message = AppointmentData.Add(booking);
55      if (message ≠ "")
56      {
57          // e.g. "That mechanic already has a booking in that time."
58          Msg.Problem(message);
59          // the list may have changed since it was filled
60          RefreshMechanics();
61          return;
62      }
63      DialogResult = DialogResult.OK;
64  }
65
66  private void MoveBooking()
67  {
68      string message = AppointmentData.Move(appointmentID, ChosenStart());
69      if (message ≠ "")
70      {
71          Msg.Problem(message);
72          return;
73      }
74      DialogResult = DialogResult.OK;
75  }
76
77  // A combo box with nothing chosen gives null; this turns that into 0.
78  private int ChosenID(ComboBox combo)
79  {
80      if (combo.SelectedValue == null)
81      {
82          return 0;
83      }
84      return (int)combo.SelectedValue;
85  }
```

**1 The Book button, or the Move button**

Lines 1 to 7

If this form is moving a booking, MoveBooking does the work, and return ends the method here.

**2 Check that a customer was chosen**

Lines 11 to 16

One problem at a time, as in chapter 6: show the first problem, put the cursor in that box with Focus, and stop with return.

**3 Check that a service was chosen**

Lines 17 to 22

The same shape.

**4 Check that a mechanic was chosen**

Lines 23 to 28

The same shape. The start time needs no check, because the list only offers times in 15-minute steps.

**5 Check that the bike was described**

Lines 31 to 36

Trim() removes spaces at either end, so a box with only spaces in it counts as empty.

**6 Pack what was chosen into one object**

Lines 38 to 50

An Appointment object holds the whole booking in one place. Empty notes become null, which the database stores as nothing. Session.StaffID is whoever signed in.

**7 Send it, and show the answer**

Lines 54 to 64

The database does the checks and answers in words. A sentence is shown with Msg.Problem, and the mechanic list is refreshed because it may have changed. An empty answer means it worked, and DialogResult.OK closes the form and tells the diary.

**8 A move sends only the new time**

Lines 66 to 75

The same form used for a move. Only the booking number and the new start time go to the database.

**9 Turn nothing chosen into 0**

Lines 78 to 85

A combo box with nothing chosen gives null. This small method turns that into 0, so the checks above can compare with a number.

**Why you type it** The message check is the whole point. The database decides; the form only shows what it said and refreshes the list.

# Put the whole file behind the form, and build.

Here is the whole file: the four pieces you have just read, together. The yellow lines are the two parts you type. Everything else is pasted in one go, over the top of what the designer made.

**Type this one** **Forms/frmAppointmentEdit.cs: the whole file** C#

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
using System;
using System.Collections.Generic;
using System.Data;
using System.Drawing;
using System.Windows.Forms;
using BikeShop.Data;
using BikeShop.Models;

namespace BikeShop.Forms
{
    public partial class frmAppointmentEdit : Form
    {
        // 0 = a new booking; otherwise the booking being moved
        private readonly int appointmentID;
        // the date and time the form opens on
        private readonly DateTime startWith;
        // "Aoife McCann · Puncture repair with Conor" when moving
        private readonly string movingWhat;

        // New booking on a chosen day
        public frmAppointmentEdit(DateTime day)
        {
            InitializeComponent();
            startWith = day.Date.AddHours(9);
        }

        // Move an existing booking: only the date and time can change
        public frmAppointmentEdit(int id, DateTime currentStart, string summary)
        {
            InitializeComponent();
            appointmentID = id;
            startWith = currentStart;
            movingWhat = summary;
        }

        #region Load
        private void frmAppointmentEdit_Load(object sender, EventArgs e)
        {
            // The two text boxes stop typing at the length the database column
            // holds.
            txtBike.MaxLength = 80;
            txtNotes.MaxLength = 200;

            FillTimes();
            dtpDate.Value = startWith.Date;
            cboStart.SelectedItem = startWith.ToString("HH:mm");

            // Set the two names first, then the DataSource, so the combo box
            // knows what to show and what to keep.
            cboCustomer.DisplayMember = "FullName";
            cboCustomer.ValueMember = "CustomerID";
            cboCustomer.DataSource = CustomerData.Search("");

```

```
            // "Puncture repair · 30 min · £15.00"
            cboService.DisplayMember = "Display";
            cboService.ValueMember = "ServiceTypeID";
            cboService.DataSource = ServiceTypeData.GetAll();

            if (appointmentID ≠ 0)
            {
                Text = "Move a booking";
                lblMoving.Text = "Moving: " + movingWhat;
                btnBook.Text = "Move";
                cboCustomer.Enabled = false;
                cboService.Enabled = false;
                cboMechanic.Enabled = false;
                txtBike.Enabled = false;
                txtNotes.Enabled = false;
                lblMechanicNote.Text = "The same mechanic keeps the job.";
            }
            else
            {
                lblMoving.Text = "";
                RefreshMechanics();
            }
        }

        // The shop opens 08:30 and the last slot starts 17:15: a time every 15
        // minutes.
        private void FillTimes()
        {
            cboStart.Items.Clear();
            TimeSpan time = new TimeSpan(8, 30, 0);
            while (time ⩽ new TimeSpan(17, 15, 0))
            {
                cboStart.Items.Add(time.ToString(@"hh\:mm"));
                time = time.Add(TimeSpan.FromMinutes(15));
            }
        }
        #endregion

        // The chosen date and time as one value, which is what the database
        // stores.
        private DateTime ChosenStart()
        {
            string time = "09:00";
            if (cboStart.SelectedItem ≠ null)
            {
                time = (string)cboStart.SelectedItem;
            }
            return dtpDate.Value.Date + TimeSpan.Parse(time);
        }

        #region FreeMechanics
        // Asks the database which mechanics are free for this service at this
        // time.
        // Runs again whenever the date, the time or the service changes.
        private void RefreshMechanics()
        {
            if (appointmentID ≠ 0 || cboService.SelectedValue == null
                || cboStart.SelectedItem == null)
```

```
            {
                return;
            }

            cboMechanic.DisplayMember = "FullName";
            cboMechanic.ValueMember = "StaffID";
            cboMechanic.DataSource = AppointmentData.GetFreeMechanics(
                ChosenStart(), (int)cboService.SelectedValue);

            int free = cboMechanic.Items.Count;
            if (free == 0)
            {
                lblMechanicNote.Text =
                    "No mechanic is free then. Try another time.";
            }
            else if (free == 1)
            {
                lblMechanicNote.Text = "1 mechanic is free.";
            }
            else
            {
                lblMechanicNote.Text = free + " mechanics are free.";
            }
            btnBook.Enabled = free > 0;
        }

        private void cboService_SelectedIndexChanged(object sender, EventArgs e)
        {
            RefreshMechanics();
        }

        private void dtpDate_ValueChanged(object sender, EventArgs e)
        {
            RefreshMechanics();
        }

        private void cboStart_SelectedIndexChanged(object sender, EventArgs e)
        {
            RefreshMechanics();
        }
        #endregion

        #region Book
        private void btnBook_Click(object sender, EventArgs e)
        {
            if (appointmentID != 0)
            {
                MoveBooking();
                return;
            }

            // Check what was chosen and typed, one problem at a time: show the
            // first and put the cursor there.
            if (ChosenID(cboCustomer) == 0)
            {
                Msg.Problem("Choose the customer.");
                cboCustomer.Focus();
                return;
```

```
170             }
171             if (ChosenID(cboService) == 0)
172             {
173                 Msg.Problem("Choose the service.");
174                 cboService.Focus();
175                 return;
176             }
177             if (ChosenID(cboMechanic) == 0)
178             {
179                 Msg.Problem("Choose a mechanic.");
180                 cboMechanic.Focus();
181                 return;
182             }
183             // The start time needs no check: the list only offers times in
184             // 15-minute steps.
185             if (txtBike.Text.Trim() == "")
186             {
187                 Msg.Problem("Type a description of the bike.");
188                 txtBike.Focus();
189                 return;
190             }
191
192             Appointment booking = new Appointment();
193             booking.CustomerID = ChosenID(cboCustomer);
194             booking.ServiceTypeID = ChosenID(cboService);
195             booking.MechanicID = ChosenID(cboMechanic);
196             booking.StartTime = ChosenStart();
197             booking.BikeDescription = txtBike.Text.Trim();
198             booking.Notes = txtNotes.Text.Trim();
199             if (booking.Notes == "")
200             {
201                 // nothing typed: store a NULL, not an empty piece of text
202                 booking.Notes = null;
203             }
204             booking.BookedByStaffID = Session.StaffID;
205
206             // The database checks the clash, the hours and the day, and answers
207             // in words.
208             string message = AppointmentData.Add(booking);
209             if (message ≠ "")
210             {
211                 // e.g. "That mechanic already has a booking in that time."
212                 Msg.Problem(message);
213                 // the list may have changed since it was filled
214                 RefreshMechanics();
215                 return;
216             }
217             DialogResult = DialogResult.OK;
218         }
219
220         private void MoveBooking()
221         {
222             string message = AppointmentData.Move(appointmentID, ChosenStart());
223             if (message ≠ "")
224             {
225                 Msg.Problem(message);
226                 return;
227             }
```

```
228             DialogResult = DialogResult.OK;
229         }
230
231         // A combo box with nothing chosen gives null; this turns that into 0.
232         private int ChosenID(ComboBox combo)
233         {
234             if (combo.SelectedValue == null)
235             {
236                 return 0;
237             }
238             return (int)combo.SelectedValue;
239         }
240         #endregion
241     }
242 }
```

**1 The usings**

Lines 1 to 7

The tools this form needs: Windows Forms for the window itself, System.Data for the tables that fill the lists, and your own Data and Models folders.

**2 The form's class, and three facts it remembers**

Lines 9 to 18

partial means the designer keeps the other half of this class, the controls you dragged, in its own file. readonly means set once, when the form is made. appointmentID is 0 for a new booking, and any other number means a booking that is being moved.

**3 The two ways the form is opened**

Lines 21 to 34

A constructor runs when the form is made. The diary uses the first one for a new booking on a day, and the second one to move a booking it already has.

**4 When the form loads**

Lines 36 to 90

This runs once, just before the form appears. It fills the three lists and, when a booking is being moved, locks everything except the date and the time. FillTimes makes the list of start times.

**5 The chosen date and time as one value**

Lines 94 to 102

The date from the date picker and the time from the list are joined into one value, which is what the database stores.

**6 Which mechanics are free: the first part you type**

Lines 104 to 152

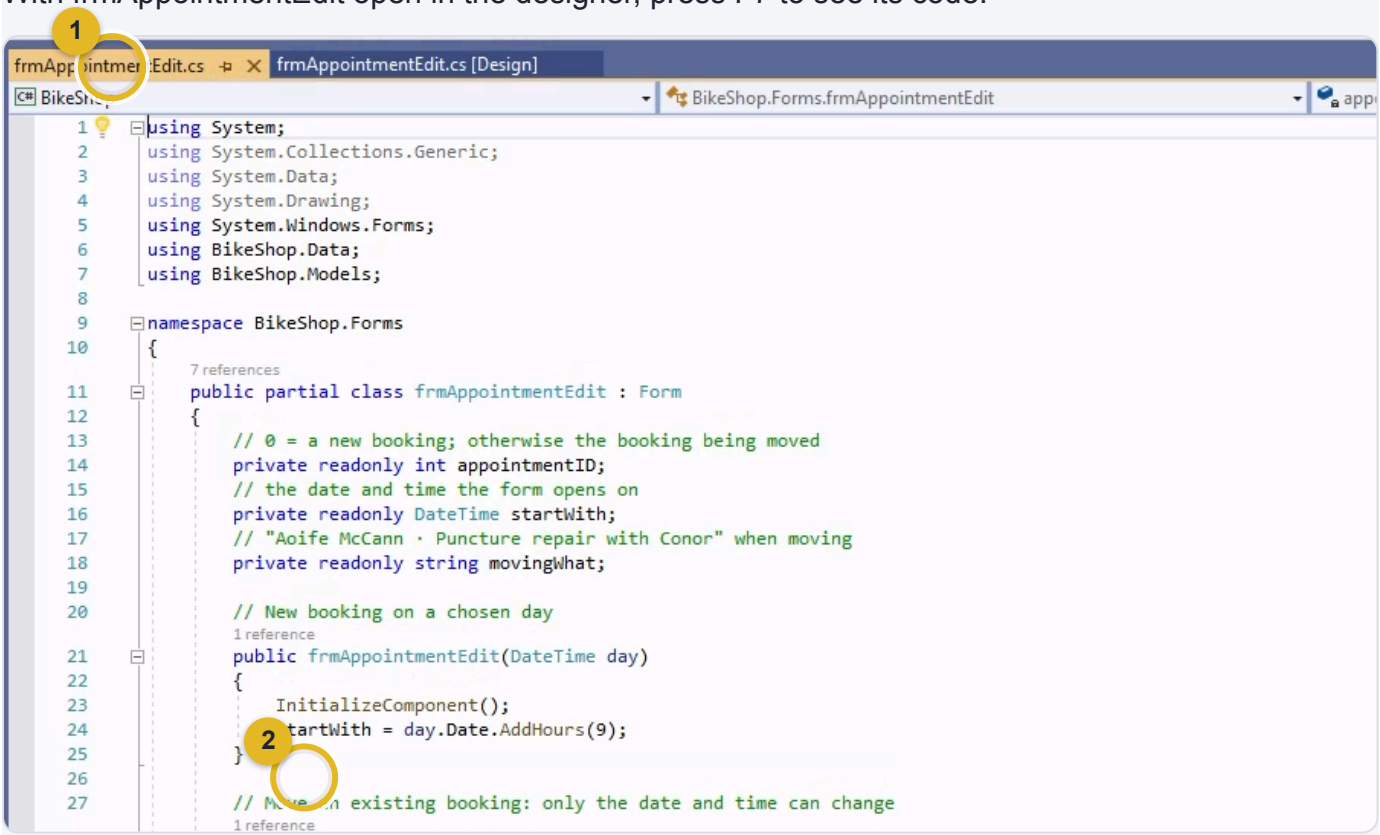
Whenever the date, the time or the service changes, the form asks the database who is free, and switches the Book button off when nobody is.

**7 The Book button: the second part you type**

Lines 154 to 242

The checks, then one call to the database. The database decides, and the form shows what it said. MoveBooking does the same for a move, and ChosenID turns nothing chosen into 0.

With frmAppointmentEdit open in the designer, press F7 to see its code.



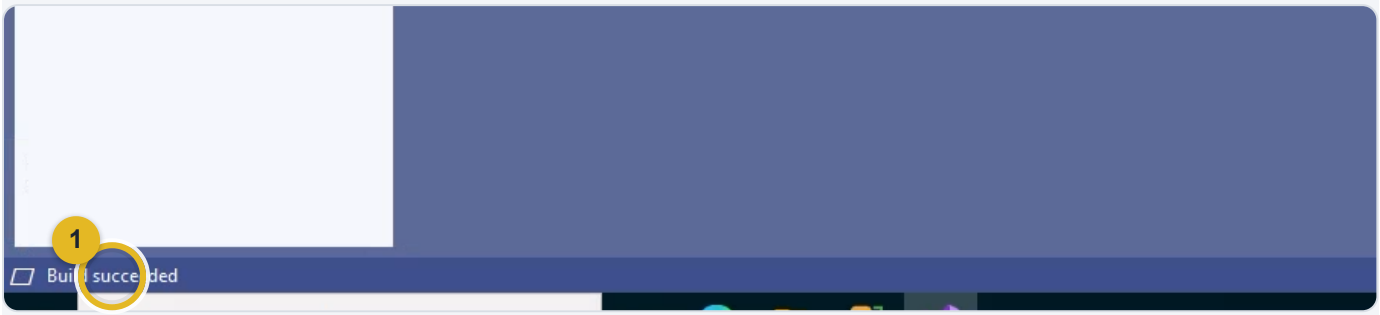

F7 shows the code behind the form that is open in the designer. Shift and F7 go back to the designer.

1. The code tab, frmAppointmentEdit.cs, is in front.

2. Paste inside the class, after the last method the designer made.

**You should see** The code tab frmAppointmentEdit.cs comes to the front.

Press Ctrl and A together to select everything in the code tab. Press the Copy button under this step, and paste over it. Then click on each line that says YOU TYPE THIS PART, type the yellow lines in its place, and delete the marker line. Press F6 to build.




Build succeeded, at the bottom left. Anything else means an error list to read.

1 Build succeeded. Press F6 to build at any time.

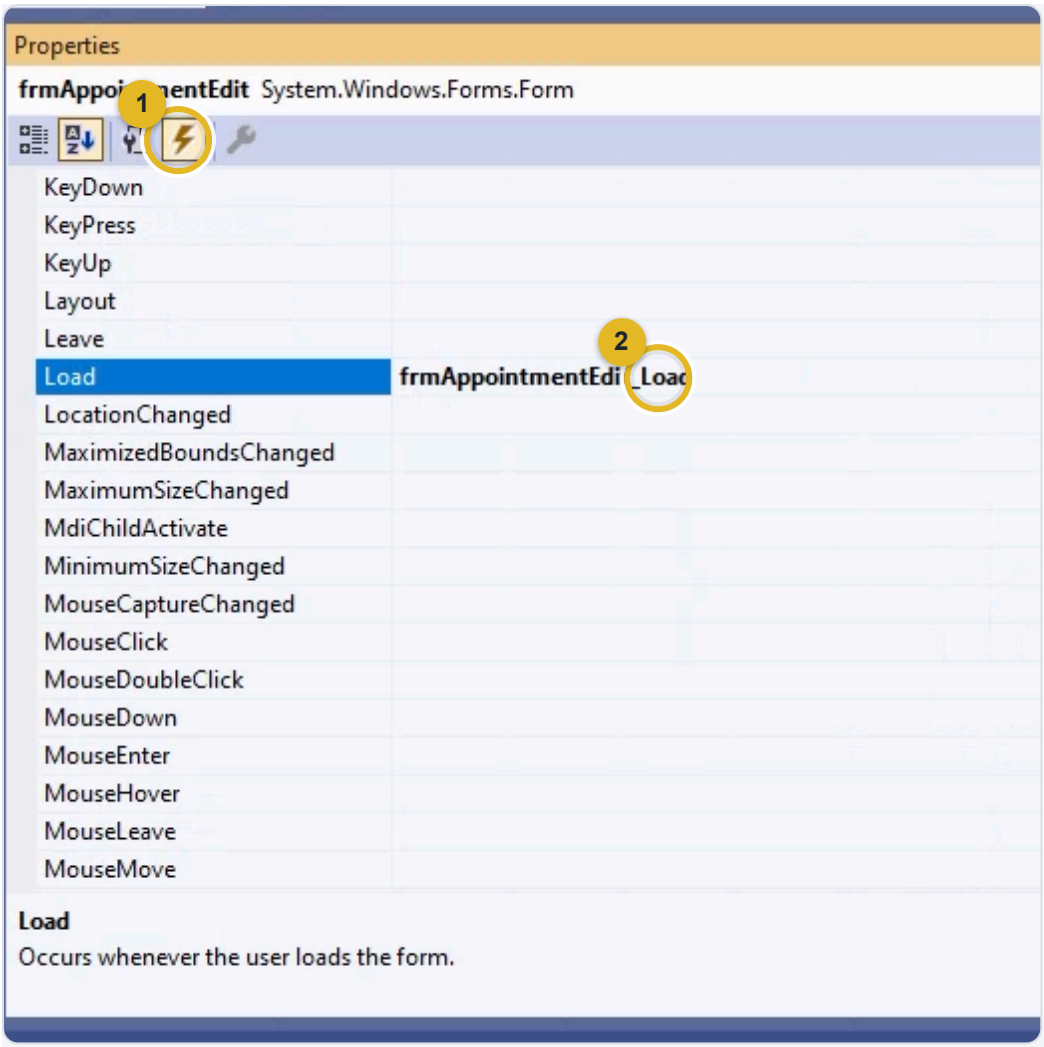
> **You should see** Build succeeded. The form is not reachable from a menu yet. The diary opens it, in stage 6.

**IF IT GOES WRONG** frmAppointmentEdit_Load or btnBook_Click is underlined: no suitable method found

The designer did not make that event. Go back to the designer, click the form (or the Book button), open the Properties window, press the lightning bolt, find Load (or Click) and double-click the empty box beside it.

**IF IT GOES WRONG** 'The name Session does not exist'

Session.StaffID comes from the sign-in in chapter 5. Add that class if you skipped it.




The lightning bolt shows the events, in alphabetical order. Double-click the empty box beside one to make its method.

1. Press the lightning bolt.
2. Double-click the empty box beside the event named in the table. Load is part of the way down, just after Leave.

STAGE 6 OF 8 · THE DIARY: ONE DAY AT A GLANCE · SCREEN 1 OF 9

# Read the stored procedure that gives one whole day, then add it to your script.

The diary shows one day. The counter staff pick a day, see every booking with names and never numbers, and press New booking, Move, Mark done or Cancel booking. One stored procedure gives the whole day in one trip to the database, with the names joined in.

Copy this one **usp_Appointment_GetByDay: every booking on one day, with names** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
CREATE OR ALTER PROCEDURE usp_Appointment_GetByDay
    @Day DATE
AS
BEGIN
    SELECT a.AppointmentID,
           a.StartTime,
           a.EndTime,
           a.MechanicID,
           m.FirstName + ' ' + m.LastName AS Mechanic,
           c.FirstName + ' ' + c.LastName AS Customer,
           st.Name                        AS Service,
           a.BikeDescription,
           a.Status,
           a.Notes
    FROM Appointment a
    JOIN Staff       m  ON m.StaffID        = a.MechanicID
    JOIN Customer    c  ON c.CustomerID     = a.CustomerID
    JOIN ServiceType st ON st.ServiceTypeID = a.ServiceTypeID
    WHERE CAST(a.StartTime AS DATE) = @Day
    ORDER BY a.StartTime, m.FirstName;
END
GO
```

**1 The name and the one parameter**

Lines 1 to 2

This one takes a date and gives back a list of rows.

**2 The start**

Lines 3 to 4

AS and BEGIN mark where the work starts. No @Message: a list is never refused.

**3 The columns the diary shows**

Lines 5 to 14

a, m, c and st are short names for the four tables, given in the FROM and JOIN lines below. Each + ' ' + joins a first name and a last name into one column, named with AS. The number columns come too, hidden later, so the program knows which row is which.

**4 Join the names in**

Lines 15 to 18

Each JOIN pairs a booking row with the matching row in another table: the mechanic's Staff row, the customer's row and the service's row. ON says which columns must match. Three JOINs turn three numbers into three names.

**5 Only that day, in time order**

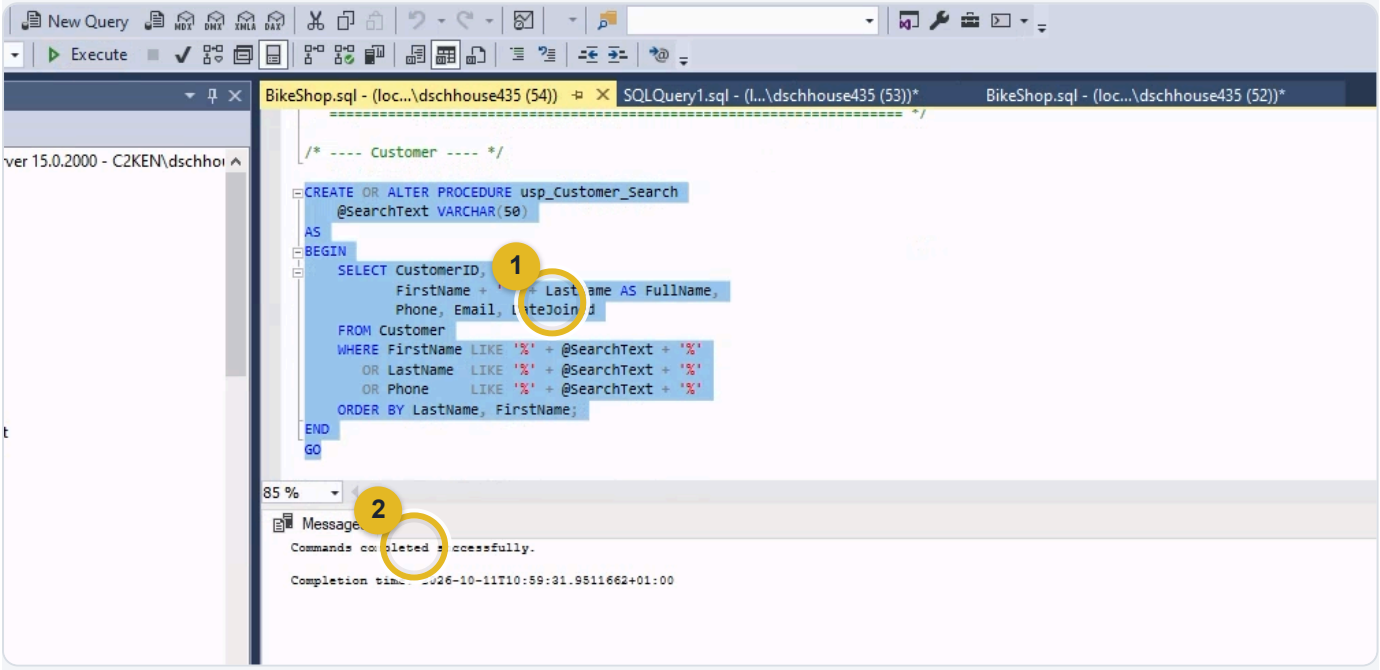
Lines 19 to 22

CAST(… AS DATE) throws away the clock part, so every booking on that date is kept whatever its time. ORDER BY sorts the rows by start time. GO ends the batch, ready for the next stored procedure.

Press the Copy button under this step and paste it on the empty line above the line that prints BikeShop is ready.

**You should see** The stored procedure sits just above the line that prints BikeShop is ready.

Select the lines you just added, from CREATE OR ALTER PROCEDURE down to GO, and press F5.




Select the lines you just added, then press F5. Only the highlighted lines run, so the test bookings you have made stay in the database.

1. Drag from CREATE OR ALTER PROCEDURE down to GO, so the whole block is highlighted.

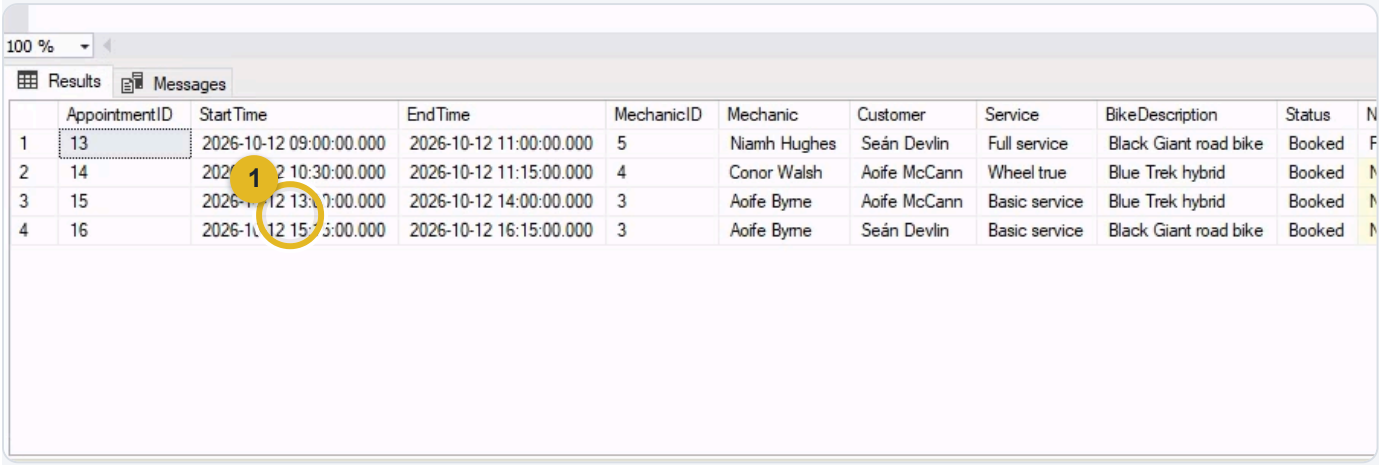
2. The Messages pane says: Commands completed successfully.

**You should see** The Messages pane says: Commands completed successfully. Your test bookings are still there, because only the selected lines ran.

**IF IT GOES WRONG** The Messages pane shows (6 rows affected) and BikeShop is ready

Nothing was selected, so the whole script ran and the sample data was put back. Your test bookings are gone. Run the stage 2 test again to make a new one, and write down its new number.

Click New Query and run EXEC usp_Appointment_GetByDay @Day = '2026-10-12', with your test day in place of that date.




The whole day in one trip to the database, with names, not numbers.

1 Two bookings from the sample data and your two test bookings, in time order.

> **You should see** Four rows: two bookings that the sample data put on that day, and your two test bookings, each with the mechanic's name, the customer's name and the service's name.

## Add the class that draws the time line.

The diary has a picture of each mechanic's day above the grid, so that the gaps are obvious. It is one class that draws itself. It makes no decisions, so you copy it whole. After one build it appears in the Toolbox like any other control.

Copy this one **Controls/DiaryTimeline.cs** C#

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
using System;
using System.Collections.Generic;
using System.Data;
using System.Drawing;
using System.Drawing.Text;
using System.Windows.Forms;

namespace BikeShop.Controls
{
    // Raised when the user clicks a booking block.
    public class BookingPickedEventArgs : EventArgs
    {
        public int AppointmentID { get; set; }
    }

    // A picture of one day: a row for each mechanic, 08:30 to 17:30 across,
    // each booking as a block.
    // Optional in the coursework: the grid already shows everything.  Drag it
    // from the Toolbox after one build.
    public class DiaryTimeline : Panel
    {
        private const int LabelWidth = 110;
        private const int RowHeight = 44;
        private const int HeaderHeight = 22;
        private static readonly TimeSpan DayStart = new TimeSpan(8, 30, 0);
        private static readonly TimeSpan DayEnd = new TimeSpan(17, 30, 0);

        private class Block
        {
            public Rectangle Area;
            public int AppointmentID;
            public string Text;
            public string Status;
        }

        private readonly List<string> lanes = new List<string>();
        private readonly List<Block> blocks = new List<Block>();
        private int highlighted;
        private DateTime day;

        public event EventHandler<BookingPickedEventArgs> BookingPicked;

        public DiaryTimeline()
        {
            DoubleBuffered = true;        // no flicker when it repaints
            BackColor = Color.White;
            BorderStyle = BorderStyle.FixedSingle;
            Cursor = Cursors.Hand;
        }

        #region ShowDay
        // Takes the same table the grid shows (usp_Appointment_GetByDay) and
        // lays the blocks out.
```

```
54          public void ShowDay(DateTime whichDay, DataTable bookings)
55          {
56              day = whichDay.Date;
57              lanes.Clear();
58              blocks.Clear();
59
60              foreach (DataRow row in bookings.Rows)
61              {
62                  string mechanic = Convert.ToString(row["Mechanic"]);
63                  if (!lanes.Contains(mechanic)) lanes.Add(mechanic);
64              }
65              lanes.Sort();
66
67              foreach (DataRow row in bookings.Rows)
68              {
69                  string status = Convert.ToString(row["Status"]);
70                  // a cancelled booking frees the time, so it is not drawn
71                  if (status == "Cancelled") continue;
72
73                  Block block = new Block();
74                  block.AppointmentID = Convert.ToInt32(row["AppointmentID"]);
75                  block.Status = status;
76                  block.Text = Convert.ToString(row["Customer"]) + " · "
77                      + Convert.ToString(row["Service"]);
78                  int lane = lanes.IndexOf(Convert.ToString(row["Mechanic"]));
79                  int x1 = XFor(Convert.ToDateTime(row["StartTime"]).TimeOfDay);
80                  int x2 = XFor(Convert.ToDateTime(row["EndTime"]).TimeOfDay);
81                  block.Area = new Rectangle(x1, HeaderHeight + lane * RowHeight
82                      + 6, Math.Max(x2 - x1 - 2, 6), RowHeight - 12);
83                  blocks.Add(block);
84              }
85              Invalidate();
86          }
87
88          public void Highlight(int appointmentID)
89          {
90              highlighted = appointmentID;
91              Invalidate();
92          }
93          #endregion
94
95          // Where a time falls across the control: 08:30 at the left edge of the
96          // lanes, 17:30 at the right.
97          private int XFor(TimeSpan time)
98          {
99              double minutesIn = (time - DayStart).TotalMinutes;
100             double minutesAll = (DayEnd - DayStart).TotalMinutes;
101             double fraction = minutesIn / minutesAll;
102             fraction = Math.Max(0, Math.Min(1, fraction));
103             return LabelWidth
104                 + (int)Math.Round(fraction * (Width - LabelWidth - 10));
105         }
106
107         #region Paint
108         protected override void OnPaint(PaintEventArgs e)
109         {
110             base.OnPaint(e);
111             Graphics g = e.Graphics;
```

```
            g.TextRenderingHint = TextRenderingHint.ClearTypeGridFit;

            using (Font small = new Font(Font.FontFamily, 8f))
            using (Pen hairline = new Pen(Color.FromArgb(220, 224, 230)))
            using (Brush grey = new SolidBrush(Color.FromArgb(92, 103, 116)))
            {
                // hour lines and labels
                for (TimeSpan t = new TimeSpan(9, 0, 0); t < DayEnd;
                    t = t.Add(TimeSpan.FromHours(1)))
                {
                    int x = XFor(t);
                    g.DrawLine(hairline, x, HeaderHeight, x, Height);
                    g.DrawString(t.ToString(@"hh\:mm"), small, grey, x - 16, 4);
                }
                // mechanic names
                for (int i = 0; i < lanes.Count; i++)
                {
                    int y = HeaderHeight + i * RowHeight;
                    g.DrawLine(hairline, 0, y, Width, y);
                    g.DrawString(lanes[i], Font, Brushes.Black, 8, y + 12);
                }
                if (lanes.Count == 0)
                {
                    string when = day.ToString("dddd d MMMM");
                    string none = "No bookings on " + when;
                    int y = HeaderHeight + 14;
                    g.DrawString(none, Font, grey, LabelWidth, y);
                }
            }

            foreach (Block block in blocks)
            {
                Color fill = Color.FromArgb(79, 163, 227);
                if (block.Status == "Done")
                {
                    fill = Color.FromArgb(63, 185, 80);
                }
                using (Brush brush = new SolidBrush(fill))
                    g.FillRectangle(brush, block.Area);
                if (block.AppointmentID == highlighted)
                    using (Pen ring = new Pen(Color.FromArgb(228, 184, 36), 3))
                        g.DrawRectangle(ring, block.Area);

                Rectangle textArea = block.Area;
                textArea.Inflate(-4, -2);
                TextFormatFlags flags = TextFormatFlags.EndEllipsis
                    | TextFormatFlags.VerticalCenter
                    | TextFormatFlags.NoPadding;
                TextRenderer.DrawText(g, block.Text, Font, textArea,
                    Color.White, flags);
            }
        }

        protected override void OnResize(EventArgs eventargs)
        {
            base.OnResize(eventargs);
            Invalidate();
        }
```

```
170         #endregion
171
172         #region Click
173         protected override void OnMouseClick(MouseEventArgs e)
174         {
175             base.OnMouseClick(e);
176             foreach (Block block in blocks)
177             {
178                 if (block.Area.Contains(e.Location))
179                 {
180                     BookingPickedEventArgs args = new BookingPickedEventArgs();
181                     args.AppointmentID = block.AppointmentID;
182                     if (BookingPicked ≠ null)
183                     {
184                         BookingPicked(this, args);
185                     }
186                     return;
187                 }
188             }
189         }
190         #endregion
191     }
192 }
```

**1 The usings**

Lines 1 to 6

System.Drawing holds the pens, brushes and colours. System.Windows.Forms holds Panel, which this control is built on.

**2 The message a click sends**

Lines 8 to 14

When the user clicks a block, the control raises an event with the booking's number in it. This small class carries that number to the diary form.

**3 The control and its fixed numbers**

Lines 20 to 26

The words : Panel mean it starts as an ordinary blank panel. const and static readonly are numbers that never change: the width of the names column, the height of a row, and the opening hours.

**4 One drawn block**

Lines 28 to 34

Each booking becomes a Block: where it is drawn (Area), which booking it is, what text it shows, and its status.

**5 What the control remembers**

Lines 36 to 41

lanes is the list of mechanics' names, one row each. blocks is every drawn booking. highlighted is the booking to ring. BookingPicked is the event the diary form listens to.

**6 Set up when it is made**

Lines 43 to 49

DoubleBuffered stops flicker when it repaints. A white background, a thin border, and a hand cursor so that it looks clickable.

**7 ShowDay: one row per mechanic**

Lines 51 to 65

The diary hands over the same table the grid shows. First pass: collect each mechanic's name once, then sort the names.

**8 Second pass: a block for each booking**

Lines 67 to 86

Cancelled rows are skipped, because they free the time. XFor turns a time into a position across the control. The rectangle is placed in that mechanic's row. Invalidate asks Windows to repaint.

**9 Highlight: which block gets the gold ring**

Lines 88 to 93

The diary calls this when the selected row changes.

**10 XFor: a time becomes a position**

Lines 97 to 105

08:30 is none of the way across and 17:30 is all of the way. The fraction is kept between 0 and 1, then scaled to the width left of the names column.

**11 OnPaint: draw everything**

Lines 107 to 112

override replaces the panel's own painting. Windows calls OnPaint whenever the control needs drawing. g is the pen-and-paper object you draw with.

**12 Three drawing tools, tidied away after**

Lines 114 to 117

using (…) makes a font, a pen and a brush, and throws them away at the closing brace.

**13 Hour lines and labels**

Lines 119 to 125

A faint line and a time label for each hour from 09:00.

**14 A line and a name for each mechanic**

Lines 127 to 140

One row per mechanic, with the name at the left. When the day is empty, a sentence says so.

**15 Draw each block**

Lines 142 to 163

Blue for Booked, green for Done. The highlighted block gets a gold ring. The text is drawn inside, cut short with … when the block is narrow.

**16 Repaint when the size changes**

Lines 165 to 170

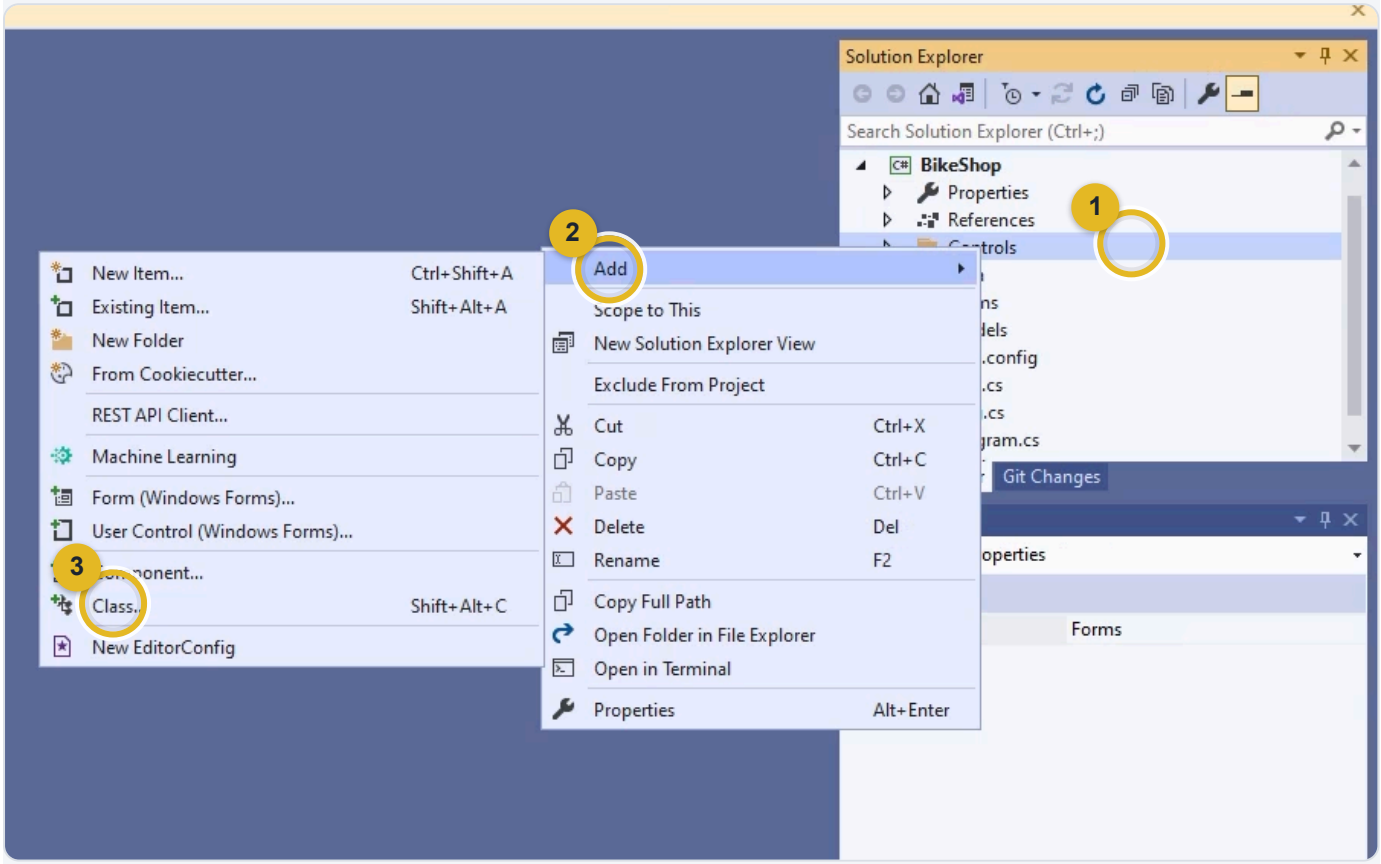
The positions depend on the width, so a resize means a repaint.

**17 A click finds the block under the mouse**

Lines 172 to 192

OnMouseClick runs when the user clicks. Area.Contains asks whether the click landed inside a block. If it did, the event is raised with that booking's number, and the diary form picks the row.

In Solution Explorer in Visual Studio 2019, right-click the project name BikeShop, point at Add, and click New Folder. Type Controls as its name. Then right-click the new Controls folder, point at Add, click Class…, type DiaryTimeline.cs as the name and press Add.

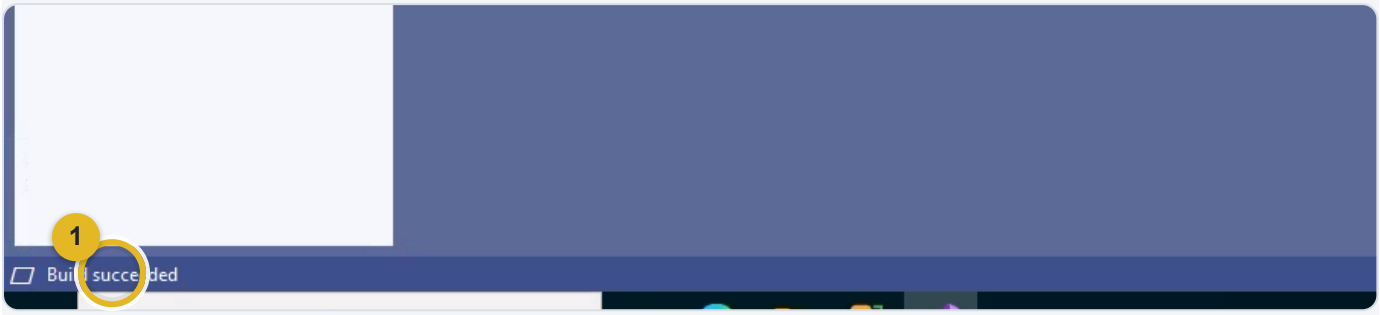



Adding a class to the Controls folder. Taken in the finished project, so Solution Explorer lists more files than yours.

1. Right-click the Controls folder in Solution Explorer.
2. Point at Add.
3. Click Class…

**You should see** A new file opens with an empty class in it.

Click in the new file, press Ctrl and A together to select everything, press the Copy button under this step, and paste over it. Then press F6 to build.



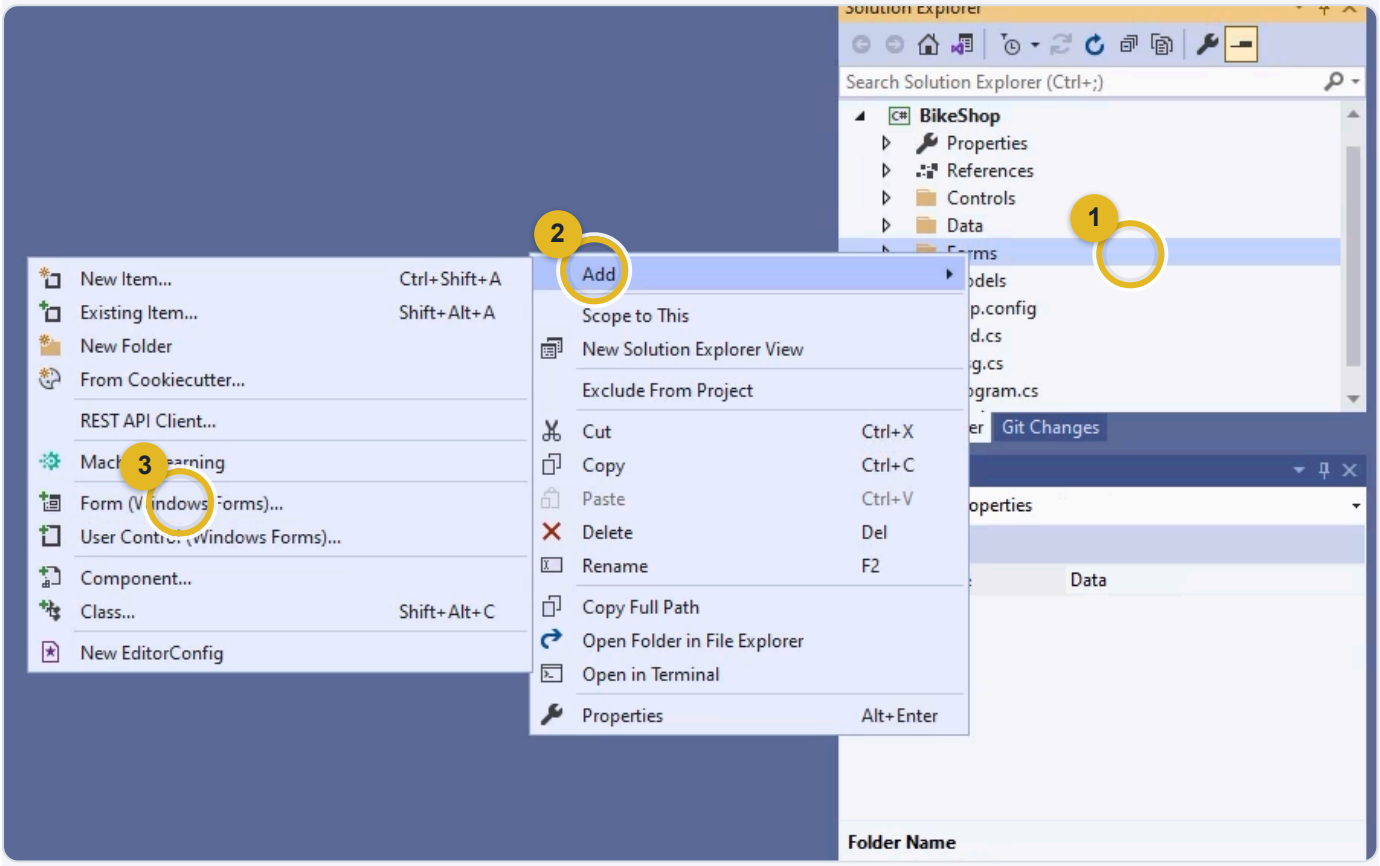

Build succeeded, at the bottom left. Anything else means an error list to read.

1. Build succeeded. Press F6 to build at any time.

> **You should see** Build succeeded. DiaryTimeline now appears at the top of the Toolbox.

# Add the diary form.

In Solution Explorer, right-click the Forms folder, point at Add, and click Form (Windows Forms)…. Type frmDiary as the name and press Add.




Adding a form to the Forms folder. Taken in the finished project, so Solution Explorer lists more files than yours.

1. Right-click the Forms folder.
2. Point at Add.
3. Click Form (Windows Forms)…, type the name, press Add.

> **You should see** An empty form opens in the designer.

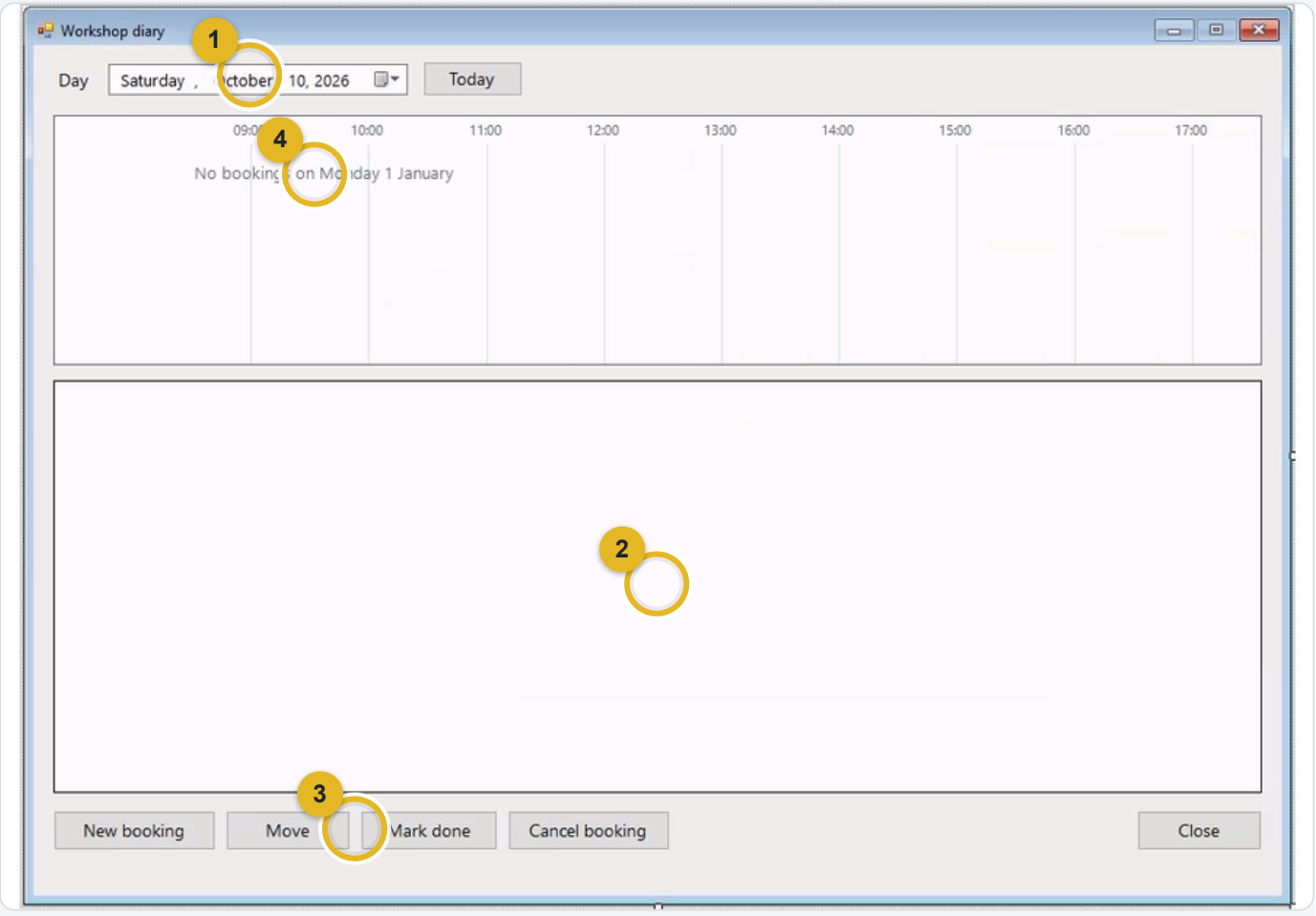
# Drag the diary's controls from the Toolbox and set their properties.

Set the form's own properties first, then work down the table. The grid gets the same settings you gave the customers grid in chapter 5. DiaryTimeline is your own control from the last build. Drag it from the top of the Toolbox like any other, name it timeline, and size it 968 by 200.

One day of repair bookings: a picture of each mechanic's day on top, the same bookings as a grid below.

**Click the form itself, then set these in the Properties window**

| | | | |
|---|---|---|---|
| Text | `Workshop diary` | Size | `1000, 680` |
| FormBorderStyle | `FixedSingle` | StartPosition | `CenterParent` |
| MaximizeBox | `False` | | |

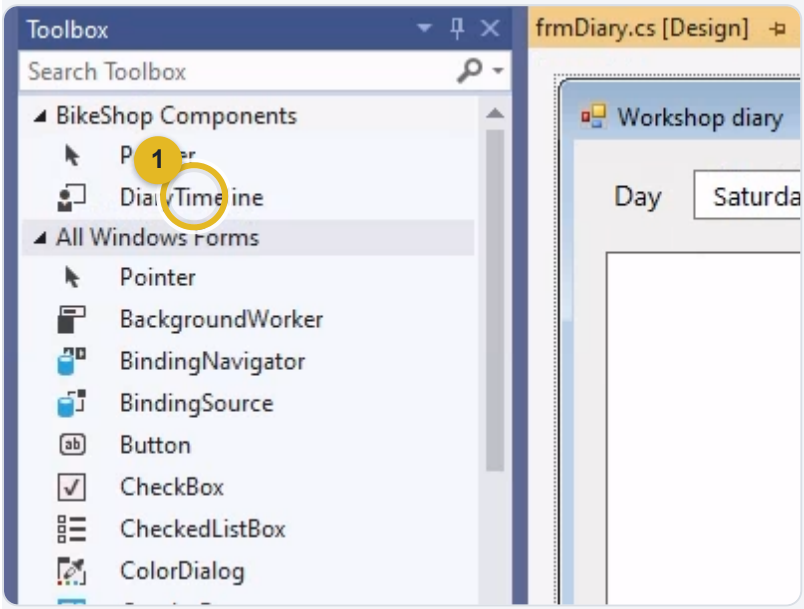



frmDiary in the designer with every control in place, the time line included. The grid fills the middle.

1. Day, the date picker and Today along the top.
2. The grid.
3. New booking, Move, Mark done, Cancel booking and Close along the bottom.
4. The time line. In the designer it says there are no bookings, because there is no day yet.

| DRAG THIS FROM THE TOOLBOX | NAME IT (THE NAME PROPERTY) | SET THESE IN THE PROPERTIES WINDOW | THEN |
|---|---|---|---|
| Label | `lblDay` | `Text = Day` | |
| DateTimePicker | `dtpDay` | `Format = Long`<br>Pick a day; the diary reloads. | Double-click it. Visual Studio makes `dtpDay_ValueChanged` for you; the code goes inside it. |
| Button | `btnToday` | `Text = Today` | Double-click it. Visual Studio makes `btnToday_Click` for you; the code goes inside it. |
| DiaryTimeline (your own, after one build) | `timeline` | nothing else<br>Your own drawn time line (chapter 7 stage 6). It appears at the top of the Toolbox after one build. | Events tab (the lightning bolt) → BookingPicked: type `timeline_BookingPicked` beside it and press Enter. |
| DataGridView | `dgvBookings` | `the grid settings (chapter 6)`<br>The day's bookings with names, not ID numbers. | Events tab (the lightning bolt) → SelectionChanged: type `dgvBookings_SelectionChanged` beside it and press Enter. |
| Button | `btnNew` | `Text = New booking` | Double-click it. Visual Studio makes `btnNew_Click` for you; the code goes inside it. |
| Button | `btnMove` | `Text = Move`<br>`Enabled = False`<br>Only for a booking that is still "Booked". | Double-click it. Visual Studio makes `btnMove_Click` for you; the code goes inside it. |
| Button | `btnDone` | `Text = Mark done`<br>`Enabled = False` | Double-click it. Visual Studio makes `btnDone_Click` for you; the code goes inside it. |
| Button | `btnCancelBooking` | `Text = Cancel booking`<br>`Enabled = False`<br>A status change, not a delete: the booking stays in the diary as Cancelled. | Double-click it. Visual Studio makes `btnCancelBooking_Click` for you; the code goes inside it. |

| DRAG THIS FROM THE TOOLBOX | NAME IT (THE NAME PROPERTY) | SET THESE IN THE PROPERTIES WINDOW | THEN |
|---|---|---|---|
| Button | `btnClose` | `Text = Close` | Double-click it. Visual Studio makes `btnClose_Click` for you; the code goes inside it. |




After one build, your own control appears at the top of the Toolbox.

1. Drag DiaryTimeline on to frmDiary, above the grid.

# Read how the diary loads a day.

Three pieces of code go behind the diary. Read them one at a time. Then you put the whole file in at once, and type the yellow lines. This first piece loads a day.

Read this one **Forms/frmDiary.cs: loading the day** C#

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  // One trip to the database gives the day's bookings; the grid and the
2  // timeline both show them.
3  private void LoadDay()
4  {
5      DataTable bookings = AppointmentData.GetByDay(dtpDay.Value);
6      dgvBookings.DataSource = bookings;
7      Grid.Tidy(dgvBookings, "AppointmentID", "MechanicID");
8      timeline.ShowDay(dtpDay.Value, bookings);
9  }
```

**1 One method loads the day**

Lines 3 to 4

It is called when the diary opens, when the day changes, and after every change to a booking.

**2 Fetch the day and show it in the grid**

Lines 5 to 7

One trip to the database. The table becomes the grid's DataSource. Grid.Tidy is your helper from chapter 5: it hides the two number columns and tidies the headings.

**3 Give the same table to the drawn time line**

Lines 8 to 9

The picture and the grid show the same bookings, because they are given the same table.

STAGE 6 OF 8 · THE DIARY: ONE DAY AT A GLANCE · SCREEN 6 OF 9

## Read how the diary decides which buttons are allowed.

Move, Mark done and Cancel booking only make sense for a booking that is still open. This piece switches them on and off as the user clicks around the grid.

Read this one **Forms/frmDiary.cs: which buttons are allowed** C#

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  // Move, Mark done and Cancel only apply to a booking that is still
2  // "Booked".
3  private void dgvBookings_SelectionChanged(object sender, EventArgs e)
4  {
5      bool picked = dgvBookings.SelectedRows.Count > 0;
6      bool booked = picked
7          && Grid.SelectedText(dgvBookings, "Status") == "Booked";
8      btnMove.Enabled = booked;
9      btnDone.Enabled = booked;
10     btnCancelBooking.Enabled = booked;
11
12     if (picked)
13     {
14         timeline.Highlight(Grid.SelectedID(dgvBookings,
15             "AppointmentID"));
16     }
17     else
18     {
19         timeline.Highlight(0);
20     }
21 }
22
23 // A click on a block in the timeline picks the same booking in the
24 // grid.
25 private void timeline_BookingPicked(object sender,
26     BikeShop.Controls.BookingPickedEventArgs e)
27 {
28     foreach (DataGridViewRow row in dgvBookings.Rows)
29     {
30         int id = Convert.ToInt32(row.Cells["AppointmentID"].Value);
31         if (id == e.AppointmentID)
32         {
33             row.Selected = true;
34             dgvBookings.CurrentCell = row.Cells["Customer"];
35             return;
36         }
37     }
38 }
```

**1 Switch the three buttons on or off**

Lines 3 to 10

bool is a true-or-false value. picked asks: is a row selected? booked asks: is it selected, and is its Status still Booked? The three buttons are switched on only when booked is true.

**2 Ring the chosen booking on the time line**

Lines 12 to 21

Highlight draws a gold ring round the block with that number. 0 means no ring.

**3 A click on the picture picks the row in the grid**

Lines 25 to 38

foreach looks at every row of the grid. When a row's AppointmentID matches the block that was clicked, that row becomes the selected row, and return stops the search.

STAGE 6 OF 8 · THE DIARY: ONE DAY AT A GLANCE · SCREEN 7 OF 9

# Read the code behind the diary’s buttons.

The four buttons. New booking and Move open the booking form. Mark done and Cancel booking ask a question first, then tell the database. The yellow lines are the ones you type: the question before a cancel is a sentence the user reads, so it is yours to write.

Read this one **Forms/frmDiary.cs: the four buttons** C#

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  private void btnNew_Click(object sender, EventArgs e)
2  {
3      frmAppointmentEdit booking = new frmAppointmentEdit(dtpDay.Value);
4      if (booking.ShowDialog(this) == DialogResult.OK)
5      {
6          LoadDay();
7      }
8  }
9
10 private void btnMove_Click(object sender, EventArgs e)
11 {
12     int appointmentID = Grid.SelectedID(dgvBookings, "AppointmentID");
13     DataGridViewRow row = dgvBookings.SelectedRows[0];
14     DateTime start = Convert.ToDateTime(row.Cells["StartTime"].Value);
15     string summary = Grid.SelectedText(dgvBookings, "Customer") + " · "
16         + Grid.SelectedText(dgvBookings, "Service") + " with "
17         + Grid.SelectedText(dgvBookings, "Mechanic");
18
19     frmAppointmentEdit move = new frmAppointmentEdit(appointmentID,
20         start, summary);
21     if (move.ShowDialog(this) == DialogResult.OK)
22     {
23         LoadDay();
24     }
25 }
26
27 private void btnDone_Click(object sender, EventArgs e)
28 {
29     if (!Msg.Ask("Mark the booking for "
30         + Grid.SelectedText(dgvBookings, "Customer") + " as done?"))
31     {
32         return;
33     }
34
35     string message =
36         AppointmentData.MarkDone(Grid.SelectedID(dgvBookings,
37         "AppointmentID"));
38     if (message != "")
39     {
40         Msg.Problem(message);
41         return;
42     }
43     LoadDay();
44 }
45
46 private void btnCancelBooking_Click(object sender, EventArgs e)
47 {
48     string who = Grid.SelectedText(dgvBookings, "Customer");
49     if (!Msg.Ask("Cancel " + who
50         + "'s booking? It stays in the diary marked Cancelled."))
51     {
52         return;
53     }
```

```
54
55      string message =
56          AppointmentData.Cancel(Grid.SelectedID(dgvBookings,
57          "AppointmentID"));
58      if (message ≠ "")
59      {
60          Msg.Problem(message);
61          return;
62      }
63      LoadDay();
64  }
```

**1 New booking opens the form for this day**

Lines 1 to 8

ShowDialog opens the booking form and waits. It answers OK only when the database accepted the booking. Then the day is loaded again, so the new booking appears.

**2 Move opens the same form with the booking in it**

Lines 10 to 25

The booking number, the start time and a one-line summary come from the selected row of the grid. Grid.SelectedID and Grid.SelectedText are your helpers from chapter 5.

**3 Mark done asks first, then tells the database**

Lines 27 to 44

Msg.Ask shows a yes-or-no question with the customer's name in it. No means return. The database answers in words. An empty answer means done, so the day is loaded again.

**4 Cancel booking asks a question you type**

Lines 46 to 64

The question names the customer and says that the booking stays in the diary, marked Cancelled. The rest is the same shape as Mark done.

**Why you type it** The question before a cancel is typed by you, because it is a sentence the user reads: it names the customer and says what will happen.

# Put the whole file behind the diary, and open it from the main menu.

Here is the whole file: the three pieces together, with the small methods that open and close the diary and change the day. The yellow lines are the part you type.

**Type this one** **Forms/frmDiary.cs: the whole file** C#

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
using System;
using System.Collections.Generic;
using System.Data;
using System.Drawing;
using System.Windows.Forms;
using BikeShop.Data;
using BikeShop.Models;

namespace BikeShop.Forms
{
    public partial class frmDiary : Form
    {
        public frmDiary()
        {
            InitializeComponent();
        }

        private void frmDiary_Load(object sender, EventArgs e)
        {
            dtpDay.Value = DateTime.Today;
            LoadDay();
        }

        #region LoadDay
        // One trip to the database gives the day's bookings; the grid and the
        // timeline both show them.
        private void LoadDay()
        {
            DataTable bookings = AppointmentData.GetByDay(dtpDay.Value);
            dgvBookings.DataSource = bookings;
            Grid.Tidy(dgvBookings, "AppointmentID", "MechanicID");
            timeline.ShowDay(dtpDay.Value, bookings);
        }
        #endregion

        private void dtpDay_ValueChanged(object sender, EventArgs e)
        {
            LoadDay();
        }

        private void btnToday_Click(object sender, EventArgs e)
        {
            dtpDay.Value = DateTime.Today;
        }

        #region Selection
        // Move, Mark done and Cancel only apply to a booking that is still
        // "Booked".
        private void dgvBookings_SelectionChanged(object sender, EventArgs e)
        {
            bool picked = dgvBookings.SelectedRows.Count > 0;
            bool booked = picked
                && Grid.SelectedText(dgvBookings, "Status") == "Booked";
```

```
            btnMove.Enabled = booked;
            btnDone.Enabled = booked;
            btnCancelBooking.Enabled = booked;

            if (picked)
            {
                timeline.Highlight(Grid.SelectedID(dgvBookings,
                    "AppointmentID"));
            }
            else
            {
                timeline.Highlight(0);
            }
        }

        // A click on a block in the timeline picks the same booking in the
        // grid.
        private void timeline_BookingPicked(object sender,
            BikeShop.Controls.BookingPickedEventArgs e)
        {
            foreach (DataGridViewRow row in dgvBookings.Rows)
            {
                int id = Convert.ToInt32(row.Cells["AppointmentID"].Value);
                if (id == e.AppointmentID)
                {
                    row.Selected = true;
                    dgvBookings.CurrentCell = row.Cells["Customer"];
                    return;
                }
            }
        }
        #endregion

        #region Actions
        private void btnNew_Click(object sender, EventArgs e)
        {
            frmAppointmentEdit booking = new frmAppointmentEdit(dtpDay.Value);
            if (booking.ShowDialog(this) == DialogResult.OK)
            {
                LoadDay();
            }
        }

        private void btnMove_Click(object sender, EventArgs e)
        {
            int appointmentID = Grid.SelectedID(dgvBookings, "AppointmentID");
            DataGridViewRow row = dgvBookings.SelectedRows[0];
            DateTime start = Convert.ToDateTime(row.Cells["StartTime"].Value);
            string summary = Grid.SelectedText(dgvBookings, "Customer") + " · "
                + Grid.SelectedText(dgvBookings, "Service") + " with "
                + Grid.SelectedText(dgvBookings, "Mechanic");

            frmAppointmentEdit move = new frmAppointmentEdit(appointmentID,
                start, summary);
            if (move.ShowDialog(this) == DialogResult.OK)
            {
                LoadDay();
            }
```

```
112         }
113
114         private void btnDone_Click(object sender, EventArgs e)
115         {
116             if (!Msg.Ask("Mark the booking for "
117                 + Grid.SelectedText(dgvBookings, "Customer") + " as done?"))
118             {
119                 return;
120             }
121
122             string message =
123                 AppointmentData.MarkDone(Grid.SelectedID(dgvBookings,
124                 "AppointmentID"));
125             if (message ≠ "")
126             {
127                 Msg.Problem(message);
128                 return;
129             }
130             LoadDay();
131         }
132
133         private void btnCancelBooking_Click(object sender, EventArgs e)
134         {
135             string who = Grid.SelectedText(dgvBookings, "Customer");
136             if (!Msg.Ask("Cancel " + who
137                 + "'s booking? It stays in the diary marked Cancelled."))
138             {
139                 return;
140             }
141
142             string message =
143                 AppointmentData.Cancel(Grid.SelectedID(dgvBookings,
144                 "AppointmentID"));
145             if (message ≠ "")
146             {
147                 Msg.Problem(message);
148                 return;
149             }
150             LoadDay();
151         }
152         #endregion
153
154         private void btnClose_Click(object sender, EventArgs e)
155         {
156             Close();
157         }
158     }
159 }
```

**1 The usings**

Lines 1 to 7

The same tools as the booking form.

**2 The diary's class**

Lines 9 to 12

partial again: the designer keeps the controls you dragged in its own file.

**3 Made, then loaded**

Lines 13 to 22

InitializeComponent builds the controls. When the form loads, the date picker is set to today, and today is loaded.

**4 Loading the day**

Lines 24 to 34

One trip to the database. The same table goes to the grid and to the drawn time line.

**5 A new day, or Today**

Lines 36 to 44

Changing the date loads that day. The Today button sets the date picker back to today, which loads today.

**6 Which buttons are allowed**

Lines 46 to 85

The three buttons are on only for a booking that is still Booked. A click on the picture picks that row in the grid.

**7 The four buttons: the part you type**

Lines 87 to 152

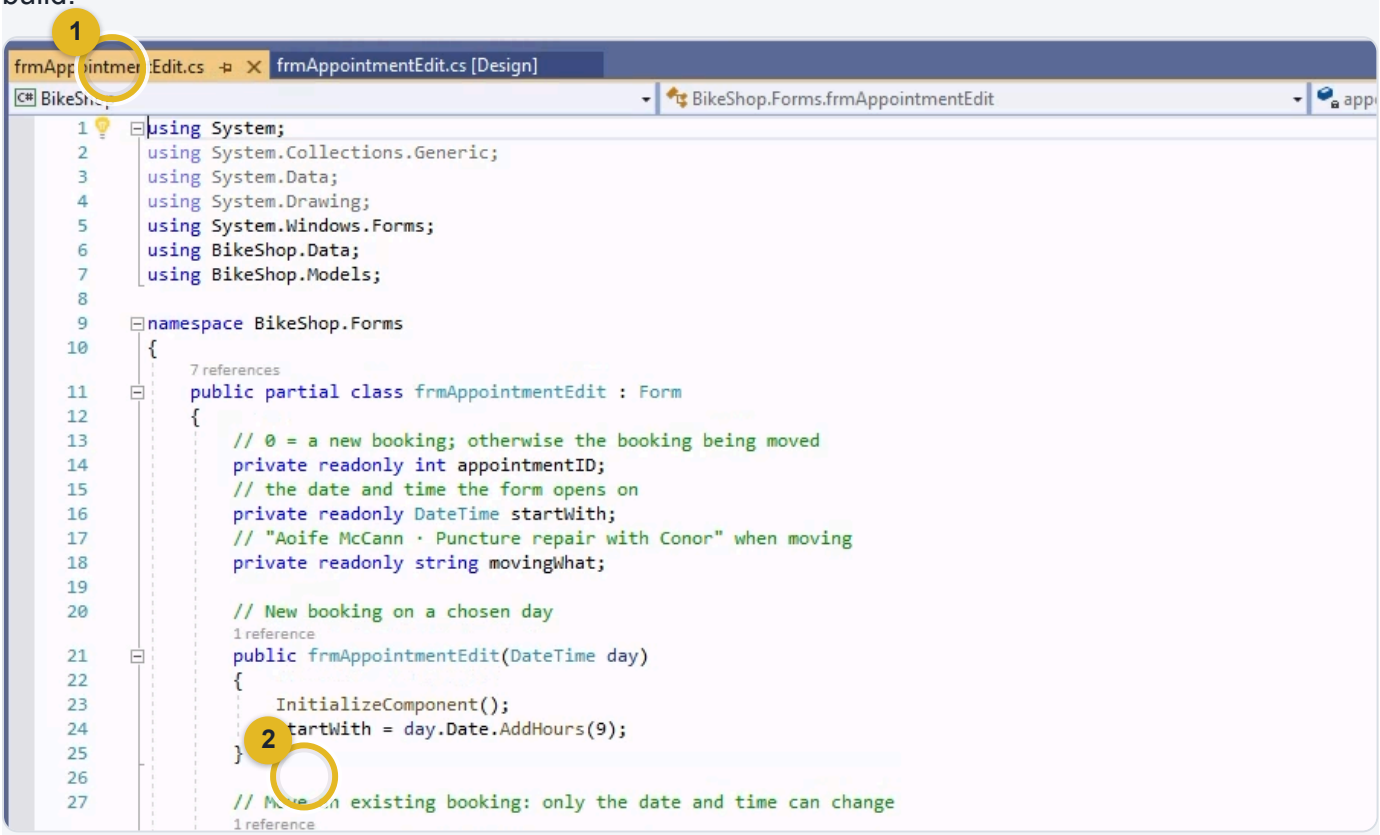
New booking and Move open the booking form. Mark done and Cancel booking ask first, then tell the database.

**8 Close**

Lines 154 to 159

Close shuts the diary. The two closing braces end the class and the namespace.

With frmDiary open in the designer, press F7. Press Ctrl and A together to select everything in the code tab, press the Copy button under this step, and paste over it. Then click on the line that says YOU TYPE THIS PART, type the yellow lines in its place, and delete the marker line. Press F6 to build.



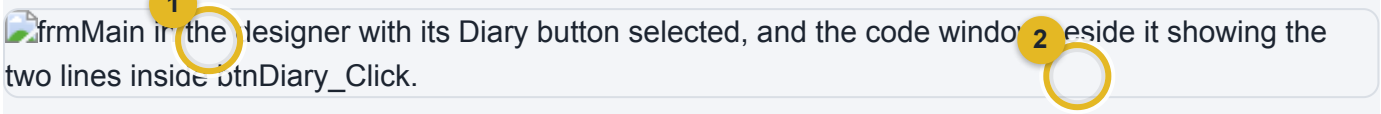

F7 shows the code behind the form that is open in the designer. Shift and F7 go back to the designer.

1. The code tab, frmAppointmentEdit.cs, is in front.

2. Paste inside the class, after the last method the designer made.

> **You should see** Build succeeded.

Open frmMain in the designer and double-click its Diary button. Inside the method that appears, type the same two lines you typed for the Customers button in chapter 6, with frmDiary in place of frmCustomers: frmDiary diary = new frmDiary(); and then diary.ShowDialog(this);



frmMain in the designer with its Diary button selected, and the code window beside it showing the two lines inside btnDiary_Click.

The Diary button on your main menu opens the diary.

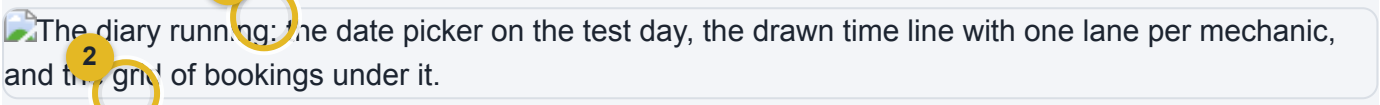
1. Double-click the Diary button to make its Click method.
2. Type the two lines inside it.

> **You should see** Build succeeded. Press F5 to run the program, sign in, and press Diary.

# Run the program and make a booking, while Siobhán books the same slot.

This screen plays out the story from stage 1. You are at the diary. Siobhán is at the counter, which here is your test window in SQL Server Management Studio (SSMS) from stage 2. Both of you want 13:00 on the test day.

In the running diary, pick your test day in the date picker.



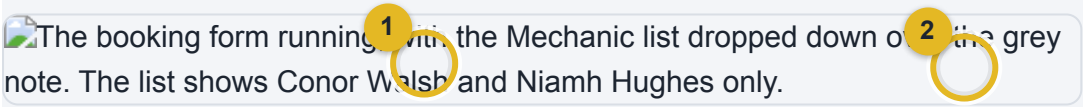

The diary on your test day: two bookings from the sample data and your two test bookings.

1. Pick your test day here.

2. New booking opens the booking form.

> **You should see** Four bookings in the grid, with names in every column: two from the sample data, and your two from stages 2 and 3. The time line above the grid shows the same four as blocks.

Press New booking. Choose Hannah Boyle as the customer, Puncture repair as the service, and 13:00 as the start time. Then open the Mechanic list.




Only the free mechanics are offered. The open list covers the grey note. Close the list to read it.

1. Aoife Byrne is not in the list.

2. The grey note is under the open list. It says: 2 mechanics are free.

> **You should see** Conor Walsh and Niamh Hughes are in the list. Aoife Byrne is not, because of your 13:00 booking. Close the list again: the grey note under it says 2 mechanics are free.

Change the service to Basic service. Leave the form open, with no mechanic chosen yet.

> **You should see** The Mechanic list fills again, because a longer job could clash with a different booking. It still offers Conor and Niamh.

Now be Siobhán. Go to your SSMS test window from stage 2. On the EXEC line, change @CustomerID = 1 to @CustomerID = 3, change @MechanicID = 3 to @MechanicID = 4, and change 'Blue Trek hybrid' to 'Grey Carrera bike'. Press F5.

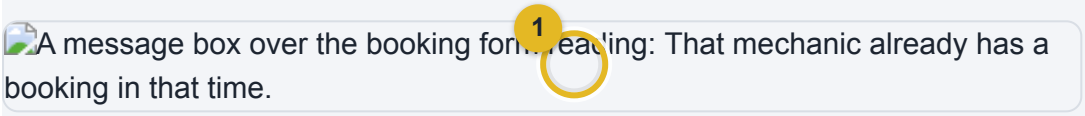
The test window with the line @CustomerID = 1, @MechanicID = 3, @ServiceTypeID = 6 and the number 3 highlighted.

The customer and the mechanic are numbers on the EXEC line. Customer 1 is Aoife McCann and 3 is Ciara Quinn. Mechanic 3 is Aoife Byrne and 4 is Conor Walsh.

1. Change the 1 after @CustomerID to 3, and the 3 after @MechanicID to 4.

> **You should see** Booked. New appointment number 17 at 13:00 on your test day. Ciara Quinn is booked in with Conor Walsh while your form is still open.

Back in your program, choose Conor in the Mechanic list and press Book.



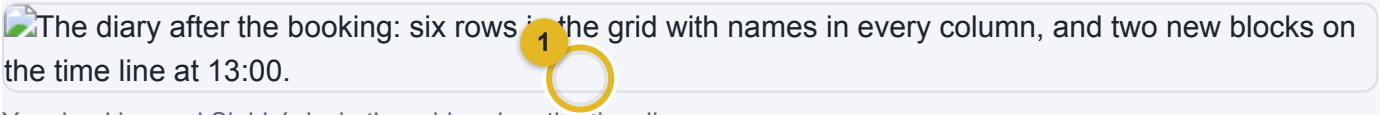
A message box over the booking form reading: That mechanic already has a booking in that time.

The refusal, as the program shows it. The words came from the stored procedure.

1. The sentence from @Message, shown by Msg.Problem.

> **You should see** A message box says: That mechanic already has a booking in that time. The list was filled before Siobhán booked, so the list missed it, and the database caught it. Press OK. The list fills again, Conor is gone, and the grey note says 1 mechanic is free.

Choose Niamh and press Book.



The diary after the booking: six rows in the grid with names in every column, and two new blocks on the time line at 13:00.

Your booking and Siobhán's, in the grid and on the time line.

1. Names in every column. No numbers.

> **You should see** The form closes. Hannah Boyle's booking is in the grid under Niamh Hughes, and Ciara Quinn's is there under Conor Walsh, with names in every column. Six rows in all.

**IF IT GOES WRONG** The grid shows AppointmentID and MechanicID columns

Grid.Tidy is not being called, or the column names you passed to it are spelled differently from the stored procedure's.

**IF IT GOES WRONG** 'Choose a mechanic.' when you press Book after the refusal

The list filled again and lost your choice. Pick Niamh in the list, then press Book.

**IF IT GOES WRONG** Nothing happens when you press Diary

The Click event is not wired. In the designer, click the button, open the Properties window, press the lightning bolt and check that Click has a method beside it.

**IF IT GOES WRONG** The grid is empty on your test day

Look at the date picker. The diary opens on today. Pick the test day.

# Read why a cancelled booking is not deleted, and type the stored procedure that cancels one.

A cancelled booking is not deleted. The shop needs to know that it happened: who cancelled, how often, and which customers never turn up. So a cancel is a change of status. The clash test ignores cancelled rows, which frees the slot. This stored procedure is short enough to type whole.

**Type this one** **usp_Appointment_Cancel: mark a booking Cancelled** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
CREATE OR ALTER PROCEDURE usp_Appointment_Cancel
    @AppointmentID INT,
    @Message VARCHAR(200) OUTPUT
AS
BEGIN
    SET @Message = '';
    -- A cancel is a status change.  The row stays: it is the shop's history.
    IF NOT EXISTS (SELECT * FROM Appointment
        WHERE AppointmentID = @AppointmentID AND Status = 'Booked')
    BEGIN
        SET @Message = 'Only an open booking can be cancelled.';
        RETURN;
    END

    UPDATE Appointment
    SET Status = 'Cancelled'
    WHERE AppointmentID = @AppointmentID AND Status = 'Booked';
END
GO
```

**1 The name, and what goes in and out**

Lines 1 to 3

Only the booking number goes in. @Message comes back out.

**2 Start with a blank message**

Lines 4 to 6

AS and BEGIN mark where the work starts. The first job is to make @Message empty. An empty message means it worked. Every refusal below puts a sentence into @Message instead, so the program always knows which it was.

**3 Refuse unless there is an open booking with this number**

Lines 8 to 13

If there is no row with this number whose status is Booked, the booking was already cancelled, already done, or never existed. Refuse with a sentence.

**4 Change the status and keep the row**

Lines 15 to 19

UPDATE changes the status to Cancelled. The row stays in the table. The shop keeps its history, and the clash test ignores the row from now on. GO ends the batch, ready for the next stored procedure.

**Why you type it** Typed whole. The check and the UPDATE both say the same two things: the right booking, and only if it is still open.

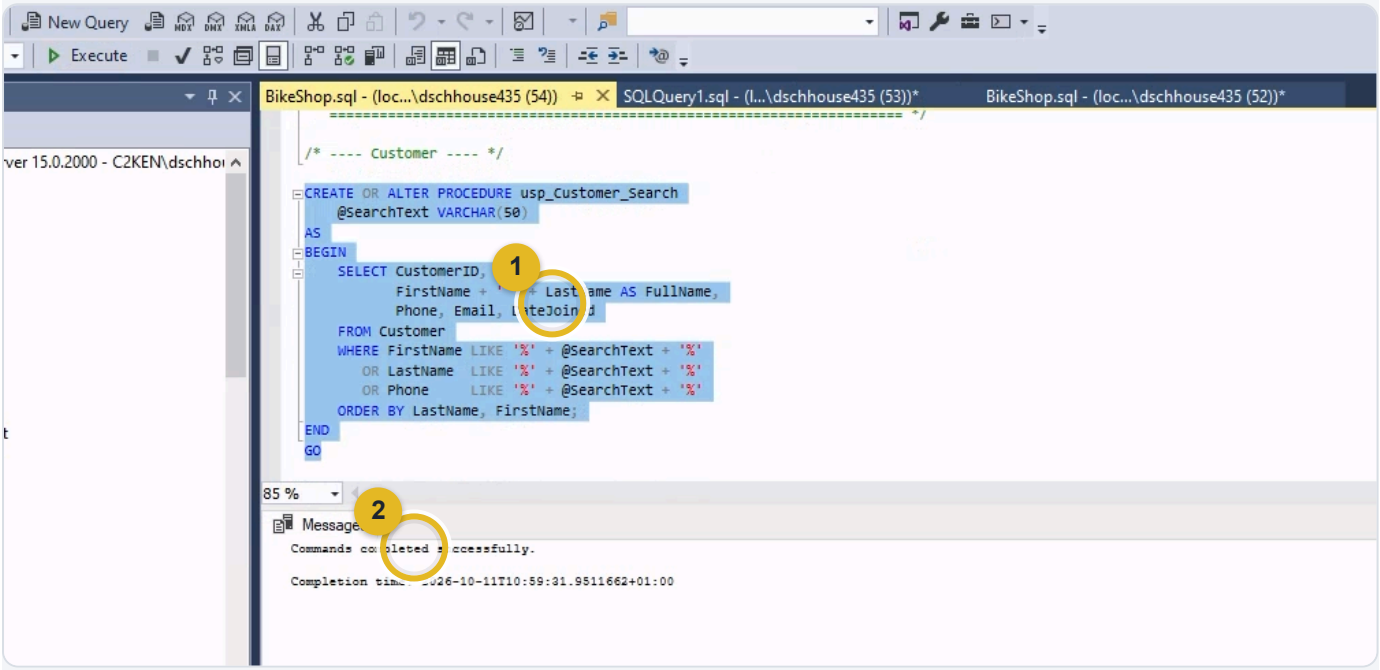
Click on the empty line above the line that prints BikeShop is ready and type the usp_Appointment_Cancel stored procedure exactly as shown, GO included.

> **You should see** Your typed lines match the block, and there is no red wavy underline anywhere in them.

**IF IT GOES WRONG** A red wavy line under a word

Rest the mouse on it to read the message, and compare your typed lines with the block, character by character.

Select the lines you typed, from CREATE OR ALTER PROCEDURE down to GO, and press F5.




Select the lines you just added, then press F5. Only the highlighted lines run, so the test bookings you have made stay in the database.

1. Drag from CREATE OR ALTER PROCEDURE down to GO, so the whole block is highlighted.

2. The Messages pane says: Commands completed successfully.

> **You should see** The Messages pane says: Commands completed successfully.

# Copy its twin, which marks a booking done.

Marking a booking done is the same shape with a different word. You have just typed its twin, so this one is copied.

**Copy this one** **usp_Appointment_MarkDone: mark a booking Done** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  CREATE OR ALTER PROCEDURE usp_Appointment_MarkDone
2      @AppointmentID INT,
3      @Message VARCHAR(200) OUTPUT
4  AS
5  BEGIN
6      SET @Message = '';
7      IF NOT EXISTS (SELECT * FROM Appointment
8          WHERE AppointmentID = @AppointmentID AND Status = 'Booked')
9      BEGIN
10         SET @Message = 'Only an open booking can be marked done.';
11         RETURN;
12     END
13
14     UPDATE Appointment
15     SET Status = 'Done'
16     WHERE AppointmentID = @AppointmentID AND Status = 'Booked';
17 END
18 GO
```

**1 The name, and what goes in and out**

Lines 1 to 3

The same two parameters as Cancel.

**2 Start with a blank message**

Lines 4 to 6

AS and BEGIN mark where the work starts. The first job is to make @Message empty. An empty message means it worked. Every refusal below puts a sentence into @Message instead, so the program always knows which it was.

**3 Refuse unless there is an open booking with this number**

Lines 7 to 12

The same check as Cancel, with its own sentence.

**4 Change the status to Done**

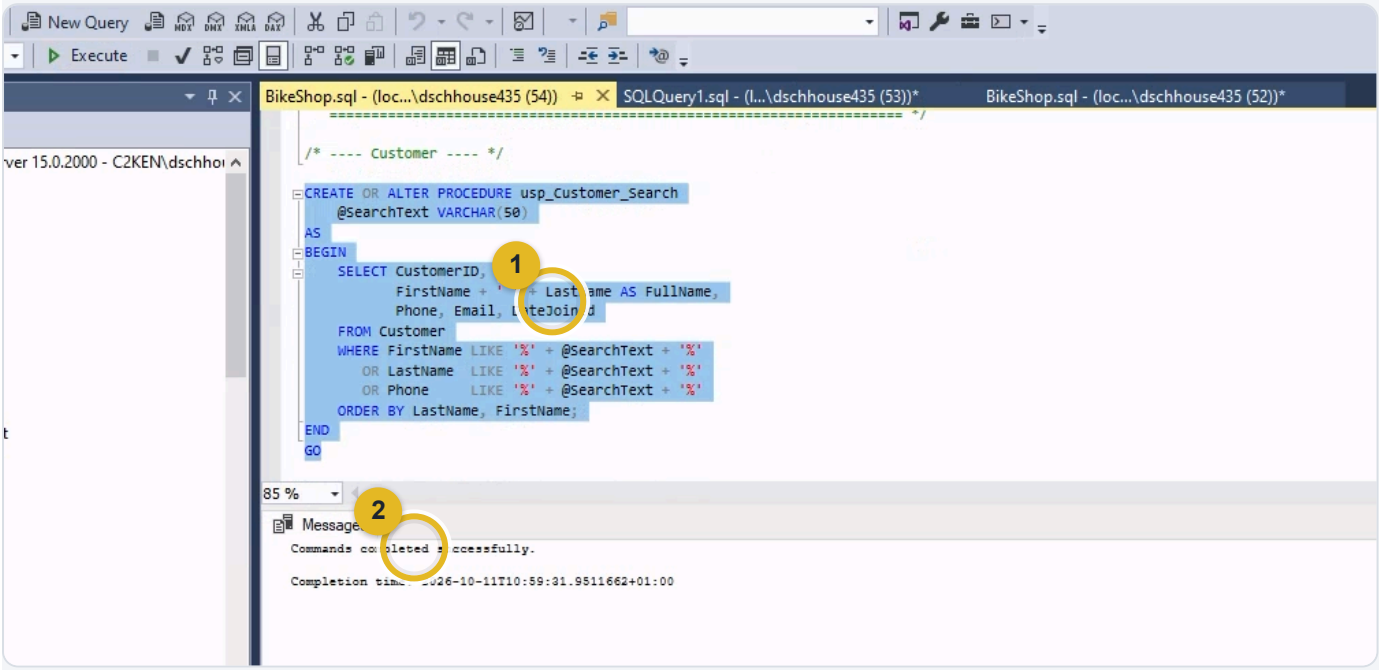
Lines 14 to 18

The row stays, with its status changed. GO ends the batch, ready for the next stored procedure.

Press the Copy button under this step and paste it on the empty line above the line that prints BikeShop is ready.

> **You should see** The stored procedure sits just above the line that prints BikeShop is ready.

Select the lines you just added, from CREATE OR ALTER PROCEDURE down to GO, and press F5.




Select the lines you just added, then press F5. Only the highlighted lines run, so the test bookings you have made stay in the database.

1. Drag from CREATE OR ALTER PROCEDURE down to GO, so the whole block is highlighted.

2. The Messages pane says: Commands completed successfully.

> **You should see** The Messages pane says: Commands completed successfully.

# Cancel your two test bookings and look at the day.

**Copy this one** **A test that cancels one booking by its number, then shows the day** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
-- Cancel one booking by its number, then look at the day: the row is still
-- there, marked Cancelled.
DECLARE @ID INT, @Day DATE, @Message VARCHAR(200);
-- ⟵ put the number that the first section printed for you here
SET @ID = 0;

EXEC usp_Appointment_Cancel @AppointmentID = @ID, @Message = @Message OUTPUT;
IF @Message = ''
BEGIN
    PRINT 'Cancelled number ' + CAST(@ID AS VARCHAR)
        + '. It is still in the diary, marked Cancelled.';
END
ELSE BEGIN PRINT @Message; END

SET @Day = DATEADD(DAY, 1, CAST(GETDATE() AS DATE));
IF DATENAME(WEEKDAY, @Day) = 'Sunday' SET @Day = DATEADD(DAY, 1, @Day);
EXEC usp_Appointment_GetByDay @Day = @Day;
```

**1 Put your booking number here**

Lines 3 to 5

Replace the 0 with the number that the first test printed for you in stage 2.

**2 Cancel it and print what happened**

Lines 7 to 13

An empty message means it was cancelled. A sentence means it was refused, and the sentence is printed.

**3 Look at the day**

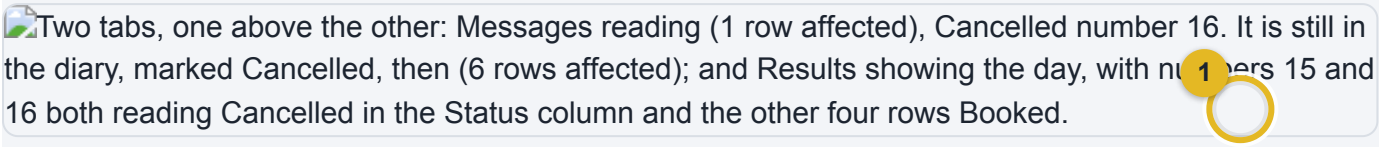
Lines 15 to 17

The same day as your tests. The cancelled row is still in the list, with its Status reading Cancelled.

Click New Query, press the Copy button under this step, paste the test, replace the 0 on the SET @ID line with your first booking number from stage 2, and press F5.

> **You should see** The Messages pane says (1 row affected), then Cancelled number 15. It is still in the diary, marked Cancelled, then (6 rows affected). Your number is the one you typed.

Change the number on the SET @ID line to your second test booking, the one that stage 3 moved to 15:15, and press F5 again.



Two tabs, one above the other: Messages reading (1 row affected), Cancelled number 16. It is still in the diary, marked Cancelled, then (6 rows affected); and Results showing the day, with numbers 15 and 16 both reading Cancelled in the Status column and the other four rows Booked.

Messages and Results are two tabs at the bottom of the window. Click each in turn. The cancelled rows are still there. Only their status changed.

1. Status reads Cancelled. The row was not deleted.

> **You should see** Cancelled number 16, in the same shape. Now click the Results tab at the bottom of the window: the day has six rows, and both of your test bookings are still in it, with Status reading Cancelled. Click Messages to come back.

# Check that the slot is free again.

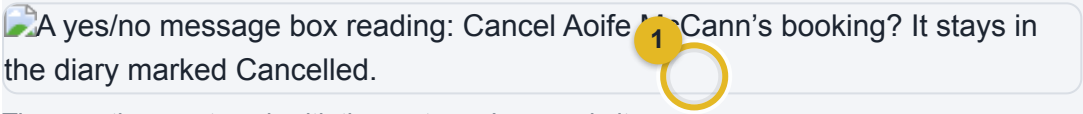
Go to your stage 2 test window and put the EXEC line back as it was: @CustomerID = 1, @MechanicID = 3, and 'Blue Trek hybrid'. Press F5.

> **You should see** Booked, with a new number. Aoife's 13:00 slot is free again, because the clash test leaves out cancelled rows.

In the cancel test window, leave the number as it is, your second booking, and press F5 again.

> **You should see** Only an open booking can be cancelled. The IF NOT EXISTS check caught it: that booking is no longer open.

In your program, close the diary and open it again from the main menu, pick the test day, select Aoife McCann's 13:00 booking, which is the one you just made, and press Cancel booking.




The question you typed, with the customer's name in it.

1. Yes cancels; No does nothing.

> **You should see** A yes-or-no question: Cancel Aoife McCann's booking? It stays in the diary marked Cancelled. After Yes, the row stays, with its Status reading Cancelled, and the three buttons switch off for it.

**RULE THAT NEVER BREAKS · NO DOUBLE BOOKING**

Cancelling frees the slot and keeps the row. Deleting the row would lose the shop's history, and the examiner marks that down.

# Run every test in your program.

Every row below is a test the examiner would run. The ones marked Must be refused are the important ones. If one of those gets through, the rule is broken, however nice the diary looks. Do each one in your program, tick it, and keep a screenshot for your testing evidence in chapter 10.

| THE TEST | WHAT MUST HAPPEN | HOW TO DO IT, AND WHAT YOU SHOULD SEE |
|---|---|---|
| A booking in a free slot | Must be allowed | New booking, a free mechanic, a free time. It appears in the grid. |
| The same mechanic, the same start time | Must be refused | Book it again for the same mechanic. Message: That mechanic already has a booking in that time. |
| The same mechanic, starting inside the booking | Must be refused | Start 15 minutes after the first booking's start. Refused with the same message. |
| The same mechanic, overlapping the start | Must be refused | Start 30 minutes before the first booking, with a service long enough to run into it. Refused. |
| The same mechanic, overlapping the end | Must be refused | Start 15 minutes before the first booking ends. Refused. |
| A booking that starts exactly when another ends | Must be allowed | Start at the first booking's end time. Allowed: touching is not overlapping. |
| Another mechanic at the same time | Must be allowed | The same time, a different mechanic. Allowed. |
| A cancelled slot booked again | Must be allowed | Cancel a booking, then book the same mechanic at that time. Allowed. |
| Move a booking onto itself | Must be allowed | Move, keep the same time, press Move. Allowed. |
| Move a booking onto another of the same mechanic | Must be refused | Move it to a time inside another of that mechanic's bookings. Refused. |
| A Sunday | Must be refused | Pick a Sunday. Message: The workshop is closed on Sundays. |

| THE TEST | WHAT MUST HAPPEN | HOW TO DO IT, AND WHAT YOU SHOULD SEE |
|---|---|---|
| A time that has passed | **Must be refused** | Pick yesterday. Message: That time has already passed. |
| Cancel keeps the row | **Must be allowed** | After a cancel, the row is still in the grid with Status Cancelled. |
| Cancel a cancelled booking | **Must be refused** | The Cancel booking button is off. In SQL Server Management Studio (SSMS), run the cancel test again: Only an open booking can be cancelled. |

# Finish the chapter.

Fourteen ticks are your evidence that the rule holds. Keep the screenshots for your testing evidence in chapter 10. When you want a clean day again, click in BikeShop.sql and press F5 with nothing selected: the whole script runs, the sample data comes back, and your test bookings go.

**Make it yours**

Your scenario will not have mechanics and bikes. Write down what the equivalent is: the thing that cannot be in two places at once, such as a room, a pod, a van or a tutor, and the start and end of its use. The lines you typed three times are the same for all of them. Only the table and column names change. Chapter 12 goes through this for your own tables.

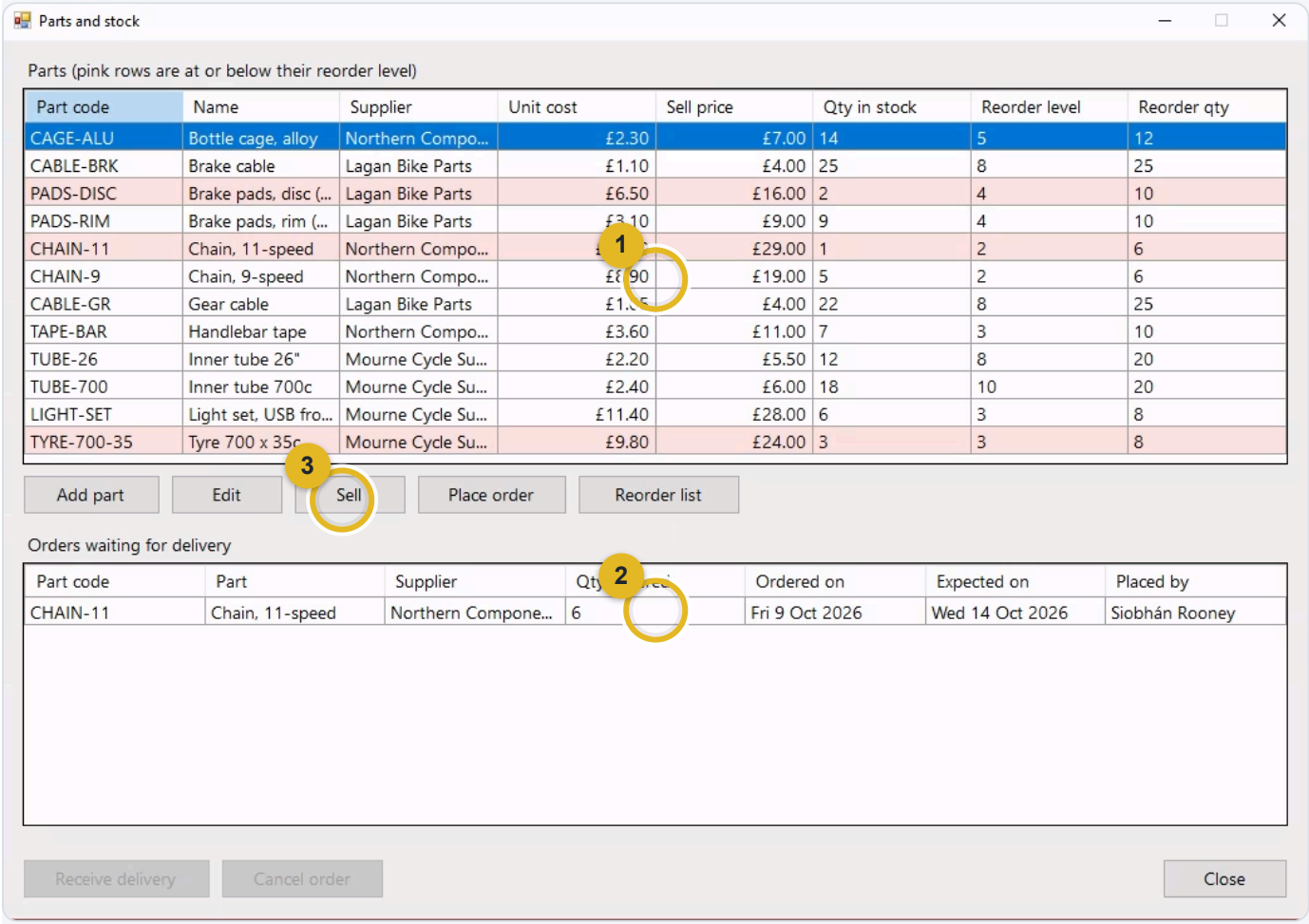
# Stock

Stock that is always right: sell, receive and reorder, with a count that can never go below zero.

STAGE 1 OF 10 · TWO KINDS OF STOCK · SCREEN 1 OF 4

## See what you will build in this chapter.

The shop keeps two kinds of stock. Parts are counted: there are five 9-speed chains on the shelf, and the number goes down when one is sold and up when a box arrives. Bikes are one thing each: a bike has a frame number, and it is available, reserved, sold or in repair. Nobody counts bikes, because each one is its own row. This chapter builds the screens for both, and the stored procedures underneath them that keep every count and every status right.




What you will build in this chapter: the parts screen with its pink reorder rows, the bikes screen and the hire screen, all backed by stored procedures that keep every count right.

1. Every part with its real count. Pink rows are at or below their reorder level.
2. The orders still waiting for a delivery. Receive closes one and adds to the count in one go.
3. Sell opens the counter sale. The database refuses if the count is short.

By the end, the counter staff can sell a part, refund it, order more and receive the delivery. They can reserve a bike, sell it once and only once, and send it for repair. They can hand a hire bike over, take it back and cancel a hire that never started. Every one of those is one stored procedure, and every stored procedure refuses when the action makes no sense.

# Check that you have these ready.

**Before you start**

- Chapter 7 is finished: your BikeShop.sql has the booking and hire stored procedures in it, and the diary runs.
- SQL Server Management Studio (SSMS) is open, connected to your BikeShop database.
- Your BikeShop solution is open in Visual Studio 2019.
- This guide is open in the Edge browser beside you. Tick each step as you do it, and the chapter remembers where you are.

**Words used in this chapter**

| | |
|---|---|
| `Stock` | The things the shop has to sell or to hire out right now. |
| `Count` | The number of one part on the shelf, which the Part table keeps in its QtyInStock column. |
| `Guard` | A check inside a stored procedure that refuses an action before any row is changed. |
| `Reorder level` | The count at which the shop should order more of a part. |
| `Reorder quantity` | How many of a part the shop usually orders at a time. |
| `Walk-in customer` | A customer who buys at the counter without an account, saved as no customer at all. |
| `Refund` | Taking a sale back, so the parts return to the shelf and the sale row is marked Refunded. |
| `Delivery` | A box from a supplier arriving, which closes an order and adds its contents to the count. |
| `Status` | One word that says where a thing is in its life, such as Available, Reserved or Sold. |
| `Reserve` | Holding a bike for sale for one customer, so nobody else can buy it. |

# Read how a count stays right.

Every change to the count of a part goes through one of three stored procedures: a sale takes some off, a refund puts them back, and a delivery adds a box. No screen ever types a new count into the Part table. That is the whole idea. If the only way in is through those three doors, and each door checks before it opens, the count is always right.

The table below plays out one part through a day. Press the button to switch the guard off and watch what happens to the count.

**With the guard in the stored procedure.**

| What happens | Change | 9-speed chains in stock after it |
|---|---|---|
| The day starts with five chains on the shelf | | 5 |
| A walk-in customer buys three | -3 | 2 |
| Another customer asks for three more<br>**Only 2 in stock.** | ~~-3~~ | 2 |
| A box of ten arrives from the supplier | +10 | 12 |
| The first customer brings their three back for a refund | +3 | 15 |

Switch the guard off:

| What happens | Change | 9-speed chains in stock after it |
|---|---|---|
| The day starts with five chains on the shelf | | 5 |
| A walk-in customer buys three | -3 | 2 |
| Another customer asks for three more | -3 | -1 |
| A box of ten arrives from the supplier | +10 | 9 |
| The first customer brings their three back for a refund | +3 | 12 |

Without the guard, the database would have let the shop sell what it does not have, and the count would be a lie from then on.

**RULE THAT NEVER BREAKS · STOCK IS ALWAYS RIGHT**

The count of a part is only ever changed by a sale, a refund or a delivery, and a sale is refused before the count can go below zero. The check is inside the stored procedure, so every screen gets it for free.

# Read how a bike is one thing, not a count.

A bike is never counted, because every bike is different: its own frame number, size and colour. Instead, each bike has a status. A bike for sale starts as Available. A customer can reserve it, which makes it Reserved and remembers who. Then it is sold, once, and becomes Sold with the date. Any bike can go into repair and come back.

**The life of a bike for sale**

Available, then Reserved for one customer, then Sold to that customer. Reserved can go back to Available if the customer changes their mind. Sold is the end: a sold bike is never sold again, and the stored procedure in stage 6 makes sure of it.

**The life of a hire bike**

A hire bike stays Available and goes out and comes back through its hires. Each hire has its own status: Booked, then Out, then Returned, or Cancelled if it never started. The free list in stage 6 offers only the hire bikes with no Booked or Out hire on those dates, which is the clash test from chapter 7 with dates instead of times.

Each step in those lives is one stored procedure, and each one checks that the bike is in the right status first. A sold bike cannot be reserved. A bike that is not in repair cannot come back from repair. The checks are the same shape every time: IF NOT EXISTS a row in the right status, refuse.

# Read the stored procedure that sells a part, then add it to your script.

A counter sale is one stored procedure. It looks up the part, checks that there are enough, takes the quantity off the count, and writes the sale row with the price at the time. The check in the middle is the stock guard. It is the one part of this stored procedure you type, because the same shape goes into your own project.

**Type this one** **usp_PartSale_Add: sell some of a part, or refuse** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
CREATE OR ALTER PROCEDURE usp_PartSale_Add
    @PartID        INT,
    @CustomerID    INT,          -- NULL for a walk-in customer
    @Quantity      INT,
    @SoldByStaffID INT,
    @NewPartSaleID INT OUTPUT,
    @Message VARCHAR(200) OUTPUT
AS
BEGIN
    SET @Message = '';

    IF @Quantity ⩽ 0
    BEGIN
        SET @Message = 'Sell at least one.';
        RETURN;
    END

    DECLARE @SellPrice DECIMAL(7,2), @InStock INT;
    SELECT @SellPrice = SellPrice, @InStock = QtyInStock FROM Part
        WHERE PartID = @PartID;
    IF @SellPrice IS NULL
    BEGIN
        SET @Message = 'Choose a part.';
        RETURN;
    END

    -- Is there enough?  If not, refuse and stop.
    IF @InStock < @Quantity
    BEGIN
        SET @Message = 'Only ' + CAST(@InStock AS VARCHAR) + ' in stock.';
        RETURN;
    END

    -- Take the stock off.  The table's CHECK (QtyInStock ⩾ 0) is the
    -- database's own backstop.
    UPDATE Part
    SET QtyInStock = QtyInStock - @Quantity
    WHERE PartID = @PartID;

    INSERT INTO PartSale (PartID, CustomerID, Quantity, UnitPrice,
        SoldByStaffID)
    VALUES (@PartID, @CustomerID, @Quantity, @SellPrice, @SoldByStaffID);

    SET @NewPartSaleID = SCOPE_IDENTITY();
END
GO
```

**1 The name, and what goes in and out**

Lines 1 to 7

A sale needs the part, the customer (or NULL for a walk-in customer), how many, and who sold it. The new sale number and @Message come back out.

**2 Start with a blank message**

Lines 8 to 10

AS and BEGIN mark where the work starts. The first job is to make @Message empty. An empty message means it worked. Every refusal below puts a sentence into @Message instead, so the program always knows which it was.

**3 Refuse a quantity of nothing**

Lines 12 to 16

Selling zero, or a minus number, makes no sense, so it is refused before anything is looked up.

**4 Look up the part**

Lines 18 to 20

One SELECT fills two boxes from the part's row: its selling price and how many are on the shelf right now.

**5 Refuse a part that does not exist**

Lines 21 to 25

NULL means no row matched the number, so there is no part to sell.

**6 The stock guard: this is the part you type**

Lines 28 to 32

If fewer are on the shelf than the customer wants, refuse, and say how many there are. CAST turns the number into text so that it can sit inside the sentence.

**7 Take the stock off**

Lines 36 to 38

The count goes down by the quantity sold. The CHECK on the table from chapter 3 is a second line of defence: it would refuse a count below zero even if this guard were missing.

**8 Save the sale**

Lines 40 to 42

One new row in the PartSale table, with the price copied in. If the price changes next month, this sale still shows what was paid.

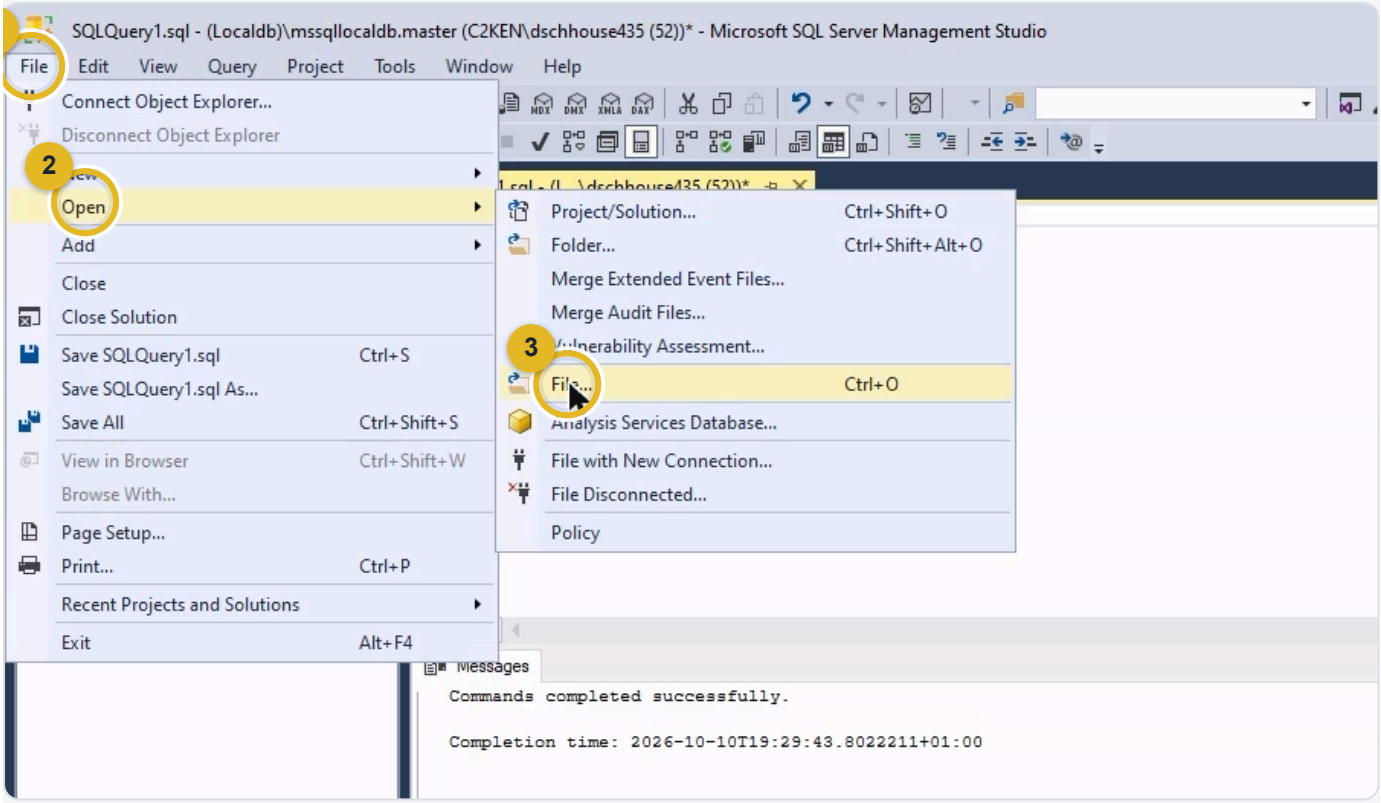
**9 Hand back the new number**

Lines 44 to 46

The number of the new row goes back through the OUTPUT parameter. GO ends the batch, ready for the next stored procedure.

**Why you type it** The stock guard is what keeps the count honest. Type it, so that you can write it again on your own tables without looking.

In SQL Server Management Studio (SSMS), click File at the top left, point at Open, and click File…




The File menu in SQL Server Management Studio (SSMS). Your script opens from here.

1. Click File at the top left.

2. Point at Open.

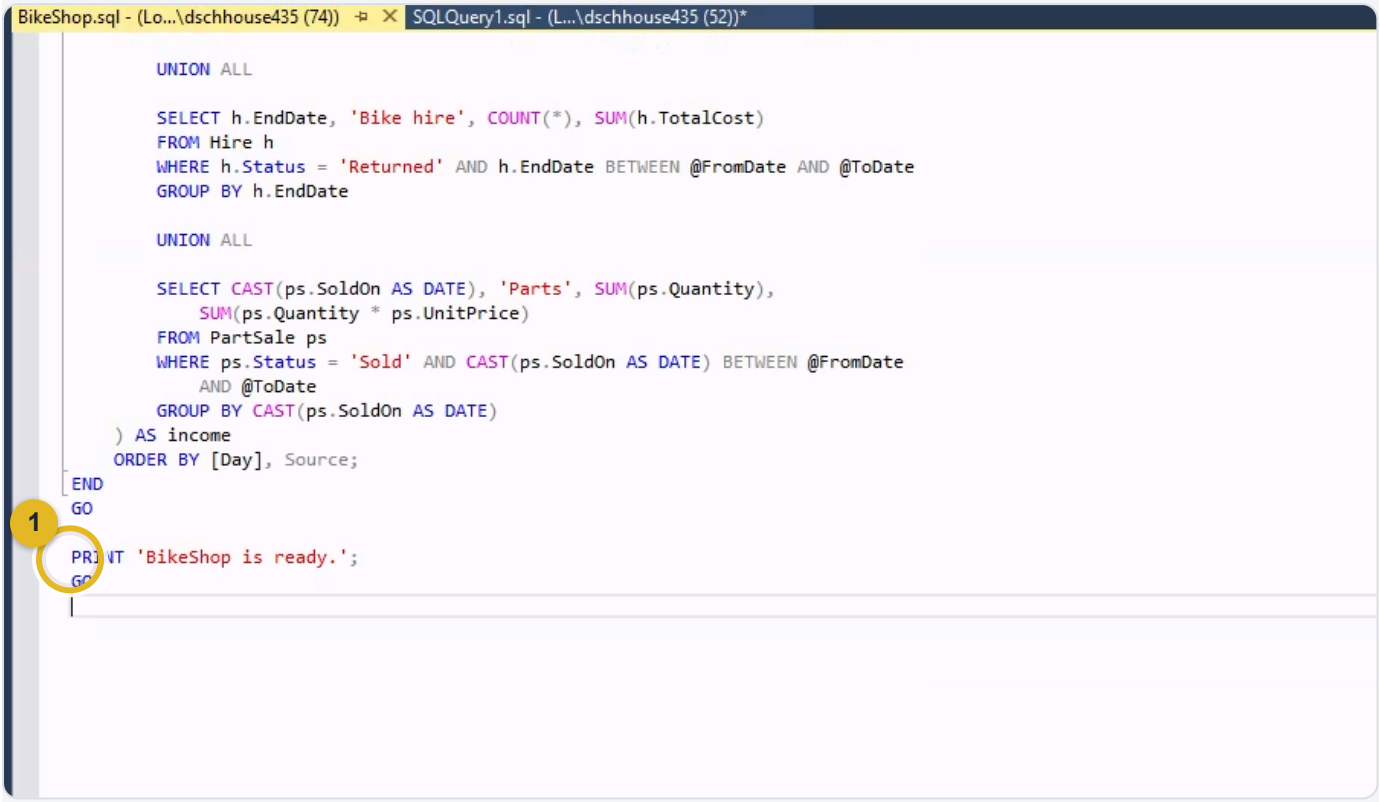
3. Click File… and choose BikeShop.sql from your OneDrive folder.

> **You should see** A window opens for you to choose a file.

Choose BikeShop.sql from your OneDrive folder and press Open.

> **You should see** Your script is in a tab, and the first line is the USE line from chapter 3.

Scroll to the very end of the script and click on the empty line above the line that prints BikeShop is ready. That line always stays last.

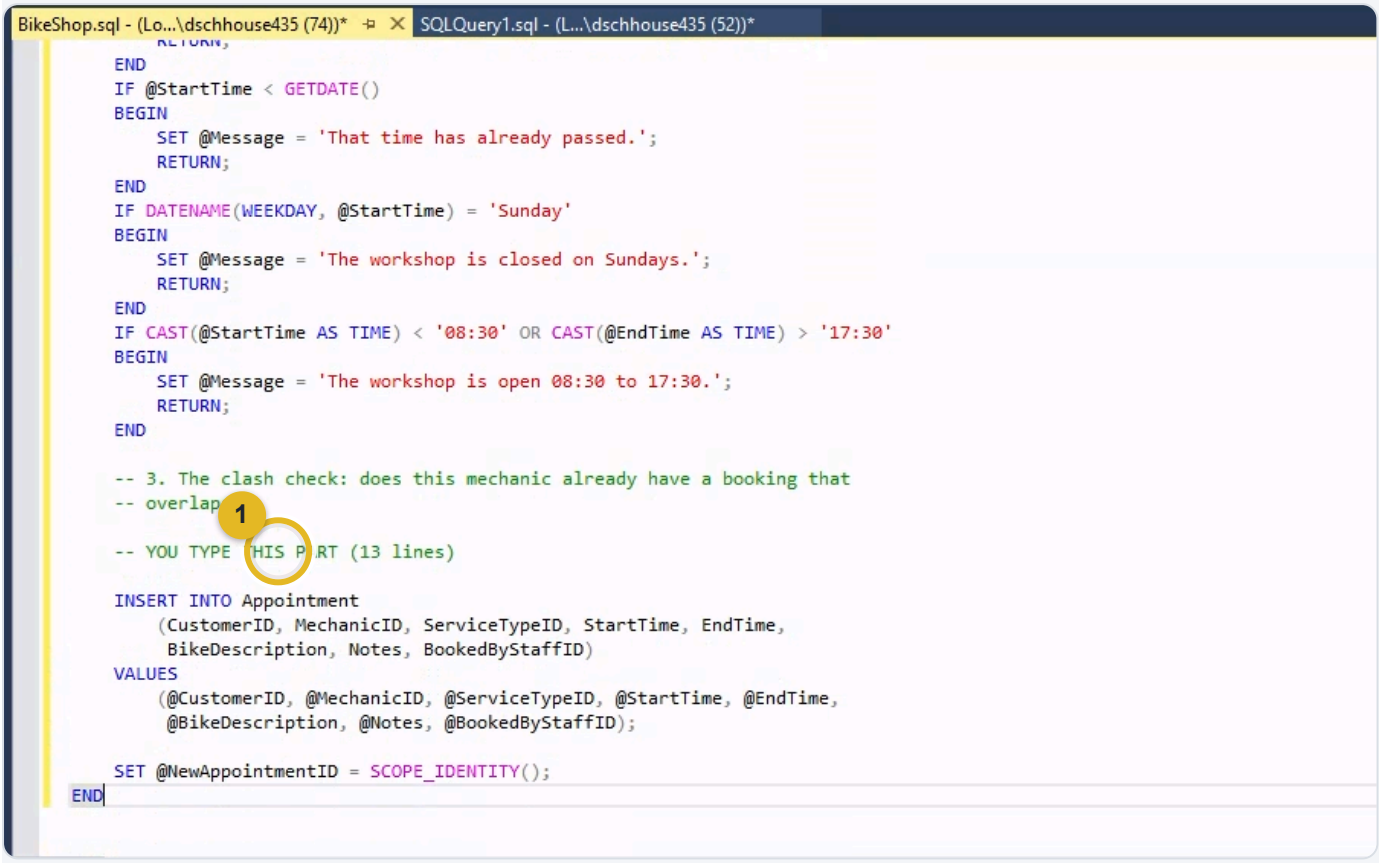



The end of your script. Everything new in this chapter goes on the empty line above the line that prints BikeShop is ready, one stored procedure after another, each ending with GO.

1. Click on the empty line above the line that prints BikeShop is ready, so the cursor blinks there.

> **You should see** The cursor blinks on the empty line, with the hire stored procedures from chapter 7 above it.

Press the Copy button under this step, click on the empty line above the line that prints BikeShop is ready, and paste.




After the paste. Everything is in except the yellow lines. The marker line shows where they go.

1 The marker line that says YOU TYPE THIS PART. The yellow lines go in its place in the next step.

> **You should see** The stored procedure is in your script, with one line that says YOU TYPE THIS PART.

Type the yellow lines in place of the line that says YOU TYPE THIS PART, so that the marker line is gone.

> **You should see** Five lines of your own, from IF @InStock to END, and no marker line.

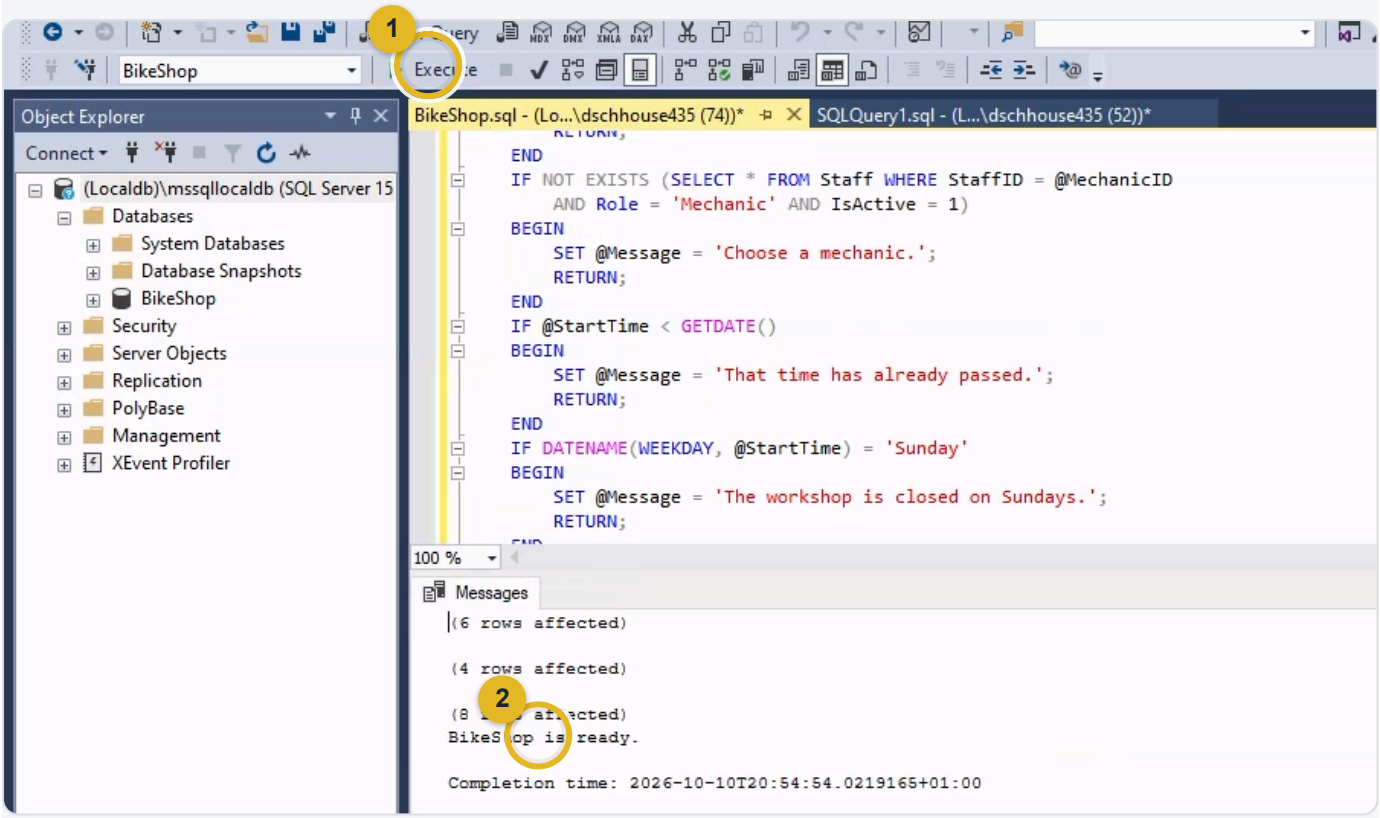
**IF IT GOES WRONG** A red wavy line under a word

A spelling slip. Compare your line with the yellow line, letter by letter, and check that each line ends the way the yellow line ends.

**IF IT GOES WRONG** 'Must declare the scalar variable …'

A name beginning with @ is spelled differently from the line that declares it. Make the two match.

Press F5, or click Execute on the toolbar, with nothing selected. This runs the whole script, so your database is built again from nothing with the new stored procedure in it.




Execute with nothing selected runs the whole script. The Messages pane counts the sample rows it put back, then says BikeShop is ready.

1. Click Execute, or press F5.

2. The last lines say BikeShop is ready, then a Completion time. Red text here is an error to read.

**You should see** The Messages pane ends with BikeShop is ready and a Completion time. No red text. The test bookings you made in chapter 7 are gone, because the sample data was put back, and that is fine.

**IF IT GOES WRONG** A red error in the Messages pane

Double-click the error. SQL Server Management Studio (SSMS) jumps to the line it is about. Compare that line with the block above, letter by letter.

## Test it: sell three, then try to sell three more.

The test sells three 9-speed chains to a walk-in customer. There are five on the shelf, so the first run works and two are left. The second run asks for three more, and the guard refuses.

Copy this one **A test that sells three chains, and prints the new count** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
-- Sell 3 of part 5 (the 9-speed chain, 5 in stock) to a walk-in customer.
-- Run it once: it sells and 2 are left.  Run it again: only 2 in stock.
DECLARE @NewID INT, @Left INT, @Message VARCHAR(200);

EXEC usp_PartSale_Add
     @PartID = 5, @CustomerID = NULL, @Quantity = 3, @SoldByStaffID = 2,
     @NewPartSaleID = @NewID OUTPUT, @Message = @Message OUTPUT;

IF @Message = ''
BEGIN
    SELECT @Left = QtyInStock FROM Part WHERE PartID = 5;
    PRINT 'Sold. New sale number ' + CAST(@NewID AS VARCHAR)
        + '. Stock of part 5 is now '
        + CAST(@Left AS VARCHAR)
        + '.';
END
ELSE
BEGIN
    PRINT @Message;
END
```

**1 Three boxes to hold the answers**

Line 3

A variable is a named box that holds one value while the script runs. @NewID will hold the number of the new sale, @Left the number of chains left, and @Message the refusal, if there is one.

**2 Run the stored procedure**

Lines 5 to 7

EXEC runs the stored procedure. Each parameter is named, so the order does not matter. Part 5 is the 9-speed chain, NULL means a walk-in customer, and staff 2 is Pádraig Maguire at the counter.

**3 If the message is empty, it worked**

Lines 9 to 16

An empty message means the sale was saved. PRINT writes one line in the Messages pane with the new number and the count now. The SELECT line above it reads the count from the Part table into @Left.

**4 Otherwise, print the refusal**

Lines 17 to 20

If @Message holds a sentence, the stored procedure refused, and the sentence is printed as it is.

Click New Query on the toolbar, or press Ctrl and N together. Press the Copy button under this step, paste the test into the empty window, and press F5.

The first run sells three chains and the count drops to 2. The second run asks for three more, and the stored procedure refuses with the real count in the sentence.

1. Write this sale number down. You refund it in stage 3.

2. The refusal. Nothing was saved and the count is still 2.

> **You should see** The Messages pane says (1 row affected) twice, then: Sold. New sale number 9. Stock of part 5 is now 2. The two counted rows are the part that changed and the sale that was saved. Write the sale number down: stage 3 refunds it.

Press F5 again, changing nothing.

> **You should see** The Messages pane says only: Only 2 in stock. The guard refused, and nothing was saved. The count is still 2.

Change the 3 after @Quantity to 2, and press F5.

> **You should see** The Messages pane says: Sold. New sale number 10. Stock of part 5 is now 0. Write this second sale number down, because stage 3 refunds both sales. Selling exactly what is on the shelf is allowed, and selling one more than that is not. Leave this window open.

**IF IT GOES WRONG** 'Choose a part.' instead of a sale

There is no part number 5 in your Part table. Look at your Part rows and use the number of a part that is there.

**IF IT GOES WRONG** The count printed is wrong

Your script was run with nothing selected since the last test, which put the sample data back. The count starts at 5 again. That is not a fault.

# Write the refund stored procedure yourself.

A refund is the sale in reverse. The sale row is not deleted: it is marked Refunded, and the quantity goes back on the shelf. The check is the same shape as the cancel in chapter 7: find the row in the right status, and refuse if there is none. You write the middle of this one yourself.

Your turn **usp_PartSale_Refund: mark a sale Refunded and put the parts back** SQL

1. In SQL Server Management Studio (SSMS), click on the empty line above the line that prints BikeShop is ready and type the first lines of the stored procedure: its name, the two parameters, AS, BEGIN and the blank message.
2. Declare two boxes, @PartID and @Quantity, and fill them with one SELECT from the PartSale row whose number matches and whose Status is Sold.
3. If @PartID is NULL, there is no such sale still open: SET @Message to 'That sale has already been refunded, or does not exist.' and RETURN.
4. Then two UPDATEs: mark the PartSale row Refunded, and add @Quantity back to QtyInStock on the Part row. Finish with END and GO.
5. Select the lines you typed, from CREATE OR ALTER PROCEDURE down to GO, and press F5.

**THE TEST** The ready-made test on the next screen refunds one sale by its number, prints the count, and then lists the sales of the last ten days.

**YOU SHOULD SEE** 'Refunded sale number 9. Stock of part 5 is now 5.' on the first run, and 'That sale has already been refunded, or does not exist.' on the second. If the second run prints Refunded again, your check is missing the Status test.

**HINT** The SELECT that fills two boxes at once looks like the one in usp_PartSale_Add: SELECT @PartID = PartID, @Quantity = Quantity FROM PartSale WHERE … The Status test goes in that WHERE, so a refunded sale is never found.

▾ **Stuck? Show me the finished procedure**

Read it, then write yours **usp_PartSale_Refund** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  CREATE OR ALTER PROCEDURE usp_PartSale_Refund
2      @PartSaleID INT,
3      @Message VARCHAR(200) OUTPUT
4  AS
5  BEGIN
6      SET @Message = '';
7
8      DECLARE @PartID INT, @Quantity INT;
9      SELECT @PartID = PartID, @Quantity = Quantity
10     FROM PartSale
11     WHERE PartSaleID = @PartSaleID AND Status = 'Sold';
12     IF @PartID IS NULL
13     BEGIN
14         SET @Message =
15             'That sale has already been refunded, or does not exist.';
16         RETURN;
17     END
18
19     UPDATE PartSale SET Status = 'Refunded' WHERE PartSaleID = @PartSaleID;
20     -- stock goes back
21     UPDATE Part SET QtyInStock = QtyInStock + @Quantity WHERE PartID = @PartID;
22 END
23 GO
```

**1 The name, and what goes in and out**

Lines 1 to 3

Only the sale number goes in. @Message comes back out.

**2 Start with a blank message**

Lines 4 to 6

AS and BEGIN mark where the work starts. The first job is to make @Message empty. An empty message means it worked. Every refusal below puts a sentence into @Message instead, so the program always knows which it was.

**3 Find the sale, if it is still open**

Lines 8 to 11

Two boxes, filled by one SELECT. The WHERE asks for the sale number and the status Sold together, so a sale that was already refunded fills nothing.

**4 Refuse if there is no open sale**

Lines 12 to 17

NULL in the box means the SELECT found no row. The sentence covers both reasons: already refunded, or never there.

**5 Mark the sale Refunded**

Line 19

The row stays, with its status changed. The shop keeps its history, and the report in chapter 9 can count refunds.

**6 Put the stock back**

Lines 21 to 23

The quantity from the sale row goes back on the count. This is the only place a refund touches the count. GO ends the batch, ready for the next stored procedure.

# Read the stored procedure that lists sales between two dates, then add it to your script.

The parts screen does not show sales, but the report in chapter 9 does, and the test on the next screen uses this to show you the refund. Read it, then add it. The ISNULL in the SELECT is new: a walk-in sale has no customer row, so the Customer column would be empty, and ISNULL puts the word Walk-in there instead.

Read this one **usp_PartSale_GetByDateRange: the sales between two dates, with names** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  CREATE OR ALTER PROCEDURE usp_PartSale_GetByDateRange
2      @FromDate DATE,
3      @ToDate   DATE
4  AS
5  BEGIN
6      SELECT ps.PartSaleID, ps.SoldOn, p.PartCode, p.Name AS Part,
7             ISNULL(c.FirstName + ' ' + c.LastName, 'Walk-in') AS Customer,
8             ps.Quantity, ps.UnitPrice,
9             ps.Quantity * ps.UnitPrice AS Total,
10            st.FirstName + ' ' + st.LastName AS SoldBy,
11            ps.Status
12     FROM PartSale ps
13     JOIN Part      p  ON p.PartID      = ps.PartID
14     LEFT JOIN Customer c ON c.CustomerID = ps.CustomerID
15     JOIN Staff     st ON st.StaffID    = ps.SoldByStaffID
16     WHERE CAST(ps.SoldOn AS DATE) BETWEEN @FromDate AND @ToDate
17     ORDER BY ps.SoldOn DESC;
18 END
19 GO
```

**1 The name, and what goes in and out**

Lines 1 to 3

Two dates go in. A table of rows comes back, so there is no @Message.

**2 What comes back**

Lines 4 to 11

Each row is one sale: its number and time, the part code and name, the customer or the word Walk-in, how many at what price, the total worked out on the spot, who sold it, and its status.

**3 Where the rows come from**

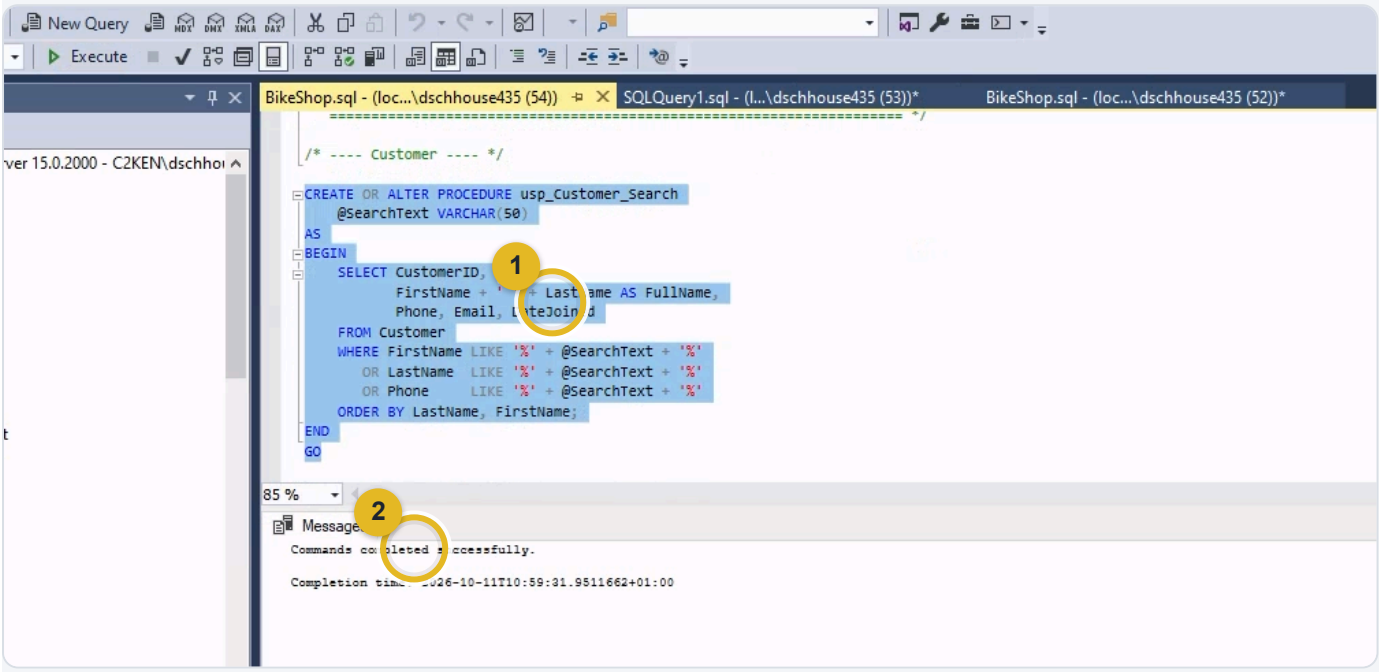
Lines 12 to 19

Four tables joined. LEFT JOIN Customer keeps the walk-in sales, which have no customer row. CAST(ps.SoldOn AS DATE) throws away the clock part so that BETWEEN works on whole days. Newest first. GO ends the batch, ready for the next stored procedure.

Press the Copy button under this step, click on the empty line above the line that prints BikeShop is ready, and paste.

> **You should see** The stored procedure is in your script, ending with GO, and the line that prints BikeShop is ready is still last.

Select the lines you just added, from CREATE OR ALTER PROCEDURE down to GO, and press F5.




Select the lines you added, then press F5. Only the highlighted lines run, so the test sales and orders you have made stay in the database.

1. Drag from CREATE OR ALTER PROCEDURE down to GO, so the whole block is highlighted.

2. The Messages pane says: Commands completed successfully.

> **You should see** The Messages pane says: Commands completed successfully. Your test sales and orders are still there.

**IF IT GOES WRONG** The Messages pane shows (6 rows affected) and BikeShop is ready

Nothing was selected, so the whole script ran and the sample data was put back. Your test sales, orders and reservations are gone. Nothing is broken: carry on, and remember that the stock of part 5 is 5 again.

# Test the refund with your sale number.

Copy this one **A test that refunds one sale, then lists the last ten days of sales** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
-- Refund one sale by its number, then look at the sales of the last ten
-- days.  Run it once: the stock goes back up.  Run it again: refused.
DECLARE @ID INT, @Left INT, @From DATE, @To DATE, @Message VARCHAR(200);
-- put your sale number here
SET @ID = 9;

EXEC usp_PartSale_Refund @PartSaleID = @ID, @Message = @Message OUTPUT;

IF @Message = ''
BEGIN
    SELECT @Left = QtyInStock FROM Part WHERE PartID = 5;
    PRINT 'Refunded sale number ' + CAST(@ID AS VARCHAR)
        + '. Stock of part 5 is now '
        + CAST(@Left AS VARCHAR)
        + '.';
END
ELSE
BEGIN
    PRINT @Message;
END

SET @To = CAST(GETDATE() AS DATE);
SET @From = DATEADD(DAY, -10, @To);
EXEC usp_PartSale_GetByDateRange @FromDate = @From, @ToDate = @To;
```

**1 The sale to refund**

Lines 3 to 5

Five boxes. @Left will hold the number of chains left. @ID is the sale number, and the SET line is where you put your own number from stage 2.

**2 Run the stored procedure**

Line 7

The sale number goes in, and @Message comes back.

**3 If the message is empty, it worked**

Lines 9 to 16

The SELECT line reads the count from the Part table into @Left, and PRINT writes the sale number and that count.

**4 Otherwise, print the refusal**

Lines 17 to 20

The sentence from the stored procedure is printed as it is.

**5 Then list the sales**

Lines 22 to 24

> Today, and ten days before it. The sales between the two come back as a table in the Results tab, your test sales at the top because they are newest.

Click New Query, press the Copy button under this step, paste the test, change the 9 on the SET @ID line to your first sale number from stage 2, and press F5.

> **You should see** The Messages pane says (1 row affected) twice, then: Refunded sale number 9. Stock of part 5 is now 3. Your number is the one you typed. Three chains are back on the shelf, on top of the zero that stage 2 left.

Press F5 again, changing nothing.

> **You should see** The Messages pane says: That sale has already been refunded, or does not exist. The count stays at 3.

Change the number on the SET @ID line to your second sale number, the one for two chains, and press F5.

> **You should see** The Messages pane says: Refunded sale number 10, with your own number, and the count is 5 again.

Click the Results tab at the bottom of the window.

> **You should see** Your sales are the top rows, with Walk-in in the Customer column and Refunded in the Status column.

STAGE 4 OF 10 · ORDERS IN: PLACE, RECEIVE, CANCEL, AND THE REORDER LIST · SCREEN 1 OF 5

# Read the stored procedure that places an order, then add it to your script.

Ordering more of a part does not change the count. The box has not arrived yet. An order is a row in the PartOrder table with the status Ordered, and nothing else happens until the delivery is received.

Copy this one **usp_PartOrder_Place: order more of a part from its supplier** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
CREATE OR ALTER PROCEDURE usp_PartOrder_Place
    @PartID          INT,
    @QtyOrdered      INT,
    @ExpectedOn      DATE,
    @PlacedByStaffID INT,
    @NewPartOrderID  INT OUTPUT,
    @Message VARCHAR(200) OUTPUT
AS
BEGIN
    SET @Message = '';
    IF @QtyOrdered ⩽ 0
    BEGIN
        SET @Message = 'Order at least one.';
        RETURN;
    END

    INSERT INTO PartOrder (PartID, QtyOrdered, ExpectedOn, PlacedByStaffID)
    VALUES (@PartID, @QtyOrdered, @ExpectedOn, @PlacedByStaffID);
    SET @NewPartOrderID = SCOPE_IDENTITY();
END
GO
```

**1 The name, and what goes in and out**

Lines 1 to 7

The part, how many, the day the box is expected, and who placed the order. The new order number and @Message come back out.

**2 Start with a blank message**

Lines 8 to 10

AS and BEGIN mark where the work starts. The first job is to make @Message empty. An empty message means it worked. Every refusal below puts a sentence into @Message instead, so the program always knows which it was.

**3 Refuse an order for nothing**

Lines 11 to 15

An order for zero is refused before anything is saved.

**4 Save the order**

Lines 17 to 18

One new row. The table fills in OrderedOn with today and Status with Ordered on its own, from the defaults you set in chapter 3.

**5 Hand back the new number**

Lines 19 to 21

The number of the new row goes back through the OUTPUT parameter. GO ends the batch, ready for the next stored procedure.

Press the Copy button under this step, click on the empty line above the line that prints BikeShop is ready, and paste.

> **You should see** The stored procedure is in your script, ending with GO, and the line that prints BikeShop is ready is still last.

Select the lines you just added, from CREATE OR ALTER PROCEDURE down to GO, and press F5.

> **You should see** The Messages pane says: Commands completed successfully. Your test sales are still there.

**IF IT GOES WRONG** The Messages pane shows (6 rows affected) and BikeShop is ready

Nothing was selected, so the whole script ran and the sample data was put back. Your test sales, orders and reservations are gone. Nothing is broken: carry on, and remember that the stock of part 5 is 5 again.

# Read the stored procedure that receives a delivery, then add it to your script.

When the box arrives, two things must both happen: the order closes, and the count goes up by what was in the box. If only one of them happened, the shop would either count the same box twice or never count it at all. Putting both UPDATEs inside one stored procedure is how the two stay together. You type those two UPDATEs.

**Type this one** **usp_PartOrder_Receive: close the order and add the box to the count** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
CREATE OR ALTER PROCEDURE usp_PartOrder_Receive
    @PartOrderID INT,
    @QtyReceived INT,
    @Message VARCHAR(200) OUTPUT
AS
BEGIN
    SET @Message = '';

    IF @QtyReceived < 0
    BEGIN
        SET @Message = 'The quantity received cannot be negative.';
        RETURN;
    END

    DECLARE @PartID INT;
    SELECT @PartID = PartID FROM PartOrder WHERE PartOrderID = @PartOrderID
        AND Status = 'Ordered';
    IF @PartID IS NULL
    BEGIN
        SET @Message = 'That order is not open.';
        RETURN;
    END

    -- Two changes that must both happen: the order is closed and the stock goes
    -- up.
    UPDATE PartOrder
    SET Status = 'Received', ReceivedOn = CAST(GETDATE() AS DATE),
        QtyReceived = @QtyReceived
    WHERE PartOrderID = @PartOrderID;

    UPDATE Part
    SET QtyInStock = QtyInStock + @QtyReceived
    WHERE PartID = @PartID;
END
GO
```

**1 The name, and what goes in and out**

Lines 1 to 4

The order number and how many actually arrived. @Message comes back out.

**2 Start with a blank message**

Lines 5 to 7

AS and BEGIN mark where the work starts. The first job is to make @Message empty. An empty message means it worked. Every refusal below puts a sentence into @Message instead, so the program always knows which it was.

**3 Refuse a minus number**

Lines 9 to 13

A box can be short, even empty, but it cannot hold fewer than nothing.

**4 Find the open order**

Lines 15 to 17

One box, filled from the PartOrder row whose number matches and whose status is still Ordered. The part number is what the second UPDATE needs.

**5 Refuse if the order is not open**

Lines 18 to 22

NULL means no open order matched. It was received already, or cancelled, or never existed.

**6 Close the order: the first line you type**

Lines 26 to 29

The order becomes Received, with today's date and the quantity that actually came.

**7 Add the box to the count: the second line you type**

Lines 31 to 35

The count goes up by what arrived, not by what was ordered. A short box adds only what was in it. GO ends the batch, ready for the next stored procedure.

**Why you type it** Two changes that belong together, in one place. Type them, so that the pattern is yours for the two-table changes in your own project.

Press the Copy button under this step, click on the empty line above the line that prints BikeShop is ready, and paste.

**You should see** The stored procedure is in your script, with one line that says YOU TYPE THIS PART.

Type the yellow lines in place of the line that says YOU TYPE THIS PART, so that the marker line is gone.

**You should see** Seven lines of your own, from UPDATE PartOrder down to WHERE PartID = @PartID; and no marker line.

**IF IT GOES WRONG** A red wavy line under a word

A spelling slip. Compare your line with the yellow line, letter by letter, and check that each line ends the way the yellow line ends.

**IF IT GOES WRONG** 'Must declare the scalar variable …'

A name beginning with @ is spelled differently from the line that declares it. Make the two match.

Select the lines you just added, from CREATE OR ALTER PROCEDURE down to GO, and press F5.

**You should see** The Messages pane says: Commands completed successfully. Your test sales and orders are still there.

**IF IT GOES WRONG** The Messages pane shows (6 rows affected) and BikeShop is ready

Nothing was selected, so the whole script ran and the sample data was put back. Your test sales, orders and reservations are gone. Nothing is broken: carry on, and remember that the stock of part 5 is 5 again.

# Test an order from placing it to receiving it.

Two tests. The first orders ten 9-speed chains, expected in a week. The second receives that order, and the count goes up by ten. Run the second one twice, to see the refusal.

**Copy this one** **A test that orders ten chains** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  -- Order 10 more of part 5 from its supplier, expected in a week.
2  DECLARE @Expected DATE, @NewID INT, @Message VARCHAR(200);
3  SET @Expected = DATEADD(DAY, 7, CAST(GETDATE() AS DATE));
4
5  EXEC usp_PartOrder_Place
6       @PartID = 5, @QtyOrdered = 10, @ExpectedOn = @Expected,
7       @PlacedByStaffID = 1,
8       @NewPartOrderID = @NewID OUTPUT, @Message = @Message OUTPUT;
9
10 IF @Message = ''
11 BEGIN
12     PRINT 'Ordered. New order number ' + CAST(@NewID AS VARCHAR)
13         + ', expected on ' + CAST(@Expected AS VARCHAR) + '.';
14 END
15 ELSE
16 BEGIN
17     PRINT @Message;
18 END
```

**1 A week from today**

Lines 2 to 3

Three boxes. @Expected is today plus seven days, with the clock part thrown away.

**2 Run the stored procedure**

Lines 5 to 8

The test orders part 5, ten of them, with the expected date, for staff 1, Siobhán Rooney the manager.

**3 If the message is empty, it worked**

Lines 10 to 14

PRINT writes the new order number and the expected date.

**4 Otherwise, print the refusal**

Lines 15 to 18

The sentence from the stored procedure is printed as it is.

Copy this one **A test that receives one order by its number** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
-- The delivery arrives: receive one order by its number.
-- Run it once: the order closes and the stock goes up.  Run it again:
-- that order is not open.
DECLARE @ID INT, @Left INT, @Message VARCHAR(200);
-- put your order number here
SET @ID = 5;

EXEC usp_PartOrder_Receive
     @PartOrderID = @ID, @QtyReceived = 10, @Message = @Message OUTPUT;

IF @Message = ''
BEGIN
    SELECT @Left = QtyInStock FROM Part WHERE PartID = 5;
    PRINT 'Received order number ' + CAST(@ID AS VARCHAR)
        + '. Stock of part 5 is now '
        + CAST(@Left AS VARCHAR)
        + '.';
END
ELSE
BEGIN
    PRINT @Message;
END
```

**1 The order to receive**

Lines 4 to 6

@Left will hold the number of chains left. The SET line is where your own order number goes.

**2 Run the stored procedure**

Lines 8 to 9

The test sends the order number and ten received, which is the whole box.

**3 If the message is empty, it worked**

Lines 11 to 18

The SELECT line reads the count from the Part table into @Left, and PRINT writes the order number and that count.

**4 Otherwise, print the refusal**

Lines 19 to 22

The sentence from the stored procedure is printed as it is.

Click New Query, press the Copy button under this step, and paste the test into the empty window.

**You should see** The test is in a new query window of its own, with your script still open in its tab.

Press F5 to run the test.

> **You should see** The Messages pane says (1 row affected), then: Ordered. New order number 5, expected on a date a week from today. Write the order number down.

Click New Query, press the Copy button under this step, paste the test, change the 5 on the SET @ID line to your order number if it is different, and press F5.

The first run closes the order and adds ten to the count in one stored procedure. The second run finds no open order with that number, so it refuses.

1. Two rows changed: the order and the part.

2. The second run is refused. The count stays at 15.

> **You should see** The Messages pane says (1 row affected) twice, then: Received order number 5. Stock of part 5 is now 15. Two rows changed in one stored procedure: the order and the part.

Press F5 again, changing nothing.

> **You should see** The Messages pane says: That order is not open. The order was received already, so the count stays at 15.

STAGE 4 OF 10 · ORDERS IN: PLACE, RECEIVE, CANCEL, AND THE REORDER LIST · SCREEN 4 OF 5

# Read the three stored procedures that cancel and list orders, then add them.

Three short ones. Cancel is the same shape as the refund: find the open row, refuse if there is none, change its status. The two lists feed the lower grid on the parts screen and the report in chapter 9.

Read this one **usp_PartOrder_Cancel: mark an open order Cancelled** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
CREATE OR ALTER PROCEDURE usp_PartOrder_Cancel
    @PartOrderID INT,
    @Message VARCHAR(200) OUTPUT
AS
BEGIN
    SET @Message = '';
    IF NOT EXISTS (SELECT * FROM PartOrder WHERE PartOrderID = @PartOrderID
        AND Status = 'Ordered')
    BEGIN
        SET @Message = 'Only an open order can be cancelled.';
        RETURN;
    END
    UPDATE PartOrder SET Status = 'Cancelled' WHERE PartOrderID = @PartOrderID
        AND Status = 'Ordered';
END
GO
```

**1 The name, and what goes in and out**

Lines 1 to 3

The order number goes in, and @Message comes back out.

**2 Start with a blank message**

Lines 4 to 6

AS and BEGIN mark where the work starts. The first job is to make @Message empty. An empty message means it worked. Every refusal below puts a sentence into @Message instead, so the program always knows which it was.

**3 Refuse if the order is not open**

Lines 7 to 12

Only a row whose status is still Ordered can be cancelled. A received order has already changed the count, so cancelling it would be wrong.

**4 Mark it Cancelled**

Lines 13 to 16

The row stays with its status changed, so the shop can see what was ordered and never came. GO ends the batch, ready for the next stored procedure.

Read this one **usp_PartOrder_GetOpen: the orders still waiting for a delivery** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  CREATE OR ALTER PROCEDURE usp_PartOrder_GetOpen
2  AS
3  BEGIN
4      SELECT o.PartOrderID, p.PartCode, p.Name AS Part, s.Name AS Supplier,
5             o.QtyOrdered, o.OrderedOn, o.ExpectedOn,
6             st.FirstName + ' ' + st.LastName AS PlacedBy
7      FROM PartOrder o
8      JOIN Part     p  ON p.PartID      = o.PartID
9      JOIN Supplier s  ON s.SupplierID  = p.SupplierID
10     JOIN Staff    st ON st.StaffID    = o.PlacedByStaffID
11     WHERE o.Status = 'Ordered'
12     ORDER BY o.ExpectedOn, p.Name;
13 END
14 GO
```

**1 The name, and AS and BEGIN**

Lines 1 to 3

This stored procedure only reads, so it has no @Message. AS and BEGIN mark where the work starts.

**2 What comes back**

Lines 4 to 6

Each row is one open order: the part code and name, the supplier, how many, when it was ordered and expected, and who placed it, as a name.

**3 Where the rows come from**

Lines 7 to 14

Four tables joined, only the rows whose status is Ordered, soonest expected first. GO ends the batch, ready for the next stored procedure.

Read this one **usp_PartOrder_GetAll: every order, open or closed** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  CREATE OR ALTER PROCEDURE usp_PartOrder_GetAll
2  AS
3  BEGIN
4      SELECT o.PartOrderID, p.PartCode, p.Name AS Part, s.Name AS Supplier,
5             o.QtyOrdered, o.OrderedOn, o.ExpectedOn, o.ReceivedOn, o.QtyReceived,
6                 o.Status
7      FROM PartOrder o
8      JOIN Part     p ON p.PartID     = o.PartID
9      JOIN Supplier s ON s.SupplierID = p.SupplierID
10     ORDER BY o.OrderedOn DESC, o.PartOrderID DESC;
11 END
12 GO
```

**1 The name, and AS and BEGIN**

Lines 1 to 3

This stored procedure only reads, so it has no @Message. AS and BEGIN mark where the work starts.

**2 What comes back**

Lines 4 to 6

Each row has the part code and name, the supplier, how many were ordered, and when it was ordered and expected, plus the day the box came, how many were in it, and the status. It does not say who placed the order.

**3 Where the rows come from**

Lines 7 to 12

Three tables are joined, every row is kept, and the newest come first. GO ends the batch, ready for the next stored procedure.

Press the Copy button under this step to copy usp_PartOrder_Cancel, click on the empty line above the line that prints BikeShop is ready, and paste.

> **You should see** usp_PartOrder_Cancel is in your script, ending with GO, and the line that prints BikeShop is ready is still last.

Press the Copy button under this step to copy usp_PartOrder_GetOpen, click on the empty line above the line that prints BikeShop is ready, and paste.

> **You should see** usp_PartOrder_GetOpen is in your script, under usp_PartOrder_Cancel.

Press the Copy button under this step to copy usp_PartOrder_GetAll, click on the empty line above the line that prints BikeShop is ready, and paste.

> **You should see** All three are in your script, each ending with GO, and the line that prints BikeShop is ready is still last.

Select all the lines you added on this screen, from the first CREATE OR ALTER PROCEDURE down to the last GO, and press F5.

> **You should see** The Messages pane says: Commands completed successfully. Your test sales and orders are still there.

**IF IT GOES WRONG** The Messages pane shows (6 rows affected) and BikeShop is ready

Nothing was selected, so the whole script ran and the sample data was put back. Your test sales, orders and reservations are gone. Nothing is broken: carry on, and remember that the stock of part 5 is 5 again.

Copy this one **A test that places an order, cancels it, and tries to cancel it again** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
-- Place an order for 6 of part 5, cancel it, then try to cancel it again.
-- Then list the orders that are still open.
DECLARE @Expected DATE, @NewID INT, @Message VARCHAR(200);
SET @Expected = DATEADD(DAY, 7, CAST(GETDATE() AS DATE));

EXEC usp_PartOrder_Place
     @PartID = 5, @QtyOrdered = 6, @ExpectedOn = @Expected,
     @PlacedByStaffID = 1,
     @NewPartOrderID = @NewID OUTPUT, @Message = @Message OUTPUT;
PRINT 'Placed order number ' + CAST(@NewID AS VARCHAR) + '.';

EXEC usp_PartOrder_Cancel @PartOrderID = @NewID, @Message = @Message OUTPUT;
IF @Message = ''
BEGIN
    PRINT 'Cancelled order number ' + CAST(@NewID AS VARCHAR) + '.';
END
ELSE
BEGIN
    PRINT @Message;
END

EXEC usp_PartOrder_Cancel @PartOrderID = @NewID, @Message = @Message OUTPUT;
IF @Message = ''
BEGIN
    PRINT 'The second cancel got through: the rule is broken.';
END
ELSE
BEGIN
    PRINT 'Second cancel refused: ' + @Message;
END

EXEC usp_PartOrder_GetOpen;
```

**1 A week from today**

Lines 3 to 4

The test has the same three boxes as the order test.

**2 Place an order for six**

Lines 6 to 10

A fresh order, so that the test does not depend on your numbers. Its number is printed.

**3 Cancel it**

Lines 12 to 20

The first cancel finds the open row and changes its status.

**4 Cancel it again**

Lines 22 to 30

The second cancel must be refused. If it got through, the test says so in plain words.

**5 Then list the open orders**

Line 32

The cancelled order is not in the list. Order 3 from the sample data, and your order from the last screen if you did not receive it, are.

Click New Query, press the Copy button under this step, and paste the test into the empty window.

> **You should see** The test is in a new query window of its own, with your script still open in its tab.

Press F5 to run the test.

> **You should see** The Messages pane says: Placed order number 6. Cancelled order number 6. Second cancel refused: Only an open order can be cancelled. The Results tab shows the open orders, without number 6.

STAGE 4 OF 10 · ORDERS IN: PLACE, RECEIVE, CANCEL, AND THE REORDER LIST · SCREEN 5 OF 5

# Read the stored procedure that makes the reorder list, then add it with the three part stored procedures.

The reorder list is the point of counting. A part belongs on it when its count is at or below its reorder level, and there is not already an open order for it. That second half is a NOT EXISTS, the same shape as the clash test in chapter 7. Without it, the shop would order the same box twice.

Copy this one **usp_Part_GetReorderList: the parts that need ordering, with the supplier to …** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
CREATE OR ALTER PROCEDURE usp_Part_GetReorderList
AS
BEGIN
    -- Parts at or below their reorder level that have no order on the way.
    SELECT p.PartID, p.PartCode, p.Name, s.Name AS Supplier, s.Phone
        AS SupplierPhone,
           p.QtyInStock, p.ReorderLevel, p.ReorderQty
    FROM Part p
    JOIN Supplier s ON s.SupplierID = p.SupplierID
    WHERE p.QtyInStock ⩽ p.ReorderLevel
      AND NOT EXISTS (SELECT * FROM PartOrder o WHERE o.PartID = p.PartID
          AND o.Status = 'Ordered')
    ORDER BY s.Name, p.Name;
END
GO
```

**1 The name, and AS and BEGIN**

Lines 1 to 3

This stored procedure only reads, so it has no @Message. AS and BEGIN mark where the work starts.

**2 What comes back**

Lines 5 to 7

The result shows the part, its supplier's name and phone number, the count, the reorder level and the usual order amount. That is everything the counter staff need to ring the supplier.

**3 Where the rows come from**

Lines 8 to 9

Part joined to Supplier, so the phone number is on the same row as the part.

**4 Low, and no order on the way**

Lines 10 to 12

Two tests joined by AND. The count is at or below the level, and NOT EXISTS finds no PartOrder row for this part whose status is Ordered.

**5 Grouped by supplier**

Lines 13 to 15

Supplier first, then part, so one phone call covers every part from that supplier. GO ends the batch, ready for the next stored procedure.

Read this one **usp_Part_GetAll: every part, with its supplier** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  CREATE OR ALTER PROCEDURE usp_Part_GetAll
2  AS
3  BEGIN
4      SELECT p.PartID, p.PartCode, p.Name, s.Name AS Supplier,
5             p.UnitCost, p.SellPrice, p.QtyInStock, p.ReorderLevel, p.ReorderQty,
6             p.SupplierID
7      FROM Part p
8      JOIN Supplier s ON s.SupplierID = p.SupplierID
9      ORDER BY p.Name;
10 END
11 GO
```

**1 The name, and AS and BEGIN**

Lines 1 to 3

This stored procedure only reads, so it has no @Message. AS and BEGIN mark where the work starts.

**2 What comes back**

Lines 4 to 6

Every column the parts grid shows, plus SupplierID at the end so that the edit form knows which supplier to pick. Grid.Tidy hides it.

**3 Where the rows come from**

Lines 7 to 11

Part joined to Supplier, in name order. GO ends the batch, ready for the next stored procedure.

Read this one **usp_Part_Add: save a new part** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
CREATE OR ALTER PROCEDURE usp_Part_Add
    @PartCode     VARCHAR(12),
    @Name         VARCHAR(60),
    @SupplierID   INT,
    @UnitCost     DECIMAL(7,2),
    @SellPrice    DECIMAL(7,2),
    @QtyInStock   INT,
    @ReorderLevel INT,
    @ReorderQty   INT,
    @NewPartID    INT OUTPUT,
    @Message VARCHAR(200) OUTPUT
AS
BEGIN
    SET @Message = '';
    IF EXISTS (SELECT * FROM Part WHERE PartCode = @PartCode)
    BEGIN
        SET @Message = 'That part code is already in use.';
        RETURN;
    END

    INSERT INTO Part (PartCode, Name, SupplierID, UnitCost, SellPrice,
        QtyInStock, ReorderLevel, ReorderQty)
    VALUES (@PartCode, @Name, @SupplierID, @UnitCost, @SellPrice, @QtyInStock,
        @ReorderLevel, @ReorderQty);
    SET @NewPartID = SCOPE_IDENTITY();
END
GO
```

**1 The name, and what goes in and out**

Lines 1 to 11

Everything about the part, including the count it starts with. The new number and @Message come back out.

**2 Start with a blank message**

Lines 12 to 14

AS and BEGIN mark where the work starts. The first job is to make @Message empty. An empty message means it worked. Every refusal below puts a sentence into @Message instead, so the program always knows which it was.

**3 Refuse a part code that is in use**

Lines 15 to 19

The part code is the name on the shelf, so two parts cannot share one.

**4 Save the part**

Lines 21 to 24

One new row with every column filled in, including the starting count. This is the only time a count is typed in: after that, only sales, refunds and deliveries change it.

### 5 Hand back the new number

Lines 25 to 27

The number of the new row goes back through the OUTPUT parameter. GO ends the batch, ready for the next stored procedure.

Read this one **usp_Part_Update: change a part, but never its count** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
CREATE OR ALTER PROCEDURE usp_Part_Update
    @PartID       INT,
    @Name         VARCHAR(60),
    @SupplierID   INT,
    @UnitCost     DECIMAL(7,2),
    @SellPrice    DECIMAL(7,2),
    @ReorderLevel INT,
    @ReorderQty   INT,
    @Message VARCHAR(200) OUTPUT
AS
BEGIN
    SET @Message = '';
    -- No QtyInStock here on purpose: stock only moves through sales, deliveries
    -- and refunds.
    UPDATE Part
    SET Name = @Name, SupplierID = @SupplierID, UnitCost = @UnitCost,
        SellPrice = @SellPrice,
        ReorderLevel = @ReorderLevel, ReorderQty = @ReorderQty
    WHERE PartID = @PartID;
END
GO
```

### 1 The name, and what goes in and out

Lines 1 to 9

The part number and the things that can change. No part code, because the code never changes, and no count, on purpose.

### 2 Start with a blank message

Lines 10 to 12

AS and BEGIN mark where the work starts. The first job is to make @Message empty. An empty message means it worked. Every refusal below puts a sentence into @Message instead, so the program always knows which it was.

### 3 Change the row, except the count

Lines 15 to 21

QtyInStock is not in the SET list. An edit screen that could type a new count would make the count a guess. This is the rule from stage 1, written into the stored procedure. GO ends the batch, ready for the next stored procedure.

Press the Copy button under this step to copy usp_Part_GetReorderList, click on the empty line above the line that prints BikeShop is ready, and paste.

> **You should see** usp_Part_GetReorderList is in your script, ending with GO, and the line that prints BikeShop is ready is still last.

Press the Copy button under this step to copy usp_Part_GetAll, click on the empty line above the line that prints BikeShop is ready, and paste.

> **You should see** usp_Part_GetAll is in your script, under usp_Part_GetReorderList.

Press the Copy button under this step to copy usp_Part_Add, click on the empty line above the line that prints BikeShop is ready, and paste.

> **You should see** usp_Part_Add is in your script, under the two lists.

Press the Copy button under this step to copy usp_Part_Update, click on the empty line above the line that prints BikeShop is ready, and paste.

> **You should see** All four are in your script, each ending with GO, and the line that prints BikeShop is ready is still last.

Select all the lines you added on this screen, from the first CREATE OR ALTER PROCEDURE down to the last GO, and press F5.

> **You should see** The Messages pane says: Commands completed successfully. Your test sales and orders are still there.

**IF IT GOES WRONG** The Messages pane shows (6 rows affected) and BikeShop is ready

Nothing was selected, so the whole script ran and the sample data was put back. Your test sales, orders and reservations are gone. Nothing is broken: carry on, and remember that the stock of part 5 is 5 again.

Copy this one **A test that lists every part, then the reorder list** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  -- Every part, then only the parts that need reordering.
2  EXEC usp_Part_GetAll;
3  EXEC usp_Part_GetReorderList;
```

**1 Every part**

Line 2

The first table in the Results tab. Look at QtyInStock beside ReorderLevel.

**2 Only the parts to order**

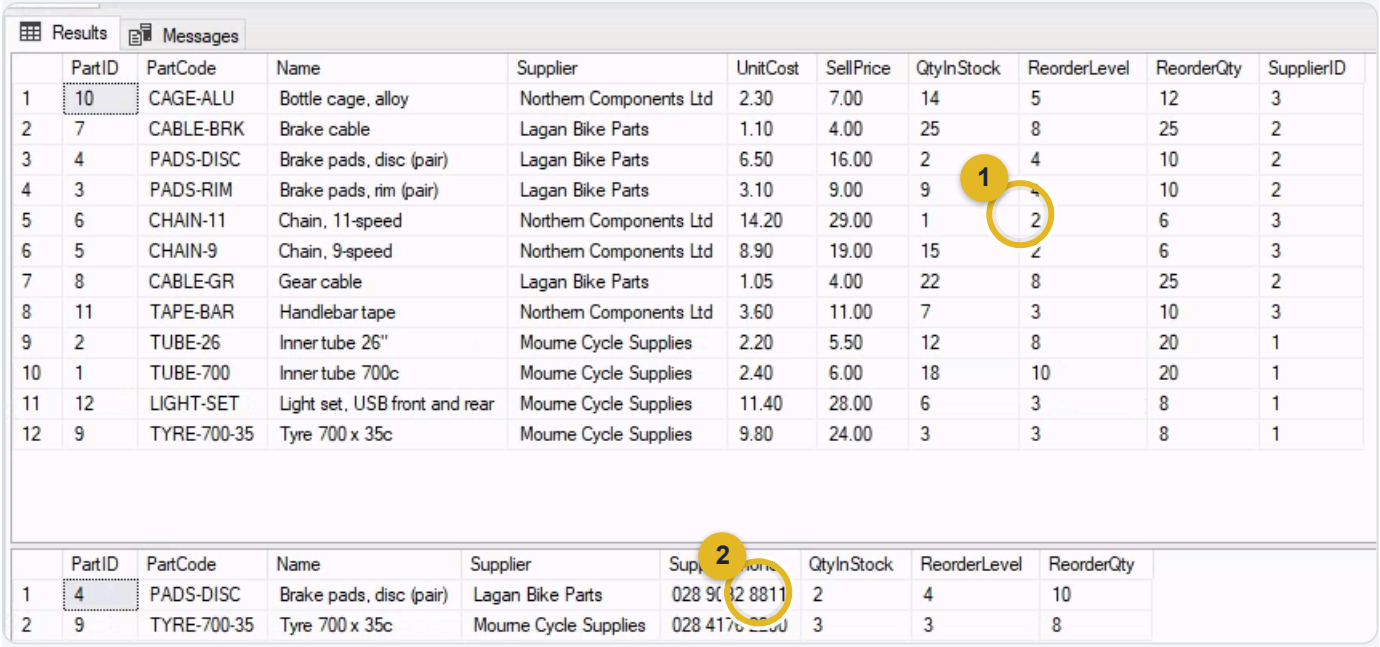
Line 3

The second table. Each row here is a phone call to make.

Click New Query, press the Copy button under this step, and paste the test into the empty window.

**You should see** The test is in a new query window of its own, with your script still open in its tab.

Press F5 to run the test.




| | PartID | PartCode | Name | Supplier | UnitCost | SellPrice | QtyInStock | ReorderLevel | ReorderQty | SupplierID |
|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 10 | CAGE-ALU | Bottle cage, alloy | Northern Components Ltd | 2.30 | 7.00 | 14 | 5 | 12 | 3 |
| 2 | 7 | CABLE-BRK | Brake cable | Lagan Bike Parts | 1.10 | 4.00 | 25 | 8 | 25 | 2 |
| 3 | 4 | PADS-DISC | Brake pads, disc (pair) | Lagan Bike Parts | 6.50 | 16.00 | 2 | 4 | 10 | 2 |
| 4 | 3 | PADS-RIM | Brake pads, rim (pair) | Lagan Bike Parts | 3.10 | 9.00 | 9 | 4 | 10 | 2 |
| 5 | 6 | CHAIN-11 | Chain, 11-speed | Northern Components Ltd | 14.20 | 29.00 | 1 | 2 | 6 | 3 |
| 6 | 5 | CHAIN-9 | Chain, 9-speed | Northern Components Ltd | 8.90 | 19.00 | 15 | 2 | 6 | 3 |
| 7 | 8 | CABLE-GR | Gear cable | Lagan Bike Parts | 1.05 | 4.00 | 22 | 8 | 25 | 2 |
| 8 | 11 | TAPE-BAR | Handlebar tape | Northern Components Ltd | 3.60 | 11.00 | 7 | 3 | 10 | 3 |
| 9 | 2 | TUBE-26 | Inner tube 26" | Mourne Cycle Supplies | 2.20 | 5.50 | 12 | 8 | 20 | 1 |
| 10 | 1 | TUBE-700 | Inner tube 700c | Mourne Cycle Supplies | 2.40 | 6.00 | 18 | 10 | 20 | 1 |
| 11 | 12 | LIGHT-SET | Light set, USB front and rear | Mourne Cycle Supplies | 11.40 | 28.00 | 6 | 3 | 8 | 1 |
| 12 | 9 | TYRE-700-35 | Tyre 700 x 35c | Mourne Cycle Supplies | 9.80 | 24.00 | 3 | 3 | 8 | 1 |

| | PartID | PartCode | Name | Supplier | Supplier phone | QtyInStock | ReorderLevel | ReorderQty |
|---|---|---|---|---|---|---|---|---|
| 1 | 4 | PADS-DISC | Brake pads, disc (pair) | Lagan Bike Parts | 028 90 32 8811 | 2 | 4 | 10 |
| 2 | 9 | TYRE-700-35 | Tyre 700 x 35c | Mourne Cycle Supplies | 028 417[illegible] | 3 | 3 | 8 |

Every part, then only the parts that need ordering. CHAIN-11 is low too (1 in stock, reorder level 2), but an order for it is already on the way, so it is left out.

1. Every part. QtyInStock beside ReorderLevel tells you which rows are low.

2. The reorder list: low stock and no open order. The supplier phone number is here so the shop can ring.

> **You should see** Two tables appear in the Results tab. The second has two rows: PADS-DISC with 2 in stock and a level of 4, and TYRE-700-35 with 3 and a level of 3. CHAIN-11 is at its level too, but order 3 is open for it, so it is left out.

# Add the Part class and its three data classes.

Now the program. One class for a part, and three data classes: one for parts, one for orders and one for sales. Each data class is the same shape as the ones from chapters 6 and 7: one small method for each stored procedure, and nothing else. The sales one has four lines you type, because a walk-in customer needs a NULL to reach the database.

**Copy this one** **Models/Part.cs: one part** C#

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  namespace BikeShop.Models
2  {
3      // One part we stock.  Plain properties only.
4      public class Part
5      {
6          public int PartID { get; set; }
7          public string PartCode { get; set; }
8          public string Name { get; set; }
9          public int SupplierID { get; set; }
10         public decimal UnitCost { get; set; }
11         public decimal SellPrice { get; set; }
12         public int QtyInStock { get; set; }
13         public int ReorderLevel { get; set; }
14         public int ReorderQty { get; set; }
15     }
16 }
```

**1 The class**

Lines 1 to 5

No using lines, because a part needs only whole numbers, words and money. It sits in the Models folder with Customer, Appointment and Hire.

**2 One property for each column**

Lines 6 to 16

The class has nine properties, one for each column of the Part table, with the same names. decimal is the type for money. The two closing braces end the class and the namespace.

Copy this one **Data/PartData.cs: the four part stored procedures** C#

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
using System.Data;
using System.Data.SqlClient;
using BikeShop.Models;

namespace BikeShop.Data
{
    public static class PartData
    {
        // Every part.
        public static DataTable GetAll()
        {
            return Db.GetTable("usp_Part_GetAll");
        }

        #region GetReorderList
        // The parts whose stock has fallen to the reorder level or below.
        public static DataTable GetReorderList()
        {
            return Db.GetTable("usp_Part_GetReorderList");
        }
        #endregion

        // Saves a new part.  Returns "" if it worked, or the sentence to show
        // the user.
        public static string Add(Part part)
        {
            SqlParameter newID = Db.Out("@NewPartID");
            string message = Db.Run("usp_Part_Add",
                Db.P("@PartCode", part.PartCode),
                Db.P("@Name", part.Name),
                Db.P("@SupplierID", part.SupplierID),
                Db.P("@UnitCost", part.UnitCost),
                Db.P("@SellPrice", part.SellPrice),
                Db.P("@QtyInStock", part.QtyInStock),
                Db.P("@ReorderLevel", part.ReorderLevel),
                Db.P("@ReorderQty", part.ReorderQty),
                newID);
            if (message == "")
            {
                part.PartID = (int)newID.Value;
            }
            return message;
        }

        // No QtyInStock here: stock only changes through sales, deliveries and
        // refunds.
        public static string Update(Part part)
        {
            return Db.Run("usp_Part_Update",
                Db.P("@PartID", part.PartID),
                Db.P("@Name", part.Name),
                Db.P("@SupplierID", part.SupplierID),
                Db.P("@UnitCost", part.UnitCost),
```

```
                Db.P("@SellPrice", part.SellPrice),
                Db.P("@ReorderLevel", part.ReorderLevel),
                Db.P("@ReorderQty", part.ReorderQty));
        }
    }
}
```

**1 The usings**

Lines 1 to 3

Each using line names a set of ready-made tools this file needs. System.Data.SqlClient is the one that talks to SQL Server. BikeShop.Models is where your own classes live.

**2 The class, and GetAll**

Lines 5 to 13

static means you never make a copy of this class: you write PartData.GetAll(). Db.GetTable runs a stored procedure and gives back a table of rows.

**3 GetReorderList: the parts to order**

Lines 15 to 21

GetReorderList has the same shape as GetAll, and it calls the reorder stored procedure. The #region and #endregion lines only fold the code up in Visual Studio 2019.

**4 Add: send the part, read the answer**

Lines 25 to 43

Db.Out makes an OUTPUT parameter. Db.Run runs the stored procedure and gives back @Message as a string. If it is empty, the new number is read out of newID and kept in the part object.

**5 Update: everything but the count**

Lines 47 to 59

Update sends seven parameters, and no QtyInStock, to match the stored procedure. The two closing braces end the class and the namespace.

Copy this one **Data/PartOrderData.cs: the five order stored procedures** C#

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
using System;
using System.Data;
using System.Data.SqlClient;

namespace BikeShop.Data
{
    public static class PartOrderData
    {
        // Orders more of a part from its supplier.  Returns "" if it worked.
        // The procedure always wants the @NewPartOrderID output parameter, but
        // this screen does not need the number.
        public static string Place(int partID, int qtyOrdered,
            DateTime expectedOn, int placedByStaffID)
        {
            SqlParameter newID = Db.Out("@NewPartOrderID");
            return Db.Run("usp_PartOrder_Place",
                Db.P("@PartID", partID),
                Db.P("@QtyOrdered", qtyOrdered),
                Db.P("@ExpectedOn", expectedOn.Date),
                Db.P("@PlacedByStaffID", placedByStaffID),
                newID);
        }

        #region Receive
        // One call: the order closes AND the stock goes up.
        public static string Receive(int partOrderID, int qtyReceived)
        {
            return Db.Run("usp_PartOrder_Receive",
                Db.P("@PartOrderID", partOrderID),
                Db.P("@QtyReceived", qtyReceived));
        }
        #endregion

        // Cancels an order that has not arrived.
        public static string Cancel(int partOrderID)
        {
            return Db.Run("usp_PartOrder_Cancel",
                Db.P("@PartOrderID", partOrderID));
        }

        // The orders still waiting for a delivery.
        public static DataTable GetOpen()
        {
            return Db.GetTable("usp_PartOrder_GetOpen");
        }

        // Every order, open or closed.
        public static DataTable GetAll()
        {
            return Db.GetTable("usp_PartOrder_GetAll");
        }
    }
```

```
53  }
```

**1 The usings**

Lines 1 to 3

using System gives the file DateTime, for the expected date. The other two talk to SQL Server.

**2 The class**

Lines 5 to 8

PartOrderData is a static class in the Data folder, like the others.

**3 Place: send the order**

Lines 12 to 22

The stored procedure insists on its @NewPartOrderID output parameter, so one is made and passed, and then ignored, because the screen does not need the number. expectedOn.Date throws away the clock part.

**4 Receive: one call, two changes**

Lines 24 to 32

Receive takes the order number and the quantity that arrived. The stored procedure closes the order and adds to the count in one go.

**5 Cancel**

Lines 35 to 39

Cancel takes one number and gives back the answer.

**6 GetOpen: the lower grid**

Lines 42 to 45

GetOpen gives back the orders still waiting for a delivery, as a table.

**7 GetAll: used in chapter 9**

Lines 48 to 53

GetAll gives back every order, for the report. The two closing braces end the class and the namespace.

**Type this one** **Data/PartSaleData.cs: the three sale stored procedures** C#

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  using System;
2  using System.Data;
3  using System.Data.SqlClient;
4
5  namespace BikeShop.Data
6  {
7      public static class PartSaleData
8      {
9          #region Add
10         // customerID 0 means a walk-in customer, who is saved as NULL.  The
11         // procedure refuses the sale if stock is short.
12         // Returns "" if it worked.  The procedure always wants the
13         // @NewPartSaleID output parameter, but this screen does not need the
14         // number.
15         public static string Add(int partID, int customerID, int quantity,
16             int soldByStaffID)
17         {
18             object customer = DBNull.Value;
19             if (customerID > 0)
20             {
21                 customer = customerID;
22             }
23             SqlParameter newID = Db.Out("@NewPartSaleID");
24             return Db.Run("usp_PartSale_Add",
25                 Db.P("@PartID", partID),
26                 Db.P("@CustomerID", customer),
27                 Db.P("@Quantity", quantity),
28                 Db.P("@SoldByStaffID", soldByStaffID),
29                 newID);
30         }
31         #endregion
32
33         // Takes a sale back and puts the parts back in stock.
34         public static string Refund(int partSaleID)
35         {
36             return Db.Run("usp_PartSale_Refund",
37                 Db.P("@PartSaleID", partSaleID));
38         }
39
40         // The sales between two dates.
41         public static DataTable GetByDateRange(DateTime fromDate,
42             DateTime toDate)
43         {
44             return Db.GetTable("usp_PartSale_GetByDateRange",
45                 Db.P("@FromDate", fromDate.Date), Db.P("@ToDate", toDate.Date));
46         }
47     }
48 }
```

**1 The usings**

Lines 1 to 3

using System gives the file DBNull and DateTime.

**2 The class**

Lines 5 to 8

PartSaleData is a static class in the Data folder, like the others.

**3 Add: send the sale, with NULL for a walk-in customer**

Lines 9 to 31

object customer starts as DBNull.Value, which is how a NULL travels to SQL Server. If a real customer number was given, it replaces the NULL. The rest is the usual Db.Run, with the output parameter made and ignored.

**4 Refund**

Lines 34 to 38

Refund takes one number and gives back the answer.

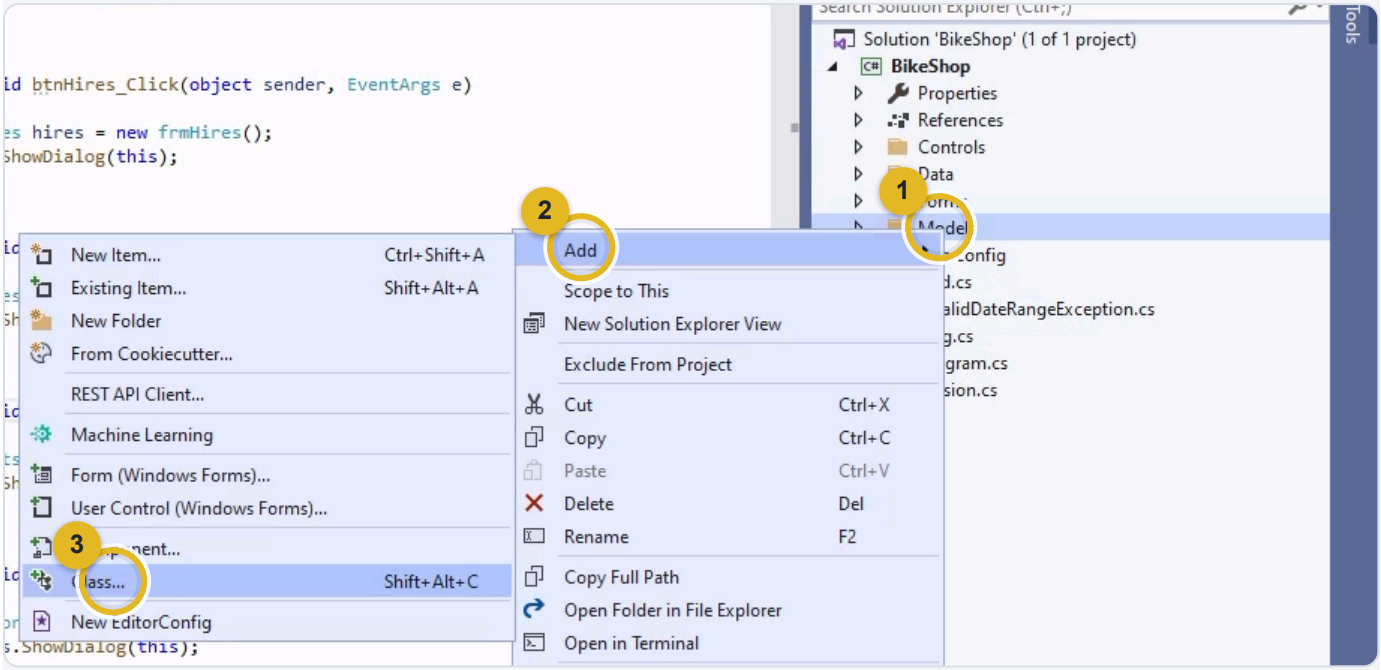
**5 GetByDateRange: used in chapter 9**

Lines 41 to 48

GetByDateRange takes two dates and gives back the sales between them. .Date throws away the clock part of each. The two closing braces end the class and the namespace.

**Why you type it** A walk-in customer is 0 in the program and NULL in the database. These four lines are the translation, and you will need the same trick for any column that can be empty in your own project.

In Visual Studio 2019, in Solution Explorer, right-click the Models folder, point at Add, and click Class…. Type Part.cs in the Name box and press Add.




Adding a class to the Models folder. Solution Explorer lists the classes you made in chapters 5, 6 and 7.

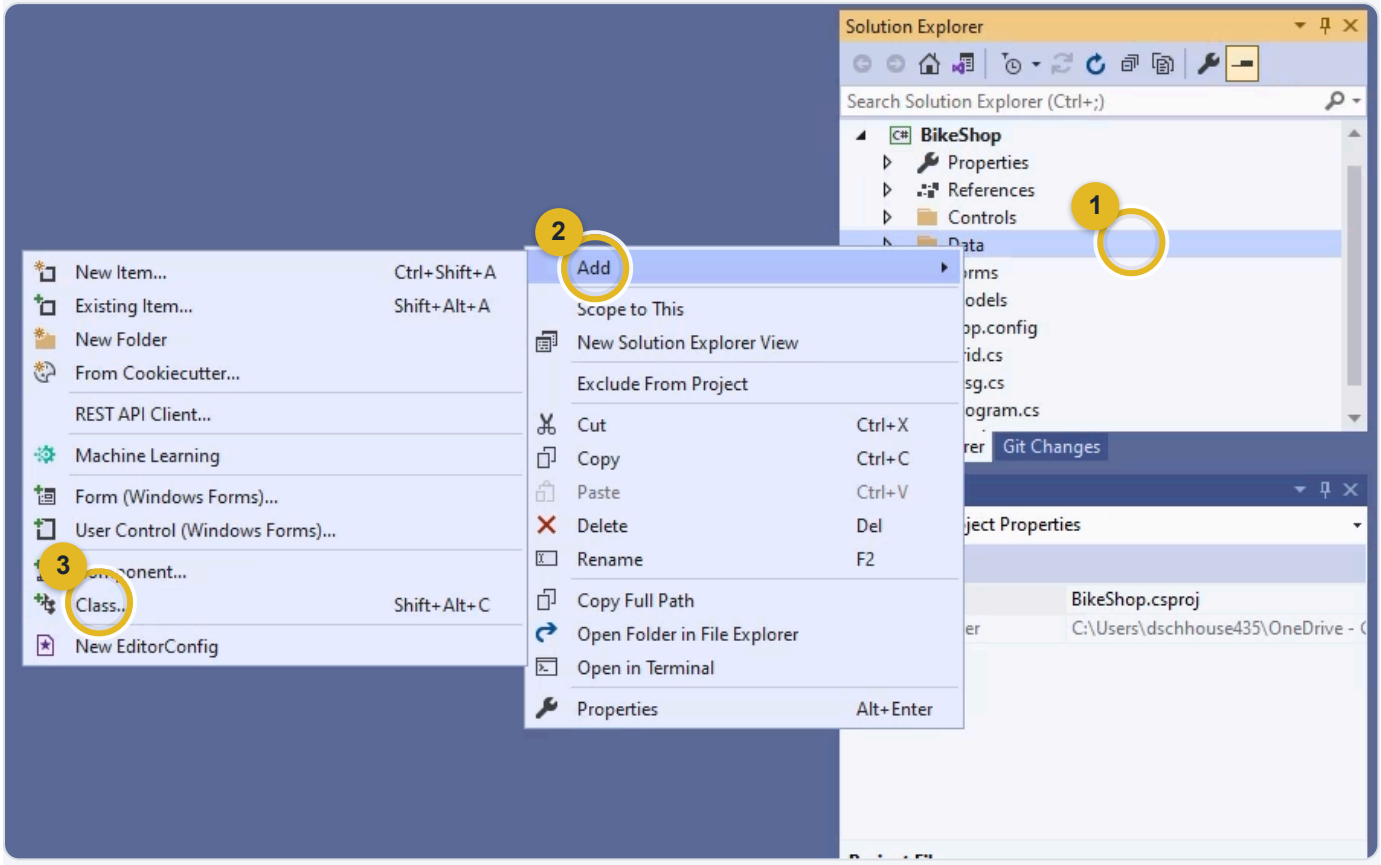
1. Right-click the Models folder in Solution Explorer.
2. Point at Add.
3. Click Class…

> **You should see** A new file called Part.cs opens with an empty class in it.

Click in the new file, press Ctrl and A together to select everything, press the Copy button under this step, and paste over it.

> **You should see** Part.cs holds the Part class with its nine properties, and no red wavy line anywhere.

In Solution Explorer, right-click the Data folder, point at Add, and click Class…




Adding a class to the Data folder. This picture was taken in the finished project, so Solution Explorer lists files you have not made yet.

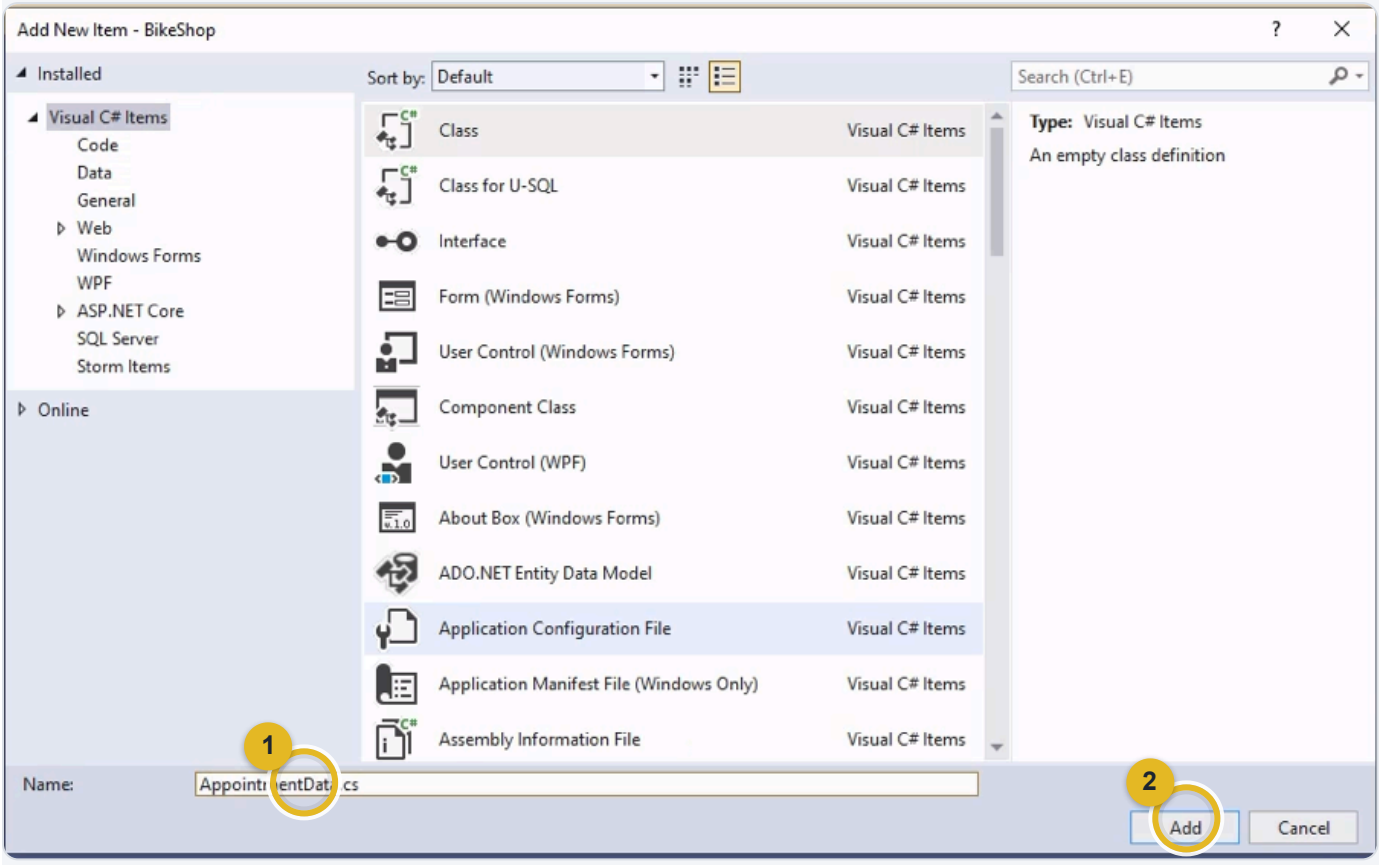
1. Right-click the Data folder in Solution Explorer.

2. Point at Add.

3. Click Class…

> **You should see** The Add New Item window opens with Class selected.

In the Name box at the bottom, type PartData.cs and press Add.



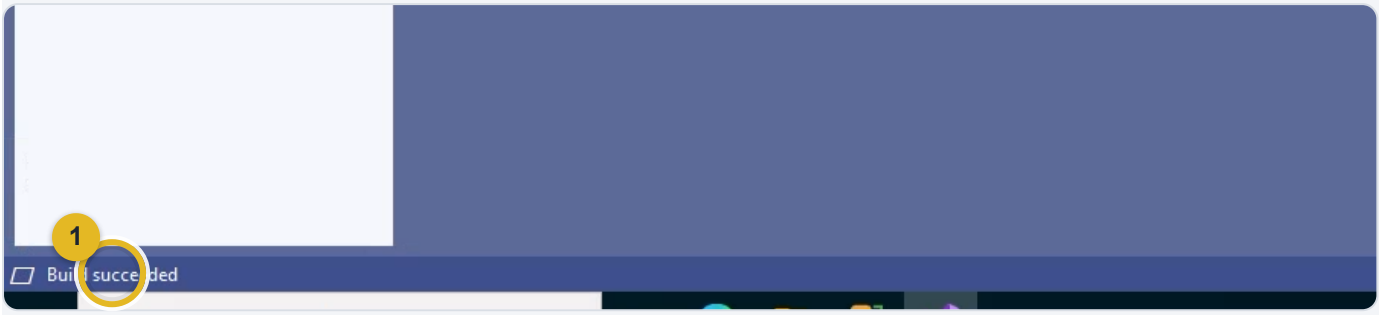

Name the new class before you press Add.

1. Type the name in the Name box, with .cs on the end.

2. Press Add.

**You should see** A new file opens with an empty class in it.

Click in the new file, press Ctrl and A together to select everything, press the Copy button under this step, and paste over it. Then press F6 to build.




Build succeeded, at the bottom left. Anything else means an error list to read.

1 Build succeeded. Press F6 to build at any time.

> **You should see** Build succeeded, at the bottom left.

**IF IT GOES WRONG** 'The name Db does not exist'

Your helper class from chapter 5 is called something else, or is in a different namespace. Match the name you used.

**IF IT GOES WRONG** 'The type or namespace name Part could not be found'

The Part class is missing from the Models folder. Go back to step 8.5.1 and add it.

Right-click the Data folder again, point at Add, and click Class…

> **You should see** The Add New Item window opens with Class selected.

In the Name box, type PartOrderData.cs and press Add.

> **You should see** A new file opens with an empty class in it.

Click in the new file, press Ctrl and A together to select everything, press the Copy button under this step, and paste over it.

> **You should see** PartOrderData.cs holds five methods, ending with GetAll, and no red wavy line anywhere.

Right-click the Data folder once more, point at Add, and click Class…

> **You should see** The Add New Item window opens with Class selected.

In the Name box, type PartSaleData.cs and press Add.

> **You should see** A new file opens with an empty class in it.

Click in the new file, press Ctrl and A together to select everything, press the Copy button under this step, and paste over it.

> **You should see** PartSaleData.cs holds the file, with one line that says YOU TYPE THIS PART.

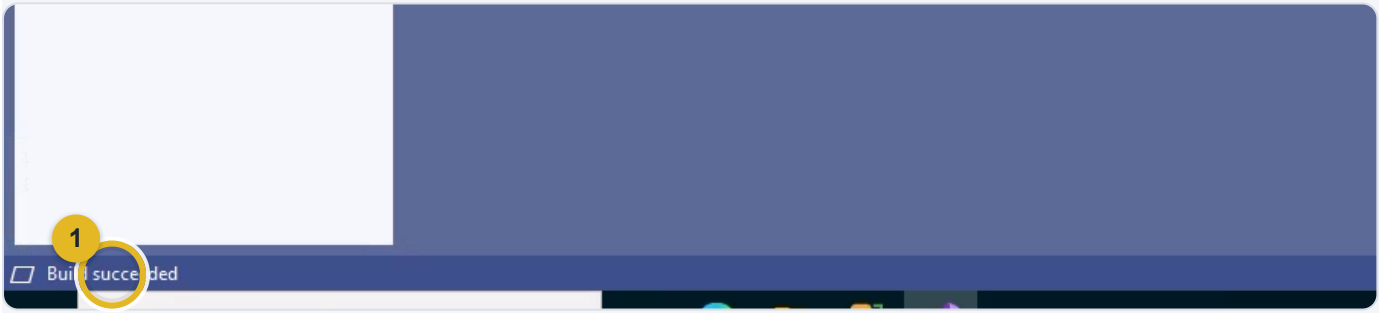
Click on the line that says YOU TYPE THIS PART, type the yellow lines in its place, and delete the marker line.

> **You should see** Four lines of your own, from object customer down to customer = customerID; and no marker line.

**IF IT GOES WRONG** A red wavy line under a word

A spelling slip. Compare your line with the yellow line, letter by letter, and check that each line ends with a semicolon where the yellow line does.

Press F6 to build.




Build succeeded, at the bottom left. Anything else means an error list to read.

1. Build succeeded. Press F6 to build at any time.

> **You should see** Build succeeded, at the bottom left.

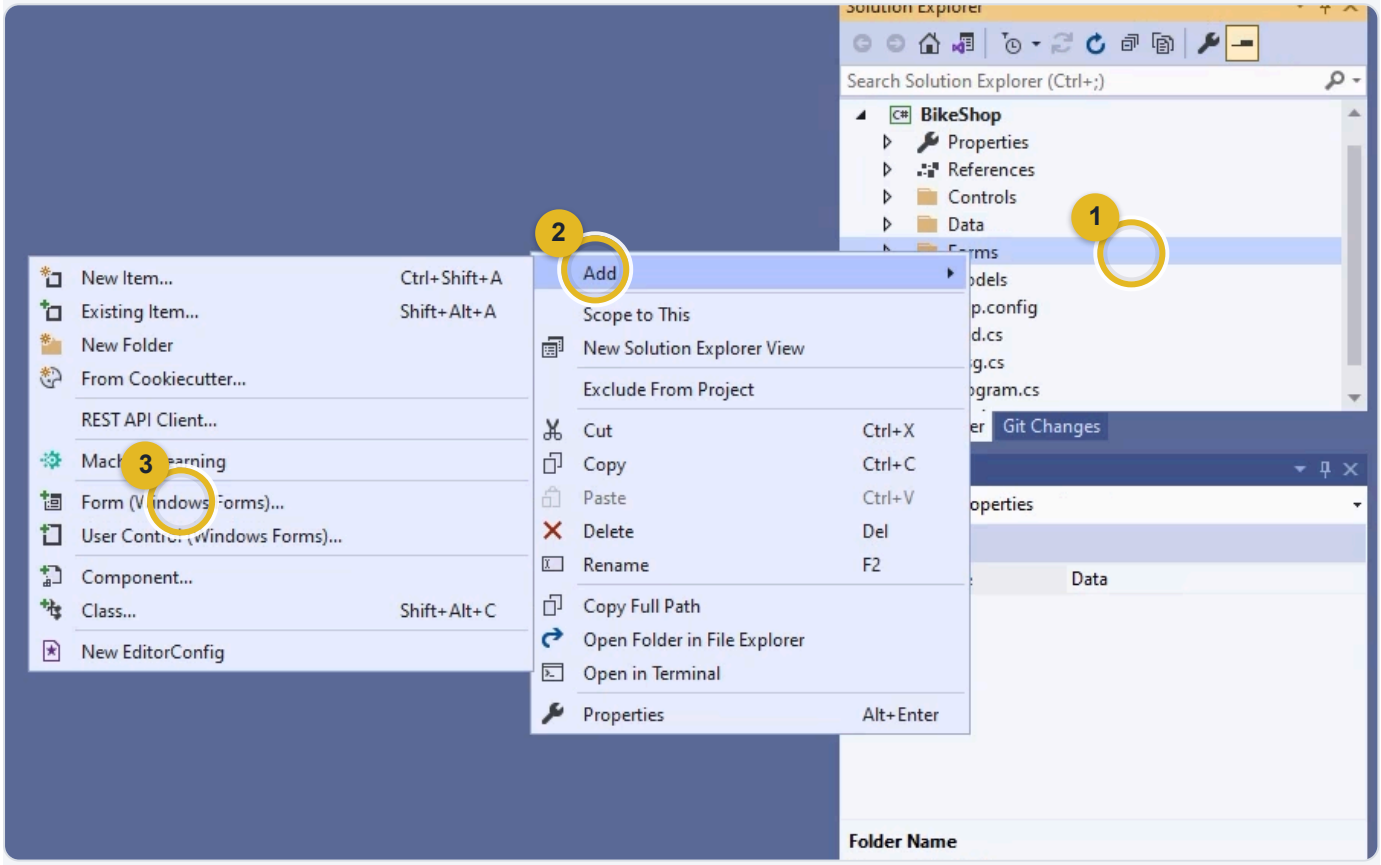
**IF IT GOES WRONG** 'The name DBNull does not exist'

The using System; line at the top is missing. Add it above the other using lines.

# Add the part edit form and set its controls.

Five forms go into this stage. The four small ones come first, because the big parts screen opens them, and Visual Studio 2019 cannot build a form that opens a form that does not exist yet. The first is the add-or-edit form for a part. It has a stock box that is only shown when the part is new, and a grey note that takes its place when the part is being edited.

In Solution Explorer, right-click the Forms folder, point at Add, and click Form (Windows Forms)…. Type frmPartEdit as the name and press Add.

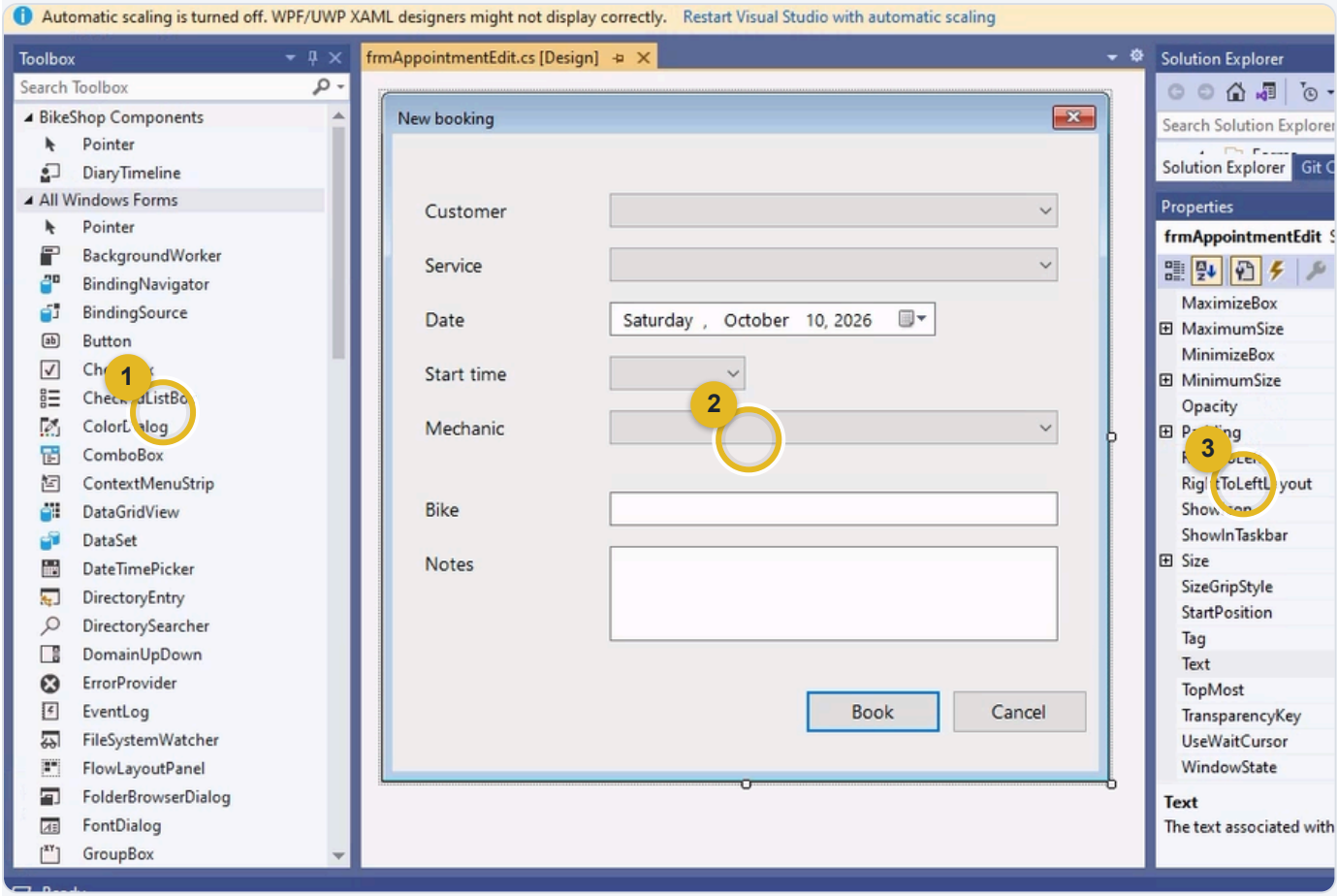



Adding a form to the Forms folder. Taken in the finished project, so Solution Explorer lists more files than yours.

1. Right-click the Forms folder.
2. Point at Add.
3. Click Form (Windows Forms)…, type the name, press Add.

**You should see** An empty form opens in the designer.

Open the Toolbox (View, then Toolbox) and the Properties window (View, then Properties Window), if they are not already open.




The three parts of the designer: the Toolbox you drag from, the form you drop on, the Properties window where you name and set each control.

1. The Toolbox. Drag a control from here on to the form.
2. The form. Drop the control where the picture shows it.
3. The Properties window. Set Name first, then the rest of the row.

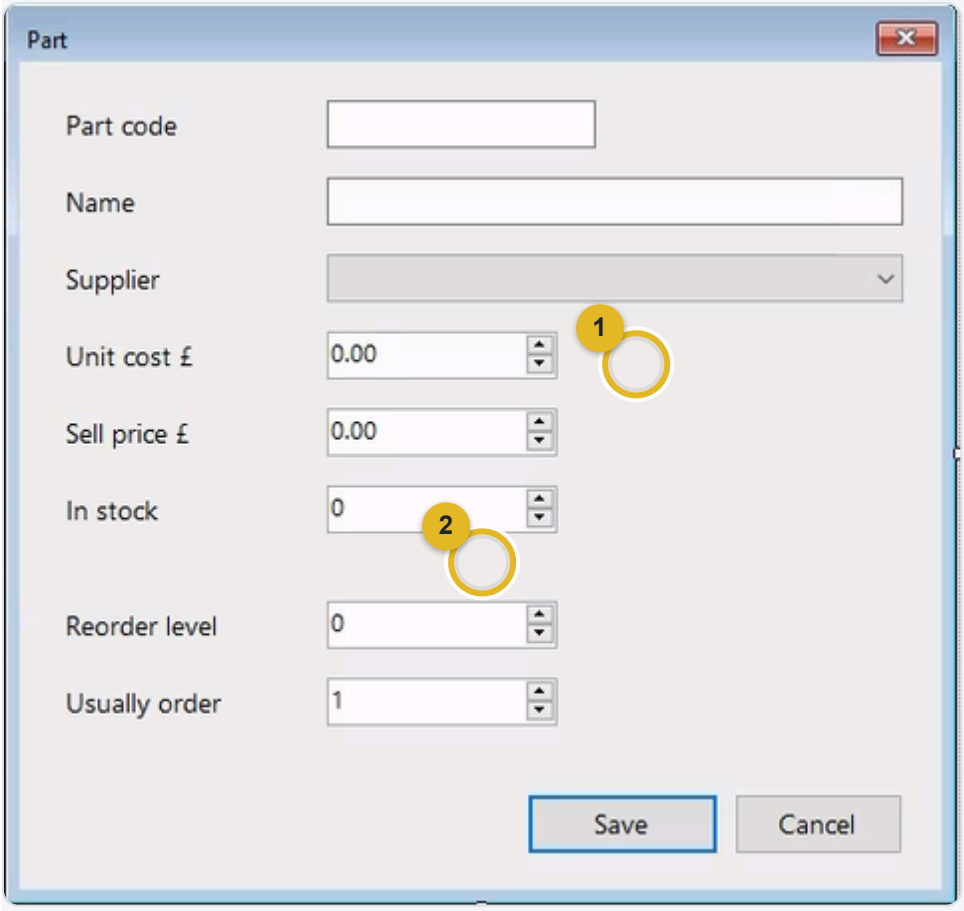
> **You should see** The Toolbox is on the left, the form in the middle, the Properties window on the right.

Click the form itself first and set the form's own properties. Then work down the table: drag the control from the Toolbox, drop it on the form, set its Name in the Properties window, and set the other settings in the row. Where the Then column says double-click, double-clicking the control makes its event method for you.

Add or edit a part. When editing, the stock box is hidden: stock only changes through sales, refunds and deliveries.

**Click the form itself, then set these in the Properties window**

| | | | |
|---|---|---|---|
| Text | `Part` | Size | `480, 430` |
| FormBorderStyle | `FixedDialog` | StartPosition | `CenterParent` |
| MaximizeBox | `False` | MinimizeBox | `False` |
| ShowInTaskbar | `False` | AcceptButton | `btnSave (after you add it)` |
| CancelButton | `btnCancel (after you add it)` | | |




frmPartEdit in the designer. The stock box is only used when a part is new.

1. Each box lines up with its label.
2. The grey note replaces the stock box when a part is being edited.

| DRAG THIS FROM THE TOOLBOX | NAME IT (THE NAME PROPERTY) | SET THESE IN THE PROPERTIES WINDOW | THEN |
|---|---|---|---|
| Label | `lblCode` | `Text = Part code` | |
| TextBox | `txtCode` | nothing else<br>Short and unique: "TUBE-700". | |
| Label | `lblName` | `Text = Name` | |
| TextBox | `txtName` | nothing else | |
| Label | `lblSupplier` | `Text = Supplier` | |
| ComboBox | `cboSupplier` | `DropDownStyle = DropDownList` | |
| Label | `lblUnitCost` | `Text = Unit cost £` | |
| NumericUpDown | `nudUnitCost` | `DecimalPlaces = 2`<br>`Maximum = 10000` | |
| Label | `lblSellPrice` | `Text = Sell price £` | |
| NumericUpDown | `nudSellPrice` | `DecimalPlaces = 2`<br>`Maximum = 10000` | |
| Label | `lblInStock` | `Text = In stock` | |
| NumericUpDown | `nudStock` | `Maximum = 100000`<br>Only for a new part. | |
| Label | `lblStock` | `ForeColor = grey (92, 103, 116)`<br>When editing: "Stock changes only through sales, refunds and deliveries." | |
| Label | `lblReorderLevel` | `Text = Reorder level` | |
| NumericUpDown | `nudReorderLevel` | `Maximum = 100000`<br>When stock falls to this, the row turns pink. | |
| Label | `lblReorderQty` | `Text = Usually order` | |
| NumericUpDown | `nudReorderQty` | `Maximum = 100000`<br>`Minimum = 1`<br>`Value = 1` | |
| Button | `btnSave` | `Text = Save` | Double-click it. Visual Studio makes `btnSave_Click` for you; the code goes inside it. |

| DRAG THIS FROM THE TOOLBOX | NAME IT (THE NAME PROPERTY) | SET THESE IN THE PROPERTIES WINDOW | THEN |
|---|---|---|---|
| Button | `btnCancel` | `Text = Cancel`<br>`DialogResult = Cancel` | |

# Put the code behind the part edit form.

Read the file, then put it in. There are two constructors: one for a new part, and one that takes the row the grid already holds, so that the form opens filled in. When the part already exists, the Load method hides the stock box and shows the note, because a count is never edited by hand.

Copy this one **Forms/frmPartEdit.cs: the whole file** C#

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
using System;
using System.Collections.Generic;
using System.Data;
using System.Drawing;
using System.Windows.Forms;
using BikeShop.Data;
using BikeShop.Models;

namespace BikeShop.Forms
{
    public partial class frmPartEdit : Form
    {
        // 0 = adding; otherwise the part being edited
        private readonly int partID;

        // Add a part
        public frmPartEdit()
        {
            InitializeComponent();
        }

        // Edit a part: the values come from the row the grid already holds
        public frmPartEdit(DataRow part)
        {
            InitializeComponent();
            partID = (int)part["PartID"];
            txtCode.Text = (string)part["PartCode"];
            txtName.Text = (string)part["Name"];
            nudUnitCost.Value = (decimal)part["UnitCost"];
            nudSellPrice.Value = (decimal)part["SellPrice"];
            nudReorderLevel.Value = (int)part["ReorderLevel"];
            nudReorderQty.Value = (int)part["ReorderQty"];
            // kept until the supplier list has loaded
            cboSupplier.Tag = part["SupplierID"];
        }

        private void frmPartEdit_Load(object sender, EventArgs e)
        {
            // The boxes stop typing at the length the database column holds.
            txtCode.MaxLength = 12;
            txtName.MaxLength = 60;

            cboSupplier.DisplayMember = "Name";
            cboSupplier.ValueMember = "SupplierID";
            cboSupplier.DataSource = SupplierData.GetAll();

            if (partID == 0)
            {
                Text = "Add a part";
                return;
            }

            Text = "Edit part";
```

```
            cboSupplier.SelectedValue = cboSupplier.Tag;
            // the code is the part's name on the shelf: it does not change
            txtCode.Enabled = false;
            lblStock.Text =
                "Stock changes only through sales, refunds and deliveries.";
            nudStock.Visible = false;
        }

        #region Save
        private void btnSave_Click(object sender, EventArgs e)
        {
            // Check what was typed, one problem at a time: show the first and
            // put the cursor there.
            // The number boxes need no check: none of them goes below 0, and
            // the reorder quantity starts at 1.
            if (txtCode.Text.Trim() == "")
            {
                Msg.Problem("Type the part code.");
                txtCode.Focus();
                return;
            }
            if (txtName.Text.Trim() == "")
            {
                Msg.Problem("Type the part's name.");
                txtName.Focus();
                return;
            }
            if (cboSupplier.SelectedValue == null)
            {
                Msg.Problem("Choose the part's supplier.");
                cboSupplier.Focus();
                return;
            }

            Part part = new Part();
            part.PartID = partID;
            part.PartCode = txtCode.Text.Trim().ToUpper();
            part.Name = txtName.Text.Trim();
            part.SupplierID = (int)cboSupplier.SelectedValue;
            part.UnitCost = nudUnitCost.Value;
            part.SellPrice = nudSellPrice.Value;
            part.QtyInStock = (int)nudStock.Value;
            part.ReorderLevel = (int)nudReorderLevel.Value;
            part.ReorderQty = (int)nudReorderQty.Value;

            string message;
            if (partID == 0)
            {
                message = PartData.Add(part);
            }
            else
            {
                message = PartData.Update(part);
            }
            if (message != "")
            {
                // e.g. that part code is already in use
                Msg.Problem(message);
```

```
112                 return;
113             }
114             DialogResult = DialogResult.OK;
115         }
116         #endregion
117     }
118 }
```

**1 The usings**

Lines 1 to 7

The tools this form needs: Windows Forms for the window itself, System.Data for the tables that fill the grids and lists, System.Drawing for colours, and your own Data and Models folders.

**2 The form’s class, one fact it remembers, and the first constructor**

Lines 9 to 20

partial means the designer keeps the other half of this class, the controls you dragged, in its own file. partID is 0 for a new part. The first constructor is for adding.

**3 The second constructor: editing**

Lines 23 to 35

The grid hands over the row it already holds, and each box is filled from it. The supplier number is parked in the list's Tag until the list has been filled, in Load.

**4 When the form loads**

Lines 37 to 60

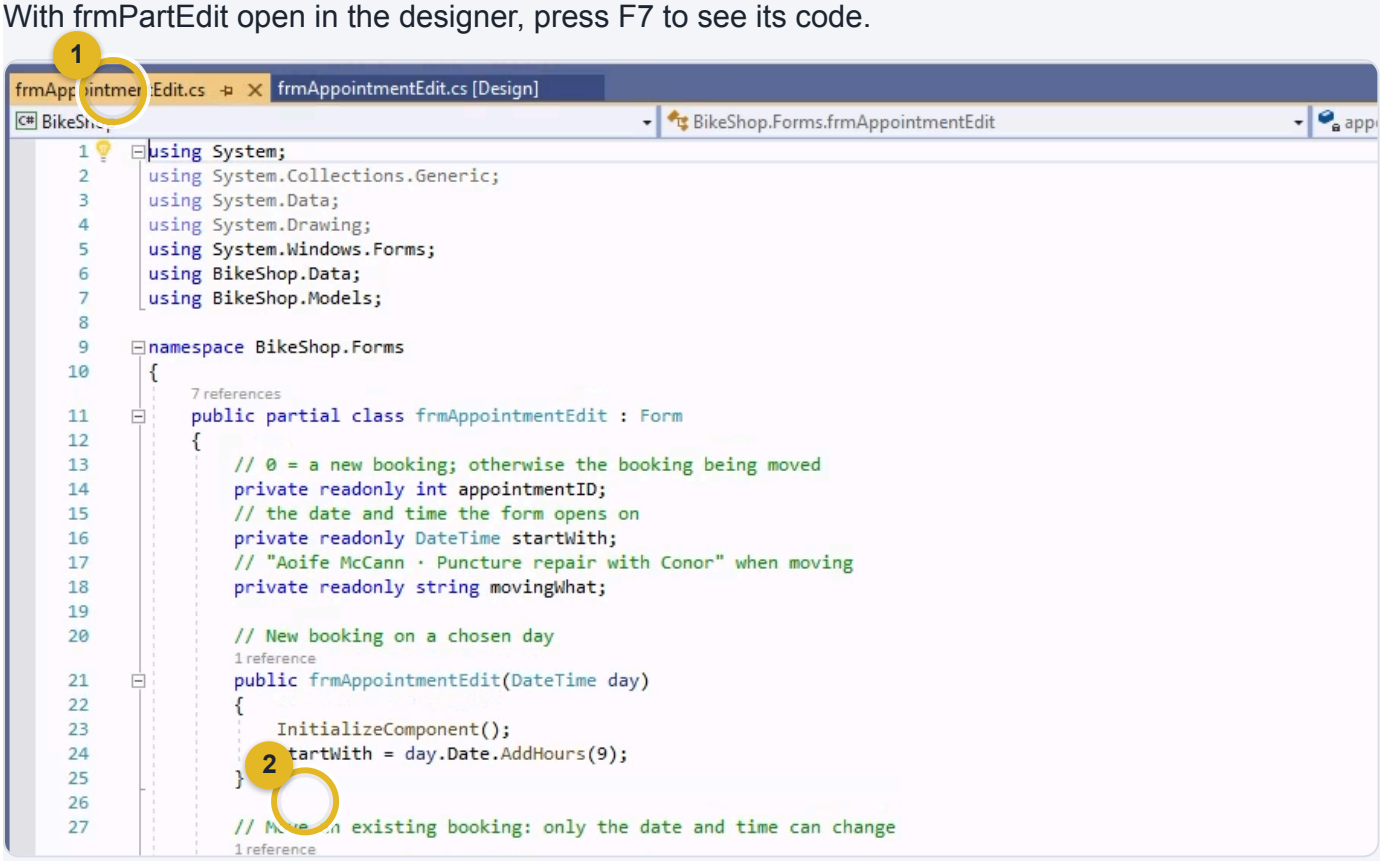
The boxes stop typing at the length the columns hold. The supplier list is filled, and then the form decides which it is: Add a part, or Edit part. Editing locks the code, hides the stock box, and shows the note.

**5 The Save button**

Lines 62 to 118

The Save button makes three checks, one problem at a time, each putting the cursor where the problem is. Then a Part object is filled from the boxes, and one of two data methods is called: Add for a new part, Update for an old one. The database decides, and the form shows what it said.

With frmPartEdit open in the designer, press F7 to see its code.




F7 shows the code behind the form that is open in the designer. Shift and F7 go back to the designer.

1. The code tab is in front.

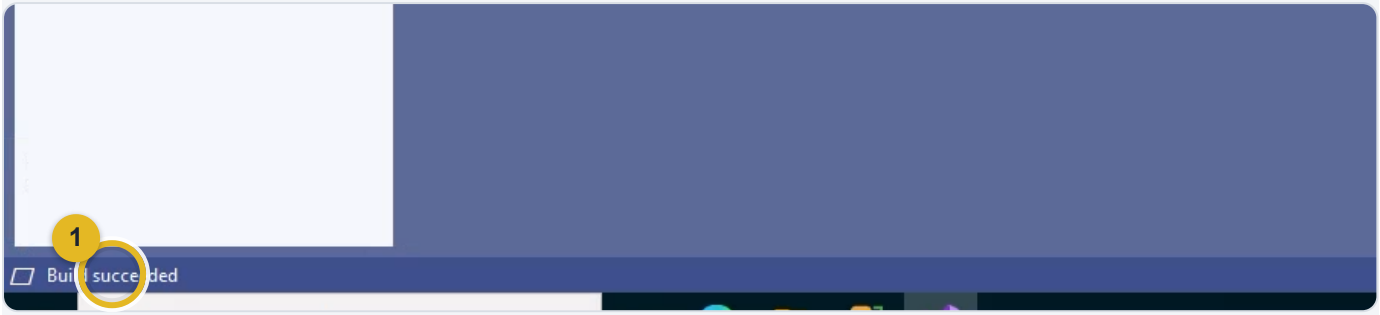
2. Select everything here before you paste over it.

> **You should see** The code tab for the form comes to the front.

Press Ctrl and A together to select everything in the code tab, press the Copy button under this step, and paste over it.

> **You should see** The code tab holds the whole file, and the old lines are gone.

Press F6 to build.




Build succeeded, at the bottom left. Anything else means an error list to read.

1 Build succeeded. Press F6 to build at any time.

**You should see** Build succeeded, at the bottom left.

**IF IT GOES WRONG** frmPartEdit_Load or btnSave_Click is underlined: no suitable method found

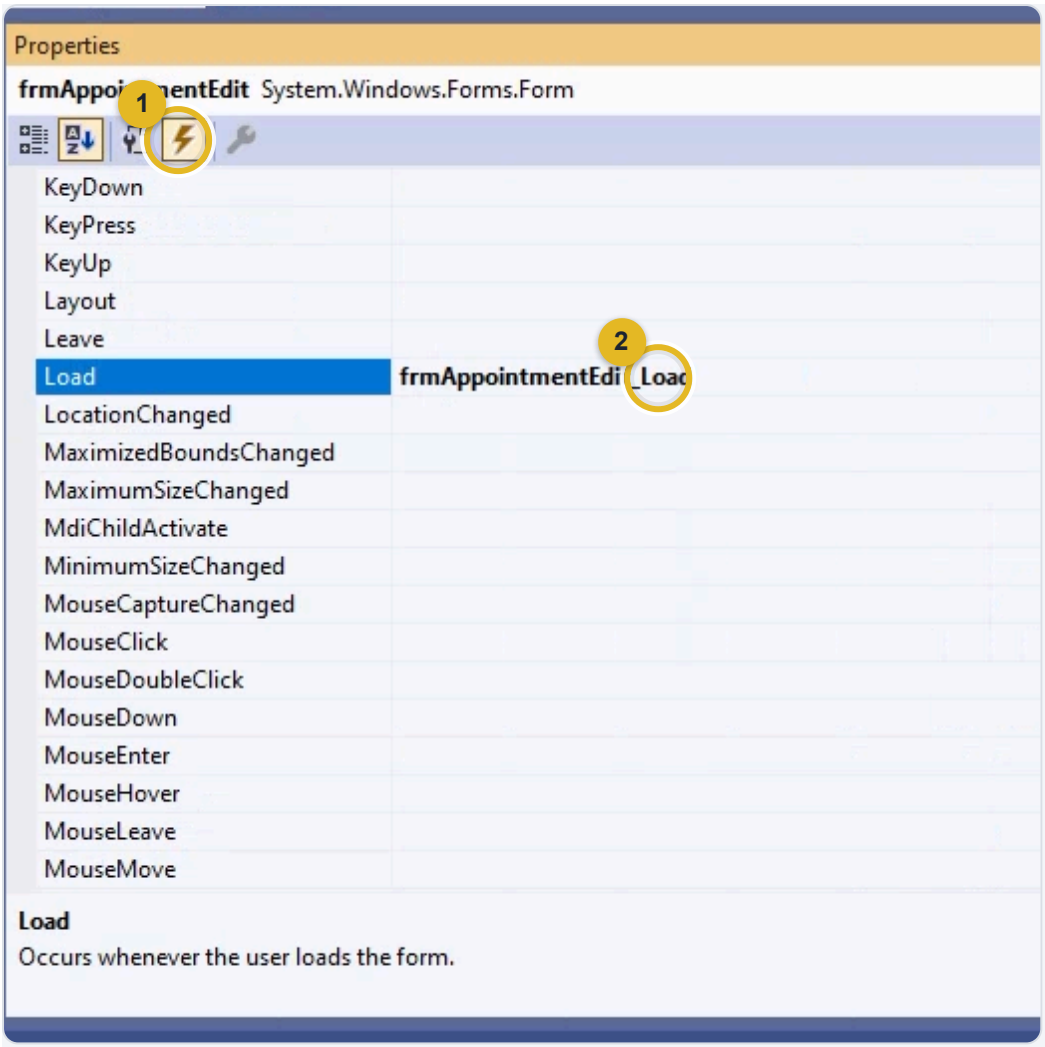
The designer did not make that event. Go back to the designer, click the form (or the button), open the Properties window, press the lightning bolt, and double-click the empty box beside the event. Then build again.

**IF IT GOES WRONG** 'The name Grid does not exist' or 'The name Msg does not exist'

Your grid and message helpers from chapter 6 are called something else, or are in a different namespace. Match the names you used.

**IF IT GOES WRONG** 'The name SupplierData does not exist'

Your supplier data class from chapter 6 is called something else. Match the name you used, and the name of its method that gives every supplier.



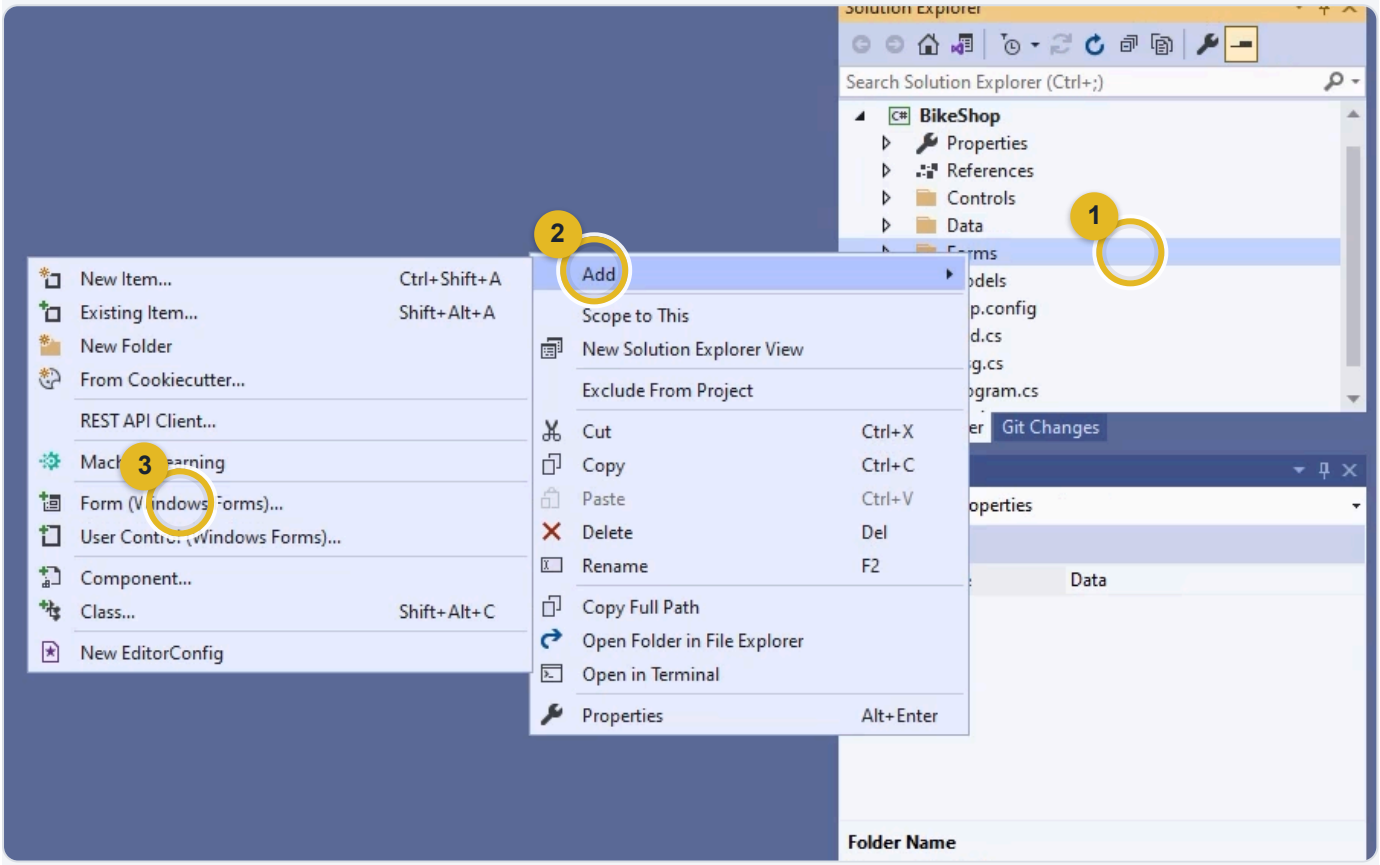

The lightning bolt shows the events, in alphabetical order. Double-click the empty box beside one to make its method.

1. Press the lightning bolt.
2. Double-click the empty box beside the event named in the table. Load is part of the way down, after Leave.

# Add the counter sale form and set its controls.

The counter sale form is small: the part and its count at the top, a Walk-in customer tick box that switches the customer list off, a quantity, and a Sell button.

In Solution Explorer, right-click the Forms folder, point at Add, and click Form (Windows Forms)…. Type frmPartSell as the name and press Add.




Adding a form to the Forms folder. Taken in the finished project, so Solution Explorer lists more files than yours.

1. Right-click the Forms folder.
2. Point at Add.
3. Click Form (Windows Forms)…, type the name, press Add.

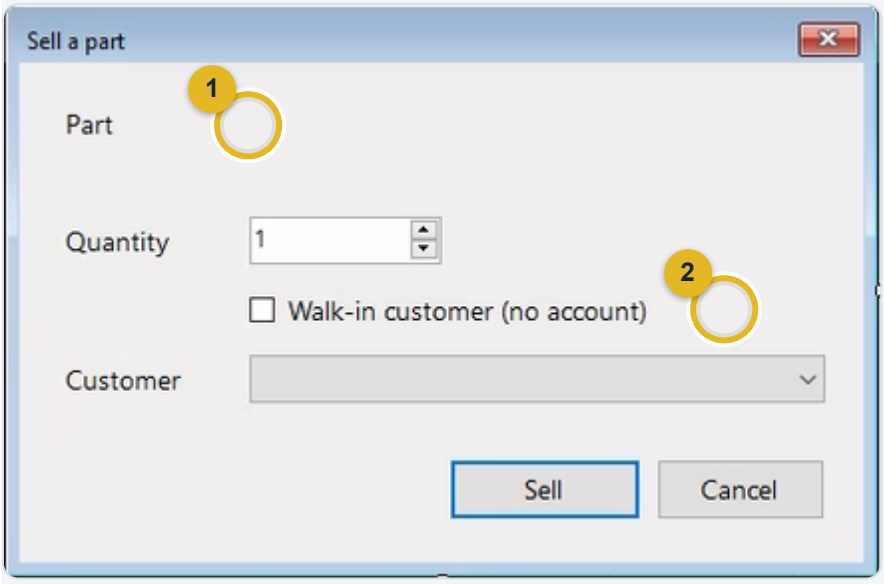
> **You should see** An empty form opens in the designer.

Click the form itself first and set the form's own properties. Then work down the table: drag the control from the Toolbox, drop it on the form, set its Name in the Properties window, and set the other settings in the row. Where the Then column says double-click, double-clicking the control makes its event method for you.

A counter sale. The database takes the stock off, and refuses if there is not enough.

**Click the form itself, then set these in the Properties window**

| | | | |
|---|---|---|---|
| Text | `Sell a part` | Size | `440, 260` |
| FormBorderStyle | `FixedDialog` | StartPosition | `CenterParent` |
| MaximizeBox | `False` | MinimizeBox | `False` |
| ShowInTaskbar | `False` | AcceptButton | `btnSell (after you add it)` |
| CancelButton | `btnCancel (after you add it)` | | |




frmPartSell in the designer. A small window for a counter sale.

1. The part name and In stock line are filled in when the form opens.
2. Walk-in customer ticked switches the customer list off.

| DRAG THIS FROM THE TOOLBOX | NAME IT (THE NAME PROPERTY) | SET THESE IN THE PROPERTIES WINDOW | THEN |
|---|---|---|---|
| Label | `lblPartCaption` | `Text = Part` | |
| Label | `lblPart` | nothing else<br>Filled in: the part being sold. | |
| Label | `lblInStock` | `ForeColor = grey (92, 103, 116)` | |
| Label | `lblQty` | `Text = Quantity` | |
| NumericUpDown | `nudQuantity` | `Maximum = 1000`<br>`Minimum = 1`<br>`Value = 1` | |
| CheckBox | `chkWalkIn` | `Text = Walk-in customer (no account)`<br>Ticked: the customer list is skipped. | Double-click it. Visual Studio makes `chkWalkIn_CheckedChanged` for you; the code goes inside it. |
| Label | `lblCustomer` | `Text = Customer` | |
| ComboBox | `cboCustomer` | `DropDownStyle = DropDownList` | |
| Button | `btnSell` | `Text = Sell` | Double-click it. Visual Studio makes `btnSell_Click` for you; the code goes inside it. |
| Button | `btnCancel` | `Text = Cancel`<br>`DialogResult = Cancel` | |

# Put the code behind the counter sale form, and type the part that calls the database.

The Sell button is where the stock guard reaches the screen. The form sends the sale and shows whatever sentence comes back. It never checks the count itself, because the count it was shown when it opened may already be out of date. You type that part.

**Type this one** **Forms/frmPartSell.cs: the whole file** C#

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
using System;
using System.Collections.Generic;
using System.Data;
using System.Drawing;
using System.Windows.Forms;
using BikeShop.Data;
using BikeShop.Models;

namespace BikeShop.Forms
{
    public partial class frmPartSell : Form
    {
        private readonly int partID;

        public frmPartSell(int id, string partName, int inStock)
        {
            InitializeComponent();
            partID = id;
            lblPart.Text = partName;
            lblInStock.Text = "In stock: " + inStock;
        }

        private void frmPartSell_Load(object sender, EventArgs e)
        {
            cboCustomer.DisplayMember = "FullName";
            cboCustomer.ValueMember = "CustomerID";
            cboCustomer.DataSource = CustomerData.Search("");

            chkWalkIn.Checked = true;
        }

        // Most counter sales are to someone without an account: tick the box
        // and the customer list is skipped.
        private void chkWalkIn_CheckedChanged(object sender, EventArgs e)
        {
            cboCustomer.Enabled = !chkWalkIn.Checked;
        }

        #region Sell
        private void btnSell_Click(object sender, EventArgs e)
        {
            // 0 = a walk-in sale, no customer
            int customerID = 0;
            if (!chkWalkIn.Checked)
            {
                if (cboCustomer.SelectedValue == null)
                {
                    Msg.Problem("Choose the customer, or tick Walk-in " +
                        "customer.");
                    cboCustomer.Focus();
                    return;
                }
                customerID = (int)cboCustomer.SelectedValue;
```

```
54              }
55
56              // The quantity box starts at 1, so it needs no check.
57              // The procedure takes the stock off and refuses if there is not
58              // enough: "Only 2 in stock."
59              string message = PartSaleData.Add(partID, customerID,
60                  (int)nudQuantity.Value, Session.StaffID);
61              if (message ≠ "")
62              {
63                  Msg.Problem(message);
64                  return;
65              }
66              DialogResult = DialogResult.OK;
67          }
68          #endregion
69      }
70  }
```

**1 The usings**

Lines 1 to 7

The tools this form needs: Windows Forms for the window itself, System.Data for the tables that fill the grids and lists, System.Drawing for colours, and your own Data and Models folders.

**2 The form's class and its constructor**

Lines 9 to 21

The parts screen hands over the part number, its name and its count. The number is kept, and the name and count are shown at the top of the form.

**3 When the form loads**

Lines 23 to 30

The customer list is filled from your customer search with an empty search, which gives everyone. Walk-in starts ticked, because most counter sales are to someone without an account.

**4 The tick box**

Lines 34 to 37

Ticked means the customer list is switched off. The exclamation mark means not.

**5 The Sell button: the part you type**

Lines 39 to 70

customerID starts at 0, which the data class turns into NULL. If Walk-in is not ticked, a customer must be chosen, and their number is used. Then one call to the database, and the message check.

**Why you type it** The call and the message check are the whole point. The database decides, and the form only shows what it said.

With frmPartSell open in the designer, press F7 to see its code.

**You should see** The code tab for the form comes to the front.

Press Ctrl and A together to select everything in the code tab, press the Copy button under this step, and paste over it.

> **You should see** The code tab holds the whole file, with one line that says YOU TYPE THIS PART.

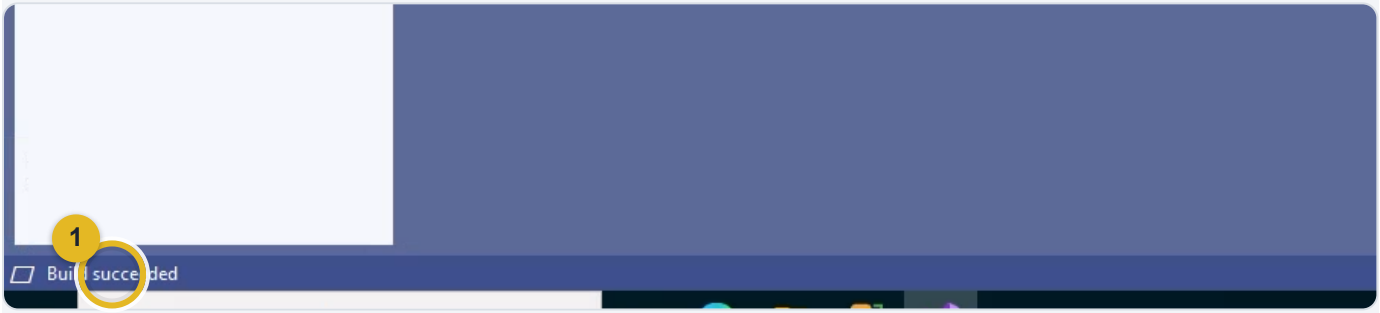
Click on the line that says YOU TYPE THIS PART, type the yellow lines in its place, and delete the marker line.

> **You should see** The yellow lines are in your file, and no line says YOU TYPE THIS PART any more.

**IF IT GOES WRONG** A red wavy line under a word

A spelling slip. Compare your line with the yellow line, letter by letter, and check that each line ends with a semicolon where the yellow line does.

Press F6 to build.




Build succeeded, at the bottom left. Anything else means an error list to read.

1. Build succeeded. Press F6 to build at any time.

> **You should see** Build succeeded, at the bottom left.

**IF IT GOES WRONG** frmPartSell_Load or btnSell_Click is underlined: no suitable method found

The designer did not make that event. Go back to the designer, click the form (or the button), open the Properties window, press the lightning bolt, and double-click the empty box beside the event. Then build again.

**IF IT GOES WRONG** 'The name Session does not exist'

Your sign-in class from chapter 5 is called something else. Match the name you used, and the name of its StaffID property.

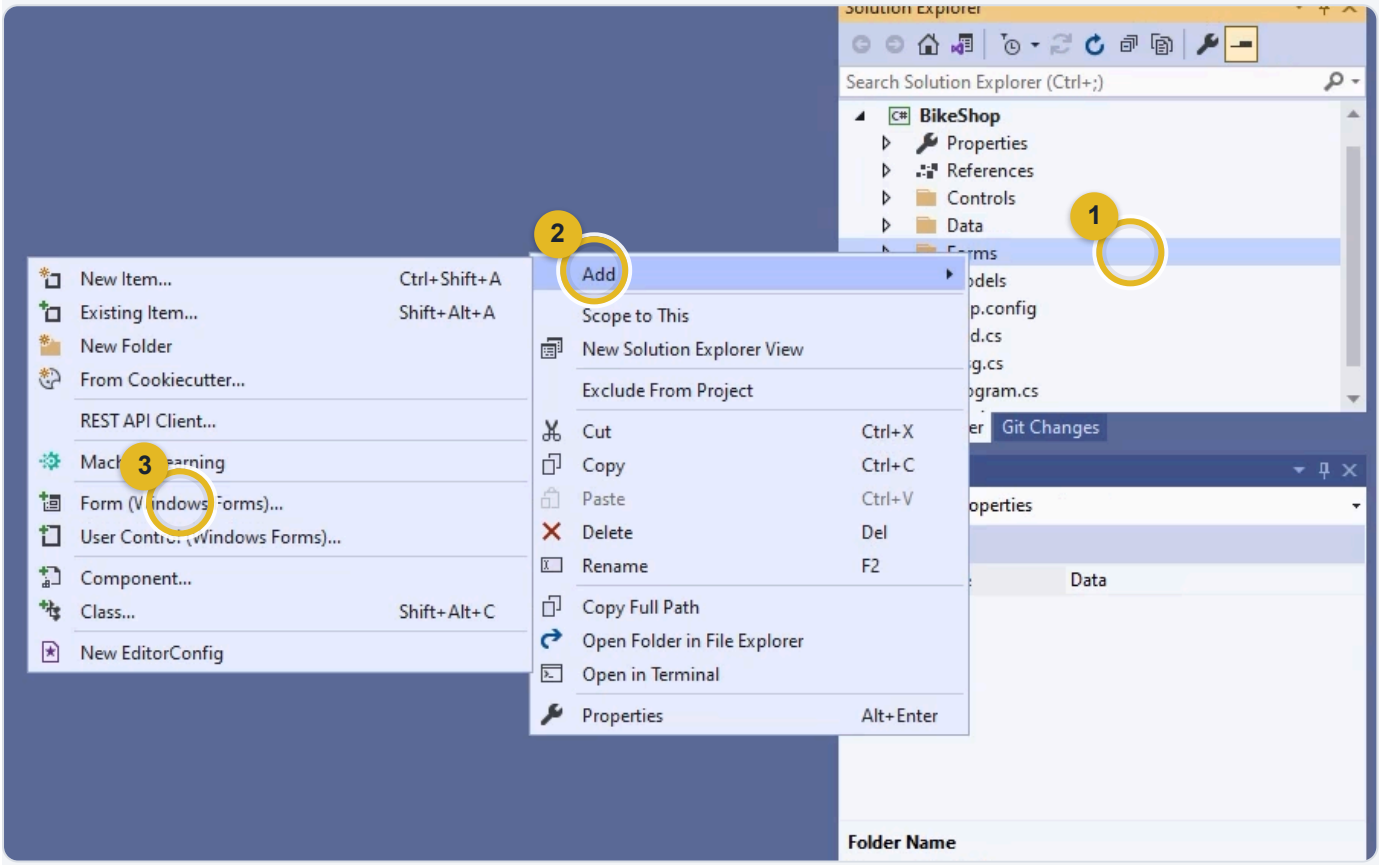
**IF IT GOES WRONG** 'The name CustomerData does not exist' or 'no definition for Search'

Your customer data class from chapter 6 is called something else, or its search method has another name. Match the names you used.

# Add the two order forms and set their controls.

This stage adds two more small forms: one to place an order and one to receive the delivery. Each has a grey note that tells the counter staff what will happen to the count.

In Solution Explorer, right-click the Forms folder, point at Add, and click Form (Windows Forms)…. Type frmPartOrderPlace as the name and press Add.




Adding a form to the Forms folder. Taken in the finished project, so Solution Explorer lists more files than yours.

1. Right-click the Forms folder.
2. Point at Add.
3. Click Form (Windows Forms)…, type the name, press Add.

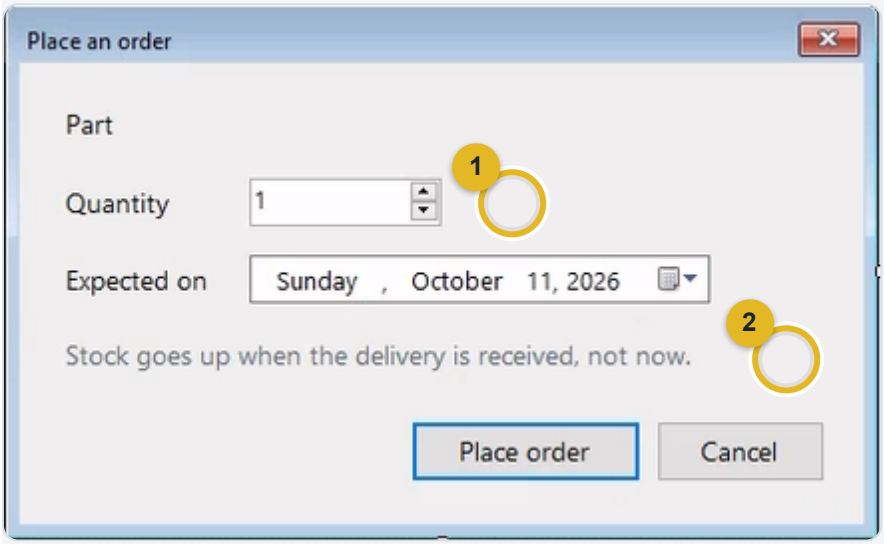
> **You should see** An empty form opens in the designer.

Click the form itself first and set the form's own properties. Then work down the table: drag the control from the Toolbox, drop it on the form, set its Name in the Properties window, and set the other settings in the row. Where the Then column says double-click, double-clicking the control makes its event method for you.

Orders more of one part from its supplier. Nothing changes in stock until the delivery is received.

**Click the form itself, then set these in the Properties window**

| | | | |
|---|---|---|---|
| Text | `Place an order` | Size | `440, 240` |
| FormBorderStyle | `FixedDialog` | StartPosition | `CenterParent` |
| MaximizeBox | `False` | MinimizeBox | `False` |
| ShowInTaskbar | `False` | AcceptButton | `btnPlace (after you add it)` |
| CancelButton | `btnCancel (after you add it)` | | |




frmPartOrderPlace in the designer. The note tells the counter staff that the count does not change yet.

1. Quantity starts at the part's usual reorder amount.
2. The grey note: stock goes up when the delivery is received, not now.

| DRAG THIS FROM THE TOOLBOX | NAME IT (THE NAME PROPERTY) | SET THESE IN THE PROPERTIES WINDOW | THEN |
|---|---|---|---|
| Label | `lblPartCaption` | `Text = Part` | |
| Label | `lblPart` | nothing else | |
| Label | `lblQty` | `Text = Quantity` | |
| NumericUpDown | `nudQuantity` | `Maximum = 100000`<br>`Minimum = 1`<br>`Value = 1`<br>Starts at the part's usual order amount. | |
| Label | `lblExpected` | `Text = Expected on` | |
| DateTimePicker | `dtpExpected` | `Format = Long` | |
| Label | `lblNote` | `Text = Stock goes up when the delivery is received, not now.`<br>`ForeColor = grey (92, 103, 116)` | |
| Button | `btnPlace` | `Text = Place order` | Double-click it. Visual Studio makes `btnPlace_Click` for you; the code goes inside it. |
| Button | `btnCancel` | `Text = Cancel`<br>`DialogResult = Cancel` | |

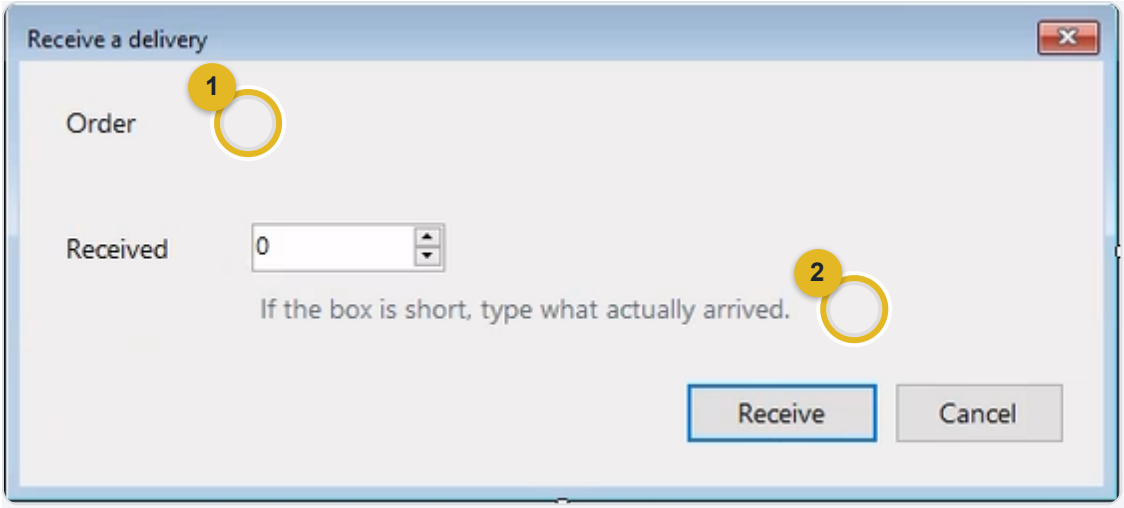
Add the second form the same way: right-click the Forms folder, point at Add, click Form (Windows Forms)…, type frmPartOrderReceive as the name and press Add.

> **You should see** A second empty form opens in the designer.

The delivery has arrived. One click closes the order and adds the parts to stock, inside one transaction.

**Click the form itself, then set these in the Properties window**

| | | | |
|---|---|---|---|
| Text | `Receive a delivery` | Size | `560, 220` |
| FormBorderStyle | `FixedDialog` | StartPosition | `CenterParent` |
| MaximizeBox | `False` | MinimizeBox | `False` |
| ShowInTaskbar | `False` | AcceptButton | `btnReceive (after you add it)` |
| CancelButton | `btnCancel (after you add it)` | | |




frmPartOrderReceive in the designer. The received box starts at the ordered amount.

1. The order summary and the quantity ordered are filled in when the form opens.
2. If the box is short, the staff type what actually arrived.

| DRAG THIS FROM THE TOOLBOX | NAME IT (THE NAME PROPERTY) | SET THESE IN THE PROPERTIES WINDOW | THEN |
|---|---|---|---|
| Label | `lblOrderCaption` | `Text = Order` | |
| Label | `lblOrder` | nothing else | |
| Label | `lblOrdered` | `ForeColor = grey (92, 103, 116)` | |
| Label | `lblReceived` | `Text = Received` | |
| NumericUpDown | `nudReceived` | `Maximum = 100000` Starts at the amount ordered. Change it if the box is short. | |
| Label | `lblNote` | `Text = If the box is short, type what actually arrived.` `ForeColor = grey (92, 103, 116)` | |
| Button | `btnReceive` | `Text = Receive` | Double-click it. Visual Studio makes `btnReceive_Click` for you; the code goes inside it. |
| Button | `btnCancel` | `Text = Cancel` `DialogResult = Cancel` | |

# Put the code behind the two order forms.

The place form is all copy. The receive form has the call you type: the one call that closes the order and adds to the count.

Copy this one **Forms/frmPartOrderPlace.cs: the whole file** C#

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  using System;
2  using System.Collections.Generic;
3  using System.Data;
4  using System.Drawing;
5  using System.Windows.Forms;
6  using BikeShop.Data;
7  using BikeShop.Models;
8
9  namespace BikeShop.Forms
10 {
11     public partial class frmPartOrderPlace : Form
12     {
13         private readonly int partID;
14
15         public frmPartOrderPlace(int id, string partName, int suggestedQty)
16         {
17             InitializeComponent();
18             partID = id;
19             lblPart.Text = partName;
20             // the part's usual reorder amount, ready to change
21             nudQuantity.Value = Math.Max(1, suggestedQty);
22         }
23
24         private void frmPartOrderPlace_Load(object sender, EventArgs e)
25         {
26             dtpExpected.Value = DateTime.Today.AddDays(7);
27         }
28
29         #region Place
30         private void btnPlace_Click(object sender, EventArgs e)
31         {
32             // The quantity box starts at 1, so it needs no check.  The database
33             // checks the expected date.
34             string message = PartOrderData.Place(partID, (int)nudQuantity.Value,
35                 dtpExpected.Value, Session.StaffID);
36             if (message ≠ "")
37             {
38                 // e.g. the expected date is in the past
39                 Msg.Problem(message);
40                 return;
41             }
42             DialogResult = DialogResult.OK;
43         }
44         #endregion
45     }
46 }
```

**1 The usings**

Lines 1 to 7

The tools this form needs: Windows Forms for the window itself, System.Data for the tables that fill the grids and lists, System.Drawing for colours, and your own Data and Models folders.

**2 The form’s class and its constructor**

Lines 9 to 22

The parts screen hands over the part number, its name and its usual reorder amount. The quantity box starts at that amount, or at 1 if the amount is 0, ready to change.

**3 When the form loads**

Lines 24 to 27

The expected date starts a week from today.

**4 The Place order button**

Lines 29 to 46

The button makes one call to the database with the part, the quantity, the date and who is signed in. Then it checks the message, and the form closes with OK.

**Type this one** **Forms/frmPartOrderReceive.cs: the whole file** C#

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  using System;
2  using System.Collections.Generic;
3  using System.Data;
4  using System.Drawing;
5  using System.Windows.Forms;
6  using BikeShop.Data;
7  using BikeShop.Models;
8
9  namespace BikeShop.Forms
10 {
11     public partial class frmPartOrderReceive : Form
12     {
13         private readonly int partOrderID;
14
15         public frmPartOrderReceive(int id, string summary, int qtyOrdered)
16         {
17             InitializeComponent();
18             partOrderID = id;
19             lblOrder.Text = summary;
20             lblOrdered.Text = qtyOrdered + " ordered";
21             // usually the whole order arrives; change it if the box is short
22             nudReceived.Value = qtyOrdered;
23         }
24
25         #region Receive
26         private void btnReceive_Click(object sender, EventArgs e)
27         {
28             // One procedure call closes the order AND adds to the stock.
29             // The quantity box does not go below 0, so it needs no check.
30             string message = PartOrderData.Receive(partOrderID,
31                 (int)nudReceived.Value);
32             if (message ≠ "")
33             {
34                 Msg.Problem(message);
35                 return;
36             }
37             DialogResult = DialogResult.OK;
38         }
39         #endregion
40     }
41 }
```

### 1 The usings

Lines 1 to 7

The tools this form needs: Windows Forms for the window itself, System.Data for the tables that fill the grids and lists, System.Drawing for colours, and your own Data and Models folders.

### 2 The form’s class and its constructor

Lines 9 to 23

The parts screen hands over the order number, a one-line summary of the order, and how many were ordered. The received box starts at the ordered amount.

**3 The Receive button: the part you type**

Lines 25 to 41

The button makes one call with the order number and what arrived. Then it checks the message, and the form closes with OK.

**Why you type it** One call does two things in the database. Type it, so that you remember that the screen never adds to the count itself.

With frmPartOrderPlace open in the designer, press F7 to see its code.

**You should see** The code tab for the form comes to the front.

Press Ctrl and A together to select everything in the code tab, press the Copy button under this step, and paste over it.

**You should see** The code tab holds the whole file, and the old lines are gone.

Press F6 to build.

**You should see** Build succeeded, at the bottom left.

**IF IT GOES WRONG** frmPartOrderPlace_Load or btnPlace_Click is underlined: no suitable method found

The designer did not make that event. Go back to the designer, click the form (or the button), open the Properties window, press the lightning bolt, and double-click the empty box beside the event. Then build again.

**IF IT GOES WRONG** 'The name Session does not exist'

Your sign-in class from chapter 5 is called something else. Match the name you used, and the name of its StaffID property.

Open frmPartOrderReceive in the designer and press F7 to see its code.

**You should see** The code tab for the form comes to the front.

Press Ctrl and A together to select everything in the code tab, press the Copy button under this step, and paste over it.

**You should see** The code tab holds the whole file, with one line that says YOU TYPE THIS PART.

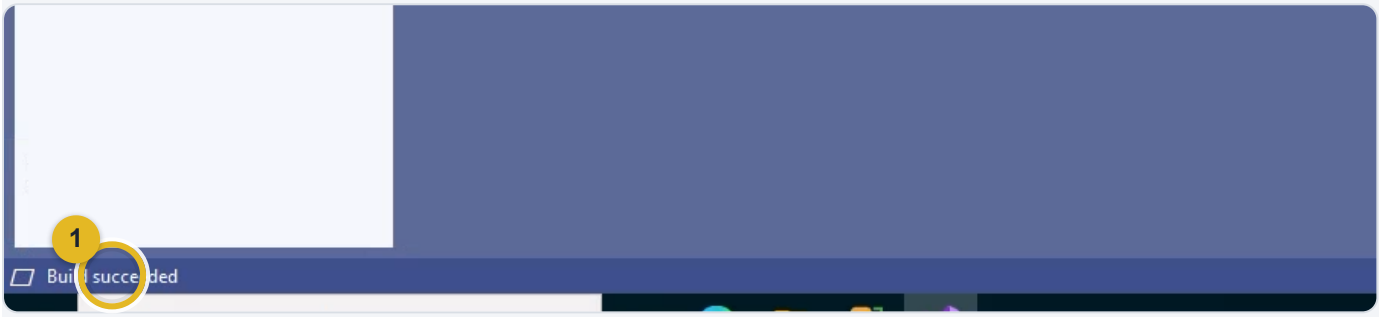
Click on the line that says YOU TYPE THIS PART, type the yellow lines in its place, and delete the marker line.

> **You should see** The yellow lines are in your file, and no line says YOU TYPE THIS PART any more.

**IF IT GOES WRONG** A red wavy line under a word

A spelling slip. Compare your line with the yellow line, letter by letter, and check that each line ends with a semicolon where the yellow line does.

Press F6 to build.




Build succeeded, at the bottom left. Anything else means an error list to read.

1. Build succeeded. Press F6 to build at any time.

> **You should see** Build succeeded, at the bottom left.

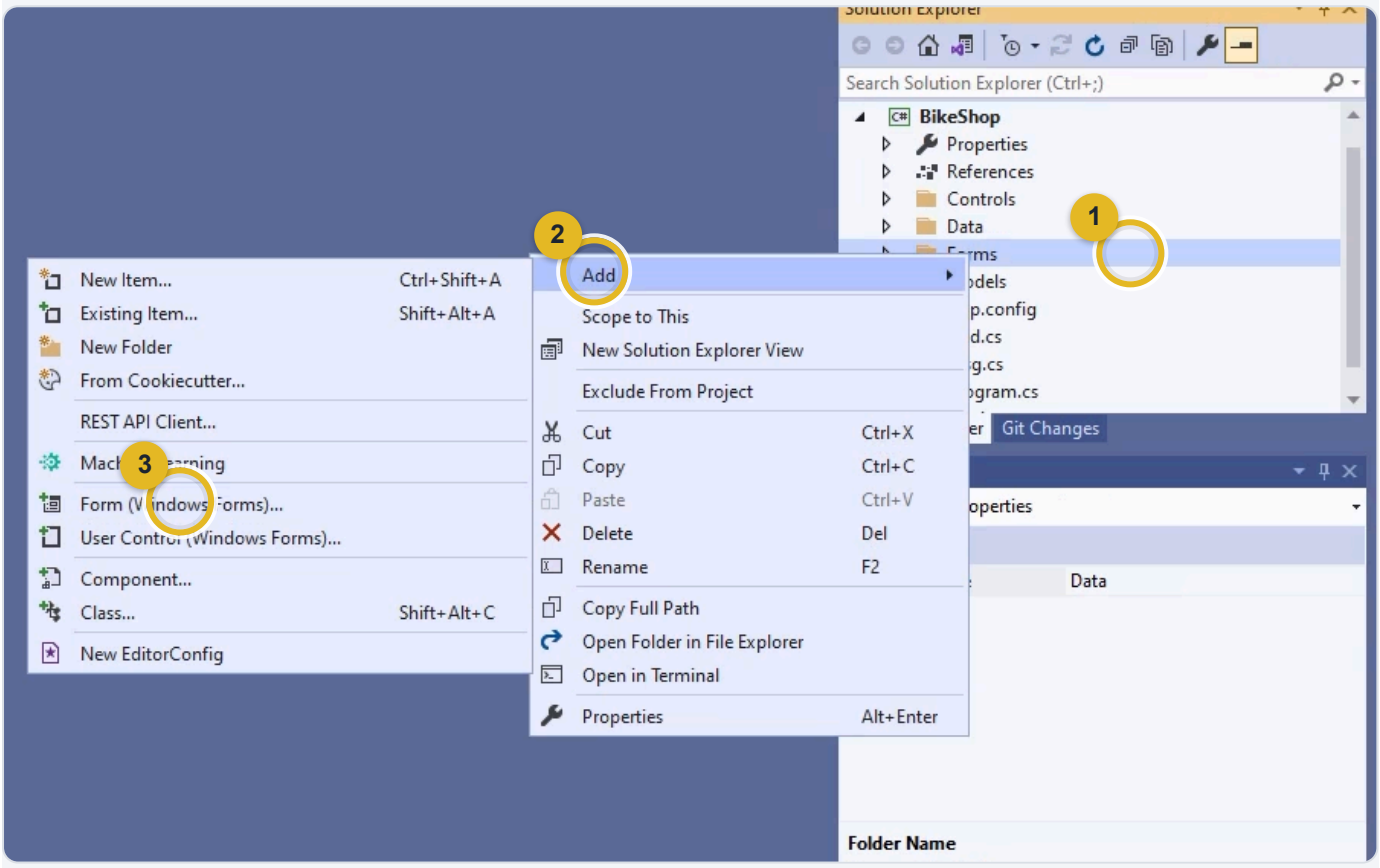
**IF IT GOES WRONG** 'btnReceive_Click is underlined: no suitable method found'

The designer did not make that event. Go back to the designer, double-click the Receive button, and build again.

# Add the parts screen and set its controls.

Now you build the big one. It has two grids: every part at the top, the open orders underneath. The buttons open the four small forms you have made, and the Reorder list button swaps the top grid between every part and only the parts to order.

In Solution Explorer, right-click the Forms folder, point at Add, and click Form (Windows Forms)…. Type frmParts as the name and press Add.




Adding a form to the Forms folder. Taken in the finished project, so Solution Explorer lists more files than yours.

1. Right-click the Forms folder.
2. Point at Add.
3. Click Form (Windows Forms)…, type the name, press Add.

> **You should see** An empty form opens in the designer.

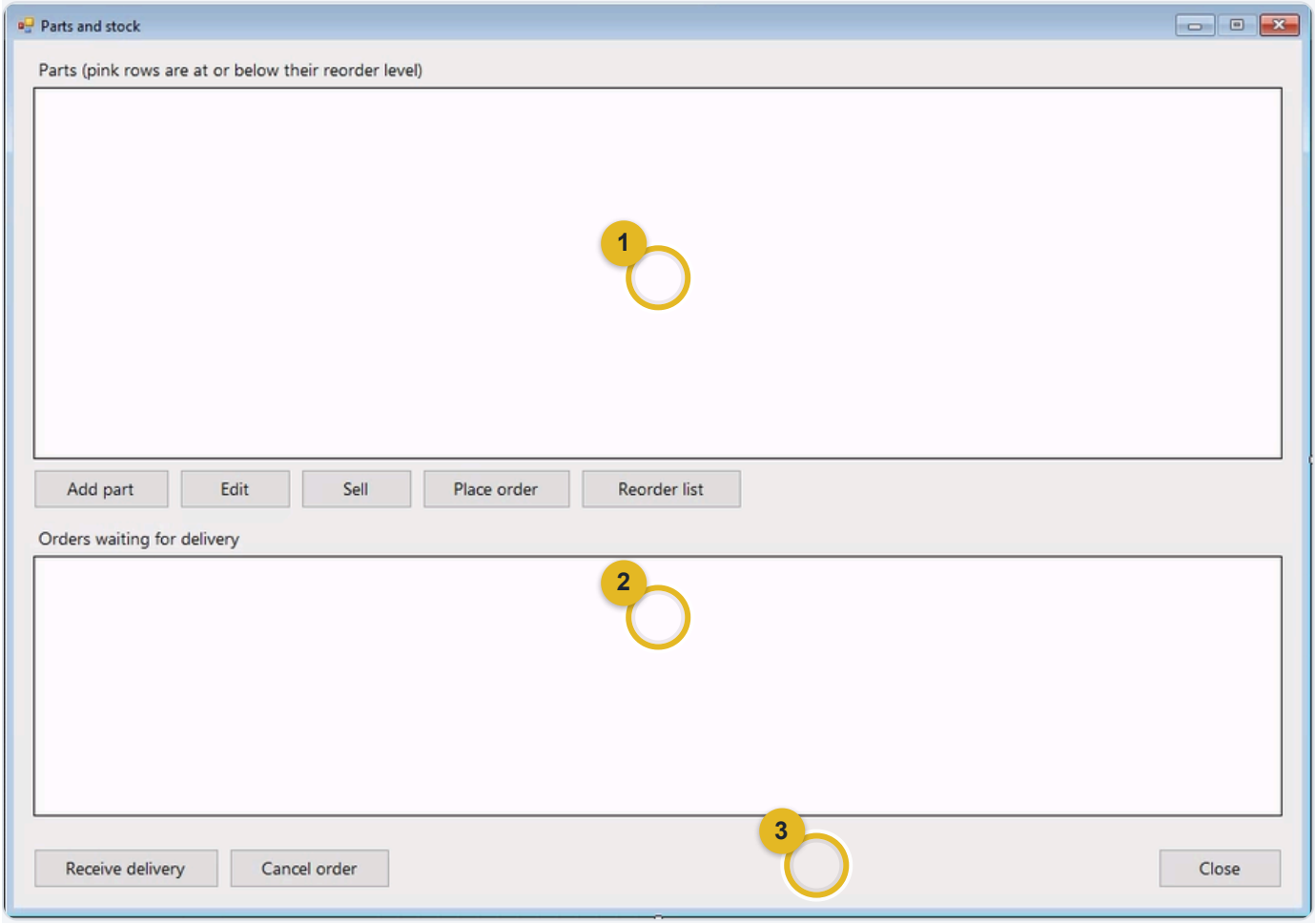
Click the form itself first and set the form's own properties. Then work down the table: drag the control from the Toolbox, drop it on the form, set its Name in the Properties window, and set the

other settings in the row. Where the Then column says double-click, double-clicking the control makes its event method for you.

Counted stock. Pink rows need reordering. The lower grid is the orders still waiting for a delivery.

**Click the form itself, then set these in the Properties window**

| | | | |
|---|---|---|---|
| Text | `Parts and stock` | Size | `1040, 700` |
| FormBorderStyle | `FixedSingle` | StartPosition | `CenterParent` |
| MaximizeBox | `False` | | |




frmParts in the designer. Yours should look like this before you add any code.

1. The parts grid, dgvParts.
2. The orders grid, dgvOrders.
3. Add part, Edit, Sell, Place order and Reorder list sit between the grids; Receive delivery, Cancel order and Close sit along the bottom.

| DRAG THIS FROM THE TOOLBOX | NAME IT (THE NAME PROPERTY) | SET THESE IN THE PROPERTIES WINDOW | THEN |
|---|---|---|---|
| Label | `lblParts` | `Text = Parts (pink rows are at or below their reorder level)` | |
| DataGridView | `dgvParts` | `the grid settings (chapter 6)` DataBindingComplete is where the pink tint is painted. | Events tab (the lightning bolt) → SelectionChanged: type `dgvParts_SelectionChanged` beside it and press Enter.,Events tab (the lightning bolt) → DataBindingComplete: type `dgvParts_DataBindingComplete` beside it and press Enter. |
| Button | `btnAddPart` | `Text = Add part` | Double-click it. Visual Studio makes `btnAddPart_Click` for you; the code goes inside it. |
| Button | `btnEditPart` | `Text = Edit` `Enabled = False` | Double-click it. Visual Studio makes `btnEditPart_Click` for you; the code goes inside it. |
| Button | `btnSell` | `Text = Sell` `Enabled = False` Over the counter. Stock can never go below zero. | Double-click it. Visual Studio makes `btnSell_Click` for you; the code goes inside it. |
| Button | `btnOrder` | `Text = Place order` `Enabled = False` Stock does not change until the delivery is received. | Double-click it. Visual Studio makes `btnOrder_Click` for you; the code goes inside it. |
| Button | `btnReorderList` | `Text = Reorder list` Swaps the grid to only the parts that need ordering, and back. | Double-click it. Visual Studio makes `btnReorderList_Click` for you; the code goes inside it. |
| Label | `lblOrders` | `Text = Orders waiting for delivery` | |
| DataGridView | `dgvOrders` | `the grid settings (chapter 6)` | Events tab (the lightning bolt) → SelectionChanged: type `dgvOrders_SelectionChanged` beside it and press Enter. |

| DRAG THIS FROM THE TOOLBOX | NAME IT (THE NAME PROPERTY) | SET THESE IN THE PROPERTIES WINDOW | THEN |
|---|---|---|---|
| Button | `btnReceive` | `Text = Receive delivery`<br>`Enabled = False`<br>Closes the order and adds to stock in one go. | Double-click it. Visual Studio makes `btnReceive_Click` for you; the code goes inside it. |
| Button | `btnCancelOrder` | `Text = Cancel order`<br>`Enabled = False` | Double-click it. Visual Studio makes `btnCancelOrder_Click` for you; the code goes inside it. |
| Button | `btnClose` | `Text = Close` | Double-click it. Visual Studio makes `btnClose_Click` for you; the code goes inside it. |

# Read the three pieces of code behind the parts screen.

There are three pieces to read before you put the whole file in. The first fills the top grid and tints the low rows pink. The next two are the buttons: all but Cancel order open a small form, and refill a grid when the form comes back with OK. Cancel order asks a yes-or-no question instead.

Read this one **Forms/frmParts.cs: filling the grid and tinting the low rows** C#

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  private void LoadParts()
2  {
3      // empty the grid first, so the two lists never mix their columns
4      dgvParts.DataSource = null;
5      dgvParts.Columns.Clear();
6      if (reorderOnly)
7      {
8          dgvParts.DataSource = PartData.GetReorderList();
9          btnReorderList.Text = "Show all parts";
10     }
11     else
12     {
13         dgvParts.DataSource = PartData.GetAll();
14         btnReorderList.Text = "Reorder list";
15     }
16     Grid.Tidy(dgvParts, "PartID", "SupplierID");
17 }
18
19 // The grid raises this when it has finished filling: the moment to tint
20 // the parts that are low on stock.
21 private void dgvParts_DataBindingComplete(object sender,
22     DataGridViewBindingCompleteEventArgs e)
23 {
24     if (reorderOnly)
25     {
26         // every part on the reorder list is low, so only the full list
27         // is tinted
28         return;
29     }
30
31     foreach (DataGridViewRow row in dgvParts.Rows)
32     {
33         int inStock = Convert.ToInt32(row.Cells["QtyInStock"].Value);
34         int reorderLevel =
35             Convert.ToInt32(row.Cells["ReorderLevel"].Value);
36         if (inStock ⩽ reorderLevel)
37         {
38             row.DefaultCellStyle.BackColor = Color.MistyRose;
39         }
40         else
41         {
42             row.DefaultCellStyle.BackColor = Color.White;
43         }
44     }
45 }
46
47 private void btnReorderList_Click(object sender, EventArgs e)
48 {
49     reorderOnly = !reorderOnly;
50     LoadParts();
51 }
```

### 1 Fill the grid, one way or the other

Lines 1 to 17

reorderOnly is a true-or-false fact the form remembers. True fills the grid from the reorder list; false fills it from every part. The grid is emptied first, columns and all, so the two lists never mix their columns. The button's words swap to say what the next press will do. Grid.Tidy hides the number columns.

### 2 Tint the low rows

Lines 21 to 45

The grid raises this event once it has finished filling. For every row, the count and the reorder level are read out of their cells, and the row goes pink when the count is at or below the level. The reorder list is skipped: every row on it is low.

### 3 The Reorder list button

Lines 47 to 51

The button changes reorderOnly from true to false, or from false to true, and then fills the grid again.

**Why you type it** The pink tint is worked out from two columns of the row, after the grid has filled. The grid tells you when with its DataBindingComplete event.

Read this one **Forms/frmParts.cs: the buttons under the parts grid** C#

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  private void btnAddPart_Click(object sender, EventArgs e)
2  {
3      frmPartEdit add = new frmPartEdit();
4      if (add.ShowDialog(this) == DialogResult.OK)
5      {
6          LoadParts();
7      }
8  }
9
10 private void btnEditPart_Click(object sender, EventArgs e)
11 {
12     // the row behind the picked line
13     DataRowView row =
14         (DataRowView)dgvParts.SelectedRows[0].DataBoundItem;
15     frmPartEdit edit = new frmPartEdit(row.Row);
16     if (edit.ShowDialog(this) == DialogResult.OK)
17     {
18         LoadParts();
19     }
20 }
21
22 private void btnSell_Click(object sender, EventArgs e)
23 {
24     DataGridViewRow row = dgvParts.SelectedRows[0];
25     int inStock = Convert.ToInt32(row.Cells["QtyInStock"].Value);
26     int partID = Grid.SelectedID(dgvParts, "PartID");
27     string name = Grid.SelectedText(dgvParts, "Name");
28     frmPartSell sell = new frmPartSell(partID, name, inStock);
29     if (sell.ShowDialog(this) == DialogResult.OK)
30     {
31         LoadParts();
32     }
33 }
34
35 private void btnOrder_Click(object sender, EventArgs e)
36 {
37     DataGridViewRow row = dgvParts.SelectedRows[0];
38     int reorderQty = Convert.ToInt32(row.Cells["ReorderQty"].Value);
39     int partID = Grid.SelectedID(dgvParts, "PartID");
40     string name = Grid.SelectedText(dgvParts, "Name");
41     frmPartOrderPlace place =
42         new frmPartOrderPlace(partID, name, reorderQty);
43     if (place.ShowDialog(this) == DialogResult.OK)
44     {
45         LoadOrders();
46         // an open order takes the part off the reorder list
47         LoadParts();
48     }
49 }
```

**1 The Add part button**

Lines 1 to 8

The button opens the edit form empty. If the form comes back with OK, the part was saved, so the grid is filled again.

**2 The Edit button**

Lines 10 to 20

The row behind the picked line is handed to the edit form, so it opens filled in.

**3 The Sell button**

Lines 22 to 33

The count, the number and the name of the picked part go to the counter sale form. OK means a sale happened, so the grid is filled again and the count changes.

**4 The Place order button**

Lines 35 to 49

The usual reorder amount goes to the place form. OK means an order was placed, so both grids are filled again. The count has not changed, but on the reorder list the part has gone, because an order for it is now open.

Read this one **Forms/frmParts.cs: the buttons under the orders grid** C#

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  private void btnReceive_Click(object sender, EventArgs e)
2  {
3      string summary = Grid.SelectedText(dgvOrders, "PartCode") + " · "
4          + Grid.SelectedText(dgvOrders, "Part") + " from "
5          + Grid.SelectedText(dgvOrders, "Supplier");
6      DataGridViewRow row = dgvOrders.SelectedRows[0];
7      int ordered = Convert.ToInt32(row.Cells["QtyOrdered"].Value);
8      int orderID = Grid.SelectedID(dgvOrders, "PartOrderID");
9      frmPartOrderReceive receive =
10         new frmPartOrderReceive(orderID, summary, ordered);
11     if (receive.ShowDialog(this) == DialogResult.OK)
12     {
13         LoadOrders();
14         // the delivery has added to the stock figures
15         LoadParts();
16     }
17 }
18
19 private void btnCancelOrder_Click(object sender, EventArgs e)
20 {
21     if (!Msg.Ask("Cancel the order for "
22         + Grid.SelectedText(dgvOrders, "Part") + "?"))
23     {
24         return;
25     }
26
27     string message =
28         PartOrderData.Cancel(Grid.SelectedID(dgvOrders, "PartOrderID"));
29     if (message ≠ "")
30     {
31         Msg.Problem(message);
32         return;
33     }
34     LoadOrders();
35 }
```

**1 The Receive delivery button**

Lines 1 to 17

A one-line summary of the picked order is put together from three cells, and goes to the receive form with the number and the quantity. OK means a delivery arrived, so both grids are filled again: the order is gone and the count is up.

**2 The Cancel order button**

Lines 19 to 35

The button asks a yes-or-no question with the part's name in it, then makes one call to the database and checks the message. Then the lower grid is filled again.

The whole file is on the next screen. The lines you type are the pink tint.

# Put the code behind the parts screen, and type the pink tint.

**Type this one** **Forms/frmParts.cs: the whole file** C#

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
using System;
using System.Collections.Generic;
using System.Data;
using System.Drawing;
using System.Windows.Forms;
using BikeShop.Data;
using BikeShop.Models;

namespace BikeShop.Forms
{
    public partial class frmParts : Form
    {
        // false = every part; true = only the parts at or below their reorder
        // level
        private bool reorderOnly;

        public frmParts()
        {
            InitializeComponent();
        }

        private void frmParts_Load(object sender, EventArgs e)
        {
            LoadParts();
            LoadOrders();
        }

        #region LoadParts
        private void LoadParts()
        {
            // empty the grid first, so the two lists never mix their columns
            dgvParts.DataSource = null;
            dgvParts.Columns.Clear();
            if (reorderOnly)
            {
                dgvParts.DataSource = PartData.GetReorderList();
                btnReorderList.Text = "Show all parts";
            }
            else
            {
                dgvParts.DataSource = PartData.GetAll();
                btnReorderList.Text = "Reorder list";
            }
            Grid.Tidy(dgvParts, "PartID", "SupplierID");
        }

        // The grid raises this when it has finished filling: the moment to tint
        // the parts that are low on stock.
        private void dgvParts_DataBindingComplete(object sender,
            DataGridViewBindingCompleteEventArgs e)
        {
            if (reorderOnly)
            {
```

```
                // every part on the reorder list is low, so only the full list
                // is tinted
                return;
            }

            foreach (DataGridViewRow row in dgvParts.Rows)
            {
                int inStock = Convert.ToInt32(row.Cells["QtyInStock"].Value);
                int reorderLevel =
                    Convert.ToInt32(row.Cells["ReorderLevel"].Value);
                if (inStock ≤ reorderLevel)
                {
                    row.DefaultCellStyle.BackColor = Color.MistyRose;
                }
                else
                {
                    row.DefaultCellStyle.BackColor = Color.White;
                }
            }
        }

        private void btnReorderList_Click(object sender, EventArgs e)
        {
            reorderOnly = !reorderOnly;
            LoadParts();
        }
        #endregion

        #region LoadOrders
        private void LoadOrders()
        {
            dgvOrders.DataSource = PartOrderData.GetOpen();
            Grid.Tidy(dgvOrders, "PartOrderID");
        }
        #endregion

        #region Selection
        private void dgvParts_SelectionChanged(object sender, EventArgs e)
        {
            bool picked = dgvParts.SelectedRows.Count > 0;
            // the reorder list has no cost columns to edit from
            btnEditPart.Enabled = picked && !reorderOnly;
            btnSell.Enabled = picked;
            btnOrder.Enabled = picked;
        }

        private void dgvOrders_SelectionChanged(object sender, EventArgs e)
        {
            bool picked = dgvOrders.SelectedRows.Count > 0;
            btnReceive.Enabled = picked;
            btnCancelOrder.Enabled = picked;
        }
        #endregion

        #region PartActions
        private void btnAddPart_Click(object sender, EventArgs e)
        {
            frmPartEdit add = new frmPartEdit();
```

```
            if (add.ShowDialog(this) == DialogResult.OK)
            {
                LoadParts();
            }
        }

        private void btnEditPart_Click(object sender, EventArgs e)
        {
            // the row behind the picked line
            DataRowView row =
                (DataRowView)dgvParts.SelectedRows[0].DataBoundItem;
            frmPartEdit edit = new frmPartEdit(row.Row);
            if (edit.ShowDialog(this) == DialogResult.OK)
            {
                LoadParts();
            }
        }

        private void btnSell_Click(object sender, EventArgs e)
        {
            DataGridViewRow row = dgvParts.SelectedRows[0];
            int inStock = Convert.ToInt32(row.Cells["QtyInStock"].Value);
            int partID = Grid.SelectedID(dgvParts, "PartID");
            string name = Grid.SelectedText(dgvParts, "Name");
            frmPartSell sell = new frmPartSell(partID, name, inStock);
            if (sell.ShowDialog(this) == DialogResult.OK)
            {
                LoadParts();
            }
        }

        private void btnOrder_Click(object sender, EventArgs e)
        {
            DataGridViewRow row = dgvParts.SelectedRows[0];
            int reorderQty = Convert.ToInt32(row.Cells["ReorderQty"].Value);
            int partID = Grid.SelectedID(dgvParts, "PartID");
            string name = Grid.SelectedText(dgvParts, "Name");
            frmPartOrderPlace place =
                new frmPartOrderPlace(partID, name, reorderQty);
            if (place.ShowDialog(this) == DialogResult.OK)
            {
                LoadOrders();
                // an open order takes the part off the reorder list
                LoadParts();
            }
        }
        #endregion

        #region OrderActions
        private void btnReceive_Click(object sender, EventArgs e)
        {
            string summary = Grid.SelectedText(dgvOrders, "PartCode") + " · "
                + Grid.SelectedText(dgvOrders, "Part") + " from "
                + Grid.SelectedText(dgvOrders, "Supplier");
            DataGridViewRow row = dgvOrders.SelectedRows[0];
            int ordered = Convert.ToInt32(row.Cells["QtyOrdered"].Value);
            int orderID = Grid.SelectedID(dgvOrders, "PartOrderID");
            frmPartOrderReceive receive =
```

```
170                 new frmPartOrderReceive(orderID, summary, ordered);
171             if (receive.ShowDialog(this) == DialogResult.OK)
172             {
173                 LoadOrders();
174                 // the delivery has added to the stock figures
175                 LoadParts();
176             }
177         }
178 
179         private void btnCancelOrder_Click(object sender, EventArgs e)
180         {
181             if (!Msg.Ask("Cancel the order for "
182                 + Grid.SelectedText(dgvOrders, "Part") + "?"))
183             {
184                 return;
185             }
186 
187             string message =
188                 PartOrderData.Cancel(Grid.SelectedID(dgvOrders, "PartOrderID"));
189             if (message != "")
190             {
191                 Msg.Problem(message);
192                 return;
193             }
194             LoadOrders();
195         }
196         #endregion
197 
198         private void btnClose_Click(object sender, EventArgs e)
199         {
200             Close();
201         }
202     }
203 }
```

**1 The usings**

Lines 1 to 7

The tools this form needs: Windows Forms for the window itself, System.Data for the tables that fill the grids and lists, System.Drawing for colours, and your own Data and Models folders.

**2 The form’s class, one fact it remembers, and its constructor**

Lines 9 to 20

reorderOnly starts false, so the form opens showing every part.

**3 When the form loads**

Lines 22 to 26

Both grids are filled.

**4 Filling the top grid, and the pink tint: the part you type**

Lines 28 to 80

These are the three methods you read on the last screen.

**5 Filling the lower grid**

Lines 82 to 88

This fills the lower grid with the open orders, with the number column hidden.

**6 Which buttons are allowed**

Lines 90 to 106

When the picked row changes in either grid, the buttons under it switch on or off. The Edit button is off on the reorder list, because that list has no cost columns to edit from.

**7 The buttons under the parts grid**

Lines 108 to 158

These are the four methods you read on the last screen.

**8 The buttons under the orders grid**

Lines 160 to 196

These are the Receive and Cancel order buttons.

**9 Close**

Lines 198 to 203

The Close button closes the form. The two closing braces end the class and the namespace.

**Why you type it** This one if statement turns a count into a pink row that the counter staff can see across the room.

With frmParts open in the designer, press F7 to see its code.

**You should see** The code tab for the form comes to the front.

Press Ctrl and A together to select everything in the code tab, press the Copy button under this step, and paste over it.

**You should see** The code tab holds the whole file, with one line that says YOU TYPE THIS PART.

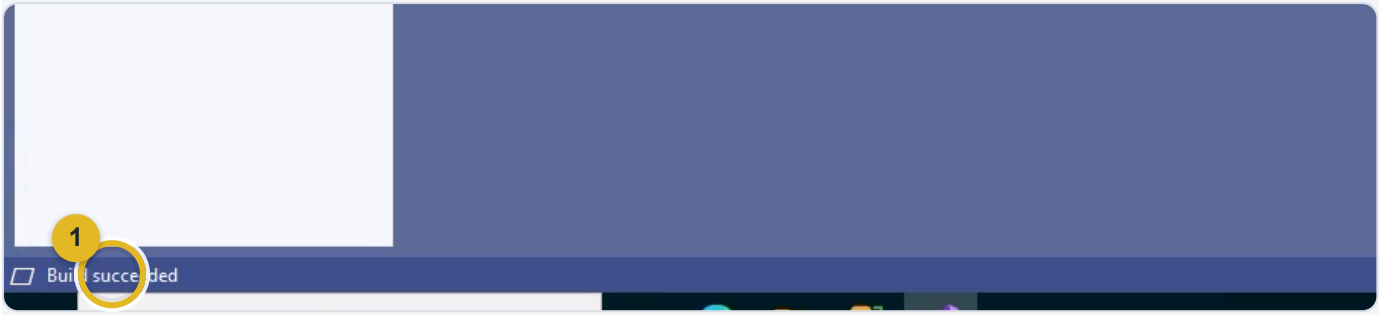
Click on the line that says YOU TYPE THIS PART, type the yellow lines in its place, and delete the marker line.

**You should see** The yellow lines are in your file, and no line says YOU TYPE THIS PART any more.

**IF IT GOES WRONG** A red wavy line under a word

A spelling slip. Compare your line with the yellow line, letter by letter, and check that each line ends with a semicolon where the yellow line does.

Press F6 to build.




Build succeeded, at the bottom left. Anything else means an error list to read.

1 Build succeeded. Press F6 to build at any time.

**You should see** Build succeeded, at the bottom left.

**IF IT GOES WRONG** frmParts_Load or btnSell_Click is underlined: no suitable method found

The designer did not make that event. Go back to the designer, click the form (or the button), open the Properties window, press the lightning bolt, and double-click the empty box beside the event. Then build again.

**IF IT GOES WRONG** dgvParts_DataBindingComplete or dgvParts_SelectionChanged is underlined

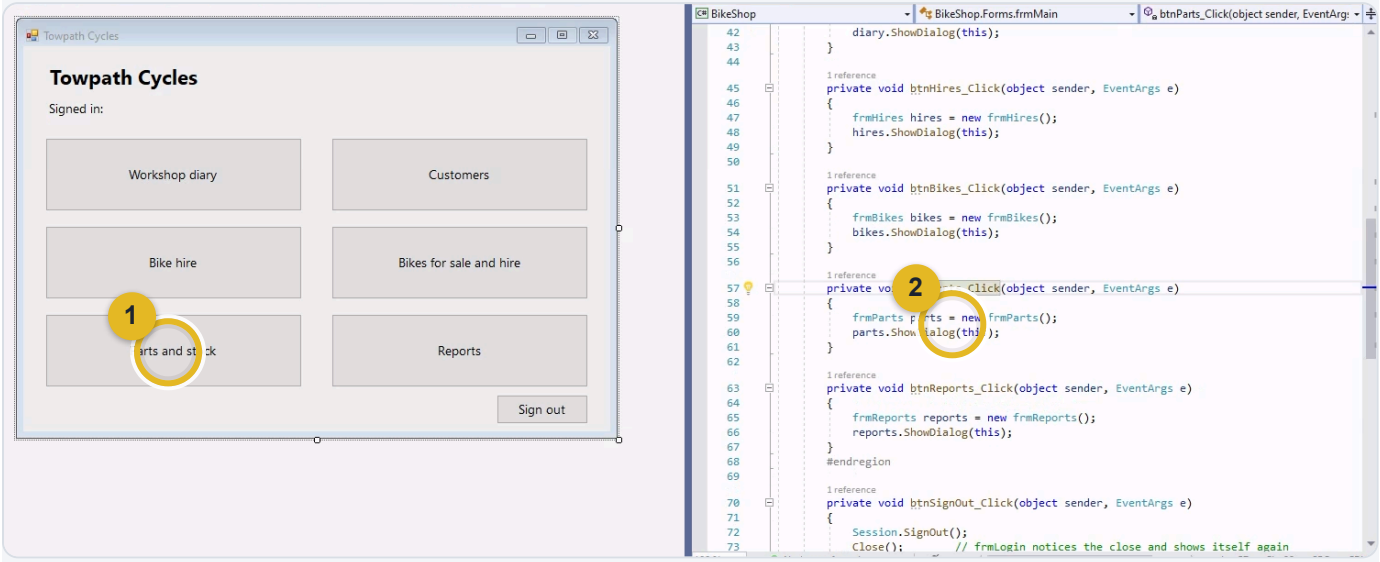
The grid's events were not made. Click the grid in the designer, open the Properties window, press the lightning bolt, and double-click the empty box beside DataBindingComplete, then beside SelectionChanged. Do the same for SelectionChanged on dgvOrders.

**IF IT GOES WRONG** 'The type or namespace name frmPartSell could not be found'

One of the four small forms is missing or is named differently. Go back to screens 2 to 7 of this stage and check the four names.

# Wire the Parts and stock button and run the program.

Open frmMain in the designer and double-click its Parts and stock button.




The three buttons on your main menu that this chapter wires up. Each one gets two lines, the same shape as the Diary button in chapter 7.

1. Double-click the button to make its Click method.

2. Type the two lines inside it.

> **You should see** The code tab opens with an empty btnParts_Click method, and the cursor is inside it.

Inside the method, type the same two lines you typed for the Diary button in chapter 7, with frmParts in place of frmDiary: frmParts parts = new frmParts(); as the first line, and parts.ShowDialog(this); as the second line.

> **You should see** Two lines sit inside the method, each ending with a semicolon.

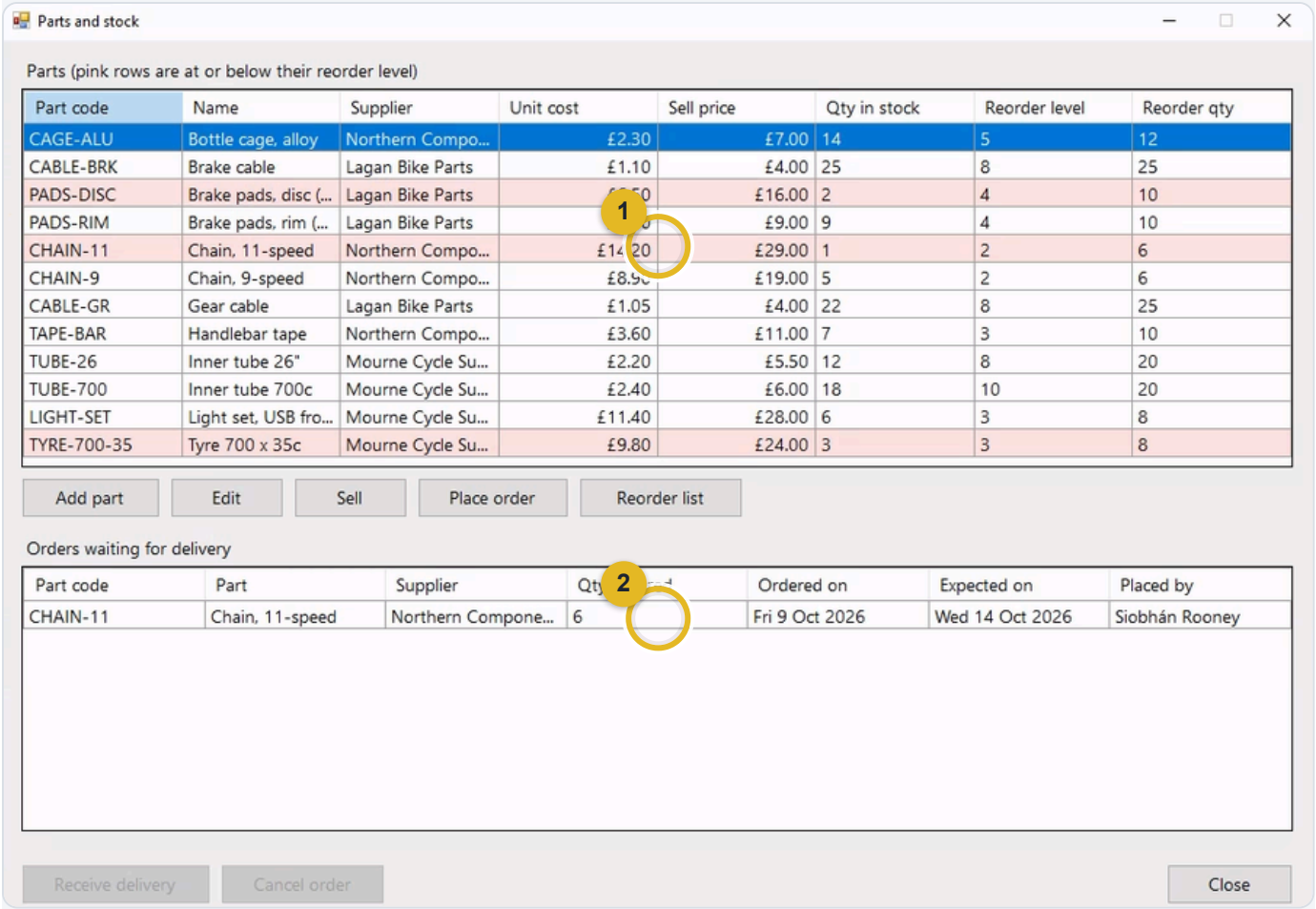
Press F6 to build.

> **You should see** Build succeeded, at the bottom left.

Press F5 to run the program, sign in, and press Parts and stock.

> **You should see** The Parts and stock screen opens over the main menu.

In the running program, look at the top grid.




The parts screen with the sample data. The pink rows are the parts at or below their reorder level.

1. PADS-DISC, CHAIN-11 and TYRE-700-35 are pink: their count is at or below the reorder level.

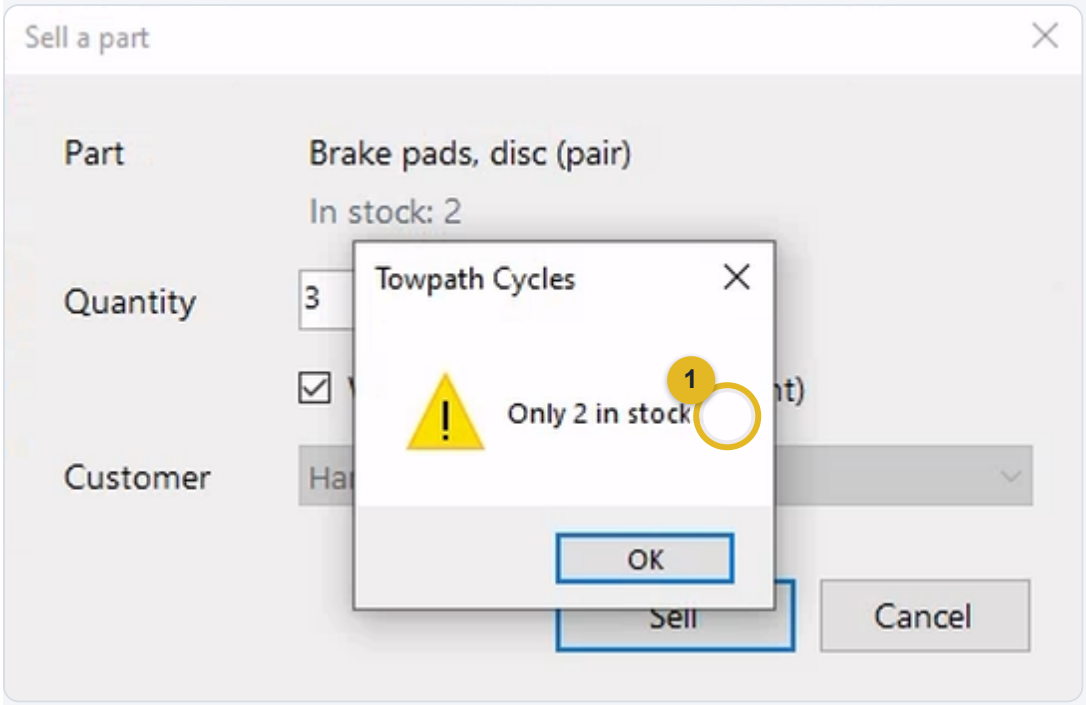
2. The orders waiting for delivery. Order 3, for CHAIN-11, is on the way.

> **You should see** Every part, with its real count. Three rows are pink: PADS-DISC, CHAIN-11 and TYRE-700-35. The lower grid holds the one order waiting for delivery: order 3, for CHAIN-11. If you ran your whole script again, the counts are the sample data again.

Pick PADS-DISC in the top grid and press Sell.

> **You should see** The counter sale form opens, with Brake pads, disc (pair) and In stock: 2 at the top, and Walk-in customer ticked.

Leave Walk-in customer ticked, set the quantity to 3, and press Sell.




The refusal, as the program shows it. The words came from the stored procedure, with the real count in them.

1 The sentence from @Message, shown by Msg.Problem.

**You should see** A message box says: Only 2 in stock. The guard in the stored procedure refused, through the form.

Press OK, set the quantity to 2, and press Sell again.

**You should see** The form closes, and the PADS-DISC row in the top grid now shows 0 in stock.

**IF IT GOES WRONG** Nothing happens when you press Parts and stock

The Click event is not wired. In the designer, click the button, open the Properties window, press the lightning bolt and check that Click has a method beside it.

**IF IT GOES WRONG** No row is pink

The DataBindingComplete event is not wired to dgvParts_DataBindingComplete. Click the grid in the designer, press the lightning bolt in the Properties window, and look for the method beside DataBindingComplete.

**IF IT GOES WRONG** The grid shows PartID and SupplierID columns

Grid.Tidy is not being called, or the column names you passed to it are spelled differently from the stored procedure's.

# Read the stored procedures that reserve a bike and let it go again, then add them.

A bike for sale can be held for one customer. Reserve checks that the bike is for sale and available, then marks it Reserved and remembers who. Unreserve is the same shape backwards, and you type its check. Every bike stored procedure in this stage uses the same shape: IF NOT EXISTS a row in the right status, refuse.

Copy this one **usp_Bike_Reserve: hold a bike for sale for one customer** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
CREATE OR ALTER PROCEDURE usp_Bike_Reserve
    @BikeID     INT,
    @CustomerID INT,
    @Message VARCHAR(200) OUTPUT
AS
BEGIN
    SET @Message = '';
    IF NOT EXISTS (SELECT * FROM Bike WHERE BikeID = @BikeID
        AND Purpose = 'Sale' AND Status = 'Available')
    BEGIN
        SET @Message = 'Only an available bike for sale can be reserved.';
        RETURN;
    END

    UPDATE Bike
    SET Status = 'Reserved', CustomerID = @CustomerID
    WHERE BikeID = @BikeID AND Purpose = 'Sale' AND Status = 'Available';
END
GO
```

**1 The name, and what goes in and out**

Lines 1 to 4

The bike and the customer in, @Message out.

**2 Start with a blank message**

Lines 5 to 7

AS and BEGIN mark where the work starts. The first job is to make @Message empty. An empty message means it worked. Every refusal below puts a sentence into @Message instead, so the program always knows which it was.

**3 Refuse unless the bike is for sale and available**

Lines 8 to 13

One check covers three wrong cases: no such bike, a hire bike, or a bike that is already reserved or sold. The same sentence does for all three.

**4 Mark it Reserved, and remember who**

Lines 15 to 19

The status changes and the customer number is kept on the row. The WHERE repeats the check, so the row only changes if it is still available at that instant. GO ends the batch, ready for the next stored procedure.

**Type this one** **usp_Bike_Unreserve: put a reserved bike back on sale** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
CREATE OR ALTER PROCEDURE usp_Bike_Unreserve
    @BikeID INT,
    @Message VARCHAR(200) OUTPUT
AS
BEGIN
    SET @Message = '';
    -- Back to stock: the reservation is dropped and the bike is for sale again.
    IF NOT EXISTS (SELECT * FROM Bike WHERE BikeID = @BikeID
        AND Status = 'Reserved')
    BEGIN
        SET @Message = 'That bike is not reserved.';
        RETURN;
    END

    UPDATE Bike
    SET Status = 'Available', CustomerID = NULL
    WHERE BikeID = @BikeID AND Status = 'Reserved';
END
GO
```

**1 The name, and what goes in and out**

Lines 1 to 3

Only the bike number goes in.

**2 Start with a blank message**

Lines 4 to 6

AS and BEGIN mark where the work starts. The first job is to make @Message empty. An empty message means it worked. Every refusal below puts a sentence into @Message instead, so the program always knows which it was.

**3 Refuse unless the bike is reserved: the part you type**

Lines 8 to 13

Only a Reserved bike can be unreserved. Anything else, refuse.

**4 Back to Available, with no customer**

Lines 15 to 19

The status goes back, and the customer number is cleared with NULL. GO ends the batch, ready for the next stored procedure.

**Why you type it** The status check again, from memory this time, for the one status this stored procedure accepts.

Press the Copy button under this step, click on the empty line above the line that prints BikeShop is ready, and paste.

> **You should see** The stored procedure is in your script, ending with GO, and the line that prints BikeShop is ready is still last.

Press the Copy button under this step, click on the empty line above the line that prints BikeShop is ready, and paste.

> **You should see** Both stored procedures are in your script, the second with one line that says YOU TYPE THIS PART.

Type the yellow lines in place of the line that says YOU TYPE THIS PART, so that the marker line is gone.

> **You should see** Six lines of your own, from IF NOT EXISTS to END, and no marker line.

**IF IT GOES WRONG** A red wavy line under a word

A spelling slip. Compare your line with the yellow line, letter by letter, and check that each line ends the way the yellow line ends.

**IF IT GOES WRONG** 'Must declare the scalar variable …'

A name beginning with @ is spelled differently from the line that declares it. Make the two match.

Select all the lines you added on this screen, from the first CREATE OR ALTER PROCEDURE down to the last GO, and press F5.

> **You should see** The Messages pane says: Commands completed successfully. Your test sales and orders are still there.

**IF IT GOES WRONG** The Messages pane shows (6 rows affected) and BikeShop is ready

Nothing was selected, so the whole script ran and the sample data was put back. Your test sales, orders and reservations are gone. Nothing is broken: carry on, and remember that the stock of part 5 is 5 again.

# Test reserving and unreserving.

**Copy this one** **A test that reserves the Cannondale, then tries two bikes that cannot be res…** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
-- Reserve bike 12 (the Cannondale, for sale) for customer 1, Aoife McCann.
-- Then try bike 10, which is already reserved, and bike 1, a hire bike.
DECLARE @Message VARCHAR(200);

EXEC usp_Bike_Reserve @BikeID = 12, @CustomerID = 1,
     @Message = @Message OUTPUT;
IF @Message = ''
BEGIN
    PRINT 'Bike 12 is reserved for Aoife McCann.';
END
ELSE
BEGIN
    PRINT @Message;
END

EXEC usp_Bike_Reserve @BikeID = 10, @CustomerID = 1,
     @Message = @Message OUTPUT;
PRINT 'Bike 10: ' + @Message;

EXEC usp_Bike_Reserve @BikeID = 1, @CustomerID = 1,
     @Message = @Message OUTPUT;
PRINT 'Bike 1: ' + @Message;
```

**1 One box for the answer**

Line 3

Only @Message is needed: reserving makes no new row.

**2 Reserve bike 12 for Aoife McCann**

Lines 5 to 14

Bike 12 is the Cannondale for sale, available in the sample data. Customer 1 is Aoife McCann.

**3 Try a bike that is already reserved**

Lines 16 to 18

Bike 10 is reserved for Méabh Cunningham in the sample data. The refusal is printed after the bike number.

**4 Try a hire bike**

Lines 20 to 22

Bike 1 is in the hire fleet. The same sentence refuses it.

**Copy this one** **A test that lets bike 12 go again** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  -- Take the reservation off bike 12.  Run it once: it is for sale again.
2  -- Run it again: that bike is not reserved.
3  DECLARE @Message VARCHAR(200);
4
5  EXEC usp_Bike_Unreserve @BikeID = 12, @Message = @Message OUTPUT;
6  IF @Message = ''
7  BEGIN
8      PRINT 'Bike 12 is available again.';
9  END
10 ELSE
11 BEGIN
12     PRINT @Message;
13 END
```

**1 One box for the answer**

Line 3

The same single box.

**2 Unreserve bike 12, and print the answer**

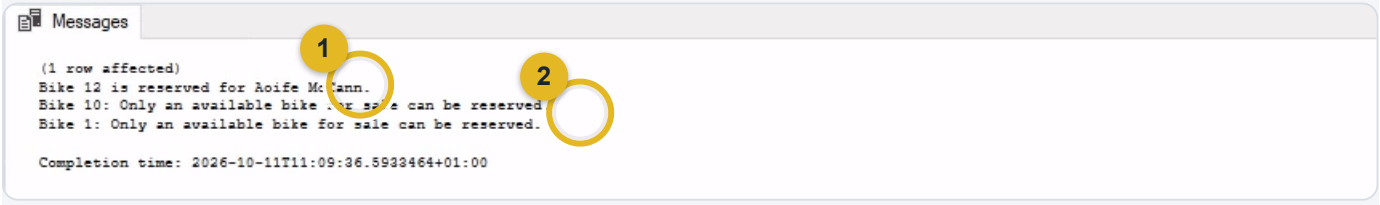
Lines 5 to 13

The first run puts the bike back on sale. The second run finds no Reserved row for bike 12, so it is refused.

Click New Query, press the Copy button under this step, and paste the test into the empty window.

**You should see** The test is in a new query window of its own, with your script still open in its tab.

Press F5 to run the test.




One reservation made and two refused: bike 10 is already reserved, and bike 1 is a hire bike.

1 Bike 12 changed status: Available to Reserved.

2 The same sentence twice, for two different reasons. One check covers both.

**You should see** The Messages pane says (1 row affected), then Bike 12 is reserved for Aoife McCann. Then Bike 10: Only an available bike for sale can be reserved. Then the same sentence for bike 1.

Click New Query, press the Copy button under this step, and paste the test into the empty window.

> **You should see** The test is in a new query window of its own, with your script still open in its tab.

Press F5 to run the test.

> **You should see** The Messages pane says (1 row affected), then Bike 12 is available again.

Press F5 again, changing nothing.

> **You should see** The Messages pane says: That bike is not reserved.

Go back to the reserve test window and press F5 there once more.

> **You should see** The Messages pane says (1 row affected), then Bike 12 is reserved for Aoife McCann, then the two refusals again. Stage 7 needs bike 12 reserved.

# Write the stored procedure that sells a bike once, and only once.

Selling a bike is the one that matters most. A sold bike must never be sold again, and a reserved bike must only go to the customer who reserved it. The stored procedure has three checks before its UPDATE. The last check is given. You write the first two.

Your turn **usp_Bike_Sell: add the sold check and the reserved-for-someone-else check** SQL

1. In SQL Server Management Studio (SSMS), click on the empty line above the line that prints BikeShop is ready and type the stored procedure's name, its three parameters, AS, BEGIN and the blank message.
2. Add the first check: IF EXISTS a Bike row with this number and the status Sold, SET @Message = 'That bike has already been sold.'; and RETURN.
3. Add the second check: IF EXISTS a Bike row with this number, the status Reserved, and a CustomerID that is not @CustomerID, SET @Message = 'That bike is reserved for another customer.'; and RETURN.
4. Then copy the last check and the UPDATE from the fold-out below: the bike must be for sale and either Available or Reserved, and the UPDATE marks it Sold with the customer and today's date. Finish with END and GO.
5. Select the lines you typed, from CREATE OR ALTER PROCEDURE down to GO, and press F5.

**THE TEST** The ready-made test on the next screen tries three sales: a bike sold ten days ago, a reserved bike to the wrong customer, and the same reserved bike to the customer who reserved it.

**YOU SHOULD SEE** 'That bike has already been sold.' then 'That bike is reserved for another customer.' then 'sold to Méabh Cunningham.' If the first or second line says sold, one of your checks is missing or has the wrong status word.

**HINT** Not equal in SQL is <>. The second check reads AND Status = 'Reserved' AND CustomerID <> @CustomerID. The order of the three checks matters: a sold bike must be caught first, so that the message says why.

**Stuck? Show me the finished procedure**

**Read it, then write yours** **usp_Bike_Sell** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
CREATE OR ALTER PROCEDURE usp_Bike_Sell
    @BikeID     INT,
    @CustomerID INT,
    @Message VARCHAR(200) OUTPUT
AS
BEGIN
    SET @Message = '';
    IF EXISTS (SELECT * FROM Bike WHERE BikeID = @BikeID AND Status = 'Sold')
    BEGIN
        SET @Message = 'That bike has already been sold.';
        RETURN;
    END
    IF EXISTS (SELECT * FROM Bike WHERE BikeID = @BikeID
        AND Status = 'Reserved' AND CustomerID <> @CustomerID)
    BEGIN
        SET @Message = 'That bike is reserved for another customer.';
        RETURN;
    END

    -- The WHERE clause only matches a bike that can still be sold, so a bike is
    -- sold once.
    IF NOT EXISTS (SELECT * FROM Bike WHERE BikeID = @BikeID
        AND Purpose = 'Sale' AND Status IN ('Available', 'Reserved'))
    BEGIN
        SET @Message = 'That bike is not for sale right now.';
        RETURN;
    END

    UPDATE Bike
    SET Status = 'Sold', CustomerID = @CustomerID,
        SoldOn = CAST(GETDATE() AS DATE)
    WHERE BikeID = @BikeID AND Purpose = 'Sale'
        AND Status IN ('Available', 'Reserved');
END
GO
```

**1 The name, and what goes in and out**

Lines 1 to 4

The bike and the customer buying it. @Message comes back out.

**2 Start with a blank message**

Lines 5 to 7

AS and BEGIN mark where the work starts. The first job is to make @Message empty. An empty message means it worked. Every refusal below puts a sentence into @Message instead, so the program always knows which it was.

**3 Refuse a bike that is sold: the first check you write**

Lines 8 to 12

A sold bike is never sold again. This comes first so that the sentence says the real reason.

**4 Refuse a bike reserved for someone else: the second check you write**

Lines 13 to 18

Reserved, and the customer on the row is not this customer. The reserving customer can still buy it, because for them the CustomerID matches.

**5 Refuse anything that is not for sale**

Lines 22 to 27

A hire bike, a bike in repair, or a number that matches nothing. After the two checks above, this catches everything else.

**6 Mark it Sold**

Lines 29 to 35

The status, the customer and today's date. The WHERE repeats the conditions, so the row changes only if it can still be sold at that instant. GO ends the batch, ready for the next stored procedure.

# Test the bike sale three ways.

**Copy this one** **A test that tries three sales, of which only the third should work** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  -- Three sales: bike 11 was sold ten days ago, bike 10 is reserved for
2  -- Méabh Cunningham (customer 9).  Only the third one should work.
3  DECLARE @Message VARCHAR(200);
4
5  EXEC usp_Bike_Sell @BikeID = 11, @CustomerID = 1,
6       @Message = @Message OUTPUT;
7  PRINT 'Bike 11 to customer 1: ' + @Message;
8
9  EXEC usp_Bike_Sell @BikeID = 10, @CustomerID = 1,
10      @Message = @Message OUTPUT;
11 PRINT 'Bike 10 to customer 1: ' + @Message;
12
13 EXEC usp_Bike_Sell @BikeID = 10, @CustomerID = 9,
14      @Message = @Message OUTPUT;
15 IF @Message = ''
16 BEGIN
17     PRINT 'Bike 10 to customer 9: sold to Méabh Cunningham.';
18 END
19 ELSE
20 BEGIN
21     PRINT 'Bike 10 to customer 9: ' + @Message;
22 END
```

**1 One box for the answer**

Line 3

Only @Message is needed.

**2 A bike that was sold ten days ago**

Lines 5 to 7

Bike 11 was sold to Oisín Fegan in the sample data. Selling it to Aoife McCann must be refused.

**3 A reserved bike to the wrong customer**

Lines 9 to 11

Bike 10 is reserved for Méabh Cunningham, customer 9. Aoife cannot buy it.

**4 The reserved bike to its own customer**

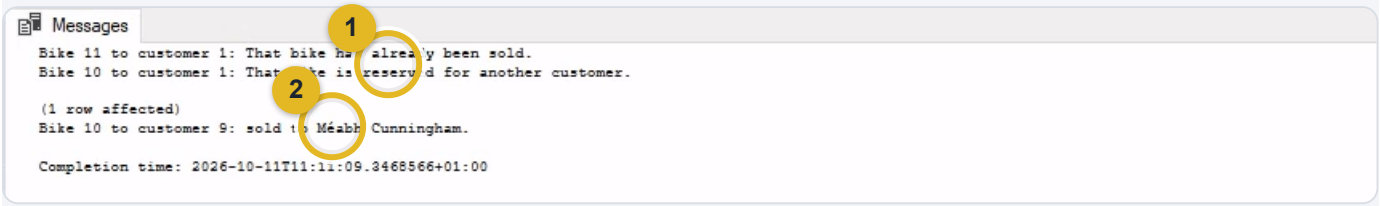
Lines 13 to 22

The one sale that should work. Méabh buys the bike she reserved.

Click New Query, press the Copy button under this step, and paste the test into the empty window.

> **You should see** The test is in a new query window of its own, with your script still open in its tab.

Press F5 to run the test.




Two refusals and one sale. A bike is sold once, and a reserved bike goes only to the customer who reserved it.

1. The sold bike and the reserved bike are both refused, each with its own sentence.

2. The sale that is allowed: the reserved bike to its own customer.

> **You should see** The Messages pane says: Bike 11 to customer 1: That bike has already been sold. Bike 10 to customer 1: That bike is reserved for another customer. Then (1 row affected) and Bike 10 to customer 9: sold to Méabh Cunningham.

Press F5 again, changing nothing.

> **You should see** The third line now reads: Bike 10 to customer 9: That bike has already been sold. Méabh bought it a moment ago, and the first check catches even her.

# Read the stored procedures for repair, for a new bike and for the bike list, then add them.

Three more. Repair goes both ways in one stored procedure, with IF and ELSE choosing which way. Add is the same shape as adding a part. The list takes a filter, so one stored procedure serves the three choices on the bikes screen.

Read this one **usp_Bike_SetRepair: into repair, or back in service** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
CREATE OR ALTER PROCEDURE usp_Bike_SetRepair
    @BikeID   INT,
    @InRepair BIT,
    @Message VARCHAR(200) OUTPUT
AS
BEGIN
    SET @Message = '';
    IF @InRepair = 1
    BEGIN
        -- Only a bike that is available can go into repair.
        IF NOT EXISTS (SELECT * FROM Bike WHERE BikeID = @BikeID
            AND Status = 'Available')
        BEGIN
            SET @Message = 'Only an available bike can go into repair.';
            RETURN;
        END
        UPDATE Bike SET Status = 'In repair' WHERE BikeID = @BikeID;
    END
    ELSE
    BEGIN
        -- Only a bike that is in repair can come back.
        IF NOT EXISTS (SELECT * FROM Bike WHERE BikeID = @BikeID
            AND Status = 'In repair')
        BEGIN
            SET @Message = 'Only a bike in repair can come back.';
            RETURN;
        END
        UPDATE Bike SET Status = 'Available' WHERE BikeID = @BikeID;
    END
END
GO
```

**1 The name, and what goes in and out**

Lines 1 to 4

The bike, and a BIT: 1 means into repair, 0 means back in service.

**2 Start with a blank message**

Lines 5 to 7

AS and BEGIN mark where the work starts. The first job is to make @Message empty. An empty message means it worked. Every refusal below puts a sentence into @Message instead, so the program always knows which it was.

**3 Which way?**

Lines 8 to 9

IF with a BEGIN and END block for each way. The first block is for going into repair.

**4 Into repair: only from Available**

Lines 11 to 16

A reserved or sold bike, or one already in repair, is refused.

**5 Mark it In repair**

Lines 17 to 18

The status changes. The END after it closes the first block.

**6 Otherwise, the second way**

Lines 19 to 20

ELSE runs when @InRepair is 0: the bike is coming back.

**7 Back in service: only from In repair**

Lines 22 to 27

A bike that is not in repair cannot come back from it.

**8 Mark it Available**

Lines 28 to 31

The status goes back. Two ENDs close the second block and the stored procedure. GO ends the batch, ready for the next stored procedure.

Read this one **usp_Bike_Add: save a new bike** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
CREATE OR ALTER PROCEDURE usp_Bike_Add
    @FrameNumber VARCHAR(20),
    @Make        VARCHAR(30),
    @Model       VARCHAR(40),
    @Size        VARCHAR(2),
    @Colour      VARCHAR(20),
    @Purpose     VARCHAR(4),
    @DailyRate   DECIMAL(6,2),
    @SalePrice   DECIMAL(8,2),
    @NewBikeID   INT OUTPUT,
    @Message VARCHAR(200) OUTPUT
AS
BEGIN
    SET @Message = '';
    IF EXISTS (SELECT * FROM Bike WHERE FrameNumber = @FrameNumber)
    BEGIN
        SET @Message = 'A bike with that frame number is already on record.';
        RETURN;
    END

    INSERT INTO Bike (FrameNumber, Make, Model, Size, Colour, Purpose,
        DailyRate, SalePrice)
    VALUES (@FrameNumber, @Make, @Model, @Size, @Colour, @Purpose, @DailyRate,
        @SalePrice);
    SET @NewBikeID = SCOPE_IDENTITY();
END
GO
```

**1 The name, and what goes in and out**

Lines 1 to 11

Everything about the bike. One of the two prices is NULL, because a hire bike has no sale price and a bike for sale has no daily rate.

**2 Start with a blank message**

Lines 12 to 14

AS and BEGIN mark where the work starts. The first job is to make @Message empty. An empty message means it worked. Every refusal below puts a sentence into @Message instead, so the program always knows which it was.

**3 Refuse a frame number that is on record**

Lines 15 to 19

The frame number is stamped on the bike, so two rows cannot share one.

**4 Save the bike**

Lines 21 to 24

One new row. Status starts as Available from the table's default.

**5 Hand back the new number**

Lines 25 to 27

The number of the new row goes back through the OUTPUT parameter. GO ends the batch, ready for the next stored procedure.

Read this one **usp_Bike_GetAll: every bike, or only one purpose** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
CREATE OR ALTER PROCEDURE usp_Bike_GetAll
    @Purpose VARCHAR(4)      -- 'Hire', 'Sale' or NULL for every bike
AS
BEGIN
    SELECT b.BikeID, b.FrameNumber,
           b.Make + ' ' + b.Model AS Bike,
           b.Size, b.Colour, b.Purpose, b.DailyRate, b.SalePrice, b.Status,
           c.FirstName + ' ' + c.LastName AS Customer,
           b.SoldOn
    FROM Bike b
    LEFT JOIN Customer c ON c.CustomerID = b.CustomerID
    WHERE @Purpose IS NULL OR b.Purpose = @Purpose
    ORDER BY b.Purpose, b.Make, b.Model, b.Size;
END
GO
```

**1 The name, and AS and BEGIN**

Lines 1 to 4

One parameter, @Purpose: Hire, Sale, or NULL for every bike. No @Message, because this only reads.

**2 What comes back**

Lines 5 to 9

The make and model joined into one Bike column, the two prices, the status, the customer as a name when there is one, and the sale date.

**3 Where the rows come from, and the filter**

Lines 10 to 15

LEFT JOIN Customer keeps the bikes that have no customer. The WHERE is the filter: when @Purpose is NULL every row passes, otherwise only the rows with that purpose. GO ends the batch, ready for the next stored procedure.

Press the Copy button under this step, click on the empty line above the line that prints BikeShop is ready, and paste.

> **You should see** The stored procedure is in your script, ending with GO, and the line that prints BikeShop is ready is still last.

Press the Copy button under this step, click on the empty line above the line that prints BikeShop is ready, and paste.

> **You should see** The add stored procedure is in your script, under the repair one.

Press the Copy button under this step, click on the empty line above the line that prints BikeShop is ready, and paste.

> **You should see** All three are in your script, each ending with GO.

Select all the lines you added on this screen, from the first CREATE OR ALTER PROCEDURE down to the last GO, and press F5.

> **You should see** The Messages pane says: Commands completed successfully. Your test sales and orders are still there.

**IF IT GOES WRONG** The Messages pane shows (6 rows affected) and BikeShop is ready

Nothing was selected, so the whole script ran and the sample data was put back. Your test sales, orders and reservations are gone. Nothing is broken: carry on, and remember that the stock of part 5 is 5 again.

Copy this one **A test that brings bike 6 back, sends it in again, and tries once more** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
-- Bike 6 (the Frog 62) is in repair.  Bring it back, send it in again,
-- then try to send it in a second time.
DECLARE @Message VARCHAR(200);

EXEC usp_Bike_SetRepair @BikeID = 6, @InRepair = 0,
     @Message = @Message OUTPUT;
PRINT 'Back in service: ' + @Message;

EXEC usp_Bike_SetRepair @BikeID = 6, @InRepair = 1,
     @Message = @Message OUTPUT;
PRINT 'Into repair: ' + @Message;

EXEC usp_Bike_SetRepair @BikeID = 6, @InRepair = 1,
     @Message = @Message OUTPUT;
PRINT 'Into repair again: ' + @Message;
```

**1 One box for the answer**

Line 3

Only @Message is needed.

**2 Bring bike 6 back**

Lines 5 to 7

Bike 6, the Frog 62, is in repair in the sample data. With 0 it comes back, and the message is empty.

**3 Send it in again**

Lines 9 to 11

Now it is available, so with 1 it goes into repair, and the message is empty again.

**4 Send it in once more**

Lines 13 to 15

It is already in repair, so this one is refused.

Click New Query, press the Copy button under this step, and paste the test into the empty window.

> **You should see** The test is in a new query window of its own, with your script still open in its tab.

Press F5 to run the test.

> **You should see** The Messages pane says: Back in service: with nothing after it, then Into repair: with nothing after it, then Into repair again: Only an available bike can go into repair. An empty message is the stored procedure saying it worked.

# Read the stored procedure that finds the free hire bikes, then add it.

The hire form in stage 8 asks one question: which hire bikes are free for every day from the start date to the return date? The answer is the clash test from chapter 7, inside NOT EXISTS, with dates in place of times. A bike is free when no hire of it that is Booked or Out overlaps the dates.

Copy this one **usp_Bike_GetAvailableForHire: the hire bikes free for every day between tw…** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
CREATE OR ALTER PROCEDURE usp_Bike_GetAvailableForHire
    @StartDate DATE,
    @EndDate   DATE
AS
BEGIN
    SELECT b.BikeID,
           b.Make + ' ' + b.Model + ' · ' + b.Size + ' · ' + b.Colour + ' ('
               + b.FrameNumber + ')' AS Display,
           b.DailyRate
    FROM Bike b
    WHERE b.Purpose = 'Hire'
      AND b.Status  = 'Available'
      AND NOT EXISTS (SELECT *
                      FROM Hire h
                      WHERE h.BikeID = b.BikeID
                        AND h.Status IN ('Booked', 'Out')
                        AND @StartDate < h.EndDate
                        AND @EndDate   > h.StartDate)
    ORDER BY b.Make, b.Model, b.Size;
END
GO
```

**1 The name, and what goes in and out**

Lines 1 to 3

The start date and the return date. A table comes back.

**2 What comes back**

Lines 4 to 9

The bike number, one Display column that joins the make, model, size, colour and frame number into a sentence for the list, and the daily rate for the cost line.

**3 Only hire bikes that are available**

Lines 10 to 12

A bike for sale is never offered, and neither is a bike in repair.

**4 And no hire of it overlaps the dates**

Lines 13 to 18

The clash test. A hire of this bike, Booked or Out, that starts before the new return date and ends after the new start date, overlaps. If there is one, the bike is left out.

**5 In a sensible order**

Lines 19 to 21

Make, model, then size, so the list reads like a catalogue. GO ends the batch, ready for the next stored procedure.

Press the Copy button under this step, click on the empty line above the line that prints BikeShop is ready, and paste.

> **You should see** The stored procedure is in your script, ending with GO, and the line that prints BikeShop is ready is still last.

Select the lines you just added, from CREATE OR ALTER PROCEDURE down to GO, and press F5.

> **You should see** The Messages pane says: Commands completed successfully. Your test sales and orders are still there.

**IF IT GOES WRONG** The Messages pane shows (6 rows affected) and BikeShop is ready

Nothing was selected, so the whole script ran and the sample data was put back. Your test sales, orders and reservations are gone. Nothing is broken: carry on, and remember that the stock of part 5 is 5 again.

Copy this one **A test that asks which hire bikes are free for two different date ranges** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
-- Which hire bikes are free for one day, four days from now?  Then the
-- same question for two days, nine days from now.
DECLARE @From DATE, @To DATE;
SET @From = DATEADD(DAY, 4, CAST(GETDATE() AS DATE));
SET @To = DATEADD(DAY, 1, @From);
EXEC usp_Bike_GetAvailableForHire @StartDate = @From, @EndDate = @To;

SET @From = DATEADD(DAY, 9, CAST(GETDATE() AS DATE));
SET @To = DATEADD(DAY, 2, @From);
EXEC usp_Bike_GetAvailableForHire @StartDate = @From, @EndDate = @To;
```

**1 One day, four days from now**

Lines 3 to 6

Hire 4 in the sample data has bike 1 from three days from now for three days, so bike 1 is busy on this day.

**2 Two days, nine days from now**

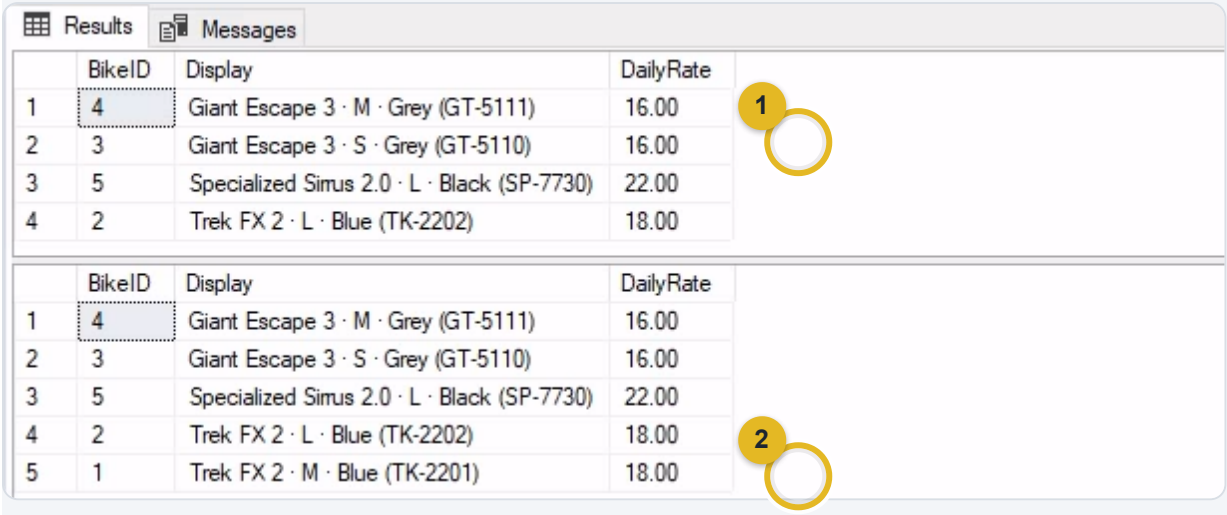
Lines 8 to 10

Nothing is booked then, so every available hire bike is free, bike 1 included.

Click New Query, press the Copy button under this step, and paste the test into the empty window.

> **You should see** The test is in a new query window of its own, with your script still open in its tab.

Press F5 to run the test.




The free bikes for two different date ranges. The Trek FX 2 M is in the second table but not the first, because hire 4 has it for the first dates.

1. Four bikes free. Bike 1 is missing because of the booked hire.

2. Five bikes free, bike 1 among them. The Display column is the sentence the hire form shows in its list.

> **You should see** Two tables appear in the Results tab. The first has four bikes, and the Trek FX 2 M is not among them. The second has five, with the Trek FX 2 M back. Each row shows the Display sentence the hire form will list.

**IF IT GOES WRONG** The first table has five bikes too

Your clash test is letting hire 4 through. Check the two date questions: @StartDate < h.EndDate and @EndDate > h.StartDate, and that the status list reads ('Booked', 'Out').

# Add the Bike class and BikeData.

One class for a bike, and one data class with a method for each of the seven bike stored procedures. The two prices in the class are decimal with a question mark, which means they can be empty, because a bike has only one of them.

**Copy this one** **Models/Bike.cs: one bike** C#

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  namespace BikeShop.Models
2  {
3      // One bike, for hire or for sale.  Plain properties only.  The price that
4      // does not apply is null.
5      public class Bike
6      {
7          public int BikeID { get; set; }
8          public string FrameNumber { get; set; }
9          public string Make { get; set; }
10         public string Model { get; set; }
11         public string Size { get; set; }
12         public string Colour { get; set; }
13         public string Purpose { get; set; }
14         public decimal? DailyRate { get; set; }
15         public decimal? SalePrice { get; set; }
16     }
17 }
```

**1 The class**

Lines 1 to 6

No using lines. It sits in the Models folder with Part.

**2 One property for each column**

Lines 7 to 17

Nine properties with the same names as the Bike table's columns. decimal? with the question mark means a decimal that can be empty, for the price that does not apply.

Copy this one **Data/BikeData.cs: one method for each bike stored procedure** C#

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
using System;
using System.Data;
using System.Data.SqlClient;
using BikeShop.Models;

namespace BikeShop.Data
{
    public static class BikeData
    {
        // purpose = "Hire", "Sale" or null for every bike
        public static DataTable GetAll(string purpose)
        {
            return Db.GetTable("usp_Bike_GetAll", Db.P("@Purpose", purpose));
        }

        #region GetAvailableForHire
        // The hire bikes that are free for every day from startDate to endDate.
        public static DataTable GetAvailableForHire(DateTime startDate,
            DateTime endDate)
        {
            return Db.GetTable("usp_Bike_GetAvailableForHire",
                Db.P("@StartDate", startDate.Date),
                Db.P("@EndDate", endDate.Date));
        }
        #endregion

        // Saves a new bike.  Returns "" if it worked, or the sentence to show
        // the user.
        public static string Add(Bike bike)
        {
            SqlParameter newID = Db.Out("@NewBikeID");
            string message = Db.Run("usp_Bike_Add",
                Db.P("@FrameNumber", bike.FrameNumber),
                Db.P("@Make", bike.Make),
                Db.P("@Model", bike.Model),
                Db.P("@Size", bike.Size),
                Db.P("@Colour", bike.Colour),
                Db.P("@Purpose", bike.Purpose),
                Db.P("@DailyRate", bike.DailyRate),
                Db.P("@SalePrice", bike.SalePrice),
                newID);
            if (message == "")
            {
                bike.BikeID = (int)newID.Value;
            }
            return message;
        }

        #region Lifecycle
        // Reserves a sale bike for a customer.
        public static string Reserve(int bikeID, int customerID)
        {
            return Db.Run("usp_Bike_Reserve", Db.P("@BikeID", bikeID),
```

```
54                  Db.P("@CustomerID", customerID));
55          }
56
57          // Puts a reserved bike back on sale.
58          public static string Unreserve(int bikeID)
59          {
60              return Db.Run("usp_Bike_Unreserve", Db.P("@BikeID", bikeID));
61          }
62
63          // Sells a bike, once only.
64          public static string Sell(int bikeID, int customerID)
65          {
66              return Db.Run("usp_Bike_Sell", Db.P("@BikeID", bikeID),
67                  Db.P("@CustomerID", customerID));
68          }
69
70          // Marks a bike as in the repair stand (true) or back in the shop
71          // (false).
72          public static string SetRepair(int bikeID, bool inRepair)
73          {
74              return Db.Run("usp_Bike_SetRepair", Db.P("@BikeID", bikeID),
75                  Db.P("@InRepair", inRepair));
76          }
77          #endregion
78      }
79  }
```

**1 The usings**

Lines 1 to 4

Each using line names a set of ready-made tools this file needs. System.Data.SqlClient is the one that talks to SQL Server. BikeShop.Models is where your own classes live.

**2 The class, and GetAll**

Lines 6 to 14

The purpose is passed straight through: Hire, Sale, or null for every bike. Db.P with null sends a NULL to the stored procedure.

**3 GetAvailableForHire: the hire form's list**

Lines 16 to 25

Two dates in, the free bikes out. .Date throws away the clock part.

**4 Add: send the bike, read the answer**

Lines 29 to 47

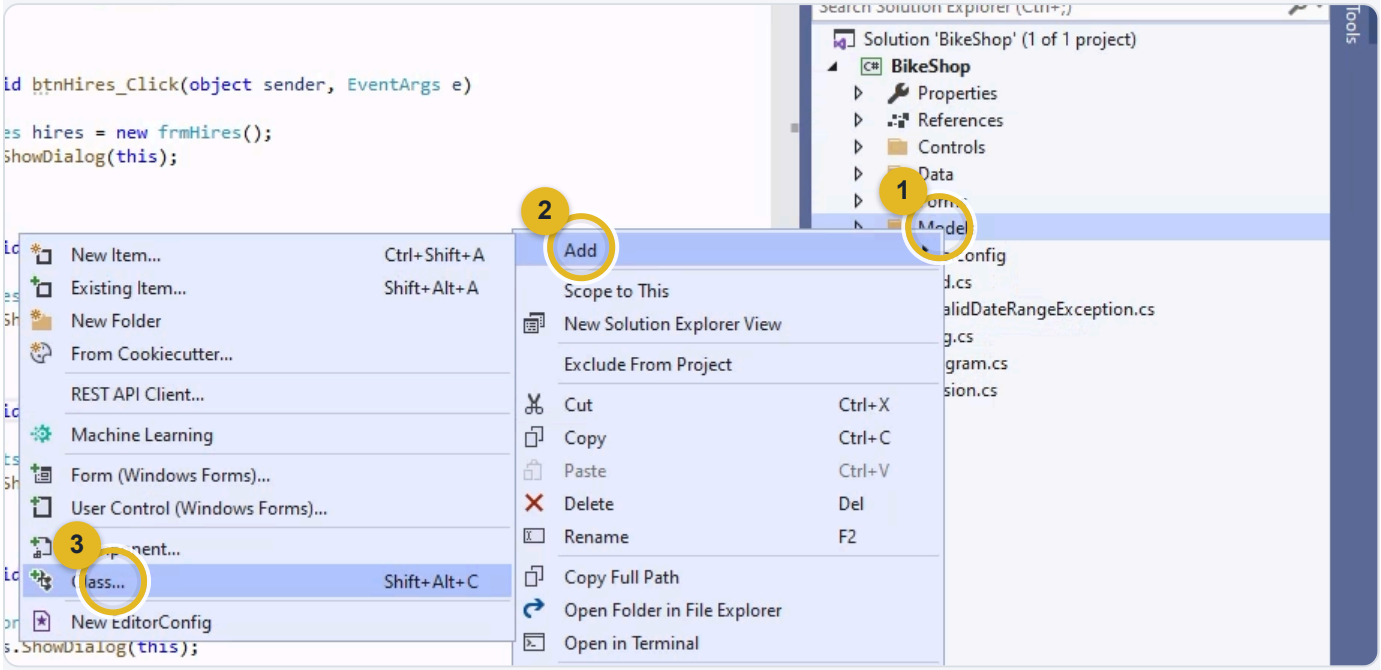
The same shape as adding a part. An empty price goes to the database as NULL on its own, because the property can be empty.

**5 Reserve, Unreserve, Sell and SetRepair**

Lines 49 to 79

Four short methods, one call each. SetRepair takes true or false, which SQL Server reads as 1 or 0 for the BIT. The two closing braces end the class and the namespace.

In Visual Studio 2019, in Solution Explorer, right-click the Models folder, point at Add, and click Class…




Adding a class to the Models folder. Solution Explorer lists the classes you made in chapters 5, 6 and 7.

1. Right-click the Models folder in Solution Explorer.
2. Point at Add.
3. Click Class…

> **You should see** The Add New Item window opens with Class selected.

In the Name box, type Bike.cs and press Add.

> **You should see** A new file opens with an empty class in it.

Click in the new file, press Ctrl and A together to select everything, press the Copy button under this step, and paste over it.

> **You should see** Bike.cs holds the Bike class with its nine properties, and no red wavy line anywhere.

Right-click the Data folder, point at Add, and click Class…

> **You should see** The Add New Item window opens with Class selected.

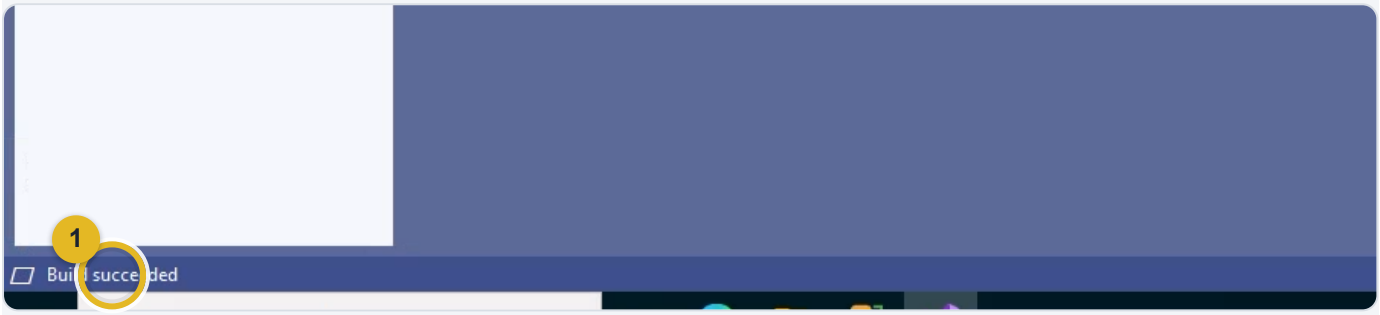
In the Name box, type BikeData.cs and press Add.

> **You should see** A new file opens with an empty class in it.

Click in the new file, press Ctrl and A together to select everything, press the Copy button under this step, and paste over it.

> **You should see** BikeData.cs holds seven methods, ending with SetRepair, and no red wavy line anywhere.

Press F6 to build.




Build succeeded, at the bottom left. Anything else means an error list to read.

1. Build succeeded. Press F6 to build at any time.

> **You should see** Build succeeded, at the bottom left.

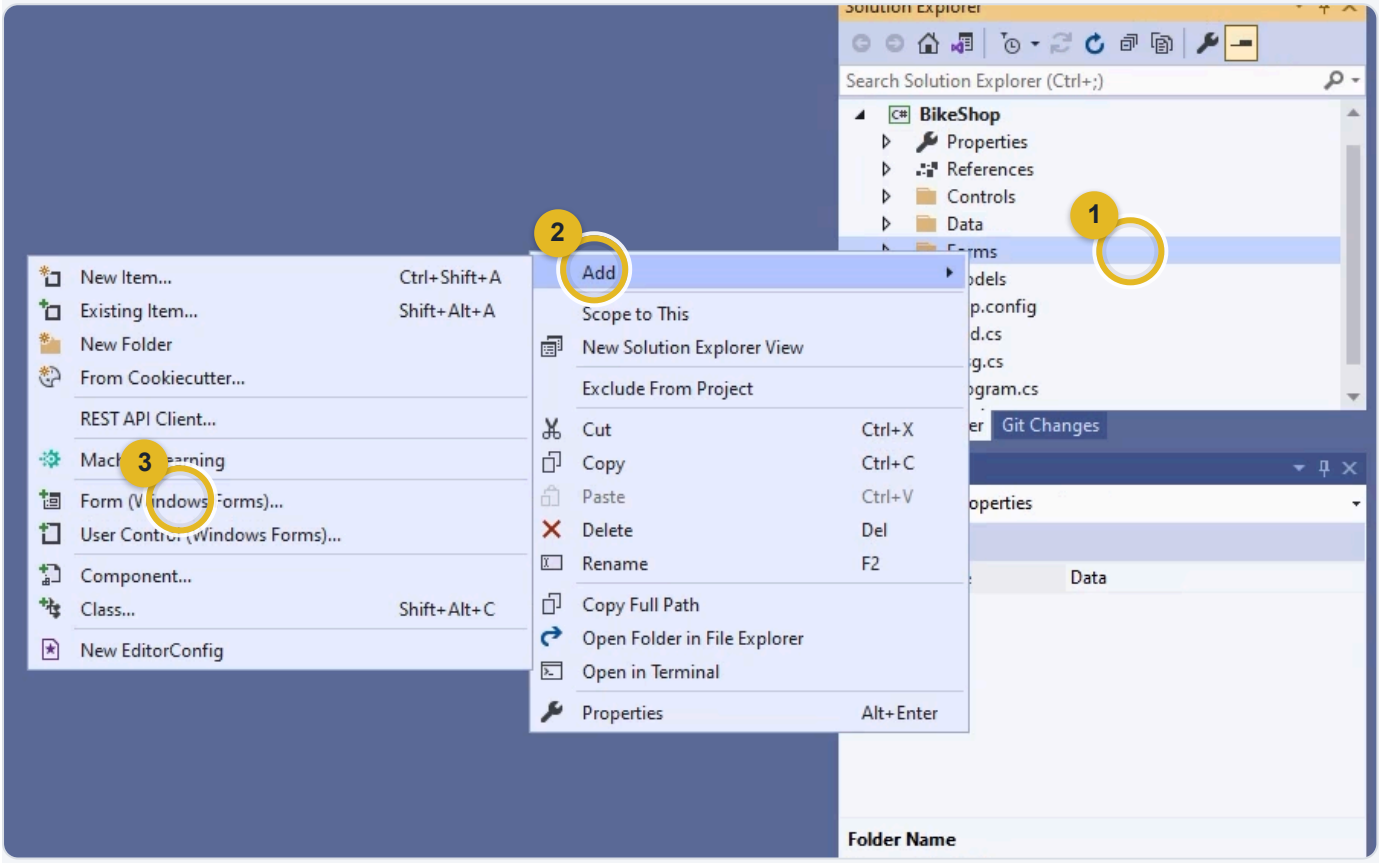
**IF IT GOES WRONG** 'The type or namespace name Bike could not be found'

The Bike class is missing from the Models folder. Go back to step 8.7.1 and add it.

# Add the new bike form and set its controls.

The new bike form first, because the bikes screen opens it. It has two round buttons, Hire and Sale, and only the price box that matches is switched on.

Right-click the Forms folder in Solution Explorer, point at Add, and click Form (Windows Forms)…




Adding a form to the Forms folder. Taken in the finished project, so Solution Explorer lists more files than yours.

1. Right-click the Forms folder.
2. Point at Add.
3. Click Form (Windows Forms)…, type the name, press Add.

> **You should see** The Add New Item window opens with Form (Windows Forms) selected.

Type frmBikeEdit in the Name box and press Add.

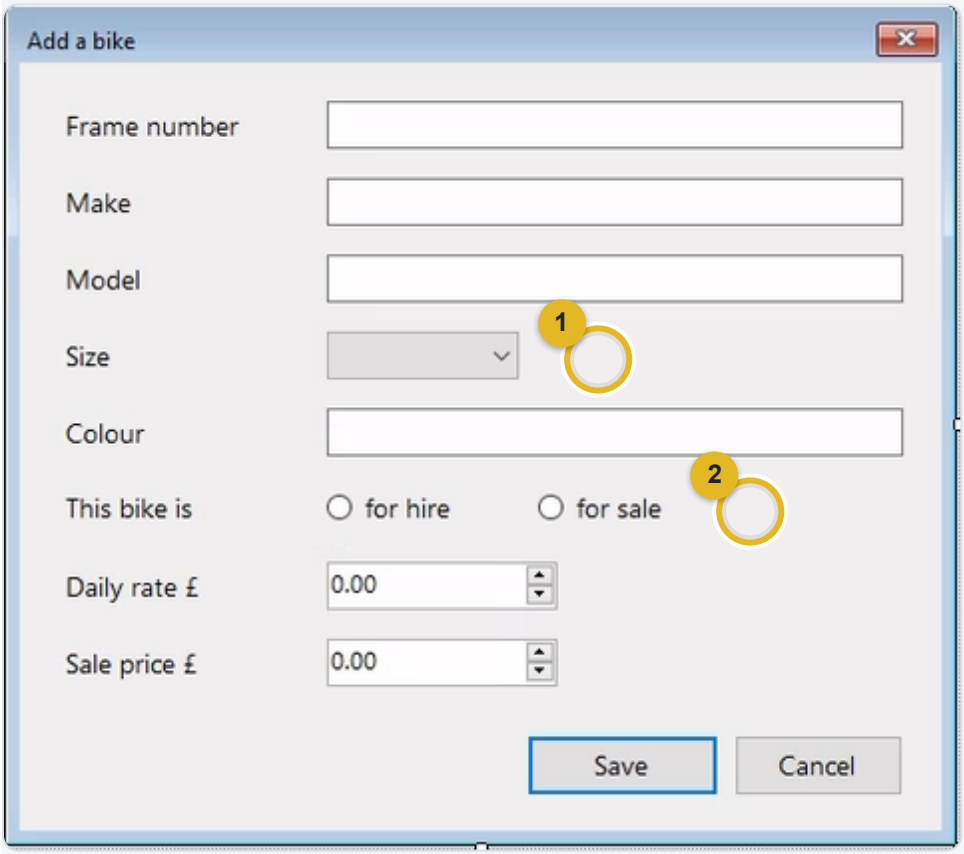
> **You should see** An empty form opens in the designer.

Click the form itself first and set the form's own properties. Then work down the table: drag the control from the Toolbox, drop it on the form, set its Name in the Properties window, and set the other settings in the row. Where the Then column says double-click, double-clicking the control makes its event method for you.

A hire bike has a daily rate; a bike for sale has a price. Only the box that applies is switched on.

**Click the form itself, then set these in the Properties window**

| | | | |
|---|---|---|---|
| Text | `Add a bike` | Size | `480, 400` |
| FormBorderStyle | `FixedDialog` | StartPosition | `CenterParent` |
| MaximizeBox | `False` | MinimizeBox | `False` |
| ShowInTaskbar | `False` | AcceptButton | `btnSave (after you add it)` |
| CancelButton | `btnCancel (after you add it)` | | |




frmBikeEdit in the designer. Only the price box that matches the purpose is switched on.

1. The five boxes that describe the bike.
2. for hire or for sale, then the one price that applies.

| DRAG THIS FROM THE TOOLBOX | NAME IT (THE NAME PROPERTY) | SET THESE IN THE PROPERTIES WINDOW | THEN |
|---|---|---|---|
| Label | `lblFrame` | `Text = Frame number` | |
| TextBox | `txtFrame` | nothing else<br>Stamped on the frame; no two bikes share one. | |
| Label | `lblMake` | `Text = Make` | |
| TextBox | `txtMake` | nothing else | |
| Label | `lblModel` | `Text = Model` | |
| TextBox | `txtModel` | nothing else | |
| Label | `lblSize` | `Text = Size` | |
| ComboBox | `cboSize` | `DropDownStyle = DropDownList`<br>`Items = XS / S / M / L / XL` | |
| Label | `lblColour` | `Text = Colour` | |
| TextBox | `txtColour` | nothing else | |
| Label | `lblPurpose` | `Text = This bike is` | |
| RadioButton | `rdoHire` | `Text = for hire` | Double-click it. Visual Studio makes `rdoHire_CheckedChanged` for you; the code goes inside it. |
| RadioButton | `rdoSale` | `Text = for sale` | Double-click it. Visual Studio makes `rdoSale_CheckedChanged` for you; the code goes inside it. |
| Label | `lblRate` | `Text = Daily rate £` | |
| NumericUpDown | `nudRate` | `DecimalPlaces = 2`<br>`Maximum = 1000` | |
| Label | `lblPrice` | `Text = Sale price £` | |
| NumericUpDown | `nudPrice` | `DecimalPlaces = 2`<br>`Maximum = 100000` | |
| Button | `btnSave` | `Text = Save` | Double-click it. Visual Studio makes `btnSave_Click` for you; the code goes |

| DRAG THIS FROM THE TOOLBOX | NAME IT (THE NAME PROPERTY) | SET THESE IN THE PROPERTIES WINDOW | THEN |
|---|---|---|---|
| | | | inside it. |
| Button | `btnCancel` | `Text = Cancel`<br>`DialogResult = Cancel` | |

STAGE 7 OF 10 · THE BIKES SCREEN · SCREEN 3 OF 6

# Put the code behind the new bike form.

Copy this one **Forms/frmBikeEdit.cs: the whole file** C#

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
using System;
using System.Collections.Generic;
using System.Data;
using System.Drawing;
using System.Windows.Forms;
using BikeShop.Data;
using BikeShop.Models;

namespace BikeShop.Forms
{
    public partial class frmBikeEdit : Form
    {
        public frmBikeEdit()
        {
            InitializeComponent();
        }

        private void frmBikeEdit_Load(object sender, EventArgs e)
        {
            // The boxes stop typing at the length the database column holds.
            txtFrame.MaxLength = 20;
            txtMake.MaxLength = 30;
            txtModel.MaxLength = 40;
            txtColour.MaxLength = 20;

            cboSize.SelectedIndex = 2;      // M
            rdoHire.Checked = true;
            ShowPriceBox();
        }

        #region Purpose
        // A hire bike has a daily rate; a bike for sale has a price.  Only the
        // box that applies is switched on.
        private void ShowPriceBox()
        {
            lblRate.Enabled = rdoHire.Checked;
            nudRate.Enabled = rdoHire.Checked;
            lblPrice.Enabled = rdoSale.Checked;
            nudPrice.Enabled = rdoSale.Checked;
        }

        private void rdoHire_CheckedChanged(object sender, EventArgs e)
        {
            ShowPriceBox();
        }

        private void rdoSale_CheckedChanged(object sender, EventArgs e)
        {
            ShowPriceBox();
        }
        #endregion

        #region Save
```

```
        private void btnSave_Click(object sender, EventArgs e)
        {
            // Check what was typed, one problem at a time: show the first and
            // put the cursor there.
            // The size list and the Hire / Sale buttons need no check: one
            // choice is always picked.
            if (txtFrame.Text.Trim() == "")
            {
                Msg.Problem("Type the frame number.");
                txtFrame.Focus();
                return;
            }
            if (txtMake.Text.Trim() == "")
            {
                Msg.Problem("Type the bike's make.");
                txtMake.Focus();
                return;
            }
            if (txtModel.Text.Trim() == "")
            {
                Msg.Problem("Type the bike's model.");
                txtModel.Focus();
                return;
            }
            if (txtColour.Text.Trim() == "")
            {
                Msg.Problem("Type the bike's colour.");
                txtColour.Focus();
                return;
            }
            if (rdoHire.Checked && nudRate.Value <= 0)
            {
                Msg.Problem("A hire bike needs a daily rate above £0.");
                nudRate.Focus();
                return;
            }
            if (rdoSale.Checked && nudPrice.Value <= 0)
            {
                Msg.Problem("A bike for sale needs a sale price above £0.");
                nudPrice.Focus();
                return;
            }

            Bike bike = new Bike();
            bike.FrameNumber = txtFrame.Text.Trim().ToUpper();
            bike.Make = txtMake.Text.Trim();
            bike.Model = txtModel.Text.Trim();
            bike.Size = cboSize.Text;
            bike.Colour = txtColour.Text.Trim();
            if (rdoHire.Checked)
            {
                bike.Purpose = "Hire";
                bike.DailyRate = nudRate.Value;
            }
            else
            {
                bike.Purpose = "Sale";
                bike.SalePrice = nudPrice.Value;
```

```
112             }
113
114             string message = BikeData.Add(bike);
115             if (message ≠ "")
116             {
117                 // e.g. that frame number is already on a bike
118                 Msg.Problem(message);
119                 return;
120             }
121             DialogResult = DialogResult.OK;
122         }
123         #endregion
124     }
125 }
```

**1 The usings**

Lines 1 to 7

The tools this form needs: Windows Forms for the window itself, System.Data for the tables that fill the grids and lists, System.Drawing for colours, and your own Data and Models folders.

**2 The form's class and its constructor**

Lines 9 to 16

This form only adds, so there is nothing to remember.

**3 When the form loads**

Lines 18 to 29

The boxes stop typing at the length the columns hold. Size starts at M, Hire starts chosen, and ShowPriceBox switches the right price box on.

**4 Hire or Sale**

Lines 31 to 51

ShowPriceBox switches the daily rate on for a hire bike and the sale price on for a bike for sale. Both round buttons call it when they change.

**5 The Save button**

Lines 53 to 125

The Save button runs six checks, one problem at a time. Then a Bike object is filled, with only the price that applies, and one call goes to the database. If the message is empty, the form closes with OK.

With frmBikeEdit open in the designer, press F7 to see its code.

> **You should see** The code tab for the form comes to the front.

Press Ctrl and A together to select everything in the code tab, press the Copy button under this step, and paste over it.

> **You should see** The code tab holds the whole file, and the old lines are gone.

Press F6 to build.

**You should see** Build succeeded, at the bottom left.

**IF IT GOES WRONG** frmBikeEdit_Load or btnSave_Click is underlined: no suitable method found

The designer did not make that event. Go back to the designer, click the form (or the button), open the Properties window, press the lightning bolt, and double-click the empty box beside the event. Then build again.

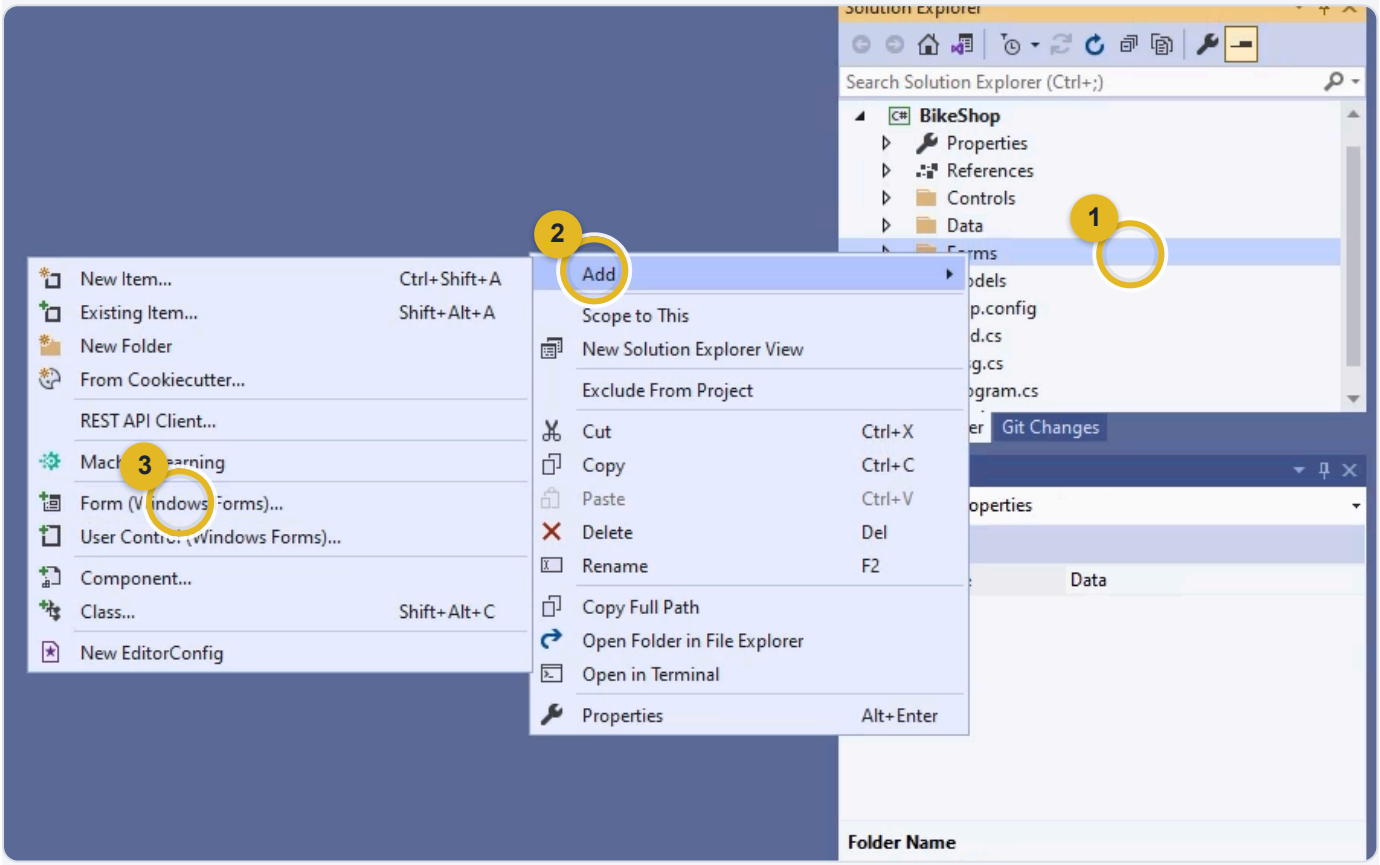
**IF IT GOES WRONG** rdoHire_CheckedChanged or rdoSale_CheckedChanged is underlined

The round buttons' events were not made. Double-click each round button in the designer, which makes its CheckedChanged method, and build again.

# Add the bikes screen and set its controls.

One grid for every bike, a filter above it, a customer list for reserving and selling, and the buttons that move a bike through its life. The repair button changes its own words to say which way it will send the bike.

Right-click the Forms folder in Solution Explorer, point at Add, and click Form (Windows Forms)…




Adding a form to the Forms folder. Taken in the finished project, so Solution Explorer lists more files than yours.

1. Right-click the Forms folder.
2. Point at Add.
3. Click Form (Windows Forms)…, type the name, press Add.

> **You should see** The Add New Item window opens with Form (Windows Forms) selected.

Type frmBikes in the Name box and press Add.

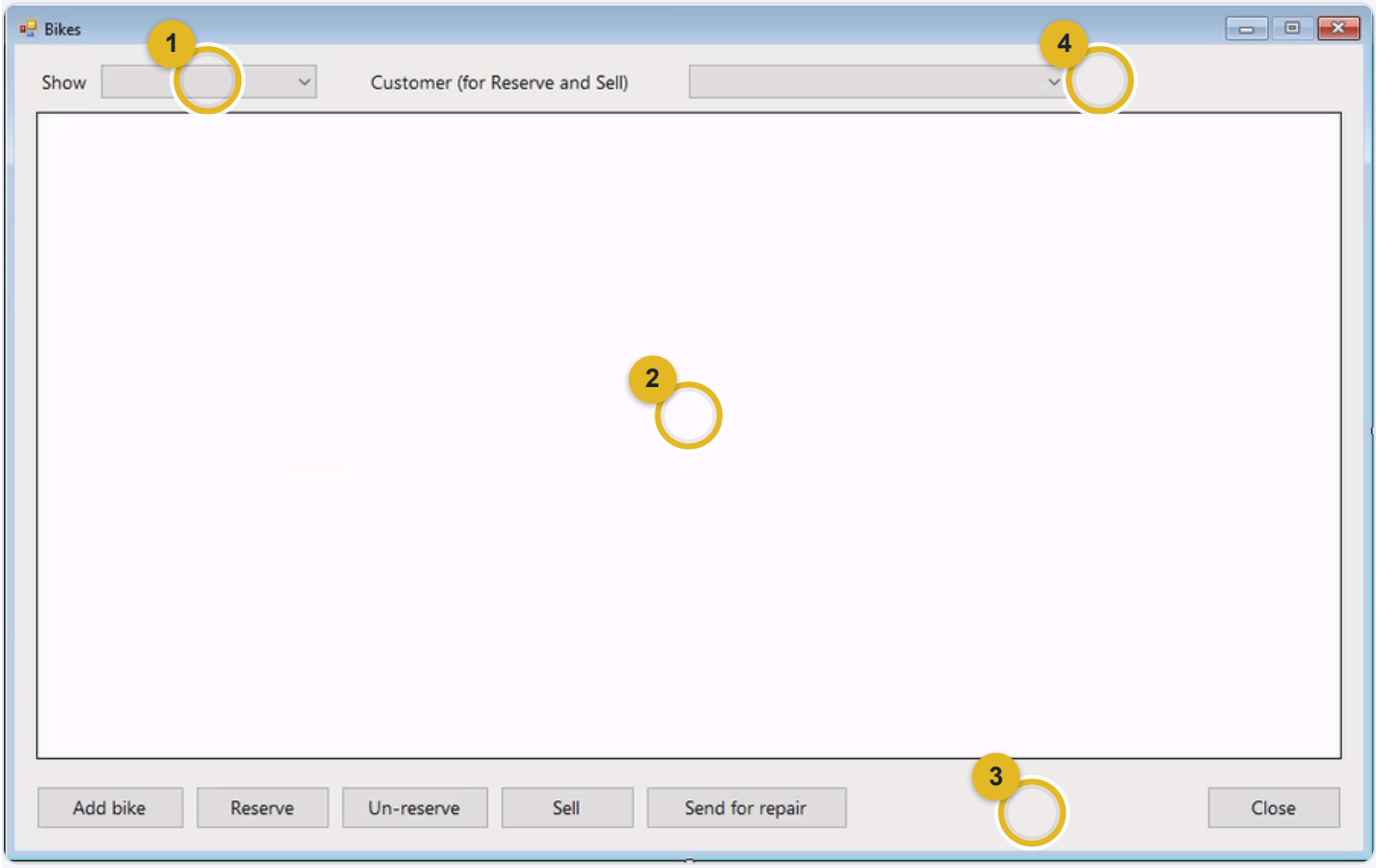
> **You should see** An empty form opens in the designer.

Click the form itself first and set the form's own properties. Then work down the table: drag the control from the Toolbox, drop it on the form, set its Name in the Properties window, and set the other settings in the row. Where the Then column says double-click, double-clicking the control makes its event method for you.

Every bike the shop owns. Sale bikes go Available → Reserved → Sold; any bike can go in and out of repair.

**Click the form itself, then set these in the Properties window**

| | | | |
|---|---|---|---|
| Text | `Bikes` | Size | `1000, 600` |
| FormBorderStyle | `FixedSingle` | StartPosition | `CenterParent` |
| MaximizeBox | `False` | | |




frmBikes in the designer. One grid for every bike, with the buttons that move a bike through its life.

1. The Show list: All bikes, For sale, Hire fleet.
2. The bikes grid, dgvBikes.
3. The buttons that move a bike through its life, and Close.
4. The customer list: Reserve and Sell use the customer picked here.

| DRAG THIS FROM THE TOOLBOX | NAME IT (THE NAME PROPERTY) | SET THESE IN THE PROPERTIES WINDOW | THEN |
|---|---|---|---|
| Label | `lblShow` | `Text = Show` | |
| ComboBox | `cboFilter` | `DropDownStyle = DropDownList` `Items = All bikes / For sale / Hire fleet` | Double-click it. Visual Studio makes `cboFilter_SelectedIndexChanged` for you; the code goes inside it. |
| Label | `lblCustomer` | `Text = Customer (for Reserve and Sell)` | |
| ComboBox | `cboCustomer` | `DropDownStyle = DropDownList` | |
| DataGridView | `dgvBikes` | `the grid settings (chapter 6)` | Events tab (the lightning bolt) → SelectionChanged: type `dgvBikes_SelectionChanged` beside it and press Enter. |
| Button | `btnAdd` | `Text = Add bike` | Double-click it. Visual Studio makes `btnAdd_Click` for you; the code goes inside it. |
| Button | `btnReserve` | `Text = Reserve` `Enabled = False` | Double-click it. Visual Studio makes `btnReserve_Click` for you; the code goes inside it. |
| Button | `btnUnreserve` | `Text = Un-reserve` `Enabled = False` Back to Available: the bike returns to stock. | Double-click it. Visual Studio makes `btnUnreserve_Click` for you; the code goes inside it. |
| Button | `btnSell` | `Text = Sell` `Enabled = False` Once. The database refuses a second sale. | Double-click it. Visual Studio makes `btnSell_Click` for you; the code goes inside it. |
| Button | `btnRepair` | `Text = Send for repair` `Enabled = False` Reads "Back in service" for a bike in repair. | Double-click it. Visual Studio makes `btnRepair_Click` for you; the code goes inside it. |

| DRAG THIS FROM THE TOOLBOX | NAME IT (THE NAME PROPERTY) | SET THESE IN THE PROPERTIES WINDOW | THEN |
|---|---|---|---|
| Button | `btnClose` | `Text = Close` | Double-click it. Visual Studio makes `btnClose_Click` for you; the code goes inside it. |

# Read how the buttons follow the status, then put the code behind the bikes screen.

The part to read first is the selection method. When the picked row changes, every button is switched on or off from two facts: whether the bike is for sale, and what its status is. That is the life of a bike from stage 1, written as five lines. You type those lines.

Read this one **Forms/frmBikes.cs: which buttons are allowed** C#

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  // Which buttons make sense depends on the bike's purpose and where it
2  // is in its life:
3  // Available → Reserved → Sold for a sale bike; Available ↔ In repair
4  // for any bike.
5  private void dgvBikes_SelectionChanged(object sender, EventArgs e)
6  {
7      bool picked = dgvBikes.SelectedRows.Count > 0;
8      string status = "";
9      if (picked)
10     {
11         status = Grid.SelectedText(dgvBikes, "Status");
12     }
13     bool forSale = picked
14         && Grid.SelectedText(dgvBikes, "Purpose") == "Sale";
15
16     btnReserve.Enabled = forSale && status == "Available";
17     btnUnreserve.Enabled = forSale && status == "Reserved";
18     btnSell.Enabled = forSale
19         && (status == "Available" || status == "Reserved");
20     btnRepair.Enabled = picked
21         && (status == "Available" || status == "In repair");
22     if (status == "In repair")
23     {
24         btnRepair.Text = "Back in service";
25     }
26     else
27     {
28         btnRepair.Text = "Send for repair";
29     }
30
31     // A reserved bike can only be sold to the customer who reserved it,
32     // so pick them in the box.
33     if (status == "Reserved")
34     {
35         cboCustomer.SelectedIndex =
36             cboCustomer.FindStringExact(Grid.SelectedText(dgvBikes,
37             "Customer"));
38     }
39 }
```

**1 Start with no status**

Lines 5 to 8

picked is true when a row is selected. status starts empty, in case nothing is picked.

**2 Read the status from the row**

Lines 9 to 12

Grid.SelectedText reads one cell of the picked row as words.

**3 Switch the buttons**

Lines 13 to 29

Reserve needs an available bike for sale. Unreserve needs a reserved one. Sell works on an available bike for sale or a reserved one. Repair works on any bike that is available or in repair, and its words say which way it will go.

**4 Pick the reserving customer**

Lines 33 to 39

A reserved bike can only be sold to the customer who reserved it, so the customer list is set to them, by name.

**Why you type it** Five lines that make the wrong button impossible to press. The stored procedure would refuse anyway, but the counter staff should never see a button that cannot work.

**Type this one** **Forms/frmBikes.cs: the whole file** C#

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
using System;
using System.Collections.Generic;
using System.Data;
using System.Drawing;
using System.Windows.Forms;
using BikeShop.Data;
using BikeShop.Models;

namespace BikeShop.Forms
{
    public partial class frmBikes : Form
    {
        public frmBikes()
        {
            InitializeComponent();
        }

        private void frmBikes_Load(object sender, EventArgs e)
        {
            cboCustomer.DisplayMember = "FullName";
            cboCustomer.ValueMember = "CustomerID";
            cboCustomer.DataSource = CustomerData.Search("");

            // "All bikes" — this also fills the grid (see below)
            cboFilter.SelectedIndex = 0;
        }

        #region LoadBikes
        private void LoadBikes()
        {
            // The filter box holds "All bikes", "For sale" and "Hire fleet";
            // the procedure takes Sale, Hire or nothing.
            string purpose = null;
            if (cboFilter.SelectedIndex == 1)
            {
                purpose = "Sale";
            }
            else if (cboFilter.SelectedIndex == 2)
            {
                purpose = "Hire";
            }

            dgvBikes.DataSource = BikeData.GetAll(purpose);
            Grid.Tidy(dgvBikes, "BikeID");
        }

        private void cboFilter_SelectedIndexChanged(object sender, EventArgs e)
        {
            LoadBikes();
        }
        #endregion

        #region Selection
```

```
54          // Which buttons make sense depends on the bike's purpose and where it
55          // is in its life:
56          // Available → Reserved → Sold for a sale bike; Available ↔ In repair
57          // for any bike.
58          private void dgvBikes_SelectionChanged(object sender, EventArgs e)
59          {
60              bool picked = dgvBikes.SelectedRows.Count > 0;
61              string status = "";
62              if (picked)
63              {
64                  status = Grid.SelectedText(dgvBikes, "Status");
65              }
66              bool forSale = picked
67                  && Grid.SelectedText(dgvBikes, "Purpose") == "Sale";
68
69              btnReserve.Enabled = forSale && status == "Available";
70              btnUnreserve.Enabled = forSale && status == "Reserved";
71              btnSell.Enabled = forSale
72                  && (status == "Available" || status == "Reserved");
73              btnRepair.Enabled = picked
74                  && (status == "Available" || status == "In repair");
75              if (status == "In repair")
76              {
77                  btnRepair.Text = "Back in service";
78              }
79              else
80              {
81                  btnRepair.Text = "Send for repair";
82              }
83
84              // A reserved bike can only be sold to the customer who reserved it,
85              // so pick them in the box.
86              if (status == "Reserved")
87              {
88                  cboCustomer.SelectedIndex =
89                      cboCustomer.FindStringExact(Grid.SelectedText(dgvBikes,
90                      "Customer"));
91              }
92          }
93          #endregion
94
95          #region Actions
96          private void btnAdd_Click(object sender, EventArgs e)
97          {
98              frmBikeEdit add = new frmBikeEdit();
99              if (add.ShowDialog(this) == DialogResult.OK)
100             {
101                 LoadBikes();
102             }
103         }
104
105         private void btnReserve_Click(object sender, EventArgs e)
106         {
107             if (cboCustomer.SelectedValue == null)
108             {
109                 Msg.Problem("Choose the customer who is reserving the bike.");
110                 cboCustomer.Focus();
111                 return;
```

```
112             }
113
114             string message =
115                 BikeData.Reserve(Grid.SelectedID(dgvBikes, "BikeID"),
116                 (int)cboCustomer.SelectedValue);
117             if (message ≠ "")
118             {
119                 Msg.Problem(message);
120                 return;
121             }
122             LoadBikes();
123         }
124
125         private void btnUnreserve_Click(object sender, EventArgs e)
126         {
127             if (!Msg.Ask("Take the reservation off the "
128                 + Grid.SelectedText(dgvBikes, "Bike") + "?"))
129             {
130                 return;
131             }
132
133             string message =
134                 BikeData.Unreserve(Grid.SelectedID(dgvBikes, "BikeID"));
135             if (message ≠ "")
136             {
137                 Msg.Problem(message);
138                 return;
139             }
140             LoadBikes();
141         }
142
143         private void btnSell_Click(object sender, EventArgs e)
144         {
145             if (cboCustomer.SelectedValue == null)
146             {
147                 Msg.Problem("Choose the customer who is buying the bike.");
148                 cboCustomer.Focus();
149                 return;
150             }
151             string bike = Grid.SelectedText(dgvBikes, "Bike");
152             string customer = cboCustomer.Text;
153             if (!Msg.Ask("Sell the " + bike + " to " + customer
154                 + "? A sold bike cannot be sold again."))
155             {
156                 return;
157             }
158
159             string message = BikeData.Sell(Grid.SelectedID(dgvBikes, "BikeID"),
160                 (int)cboCustomer.SelectedValue);
161             if (message ≠ "")
162             {
163                 Msg.Problem(message);
164                 return;
165             }
166             LoadBikes();
167         }
168
169         private void btnRepair_Click(object sender, EventArgs e)
```

```
170         {
171             // The button says "Send for repair" or "Back in service" depending
172             // on the bike's status.
173             bool sendForRepair = Grid.SelectedText(dgvBikes, "Status")
174                 ≠ "In repair";
175 
176             string message =
177                 BikeData.SetRepair(Grid.SelectedID(dgvBikes, "BikeID"),
178                 sendForRepair);
179             if (message ≠ "")
180             {
181                 Msg.Problem(message);
182                 return;
183             }
184             LoadBikes();
185         }
186         #endregion
187 
188         private void btnClose_Click(object sender, EventArgs e)
189         {
190             Close();
191         }
192     }
193 }
```

**1 The usings**

Lines 1 to 7

The tools this form needs: Windows Forms for the window itself, System.Data for the tables that fill the grids and lists, System.Drawing for colours, and your own Data and Models folders.

**2 The form’s class and its constructor**

Lines 9 to 16

The form has nothing of its own to remember, because the grid holds every bike's details.

**3 When the form loads**

Lines 18 to 26

The customer list is filled. Then the filter is set to All bikes, which fires its change event, which fills the grid.

**4 Filling the grid**

Lines 28 to 51

The filter's position is turned into the word the stored procedure wants: null, Sale or Hire. Then one call, and Grid.Tidy hides the number column.

**5 Which buttons are allowed: the part you type**

Lines 53 to 93

The method you read above.

**6 The five buttons**

Lines 95 to 186

Add opens the new bike form. Reserve and Sell need a customer picked, and Sell and Un-reserve ask a yes-or-no question first, with the bike's name in it. Repair works out which way from the status. Every one

of them makes one call, shows the message if there is one, and fills the grid again.

**7 Close**

Lines 188 to 193

The Close button closes the form. The two closing braces end the class and the namespace.

**Why you type it** These five lines give the life of a bike as true or false. Type them, so that you can write the same kind of lines for the statuses in your own project.

With frmBikes open in the designer, press F7 to see its code.

**You should see** The code tab for the form comes to the front.

Press Ctrl and A together to select everything in the code tab, press the Copy button under this step, and paste over it.

**You should see** The code tab holds the whole file, with one line that says YOU TYPE THIS PART.

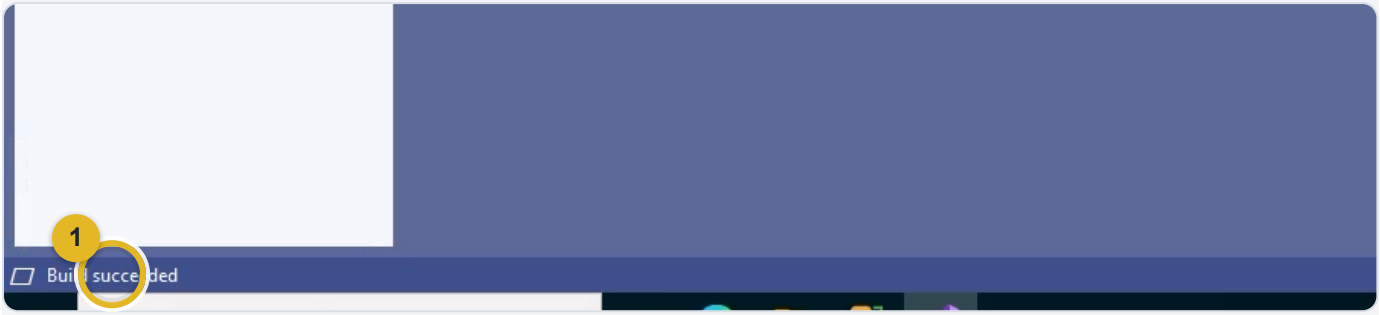
Click on the line that says YOU TYPE THIS PART, type the yellow lines in its place, and delete the marker line.

**You should see** The yellow lines are in your file, and no line says YOU TYPE THIS PART any more.

**IF IT GOES WRONG** A red wavy line under a word

A spelling slip. Compare your line with the yellow line, letter by letter, and check that each line ends with a semicolon where the yellow line does.

Press F6 to build.




Build succeeded, at the bottom left. Anything else means an error list to read.

1 Build succeeded. Press F6 to build at any time.

**You should see** Build succeeded, at the bottom left.

**IF IT GOES WRONG** frmBikes_Load or btnSell_Click is underlined: no suitable method found

The designer did not make that event. Go back to the designer, click the form (or the button), open the Properties window, press the lightning bolt, and double-click the empty box beside the event. Then build again.

**IF IT GOES WRONG** dgvBikes_SelectionChanged or cboFilter_SelectedIndexChanged is underlined

An event was not made. Click the grid, or the filter list, in the designer, press the lightning bolt in the Properties window, and double-click the empty box beside the event named in the table.

**IF IT GOES WRONG** 'The type or namespace name frmBikeEdit could not be found'

The new bike form is missing or named differently. Go back to screen 2 of this stage.

# Wire the Bikes button and run the bikes screen.

Open frmMain in the designer and double-click its Bikes for sale and hire button.

> **You should see** The code tab opens with an empty btnBikes_Click method, and the cursor is inside it.

Inside the method, type frmBikes bikes = new frmBikes(); as the first line, and bikes.ShowDialog(this); as the second line.

> **You should see** Two lines sit inside the method, each ending with a semicolon.

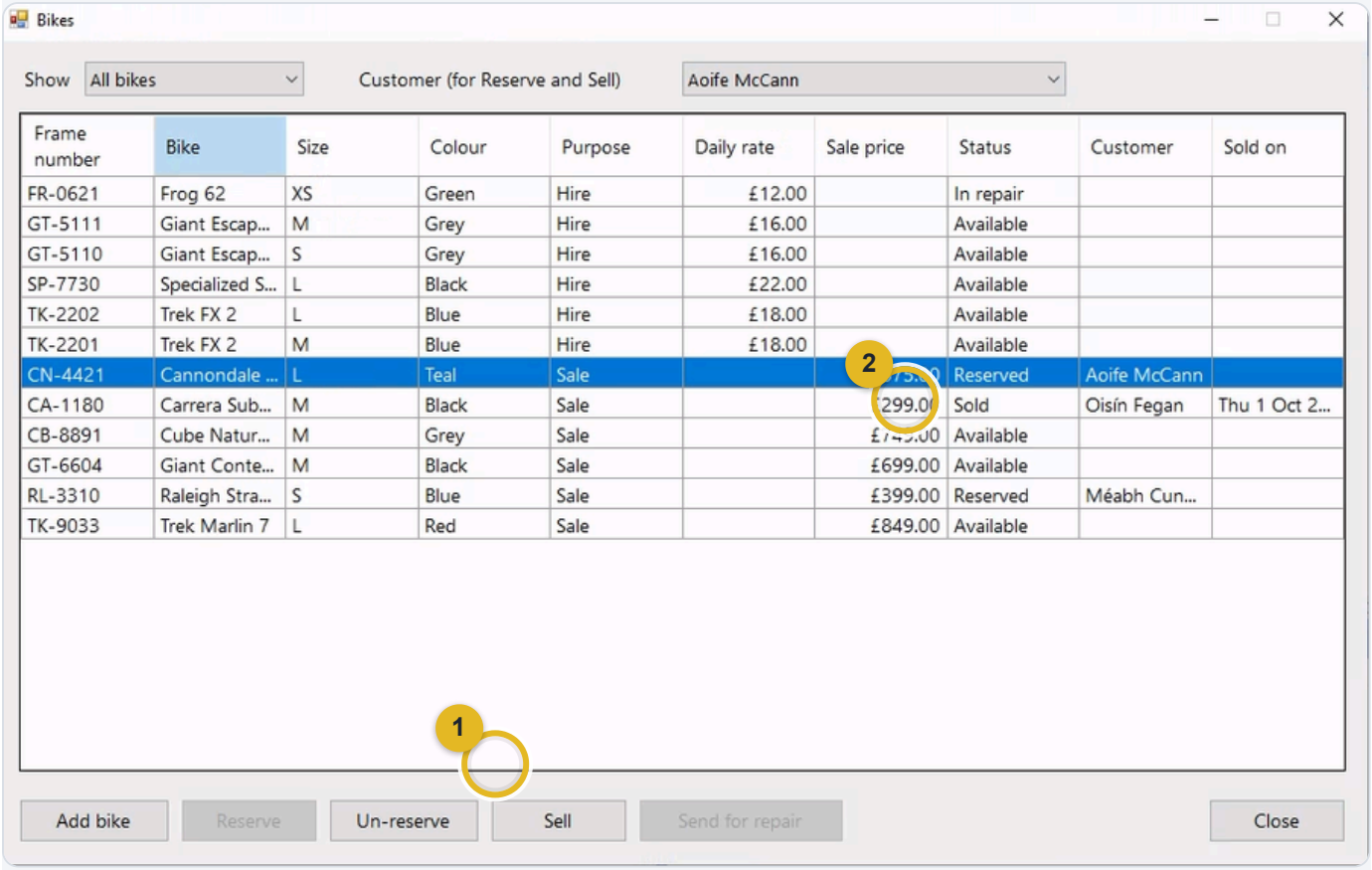
Press F6 to build.

> **You should see** Build succeeded, at the bottom left.

Press F5 to run the program, sign in, and press Bikes for sale and hire.

> **You should see** The bikes screen opens over the main menu, with every bike in the grid.

In the running bikes screen, pick the Cannondale, which stage 6 left reserved for Aoife McCann.




The bikes screen. The buttons switch on and off to match the status of the bike you pick.

1. A reserved bike for sale: Un-reserve and Sell are on, and Reserve and Send for repair are off.

2. With a sold bike picked, every button but Add bike and Close goes off.

> **You should see** The customer list jumps to Aoife McCann on its own. Un-reserve and Sell are on, Reserve is off, and the repair button is off because a reserved bike cannot go into repair.

Change the customer list to Seán Devlin and press Sell. Answer Yes.

> **You should see** A message box says: That bike is reserved for another customer. Press OK. The bike is still Reserved.

Put the customer list back to Aoife McCann and press Sell. Answer Yes.

> **You should see** The grid fills again. The Cannondale is Sold, with Aoife McCann in the Customer column and today's date in the Sold on column. Pick it again: every button but Add bike and Close is off.

Set the filter to Hire fleet, pick the Frog 62, and press Back in service.

> **You should see** The grid fills again and the Frog 62 is Available. Pick it again: the button now reads Send for repair. Press it, and the bike is In repair once more.

**IF IT GOES WRONG** Nothing happens when you press Bikes for sale and hire

The Click event is not wired. In the designer, click the button, open the Properties window, press the lightning bolt and check that Click has a method beside it.

**IF IT GOES WRONG** The buttons never change

The SelectionChanged event is not wired to dgvBikes_SelectionChanged. Click the grid in the designer, press the lightning bolt in the Properties window, and look for the method beside SelectionChanged.

**IF IT GOES WRONG** The grid is empty

The filter's SelectedIndexChanged event is not wired, so nothing fills the grid. Double-click the filter list in the designer and build again.

# Read the stored procedures that hand a hire bike over and take it back, then add them.

A hire has four statuses in its life: Booked, Out, Returned, and Cancelled if it never started. Check out moves Booked to Out, and return moves Out to Returned. Each one accepts exactly one status, in the same shape as the bike stored procedures. You type the check in Return.

**Copy this one** **usp_Hire_CheckOut: the customer takes the bike** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  CREATE OR ALTER PROCEDURE usp_Hire_CheckOut
2      @HireID INT,
3      @Message VARCHAR(200) OUTPUT
4  AS
5  BEGIN
6      SET @Message = '';
7      IF NOT EXISTS (SELECT * FROM Hire WHERE HireID = @HireID
8          AND Status = 'Booked')
9      BEGIN
10         SET @Message = 'Only a booked hire can be checked out.';
11         RETURN;
12     END
13     UPDATE Hire SET Status = 'Out' WHERE HireID = @HireID AND Status = 'Booked';
14 END
15 GO
```

**1 The name, and what goes in and out**

Lines 1 to 3

The hire number in, @Message out.

**2 Start with a blank message**

Lines 4 to 6

AS and BEGIN mark where the work starts. The first job is to make @Message empty. An empty message means it worked. Every refusal below puts a sentence into @Message instead, so the program always knows which it was.

**3 Refuse unless the hire is Booked**

Lines 7 to 12

A hire that is already out, returned or cancelled cannot go out.

**4 Booked becomes Out**

Lines 13 to 15

The WHERE repeats the status, so the row changes only if it is still Booked at that instant. GO ends the batch, ready for the next stored procedure.

**Type this one** **usp_Hire_Return: the bike comes back** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  CREATE OR ALTER PROCEDURE usp_Hire_Return
2      @HireID INT,
3      @Message VARCHAR(200) OUTPUT
4  AS
5  BEGIN
6      SET @Message = '';
7      IF NOT EXISTS (SELECT * FROM Hire WHERE HireID = @HireID AND Status = 'Out')
8      BEGIN
9          SET @Message = 'Only a hire that is out can be returned.';
10         RETURN;
11     END
12     UPDATE Hire SET Status = 'Returned' WHERE HireID = @HireID
13         AND Status = 'Out';
14 END
15 GO
```

**1 The name, and what goes in and out**

Lines 1 to 3

The hire number in, @Message out.

**2 Start with a blank message**

Lines 4 to 6

AS and BEGIN mark where the work starts. The first job is to make @Message empty. An empty message means it worked. Every refusal below puts a sentence into @Message instead, so the program always knows which it was.

**3 Refuse unless the hire is Out: the part you type**

Lines 7 to 11

Only a hire that is out can come back.

**4 Out becomes Returned**

Lines 12 to 15

The bike is free again for any dates, because the clash test only looks at Booked and Out. GO ends the batch, ready for the next stored procedure.

**Why you type it** The status check once more, for Out this time. By now you can write it without looking.

Press the Copy button under this step, click on the empty line above the line that prints BikeShop is ready, and paste.

**You should see** The stored procedure is in your script, ending with GO, and the line that prints BikeShop is ready is still last.

Press the Copy button under this step, click on the empty line above the line that prints BikeShop is ready, and paste.

> **You should see** Both stored procedures are in your script, the second with one line that says YOU TYPE THIS PART.

Type the yellow lines in place of the line that says YOU TYPE THIS PART, so that the marker line is gone.

> **You should see** Five lines of your own, from IF NOT EXISTS to END, and no marker line.

**IF IT GOES WRONG** A red wavy line under a word

A spelling slip. Compare your line with the yellow line, letter by letter, and check that each line ends the way the yellow line ends.

**IF IT GOES WRONG** 'Must declare the scalar variable …'

A name beginning with @ is spelled differently from the line that declares it. Make the two match.

Select all the lines you added on this screen, from the first CREATE OR ALTER PROCEDURE down to the last GO, and press F5.

> **You should see** The Messages pane says: Commands completed successfully. Your test sales and orders are still there.

**IF IT GOES WRONG** The Messages pane shows (6 rows affected) and BikeShop is ready

Nothing was selected, so the whole script ran and the sample data was put back. Your test sales, orders and reservations are gone. Nothing is broken: carry on, and remember that the stock of part 5 is 5 again.

Copy this one **A test that hands over hire 4, then tries a hire that came back weeks ago** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
-- Hand bike 1 to Jack O'Hare: hire 4 is booked.  Then try hire 1, which
-- came back weeks ago.
DECLARE @Message VARCHAR(200);

EXEC usp_Hire_CheckOut @HireID = 4, @Message = @Message OUTPUT;
IF @Message = ''
BEGIN
    PRINT 'Hire 4 is out.';
END
ELSE
BEGIN
    PRINT 'Hire 4: ' + @Message;
END

EXEC usp_Hire_CheckOut @HireID = 1, @Message = @Message OUTPUT;
PRINT 'Hire 1: ' + @Message;
```

**1 One box for the answer**

Line 3

Only @Message is needed.

**2 Hire 4 goes out**

Lines 5 to 13

Hire 4 is Jack O'Hare's booked hire of bike 1, starting in three days. Handing it over early is allowed: the hire is Booked.

**3 Hire 1 cannot**

Lines 15 to 16

Hire 1 was returned weeks ago in the sample data.

Copy this one **A test that brings hire 4 back** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  -- Bike 1 comes back: hire 4 is out.  Run it once: returned.  Run it
2  -- again: only a hire that is out can be returned.
3  DECLARE @Message VARCHAR(200);
4
5  EXEC usp_Hire_Return @HireID = 4, @Message = @Message OUTPUT;
6  IF @Message = ''
7  BEGIN
8      PRINT 'Hire 4 is returned.';
9  END
10 ELSE
11 BEGIN
12     PRINT 'Hire 4: ' + @Message;
13 END
```

**1 One box for the answer**

Line 3

The same single box.

**2 Return hire 4, and print the answer**

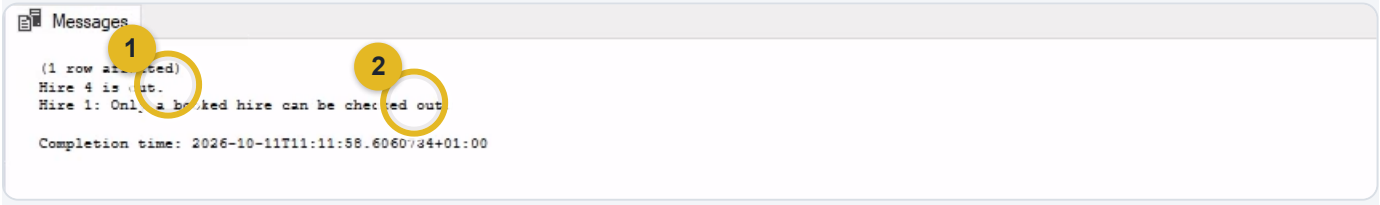
Lines 5 to 13

The first run brings it back. The second run finds no Out row for hire 4, so it is refused.

Click New Query, press the Copy button under this step, and paste the test into the empty window.

> **You should see** The test is in a new query window of its own, with your script still open in its tab.

Press F5 to run the test.




Hire 4 goes out. Hire 1 came back weeks ago, so it cannot go out again.

1 Booked became Out for hire 4.

2 The refusal for a hire that is not Booked.

> **You should see** The Messages pane says (1 row affected), then Hire 4 is out. Then Hire 1: Only a booked hire can be checked out.

Click New Query, press the Copy button under this step, and paste the test into the empty window.

> **You should see** The test is in a new query window of its own, with your script still open in its tab.

Press F5 to run the test.

> **You should see** The Messages pane says (1 row affected), then Hire 4 is returned.

Press F5 again, changing nothing.

> **You should see** The Messages pane says: Hire 4: Only a hire that is out can be returned.

# Write the stored procedure that cancels a hire.

Cancel is the last status change, and you write all of it. A cancelled hire stays in the table, marked Cancelled, and the clash test ignores it, so the bike is free for those dates again.

Your turn **usp_Hire_Cancel: mark a booked hire Cancelled** SQL

1. In SQL Server Management Studio (SSMS), click on the empty line above the line that prints BikeShop is ready and type the stored procedure: its name, the hire number and @Message as parameters, AS, BEGIN and the blank message.
2. Add the check: IF NOT EXISTS a Hire row with this number and the status Booked, SET @Message = 'Only a booked hire can be cancelled.'; and RETURN.
3. Then the UPDATE: set the status to Cancelled where the number matches and the status is still Booked. Finish with END and GO.
4. Select the lines you typed, from CREATE OR ALTER PROCEDURE down to GO, and press F5.

**THE TEST** The ready-made test below cancels hire 5, tries to cancel it again, and then lists every hire of bike 1.

**YOU SHOULD SEE** 'Cancelled hire number 5.' then 'Second cancel refused: Only a booked hire can be cancelled.' and a table of bike 1's hires with hire 5 marked Cancelled. If the second line says the rule is broken, your check is missing or tests the wrong status.

**HINT** It is usp_Hire_CheckOut with two words changed: the status it accepts stays Booked, and the status it sets becomes Cancelled.

▾ **Stuck? Show me the finished procedure**

Read it, then write yours **usp_Hire_Cancel** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
CREATE OR ALTER PROCEDURE usp_Hire_Cancel
    @HireID INT,
    @Message VARCHAR(200) OUTPUT
AS
BEGIN
    SET @Message = '';
    IF NOT EXISTS (SELECT * FROM Hire WHERE HireID = @HireID
        AND Status = 'Booked')
    BEGIN
        SET @Message = 'Only a booked hire can be cancelled.';
        RETURN;
    END
    UPDATE Hire SET Status = 'Cancelled' WHERE HireID = @HireID
        AND Status = 'Booked';
END
GO
```

**1 The name, and what goes in and out**

Lines 1 to 3

The hire number in, @Message out.

**2 Start with a blank message**

Lines 4 to 6

AS and BEGIN mark where the work starts. The first job is to make @Message empty. An empty message means it worked. Every refusal below puts a sentence into @Message instead, so the program always knows which it was.

**3 Refuse unless the hire is Booked**

Lines 7 to 12

A hire that has gone out cannot be cancelled: it has to come back instead.

**4 Booked becomes Cancelled**

Lines 13 to 16

The row stays, so the shop knows the hire was booked and dropped. GO ends the batch, ready for the next stored procedure.

Copy this one **A test that cancels hire 5 twice, then lists the hires of bike 1** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
-- Cancel hire 5, a booked hire of bike 1, then try to cancel it again.
-- Then look at the whole history of bike 1.
DECLARE @Message VARCHAR(200);

EXEC usp_Hire_Cancel @HireID = 5, @Message = @Message OUTPUT;
IF @Message = ''
BEGIN
    PRINT 'Cancelled hire number 5.';
END
ELSE
BEGIN
    PRINT 'Hire 5: ' + @Message;
END

EXEC usp_Hire_Cancel @HireID = 5, @Message = @Message OUTPUT;
IF @Message = ''
BEGIN
    PRINT 'The second cancel got through: the rule is broken.';
END
ELSE
BEGIN
    PRINT 'Second cancel refused: ' + @Message;
END

EXEC usp_Hire_GetForBike @BikeID = 1;
```

**1 One box for the answer**

Line 3

Only @Message is needed.

**2 Cancel hire 5**

Lines 5 to 13

Hire 5 is Róisín Campbell's booked hire of bike 1, starting in six days.

**3 Cancel it again**

Lines 15 to 23

The second cancel must be refused. If it got through, the test says so in plain words.

**4 Then the history of bike 1**

Line 25

Every hire of bike 1, newest first, in the Results tab. This stored procedure is on the next screen.

Click New Query, press the Copy button under this step, and paste the test into the empty window. Do not press F5 yet: the last line of the test uses a stored procedure that you add on the next screen.

> **You should see** The test is in a new query window of its own, waiting. The next screen adds the two list stored procedures, and its last step runs this test.

# Read the two stored procedures that list hires, then add them.

The hire screen shows the hires that touch two dates, and the bike history is for the test above and the report in chapter 9. Both are reads, with the names joined in.

**Read this one** **usp_Hire_GetByDateRange: the hires that touch two dates** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
CREATE OR ALTER PROCEDURE usp_Hire_GetByDateRange
    @FromDate DATE,
    @ToDate   DATE
AS
BEGIN
    SELECT h.HireID,
           b.Make + ' ' + b.Model + ' (' + b.Size + ')' AS Bike,
           c.FirstName + ' ' + c.LastName AS Customer,
           h.StartDate, h.EndDate,
           DATEDIFF(DAY, h.StartDate, h.EndDate) AS Days,
           h.TotalCost, h.DepositPaid, h.Status
    FROM Hire h
    JOIN Bike     b ON b.BikeID     = h.BikeID
    JOIN Customer c ON c.CustomerID = h.CustomerID
    WHERE h.StartDate ⩽ @ToDate AND h.EndDate ⩾ @FromDate
    ORDER BY h.StartDate, b.Make;
END
GO
```

**1 The name, and what goes in and out**

Lines 1 to 3

Two dates in. A table comes back.

**2 What comes back**

Lines 4 to 11

The bike as make, model and size in one column, the customer as a name, the dates, the number of days worked out by DATEDIFF, the money, and the status.

**3 Where the rows come from**

Lines 12 to 18

Three tables joined. The WHERE keeps a hire that starts before the To date and ends after the From date, which is every hire that touches the range. GO ends the batch, ready for the next stored procedure.

Read this one **usp_Hire_GetForBike: the history of one bike** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
CREATE OR ALTER PROCEDURE usp_Hire_GetForBike
    @BikeID INT
AS
BEGIN
    SELECT h.HireID, c.FirstName + ' ' + c.LastName AS Customer,
           h.StartDate, h.EndDate, h.TotalCost, h.DepositPaid, h.Status
    FROM Hire h
    JOIN Customer c ON c.CustomerID = h.CustomerID
    WHERE h.BikeID = @BikeID
    ORDER BY h.StartDate DESC;
END
GO
```

**1 The name, and what goes in and out**

Lines 1 to 2

The bike number in. A table comes back.

**2 What comes back**

Lines 3 to 6

One row per hire of that bike, with the customer's name.

**3 Where the rows come from**

Lines 7 to 12

Hire joined to Customer, this bike only, newest first. GO ends the batch, ready for the next stored procedure.

Press the Copy button under this step, click on the empty line above the line that prints BikeShop is ready, and paste.

> **You should see** The stored procedure is in your script, ending with GO, and the line that prints BikeShop is ready is still last.

Press the Copy button under this step, click on the empty line above the line that prints BikeShop is ready, and paste.

> **You should see** Both are in your script, each ending with GO.

Select all the lines you added on this screen, from the first CREATE OR ALTER PROCEDURE down to the last GO, and press F5.

> **You should see** The Messages pane says: Commands completed successfully. Your test sales and orders are still there.

**IF IT GOES WRONG** The Messages pane shows (6 rows affected) and BikeShop is ready

Nothing was selected, so the whole script ran and the sample data was put back. Your test sales, orders and reservations are gone. Nothing is broken: carry on, and remember that the stock of part 5 is 5 again.

Go back to the query window that holds the cancel test from the last screen, and press F5.

> **You should see** The Messages pane says (1 row affected), Cancelled hire number 5, then Second cancel refused: Only a booked hire can be cancelled. The Results tab lists the hires of bike 1, with hire 5 marked Cancelled and hire 4 marked Returned.

# Add HireData.

The Hire class is already in your Models folder from chapter 7, step 7.5.7, so only the data class is new. Its Add method calls the hire stored procedure from chapter 7, and the other five call the ones from this stage.

Copy this one **Data/HireData.cs: one method for each hire stored procedure** C#

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
using System;
using System.Data;
using System.Data.SqlClient;
using BikeShop.Models;

namespace BikeShop.Data
{
    public static class HireData
    {
        #region Add
        // Saves a new hire.  The procedure refuses if the bike is already out
        // on those days.
        // Returns "" if it worked, or the sentence to show the user.
        public static string Add(Hire hire)
        {
            SqlParameter newID = Db.Out("@NewHireID");
            string message = Db.Run("usp_Hire_Add",
                Db.P("@BikeID", hire.BikeID),
                Db.P("@CustomerID", hire.CustomerID),
                Db.P("@StartDate", hire.StartDate),
                Db.P("@EndDate", hire.EndDate),
                Db.P("@DepositPaid", hire.DepositPaid),
                Db.P("@BookedByStaffID", hire.BookedByStaffID),
                newID);
            if (message == "")
            {
                hire.HireID = (int)newID.Value;
            }
            return message;
        }
        #endregion

        // The hires that start between two dates.
        public static DataTable GetByDateRange(DateTime fromDate,
            DateTime toDate)
        {
            return Db.GetTable("usp_Hire_GetByDateRange",
                Db.P("@FromDate", fromDate.Date), Db.P("@ToDate", toDate.Date));
        }

        // The hire history of one bike.
        public static DataTable GetForBike(int bikeID)
        {
            return Db.GetTable("usp_Hire_GetForBike", Db.P("@BikeID", bikeID));
        }

        // The customer picks the bike up.
        public static string CheckOut(int hireID)
        {
            return Db.Run("usp_Hire_CheckOut", Db.P("@HireID", hireID));
        }

        // The customer brings the bike back.
```

```
54          public static string Return(int hireID)
55          {
56              return Db.Run("usp_Hire_Return", Db.P("@HireID", hireID));
57          }
58
59          // Cancels a hire that has not started yet.
60          public static string Cancel(int hireID)
61          {
62              return Db.Run("usp_Hire_Cancel", Db.P("@HireID", hireID));
63          }
64      }
65  }
```

**1 The usings**

Lines 1 to 4

Each using line names a set of ready-made tools this file needs. System.Data.SqlClient is the one that talks to SQL Server. BikeShop.Models is where your own classes live.

**2 The class**

Lines 6 to 9

A static class in the Data folder, like the others.

**3 Add: the hire stored procedure from chapter 7**

Lines 10 to 31

The Hire object's six facts go in. If the message is empty, the new number is kept in the object.

**4 GetByDateRange: the hire screen’s grid**

Lines 34 to 39

Two dates in, the hires that touch them out.

**5 GetForBike: used in chapter 9**

Lines 42 to 45

Every hire of one bike, newest first.

**6 CheckOut**

Lines 48 to 51

One number in, the answer out.

**7 Return**

Lines 54 to 57

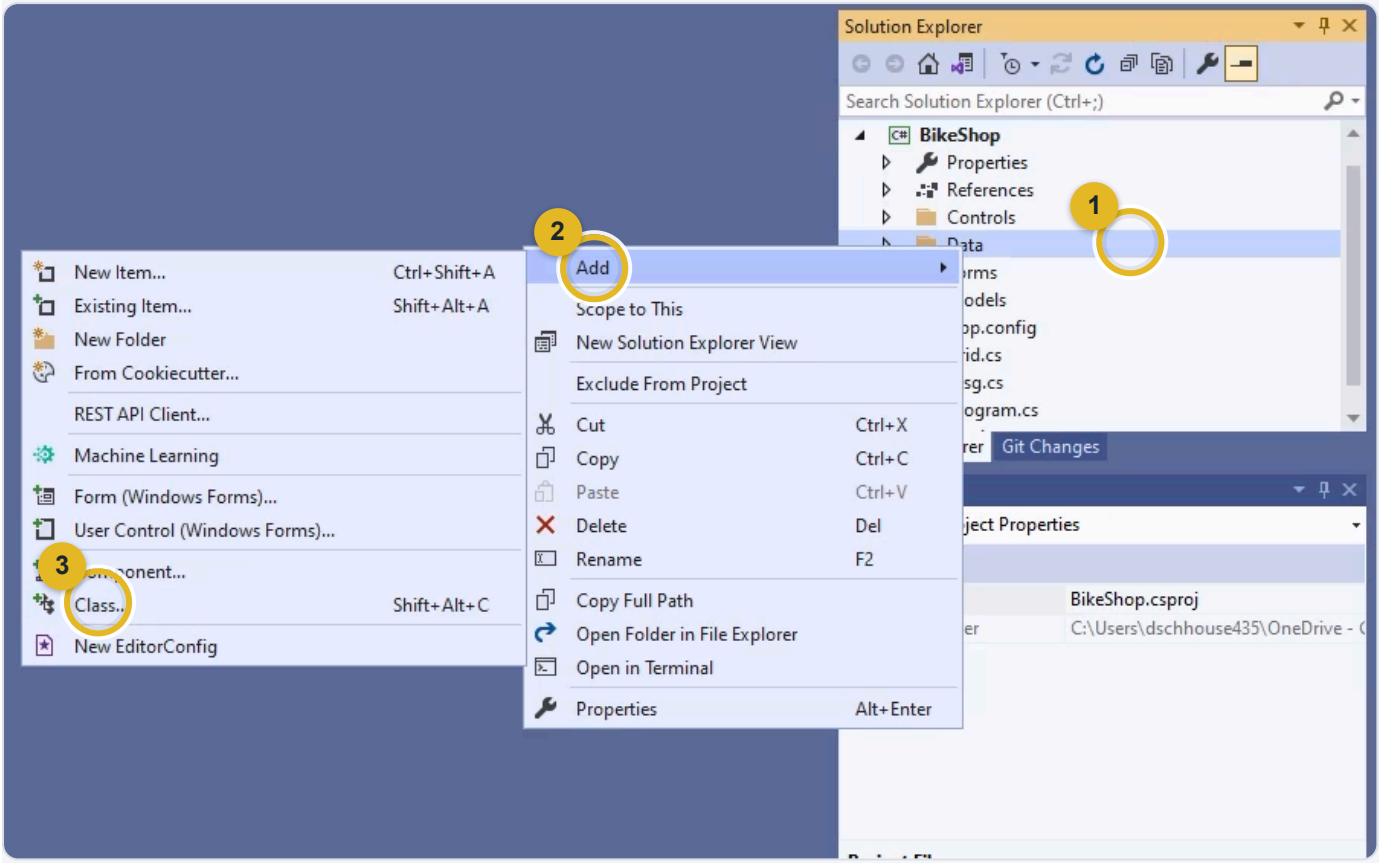
The same shape as CheckOut, for a different status.

**8 Cancel**

Lines 60 to 65

The same shape again. The two closing braces end the class and the namespace.

In Visual Studio 2019, in Solution Explorer, right-click the Data folder, point at Add, and click Class…




Adding a class to the Data folder. This picture was taken in the finished project, so Solution Explorer lists files you have not made yet.

1. Right-click the Data folder in Solution Explorer.
2. Point at Add.
3. Click Class…

> **You should see** The Add New Item window opens with Class selected.

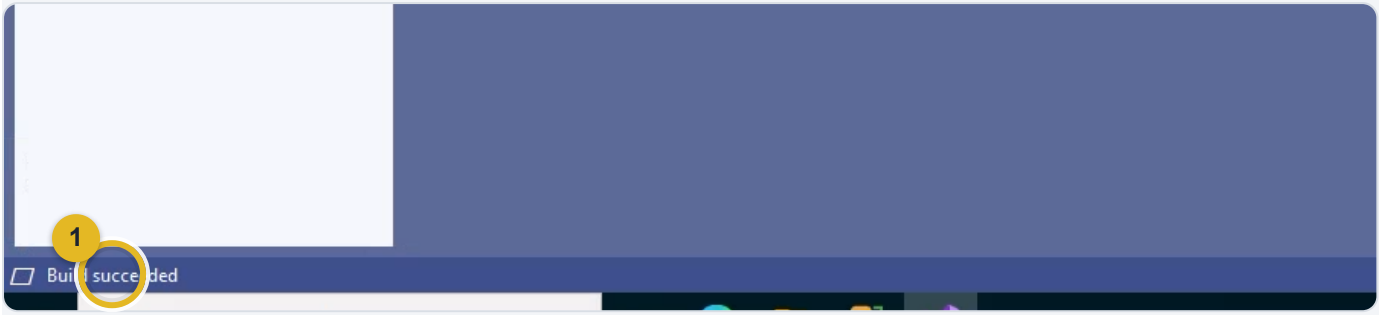
In the Name box, type HireData.cs and press Add.

> **You should see** A new file opens with an empty class in it.

Click in the new file, press Ctrl and A together to select everything, press the Copy button under this step, and paste over it.

> **You should see** HireData.cs holds six methods, ending with Cancel, and no red wavy line anywhere.

Press F6 to build.




Build succeeded, at the bottom left. Anything else means an error list to read.

1 Build succeeded. Press F6 to build at any time.

**You should see** Build succeeded, at the bottom left.

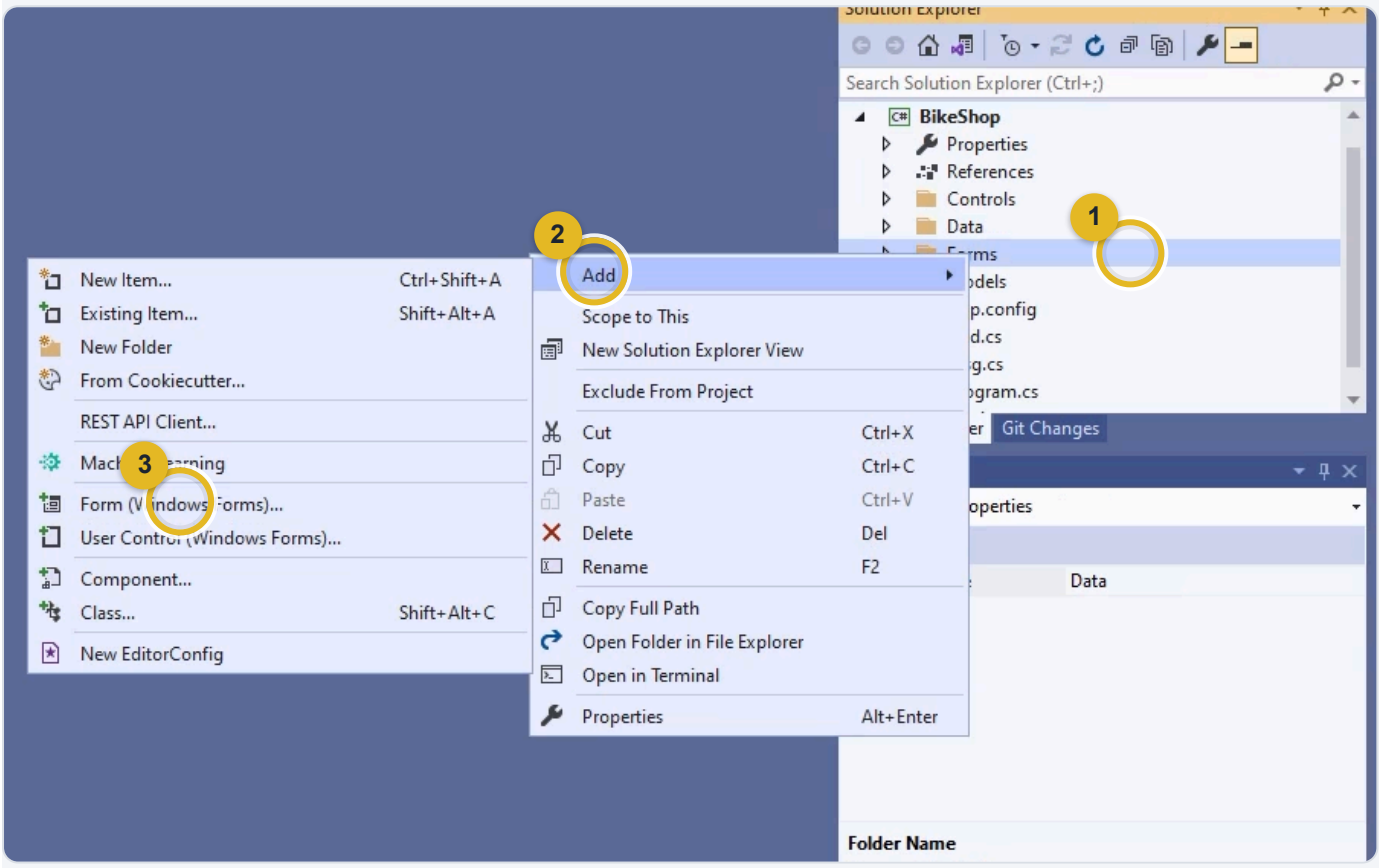
**IF IT GOES WRONG** 'The type or namespace name Hire could not be found'

The Hire class from chapter 7 is missing from the Models folder. Go back to step 7.5.7 of chapter 7 and add it.

# Add the new hire form and set its controls.

The new hire form is the clever one. The dates come first. The bike list is filled from the dates, and only shows the bikes free for all of those days. The cost line and the smallest deposit are worked out as the choices change.

In Solution Explorer, right-click the Forms folder, point at Add, and click Form (Windows Forms)…. Type frmHireEdit as the name and press Add.




Adding a form to the Forms folder. Taken in the finished project, so Solution Explorer lists more files than yours.

1. Right-click the Forms folder.
2. Point at Add.
3. Click Form (Windows Forms)…, type the name, press Add.

> **You should see** An empty form opens in the designer.

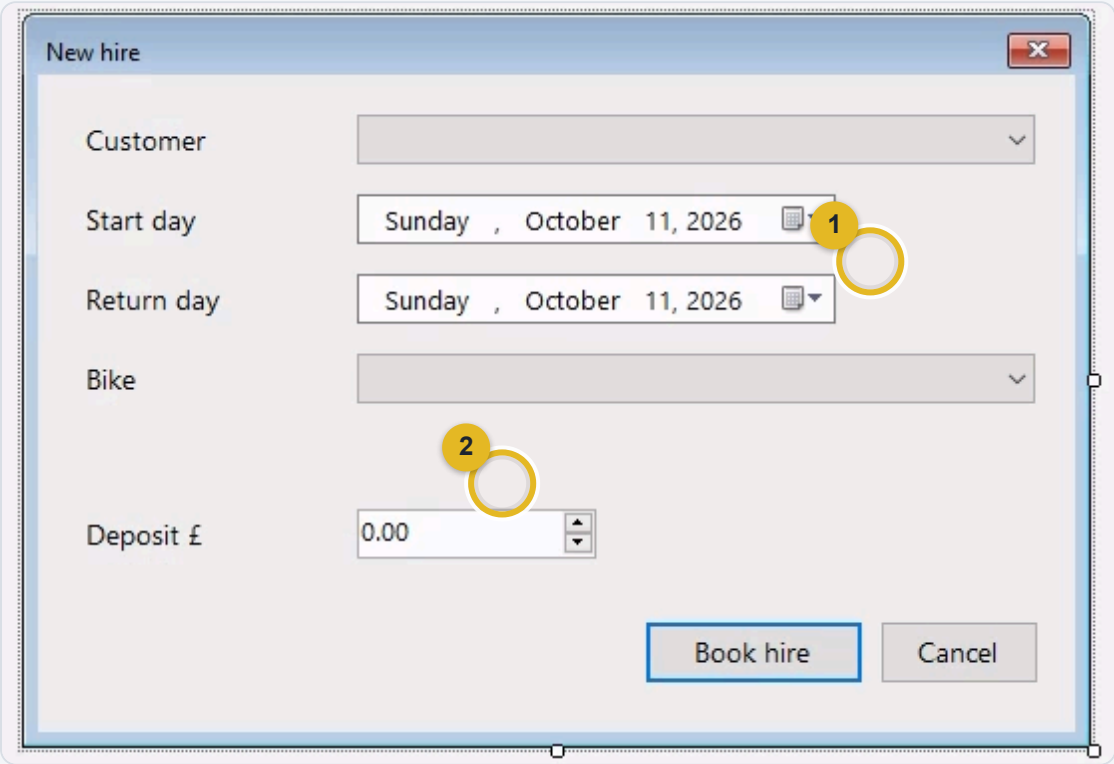
Click the form itself first and set the form's own properties. Then work down the table: drag the control from the Toolbox, drop it on the form, set its Name in the Properties window, and set the

other settings in the row. Where the Then column says double-click, double-clicking the control makes its event method for you.

Pick the dates first; the bike list then shows only bikes free for the whole of those dates, and the price works itself out.

**Click the form itself, then set these in the Properties window**

| | | | |
|---|---|---|---|
| Text | `New hire` | Size | `520, 330` |
| FormBorderStyle | `FixedDialog` | StartPosition | `CenterParent` |
| MaximizeBox | `False` | MinimizeBox | `False` |
| ShowInTaskbar | `False` | AcceptButton | `btnBook (after you add it)` |
| CancelButton | `btnCancel (after you add it)` | | |




frmHireEdit in the designer. The bike list is filled from the dates, and the cost line works itself out.

1. Pick the dates first. The bike list then shows only the bikes free for all of those days.
2. The cost line and the deposit, worked out by the form.

| DRAG THIS FROM THE TOOLBOX | NAME IT (THE NAME PROPERTY) | SET THESE IN THE PROPERTIES WINDOW | THEN |
|---|---|---|---|
| Label | `lblCustomer` | `Text = Customer` | |
| ComboBox | `cboCustomer` | `DropDownStyle = DropDownList` | |
| Label | `lblStart` | `Text = Start day` | |
| DateTimePicker | `dtpStart` | `Format = Long` | Double-click it. Visual Studio makes `dtpStart_ValueChanged` for you; the code goes inside it. |
| Label | `lblEnd` | `Text = Return day` | |
| DateTimePicker | `dtpEnd` | `Format = Long` | Double-click it. Visual Studio makes `dtpEnd_ValueChanged` for you; the code goes inside it. |
| Label | `lblBike` | `Text = Bike` | |
| ComboBox | `cboBike` | `DropDownStyle = DropDownList`<br>Only bikes free for those dates. | Double-click it. Visual Studio makes `cboBike_SelectedIndexChanged` for you; the code goes inside it. |
| Label | `lblCost` | nothing else<br>"3 days × £18.00 = £54.00 · deposit at least £10.80" | |
| Label | `lblDeposit` | `Text = Deposit £` | |
| NumericUpDown | `nudDeposit` | `DecimalPlaces = 2`<br>`Maximum = 10000`<br>Filled with the smallest deposit; can be raised. | |
| Button | `btnBook` | `Text = Book hire` | Double-click it. Visual Studio makes `btnBook_Click` for you; the code goes inside it. |
| Button | `btnCancel` | `Text = Cancel`<br>`DialogResult = Cancel` | |

# Read the three pieces of code behind the new hire form, then put the whole file in.

Three pieces to read. The first asks the database for the free bikes whenever a date changes. The second works out the cost line. The third is the Book button, which sends the hire and shows what the database said.

Read this one **Forms/frmHireEdit.cs: which bikes are free** C#

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  // Asks the database which bikes are free for the whole of the chosen
2  // dates.
3  private void RefreshBikes()
4  {
5      cboBike.DisplayMember = "Display";      // "Trek FX 2 · M · Blue"
6      cboBike.ValueMember = "BikeID";
7      cboBike.DataSource = BikeData.GetAvailableForHire(dtpStart.Value,
8          dtpEnd.Value);
9      ShowCost();
10 }
11
12 private void dtpStart_ValueChanged(object sender, EventArgs e)
13 {
14     // The return day must come after the start day: push it along if it
15     // does not.
16     if (dtpEnd.Value.Date ⩽ dtpStart.Value.Date)
17     {
18         dtpEnd.Value = dtpStart.Value.Date.AddDays(1);
19     }
20     RefreshBikes();
21 }
22
23 private void dtpEnd_ValueChanged(object sender, EventArgs e)
24 {
25     RefreshBikes();
26 }
27
28 private void cboBike_SelectedIndexChanged(object sender, EventArgs e)
29 {
30     ShowCost();
31 }
```

**1 Fill the bike list from the dates**

Lines 3 to 10

Display is the sentence column from the stored procedure, and BikeID is the value kept behind it. One call with the two dates, then the cost line is worked out again.

**2 When the start date changes**

Lines 12 to 21

If the return date is now on or before the start, it is pushed to the next day. Then the list is filled again.

**3 When the return date or the bike changes**

Lines 23 to 31

A new return date fills the list again. A different bike only changes the cost line.

**Why you type it** The list is never out of date, because it is filled again every time a date changes.

Read this one **Forms/frmHireEdit.cs: the cost line** C#

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  // Works out the price line as the choices change: days × daily rate,
2  // and the smallest deposit (20%).
3  private void ShowCost()
4  {
5      if (cboBike.SelectedItem == null)
6      {
7          lblCost.Text =
8              "No bike is free for those dates. Try other dates.";
9          btnBook.Enabled = false;
10         return;
11     }
12
13     // the row behind the chosen item
14     DataRowView bike = (DataRowView)cboBike.SelectedItem;
15     int days = (dtpEnd.Value.Date - dtpStart.Value.Date).Days;
16     decimal rate = (decimal)bike["DailyRate"];
17     decimal total = days * rate;
18     decimal smallestDeposit = Math.Round(total * 0.2m, 2);
19
20     string dayWord = "days";
21     if (days == 1)
22     {
23         dayWord = "day";
24     }
25     lblCost.Text = days + " " + dayWord + " × £" + rate.ToString("0.00")
26         + " = £" + total.ToString("0.00")
27         + " · deposit at least £" + smallestDeposit.ToString("0.00");
28     nudDeposit.Value = Math.Min(smallestDeposit, nudDeposit.Maximum);
29     btnBook.Enabled = true;
30 }
```

**1 No bike free**

Lines 3 to 11

If the list is empty, the cost line says so, and the Book button is switched off. There is nothing to book.

**2 Work out the money**

Lines 14 to 18

The row behind the chosen bike gives the daily rate. Days is the return date minus the start date. The deposit must be at least a fifth of the total, rounded to pennies, which is the same sum the stored procedure does.

**3 Say it in words**

Lines 20 to 30

One day or two days. The line is built from the numbers, the deposit box is set to the smallest deposit, and Book is switched on.

Read this one **Forms/frmHireEdit.cs: the Book button** C#

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  private void btnBook_Click(object sender, EventArgs e)
2  {
3      // Check what was chosen, one problem at a time: show the first and
4      // put the cursor there.
5      // The deposit needs no check for being negative: its box does not
6      // go below 0.
7      if (cboCustomer.SelectedValue == null)
8      {
9          Msg.Problem("Choose the customer.");
10         cboCustomer.Focus();
11         return;
12     }
13     if (dtpEnd.Value.Date <= dtpStart.Value.Date)
14     {
15         Msg.Problem("The return date must be after the start date.");
16         dtpEnd.Focus();
17         return;
18     }
19     if (cboBike.SelectedValue == null)
20     {
21         Msg.Problem("Choose a bike. If none is free, try other dates.");
22         cboBike.Focus();
23         return;
24     }
25
26     Hire hire = new Hire();
27     hire.CustomerID = (int)cboCustomer.SelectedValue;
28     hire.BikeID = (int)cboBike.SelectedValue;
29     hire.StartDate = dtpStart.Value.Date;
30     hire.EndDate = dtpEnd.Value.Date;
31     hire.DepositPaid = nudDeposit.Value;
32     hire.BookedByStaffID = Session.StaffID;
33
34     // The database checks the dates, the deposit and that the bike is
35     // free.
36     string message = HireData.Add(hire);
37     if (message != "")
38     {
39         Msg.Problem(message);
40         // the list may have changed since it was filled
41         RefreshBikes();
42         return;
43     }
44     DialogResult = DialogResult.OK;
45 }
```

**1 The first check**

Lines 1 to 12

A customer must be chosen.

**2 The second check**

Lines 13 to 18

The return date must be after the start date. The date pickers already make sure of it, so this check is a second safety net.

**3 The third check**

Lines 19 to 24

A bike must be chosen. If the list is empty, the sentence says to try other dates.

**4 Fill the hire**

Lines 26 to 32

The six facts from the boxes and the signed-in staff number.

**5 Send it, and show the answer**

Lines 36 to 45

One call. A refusal is shown, and the list is filled again in case the bike was taken while the form was open. An empty message closes the form with OK.

**Why you type it** The database checks the dates, the deposit and the clash again. The form's checks are only there to show a clear sentence before the form asks the database.

**Type this one** **Forms/frmHireEdit.cs: the whole file** C#

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  using System;
2  using System.Collections.Generic;
3  using System.Data;
4  using System.Drawing;
5  using System.Windows.Forms;
6  using BikeShop.Data;
7  using BikeShop.Models;
8
9  namespace BikeShop.Forms
10 {
11     public partial class frmHireEdit : Form
12     {
13         public frmHireEdit()
14         {
15             InitializeComponent();
16         }
17
18         private void frmHireEdit_Load(object sender, EventArgs e)
19         {
20             cboCustomer.DisplayMember = "FullName";
21             cboCustomer.ValueMember = "CustomerID";
22             cboCustomer.DataSource = CustomerData.Search("");
23
24             dtpStart.Value = DateTime.Today.AddDays(1);
25             dtpEnd.Value = DateTime.Today.AddDays(2);
26             RefreshBikes();
27         }
28
29         #region FreeBikes
30         // Asks the database which bikes are free for the whole of the chosen
31         // dates.
32         private void RefreshBikes()
33         {
34             cboBike.DisplayMember = "Display";      // "Trek FX 2 · M · Blue"
35             cboBike.ValueMember = "BikeID";
36             cboBike.DataSource = BikeData.GetAvailableForHire(dtpStart.Value,
37                 dtpEnd.Value);
38             ShowCost();
39         }
40
41         private void dtpStart_ValueChanged(object sender, EventArgs e)
42         {
43             // The return day must come after the start day: push it along if it
44             // does not.
45             if (dtpEnd.Value.Date ⩽ dtpStart.Value.Date)
46             {
47                 dtpEnd.Value = dtpStart.Value.Date.AddDays(1);
48             }
49             RefreshBikes();
50         }
51
52         private void dtpEnd_ValueChanged(object sender, EventArgs e)
53         {
```

```
            RefreshBikes();
        }

        private void cboBike_SelectedIndexChanged(object sender, EventArgs e)
        {
            ShowCost();
        }
        #endregion

        #region ShowCost
        // Works out the price line as the choices change: days × daily rate,
        // and the smallest deposit (20%).
        private void ShowCost()
        {
            if (cboBike.SelectedItem == null)
            {
                lblCost.Text =
                    "No bike is free for those dates. Try other dates.";
                btnBook.Enabled = false;
                return;
            }

            // the row behind the chosen item
            DataRowView bike = (DataRowView)cboBike.SelectedItem;
            int days = (dtpEnd.Value.Date - dtpStart.Value.Date).Days;
            decimal rate = (decimal)bike["DailyRate"];
            decimal total = days * rate;
            decimal smallestDeposit = Math.Round(total * 0.2m, 2);

            string dayWord = "days";
            if (days == 1)
            {
                dayWord = "day";
            }
            lblCost.Text = days + " " + dayWord + " × £" + rate.ToString("0.00")
                + " = £" + total.ToString("0.00")
                + " · deposit at least £" + smallestDeposit.ToString("0.00");
            nudDeposit.Value = Math.Min(smallestDeposit, nudDeposit.Maximum);
            btnBook.Enabled = true;
        }
        #endregion

        #region Book
        private void btnBook_Click(object sender, EventArgs e)
        {
            // Check what was chosen, one problem at a time: show the first and
            // put the cursor there.
            // The deposit needs no check for being negative: its box does not
            // go below 0.
            if (cboCustomer.SelectedValue == null)
            {
                Msg.Problem("Choose the customer.");
                cboCustomer.Focus();
                return;
            }
            if (dtpEnd.Value.Date ⩽ dtpStart.Value.Date)
            {
                Msg.Problem("The return date must be after the start date.");
```

```
112                 dtpEnd.Focus();
113                 return;
114             }
115             if (cboBike.SelectedValue == null)
116             {
117                 Msg.Problem("Choose a bike. If none is free, try other dates.");
118                 cboBike.Focus();
119                 return;
120             }
121
122             Hire hire = new Hire();
123             hire.CustomerID = (int)cboCustomer.SelectedValue;
124             hire.BikeID = (int)cboBike.SelectedValue;
125             hire.StartDate = dtpStart.Value.Date;
126             hire.EndDate = dtpEnd.Value.Date;
127             hire.DepositPaid = nudDeposit.Value;
128             hire.BookedByStaffID = Session.StaffID;
129
130             // The database checks the dates, the deposit and that the bike is
131             // free.
132             string message = HireData.Add(hire);
133             if (message != "")
134             {
135                 Msg.Problem(message);
136                 // the list may have changed since it was filled
137                 RefreshBikes();
138                 return;
139             }
140             DialogResult = DialogResult.OK;
141         }
142         #endregion
143     }
144 }
```

**1 The usings**

Lines 1 to 7

The tools this form needs: Windows Forms for the window itself, System.Data for the tables that fill the grids and lists, System.Drawing for colours, and your own Data and Models folders.

**2 The form's class and its constructor**

Lines 9 to 16

Nothing to remember: the boxes hold everything.

**3 When the form loads**

Lines 18 to 27

The customer list is filled, the dates start at tomorrow and the day after, and the bike list is filled for those dates.

**4 Which bikes are free: the first part you type**

Lines 29 to 61

The three methods you read above.

**5 The cost line**

Lines 63 to 94

The method you read above.

**6 The Book button: the second part you type**

Lines 96 to 144

The checks, then one call to the database.

**Why you type it** The call that fills the list from the dates, and the call that books. The database decides both.

With frmHireEdit open in the designer, press F7 to see its code.

**You should see** The code tab for the form comes to the front.

Press Ctrl and A together to select everything in the code tab, press the Copy button under this step, and paste over it.

**You should see** The code tab holds the whole file, with two lines that say YOU TYPE THIS PART.

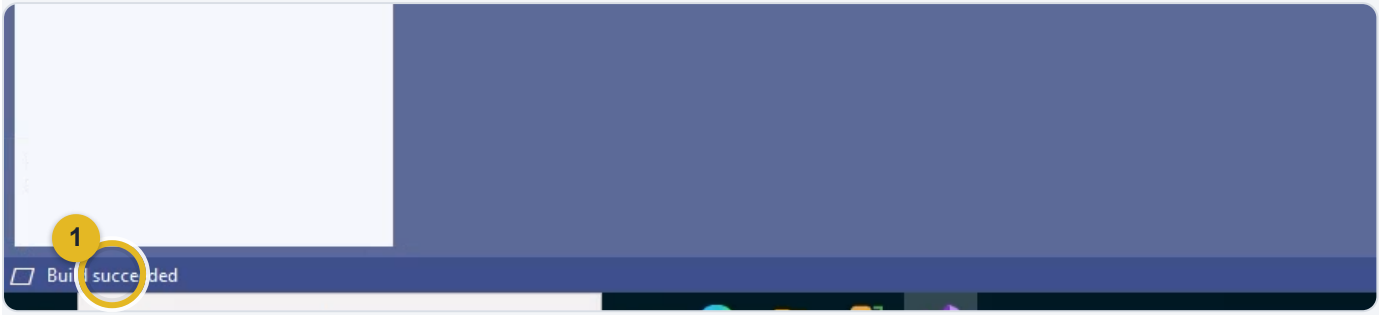
Click on each line that says YOU TYPE THIS PART in turn, type the yellow lines in its place, and delete the marker line.

**You should see** Both sets of yellow lines are in your file, and no line says YOU TYPE THIS PART any more.

**IF IT GOES WRONG** A red wavy line under a word

A spelling slip. Compare your line with the yellow line, letter by letter, and check that each line ends with a semicolon where the yellow line does.

Press F6 to build.




Build succeeded, at the bottom left. Anything else means an error list to read.

1 Build succeeded. Press F6 to build at any time.

**You should see** Build succeeded, at the bottom left.

**IF IT GOES WRONG** frmHireEdit_Load or btnBook_Click is underlined: no suitable method found

The designer did not make that event. Go back to the designer, click the form (or the button), open the Properties window, press the lightning bolt, and double-click the empty box beside the event. Then build again.

**IF IT GOES WRONG** 'The name Session does not exist'

Your sign-in class from chapter 5 is called something else. Match the name you used, and the name of its StaffID property.

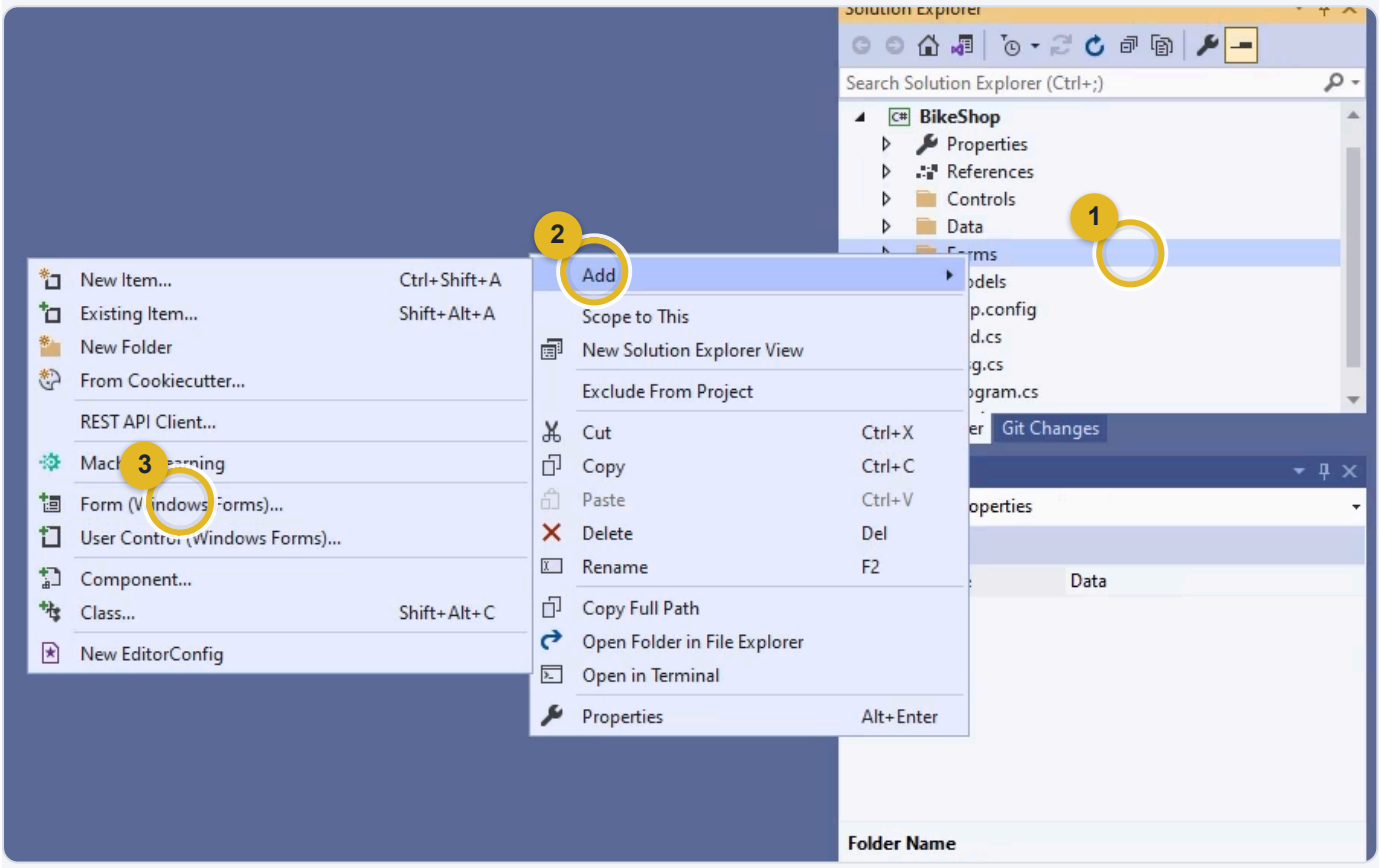
**IF IT GOES WRONG** dtpStart_ValueChanged, dtpEnd_ValueChanged or cboBike_SelectedIndexChanged is underlined

An event was not made. Click the date picker, or the bike list, in the designer, press the lightning bolt in the Properties window, and double-click the empty box beside the event named in the table.

# Add the hire screen and set its controls.

The hire screen is a grid of the hires between two dates, with one button for each step of a hire: New hire, Check out, Return and Cancel hire. The buttons follow the status, as the bike buttons do.

In Solution Explorer, right-click the Forms folder, point at Add, and click Form (Windows Forms)…. Type frmHires as the name and press Add.




Adding a form to the Forms folder. Taken in the finished project, so Solution Explorer lists more files than yours.

1. Right-click the Forms folder.
2. Point at Add.
3. Click Form (Windows Forms)…, type the name, press Add.

> **You should see** An empty form opens in the designer.

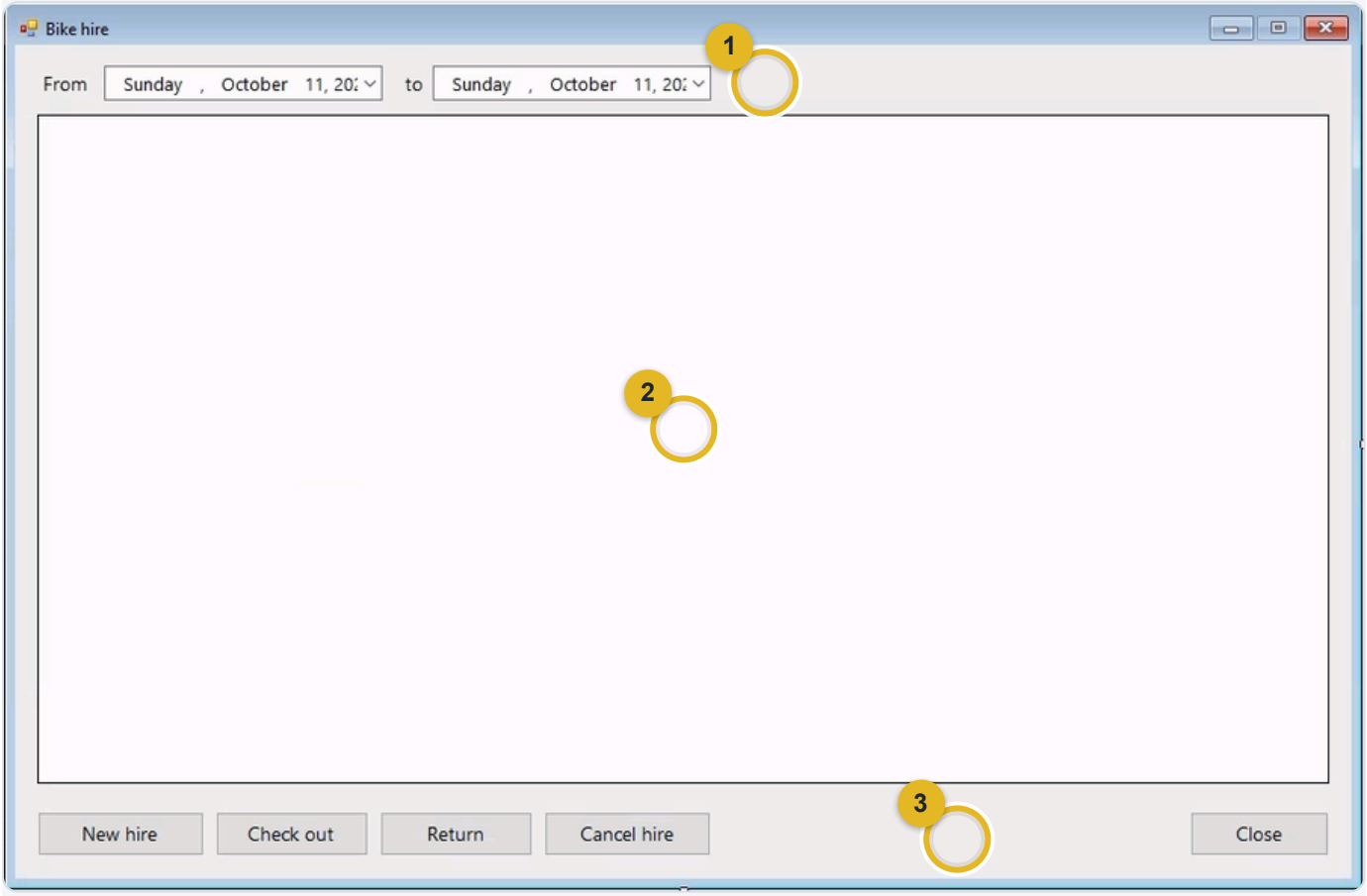
Click the form itself first and set the form's own properties. Then work down the table: drag the control from the Toolbox, drop it on the form, set its Name in the Properties window, and set the

other settings in the row. Where the Then column says double-click, double-clicking the control makes its event method for you.

Hires between two dates. Check out when the bike leaves, Returned when it comes back.

**Click the form itself, then set these in the Properties window**

| | | | |
|---|---|---|---|
| Text | `Bike hire` | Size | `960, 600` |
| FormBorderStyle | `FixedSingle` | StartPosition | `CenterParent` |
| MaximizeBox | `False` | | |




frmHires in the designer. The hires between two dates, with one button for each step of a hire.

1. From and to: the grid shows the hires that touch these dates.
2. The hires grid, dgvHires.
3. New hire, Check out, Return, Cancel hire and Close.

| DRAG THIS FROM THE TOOLBOX | NAME IT (THE NAME PROPERTY) | SET THESE IN THE PROPERTIES WINDOW | THEN |
|---|---|---|---|
| Label | `lblFrom` | `Text = From` | |
| DateTimePicker | `dtpFrom` | `Format = Long` | Double-click it. Visual Studio makes `dtpFrom_ValueChanged` for you; the code goes inside it. |
| Label | `lblTo` | `Text = to` | |
| DateTimePicker | `dtpTo` | `Format = Long` | Double-click it. Visual Studio makes `dtpTo_ValueChanged` for you; the code goes inside it. |
| DataGridView | `dgvHires` | `the grid settings (chapter 6)` | Events tab (the lightning bolt) → SelectionChanged: type `dgvHires_SelectionChanged` beside it and press Enter. |
| Button | `btnNew` | `Text = New hire` | Double-click it. Visual Studio makes `btnNew_Click` for you; the code goes inside it. |
| Button | `btnCheckOut` | `Text = Check out` `Enabled = False` Booked → Out. | Double-click it. Visual Studio makes `btnCheckOut_Click` for you; the code goes inside it. |
| Button | `btnReturn` | `Text = Returned` `Enabled = False` Out → Returned. | Double-click it. Visual Studio makes `btnReturn_Click` for you; the code goes inside it. |
| Button | `btnCancelHire` | `Text = Cancel hire` `Enabled = False` | Double-click it. Visual Studio makes `btnCancelHire_Click` for you; the code goes inside it. |
| Button | `btnClose` | `Text = Close` | Double-click it. Visual Studio makes `btnClose_Click` for you; the code goes inside it. |

# Put the code behind the hire screen.

Copy this one **Forms/frmHires.cs: the whole file** C#

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
using System;
using System.Collections.Generic;
using System.Data;
using System.Drawing;
using System.Windows.Forms;
using BikeShop.Data;
using BikeShop.Models;

namespace BikeShop.Forms
{
    public partial class frmHires : Form
    {
        public frmHires()
        {
            InitializeComponent();
        }

        private void frmHires_Load(object sender, EventArgs e)
        {
            dtpFrom.Value = DateTime.Today.AddDays(-7);
            dtpTo.Value = DateTime.Today.AddDays(28);
            LoadHires();
        }

        #region LoadHires
        private void LoadHires()
        {
            dgvHires.DataSource = HireData.GetByDateRange(dtpFrom.Value,
                dtpTo.Value);
            Grid.Tidy(dgvHires, "HireID");
        }
        #endregion

        private void dtpFrom_ValueChanged(object sender, EventArgs e)
        {
            LoadHires();
        }

        private void dtpTo_ValueChanged(object sender, EventArgs e)
        {
            LoadHires();
        }

        #region Selection
        // Each button matches one status: Booked can be checked out or
        // cancelled; Out can be returned.
        private void dgvHires_SelectionChanged(object sender, EventArgs e)
        {
            string status = "";
            if (dgvHires.SelectedRows.Count > 0)
            {
                status = Grid.SelectedText(dgvHires, "Status");
            }
```

```
            btnCheckOut.Enabled = status == "Booked";
            btnCancelHire.Enabled = status == "Booked";
            btnReturn.Enabled = status == "Out";
        }
        #endregion

        #region Actions
        private void btnNew_Click(object sender, EventArgs e)
        {
            frmHireEdit hire = new frmHireEdit();
            if (hire.ShowDialog(this) == DialogResult.OK)
            {
                LoadHires();
            }
        }

        private void btnCheckOut_Click(object sender, EventArgs e)
        {
            if (!Msg.Ask("Hand the " + Grid.SelectedText(dgvHires, "Bike")
                + " to " + Grid.SelectedText(dgvHires, "Customer") + " now?"))
            {
                return;
            }

            string message =
                HireData.CheckOut(Grid.SelectedID(dgvHires, "HireID"));
            if (message != "")
            {
                Msg.Problem(message);
                return;
            }
            LoadHires();
        }

        private void btnReturn_Click(object sender, EventArgs e)
        {
            if (!Msg.Ask("Has the " + Grid.SelectedText(dgvHires, "Bike")
                + " come back?"))
            {
                return;
            }

            string message =
                HireData.Return(Grid.SelectedID(dgvHires, "HireID"));
            if (message != "")
            {
                Msg.Problem(message);
                return;
            }
            LoadHires();
        }

        private void btnCancelHire_Click(object sender, EventArgs e)
        {
            if (!Msg.Ask("Cancel this hire for "
                + Grid.SelectedText(dgvHires, "Customer")
                + "? The bike becomes free for those dates."))
            {
```

```
112                 return;
113             }
114
115             string message =
116                 HireData.Cancel(Grid.SelectedID(dgvHires, "HireID"));
117             if (message ≠ "")
118             {
119                 Msg.Problem(message);
120                 return;
121             }
122             LoadHires();
123         }
124         #endregion
125
126         private void btnClose_Click(object sender, EventArgs e)
127         {
128             Close();
129         }
130     }
131 }
```

**1 The usings**

Lines 1 to 7

The tools this form needs: Windows Forms for the window itself, System.Data for the tables that fill the grids and lists, System.Drawing for colours, and your own Data and Models folders.

**2 The form’s class and its constructor**

Lines 9 to 16

Nothing to remember: the grid holds everything.

**3 When the form loads**

Lines 18 to 23

The dates start a week ago and four weeks ahead, then the grid is filled.

**4 Filling the grid**

Lines 25 to 32

One call with the two dates, and the number column is hidden.

**5 When the From date changes**

Lines 34 to 37

Fill the grid again.

**6 When the To date changes**

Lines 39 to 42

Fill the grid again, the same as for From.

**7 Which buttons are allowed**

Lines 44 to 58

Booked can be checked out or cancelled. Out can be returned. Anything else switches all three off.

**8 The four buttons**

Lines 60 to 124

New hire opens the form. The other three ask a yes-or-no question with the bike and the customer in it, make one call, show the message if there is one, and fill the grid again.

**9 Close**

Lines 126 to 131

Close the form. The two closing braces end the class and the namespace.

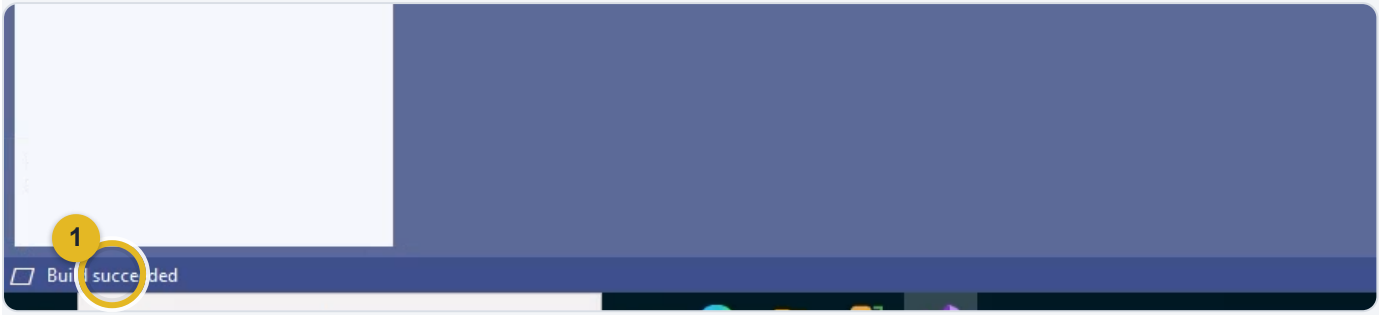
With frmHires open in the designer, press F7 to see its code.

> **You should see** The code tab for the form comes to the front.

Press Ctrl and A together to select everything in the code tab, press the Copy button under this step, and paste over it.

> **You should see** The code tab holds the whole file, and the old lines are gone.

Press F6 to build.




Build succeeded, at the bottom left. Anything else means an error list to read.

1 Build succeeded. Press F6 to build at any time.

**You should see** Build succeeded, at the bottom left.

**IF IT GOES WRONG** frmHires_Load or btnCheckOut_Click is underlined: no suitable method found

The designer did not make that event. Go back to the designer, click the form (or the button), open the Properties window, press the lightning bolt, and double-click the empty box beside the event. Then build again.

**IF IT GOES WRONG** 'The name Grid does not exist' or 'The name Msg does not exist'

Your grid and message helpers from chapter 6 are called something else, or are in a different namespace. Match the names you used.

**IF IT GOES WRONG** 'The type or namespace name frmHireEdit could not be found'

The new hire form is missing or named differently. Go back to screen 5 of this stage.

# Wire the Bike hire button and run the hire screen.

Open frmMain in the designer and double-click its Bike hire button.

> **You should see** The code tab opens with an empty btnHires_Click method, and the cursor is inside it.

Inside the method, type frmHires hires = new frmHires(); as the first line, and hires.ShowDialog(this); as the second line.

> **You should see** Two lines sit inside the method, each ending with a semicolon.

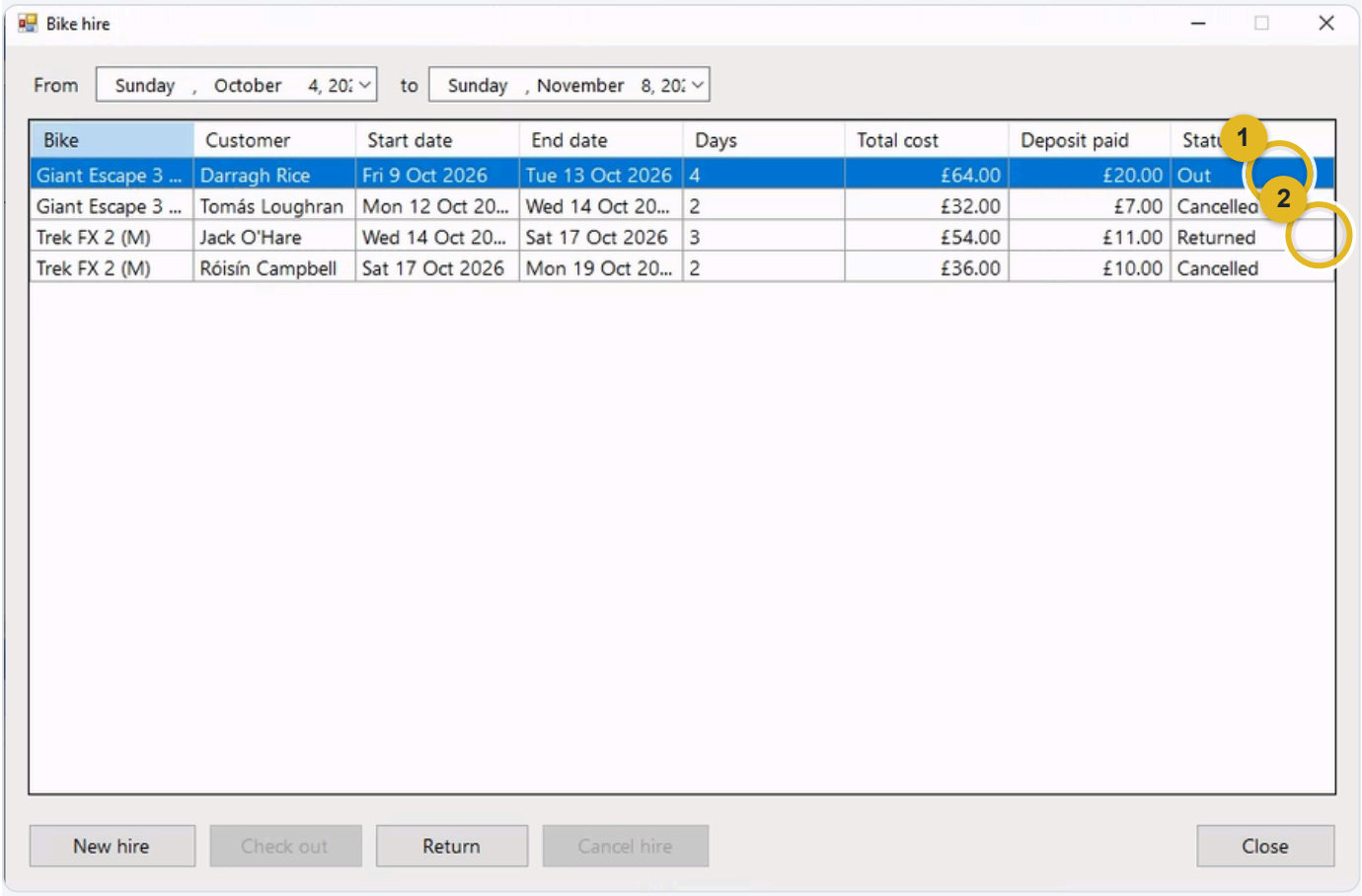
Press F6 to build.

> **You should see** Build succeeded, at the bottom left.

Press F5 to run the program, sign in, and press Bike hire.

> **You should see** The hire screen opens over the main menu, with the hires from a week ago to four weeks ahead in the grid.

In the running hire screen, pick each row in turn and watch the buttons.



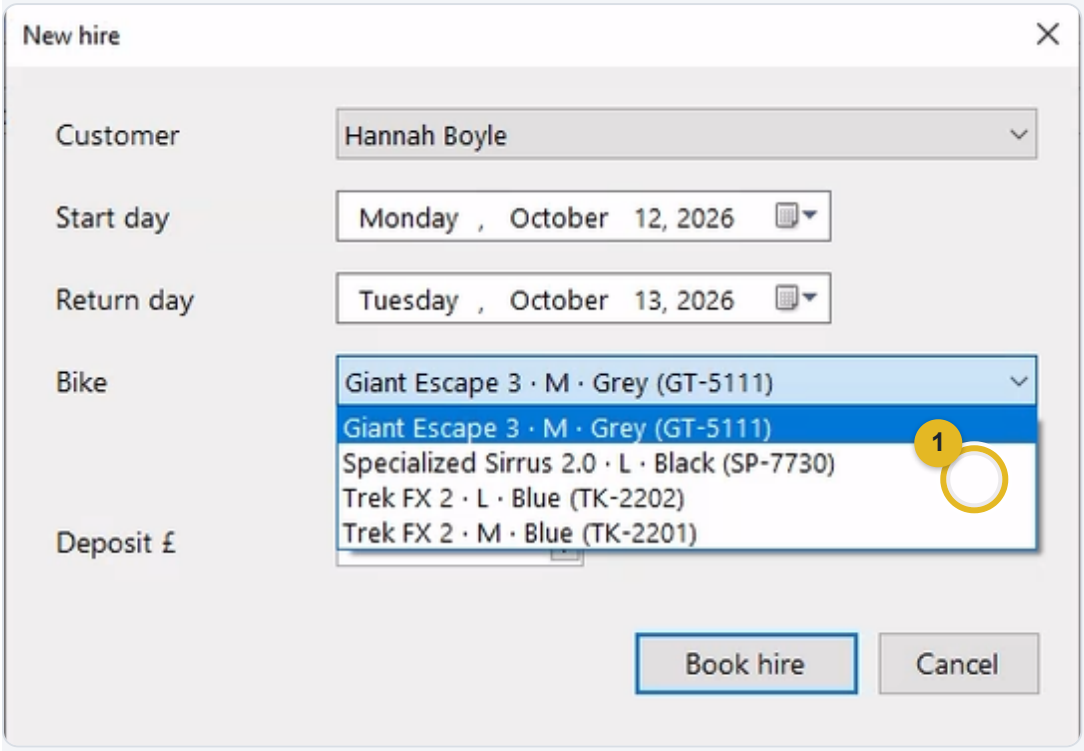

The hire screen from a week ago to four weeks ahead, after your tests in this stage. Each row is one hire, with its status in the last column.

1. Darragh Rice's hire, hire 3, is Out, so only Return is on for it.

2. Jack O'Hare's hire, hire 4, is Returned and Róisín Campbell's, hire 5, is Cancelled, from your tests. No button is on for them.

> **You should see** Darragh Rice's hire, hire 3, is Out, so only Return is on. Jack O'Hare's hire, hire 4, is Returned, from your test in this stage, and Róisín Campbell's, hire 5, is Cancelled, so nothing but New hire is on for them. The hires from weeks ago are before the From date, so they are not in the grid.

Press New hire. Pick Hannah Boyle, set the start date to tomorrow and the return date to the day after, then open the bike list.




The new hire form with the bike list open. Only the bikes free for all of those days are offered. When the list closes, the cost and the smallest deposit show under it.

1. Four bikes are offered. The Giant Escape 3 S is missing because hire 3 has it out until the day after tomorrow, and the Frog 62 is missing because it is in repair.

> **You should see** Four bikes are offered, each as a sentence with its size, colour and frame number. The Giant Escape 3 S is not among them, because hire 3 has it out until the day after tomorrow, and neither is the Frog 62, because it is in repair. The cost line reads 1 day × £16.00 = £16.00 · deposit at least £3.20 for the first bike in the list, and the deposit box holds that amount.

Pick the Trek FX 2 M, change the deposit to 1, and press Book hire.

> **You should see** A message box says: The deposit must be at least £3.60 (20% of £18.00). The stored procedure from chapter 7 worked that out, and the form only passed the sentence on.

Press OK. The form fills the bike list again, in case another booking took a bike while you typed, so the list is back on its first bike. Pick the Trek FX 2 M again: the deposit box changes to 3.60 by itself. Press Book hire.

> **You should see** The form closes, and your hire is in the grid as Booked.

Pick your new hire and press Check out. Answer Yes.

> **You should see** Booked becomes Out, and only Return is on for it.

Press Return. Answer Yes.

> **You should see** Out becomes Returned, and all three buttons are off.

**IF IT GOES WRONG** Nothing happens when you press Bike hire

The Click event is not wired. In the designer, click the button, open the Properties window, press the lightning bolt and check that Click has a method beside it.

**IF IT GOES WRONG** The bike list is empty for tomorrow

An empty list means every hire bike is already booked, out or in repair for those dates. Check the dates, then check your usp_Bike_GetAvailableForHire against the block in stage 6.

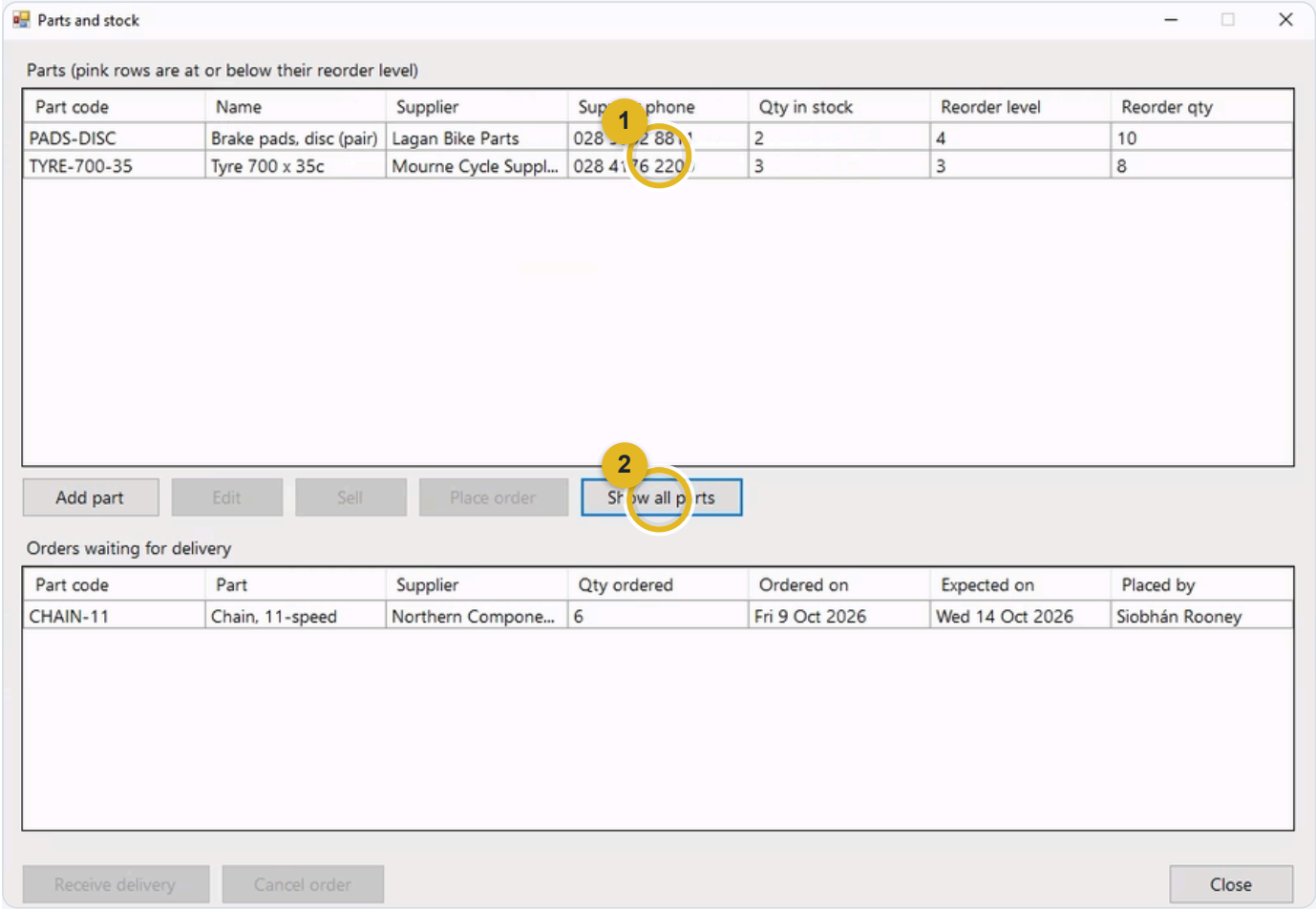
**IF IT GOES WRONG** The cost line says 0 days

The return date is the same as the start date. The date pickers push it on when they are wired: double-click dtpStart in the designer and check that its ValueChanged method is there.

# Walk a part from low stock to a full shelf.

This screen plays out stage 1 in the running program. A part runs low, the shop orders more, the box arrives, and the count is right at every step without anyone typing a number into the Part table.

Press Parts and stock on the main menu, and press Reorder list.




The reorder list in the program: the same two rows as the test in stage 4, with the supplier phone number ready to ring.

1. Only the parts to order. CHAIN-11 is gone because its order is open.

2. The button has swapped its words. Press it again to see every part.

**You should see** The top grid shows only PADS-DISC and TYRE-700-35, with a Supplier phone column, and the button now reads Show all parts. CHAIN-11 is not there, because order 3 for it is open.

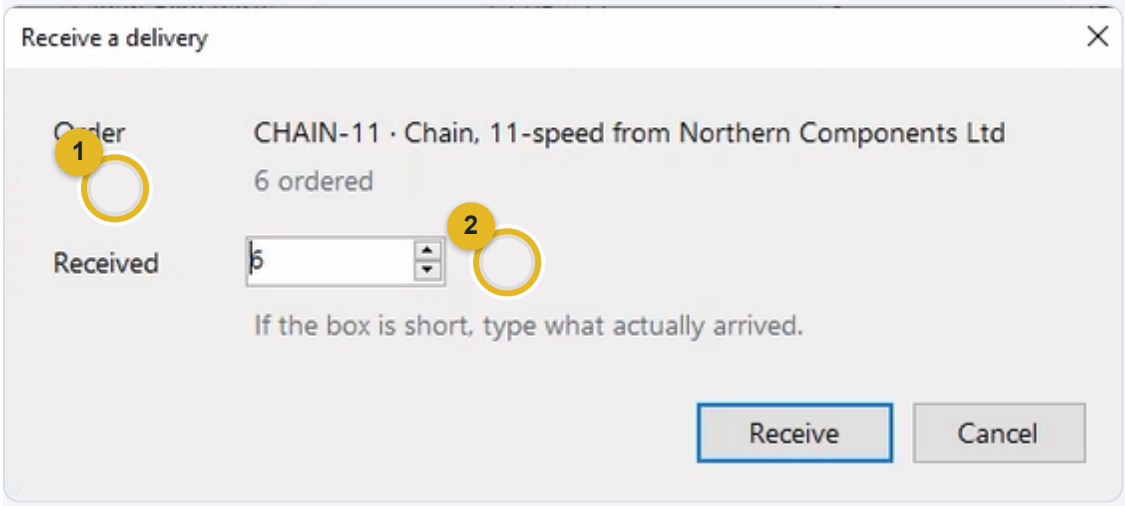
Pick TYRE-700-35 and press Place order.

> **You should see** The order form opens, with the quantity at the usual reorder amount for the tyre and the expected date a week away.

Press Place order on the order form, without changing the quantity or the date.

> **You should see** The form closes. The order is in the lower grid, and TYRE-700-35 has left the reorder list, because an order for it is now open. The count has not changed.

Press Show all parts, then pick order 3 for CHAIN-11 in the lower grid and press Receive delivery.




Receiving order 3. One press closes the order and adds six to the count of CHAIN-11.

1. The summary and the ordered amount came from the row you picked.

2. Change the number only if the box was short.

> **You should see** The receive form opens, with the order summary and 6 ordered at the top, and 6 in the received box.

Leave the number at 6 and press Receive.

> **You should see** The order leaves the lower grid, and CHAIN-11 in the top grid goes from 1 to 7 and loses its pink tint, in one press.

Pick your tyre order in the lower grid and press Cancel order. Answer Yes.

> **You should see** The order leaves the lower grid. Press Reorder list: TYRE-700-35 is back on it, because its count is still low and no order is open.

| | |
|---|---|
| **RULE THAT NEVER BREAKS · STOCK IS ALWAYS RIGHT** | Every count you saw change was changed by a stored procedure, and the screen only asked for the rows again. If the examiner types a sale for more than you have, the answer is a sentence, never a minus number. |

# Run every test in your program.

Every row below is a test the examiner would run. The ones marked Must be refused are the important ones. If one of those gets through, the count is wrong or a bike is sold twice, however nice the screens look. Do each one in your running program where it says, or in SQL Server Management Studio (SSMS) where no screen can reach it, and tick it when you have seen the result. Part 4 is PADS-DISC with 2 in stock in the sample data: run your whole script first if its count has changed.

| | THE TEST | WHAT MUST HAPPEN | HOW TO DO IT, AND WHAT YOU SHOULD SEE |
|---|---|---|---|
| | Sell a part with enough stock | Must be allowed | Parts and stock, pick CHAIN-9, Sell, quantity 1. The row's count goes down by one. |
| | Sell more than is in stock | Must be refused | Pick PADS-DISC with 2 in stock, Sell, quantity 3. Message: Only 2 in stock. |
| | The count is untouched after a refusal | Must be allowed | After the refusal, PADS-DISC still shows 2. Nothing was taken off. |
| | Sell exactly what is in stock | Must be allowed | PADS-DISC, Sell, quantity 2. The row shows 0, and it is pink. |
| | A refund puts the stock back | Must be allowed | In SQL Server Management Studio (SSMS), run the refund test from stage 3 with the number of that sale. The count printed is 2 again. |
| | Refund the same sale twice | Must be refused | Press F5 again in the refund test. Message: That sale has already been refunded, or does not exist. |
| | A delivery closes the order and adds to the count | Must be allowed | Parts and stock, pick an open order, Receive. The order leaves the lower grid and the part's count goes up by the number received. |
| | Receive the same order twice | Must be refused | The order is gone from the grid, so no button can reach it. In SQL Server Management Studio (SSMS), run the receive test from stage 4 with its number. Message: That order is not open. |
| | Cancel an order twice | Must be refused | In SQL Server Management Studio (SSMS), run the cancel test from stage 4. The second cancel is refused: Only an open order can be cancelled. |

| THE TEST | WHAT MUST HAPPEN | HOW TO DO IT, AND WHAT YOU SHOULD SEE |
|---|---|---|
| A part on the reorder list has no open order | Must be allowed | Order more of a part on the reorder list. It leaves the list until the order is received or cancelled. |
| Sell a bike that is sold | Must be refused | Bikes, pick a Sold bike: the Sell button is off. In SQL Server Management Studio (SSMS), run the bike sale test from stage 6: Bike 11 is refused with That bike has already been sold. |
| Sell a reserved bike to another customer | Must be refused | Reserve an available bike for one customer, pick another customer in the list, Sell, Yes. Message: That bike is reserved for another customer. |
| Sell a reserved bike to the customer who reserved it | Must be allowed | Pick the reserving customer, Sell, Yes. The bike is Sold with today's date. |
| A bike in repair cannot be reserved or sold | Must be refused | Pick a bike that is In repair. Reserve and Sell are off, and the repair button reads Back in service. |
| A hire whose dates overlap another hire | Must be refused | Bike hire, New hire, dates inside an existing Booked hire. That bike is not in the list. In SQL Server Management Studio (SSMS), the hire test from chapter 7 with those dates is refused: That bike is already out on those dates. |
| A hire starting the day another ends | Must be allowed | New hire, start date the day an existing hire ends. The bike is in the list, and the booking is made. |
| Too small a deposit | Must be refused | New hire, deposit 1, Book. Message: The deposit must be at least the amount the cost line shows. |
| A bike for sale cannot be hired | Must be refused | New hire: no bike for sale is ever in the list. In SQL Server Management Studio (SSMS), the hire test from chapter 7 with @BikeID = 7 is refused: Choose a hire bike. |
| Check out, then return | Must be allowed | Pick a Booked hire, Check out, Yes: it is Out. Return, Yes: it is Returned. |
| Return a hire that is not out | Must be refused | Pick a Returned hire: Return is off. In SQL Server Management Studio (SSMS), run the return test from stage 8 with its number. Message: Only a hire that is out can be returned. |

# Finish the chapter.

Twenty ticks are your evidence that the counts and the statuses hold. Keep the screenshots for your testing evidence in chapter 10. When you want the sample data back, click in BikeShop.sql and press F5 with nothing selected: every count and every status returns to where this chapter started.

**Make it yours**

Your scenario will have its own counted things and its own one-of-a-kind things. Write down which is which: what gets counted, such as tickets, seats or boxes of stock, and what has a status of its own, such as a room, a van or a loan. For each counted thing, name the three stored procedures that are the only doors to its count. For each thing with a status, draw its life as a line of status words, and write one IF NOT EXISTS check for each step along it.

CHAPTER 9 OF 12

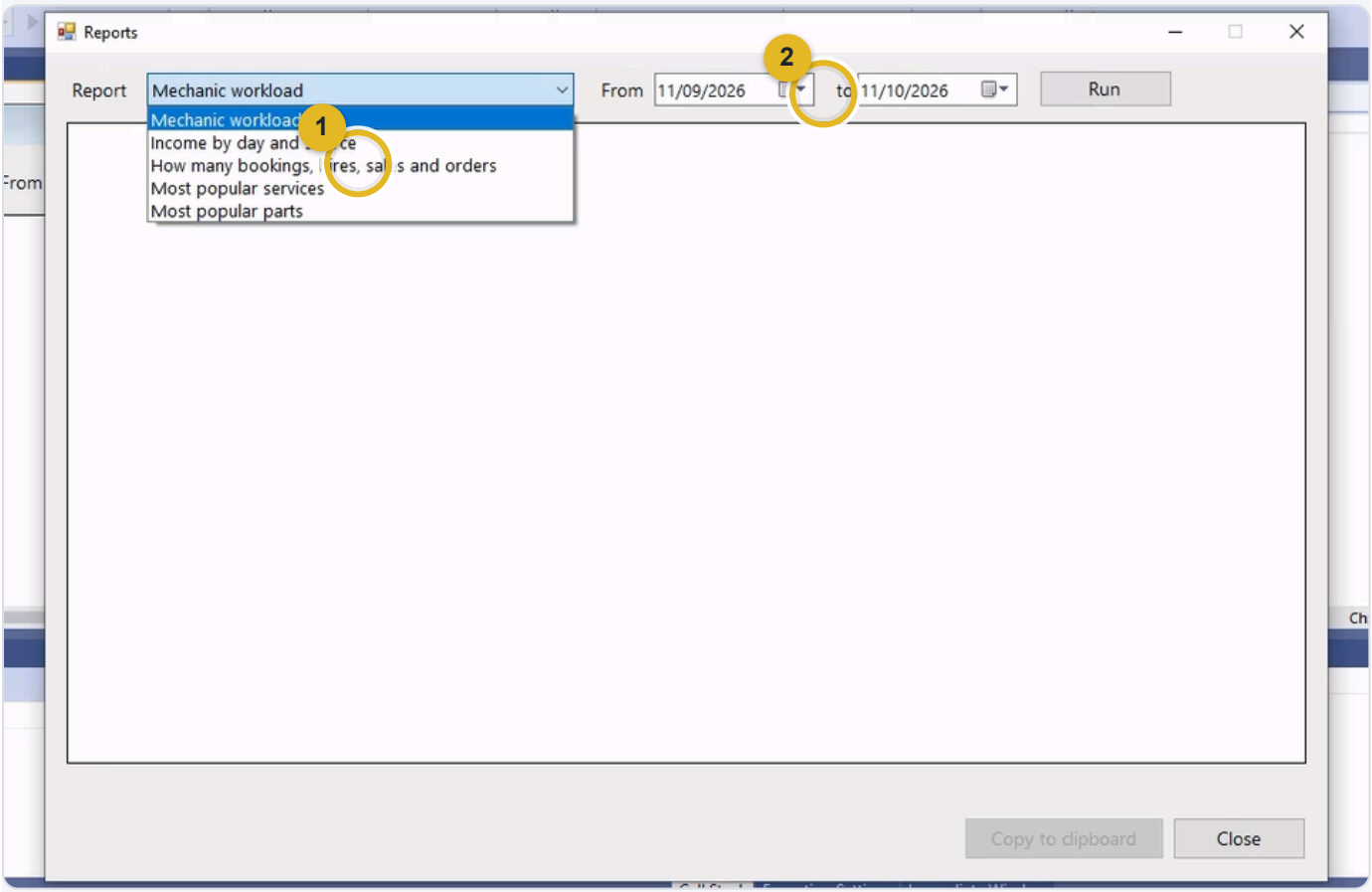
# Report across tables

A report that joins tables, filters by dates and shows totals.

STAGE 1 OF 8 · WHAT A REPORT IS · SCREEN 1 OF 3

## See what you will build in this chapter.

Every screen so far has shown one table and changed it. A report is different. A report asks one question of the whole database, for a range of dates, and shows the answer in a grid. Which mechanic did the most work in March? Where did the money come from last week? Which service is booked most? Each question needs rows from two, three or four tables at once.




The five questions the shop asks, in one list. Pick one, choose two dates, and press Run.

1. The five reports. Each one is one stored procedure.
2. The From and To dates. The form starts with the last thirty days.

By the end, the shop's manager can pick one of five reports, choose two dates, and press Run. The grid fills, and one plain sentence under it gives the answer. Five stored procedures do the work, one per report. The form checks one thing only: that the From date is not after the To date. That check is the one mistake this chapter catches, and it is caught with a class you write from nothing.

# Check that you have these ready.

**Before you start**

- Chapter 8 is finished: your BikeShop.sql has the stock, bike and hire stored procedures in it, and the parts screen runs.
- SQL Server Management Studio (SSMS) is open, connected to your BikeShop database.
- Your BikeShop solution is open in Visual Studio 2019.
- This guide is open in the Edge browser beside you. Tick each step as you do it, and the chapter remembers where you are.

**Words used in this chapter**

| | |
|---|---|
| `Report` | A question asked of the whole database for a range of dates, answered as a grid of rows. |
| `Date range` | Two dates, From and To, with every day between them counted in. |
| `LEFT JOIN` | A join that keeps every row of the first table, even the rows that match nothing in the second. |
| `GROUP BY` | The words that squash many rows into one row per thing, so COUNT and SUM can add them up. |
| `UNION ALL` | The words that stack the rows of one SELECT under the rows of another, as one list. |
| `Derived table` | A SELECT inside brackets that another SELECT reads from, as if it were a table. |
| `ISNULL` | A function that swaps an empty value for the value you give it, such as 0. |
| `DATEDIFF` | A function that counts the minutes, hours or days between two times. |
| `Exception` | An object that a method throws when something has gone wrong, carrying a sentence that says what. |
| `Catch` | The part of a method that takes an exception and does something sensible with it, instead of crashing. |

# Read what a report does, and what it never does.

A report is three things: one question, two dates, one grid. The question is fixed in the stored procedure. The dates come from the form. The grid is whatever rows the SELECT makes. Nothing else is needed, because the answer is read straight from the tables that your screens have been keeping right since chapter 6.

**The five questions Towpath Cycles asks**

Which mechanic did the most work, and earned the most for the shop? Where did the money come from, day by day, and was it the workshop, bike hire or parts? How much happened: how many bookings, hires, sales and orders? Which services are booked most? Which parts sell most? Each question is one stored procedure, and each stored procedure is one SELECT, or a stack of SELECTs.

A report never changes data. There is no INSERT, UPDATE or DELETE in any of the five. That is why none of them has an @Message parameter: there is nothing to refuse. In the program, each one is called with Db.GetTable, which hands back a table of rows, never with Db.Run. Run the same report twice and you get the same answer twice, and the database is exactly as it was.

# Read the workload stored procedure, then add it to your script.

The first report answers one question: for each mechanic, how many jobs were done between two dates, how many hours did they take, and how much did the shop earn from them. Three tables take part. Staff gives the mechanics, Appointment gives the jobs, and ServiceType gives the price of each job. Read the stored procedure below first. It uses a LEFT JOIN so a mechanic with no jobs still shows, a GROUP BY so there is one row per mechanic, and COUNT, SUM and DATEDIFF to count the jobs and add up the minutes and the money.

Copy this one **usp_Report_MechanicWorkload: jobs, hours and income per mechanic** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
CREATE OR ALTER PROCEDURE usp_Report_MechanicWorkload
    @FromDate DATE,
    @ToDate   DATE
AS
BEGIN
    SELECT s.FirstName + ' ' + s.LastName AS Mechanic,
           COUNT(a.AppointmentID) AS JobsDone,
           CAST(ISNULL(SUM(DATEDIFF(MINUTE, a.StartTime, a.EndTime)), 0) / 60.0
               AS DECIMAL(6,1)) AS HoursWorked,
           ISNULL(SUM(st.Price), 0) AS Income
    FROM Staff s
    LEFT JOIN Appointment a
           ON a.MechanicID = s.StaffID
          AND a.Status = 'Done'
          AND CAST(a.StartTime AS DATE) BETWEEN @FromDate AND @ToDate
    LEFT JOIN ServiceType st ON st.ServiceTypeID = a.ServiceTypeID
    WHERE s.Role = 'Mechanic'
    GROUP BY s.StaffID, s.FirstName, s.LastName
    ORDER BY Income DESC, Mechanic;
END
GO
```

**1 The name, and the two dates that go in**

Lines 1 to 3

Every report takes the same two things: a From date and a To date. Nothing comes back through a parameter. The answer is the table of rows that the SELECT makes.

**2 AS and BEGIN**

Lines 4 to 5

This stored procedure only reads, so it has no @Message. AS and BEGIN mark where the work starts.

**3 The first column: the mechanic**

Line 6

The first name, a space and the last name are joined into one piece of text with plus signs. AS Mechanic names the column, so the grid heading reads Mechanic and not an expression.

**4 The second column: how many jobs**

Line 7

COUNT counts the appointment numbers in the group. It counts the appointment column, not a star, because a mechanic with no jobs has no appointment number and must count as 0, not 1.

**5 The third column: the hours**

Lines 8 to 9

DATEDIFF counts the minutes from the start of each job to its end. SUM adds the minutes up. ISNULL turns an empty total into 0 for a mechanic with no jobs. Dividing by 60.0 makes hours, and CAST to DECIMAL(6,1) rounds the answer to one decimal place, such as 2.8.

**6 The fourth column: the money**

Line 10

The price of each job comes from its service type. SUM adds the prices up, and ISNULL gives 0 to a mechanic with nothing to add.

**7 Start from the staff, and bring in their jobs**

Lines 11 to 15

The report starts from Staff, because the question is about mechanics. LEFT JOIN brings in each mechanic's appointments, and keeps the mechanic even when none match. The three tests after ON pick which appointments count: the mechanic's own, finished ones, and ones that started between the two dates. CAST to DATE throws away the clock part so that BETWEEN works on whole days. The date test sits after ON and not in a WHERE on purpose: a WHERE would throw away the mechanics with no jobs in those dates.

**8 Bring in the price of each job**

Line 16

Each appointment has a service type, and the service type holds the price. This is a LEFT JOIN too, so a mechanic with no appointments, and so no service type, is still kept.

**9 Only the mechanics**

Line 17

Staff holds the manager and the counter staff too. The WHERE is on the Staff row, so it keeps the right people without losing anyone's jobs.

**10 One row per mechanic**

Line 18

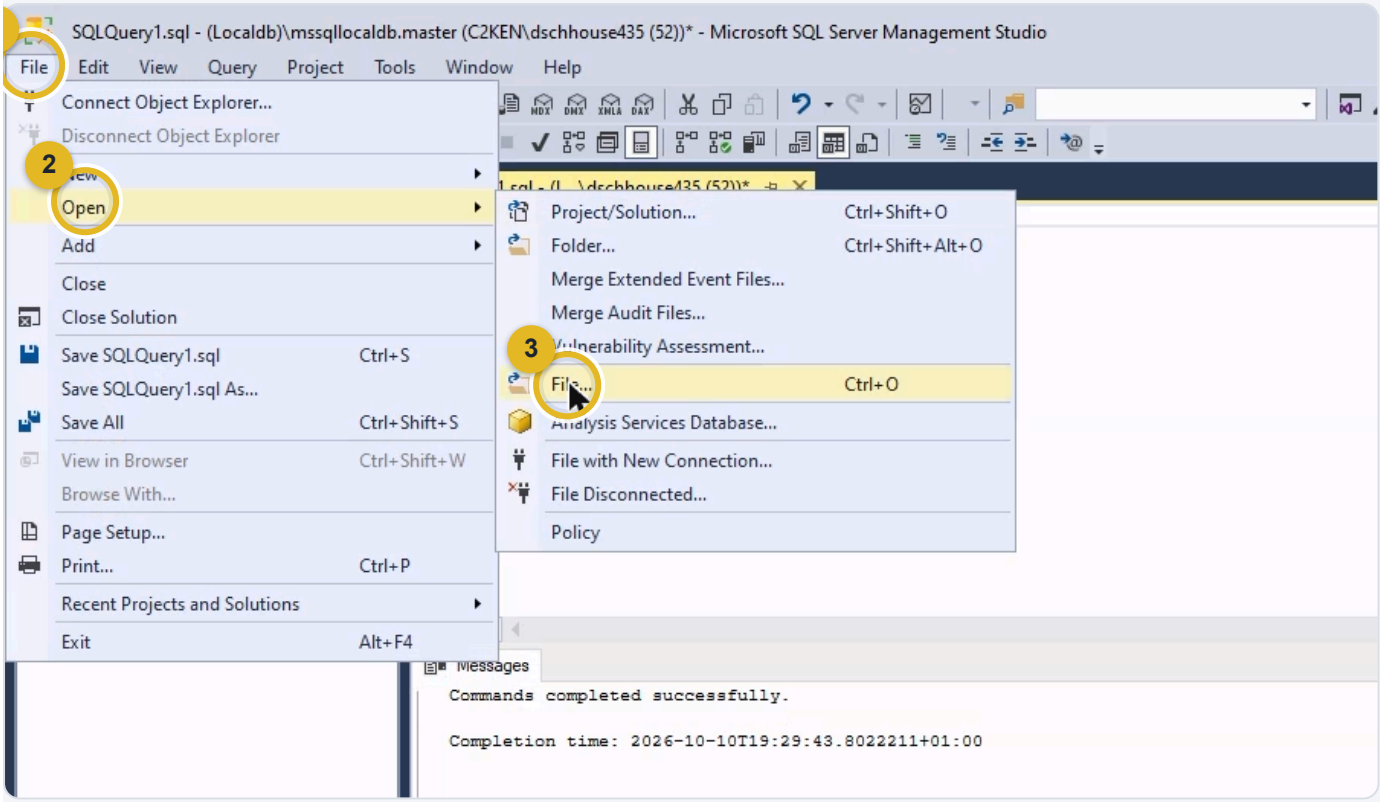
GROUP BY squashes all of one mechanic's appointment rows into one row, so that COUNT and SUM have something to add up. Every column in the SELECT that is not inside COUNT or SUM must be listed here.

**11 Biggest earner first**

Lines 19 to 21

DESC means highest first. Two mechanics with the same income are then put in name order, so the list is always in the same order. GO ends the batch, ready for the next stored procedure.

In SQL Server Management Studio (SSMS), click File, then Open, then File…, and open BikeShop.sql from your OneDrive folder.




The File menu in SQL Server Management Studio (SSMS). Your script opens from here.

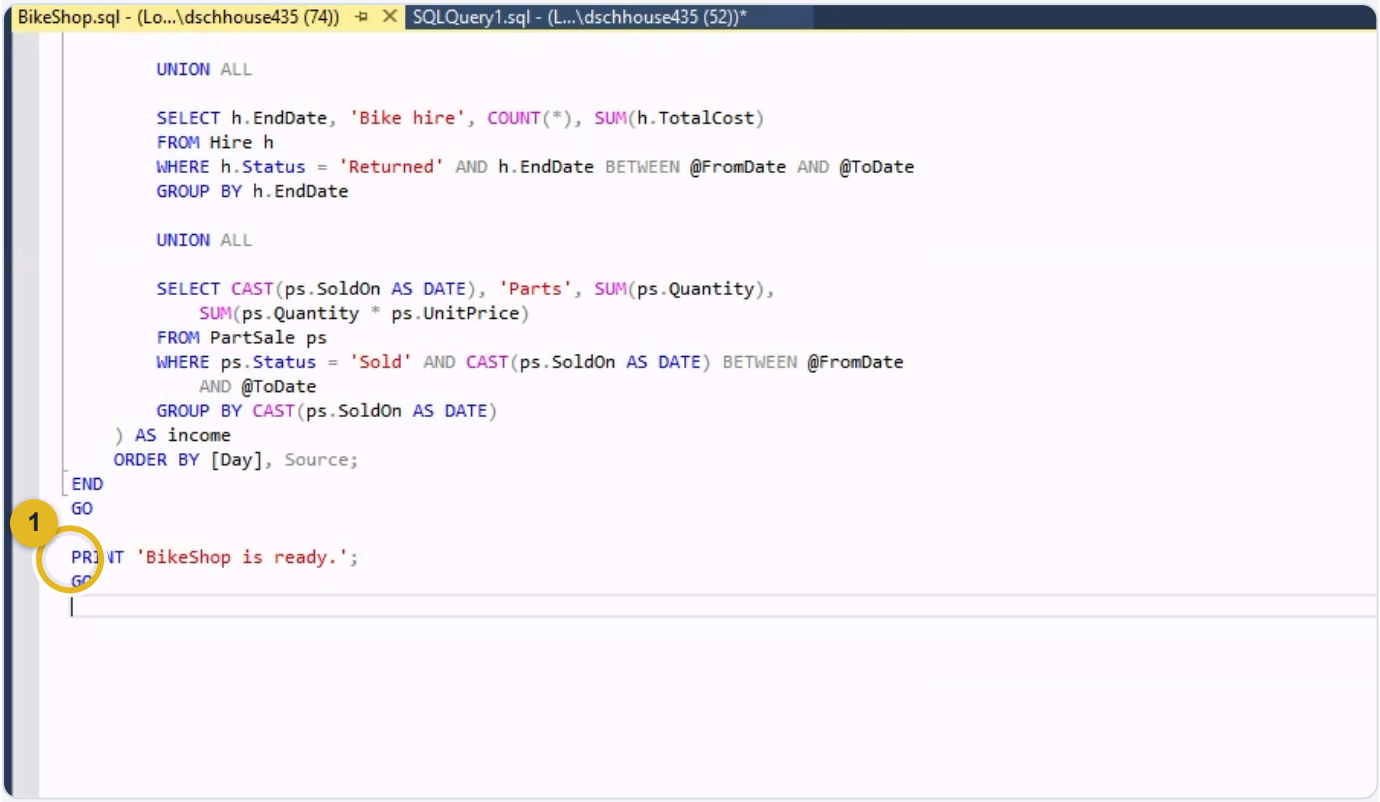
1. Click File at the top left.

2. Point at Open.

3. Click File… and choose BikeShop.sql from your OneDrive folder.

> **You should see** Your script is in a tab, and the first line is the USE line from chapter 3.

Scroll to the very end of the script and click on the empty line above the line that prints BikeShop is ready. That line always stays last.




The end of your script. Everything new in this chapter goes on the empty line above the line that prints BikeShop is ready, one stored procedure after another, each ending with GO.

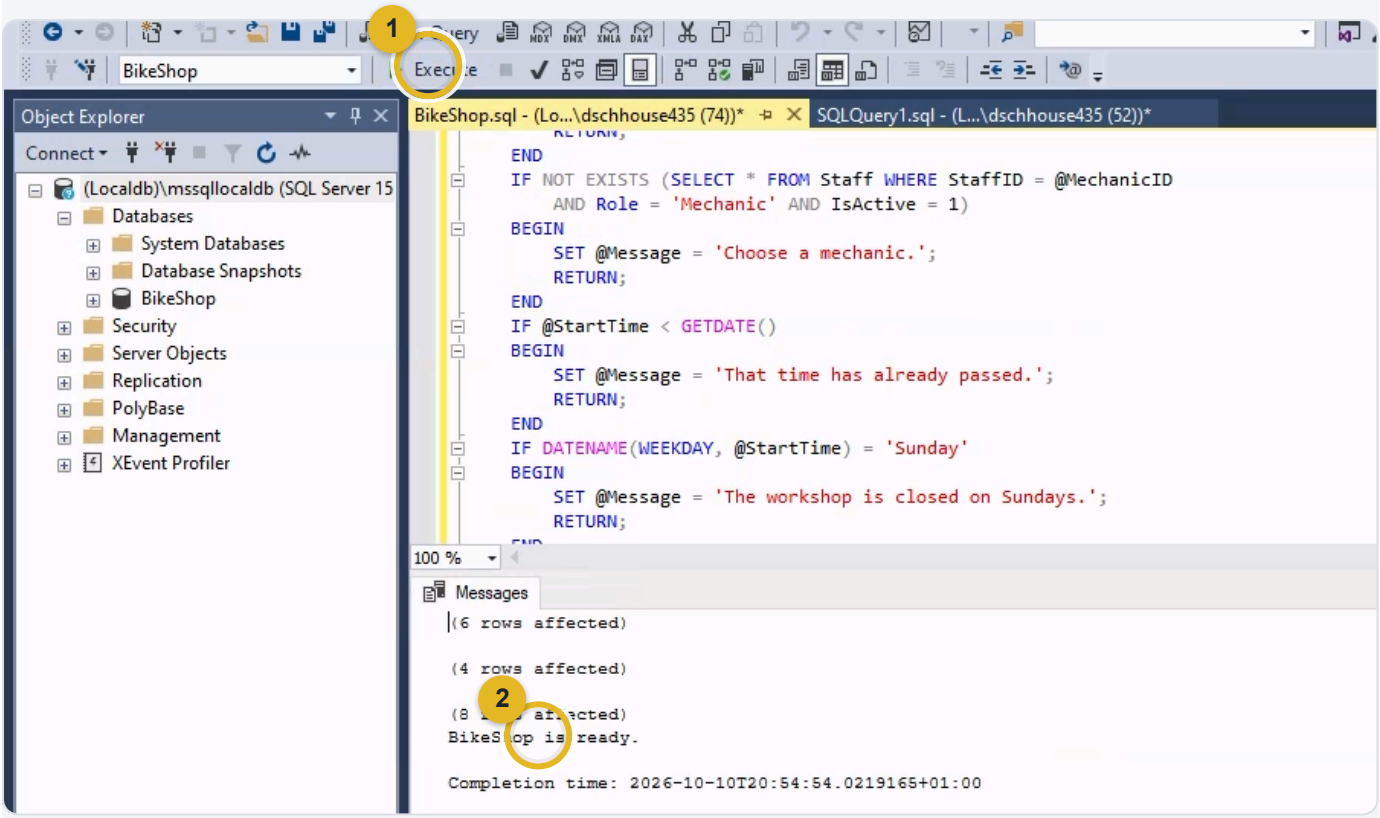
1. Click on the empty line above the line that prints BikeShop is ready, so the cursor blinks there.

> **You should see** The cursor blinks on the empty line, with the hire stored procedures from chapter 8 above it.

Press the Copy button under this step, click on the empty line above the line that prints BikeShop is ready, and paste.

> **You should see** The stored procedure is in your script, ending with GO, and the line that prints BikeShop is ready is still last.

Press F5, or click Execute on the toolbar, with nothing selected. This runs the whole script, so your database is built again from nothing with the new stored procedure in it.




Execute with nothing selected runs the whole script. The Messages pane counts the sample rows it put back, then says BikeShop is ready.

1. Click Execute, or press F5.

2. The last lines say BikeShop is ready, then a Completion time. Red text here is an error to read.

> **You should see** The Messages pane ends with BikeShop is ready and a Completion time. No red text. The sample data is back as it was at the start of chapter 8, which is what the tests in this chapter expect.

**IF IT GOES WRONG** A red error in the Messages pane

Double-click the error. SQL Server Management Studio (SSMS) jumps to the line it is about. Compare that line with the block above, letter by letter.

# Test it: the workload for this year.

The test asks for the whole of this year. The sample data is dated around today, so a whole year catches every job in it. A report hands back rows, not a message, so the answer appears in the Results tab, not the Messages pane.

**Copy this one** **A test that runs the workload report for this year** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
-- Which mechanic did the most work this year?
-- Four rows, one per mechanic, biggest earner first.
DECLARE @From DATE, @To DATE;
SET @From = '2026-01-01';
SET @To = '2026-12-31';

EXEC usp_Report_MechanicWorkload @FromDate = @From, @ToDate = @To;
```

**1 Two dates in two boxes**

Lines 3 to 5

A variable is a named box that holds one value while the script runs. The two boxes hold the first and last day of this year. Change either SET line to ask about different dates.

**2 Run the report**

Line 7

EXEC runs the stored procedure with the two dates. The rows it makes come back as a table in the Results tab.

Click New Query on the toolbar, or press Ctrl and N together.

> **You should see** A new tab opens with an empty query window, and the cursor blinks in it.

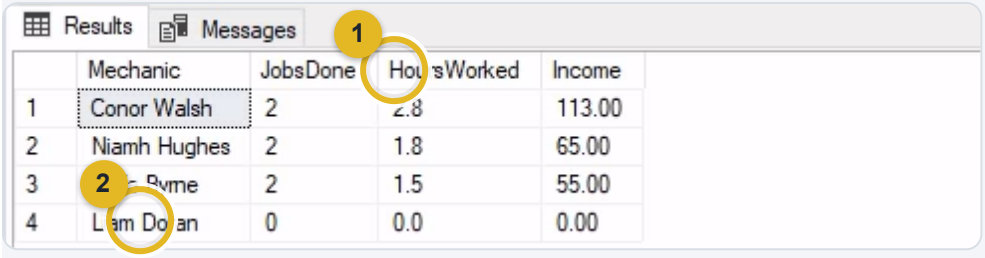
Press the Copy button under this step.

> **You should see** The button reads Copied for a moment.

Click in the empty query window and press Ctrl and V together.

> **You should see** The test is in the window, with the DECLARE line at the top and the line that runs usp_Report_MechanicWorkload at the bottom.

Press F5 to run the test.




| | Mechanic | JobsDone | HoursWorked | Income |
|---|---|---|---|---|
| 1 | Conor Walsh | 2 | 2.8 | 113.00 |
| 2 | Niamh Hughes | 2 | 1.8 | 65.00 |
| 3 | Aoife Byrne | 2 | 1.5 | 55.00 |
| 4 | Liam Doran | 0 | 0.0 | 0.00 |

The workload report for the year. Liam Doran left in the summer and did no jobs this year, and he is still listed, with zeros, because of the LEFT JOIN.

1 The column headings are the AS names from the SELECT.

2 Liam Doran: a row of zeros, not a missing row.

> **You should see** The Results tab shows four rows: Conor Walsh with 2 jobs, 2.8 hours and 113.00; Niamh Hughes with 2, 1.8 and 65.00; Aoife Byrne with 2, 1.5 and 55.00; and Liam Doran with 0, 0.0 and 0.00. The Messages tab says (4 rows affected).

Change the two dates to '2000-01-01' and '2000-01-02', and press F5 again.

> **You should see** The same four names, each with 0, 0.0 and 0.00. Nobody did a job in the year 2000, and every mechanic is still listed. That is the LEFT JOIN doing its work.

Change the two dates back to this year, and leave the window open.

> **You should see** The four rows with their real numbers are back. The next stages paste their tests into this same window.

**IF IT GOES WRONG** Liam Doran is in the list, but he left the shop

The report asks about the Role column, which still says Mechanic, not the IsActive column. That is right: a mechanic who left in the summer still did jobs earlier in the year, and the shop wants them counted.

**IF IT GOES WRONG** Conor Walsh shows 2.75 hours, not 2.8

The CAST to DECIMAL(6,1) is missing or has a different number after the comma. One decimal place is the shape the form expects.

STAGE 3 OF 8 · WHERE THE MONEY CAME FROM · SCREEN 1 OF 2

# Read the income stored procedure, and type the lines that stack it.

Money comes into the shop from three places: the workshop, bike hire and parts. They live in three different tables. The second report puts all three into one grid, one row per day per source, by stacking three SELECTs on top of each other with UNION ALL. The three SELECTs must have the same columns in the same order. The whole stack sits inside brackets as a derived table, so that one ORDER BY at the end can sort every row by day. You type the seams, where one part ends and the next begins, so that you see the columns lining up with your own hands. Add it in SQL Server Management Studio (SSMS) the same way as before.

**Type this one** **usp_Report_Income: the money per day, from three tables** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  CREATE OR ALTER PROCEDURE usp_Report_Income
2      @FromDate DATE,
3      @ToDate   DATE
4  AS
5  BEGIN
6      SELECT [Day], Source, Items, Income
7      FROM (
8          SELECT CAST(a.StartTime AS DATE) AS [Day], 'Workshop' AS Source,
9                 COUNT(*) AS Items, SUM(st.Price) AS Income
10         FROM Appointment a
11         JOIN ServiceType st ON st.ServiceTypeID = a.ServiceTypeID
12         WHERE a.Status = 'Done' AND CAST(a.StartTime AS DATE) BETWEEN @FromDate
13             AND @ToDate
14         GROUP BY CAST(a.StartTime AS DATE)
15
16         UNION ALL
17
18         SELECT h.EndDate, 'Bike hire', COUNT(*), SUM(h.TotalCost)
19         FROM Hire h
20         WHERE h.Status = 'Returned' AND h.EndDate BETWEEN @FromDate AND @ToDate
21         GROUP BY h.EndDate
22
23         UNION ALL
24
25         SELECT CAST(ps.SoldOn AS DATE), 'Parts', SUM(ps.Quantity),
26             SUM(ps.Quantity * ps.UnitPrice)
27         FROM PartSale ps
28         WHERE ps.Status = 'Sold' AND CAST(ps.SoldOn AS DATE) BETWEEN @FromDate
29             AND @ToDate
30         GROUP BY CAST(ps.SoldOn AS DATE)
31     ) AS income
32     ORDER BY [Day], Source;
33 END
34 GO
```

**1 The name, and the two dates that go in**

Lines 1 to 3

Every report takes the same two things: a From date and a To date. Nothing comes back through a parameter. The answer is the table of rows that the SELECT makes.

**2 AS and BEGIN**

Lines 4 to 5

This stored procedure only reads, so it has no @Message. AS and BEGIN mark where the work starts.

**3 The outer SELECT: four columns from a stack**

Lines 6 to 7

This SELECT reads from the bracket that follows, not from a table. Day is in square brackets because SQL Server uses that word itself. The four names are the column headings the grid will show.

### 4 The workshop part

Lines 8 to 13

One row per day of finished jobs: the day, the word Workshop as its source, how many jobs, and the prices added up. Appointment is joined to ServiceType for the price. The AS names here name the columns for the whole stack, because the first SELECT names the columns for all of them.

### 5 The first seam: the yellow lines

Lines 14 to 18

GROUP BY on the day ends the workshop part. UNION ALL stacks the next part underneath. The SELECT that starts the bike hire part gives the same four things: a day, the words Bike hire in quotes, a count and a sum. It has no AS names, because the first part named the columns already.

### 6 Where the hire money comes from

Lines 19 to 20

Only hires that came back count, on the day they came back. EndDate is already a date, so no CAST is needed.

### 7 The second seam: the yellow lines

Lines 21 to 25

The same shape again. GROUP BY on the day ends the hire part, UNION ALL stacks the parts sales underneath, and the SELECT gives a day, the word Parts, and a count. The count here is a SUM of quantities, because one sale can be three inner tubes.

### 8 Where the parts money comes from

Lines 26 to 30

The income of a sale is its quantity times the price it was sold at. Only sales still marked Sold count, so a refund drops out. CAST to DATE throws away the clock part of SoldOn.

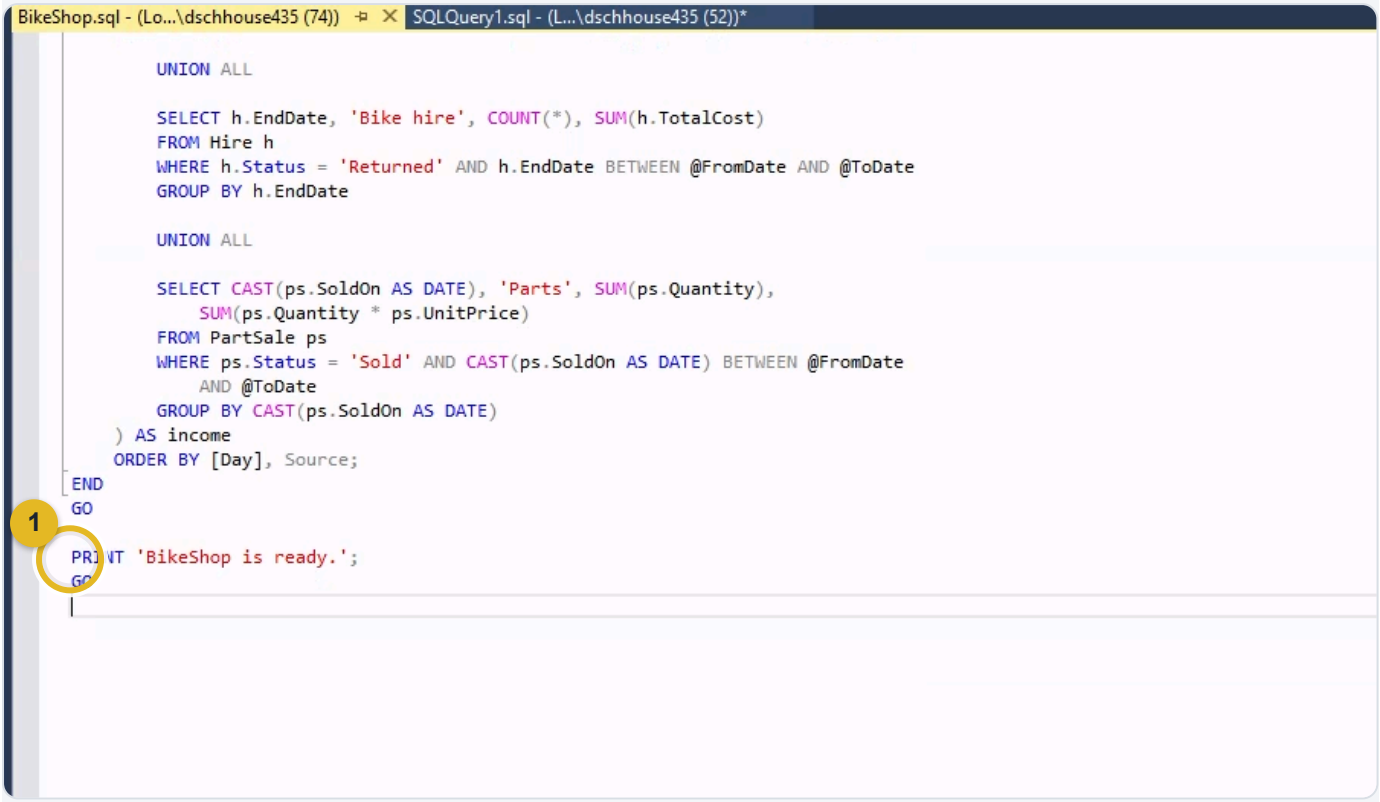
### 9 Close the stack and sort it

Lines 31 to 34

The closing bracket ends the derived table, and AS income gives it a name, which SQL Server insists on. ORDER BY sorts the whole stack by day, then by source, so the three sources of one day sit together. GO ends the batch, ready for the next stored procedure.

**Why you type it** Each yellow block is a seam: the GROUP BY that ends one part, UNION ALL, and the SELECT that starts the next. The SELECT you type must give the same four columns, in the same order, as the workshop part above it. Typing it is how you learn to line them up.

Scroll to the very end of the script and click on the empty line above the line that prints BikeShop is ready. That line always stays last.

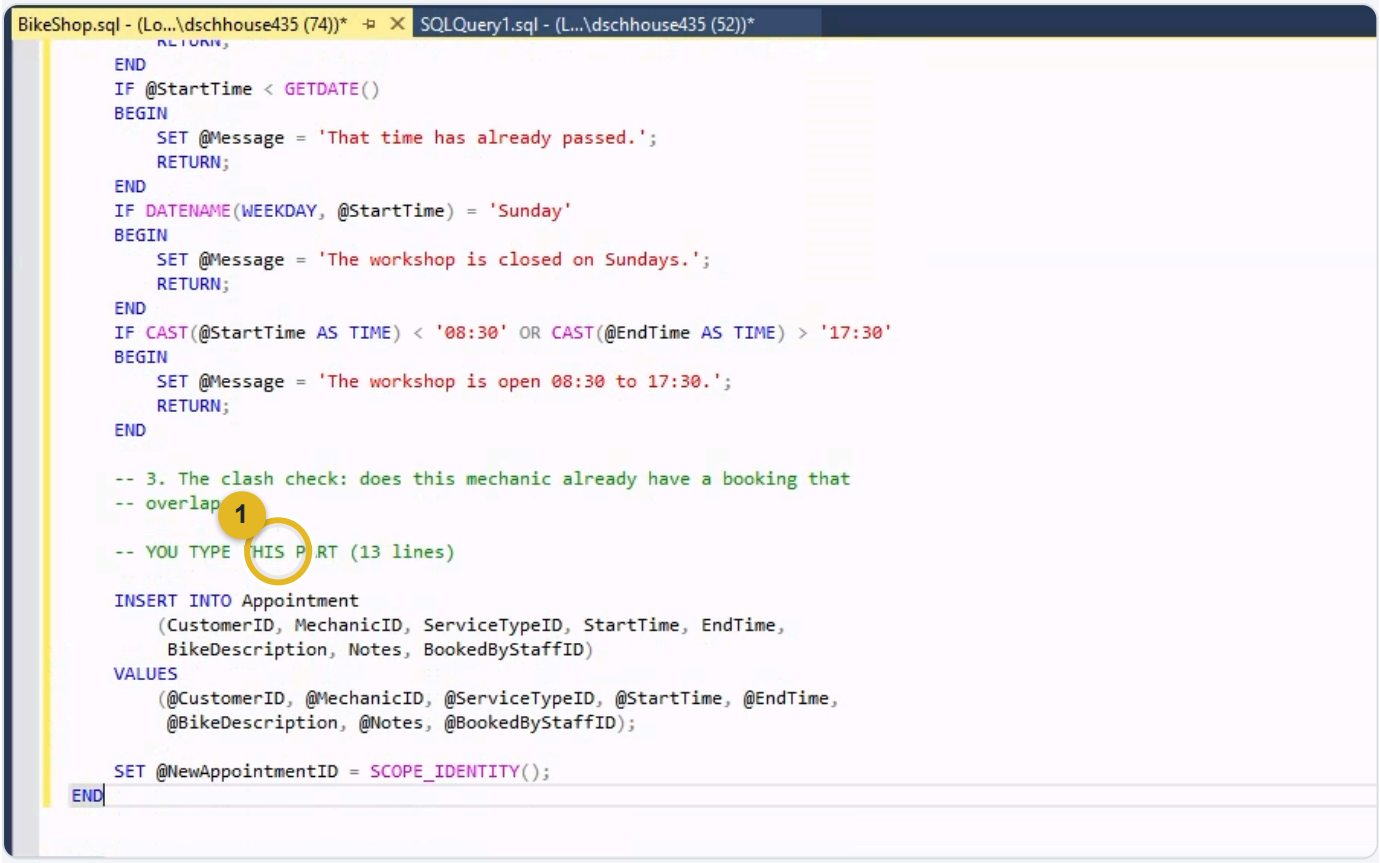



The end of your script. Everything new in this chapter goes on the empty line above the line that prints BikeShop is ready, one stored procedure after another, each ending with GO.

1. Click on the empty line above the line that prints BikeShop is ready, so the cursor blinks there.

> **You should see** The cursor blinks on the empty line, with the stored procedures you added before it above it.

Press the Copy button under this step, click on the empty line above the line that prints BikeShop is ready, and paste.




After the paste. Everything is in except the yellow lines. Each marker line shows where a yellow block goes.

1 Click on the marker line, type the yellow lines in its place, and delete the marker.

> **You should see** The stored procedure is in your script, with a line that says YOU TYPE THIS PART where each yellow block goes.

Type the yellow lines in place of each line that says YOU TYPE THIS PART, so that no marker line is left.

> **You should see** You have typed six lines of your own, three at each seam, and no marker line is left.

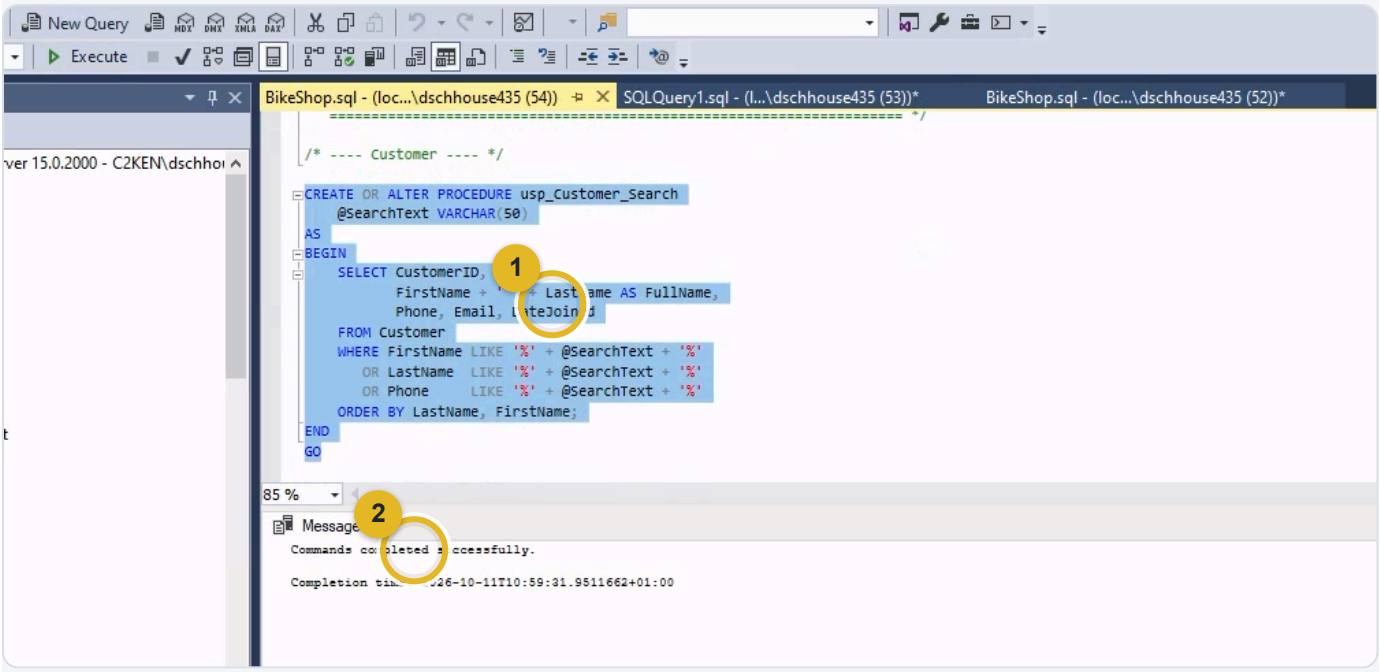
**IF IT GOES WRONG** A red wavy line under a word

A spelling slip. Compare your line with the yellow line, letter by letter, and check that each line ends the way the yellow line ends.

**IF IT GOES WRONG** 'Must declare the scalar variable …'

A name beginning with @ is spelled differently from the line that declares it. Make the two match.

Select the lines you added, from CREATE OR ALTER PROCEDURE down to GO, and press F5.




Select the lines you added, then press F5. Only the highlighted lines run.

1. Drag from CREATE OR ALTER PROCEDURE down to GO, so the whole block is highlighted.

2. The Messages pane says: Commands completed successfully.

**You should see** The Messages pane says: Commands completed successfully. Only the highlighted lines ran.

**IF IT GOES WRONG** The Messages pane shows (6 rows affected) and BikeShop is ready

Nothing was selected, so the whole script ran and the sample data was put back. Nothing is broken, because a report never changes data. Carry on.

**IF IT GOES WRONG** 'All queries combined using a UNION, INTERSECT or EXCEPT operator must have an equal number of expressions'

One SELECT in the stack has a column missing or one too many. Every SELECT joined by UNION ALL must list the same number of columns, in the same order. Count the commas in each one.

# Test it: fourteen lines of income.

**Copy this one** **A test that runs the income report for this year** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  -- Where did the money come from this year, day by day?
2  -- One row per day per source: Workshop, Bike hire or Parts.
3  DECLARE @From DATE, @To DATE;
4  SET @From = '2026-01-01';
5  SET @To = '2026-12-31';
6
7  EXEC usp_Report_Income @FromDate = @From, @ToDate = @To;
```

**1 Two dates in two boxes**

Lines 3 to 5

A variable is a named box that holds one value while the script runs. The two boxes hold the first and last day of this year. Change either SET line to ask about different dates.

**2 Run the report**

Line 7

EXEC runs the stored procedure with the two dates. The stacked rows come back as one table in the Results tab.

Press the Copy button under this step.

**You should see** The button reads Copied for a moment.

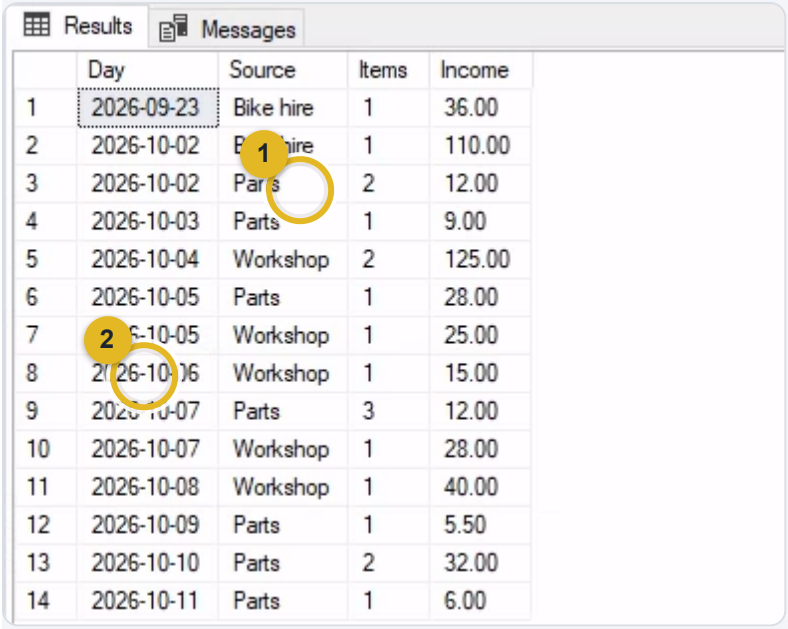
Click in your test window and press Ctrl and A together to select the old test.

**You should see** The whole of the old test is highlighted.

Press Ctrl and V together to paste the new test over the old one.

**You should see** The new test has replaced the old one, and its EXEC line runs usp_Report_Income.

Press F5 to run the test.



Results | Messages

| | Day | Source | Items | Income |
|---|---|---|---|---|
| 1 | 2026-09-23 | Bike hire | 1 | 36.00 |
| 2 | 2026-10-02 | Bike hire | 1 | 110.00 |
| 3 | 2026-10-02 | Parts | 2 | 12.00 |
| 4 | 2026-10-03 | Parts | 1 | 9.00 |
| 5 | 2026-10-04 | Workshop | 2 | 125.00 |
| 6 | 2026-10-05 | Parts | 1 | 28.00 |
| 7 | 2026-10-05 | Workshop | 1 | 25.00 |
| 8 | 2026-10-06 | Workshop | 1 | 15.00 |
| 9 | 2026-10-07 | Parts | 3 | 12.00 |
| 10 | 2026-10-07 | Workshop | 1 | 28.00 |
| 11 | 2026-10-08 | Workshop | 1 | 40.00 |
| 12 | 2026-10-09 | Parts | 1 | 5.50 |
| 13 | 2026-10-10 | Parts | 2 | 32.00 |
| 14 | 2026-10-11 | Parts | 1 | 6.00 |

The income report for the year: fourteen lines from three tables, sorted into one list by day.

1. Three sources in one Source column, because UNION ALL stacked three SELECTs.

2. The days run in order, because ORDER BY sorted the whole stack.

**You should see** The Results tab shows fourteen rows, oldest day first. The Source column holds Workshop, Bike hire or Parts. The two Bike hire rows show 36.00 and 110.00, and the Workshop rows add up to 233.00, the same total the workload report gave.

Change the two dates to '2000-01-01' and '2000-01-02', and press F5 again.

**You should see** The Results tab shows the four headings and no rows, and the Messages tab says (0 rows affected). An empty answer is still an answer.

Change the two dates back to this year.

**You should see** The fourteen rows are back.

**IF IT GOES WRONG** 'All queries combined using a UNION, INTERSECT or EXCEPT operator must have an equal number of expressions'

One SELECT in the stack has a column missing or one too many. Every SELECT joined by UNION ALL must list the same number of columns, in the same order. Count the commas in each one.

**IF IT GOES WRONG** 'Incorrect syntax near the keyword ORDER'

The derived table has no name. The line after the closing bracket must read ) AS income, and ORDER BY

comes after it.

STAGE 4 OF 8 · HOW MUCH HAPPENED · SCREEN 1 OF 2

# Type the four stacked counts of the activity stored procedure.

The third report is five numbers: bookings kept, bookings cancelled, hires, parts sold and orders placed, each between the two dates. Each number is one small SELECT that gives one row with a label and a count, and UNION ALL stacks the five rows into one grid. The label is text in quotes, so the first column reads Workshop bookings and not a column name. The first SELECT is given. In SQL Server Management Studio (SSMS), you type the other four from its pattern: same shape, a different table, a different status test.

Type this one **usp_Report_Activity: five counts in one grid** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
CREATE OR ALTER PROCEDURE usp_Report_Activity
    @FromDate DATE,
    @ToDate   DATE
AS
BEGIN
    SELECT 'Workshop bookings' AS [What], COUNT(*) AS [How many]
    FROM Appointment
    WHERE Status <> 'Cancelled'
      AND CAST(StartTime AS DATE) BETWEEN @FromDate AND @ToDate

    UNION ALL

    SELECT 'Bookings cancelled', COUNT(*)
    FROM Appointment
    WHERE Status = 'Cancelled'
      AND CAST(StartTime AS DATE) BETWEEN @FromDate AND @ToDate

    UNION ALL

    SELECT 'Bike hires', COUNT(*)
    FROM Hire
    WHERE Status <> 'Cancelled'
      AND StartDate BETWEEN @FromDate AND @ToDate

    UNION ALL

    SELECT 'Parts sold', ISNULL(SUM(Quantity), 0)
    FROM PartSale
    WHERE Status = 'Sold'
      AND CAST(SoldOn AS DATE) BETWEEN @FromDate AND @ToDate

    UNION ALL

    SELECT 'Part orders placed', COUNT(*)
    FROM PartOrder
    WHERE Status <> 'Cancelled'
      AND OrderedOn BETWEEN @FromDate AND @ToDate;
END
GO
```

**1 The name, and the two dates that go in**

Lines 1 to 3

Every report takes the same two things: a From date and a To date. Nothing comes back through a parameter. The answer is the table of rows that the SELECT makes.

**2 AS and BEGIN**

Lines 4 to 5

This stored procedure only reads, so it has no @Message. AS and BEGIN mark where the work starts.

**3 The first count: bookings kept**

Lines 6 to 9

The text in quotes is the label, and AS [What] names the column. COUNT(*) counts the appointment rows that are not cancelled and started between the dates. How many is in square brackets because it has a space in it. This SELECT names the two columns for the whole stack.

**4 The second count: bookings cancelled**

Lines 11 to 16

UNION ALL stacks one more row underneath. The same table, with the status test turned round: only the cancelled ones. No AS names, because the first SELECT named the columns.

**5 The third count: bike hires**

Lines 18 to 23

Hires that were not cancelled and started between the dates. StartDate is already a date, so there is no CAST.

**6 The fourth count: parts sold**

Lines 25 to 30

This one is a SUM, not a COUNT, because one sale row can be three inner tubes. ISNULL turns an empty sum into 0 when nothing was sold, so the row still appears with a number in it.

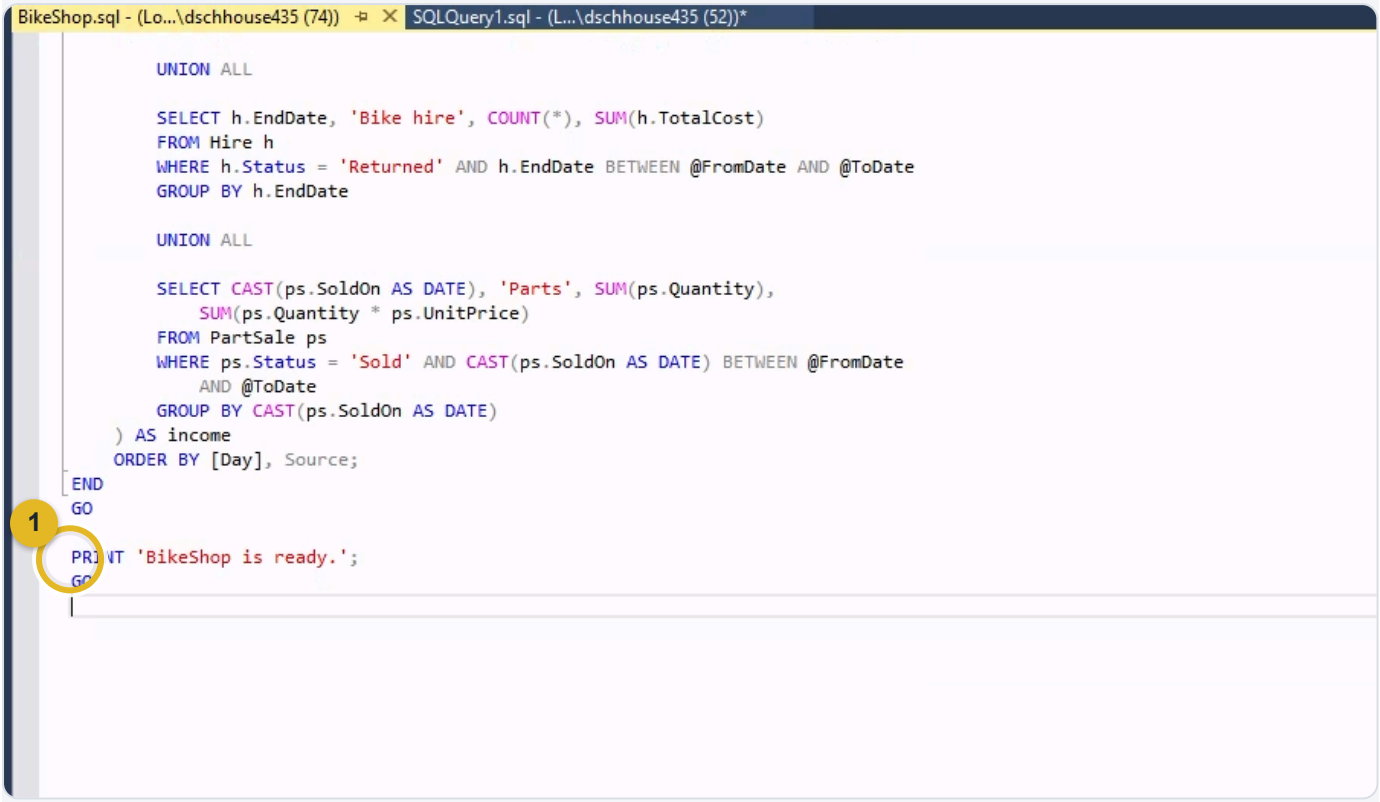
**7 The fifth count: orders placed**

Lines 32 to 39

Orders that were not cancelled and were placed between the dates. The last line ends with a semicolon, because it is the end of the whole stacked statement. GO ends the batch, ready for the next stored procedure.

**Why you type it** Four times the same four lines, each on a different table. Typing them is how the pattern sticks: a label in quotes, a COUNT or SUM, the table, the status test, the date test.

Scroll to the very end of the script and click on the empty line above the line that prints BikeShop is ready. That line always stays last.

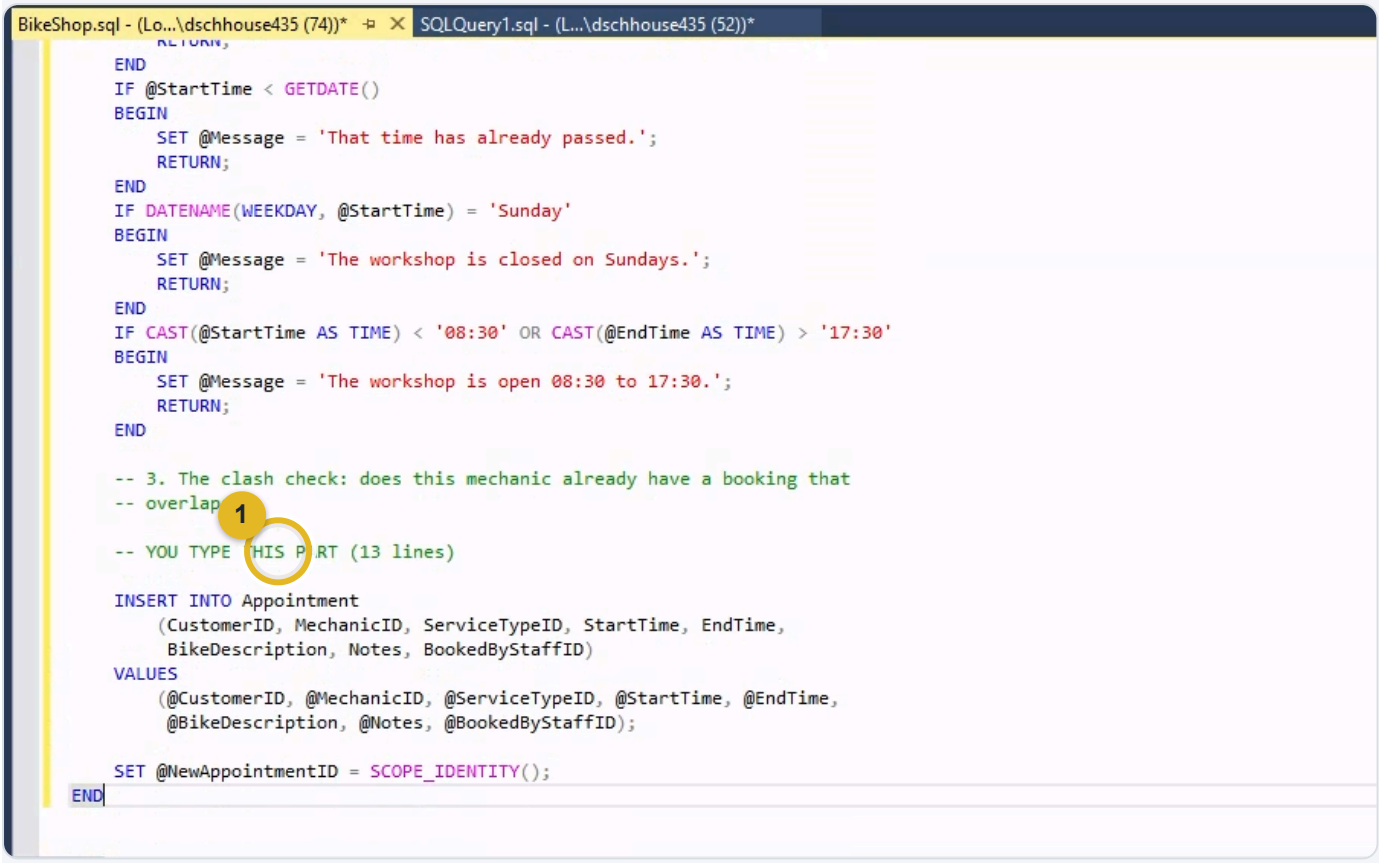



The end of your script. Everything new in this chapter goes on the empty line above the line that prints BikeShop is ready, one stored procedure after another, each ending with GO.

1. Click on the empty line above the line that prints BikeShop is ready, so the cursor blinks there.

> **You should see** The cursor blinks on the empty line, with the stored procedures you added before it above it.

Press the Copy button under this step, click on the empty line above the line that prints BikeShop is ready, and paste.




After the paste. Everything is in except the yellow lines. Each marker line shows where a yellow block goes.

1 Click on the marker line, type the yellow lines in its place, and delete the marker.

> **You should see** The stored procedure is in your script, with a line that says YOU TYPE THIS PART where each yellow block goes.

Type the yellow lines in place of each line that says YOU TYPE THIS PART, so that no marker line is left.

> **You should see** You have typed four stacked SELECTs of your own under the first one, each starting with UNION ALL, and no marker line is left.

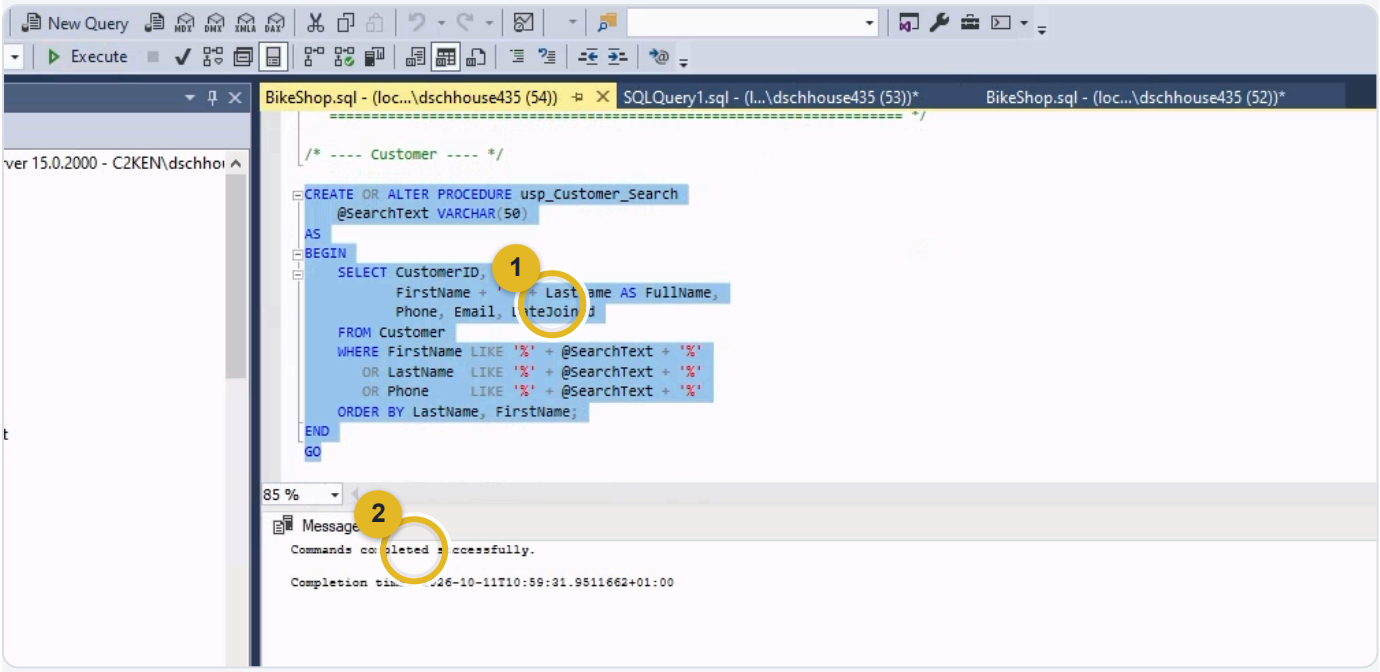
**IF IT GOES WRONG** A red wavy line under a word

A spelling slip. Compare your line with the yellow line, letter by letter, and check that each line ends the way the yellow line ends.

**IF IT GOES WRONG** 'Must declare the scalar variable …'

A name beginning with @ is spelled differently from the line that declares it. Make the two match.

Select the lines you added, from CREATE OR ALTER PROCEDURE down to GO, and press F5.




Select the lines you added, then press F5. Only the highlighted lines run.

1. Drag from CREATE OR ALTER PROCEDURE down to GO, so the whole block is highlighted.

2. The Messages pane says: Commands completed successfully.

**You should see** The Messages pane says: Commands completed successfully. Only the highlighted lines ran.

**IF IT GOES WRONG** The Messages pane shows (6 rows affected) and BikeShop is ready

Nothing was selected, so the whole script ran and the sample data was put back. Nothing is broken, because a report never changes data. Carry on.

**IF IT GOES WRONG** 'All queries combined using a UNION, INTERSECT or EXCEPT operator must have an equal number of expressions'

One SELECT in the stack has a column missing or one too many. Every SELECT joined by UNION ALL must list the same number of columns, in the same order. Count the commas in each one.

# Test it: five counts for this year.

**Copy this one** **A test that runs the activity report for this year** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  -- How much happened this year?  Five rows, one count each.
2  DECLARE @From DATE, @To DATE;
3  SET @From = '2026-01-01';
4  SET @To = '2026-12-31';
5
6  EXEC usp_Report_Activity @FromDate = @From, @ToDate = @To;
```

**1 Two dates in two boxes**

Lines 2 to 4

A variable is a named box that holds one value while the script runs. The two boxes hold the first and last day of this year. Change either SET line to ask about different dates.

**2 Run the report**

Line 6

EXEC runs the stored procedure with the two dates. Five rows come back in the Results tab, in the order the SELECTs are stacked.

Press the Copy button under this step.

**You should see** The button reads Copied for a moment.

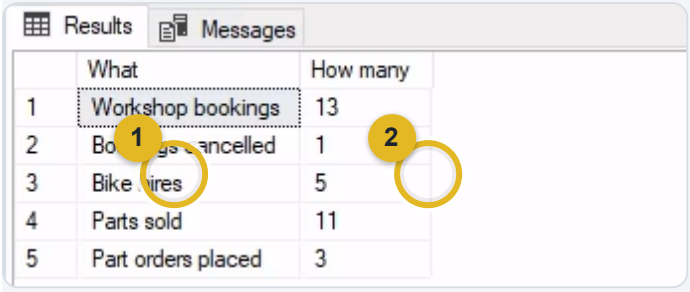
Click in your test window and press Ctrl and A together to select the old test.

**You should see** The whole of the old test is highlighted.

Press Ctrl and V together to paste the new test over the old one.

**You should see** The new test has replaced the old one, and its EXEC line runs usp_Report_Activity.

Press F5 to run the test.




The activity report for the year. Each row is one SELECT, and its label is the text in quotes.

1. The What column holds the five labels you typed.

2. The How many column holds one count each.

**You should see** The Results tab shows five rows: Workshop bookings 13, Bookings cancelled 1, Bike hires 5, Parts sold 11, Part orders placed 3.

**IF IT GOES WRONG** 'Invalid column name'

A column name in one of the four SELECTs you typed is spelled differently from the table. Double-click the error to jump to the line, and compare it with the yellow lines, letter by letter.

**IF IT GOES WRONG** Parts sold shows a different number

Count the Sold rows in your PartSale table and add up their Quantity column. If you sold or refunded parts in chapter 8 since the last whole-script run, the number is right for your data.

# Read the popular services stored procedure, then add it to your script.

The fourth report lists every service with how many times it was booked between the dates, most booked first. It lists every service, including the ones nobody booked: a manager wants to see the 0 beside Safety check, not wonder where it went. That is a LEFT JOIN from ServiceType to Appointment, with the date test after ON, the same shape as the workload report. Read the stored procedure below, then add it to your script in SQL Server Management Studio (SSMS). The next screen breaks the report on purpose, so that you see what the LEFT does.

Copy this one **usp_Report_PopularServices: every service, most booked first** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  CREATE OR ALTER PROCEDURE usp_Report_PopularServices
2      @FromDate DATE,
3      @ToDate   DATE
4  AS
5  BEGIN
6      SELECT st.Name AS Service,
7             COUNT(a.AppointmentID) AS Bookings,
8             st.Price
9      FROM ServiceType st
10     LEFT JOIN Appointment a
11            ON a.ServiceTypeID = st.ServiceTypeID
12           AND a.Status <> 'Cancelled'
13           AND CAST(a.StartTime AS DATE) BETWEEN @FromDate AND @ToDate
14     GROUP BY st.ServiceTypeID, st.Name, st.Price
15     ORDER BY Bookings DESC, Service;
16 END
17 GO
```

**1 The name, and the two dates that go in**

Lines 1 to 3

Every report takes the same two things: a From date and a To date. Nothing comes back through a parameter. The answer is the table of rows that the SELECT makes.

**2 AS and BEGIN**

Lines 4 to 5

This stored procedure only reads, so it has no @Message. AS and BEGIN mark where the work starts.

**3 Three columns**

Lines 6 to 8

The service's name, how many appointments matched it, and its price. COUNT on the appointment number gives 0 for a service with no matching appointments, because there are no numbers to count.

**4 Start from every service**

Line 9

The report starts from ServiceType, because the question is about services. Every row of this table will be in the answer.

**5 Bring in the bookings that match, and keep the services that have none**

Lines 10 to 13

LEFT JOIN keeps every service even when no appointment matches. The three tests after ON pick the appointments that count: the right service, not cancelled, and started between the dates. The date test must sit here, after ON. If it moved to a WHERE, a service with no bookings in those dates would be thrown away, and the 0 would vanish with it.

**6 One row per service**

Line 14

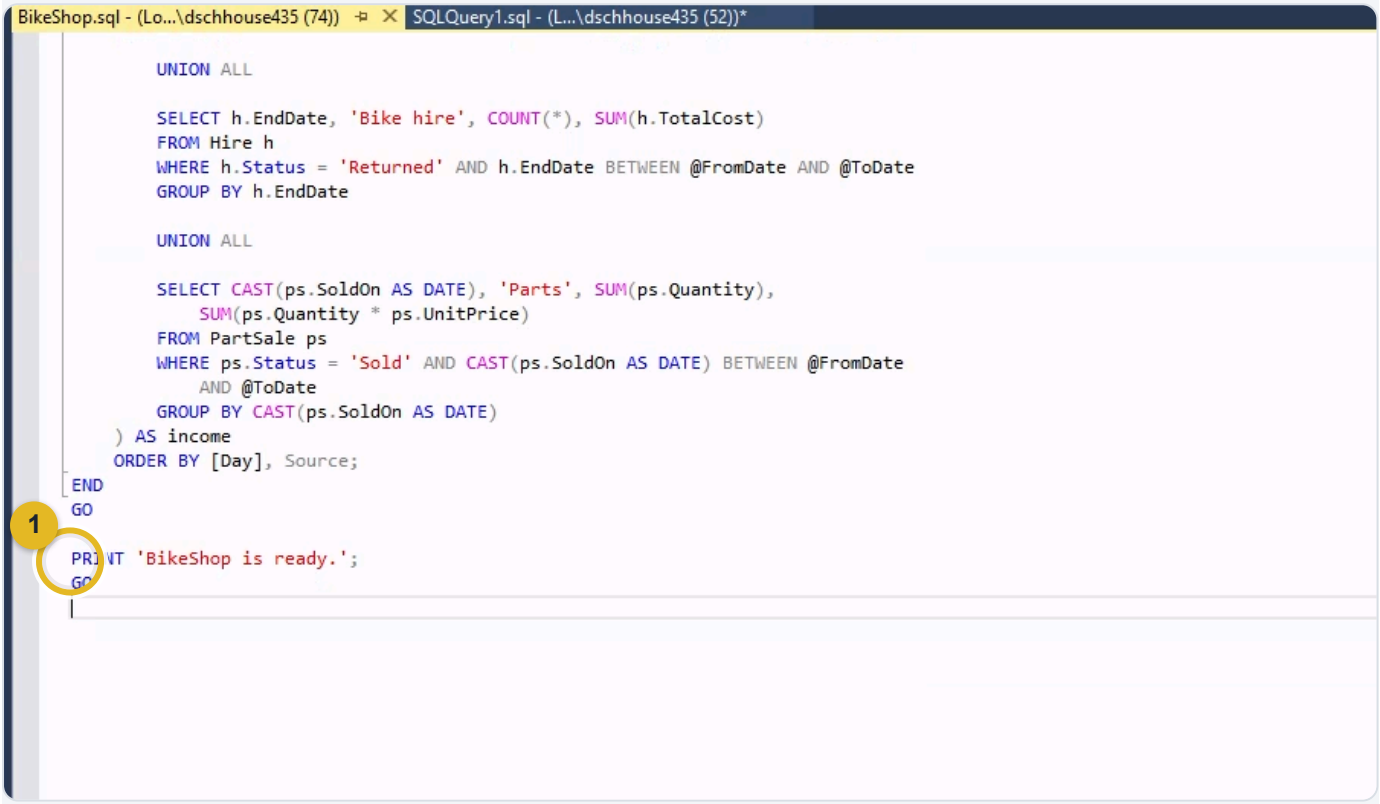
All of one service's appointment rows are squashed into one row, so COUNT has something to count. Name and Price are listed because they are in the SELECT outside the COUNT.

**7 Most booked first**

Lines 15 to 17

Highest count first. Two services with the same count are put in name order, so the list is always in the same order. GO ends the batch, ready for the next stored procedure.

Scroll to the very end of the script and click on the empty line above the line that prints BikeShop is ready. That line always stays last.




The end of your script. Everything new in this chapter goes on the empty line above the line that prints BikeShop is ready, one stored procedure after another, each ending with GO.

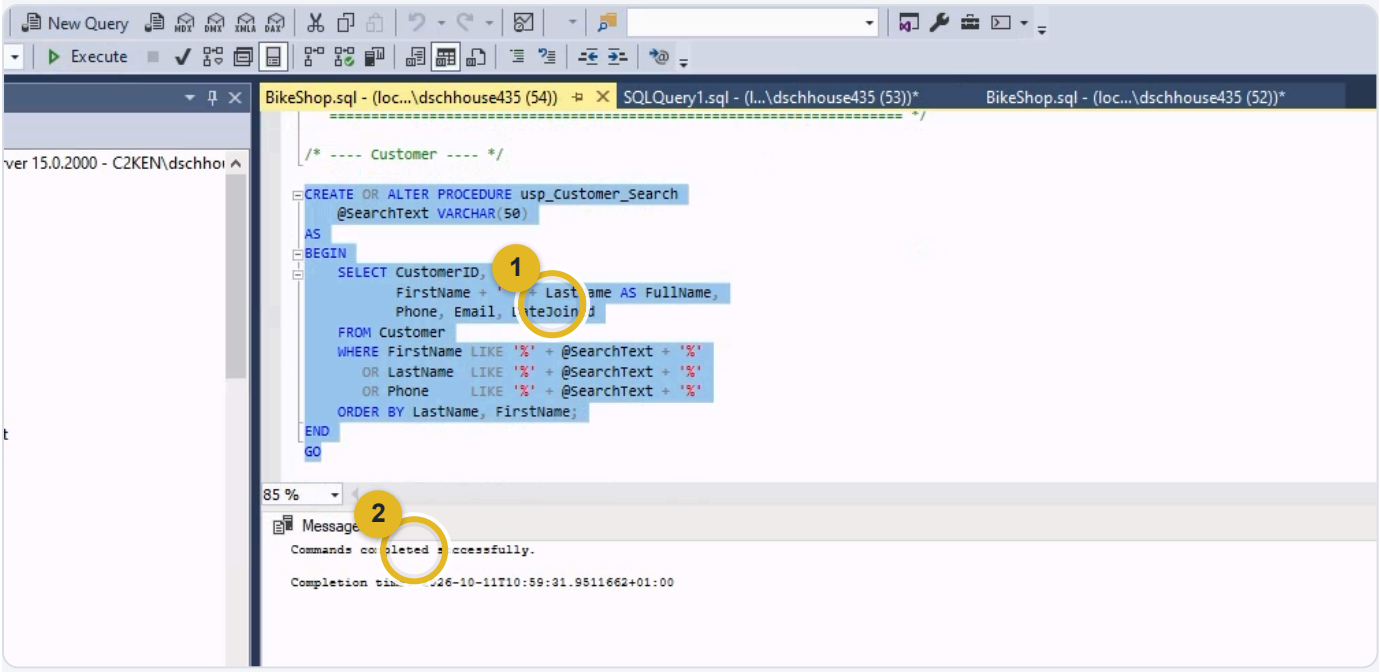
1 Click on the empty line above the line that prints BikeShop is ready, so the cursor blinks there.

**You should see** The cursor blinks on the empty line, with the stored procedures you added before it above it.

Press the Copy button under this step, click on the empty line above the line that prints BikeShop is ready, and paste.

**You should see** The stored procedure is in your script, ending with GO, and the line that prints BikeShop is ready is still last.

Select the lines you added, from CREATE OR ALTER PROCEDURE down to GO, and press F5.




Select the lines you added, then press F5. Only the highlighted lines run.

1. Drag from CREATE OR ALTER PROCEDURE down to GO, so the whole block is highlighted.

2. The Messages pane says: Commands completed successfully.

**You should see** The Messages pane says: Commands completed successfully. Only the highlighted lines ran.

**IF IT GOES WRONG** The Messages pane shows (6 rows affected) and BikeShop is ready

Nothing was selected, so the whole script ran and the sample data was put back. Nothing is broken, because a report never changes data. Carry on.

# Test it, then break it on purpose.

First you run the report as it should be. Then you take out one word, LEFT, and run the test again with the year 2000 dates, so that you see the rows go missing. Then you put the word back.

**Copy this one** **A test that runs the popular services report for this year** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
-- Which services were booked most this year?
-- Every service is listed, even one that nobody booked.
DECLARE @From DATE, @To DATE;
SET @From = '2026-01-01';
SET @To = '2026-12-31';

EXEC usp_Report_PopularServices @FromDate = @From, @ToDate = @To;
```

**1 Two dates in two boxes**

Lines 3 to 5

A variable is a named box that holds one value while the script runs. The two boxes hold the first and last day of this year. Change either SET line to ask about different dates.

**2 Run the report**

Line 7

EXEC runs the stored procedure with the two dates. One row per service comes back in the Results tab.

Press the Copy button under this step.

**You should see** The button reads Copied for a moment.

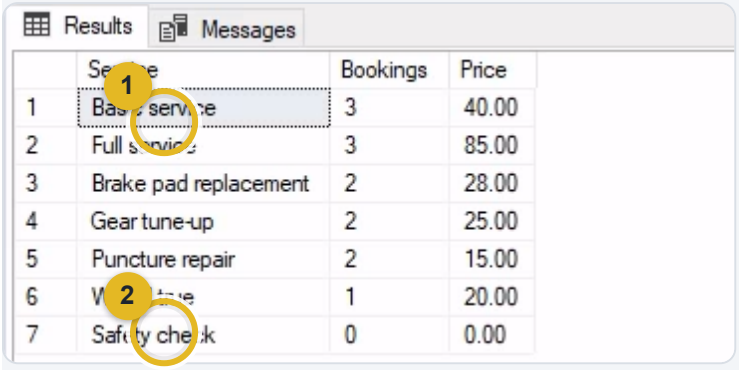
Click in your test window and press Ctrl and A together to select the old test.

**You should see** The whole of the old test is highlighted.

Press Ctrl and V together to paste the new test over the old one.

**You should see** The new test has replaced the old one, and its EXEC line runs usp_Report_PopularServices.

Press F5 to run the test.




Every service, most booked first. Safety check has no bookings this year and is still there, with 0.

1. Two services tie at 3. ORDER BY Bookings DESC, Service puts Basic service first, by name.

2. Safety check with 0: the row the LEFT JOIN keeps.

> **You should see** The Results tab shows seven rows: Basic service and Full service at the top with 3 each, then Brake pad replacement, Gear tune-up and Puncture repair with 2 each, Wheel true with 1, and Safety check at the bottom with 0.

Change the two dates to '2000-01-01' and '2000-01-02', and press F5 again.

> **You should see** Seven rows, every one with 0 in the Bookings column. Nobody booked anything in the year 2000, and every service is still listed.

Click in the tab that holds your script, find the line in usp_Report_PopularServices that reads LEFT JOIN Appointment a, and delete the word LEFT and the space after it.

> **You should see** The line now reads JOIN Appointment a, with the same indent as before.

Select the lines you added, from CREATE OR ALTER PROCEDURE down to GO, and press F5.

> **You should see** The Messages pane says: Commands completed successfully. Only the highlighted lines ran.

Click back in the test tab, and press F5 with the year 2000 dates still in it.

> **You should see** The Results tab shows the three headings and no rows at all. Without LEFT, a service with no matching appointment is thrown away, and the whole report is empty. A manager would read that as no services, which is wrong.

Click in the script tab, and type LEFT and a space back in front of JOIN Appointment a.

> **You should see** The line reads LEFT JOIN Appointment a again.

Select the lines you added, from CREATE OR ALTER PROCEDURE down to GO, and press F5.

> **You should see** The Messages pane says: Commands completed successfully. Only the highlighted lines ran.

Click back in the test tab, change the two dates back to this year, and press F5.

> **You should see** The seven rows are back, Basic service at the top with 3, Safety check at the bottom with 0.

# Your turn: write the popular parts stored procedure.

The fifth report is the fourth one with the words changed. It lists every part, how many were sold between the dates, and the money they brought in, best seller first, under the headings Part, Sold and Income. Part takes the place of ServiceType, PartSale takes the place of Appointment, and the count becomes a SUM of quantities, because one sale can be three inner tubes. Write usp_Report_PopularParts yourself, and the test on the next screen runs it.

Your turn **usp_Report_PopularParts: every part, best seller first** SQL

1. In SQL Server Management Studio (SSMS), click on the empty line above the line that prints BikeShop is ready and type the first lines of the stored procedure: CREATE OR ALTER PROCEDURE usp_Report_PopularParts, the two date parameters, AS and BEGIN.
2. Type the SELECT with three columns: p.Name AS Part, then ISNULL(SUM(ps.Quantity), 0) AS Sold, then ISNULL(SUM(ps.Quantity * ps.UnitPrice), 0) AS Income.
3. Type FROM Part p, then LEFT JOIN PartSale ps with three tests after ON: ps.PartID = p.PartID, AND ps.Status = 'Sold', AND CAST(ps.SoldOn AS DATE) BETWEEN @FromDate AND @ToDate.
4. Type GROUP BY p.PartID, p.Name and then ORDER BY Sold DESC, Part; and finish with END and GO.
5. Select the lines you typed, from CREATE OR ALTER PROCEDURE down to GO, and press F5.

**THE TEST** The ready-made test on the next screen runs your stored procedure for this year.

**YOU SHOULD SEE** Twelve rows, one per part. Brake cable and Inner tube 700c are at the top with 3 each, and six parts at the bottom show 0 and 0.00. If you see six rows instead of twelve, the LEFT is missing or the date test has moved into a WHERE.

**HINT** Open usp_Report_PopularServices in your script beside you and change one name at a time. The ISNULL round each SUM is new: a part with no sales has nothing to add up, and ISNULL turns that empty answer into 0.

**Stuck? Show me the finished procedure**

Read it, then write yours **usp_Report_PopularParts** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  CREATE OR ALTER PROCEDURE usp_Report_PopularParts
2      @FromDate DATE,
3      @ToDate   DATE
4  AS
5  BEGIN
6      SELECT p.Name AS Part,
7             ISNULL(SUM(ps.Quantity), 0) AS Sold,
8             ISNULL(SUM(ps.Quantity * ps.UnitPrice), 0) AS Income
9      FROM Part p
10     LEFT JOIN PartSale ps
11            ON ps.PartID = p.PartID
12           AND ps.Status = 'Sold'
13           AND CAST(ps.SoldOn AS DATE) BETWEEN @FromDate AND @ToDate
14     GROUP BY p.PartID, p.Name
15     ORDER BY Sold DESC, Part;
16 END
17 GO
```

**1 The name, and the two dates that go in**

Lines 1 to 3

Every report takes the same two things: a From date and a To date. Nothing comes back through a parameter. The answer is the table of rows that the SELECT makes.

**2 AS and BEGIN**

Lines 4 to 5

This stored procedure only reads, so it has no @Message. AS and BEGIN mark where the work starts.

**3 Three columns**

Lines 6 to 8

The part's name, how many were sold, and the money they made. Both sums are wrapped in ISNULL, so a part with no sales shows 0 and 0.00 instead of an empty cell.

**4 Start from every part**

Line 9

The report starts from Part, because the question is about parts, and every part must be in the answer.

**5 Bring in the sales that match, and keep the parts that have none**

Lines 10 to 13

LEFT JOIN keeps every part even when no sale matches. The three tests after ON pick the sales that count: the right part, still marked Sold, and sold between the dates. A refunded sale is not Sold, so it drops out.

**6 One row per part**

Line 14

All of one part's sale rows are squashed into one row, so the two SUMs have something to add up.

**7** **Best seller first**

Lines 15 to 17

Highest number sold first. Two parts with the same number are put in name order. GO ends the batch, ready for the next stored procedure.

# Test your popular parts stored procedure, and compare it with the finished one.

**Copy this one** **A test that runs the popular parts report for this year** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  -- Which parts sold most this year?
2  -- Every part is listed, even one that nobody bought.
3  DECLARE @From DATE, @To DATE;
4  SET @From = '2026-01-01';
5  SET @To = '2026-12-31';
6
7  EXEC usp_Report_PopularParts @FromDate = @From, @ToDate = @To;
```

**1 Two dates in two boxes**

Lines 3 to 5

A variable is a named box that holds one value while the script runs. The two boxes hold the first and last day of this year. Change either SET line to ask about different dates.

**2 Run the report**

Line 7

EXEC runs your stored procedure with the two dates. One row per part comes back in the Results tab.

Press the Copy button under this step.

> **You should see** The button reads Copied for a moment.

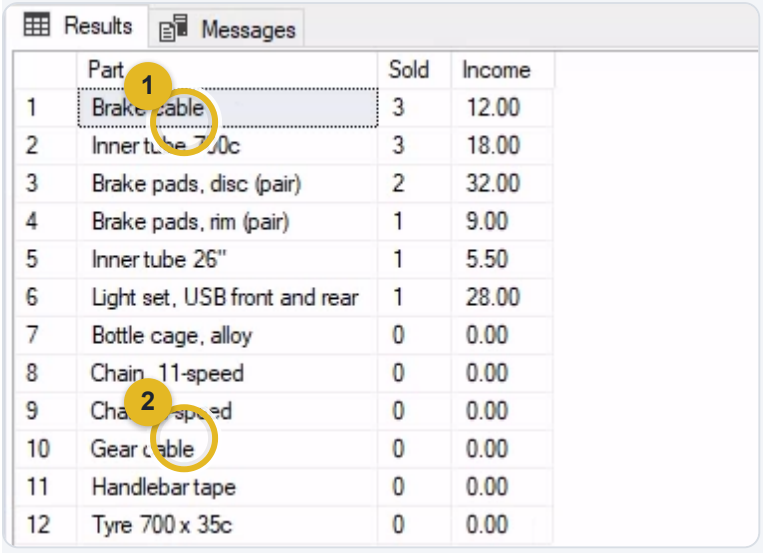
Click in your test window and press Ctrl and A together to select the old test.

> **You should see** The whole of the old test is highlighted.

Press Ctrl and V together to paste the new test over the old one.

> **You should see** The new test has replaced the old one, and its EXEC line runs usp_Report_PopularParts.

Press F5 to run the test.




Every part, best seller first, from the stored procedure you wrote yourself.

1. Brake cable and Inner tube 700c both sold 3 this year.

2. Six parts with 0. A JOIN would have dropped them.

> **You should see** The Results tab shows twelve rows: Brake cable with 3 and 12.00, Inner tube 700c with 3 and 18.00, Brake pads, disc (pair) with 2 and 32.00, then three parts with 1 each, then six parts with 0 and 0.00.

This is the finished SELECT, to hold beside yours. The whole stored procedure is in the fold-out on the last screen, part by part.

Read this one **The finished SELECT of usp_Report_PopularParts** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
SELECT p.Name AS Part,
       ISNULL(SUM(ps.Quantity), 0) AS Sold,
       ISNULL(SUM(ps.Quantity * ps.UnitPrice), 0) AS Income
FROM Part p
LEFT JOIN PartSale ps
       ON ps.PartID = p.PartID
      AND ps.Status = 'Sold'
      AND CAST(ps.SoldOn AS DATE) BETWEEN @FromDate AND @ToDate
GROUP BY p.PartID, p.Name
ORDER BY Sold DESC, Part;
```

**1 Three columns**

Lines 1 to 3

The part's name, how many were sold, and the money they made, each sum wrapped in ISNULL so that a part with no sales shows 0.

**2 Every part, with its matching sales**

Lines 4 to 8

LEFT JOIN keeps every part. The three tests after ON pick the sales that count: the right part, still marked Sold, and sold between the dates.

**3 One row per part, best seller first**

Lines 9 to 10

GROUP BY squashes each part's sales into one row, and ORDER BY puts the highest number sold at the top, with names breaking a tie.

**IF IT GOES WRONG** Six rows instead of twelve

The parts with no sales have been thrown away. Check that the join reads LEFT JOIN, and that all three tests sit after ON and not in a WHERE.

**IF IT GOES WRONG** 'Column 'Part.Name' is invalid in the select list'

The GROUP BY is missing p.Name. Every column in the SELECT that is not inside a SUM must be listed in the GROUP BY.

**IF IT GOES WRONG** The Sold column is empty for some parts

The ISNULL round the SUM is missing. A part with no sales has nothing to add up, and ISNULL turns that empty answer into 0.

# Read what an exception is, then type the one class this chapter writes from nothing.

A report can be asked one impossible thing: a From date after the To date. The form could check the two boxes itself, but the check belongs with the data code, because every report needs it and the form should not have to remember. So the data class throws an exception. An exception is an object that a method throws when it cannot carry on, carrying a sentence that says why. The method that threw it stops at once, and the sentence travels up to whoever called it, until a catch takes it. The one class you write from nothing in Visual Studio 2019 is that exception.

The first line of the class ends with a colon and the word Exception. The colon means is a kind of: InvalidDateRangeException is a kind of Exception, and gets everything an Exception already has, such as its Message. The constructor passes its sentence to base, which is the Exception underneath, so the sentence becomes the Message. The form in the next stage catches this class by its name, so nothing else is caught by mistake.

**Type this one** **InvalidDateRangeException.cs: the whole class** C#

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
1  using System;
2
3  namespace BikeShop
4  {
5      // The one mistake a report can be asked to make: a From date after the
6      // To date. It is a class of its own so the report form can catch this
7      // mistake by name and nothing else.
8      public class InvalidDateRangeException : Exception
9      {
10         public DateTime FromDate { get; set; }
11         public DateTime ToDate { get; set; }
12
13         public InvalidDateRangeException(DateTime fromDate, DateTime toDate)
14             : base("The From date (" + fromDate.ToString("d MMM yyyy")
15                   + ") is after the To date (" + toDate.ToString("d MMM yyyy")
16                   + "). Choose a From date on or before the To date.")
17         {
18             FromDate = fromDate;
19             ToDate = toDate;
20         }
21     }
22 }
```

**1 The one using line**

Line 1

Exception and DateTime both live in System, so that is the only using line this file needs.

**2 The namespace: the project itself, not a folder**

Lines 3 to 4

This file sits at the top of the project, beside Program.cs, so its namespace is BikeShop with no folder name after it. The form and the data class can both reach it.

**3 The class, and what the colon means**

Lines 8 to 9

This is a class of its own, so that the form can catch this mistake by name. The colon and Exception mean this class is a kind of Exception: it can be thrown, caught, and asked for its Message.

**4 Two properties that remember the dates**

Lines 10 to 11

The two dates that caused the problem are kept, so that whoever catches the exception can read them as dates, not only as words in a sentence.

**5 The constructor, and base**

Lines 13 to 17

The constructor takes the two dates. The line that starts with a colon and base hands one sentence to the Exception underneath, and that sentence becomes the Message. ToString with d MMM yyyy writes a date

as 31 Dec 2026, so the sentence reads like English.

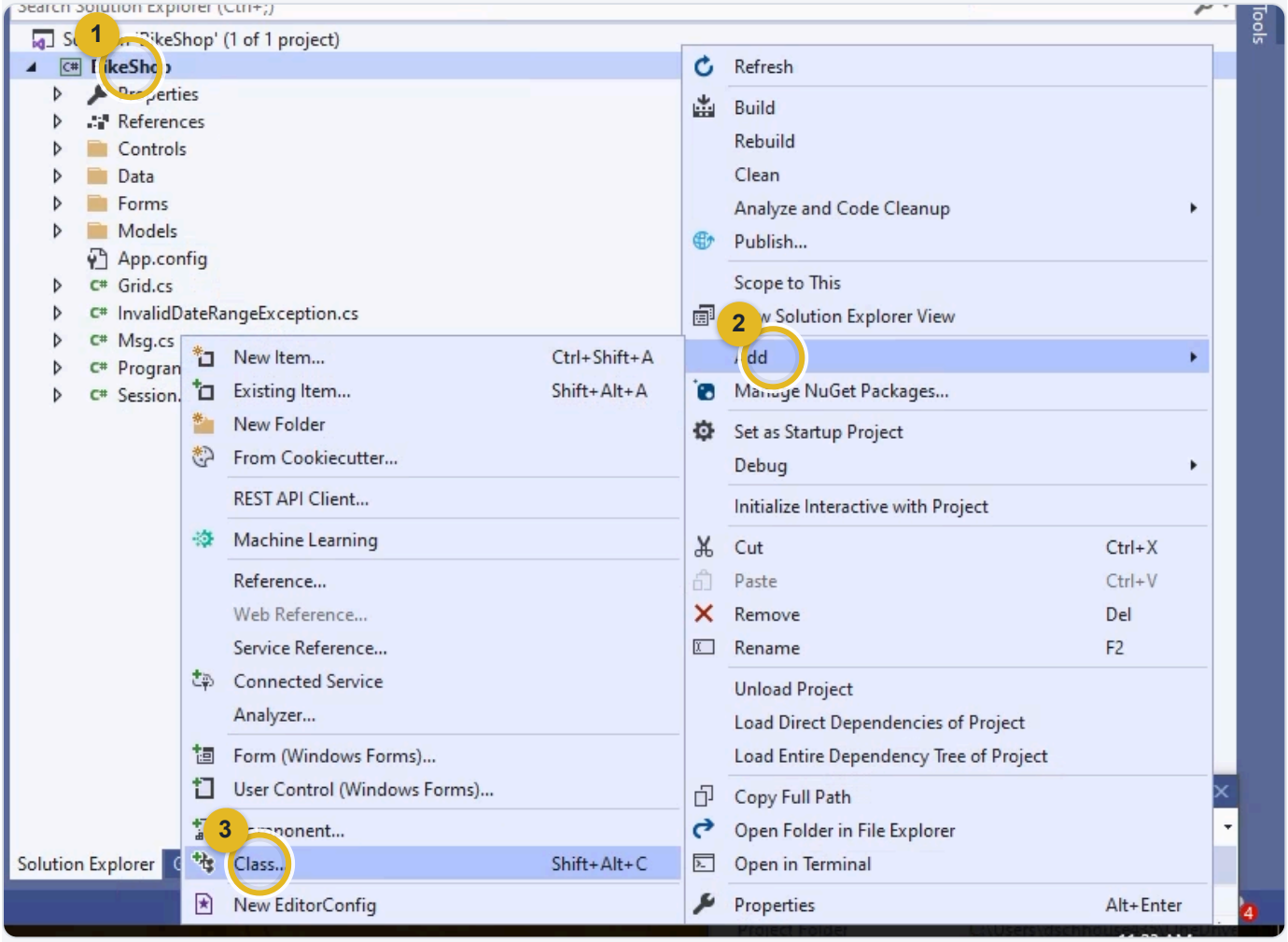
**6 Keep the two dates**

Lines 18 to 22

The two dates are copied into the properties. The closing braces end the constructor, the class and the namespace.

**Why you type it** This is the only class in the guide that you type from nothing. It is a short file, and every part of it is a shape you will use again: a class that is a kind of another class, two properties, and a constructor that hands a sentence to base.

In Solution Explorer, right-click the BikeShop project line, the one above the Data, Forms and Models folders, point at Add, and click Class….

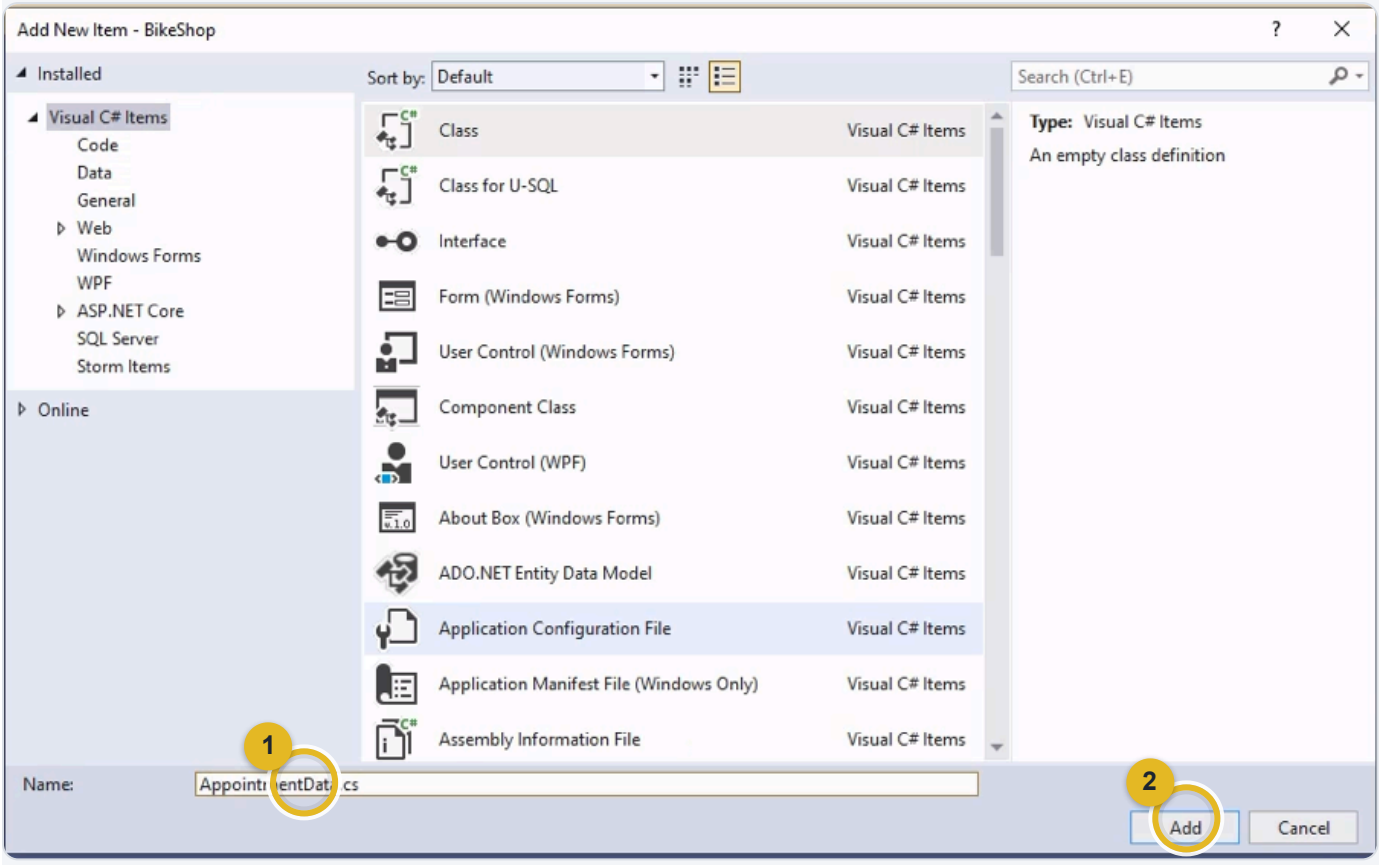



Adding a class to the project itself, not to a folder. The exception class sits at the top, beside Program.cs and Session.cs.

1. Right-click the BikeShop project line, the one above the Data, Forms and Models folders.
2. Point at Add.
3. Click Class…

> **You should see** The Add New Item window opens with Class selected.

In the Name box at the bottom, type InvalidDateRangeException.cs and press Add.




Name the new class before you press Add.

1. Type the name in the Name box, with .cs on the end.

2. Press Add.

> **You should see** A new file opens with an empty class in it, and its namespace line reads namespace BikeShop.

Click in the new file, press Ctrl and A together to select everything, and type the class above in its place, line by line, exactly as the block shows it.

> **You should see** Your file matches the block, and there is no red wavy line anywhere.

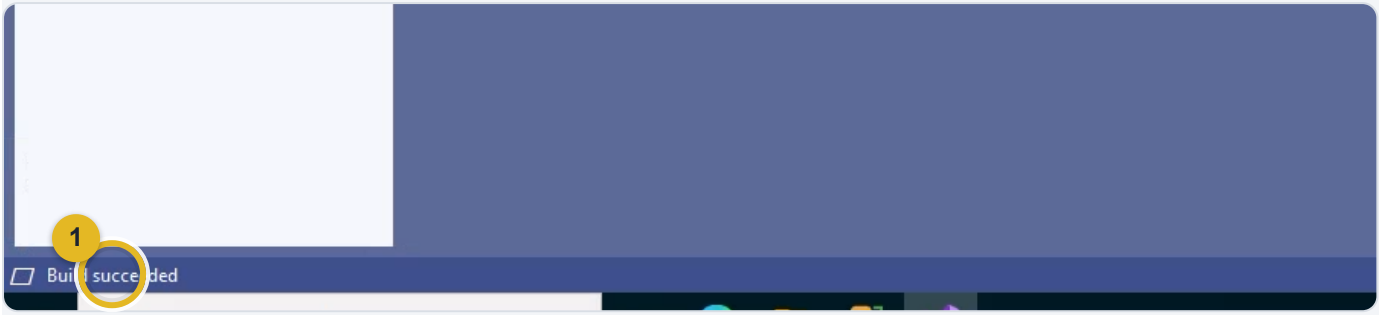
**IF IT GOES WRONG** A red wavy line under Exception

The using System; line at the top is missing. Add it above the namespace line.

**IF IT GOES WRONG** A red wavy line under base

The line that starts with a colon and base must come before the opening brace of the constructor, and the brackets after base must hold one piece of text, built with plus signs.

Press F6 to build.




Build succeeded, at the bottom left. Anything else means an error list to read.

1 Build succeeded. Press F6 to build at any time.

**You should see** Build succeeded, at the bottom left.

**IF IT GOES WRONG** 'The type or namespace name Exception could not be found'

The using System; line at the top is missing, or is spelled differently. Add it above the namespace line.

**IF IT GOES WRONG** The namespace line reads BikeShop.Data or BikeShop.Forms

The class was added inside a folder. Delete it, and add it again by right-clicking the project line itself, the one above the folders.

# Add the report data class.

One data class holds the five reports. Every method does the same two things: it runs CheckRange, and then it asks SQL Server for the table. CheckRange is where the exception is thrown. The word throw starts the exception on its way: the method stops at that line, and whoever called it gets the exception instead of a table. The dates go in with .Date on the end, which throws away the clock part, so that a report asked at four in the afternoon still includes the whole of the To day.

Copy this one **Data/ReportData.cs: the five reports and the one check** C#

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
using System;
using System.Data;

namespace BikeShop.Data
{
    public static class ReportData
    {
        // Every report starts with the same check. A From date after the To
        // date is refused before SQL Server is asked anything.
        private static void CheckRange(DateTime fromDate, DateTime toDate)
        {
            if (fromDate.Date > toDate.Date)
            {
                throw new InvalidDateRangeException(fromDate.Date, toDate.Date);
            }
        }

        public static DataTable MechanicWorkload(DateTime fromDate,
            DateTime toDate)
        {
            CheckRange(fromDate, toDate);
            return Db.GetTable("usp_Report_MechanicWorkload",
                Db.P("@FromDate", fromDate.Date), Db.P("@ToDate", toDate.Date));
        }

        public static DataTable Income(DateTime fromDate, DateTime toDate)
        {
            CheckRange(fromDate, toDate);
            return Db.GetTable("usp_Report_Income",
                Db.P("@FromDate", fromDate.Date), Db.P("@ToDate", toDate.Date));
        }

        public static DataTable Activity(DateTime fromDate, DateTime toDate)
        {
            CheckRange(fromDate, toDate);
            return Db.GetTable("usp_Report_Activity",
                Db.P("@FromDate", fromDate.Date), Db.P("@ToDate", toDate.Date));
        }

        public static DataTable PopularServices(DateTime fromDate,
            DateTime toDate)
        {
            CheckRange(fromDate, toDate);
            return Db.GetTable("usp_Report_PopularServices",
                Db.P("@FromDate", fromDate.Date), Db.P("@ToDate", toDate.Date));
        }

        public static DataTable PopularParts(DateTime fromDate,
            DateTime toDate)
        {
            CheckRange(fromDate, toDate);
            return Db.GetTable("usp_Report_PopularParts",
                Db.P("@FromDate", fromDate.Date), Db.P("@ToDate", toDate.Date));
```

```
54          }
55      }
56  }
```

**1 The usings**

Lines 1 to 2

System gives the file DateTime, and System.Data gives it DataTable, the table of rows that each report hands back.

**2 The class**

Lines 4 to 7

This is a static class in the Data folder, like the others: you write ReportData.Activity(…), and never make a copy of the class.

**3 CheckRange: the one mistake, caught before SQL Server is asked**

Lines 10 to 16

private means only this class can call it. If the From date is after the To date, a new InvalidDateRangeException is made with the two dates and thrown. The method stops there, and so does the method that called it. If the dates are in order, nothing happens, and the report goes ahead.

**4 MechanicWorkload: check, then ask**

Lines 18 to 24

The method runs CheckRange first. Then Db.GetTable runs the stored procedure with the two dates and hands back a DataTable, which the form puts straight into its grid. It is GetTable and not Run, because a report has no @Message.

**5 Income**

Lines 26 to 31

This method uses the same two lines as MechanicWorkload, and it calls usp_Report_Income.

**6 Activity**

Lines 33 to 38

This method uses the same two lines as MechanicWorkload, and it calls usp_Report_Activity.

**7 PopularServices**

Lines 40 to 46

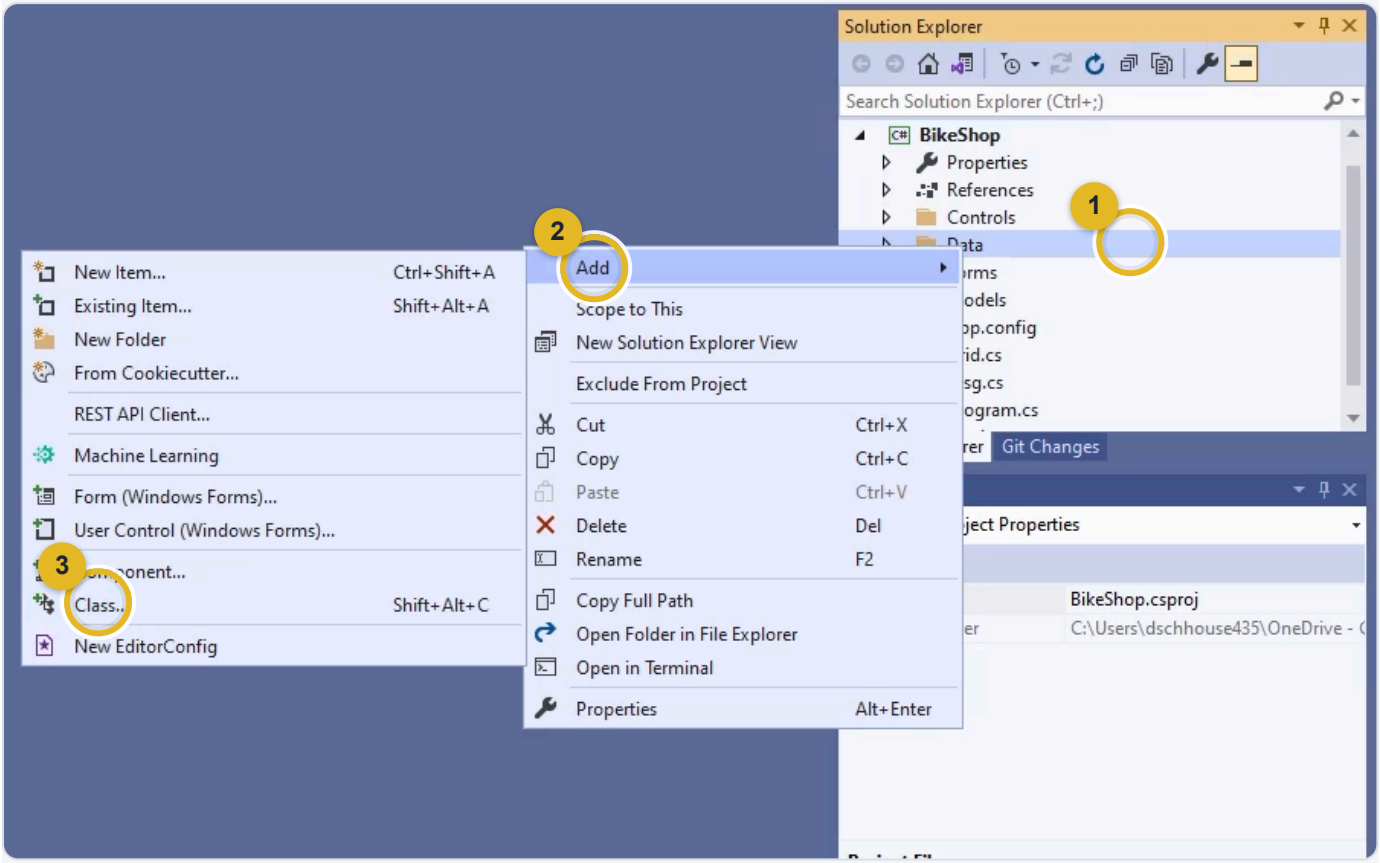
This method uses the same two lines as MechanicWorkload, and it calls usp_Report_PopularServices.

**8 PopularParts, and the end of the class**

Lines 48 to 56

This method uses the same two lines as the others, and it calls usp_Report_PopularParts, the stored procedure you wrote yourself. The two closing braces end the class and the namespace.

In Solution Explorer, right-click the Data folder, point at Add, and click Class….

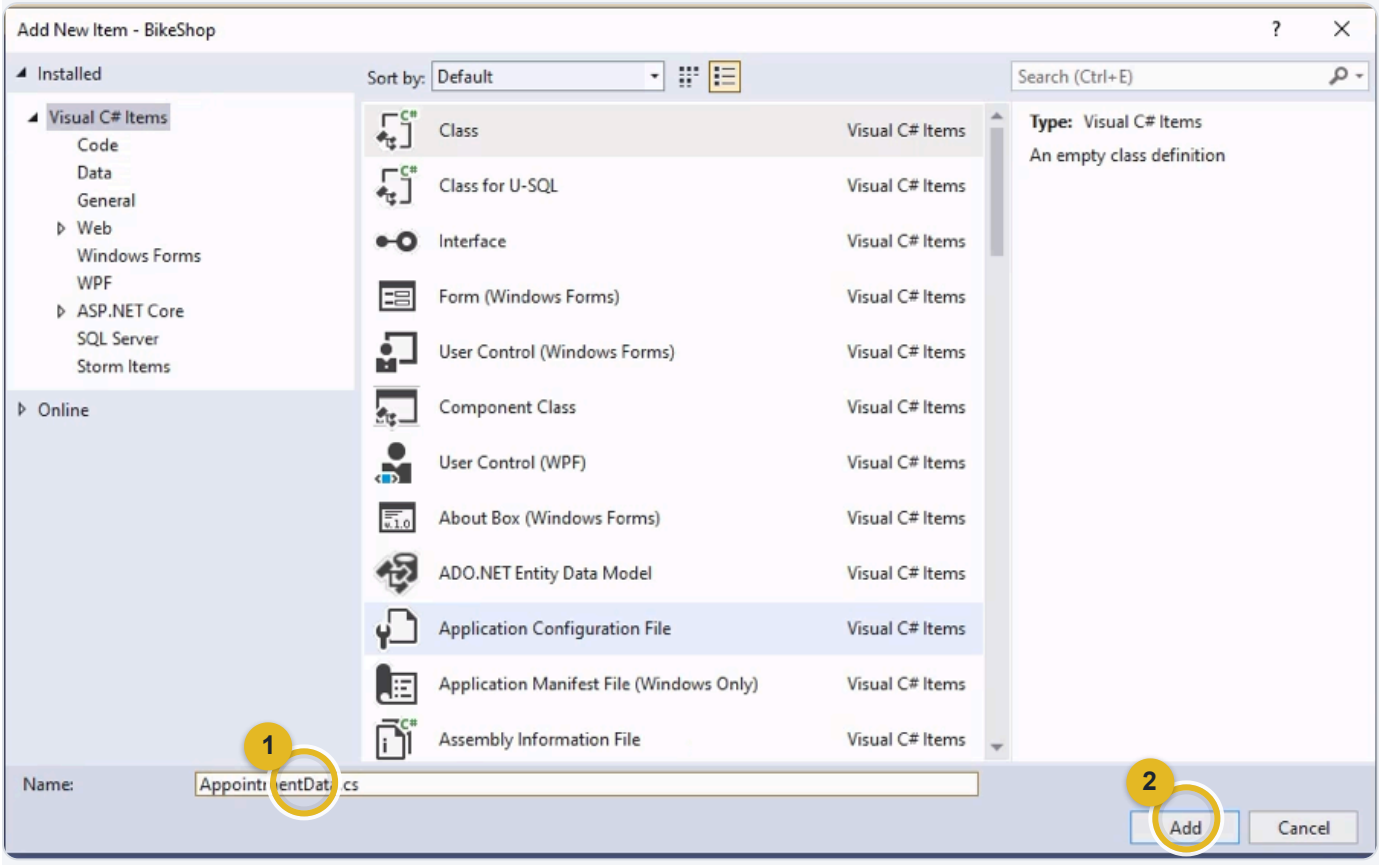



Adding a class to the Data folder. This picture was taken in the finished project, so Solution Explorer lists files you have not made yet.

1. Right-click the Data folder in Solution Explorer.
2. Point at Add.
3. Click Class…

**You should see** The Add New Item window opens with Class selected.

In the Name box at the bottom, type ReportData.cs and press Add.




Name the new class before you press Add.

1. Type the name in the Name box, with .cs on the end.

2. Press Add.

> **You should see** A new file opens with an empty class in it.

Press the Copy button under this step.

> **You should see** The button reads Copied for a moment.

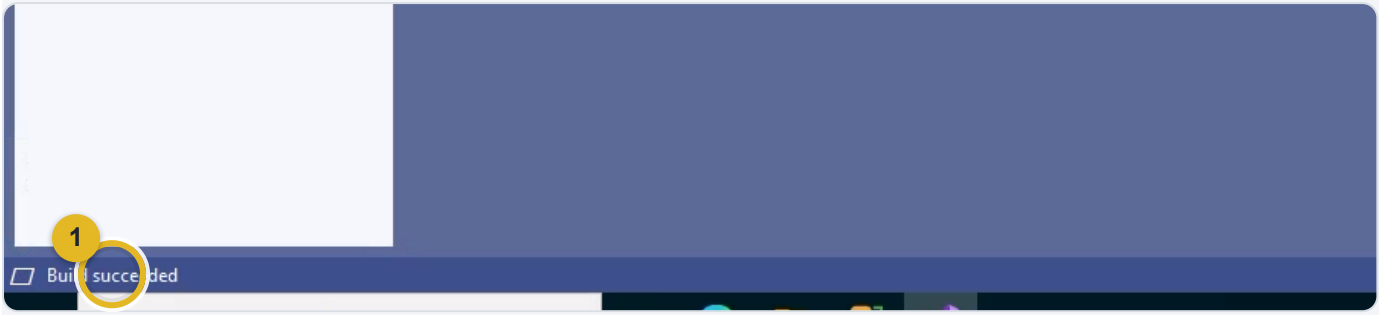
Click in the new file and press Ctrl and A together to select everything.

> **You should see** Everything in the new file is highlighted.

Press Ctrl and V together to paste the copied class over it.

> **You should see** The file holds the ReportData class, starting with the using lines.

Press F6 to build the project.




Build succeeded, at the bottom left. Anything else means an error list to read.

1 Build succeeded. Press F6 to build at any time.

**You should see** Build succeeded, at the bottom left.

**IF IT GOES WRONG** 'The type or namespace name InvalidDateRangeException could not be found'

The class from the last screen is missing, is spelled differently, or has the wrong namespace. Open it and check that its namespace line reads namespace BikeShop.

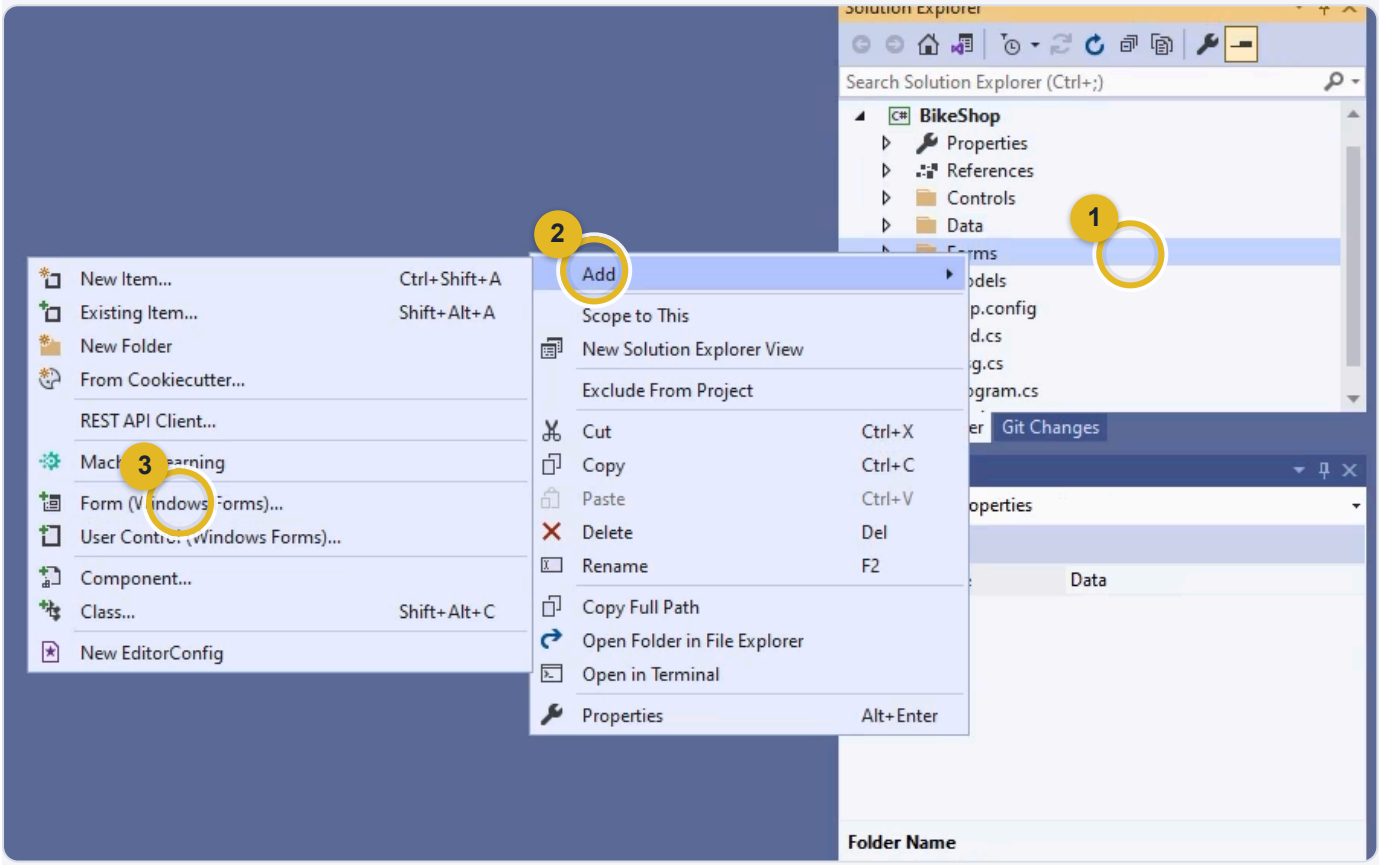
**IF IT GOES WRONG** 'The name Db does not exist'

Your helper class from chapter 5 is called something else, or is in a different namespace. Match the name you used.

# Add the report form and set its controls.

One form shows all five reports. Along the top: a list of the five report names, a From date, a To date and a Run button. Under them, a grid, a label for the one-line summary, and a Copy to clipboard button that is switched off until a report has rows. Add it in Visual Studio 2019 the same way as the forms in chapter 8.

Right-click the Forms folder in Solution Explorer, point at Add, and click Form (Windows Forms)…




Adding a form to the Forms folder. Taken in the finished project, so Solution Explorer lists more files than yours.

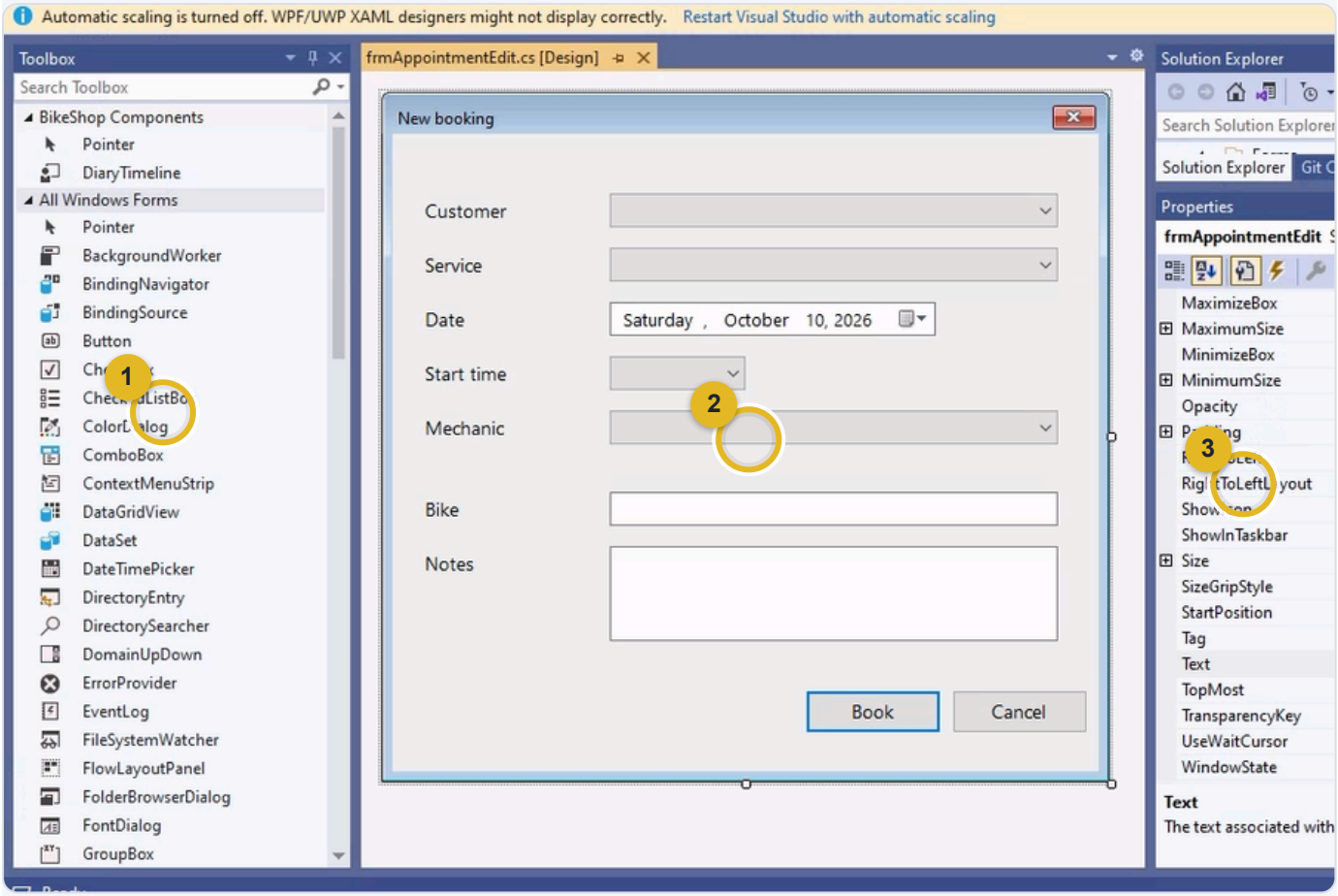
1. Right-click the Forms folder.
2. Point at Add.
3. Click Form (Windows Forms)…, type the name, press Add.

> **You should see** The Add New Item window opens with Form (Windows Forms) selected.

Type frmReports in the Name box and press Add.

> **You should see** An empty form opens in the designer.

Open the Toolbox (View, then Toolbox) and the Properties window (View, then Properties Window), if they are not already open.




The three parts of the designer: the Toolbox you drag from, the form you drop on, the Properties window where you name and set each control.

1. The Toolbox. Drag a control from here on to the form.
2. The form. Drop the control where the picture shows it.
3. The Properties window. Set Name first, then the rest of the row.

> **You should see** The Toolbox is on the left, the form in the middle, the Properties window on the right.

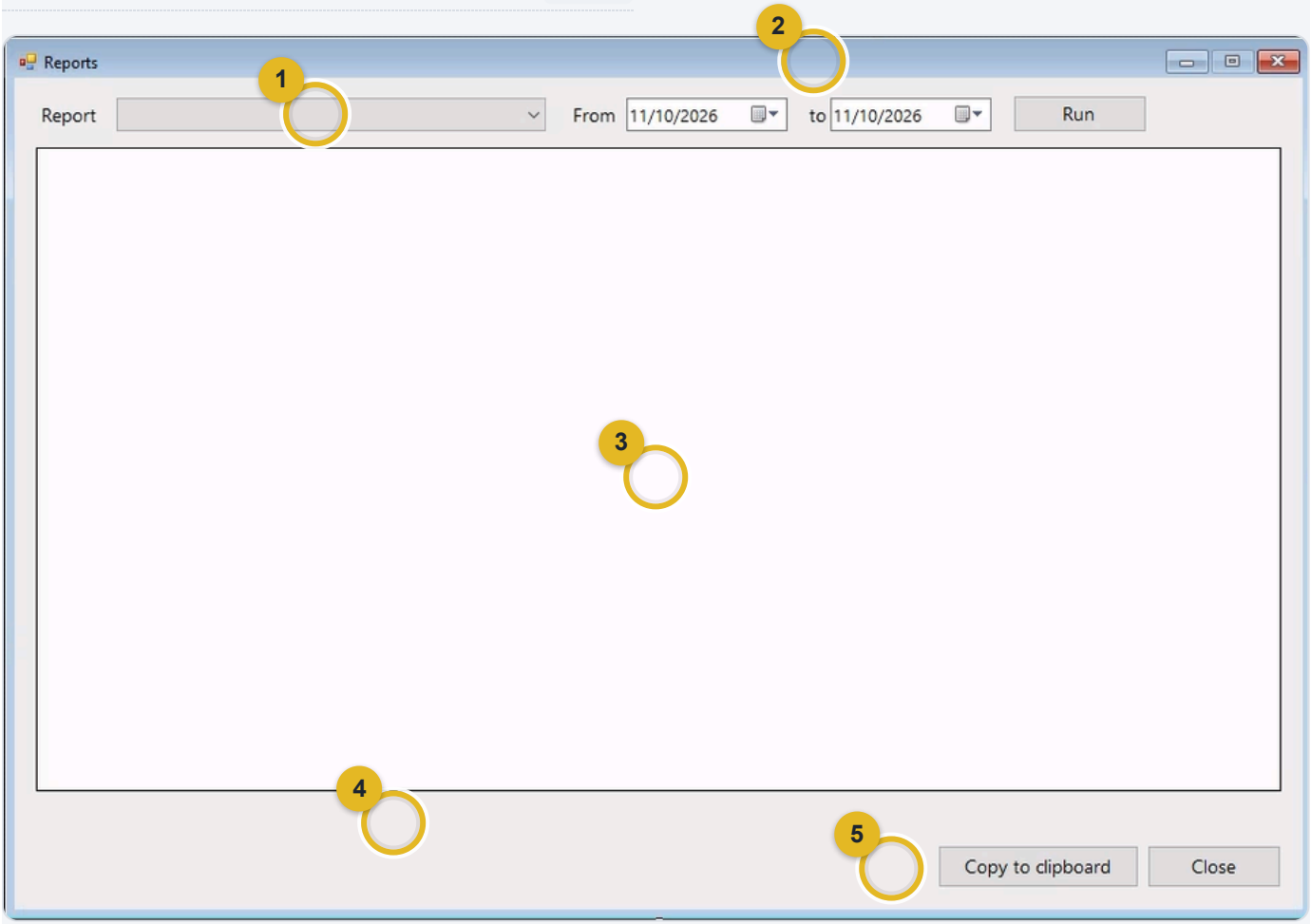
Click the form itself first and set the form's own properties. Then work down the table: drag the control from the Toolbox, drop it on the form, set its Name in the Properties window, and set the other settings in the row. Where the Then column says double-click, double-clicking the control makes its event method for you.

For the report list, the Items row needs five lines of text. Click on cboReport, find Items in the Properties window, click the three dots beside it, and type one report name per line in the window that opens, exactly as the row shows them.

Five reports across several tables for a date range, each with a one-line summary and a Copy button for the user guide and test evidence.

**Click the form itself, then set these in the Properties window**

| | | | |
|---|---|---|---|
| Text | `Reports` | Size | `960, 620` |
| FormBorderStyle | `FixedSingle` | StartPosition | `CenterParent` |
| MaximizeBox | `False` | | |




frmReports in the designer. Yours should look like this before you add any code.

1. The report list, cboReport, with its five report names.
2. The two date boxes, dtpFrom and dtpTo, with Run beside them.
3. The grid, dgvReport.
4. The label under the grid, lblTotal, where the one-line summary goes.
5. Copy to clipboard, switched off until a report has rows, and Close.

| DRAG THIS FROM THE TOOLBOX | NAME IT (THE NAME PROPERTY) | SET THESE IN THE PROPERTIES WINDOW | THEN |
|---|---|---|---|
| Label | `lblReport` | `Text = Report` | |
| ComboBox | `cboReport` | `DropDownStyle = DropDownList`<br>`Items = Mechanic workload / Income by day and source / How many bookings, hires, sales and orders / Most popular services / Most popular parts` | |
| Label | `lblFrom` | `Text = From` | |
| DateTimePicker | `dtpFrom` | `CustomFormat = dd/MM/yyyy`<br>`Format = Custom` | |
| Label | `lblTo` | `Text = to` | |
| DateTimePicker | `dtpTo` | `CustomFormat = dd/MM/yyyy`<br>`Format = Custom` | |
| Button | `btnRun` | `Text = Run` | Double-click it. Visual Studio makes `btnRun_Click` for you; the code goes inside it. |
| DataGridView | `dgvReport` | `the grid settings (chapter 6)` | |
| Label | `lblTotal` | nothing else<br>"Services listed: 12. Most popular: Full service, booked 5 times." | |
| Button | `btnCopy` | `Text = Copy to clipboard`<br>`Enabled = False`<br>Copies the grid with its headings, ready for Word or Excel. | Double-click it. Visual Studio makes `btnCopy_Click` for you; the code goes inside it. |
| Button | `btnClose` | `Text = Close` | Double-click it. Visual Studio makes `btnClose_Click` for you; the code goes inside it. |

# Put the code behind the report form.

Read the file, then put it in. The Run button asks the data class for one of the five tables, chosen by which report is picked, inside a try. If the data class throws the date exception, the catch underneath shows its sentence, puts the cursor back in the From box, and stops. If not, the grid is filled and ShowTotal writes the one-line summary. Copy to clipboard copies the grid with its headings, ready for your user guide and your test evidence.

Copy this one **Forms/frmReports.cs: the whole file** C#

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```
using System;
using System.Collections.Generic;
using System.Data;
using System.Drawing;
using System.Windows.Forms;
using BikeShop.Data;
using BikeShop.Models;

namespace BikeShop.Forms
{
    public partial class frmReports : Form
    {
        public frmReports()
        {
            InitializeComponent();
        }

        private void frmReports_Load(object sender, EventArgs e)
        {
            cboReport.SelectedIndex = 0;
            dtpFrom.Value = DateTime.Today.AddDays(-30);
            dtpTo.Value = DateTime.Today;
            lblTotal.Text = "";
        }

        #region Run
        private void btnRun_Click(object sender, EventArgs e)
        {
            DataTable table;
            try
            {
                if (cboReport.SelectedIndex == 0)
                {
                    table = ReportData.MechanicWorkload(dtpFrom.Value,
                        dtpTo.Value);
                }
                else if (cboReport.SelectedIndex == 1)
                {
                    table = ReportData.Income(dtpFrom.Value, dtpTo.Value);
                }
                else if (cboReport.SelectedIndex == 2)
                {
                    table = ReportData.Activity(dtpFrom.Value, dtpTo.Value);
                }
                else if (cboReport.SelectedIndex == 3)
                {
                    table = ReportData.PopularServices(dtpFrom.Value,
                        dtpTo.Value);
                }
                else
                {
                    table = ReportData.PopularParts(dtpFrom.Value,
                        dtpTo.Value);
```

```
                }
            }
            catch (InvalidDateRangeException ex)
            {
                // The From date is after the To date: say so, leave the grid
                // as it was, and put the cursor back in From.
                Msg.Problem(ex.Message);
                dtpFrom.Focus();
                return;
            }

            dgvReport.DataSource = table;
            Grid.Tidy(dgvReport);
            ShowTotal();
            btnCopy.Enabled = dgvReport.Rows.Count > 0;
        }

        // One plain sentence under the grid, so the answer to the report's
        // question is on screen, not left to a calculator.
        private void ShowTotal()
        {
            int n = dgvReport.Rows.Count;
            if (n == 0)
            {
                lblTotal.Text = "Nothing in those dates.";
            }
            else if (cboReport.SelectedIndex == 0)
            {
                lblTotal.Text = "Mechanics listed: " + n
                    + ". Workshop income in these dates: £"
                    + SumColumn("Income").ToString("#,##0.00") + ".";
            }
            else if (cboReport.SelectedIndex == 1)
            {
                lblTotal.Text = "Lines listed: " + n
                    + ". Total income in these dates: £"
                    + SumColumn("Income").ToString("#,##0.00") + ".";
            }
            else if (cboReport.SelectedIndex == 2)
            {
                lblTotal.Text =
                    "Each line counts one kind of thing in these dates.";
            }
            else if (cboReport.SelectedIndex == 3)
            {
                DataGridViewRow top = dgvReport.Rows[0];
                int booked = Convert.ToInt32(top.Cells["Bookings"].Value);
                if (booked == 0)
                {
                    lblTotal.Text = "Services listed: " + n
                        + ". None were booked in these dates.";
                }
                else
                {
                    lblTotal.Text = "Services listed: " + n
                        + ". Most popular: " + top.Cells["Service"].Value
                        + ", booked " + Times(booked) + ".";
                }
```

```
            }
            else
            {
                DataGridViewRow top = dgvReport.Rows[0];
                int sold = Convert.ToInt32(top.Cells["Sold"].Value);
                if (sold == 0)
                {
                    lblTotal.Text = "Parts listed: " + n
                        + ". None were sold in these dates.";
                }
                else
                {
                    lblTotal.Text = "Parts listed: " + n
                        + ". Best seller: " + top.Cells["Part"].Value
                        + ", " + sold + " sold.";
                }
            }
        }

        // Adds up one column of the grid.
        private decimal SumColumn(string column)
        {
            decimal total = 0;
            foreach (DataGridViewRow row in dgvReport.Rows)
            {
                total = total + Convert.ToDecimal(row.Cells[column].Value);
            }
            return total;
        }

        // 1 becomes "once", 2 becomes "twice", anything else "5 times".
        private string Times(int n)
        {
            if (n == 1)
            {
                return "once";
            }
            else if (n == 2)
            {
                return "twice";
            }
            else
            {
                return n + " times";
            }
        }
        #endregion

        #region Copy
        // Copies the grid with its headers, ready to paste into Word or Excel
        // for the user guide or the test evidence.
        private void btnCopy_Click(object sender, EventArgs e)
        {
            dgvReport.ClipboardCopyMode =
                DataGridViewClipboardCopyMode.EnableAlwaysIncludeHeaderText;
            dgvReport.SelectAll();
            DataObject content = dgvReport.GetClipboardContent();
            if (content ≠ null)
```

```
170             {
171                 Clipboard.SetDataObject(content);
172             }
173             dgvReport.ClearSelection();
174             Msg.Info("Copied. Paste it into Word or Excel.");
175         }
176         #endregion
177
178         private void btnClose_Click(object sender, EventArgs e)
179         {
180             Close();
181         }
182     }
183 }
```

**1 The usings**

Lines 1 to 7

These lines bring in the tools this form needs: Windows Forms for the window itself, System.Data for the table that fills the grid, System.Drawing for colours, and your own Data and Models folders.

**2 The form's class and its constructor**

Lines 9 to 16

partial means the designer keeps the other half of this class, the controls you dragged, in its own file. The constructor builds those controls.

**3 When the form loads**

Lines 18 to 24

The first report is picked in the list, the dates are set to the last thirty days, and the summary label starts empty.

**4 The Run button starts**

Lines 26 to 29

The Run button's method starts by making one box, called table, to hold whichever report comes back. The #region line only folds the code up in Visual Studio 2019.

**5 try, and the first choice**

Lines 30 to 36

Everything inside try is watched. If any line in it throws an exception, the code jumps straight to catch. SelectedIndex is the position of the picked report in the list, counting from 0, so 0 is Mechanic workload.

**6 The other four choices**

Lines 37 to 55

There is one else if for each of the next three reports, in the order of the list: 1 is income, 2 is activity and 3 is popular services. A plain else covers popular parts. Each one asks the data class for its table with the two dates from the boxes.

**7 catch: the one mistake, by name**

Lines 56 to 63

This catch takes only InvalidDateRangeException, so a different problem is not hidden. Msg.Problem shows the sentence that the exception carries. The cursor goes back to the From box, and return leaves the method, so the grid is left as it was.

**8 Show the answer**

Lines 65 to 69

The table goes into the grid, and Grid.Tidy from chapter 6 sizes the columns. ShowTotal writes the sentence under the grid. Copy to clipboard is switched on only when there is at least one row to copy.

**9 ShowTotal: no rows**

Lines 73 to 79

n is the number of rows in the grid. When it is 0, every report shows the same sentence, because an empty grid with no sentence looks like a fault.

**10 Mechanic workload, and income: a count and a sum**

Lines 80 to 91

The first two reports both have an Income column, so each sentence gives the number of rows and the column added up by SumColumn. ToString with #,##0.00 writes money with two decimal places and a comma for thousands.

**11 Activity: one fixed sentence**

Lines 92 to 96

The activity report is five counts of different things, so there is nothing sensible to add up. The sentence says what the grid is.

**12 Services: the top row is the most popular**

Lines 97 to 112

The report is sorted most booked first, so row 0 is the answer. Its Bookings cell is read as a number. If it is 0, nothing was booked at all, and the sentence says so. Otherwise the sentence names the service and how many times, with Times turning 1 into once and 2 into twice.

**13 Parts: the same shape**

Lines 113 to 129

The same idea for parts: the top row's Sold cell decides between None were sold and Best seller, and the sentence names the part.

**14 SumColumn: add up one column**

Lines 132 to 140

This method loops over every row of the grid, reading one named cell as a decimal and adding it to a running total. The column name is passed in, so the same method serves both reports that need it.

**15 Times: once, twice, or a number**

Lines 143 to 158

This method gives three ways to say how many times, so the sentence reads like English. The #endregion line closes the fold that #region Run opened.

**16 The Copy button**

Lines 160 to 176

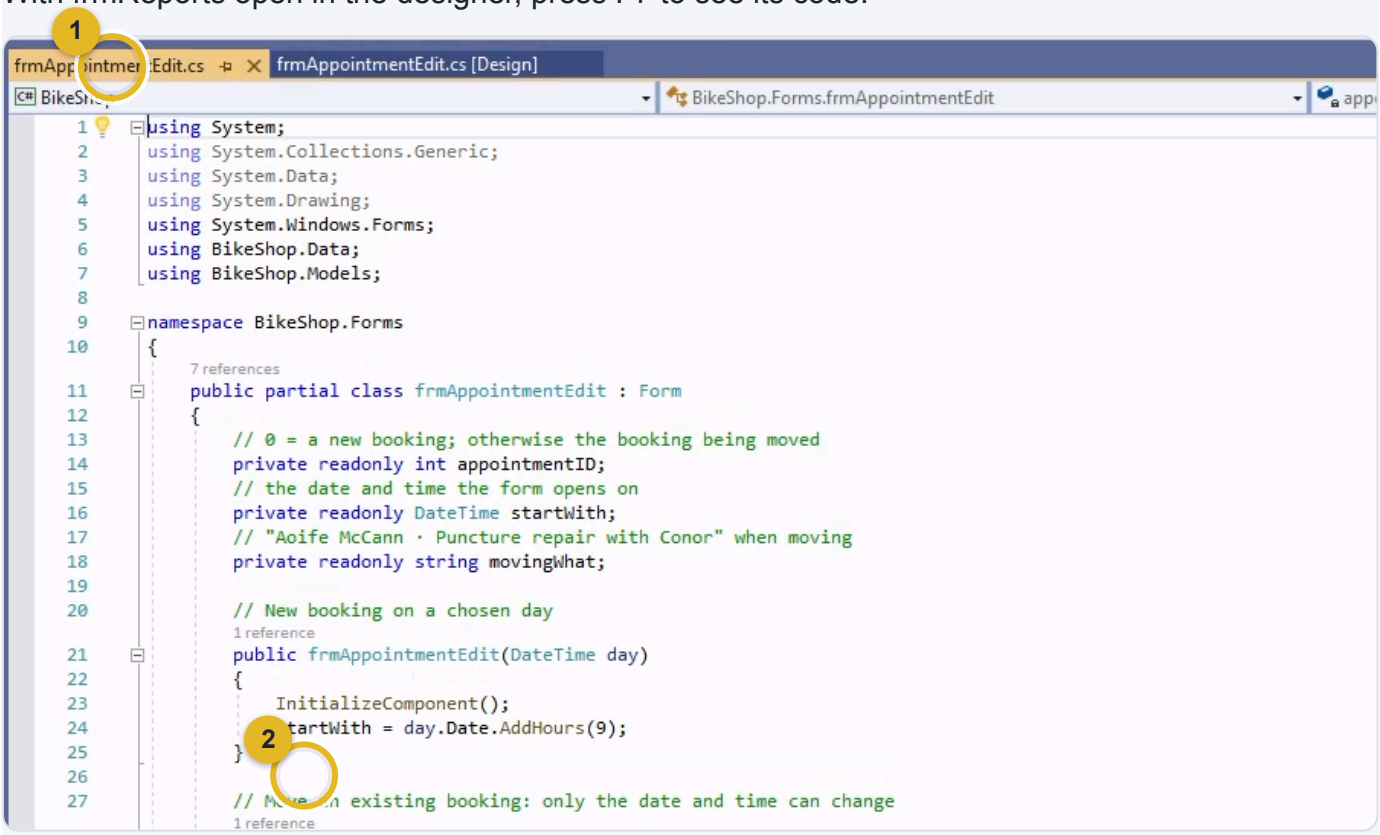
The grid is told to include its headings when copied. Every cell is selected, the selection is turned into clipboard content, and if there is any it goes on the clipboard. The selection is cleared again, and a short message says what to do next.

**17 Close**

Lines 178 to 183

Close shuts the form. The two closing braces end the class and the namespace.

With frmReports open in the designer, press F7 to see its code.




F7 shows the code behind the form that is open in the designer. Shift and F7 go back to the designer.

1. The code tab is in front.

2. Select everything here before you paste over it.

> **You should see** The code tab for the form comes to the front.

Press the Copy button under this step.

> **You should see** The button reads Copied for a moment.

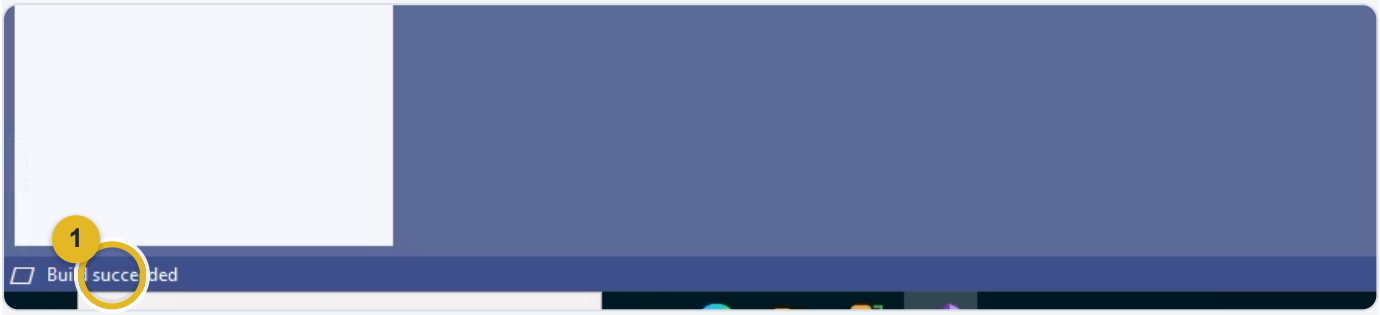
Click in the code tab and press Ctrl and A together to select everything.

> **You should see** Everything in the code tab is highlighted.

Press Ctrl and V together to paste the copied code over it.

> **You should see** The code tab holds the whole file, starting with the using lines.

Press F6 to build the project.




Build succeeded, at the bottom left. Anything else means an error list to read.

1. Build succeeded. Press F6 to build at any time.

> **You should see** Build succeeded, at the bottom left.

**IF IT GOES WRONG** 'frmReports' does not contain a definition for 'button1_Click' (or another old name)

The event was made before the control was given its Name, so the designer still calls the method by its old name. Double-click the error: Visual Studio 2019 opens frmReports.Designer.cs on the line that hooks up the event. At the end of that line, change the old name (such as button1_Click) to the control's Name followed by _Click: btnRun_Click if the line starts with this.btnRun, or frmReports_Load if the line starts with this.Load. Then press F6 to build again.

**IF IT GOES WRONG** 'The name Grid does not exist' or 'The name Msg does not exist'

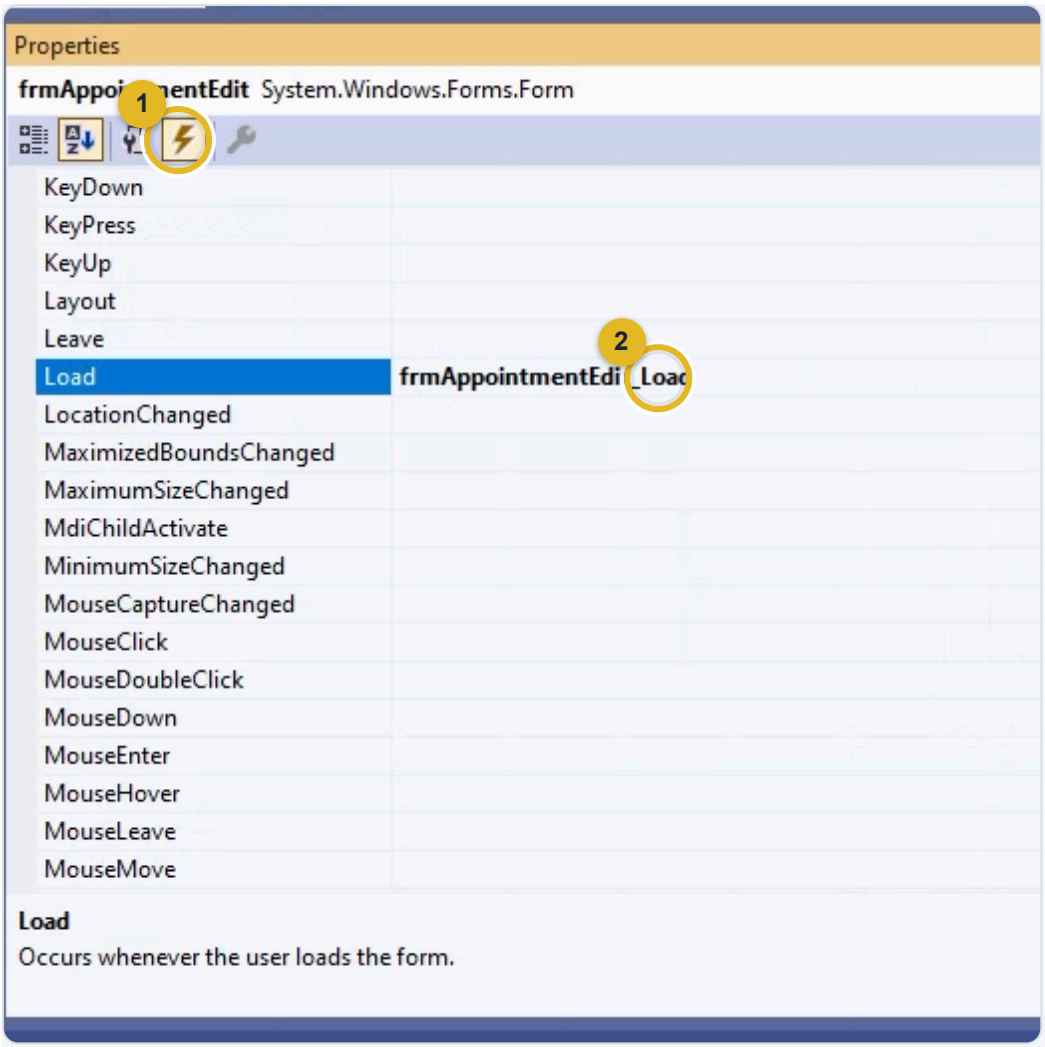
Your grid and message helpers from chapter 6 are called something else, or are in a different namespace. Match the names you used.

**IF IT GOES WRONG** 'The name ReportData does not exist'

The data class from stage 6 is missing or named differently. Check that Data/ReportData.cs is in Solution Explorer and that its class is called ReportData.

**IF IT GOES WRONG** 'The type or namespace name InvalidDateRangeException could not be found'

The exception class from stage 6 has the wrong namespace. Open it and check that its namespace line reads namespace BikeShop.



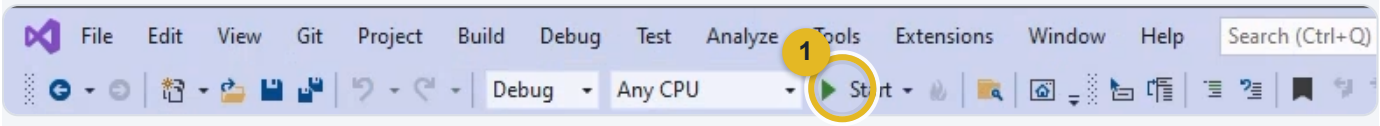

The lightning bolt shows the events, in alphabetical order. Double-click the empty box beside one to make its method.

1. Press the lightning bolt.
2. Double-click the empty box beside the event named in the table. Load is part of the way down, after Leave.

# Run the program and open the reports.

The Reports button has been on your main menu since chapter 5, switched on for a manager only. It opens this form. Sign in as the manager, open the reports, set the dates to the whole of this year, and run the first one.

Press F5 in Visual Studio 2019 to run your program.

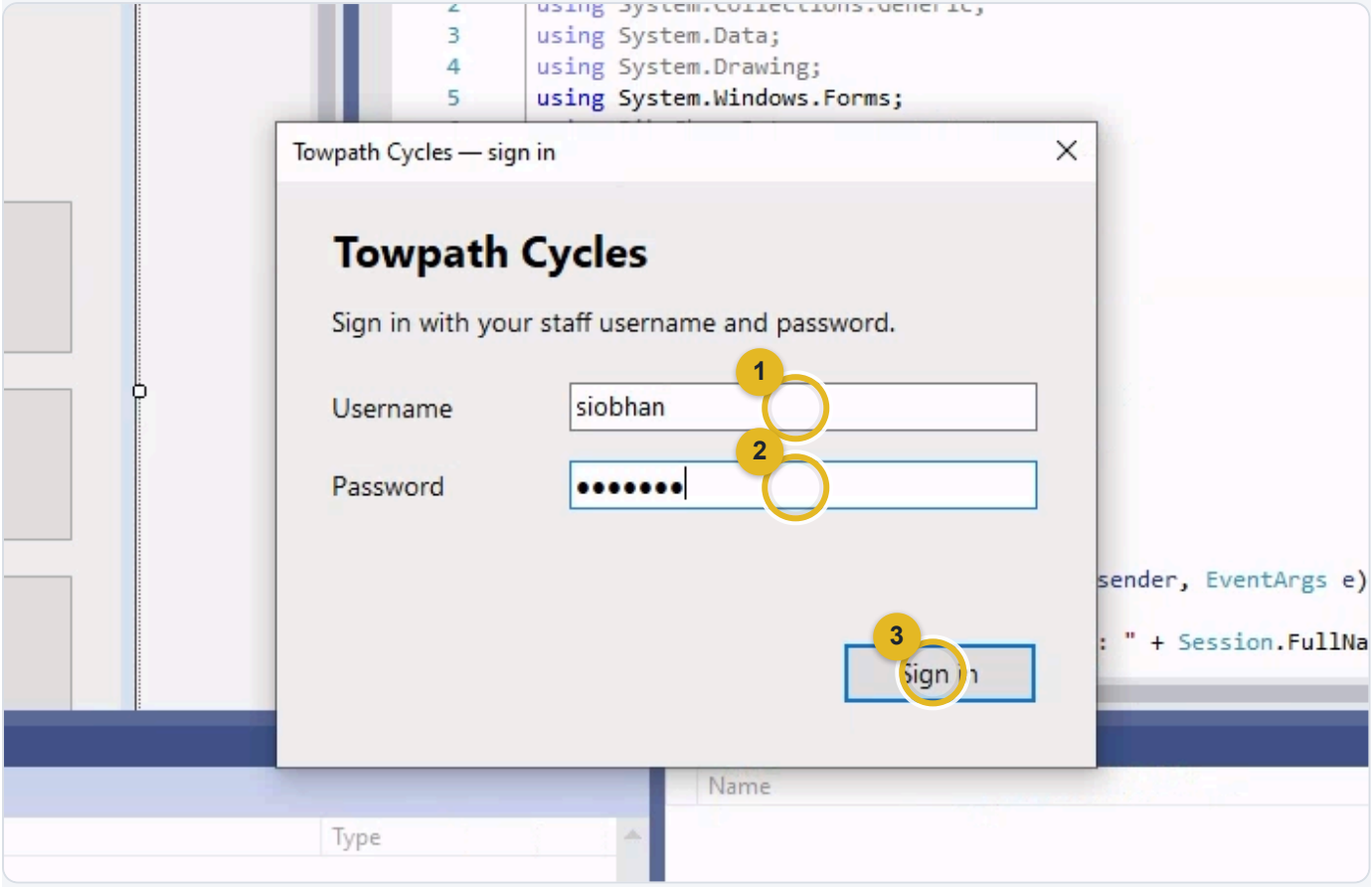



Start runs your program. Close the program window to come back to Visual Studio 2019.

1 Click Start, or press F5.

> **You should see** The sign-in window opens.

Sign in with the username siobhan and the password canal01.

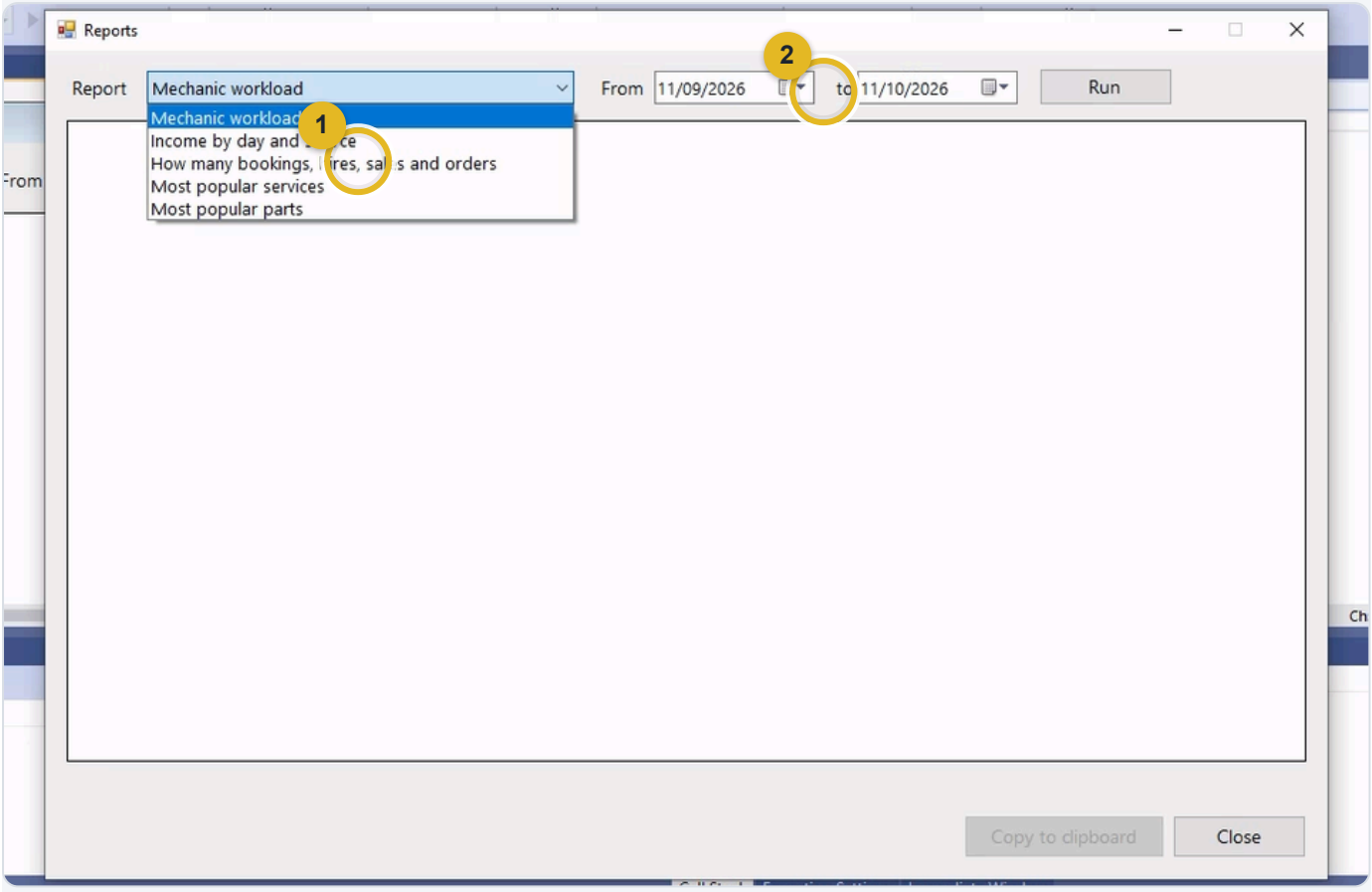



The sign-in window. Siobhán Rooney is the manager, and only a manager can open the reports.

1. Type the username siobhan.
2. Type the password canal01.
3. Click Sign in.

> **You should see** The main menu opens with Siobhán Rooney at the top, and the Reports button is switched on.

Press the Reports button on the main menu.




The five questions the shop asks, in one list. Pick one, choose two dates, and press Run.

1. The five reports. Each one is one stored procedure.

2. The From and To dates. The form starts with the last thirty days.

> **You should see** The Reports screen opens with Mechanic workload picked in the list, the dates set to the last thirty days, an empty grid, and Copy to clipboard greyed out.

Click on the day number in the From box and type 01. Press the right arrow key and type 01. Press the right arrow key again and type 2026.

> **You should see** The From box reads 01/01/2026. The right arrow key moves you from the day to the month, then to the year.

Click on the day number in the To box and type 31. Press the right arrow key and type 12. Press the right arrow key again and type 2026.

> **You should see** The To box reads 31/12/2026.

Press the Run button beside the dates.

**You should see** The grid shows four rows, Conor Walsh at the top with 113.00, and under it the sentence: Mechanics listed: 4. Workshop income in these dates: £233.00. Copy to clipboard is now switched on.

**IF IT GOES WRONG** 'Reports (manager only)' and the button is greyed out

You are signed in as someone who is not a manager. Close the program, run it again, and sign in as siobhan.

**IF IT GOES WRONG** A message that starts: Could not find stored procedure

The stored procedure named in the message is not in your database. Go back to its stage, select its lines in your script, and press F5.

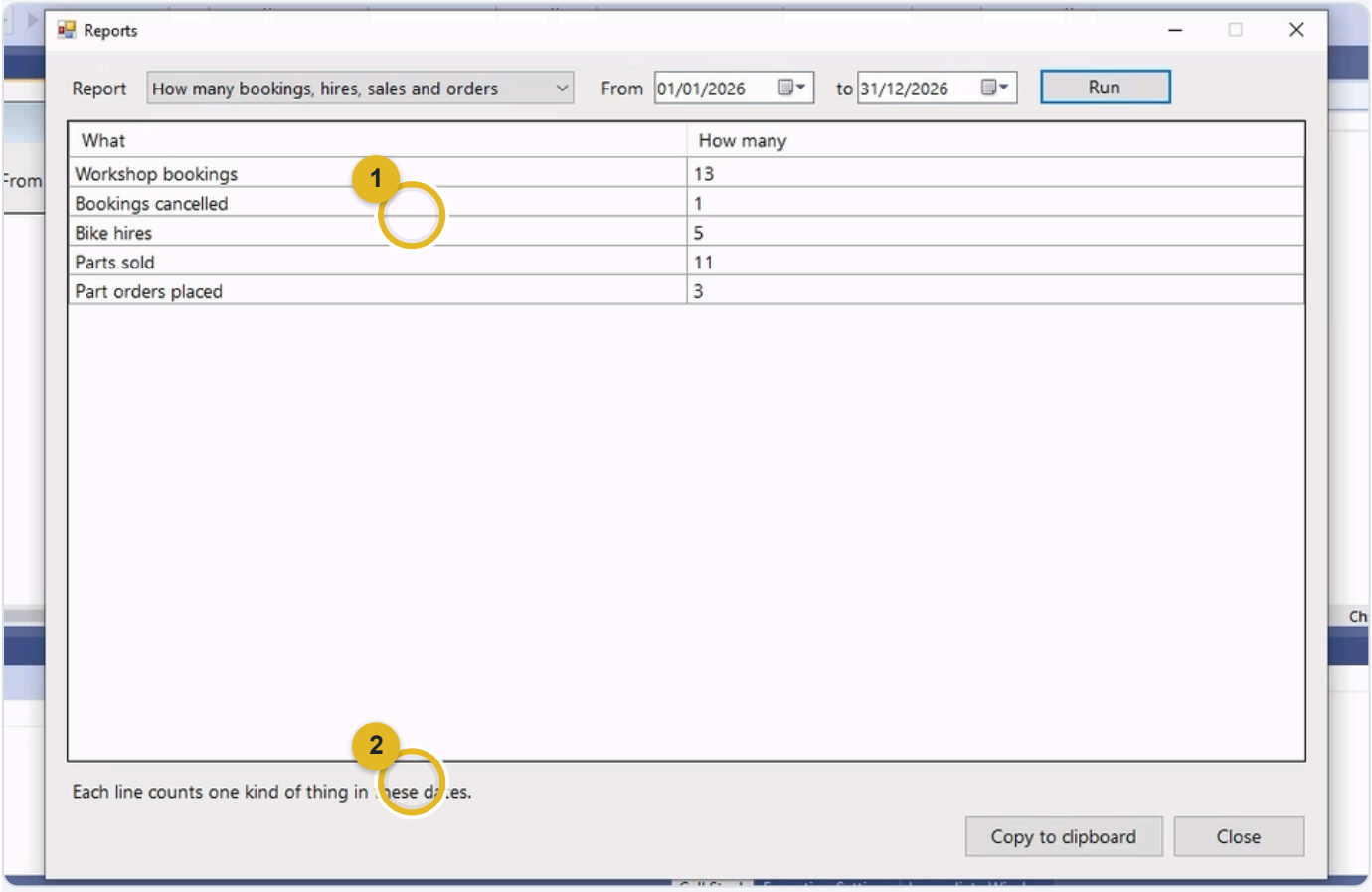
# Run each report once for this year.

With the dates still set to 1 Jan 2026 and 31 Dec 2026, run the other four reports. Each one gets its own sentence under the grid. The numbers below are for the sample data as the whole-script run in stage 2 left it; if you sold or refunded parts since, your parts numbers will differ, and that is right.

Pick Income by day and source in the list, and press Run.

> **You should see** Fourteen rows, oldest day first, with Workshop, Bike hire or Parts in the Source column, and the sentence: Lines listed: 14. Total income in these dates: £483.50.

Pick How many bookings, hires, sales and orders in the list, and press Run.



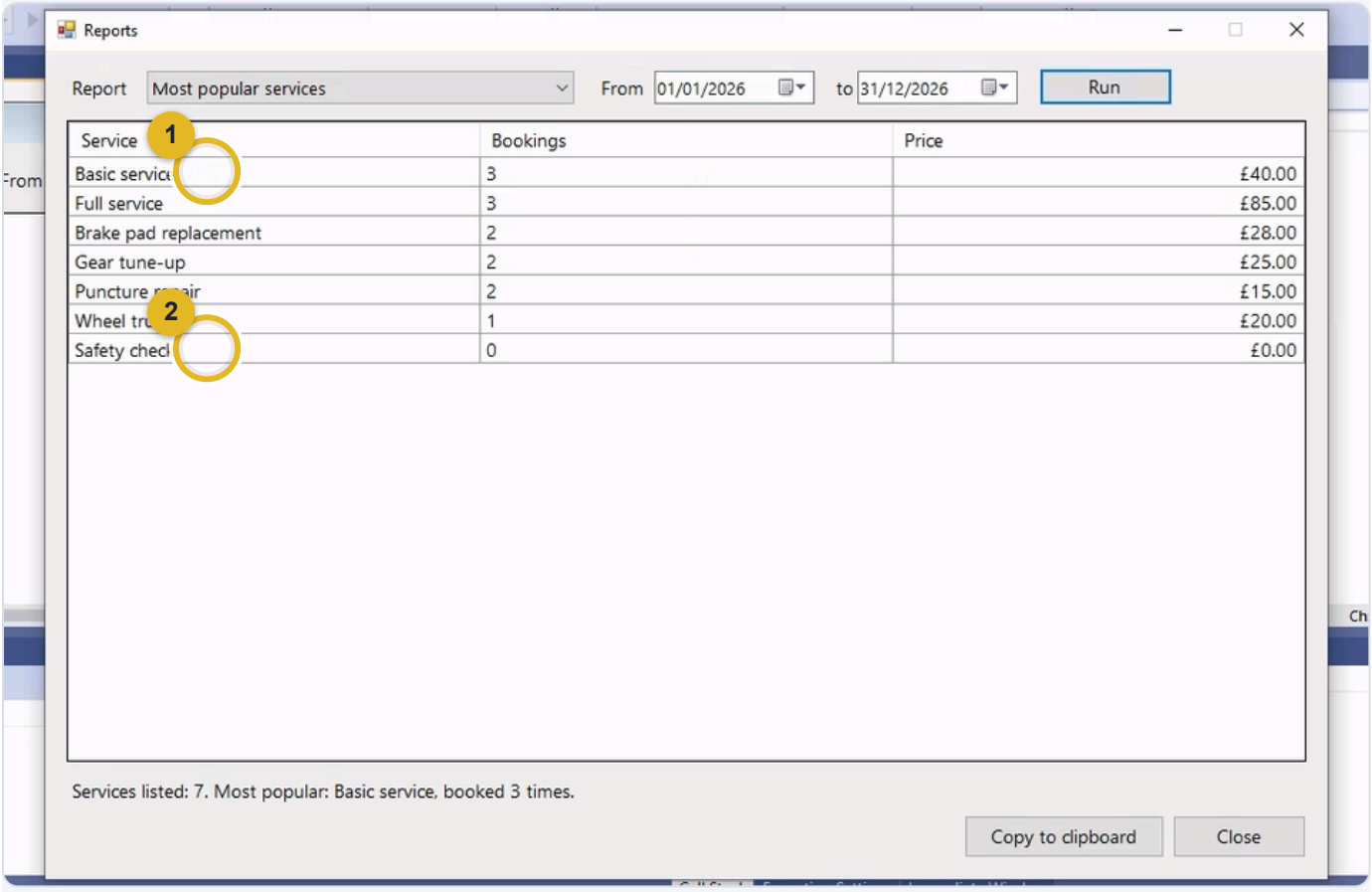

The activity report for the year. Five counts from four tables in one grid.

1. Five rows, one count each. The labels are the words in quotes in the stored procedure.

2. The sentence under the grid.

> **You should see** Five rows: Workshop bookings 13, Bookings cancelled 1, Bike hires 5, Parts sold 11, Part orders placed 3. The sentence reads: Each line counts one kind of thing in these dates.

Pick Most popular services in the list, and press Run.



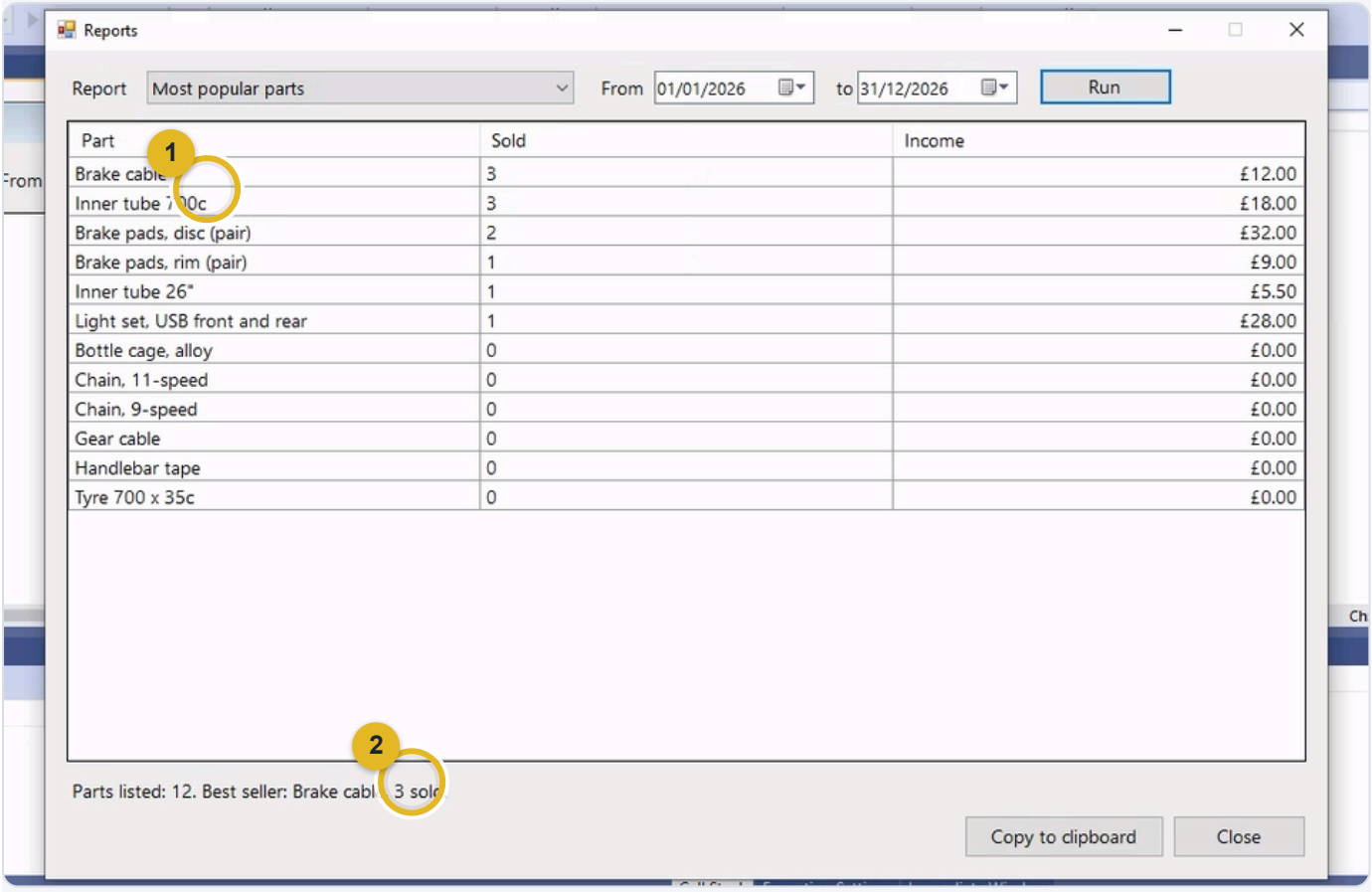

Every service, most booked first. Safety check has no bookings this year and is still in the list, with 0.

1. The top row is the most popular, and the sentence names it.

2. Safety check, with 0. A JOIN instead of a LEFT JOIN would lose this row.

> **You should see** Seven rows, Basic service at the top with 3 and Safety check at the bottom with 0, and the sentence: Services listed: 7. Most popular: Basic service, booked 3 times.

Pick Most popular parts in the list, and press Run.




Every part, best seller first. The six parts nobody bought this year are still listed, with 0.

1. Brake cable and Inner tube 700c both sold 3. Names break the tie, so Brake cable is first.

2. The sentence names the top row.

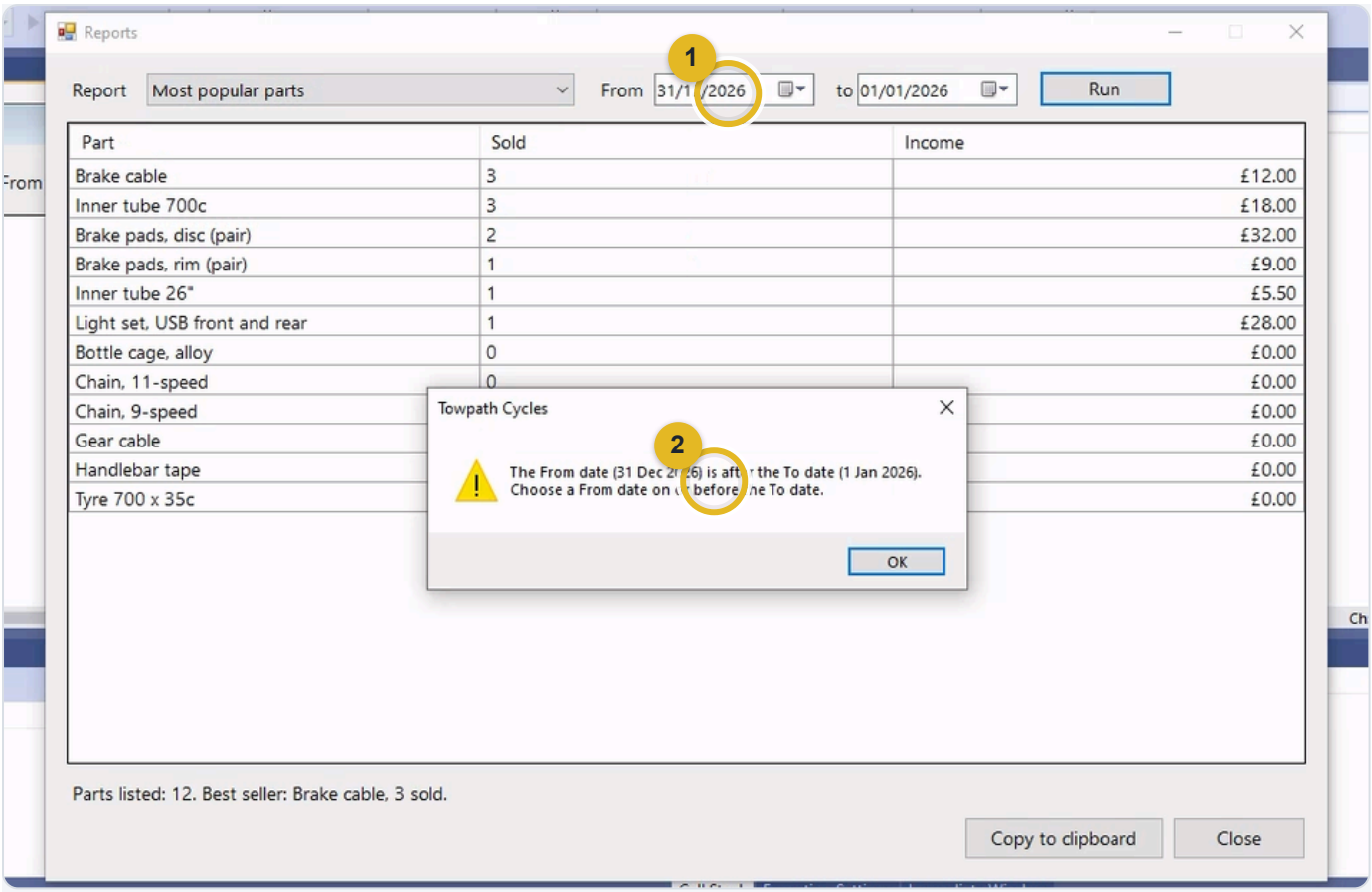
> **You should see** Twelve rows, Brake cable at the top with 3 sold and £12.00, six parts with 0 at the bottom, and the sentence: Parts listed: 12. Best seller: Brake cable, 3 sold.

# Tick the table: every report, the one mistake and the empty answer.

Every row below is a test the examiner would run. Four of them are the ones that matter most: the From date after the To date, which must be refused with a sentence and nothing else; two days with nothing in them, which must give an empty grid and a sentence, not an error; a service nobody booked, which must still be listed; and the LEFT JOIN taken out, which must lose that row, so that you know the LEFT is doing something. Do each one in your running program where it says, or in SQL Server Management Studio (SSMS) where it says, and tick it when you have seen the result.

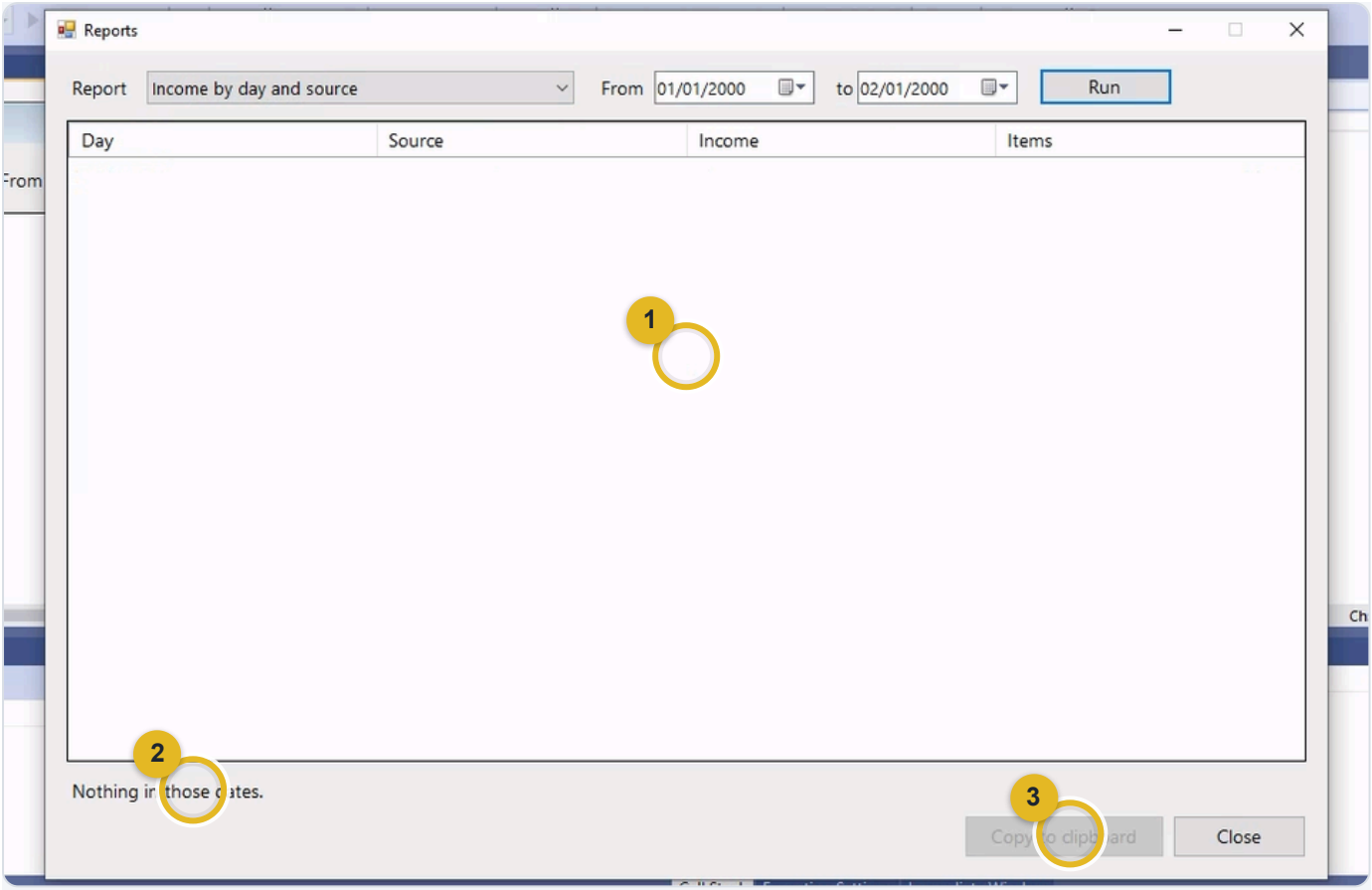
| | THE TEST | WHAT MUST HAPPEN | HOW TO DO IT, AND WHAT YOU SHOULD SEE |
|---|---|---|---|
| | Mechanic workload for the year | **Must be allowed** | Set the dates to 01/01/2026 and 31/12/2026, pick Mechanic workload, and press Run. You should see four rows, Conor Walsh first with 113.00 and Liam Doran last with zeros, and the sentence: Mechanics listed: 4. Workshop income in these dates: £233.00. |
| | Income by day and source for the year | **Must be allowed** | Fourteen rows. Sentence: Lines listed: 14. Total income in these dates: £483.50. |
| | How many bookings, hires, sales and orders for the year | **Must be allowed** | Five rows: 13, 1, 5, 11 and 3. Sentence: Each line counts one kind of thing in these dates. |
| | Most popular services for the year | **Must be allowed** | Seven rows. Sentence: Services listed: 7. Most popular: Basic service, booked 3 times. |
| | Most popular parts for the year | **Must be allowed** | Twelve rows. Sentence: Parts listed: 12. Best seller: Brake cable, 3 sold. |
| | The From date after the To date | **Must be refused** | Set From to 31/12/2026 and To to 01/01/2026, any report, Run. Message: The From date (31 Dec 2026) is after the To date (1 Jan 2026). Choose a From date on or before the To date. Press OK: the grid still shows the last report, and the cursor is back in the From box. |

| THE TEST | WHAT MUST HAPPEN | HOW TO DO IT, AND WHAT YOU SHOULD SEE |
|---|---|---|
| Two days with nothing in them | Must be allowed | Set From to 01/01/2000 and To to 02/01/2000, Income by day and source, Run. An empty grid with its headings, the sentence Nothing in those dates, and Copy to clipboard greyed out. No error. |
| A service nobody booked still shows | Must be allowed | Keep the year 2000 dates, Most popular services, Run. Seven rows, every one with 0. Sentence: Services listed: 7. None were booked in these dates. |
| A part nobody bought still shows | Must be allowed | Keep the year 2000 dates, Most popular parts, Run. Twelve rows, every one with 0 and 0.00. Sentence: Parts listed: 12. None were sold in these dates. |
| Copy to clipboard pastes into Notepad | Must be allowed | Set the dates back to 01/01/2026 and 31/12/2026, pick Most popular services, press Run, press Copy to clipboard, and press OK on the message. Open Notepad and press Ctrl and V together. The first line is the headings Service, Bookings and Price, and seven rows follow. |
| The LEFT JOIN taken out loses the service with no bookings | Must be refused | Do this in four steps in SQL Server Management Studio (SSMS): (1) in your script, delete the word LEFT from usp_Report_PopularServices as in stage 5, select the lines from CREATE OR ALTER PROCEDURE down to GO, and press F5; (2) in your test tab, with the year 2000 dates, press F5 and check the Results tab has no rows at all; (3) in your script, type LEFT back, select the same lines and press F5; (4) in your test tab, press F5 again and check there are seven rows, each with 0. |
| Counter staff cannot open the reports | Must be refused | Close the program, run it again, and sign in as padraig with the password canal02. The button reads Reports (manager only) and is greyed out, so the money is for the manager's eyes only. |




The one mistake a report can be asked to make, caught before SQL Server is asked anything. The grid behind the message is unchanged.

1. From is after To.
2. The sentence came from the exception class you typed in stage 6.




Two days in the year 2000, long before the shop opened. The report ran, found nothing, and said so. Copy to clipboard is off, because there is nothing to copy.

1. An empty grid is not an error. The question had an answer: nothing.
2. The sentence for no rows, the same for every report.
3. Copy to clipboard is greyed out.

# Finish the chapter.

Twelve ticks are your evidence that every report answers its question, that the one mistake and an empty answer are each handled with a sentence, and that the rows nobody would think to look for are still there. Take a screenshot of each result as you tick it, and keep the screenshots for your testing evidence in chapter 10. Paste one copied report into your user guide. When you want the sample data back, click in BikeShop.sql and press F5 with nothing selected.

**Make it yours**

Your scenario has its own questions. First, write down the five questions a manager would ask, each in one sentence with the word between in it, such as how many tickets were sold between two dates, or which room was booked most between two dates. Next, for each question, write down the table it starts from and the table joined to it. If the answer should list every one of something, even the ones with nothing to count, use a LEFT JOIN with the date test after ON. If the answer comes from two or three different tables, use a UNION ALL stack. Last, build one report form like frmReports, with your five questions in its list and the check that the From date is not after the To date.

CHAPTER 10 OF 12

# Testing evidence and user guide

The two short chapters of the write-up this guide covers.

STAGE 1 OF 4 · THE TEST TABLE · SCREEN 1 OF 3

## See what you will make in this chapter.

Your program works. This chapter turns that into two pieces of writing for your write-up. The first piece is your testing evidence: a table of tests in Word, with a picture under it for each test. The second piece is a user guide: a short document that shows a member of staff how to use each window of your program.

There is no code in this chapter. You use your running program, the Snipping Tool and Word. Most of the tests are ones you have already ticked at the end of chapters 7, 8 and 9, so the work here is writing them down and taking the pictures.

**Before you start**

- Chapters 7, 8 and 9 are finished, and every row of their test tables is ticked.
- Your BikeShop solution is open in Visual Studio 2019, and it builds.
- Word is open with a new, empty document.
- This guide is open in the Edge browser beside you. Tick each step as you do it, and the chapter remembers where you are.

**Words used in this chapter**

| | |
|---|---|
| `Test` | One thing you do to your program, written down with what should happen and what did happen. |
| `Test table` | A table in Word with one row for each test. |
| `Valid data` | Data the program should accept, such as a booking in a free slot. |
| `Invalid data` | Data the program should refuse, such as a booking on a Sunday. |
| `Extreme data` | Data right on the edge of what is allowed, such as selling exactly the number in stock. It should be accepted. |
| `Boundary` | The edge between allowed and refused, such as the moment one booking ends and the next one starts. |
| `Expected result` | What should happen, written before you run the test. |
| `Actual result` | What did happen when you ran the test. |
| `Evidence` | A picture of the screen that proves the actual result. |
| `Corrective action` | What you changed when a test did not give the expected result, and the test run again afterwards. |

| | |
|---|---|
| `User guide` | A short document that shows a member of staff how to use each window of your program. |
| `Snipping Tool` | The program on the Start menu that takes a picture of part of the screen. |

# Read the eight columns of the test table, and six real rows.

A test table has eight columns. Test is the number of the test. What is tested says what the test is about, in a few words. Type says whether the data is valid, invalid or extreme. What you do says exactly what you type or press. Expected result is what should happen. Actual result is what did happen. Evidence names the picture that proves it. Fixed? says what you changed, if anything was wrong.

Here are six rows, taken from the test tables at the end of chapters 7 and 8. The words are the same words you ticked there. Rows 1 to 3 are about bookings, and rows 4 to 6 are about stock and the workshop days. Row 5 is the one that went wrong the first time, and stage 2 shows how it was put right.

**Test 1: A booking in a free slot**

Type: valid. What you do: New booking, a free mechanic, a free time. Expected result: It appears in the grid. Actual result: As expected. Evidence: Figure 1, the diary with the new booking in it. Fixed?: Nothing to fix.

**Test 2: The same mechanic, the same start time**

Type: invalid. What you do: Book it again for the same mechanic. Expected result: Message: That mechanic already has a booking in that time. Actual result: As expected. Evidence: Figure 2, the message box. Fixed?: Nothing to fix.

**Test 3: A booking that starts exactly when another ends**

Type: extreme. What you do: Start at the first booking's end time. Expected result: Allowed: touching is not overlapping. Actual result: As expected. Evidence: Figure 3, the diary with both bookings in it. Fixed?: Nothing to fix.

**Test 4: Sell more than is in stock**

Type: invalid. What you do: Pick PADS-DISC with 2 in stock, Sell, quantity 3. Expected result: Message: Only 2 in stock. Actual result: As expected. Evidence: Figure 4, the message box. Fixed?: Nothing to fix.

**Test 5: Sell exactly what is in stock**

Type: extreme. What you do: PADS-DISC, Sell, quantity 2. Expected result: The row shows 0, and it is pink. Actual result: The first time, the sale was refused with the message Only 2 in stock. After the fix, as expected. Evidence: Figure 5, the wrong refusal, and Figure 6, the row showing 0 after the fix. Fixed?: Yes. The check in usp_PartSale_Add said <= where it should say <. It was changed to <, and the test was run again.

**Test 6: A Sunday**

Type: invalid. What you do: Pick a Sunday. Expected result: Message: The workshop is closed on Sundays. Actual result: As expected. Evidence: Figure 7, the message box. Fixed?: Nothing to fix.

# Make the test table in Word.

Make the table once, with its heading row, and add a row for each test as you go. Put the pictures under the table, each with a figure number and one line that says what it shows. The Evidence column then names the figure, so the examiner can find the picture for any row.

In Word, click the Layout tab at the top, between Design and References, click Orientation, and click Landscape.

> **You should see** The page turns on its side, so that eight columns have room for their words.

Type the heading Testing on the first line and press Enter.

> **You should see** The heading is on the first line, and the cursor is on the line under it.

Click the Insert tab, click Table, and click Insert Table…

> **You should see** The Insert Table box opens, with boxes for the number of columns and rows.

Type 8 in the Number of columns box and 2 in the Number of rows box, and press OK.

> **You should see** A table with eight columns and two rows appears in the document.

Type the eight headings in the top row, pressing Tab between them: Test, What is tested, Type, What you do, Expected result, Actual result, Evidence, Fixed?

> **You should see** The heading row is full, with one heading in each column.

Type the parts of test 1 from the last screen into the second row, pressing Tab to move from box to box.

> **You should see** The second row holds test 1, one part in each column.

Press Tab in the last box of the second row to add a new row, and type test 2 in it.

> **You should see** The third row holds test 2, one part in each column.

Add a row for each of tests 3 to 6 in the same way, one row each.

**Testing: the BikeShop booking and stock tests**

| Test | What is tested | Type | What you do | Expected result | Actual result | Evidence | Fixed? |
|---|---|---|---|---|---|---|---|
| 1 | A booking in a free slot | valid | New booking, a free mechanic, a free time. | It appears in the grid. | As expected. | Figure 1 | Nothing to fix. |
| 2 | The same mechanic, the same start time | invalid | Book it again for the same mechanic. | Message: That mechanic already has a booking in that time. | As expected. | Figure 2 | Nothing to fix. |
| 3 | A booking that starts exactly when another ends | extreme | Start at the first booking's end time. | Allowed: touching is not overlapping. | As expected. | Figure 3 | Nothing to fix. |
| 4 | Sell more than is in stock | invalid | Pick PADS-DISC with 2 in stock, Sell, quantity 3. | Message: Only 2 in stock. | As expected. | Figure 4 | Nothing to fix. |
| 5 | Sell exactly what is in stock | extreme | PADS-DISC, Sell, quantity 2. | The row shows 0, and it is pink. | The first time, the sale was refused with the message Only 2 in stock. After the fix, as expected. | Figure 5 and Figure 6 | Yes. The check in usp_PartSale_Add said <= where it should say <. It was changed to <, and the test was run again. |
| 6 | A Sunday | invalid | Pick a Sunday. | Message: The workshop is closed on Sundays. | As expected. | Figure 7 | Nothing to fix. |

The test table in Word, with the six rows from this chapter. Your own table has a row for every test you show.

1. The heading row: eight headings, in this order.

2. The Type column says valid, invalid or extreme.

3. The Evidence column names a figure under the table.

**You should see** The six tests are in the table, one row each, as in the picture.

**IF IT GOES WRONG** The columns are too narrow to read

Check that the page is Landscape, then drag the column edges so that What you do and Expected result are the widest columns.

STAGE 2 OF 4 · WHICH TESTS TO SHOW, AND HOW TO SHOW A FIX · SCREEN 1 OF 3

# Choose the tests that try the edges of each rule.

You cannot show every test you ran, and you do not need to. The examiner is looking for the tests that try the edges. A test in the middle, such as one booking on an empty day, shows very little. A test right on the edge shows that you thought about where the rule stops.

**The tests to show first**

Two bookings that touch: one starts at the exact minute the other ends. It must be allowed. Selling exactly the number in stock: the count goes down to 0. It must be allowed. The refusals: the same mechanic at the same time, selling more than is in stock, a Sunday, a time that has passed, refunding the same sale twice. Each one must be refused with its own sentence. The reports: the From date after the To date, and two days with nothing in them.

| | |
|---|---|
| **RULE THAT NEVER BREAKS · NO DOUBLE BOOKING** | Show the test where two bookings touch and the test where they overlap by one step. Together they prove that the line between allowed and refused is in the right place. |
| **RULE THAT NEVER BREAKS · STOCK IS ALWAYS RIGHT** | Show the test that sells exactly what is in stock and the test that sells one more. Together they prove that the count can reach 0 and can never go below it. |

For each rule, aim for at least one valid row, one invalid row and one extreme row. The six rows in stage 1 do that for bookings and for stock.

# Show a fix in three pictures: the test, the fix, the retest.

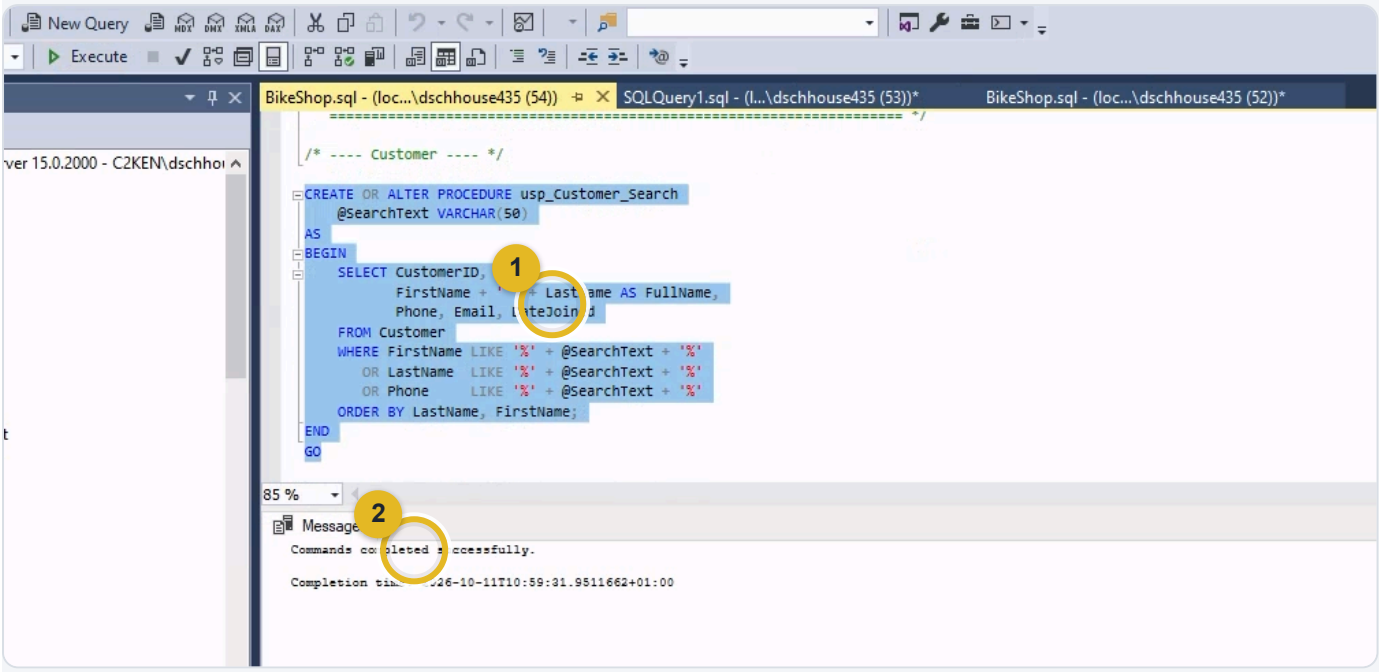
A test that fails is not a problem in your write-up. A failed test that you found, fixed and tested again is some of the best evidence you can show, because it proves that the testing was real. The examiner calls this corrective action.

Test 5 is a real example. The check in usp_PartSale_Add was typed as IF @InStock <= @Quantity. With 2 pairs of disc brake pads in stock, a sale of 2 was refused, and the message said Only 2 in stock. That message makes no sense: 2 in stock is enough for a sale of 2. That is the clue that the edge is in the wrong place.

In SQL Server Management Studio (SSMS), find the line IF @InStock < @Quantity in usp_PartSale_Add and type = straight after the <.

> **You should see** The line reads IF @InStock <= @Quantity. This is the mistake from the bike shop story, put in on purposeso that you can practise showing a fix.

Select the stored procedure, from CREATE OR ALTER PROCEDURE down to GO, and press F5.




Select the stored procedure you changed, then press F5. Only the highlighted lines run, so the test data you have made stays in the database.

1. Drag from CREATE OR ALTER PROCEDURE down to GO, so the whole block is highlighted.

2. The Messages pane says: Commands completed successfully.

> **You should see** The Messages pane says: Commands completed successfully.

Press F5 in Visual Studio 2019 to run the program.

> **You should see** The sign-in window opens.

Sign in as siobhan with the password canal01.

> **You should see** The main menu opens with Siobhán Rooney at the top.

Click the Parts and stock button on the main menu.

> **You should see** The Parts and stock window opens, with a grid of parts.

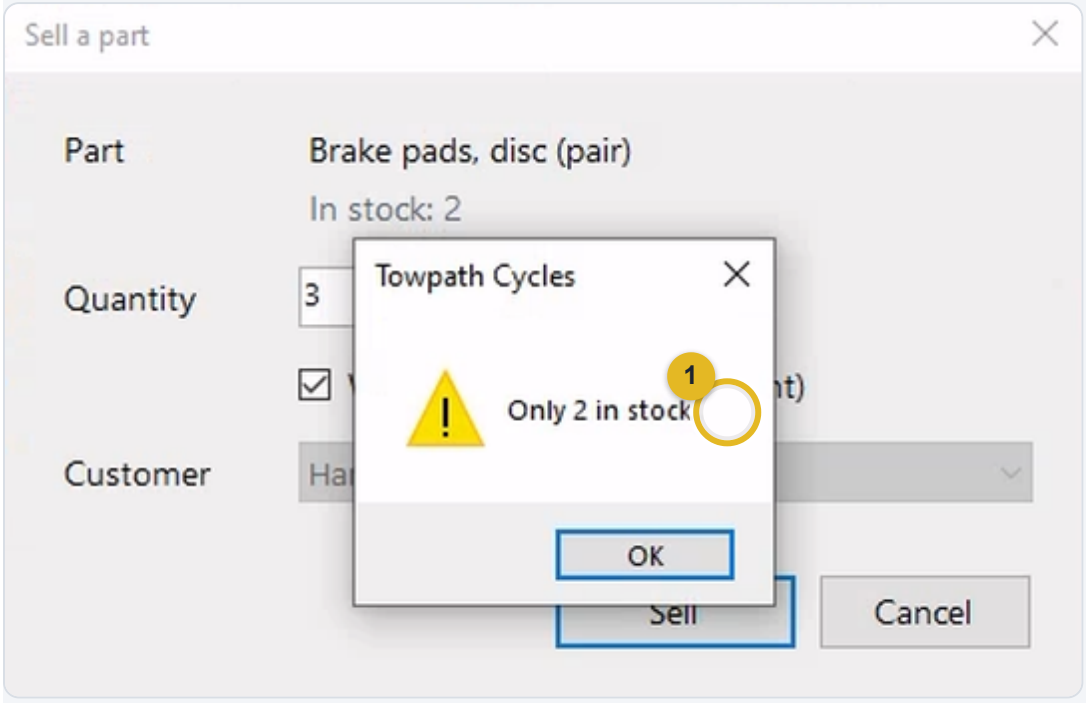
Click the PADS-DISC row in the grid.

> **You should see** The PADS-DISC row is highlighted, and it shows 2 in stock.

Click the Sell button under the grid.

> **You should see** The Sell window opens for PADS-DISC.

Set the quantity to 2 in the Sell window, and click the Sell button in that window.




The refusal, as the program shows it. The words came from the stored procedure, with the real count in them.

1. The sentence from @Message, shown by Msg.Problem.

> **You should see** A message box says Only 2 in stock. This is what Figure 5 shows; the next screen shows how to take the picture.

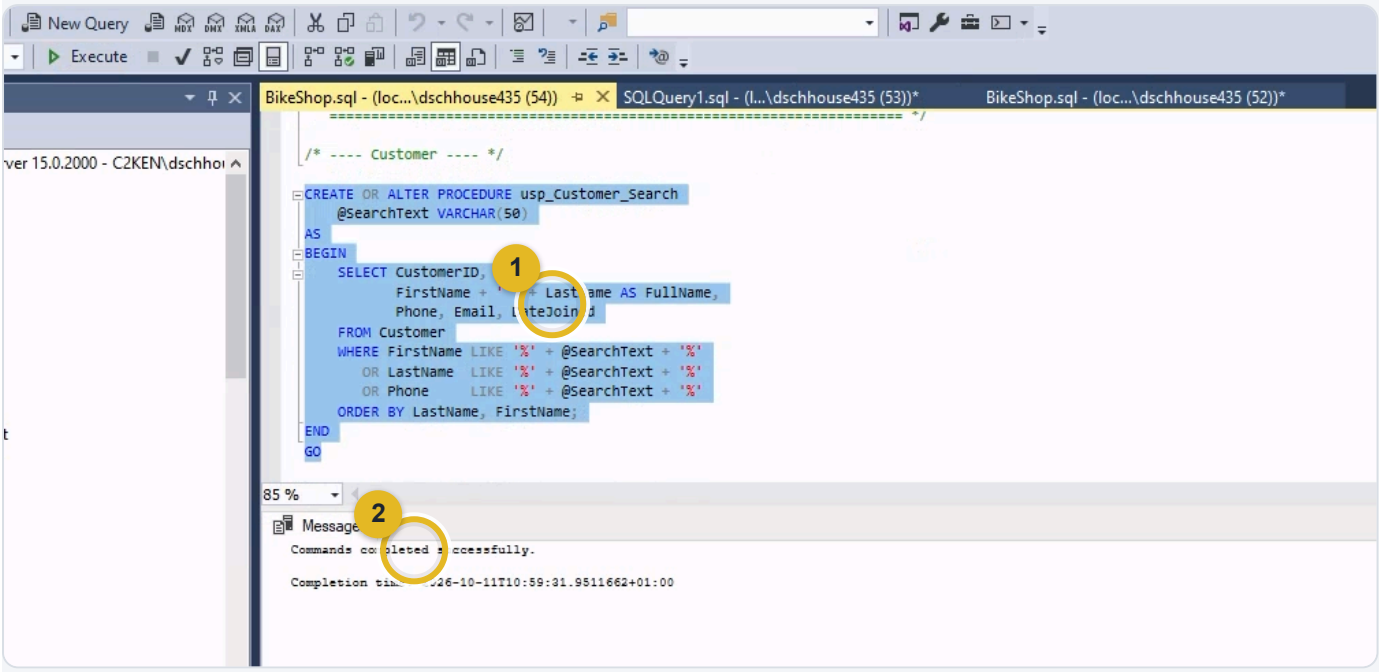
Close your program and go back to SQL Server Management Studio (SSMS).

> **You should see** usp_PartSale_Add is on the screen, with the line IF @InStock <= @Quantity.

Delete the = so that the line reads IF @InStock < @Quantity.

> **You should see** The line reads IF @InStock < @Quantity, the same as the yellow line in chapter 8.

Select the stored procedure, from CREATE OR ALTER PROCEDURE down to GO, and press F5.




Select the stored procedure you changed, then press F5. Only the highlighted lines run, so the test data you have made stays in the database.

1. Drag from CREATE OR ALTER PROCEDURE down to GO, so the whole block is highlighted.

2. The Messages pane says: Commands completed successfully.

> **You should see** The Messages pane says: Commands completed successfully.

Press F5 in Visual Studio 2019 to run the program.

> **You should see** The sign-in window opens.

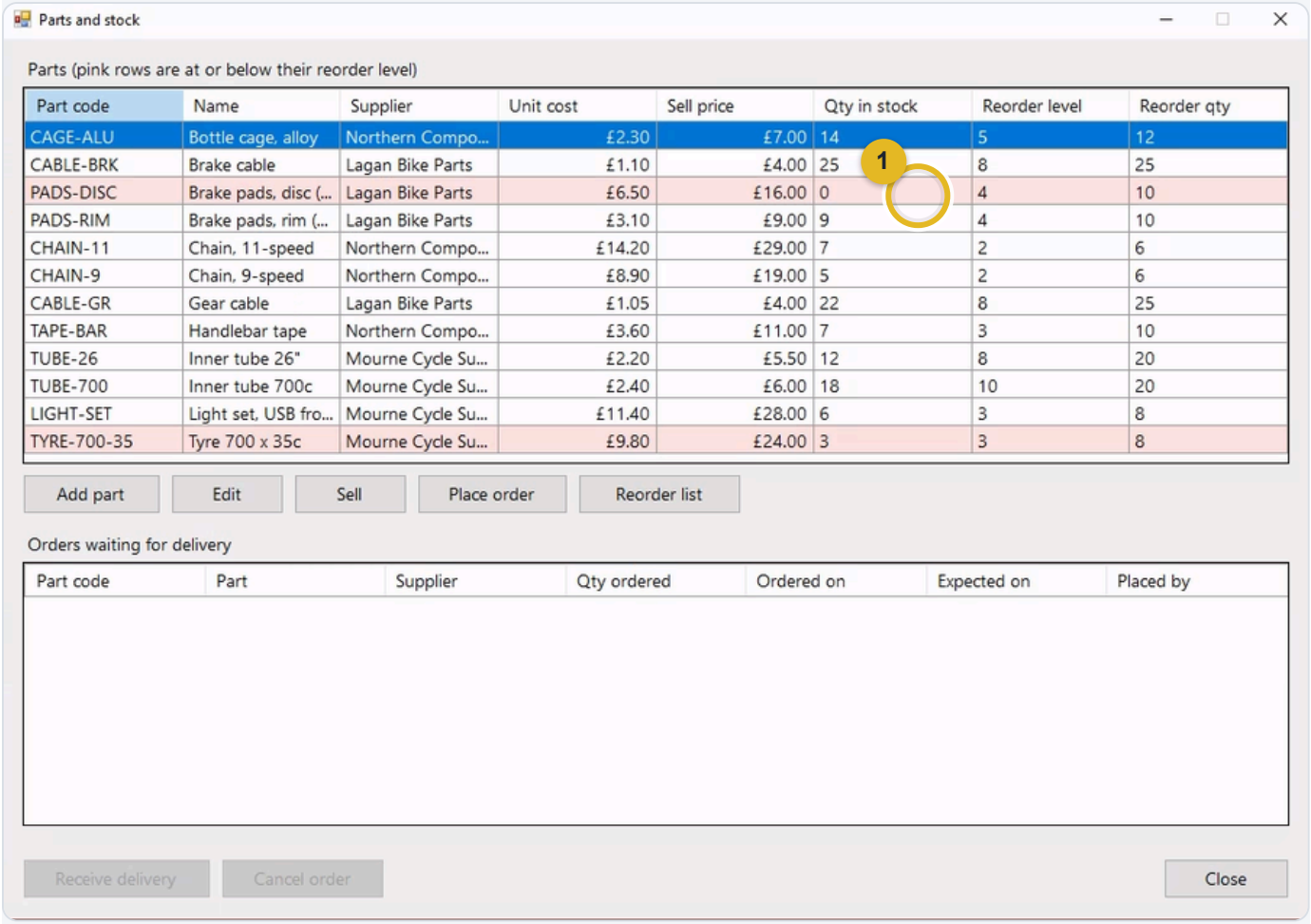
Sign in as siobhan with the password canal01.

> **You should see** The main menu opens with Siobhán Rooney at the top.

Click the Parts and stock button, then click the PADS-DISC row.

> **You should see** The PADS-DISC row is highlighted, and it shows 2 in stock.

Click the Sell button, set the quantity to 2, and click the Sell button in the Sell window.




The retest after the fix. Two pairs of disc brake pads were sold, and the count went down to 0, not below it.

1. PADS-DISC shows 0 in stock, and the row is pink because 0 is below its reorder level of 4.

> **You should see** The Sell window closes, and the PADS-DISC row shows 0 in stock and is pink. This is what Figure 6, the retest picture, shows.

In the table, the row for test 5 says what happened the first time, what you changed, and that the retest gave the expected result. Both figures are named in the Evidence column. If your own program had no mistakes like this, that is fine; show the tests that passed, and write Nothing to fix.

# Take a picture for each test and put it under the table.

Every row needs a picture that proves the actual result. Take the picture at the moment the result is on the screen: the message box while it is still open, or the grid straight after the change. A picture of the message box is enough for a refusal. For a test that changes data, show the row that changed.

Run the test in your program and leave the result on the screen.

> **You should see** The message box or the changed grid is on the screen.

Click the Start button, type Snipping Tool, and press Enter to open it.

> **You should see** The Snipping Tool window opens, with a New button at the top left.

Press New, then drag a box round the window you want.

> **You should see** The Snipping Tool shows your picture, and the picture is copied, ready to paste.

**IF IT GOES WRONG** The Snipping Tool picture shows the wrong window

The Snipping Tool takes whatever is on the screen when you press New. Bring the window you want to the front first, then press New again.

In Word, click on the empty line under your table and press Ctrl and V together.

> **You should see** The picture is under the table.

Type a line under the picture, such as Figure 2: the refusal for the same mechanic at the same start time.

> **You should see** The picture has its figure number and one line under it.

Type the figure number, such as Figure 2, in the Evidence box of that test's row.

> **You should see** The row names its picture, and the picture is under the table with the same number.

**Before the tests, put the sample data back**

The numbers in the six rows are for the sample data. Click in BikeShop.sql in SQL Server Management Studio (SSMS) and press F5 with nothing selected. The whole script runs, and

PADS-DISC has 2 in stock again.

# Read what goes on each page of the user guide.

The user guide is for the people in the scenario, not for the examiner and not for a programmer. Write it for a member of staff on their first day at Towpath Cycles. There is no code in it, and no table names.

Give each window of your program one page. The sign-in window comes first, because it is the first thing anyone sees, then the main menu, then each window in the order a member of staff would meet it. Every page has the same three parts.

**The three parts of every page**

What it is for: one or two sentences that say what the window does for the shop. The steps: numbered steps, each one starting with a doing word, such as Type, Pick or Press, with a picture of the window. What can go wrong: each message the window can show, word for word, and what to do about it.

The pictures come from your own running program, taken with the Snipping Tool, the same way as your test pictures. A picture of the window with real names and numbers in it is worth more than a paragraph of description.

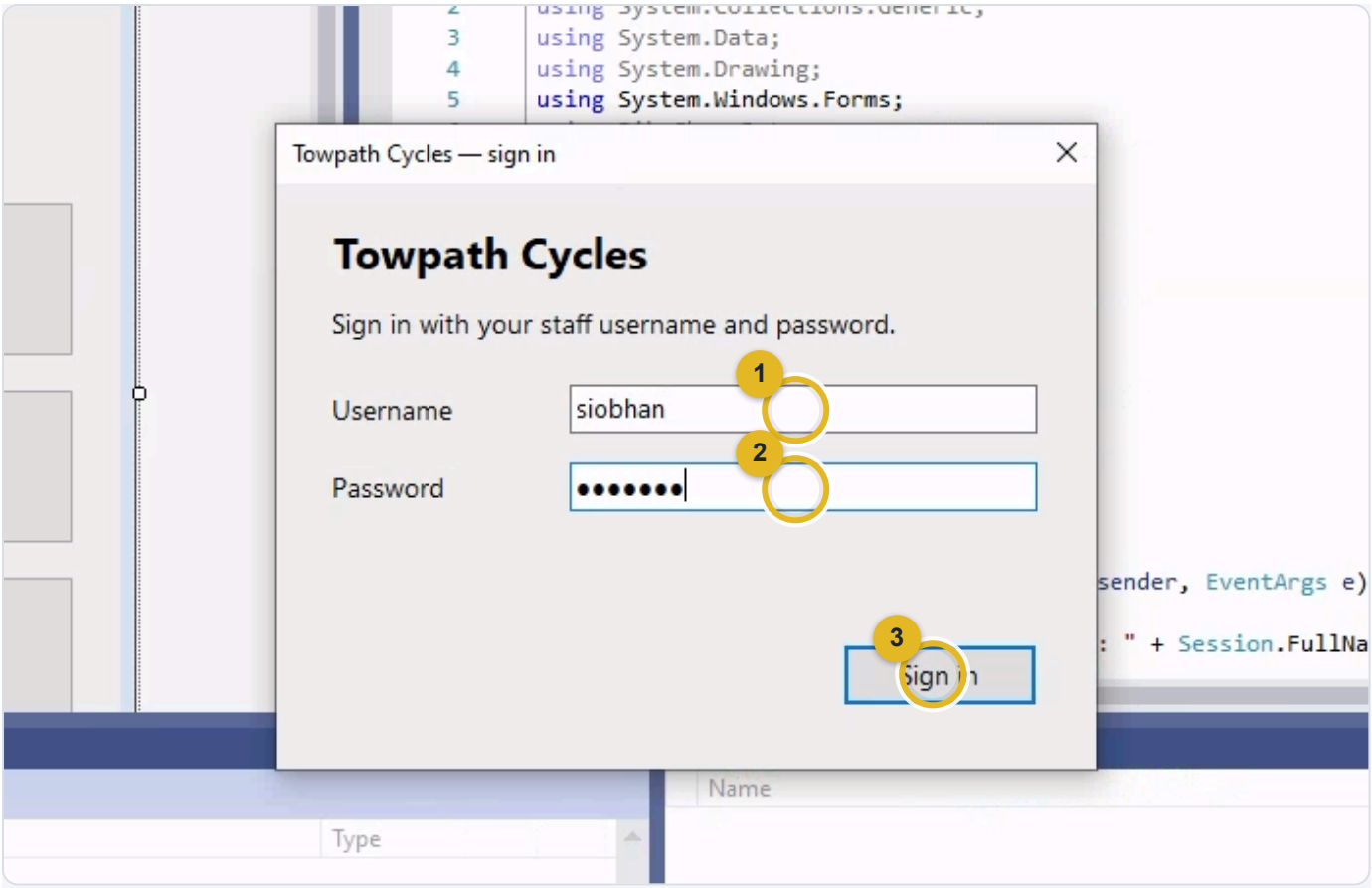
STAGE 3 OF 4 · THE USER GUIDE · SCREEN 2 OF 3

# Read the sign-in page, the first page of the guide.

Here is the first page of the Towpath Cycles user guide. Yours follows the same pattern with your own names and messages.

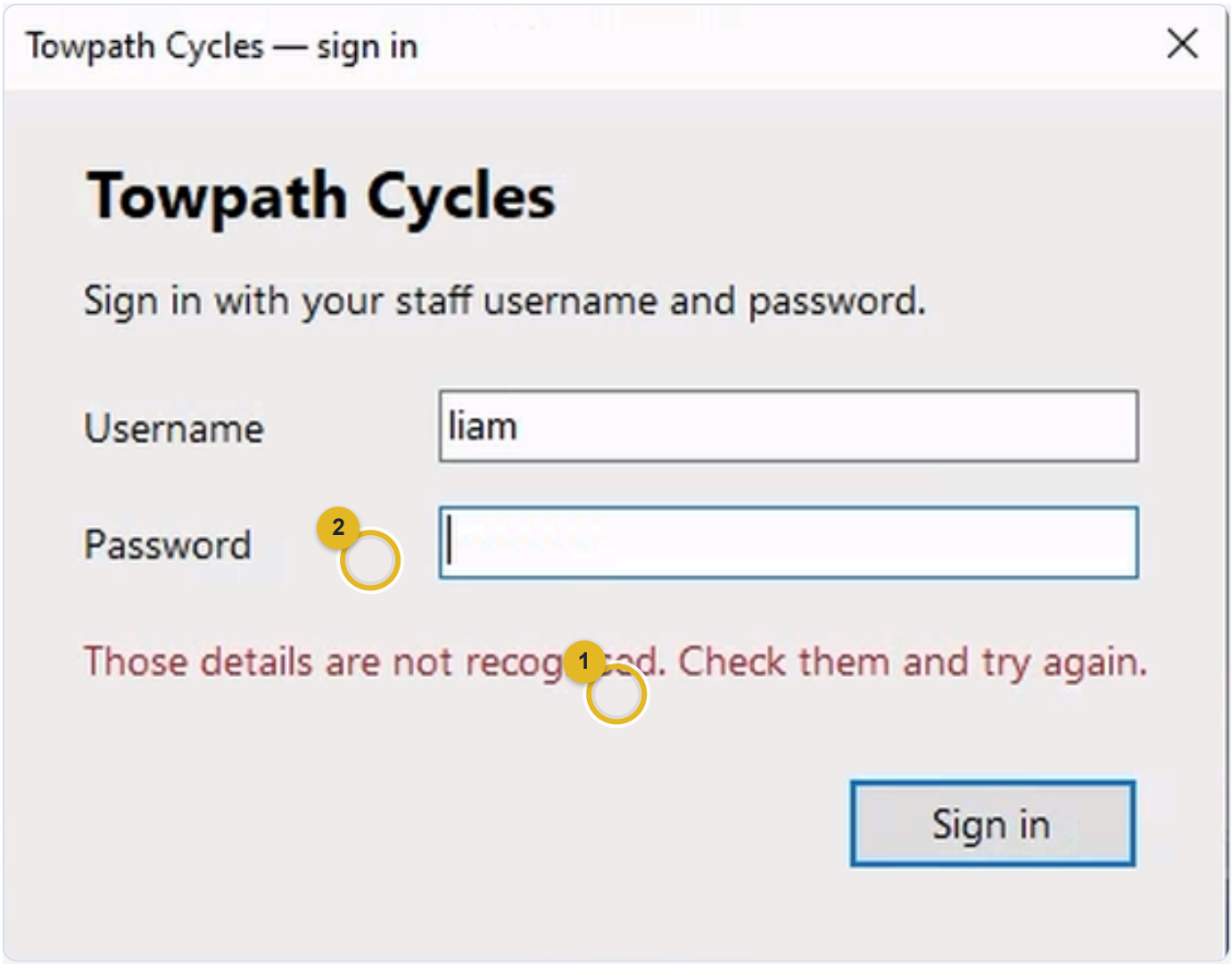
**Page 1: Signing in**

What it is for: The sign-in window lets a member of staff into the program. Each person has their own username, so the program can record who made each booking, hire and sale. The steps: 1. Type your username in the Username box. 2. Type your password in the Password box. It shows as dots. 3. Press Sign in, or press Enter. The main menu opens with your name at the top. What can go wrong: If you press Sign in with a box empty, the message Type your username and your password. appears, and you fill in both boxes. If the username or the password is wrong, the message Those details are not recognised. Check them and try again. appears, and the password box is emptied. If the program cannot reach the database, the message Could not reach the database. Is SQL Server running? appears, and you ask the manager.




The sign-in window of the finished program.

1. Type the username.
2. Type the password.
3. Click Sign in.




Sign in as liam with canal06. The sign-in stored procedure, usp_Staff_Login, sends back no rows, because Liam Doran is marked as left, so the program refuses and empties the password box.

1. The database said no, and the program turned that into a sentence.
2. The password box is emptied and the cursor is put back in it.

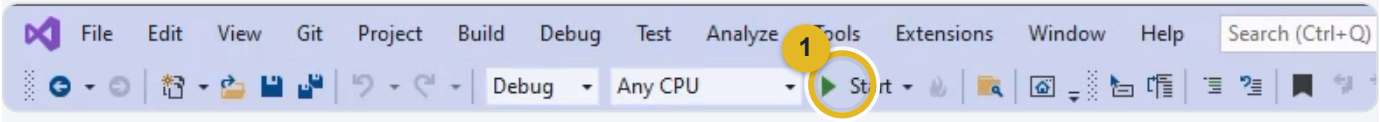
# Write a page for every window, from your own running program.

Towpath Cycles has sixteen windows, so its user guide has sixteen pages. Each small window that opens from a bigger one, such as New booking from the Workshop diary, gets its own page straight after the bigger one.

**The pages of the Towpath Cycles user guide, in order**

Signing in. The main menu. Customers, then Customer. Workshop diary, then New booking. Bike hire, then New hire. Bikes, then Add a bike. Parts and stock, then Part, Sell a part, Place an order and Receive a delivery. Reports.

Press F5 in Visual Studio 2019 to run your program.

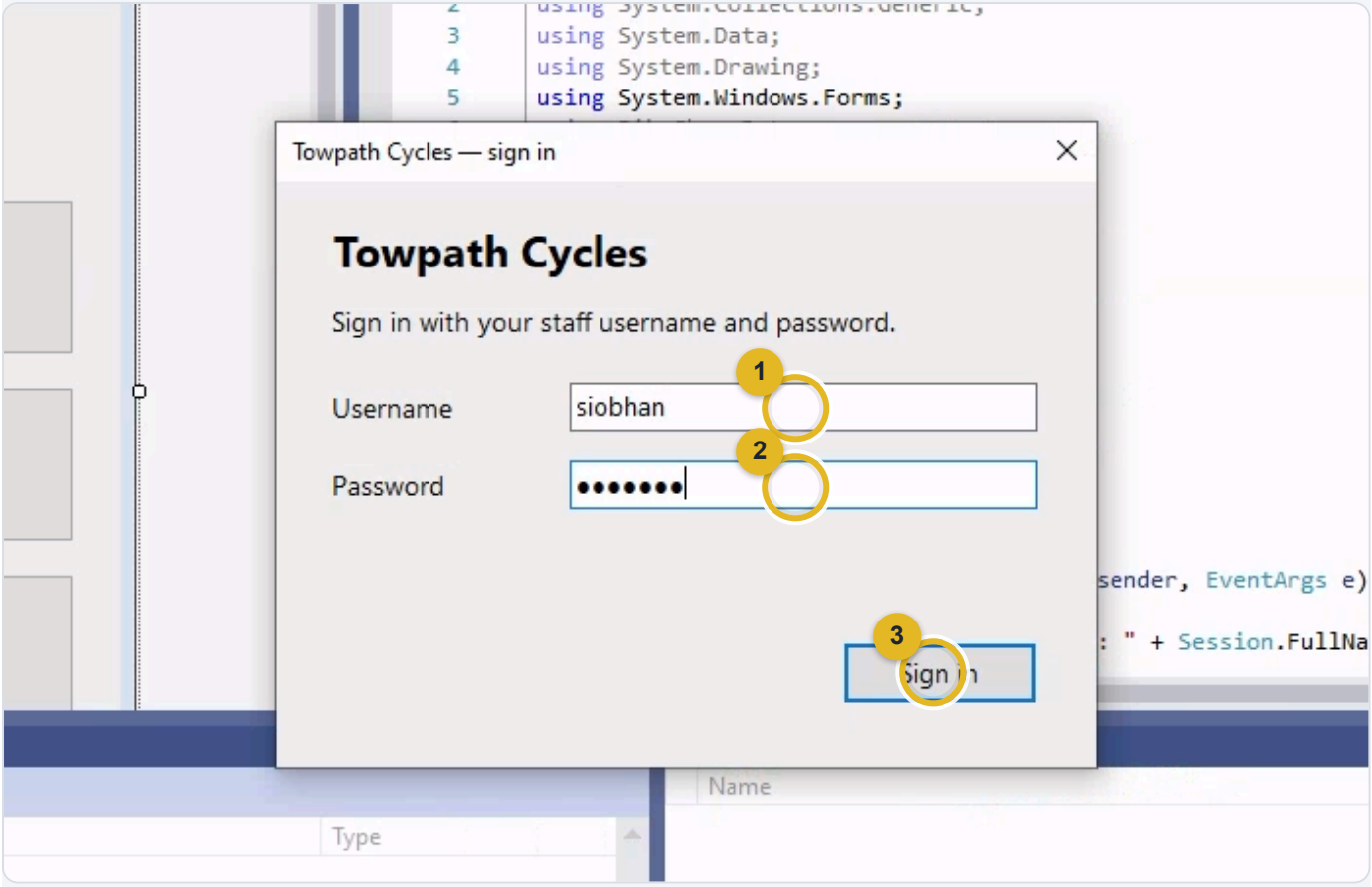



Start runs your program. Close the program window to come back to Visual Studio.

1. Click Start, or press F5.

**You should see** The sign-in window opens.

Take a picture of the sign-in window with the Snipping Tool.




The sign-in window of the finished program.

1. Type the username.
2. Type the password.
3. Click Sign in.

> **You should see** The Snipping Tool shows the sign-in window, ready to paste.

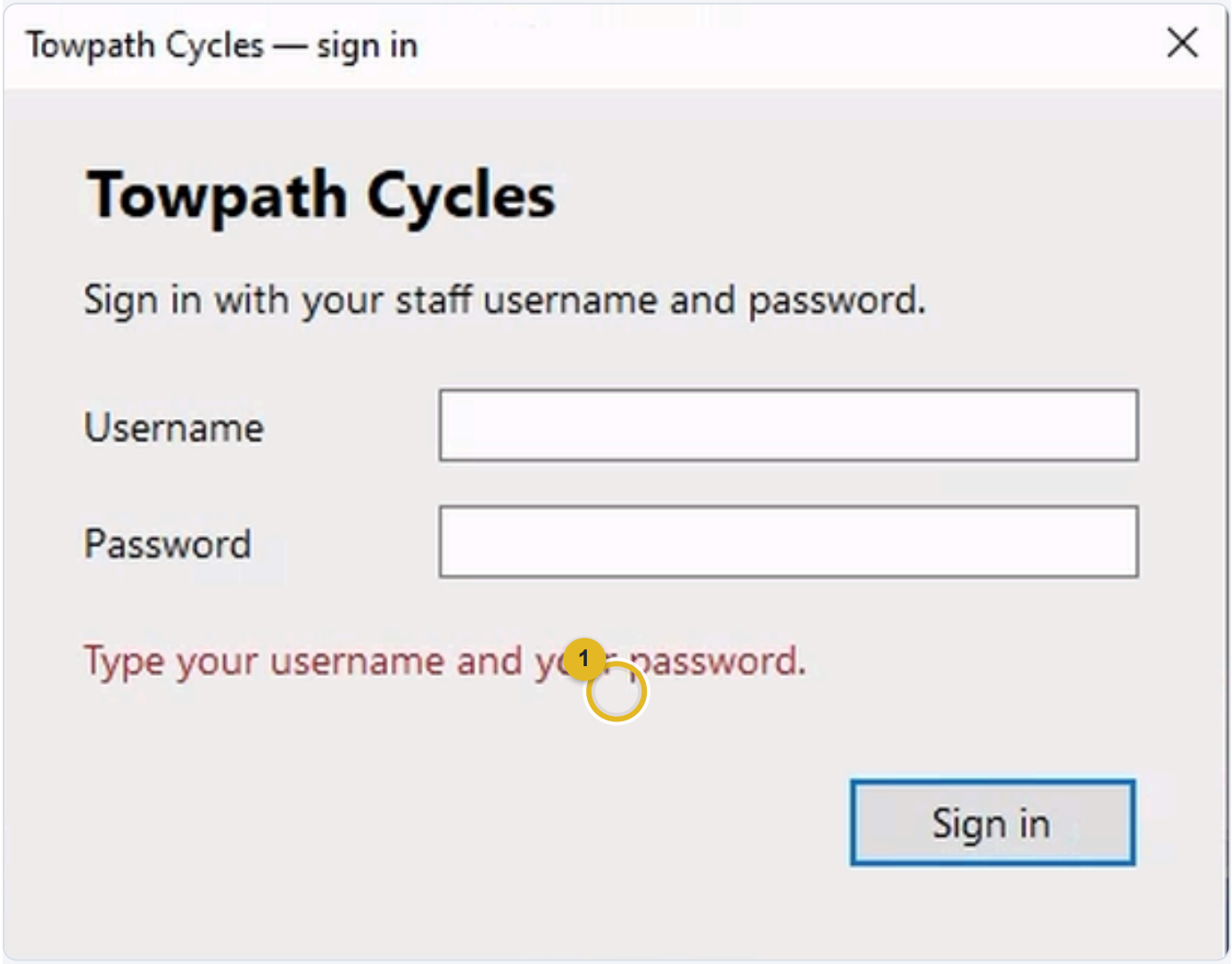
Open a new Word document and save it with the name User guide.

> **You should see** The Word title bar reads User guide.

Paste the picture on page 1 of the User guide document.

> **You should see** Page 1 has the picture of the sign-in window.

Press Sign in with both boxes empty, and take a picture of the message for the What can go wrong part.




Sign in with both boxes empty. The program refuses before it asks the database anything.

1 The red message appears here and the window stays open.

> **You should see** The red message reads: Type your username and your password.

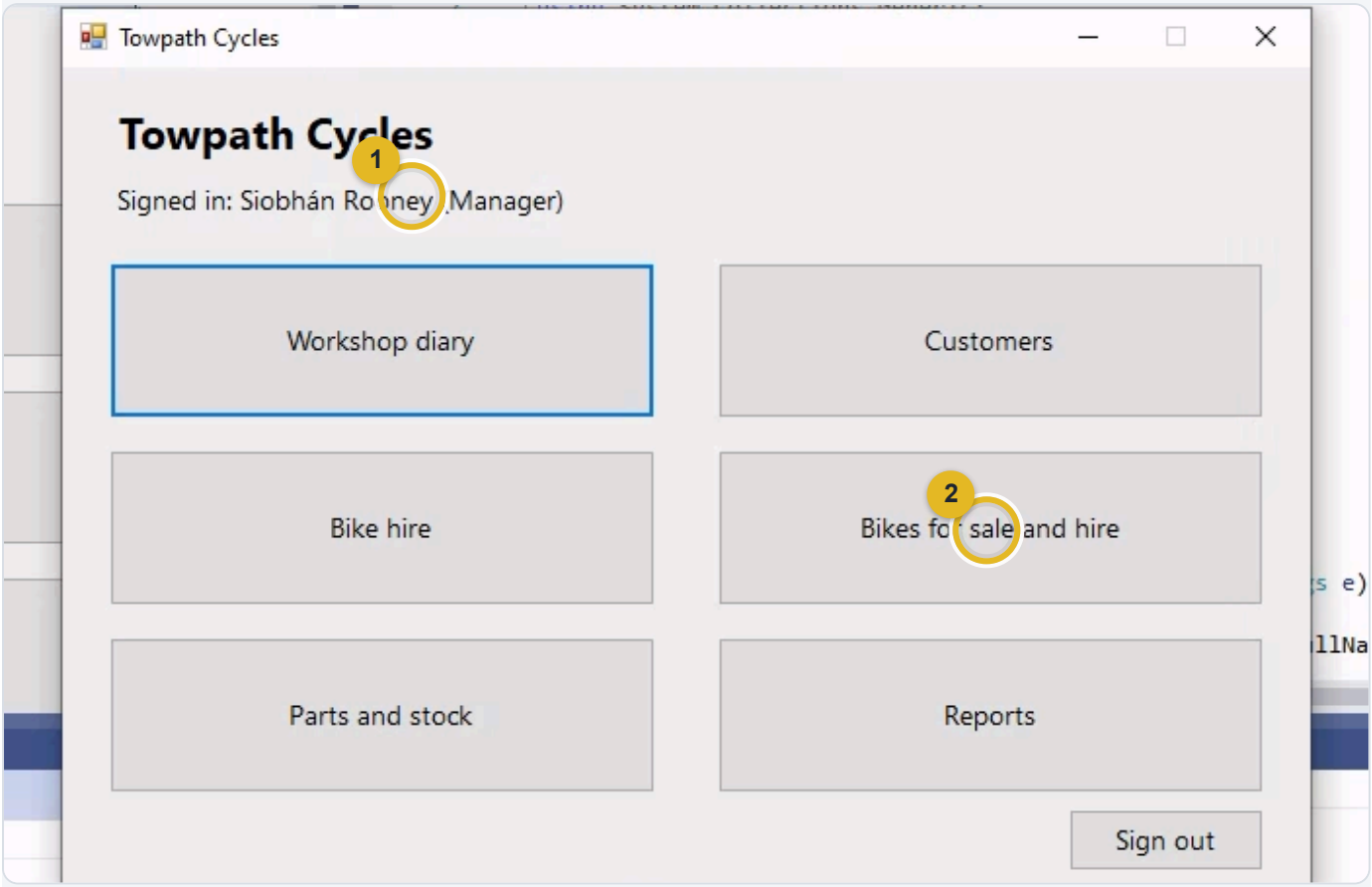
Write the three parts of page 1: What it is for, The steps, and What can go wrong.

> **You should see** Page 1 looks like the sign-in page on the last screen, with your own names and messages.

Sign in as siobhan with the password canal01.

> **You should see** The main menu opens with Siobhán Rooney at the top.

Take a picture of the main menu with the Snipping Tool, paste it on page 2 of the User guide document, and write its three parts under it.




The main menu. A manager sees every button; a mechanic sees fewer.

1. Who is signed in.
2. One button for each screen.

> **You should see** Page 2 shows the main menu with Siobhán Rooney at the top, and its three parts.

Work through every other window in the order of the list above, one page each.

> **You should see** Every window of your program has a page, with a picture, numbered steps and its messages word for word.

**IF IT GOES WRONG** A message in the guide does not match the program

Copy each message from your picture, letter by letter. The examiner compares the guide with the program, and a message that differs looks like the guide was written before the program was finished.

# Tick the checklist: the table, the pictures, the two edges and the user guide.

Every row below checks one thing the examiner will look for. Three rows are marked Must be refused. Rows 10.4.4 and 10.4.5 put a mistake back on purpose, to prove that your edge tests would catch it; put each one right straight afterwards. Before you start, click in BikeShop.sql in SQL Server Management Studio (SSMS) and press F5 with nothing selected, so that the sample data is back.

| | THE TEST | WHAT MUST HAPPEN | HOW TO DO IT, AND WHAT YOU SHOULD SEE |
|---|---|---|---|
| | The test table has its eight headings | Must be allowed | Open your Word document. The heading row reads Test, What is tested, Type, What you do, Expected result, Actual result, Evidence and Fixed?, in that order. |
| | Every row has a type | Must be allowed | Read down the Type column. Every row says valid, invalid or extreme, and each rule has at least one of each. |
| | Every row has its picture | Must be allowed | Read down the Evidence column. Every row names a figure, and that figure is under the table with a line that says what it shows. |
| | The touching-booking test catches the mistake | Must be refused | 1. In usp_Appointment_Add in SSMS, change the check to @StartTime <= EndTime. 2. Select the stored procedure and press F5. 3. Book the same mechanic at the end time of one of their bookings. The message reads: That mechanic already has a booking in that time. 4. Change the check back to @StartTime < EndTime, select the stored procedure and press F5 again. |
| | The exact-stock test catches the mistake | Must be refused | 1. In usp_PartSale_Add in SSMS, change the line to IF @InStock <= @Quantity. 2. Select the stored procedure and press F5. 3. Sell 2 of PADS-DISC. The message reads: Only 2 in stock. 4. Change the line back to IF @InStock < @Quantity, select the stored procedure and press F5 again. |
| | The exact-stock test passes after the fix | Must be allowed | Sell 2 of PADS-DISC again. The row shows 0, and it is pink. |

| | THE TEST | WHAT MUST HAPPEN | HOW TO DO IT, AND WHAT YOU SHOULD SEE |
|---|---|---|---|
| | The user guide starts with the sign-in page | Must be allowed | Page 1 is Signing in, with a picture from your own program, numbered steps, and the messages word for word. |
| | A wrong password gives the message the guide says | Must be refused | Sign in as liam with the password canal06. The red message reads: Those details are not recognised. Check them and try again. The user guide's first page has the same words. |
| | Every window has its page | Must be allowed | Count the pages of your user guide. There is one page for each window of your program, each with its three parts. |

Sign in as liam with canal06. The sign-in stored procedure, usp_Staff_Login, sends back no rows, because Liam Doran is marked as left, so the program refuses and empties the password box.

1. The database said no, and the program turned that into a sentence.
2. The password box is emptied and the cursor is put back in it.

STAGE 4 OF 4 · PROVE IT: THE WRITE-UP CHECKLIST · SCREEN 2 OF 2

# Finish the chapter.

Nine ticks are your evidence that the test table and the user guide are complete. When you want the sample data back, click in BikeShop.sql and press F5 with nothing selected.

**Make it yours**

Your scenario has its own two rules. First, write down the edge of each one: the exact moment two of your bookings touch, and the exact count that empties your stock. Next, write one valid row, one invalid row and one extreme row for each rule in your test table, before you run them, with the expected result filled in. Last, list every window of your program in the order a member of staff would meet them. That list is the contents page of your user guide.

# Mistakes that cost marks

What last year lost marks for, and how to check your own work for each one.

STAGE 1 OF 1 · PROVE IT: FOURTEEN MISTAKES, AND THE CHECK FOR EACH ONE · SCREEN 1 OF 4

## Read the five database mistakes, and where this guide does each one right.

Every mistake in this chapter was found in a real A2 project last year. No names are given. Each mistake is told in three parts: what went wrong, what it cost, and the chapter of this guide that does the same job the right way. Read each mistake, then look for it in your own project. The last screen turns all fourteen into a table of checks to tick before you hand in.

**Words used in this chapter**

| | |
|---|---|
| `Grid` | The table of rows and columns on a form, such as the list of customers. |
| `ID number` | The number the database gives each row by itself, such as CustomerID 7. It means nothing to the person using the program. |
| `Column type` | The kind of value a column holds, such as DATE for a day, DECIMAL for money, VARCHAR for text or BIT for yes or no. |
| `Stored procedure` | A saved set of instructions inside the database that your program asks it to run, such as usp_Appointment_Add. |
| `Parameter` | One piece of information sent into a stored procedure, with a name that starts with @, such as @CustomerID. |
| `Clash` | Two bookings for the same mechanic, or the same bike, at times that overlap. |
| `Status` | One word in a row that says where a thing is in its life, such as Booked, Cancelled or Sold. |
| `Click method` | The method that runs when a button is pressed, such as btnSave_Click for the Save button. |
| `Property` | A named value on a class, such as LastName on a Customer, that the program can read and change. |
| `Get part` | The code inside a property that runs when the program reads its value. |
| `DataTable` | The table of rows that a stored procedure hands back to the C# program. |
| `Null` | The word C# uses for nothing at all: no table, no text and no number. |
| `Crash` | The program stops with an error box or freezes, and the person using it loses what they were doing. |

**ID numbers in the grids**

One project showed its bookings grid with columns of numbers such as CustomerID 7 and StaffID 3, so the person at the counter could not tell who a booking was for. Last year's marking counted a grid of ID numbers as a clear weakness, because the program does not suit the person using it. Chapter 6, stage 2, screen 1, 'What a grid shows, and why it needs tidying', hides the ID columns with Grid.Tidy. Chapter 4, stage 5, 'Names, not numbers', writes the stored procedures that join the tables so the grid gets names.

**Dates kept as text**

One project kept its dates in VARCHAR columns, which hold text, and turned each date into words with ToString. Some dates came out cut short, and the program could not put the bookings in order by day or find the ones between two dates. Wrong data in normal use costs marks for the database and for the testing. Chapter 2, stage 4, screen 1, 'Choose a type for every column', explains why dates are DATE or DATETIME and never text. Chapter 3, stage 3, screen 1, 'Read the Appointment table part by part, then add it to your script', uses DATETIME for a day with a clock time, and DATE for a day on its own.

**A money column that is too small**

One project gave its price column the type DECIMAL(3,2), which allows three digits with two of them after the point, so the biggest price it can hold is 9.99. Every price of ten pounds or more was refused with an error, so the shop could not record anything that cost more. Chapter 2, stage 4, screen 1, 'Choose a type for every column', explains why money is DECIMAL. Chapter 3, stage 2, screen 3, 'Type the two checks of the ServiceType table', uses DECIMAL(8,2) for a price: eight digits, two of them after the point.

**The double-booking check only in the form**

One project checked for a clash inside the C# booking form, by loading the bookings into the program and comparing them. Two computers at the counter could still book the same mechanic at the same time, because each form only knew the bookings it had loaded when it opened. A check in the form earned some credit, but no double booking is a rule every case study tests, and full marks need the database to refuse. Chapter 7, stage 1, screen 3, 'Read what goes wrong without the rule', shows why the check belongs in the stored procedure, and stage 2 of chapter 7 writes that stored procedure.

**Cancel deletes the booking**

One project's Cancel button deleted the booking's row, so a cancelled booking vanished from the database. The shop lost its history: nobody could see what had been cancelled, and every count of bookings came out too low. A business needs its records, so a program that throws them away loses marks. Chapter 7, stage 7, screen 1, 'Read why a cancelled booking is not deleted, and type the stored procedure that cancels one', sets the booking's status to Cancelled instead.

# Read the five code mistakes, and where this guide does each one right.

These five mistakes are in the C# code, and in the way the C# code talks to the stored procedures. Each one either leaves a button doing nothing or stops the program with an error. The examiner presses every button, so each of these mistakes shows up in the examiner's testing.

**Buttons with nothing behind them**

One project had buttons whose Click methods were empty, and other buttons that were never joined to a Click method at all, so pressing them did nothing. A button that does nothing is a feature that is not there, and the marks for that feature are lost. Chapter 5, stage 6, screen 2, 'Put the code behind the main menu', gives every menu button its own Click method, and each later chapter fills in the screen that the button opens.

**Console.ReadLine in a form**

One project used Console.ReadLine, which waits for someone to type in a black command window, inside a Windows form. A form has no command window, so the program froze and had to be closed. A freeze in normal use is one of the most serious faults a program can have. Chapter 5, stage 4, screen 5, 'Add the Msg class at the top level, and read it', shows a message box instead, and every form in this guide reads what was typed from its own text boxes.

**"y" in one place and "Yes" in another**

One project saved a yes-or-no answer as the letter y in one form, and tested for the word Yes in another form. The two never matched, so the feature that depended on the answer never worked. Chapter 3, stage 2, screen 2, 'Add the Staff table, which has a UNIQUE rule and a list of roles', stores yes or no as a BIT, which can only be 1 or 0, and lets the Role column hold only the exact words in its list. Chapter 4, stage 5, screen 3, 'Copy the sign-in stored procedure, and read how one row means yes', tests IsActive = 1.

**A parameter without its @**

One project left the @ off a parameter name inside a stored procedure. Without the @, SQL Server reads the word as a column name and not as the value the form sent in. In a WHERE line, CustomerID = CustomerID is true for every row, so an update changes every customer instead of one. Chapter 4, stage 3, screen 3, 'Type the lines that make an Update different from an Add', writes WHERE CustomerID = @CustomerID, with the @ on the value that was sent in.

**Names that do not match**

One project's C# asked for a stored procedure by one name while the script had saved it under another, such as usp_AddCustomer in the code and usp_Customer_Add in the script. The program stopped with the message Could not find stored procedure. A parameter spelled differently in the two places stops the program in the same way. Chapter 4, stage 1, screen 2, 'Check that you have these ready, and learn how the stored procedures are named', sets out one naming pattern for every stored procedure. Chapter 5, stage 4, screen 4, 'Add the StaffData class to the Data folder, and read its Login method', sends the exact name usp_Staff_Login and the same @ names the stored procedure declares.

# Read the four mistakes in the screens and the data, and where this guide does each one right.

These four mistakes only show up when someone uses the program the way the examiner does: an empty report, a second sale, an edit, or a class read many times. Two of them crash the program, and two of them quietly save the wrong thing.

**The report form that crashes when nothing is found**

One project's report form crashed when the dates held no records, because its code read the first row of a DataTable that had no rows, or a DataTable that was null. The examiner tries dates with nothing in them on purpose, so this crash happens during the examiner's testing. Chapter 9, stage 7, screen 2, 'Put the code behind the report form', counts the rows first, and when there are none the form shows the sentence: Nothing in those dates.

**Sold items never marked sold**

One project recorded each sale but never changed the item's status to Sold, so the same item could be sold again the next day. Stock that is always right is the second rule every case study tests, so this mistake costs marks in the program and in the testing. Chapter 8, stage 6, screen 3, 'Write the stored procedure that sells a bike once, and only once', refuses a bike that is already sold, and sets the bike's status to Sold in the same stored procedure that records the sale.

**The last name copied from the first-name box**

One project's edit form had a copied line that set the last name from the first-name box, as in customer.LastName = txtFirstName.Text. Every customer that was edited came back with the same first name and last name, and nothing warned anyone, so the mistake only showed when someone read the grid. Chapter 6, stage 4, screen 4, 'Read Save: plain ifs, and the first problem wins', copies each box into its own property, one line each. Chapter 6, stage 5, screen 4, 'Edit the customer, two ways', is the test that catches this mistake.

**The get part that reads itself**

One project wrote a property whose get part handed back the property itself, as in get { return Price; } inside the property called Price. Reading the property ran the get part, which read the

property again, over and over, until the program crashed. Chapter 6, stage 1, screen 3, 'A class is one row in your hands', gives every plain property { get; set; }. FullName, the one property in that class with its own code, reads FirstName and LastName and never reads FullName.

STAGE 1 OF 1 · PROVE IT: FOURTEEN MISTAKES, AND THE CHECK FOR EACH ONE · SCREEN 4 OF 4

# Check your own project for all fourteen mistakes before you hand it in.

Three kinds of mistake cost the most marks last year. The first is a double-booking check that lives only in the form, because no double booking is a rule every case study tests. The second is a crash or a freeze in normal use: the report on empty dates, Console.ReadLine, the get part that reads itself, and the price that does not fit. The third is a grid of ID numbers, which the marking counts as a program that does not suit the person using it.

Do each check below in your own program, not the bike shop. A row marked Must be refused passes only when your program says no with a sentence. Tick the row when your project passes it, and if it fails, fix your project before you hand in. Then take a screenshot of each refusal for your testing evidence in chapter 10.

| THE TEST | WHAT MUST HAPPEN | HOW TO DO IT, AND WHAT YOU SHOULD SEE |
|---|---|---|
| Every grid shows names | Must be allowed | Open every screen in your program that has a grid. No column shows an ID number such as CustomerID or StaffID, and every person, thing and service appears by name. |
| Every date is kept as a date | Must be allowed | In SQL Server Management Studio (SSMS), expand each of your tables in Object Explorer and open Columns. Every column that holds a day or a time is DATE or DATETIME, never VARCHAR. |
| The biggest price fits | Must be allowed | Add one item with the highest price your scenario could ever have, such as 1250.00. The item saves, and the grid shows the price with its pennies. |
| A double booking is refused by the database | Must be refused | In SSMS, run EXEC on your booking stored procedure twice, giving the same person or thing and an overlapping time both times. The second run refuses with a sentence, and no second row is added. |
| A cancelled booking stays on record | Must be allowed | Cancel one booking in your program, then look at that day. The booking is still there, marked Cancelled, and its time is free to book again. |
| Every button does something | Must be allowed | Sign in as each role and press every button on every screen once. Each button opens a screen, saves, refuses with a sentence or closes, and no button does nothing. |

| THE TEST | WHAT MUST HAPPEN | HOW TO DO IT, AND WHAT YOU SHOULD SEE |
|---|---|---|
| No Console anywhere in the forms | Must be allowed | In Visual Studio 2019, press Ctrl, Shift and F together, search the entire solution for the word Console, and press Find All. The results list is empty. |
| Yes and no mean the same everywhere | Must be allowed | For each yes-or-no column, check that the table stores it as a BIT, and that every form and stored procedure tests it as 1 or 0, never as text such as y or Yes. |
| Every parameter keeps its @ | Must be allowed | In your script, read every WHERE line inside every stored procedure. Each value the form sent in has its @, as in WHERE CustomerID = @CustomerID. |
| The names match in both places | Must be allowed | Pick one Db.GetTable or Db.Run line in your C# and find the stored procedure with exactly that name in your script. Check that each @ name in the Db.P lines under it matches a parameter at the top of that stored procedure. Then do the same for every other Db.GetTable and Db.Run line. |
| A report with nothing in it | Must be allowed | Run every report for two days long before your business opened, such as 01/01/2000 to 02/01/2000. Each report shows an empty grid and a sentence, never an error. |
| A sold item cannot be sold twice | Must be refused | Sell one item that is one of a kind, then try to sell the same item again. The second sale is refused with a sentence, and the item's status reads Sold. |
| An edit keeps every box in its place | Must be allowed | Open one record for editing and change every box to a new value. Click Save, then open the same record again. Each box shows the value you typed into that box, and no box shows the value of another. |
| No property reads itself | Must be allowed | Open each class in your Models folder. Every property is { get; set; }, or its get part reads other properties and never its own name. |

CHAPTER 12 OF 12

# Make it yours

Turning the bike shop into your scenario: the same patterns, your tables.

STAGE 1 OF 4 · THIS YEAR'S CASE STUDY, MATCHED TO THE BIKE SHOP · SCREEN 1 OF 4

## See what you will do in this chapter.

Towpath Cycles was never your project. It was a practice business, made so that every pattern a case study asks for appears once, in a place you can see it working. Your own project is the case study that CCEA sets this year. This chapter shows you which part of the bike shop each part of this year's case study matches.

By the end you will have three things: a list that matches each thing in the case study to a bike shop table, a list of the stored procedures to copy and the names to change in each, and two upgrades that are worth extra marks. Nothing in this chapter changes your BikeShop database. Everything here is reading, matching and writing on paper.

**Before you start**

- Chapters 0 to 11 are finished, and the whole BikeShop program runs.
- Your copy of this year's case study from your teacher, The Gallery, is beside you.
- You have a sheet of paper, or a blank Word document, for your own lists.
- This guide is open in the Edge browser beside you.

**Words used in this chapter**

| | |
|---|---|
| `Case study` | The made-up business that CCEA describes each year. Your project is the program for that business. |
| `Shape` | A pattern that comes back in every case study, such as a booking of one person for a time, or stock that is counted. |
| `Unique item` | A thing there is only one of, such as one painting. It can be reserved and sold once only. |
| `Counted stock` | Things kept as a number, such as tubes of paint. The number goes down with each sale and up with each delivery. |
| `Capacity` | The most people that one class or one room can take at the same time. |
| `Deposit` | Part of the price that a customer pays first, to hold a booking. |
| `Header table` | A table with one row for a whole order: when it was placed, and whether it has arrived. |
| `Line table` | A table with one row for each thing on an order. Each row holds the number of its order, so it is linked to its header row. |

| | |
|---|---|
| `Provisional` | Held for now but not yet confirmed. A provisional booking is let go if nobody confirms it in time. |
| `ALTER TABLE` | The words that change a table that already exists, such as adding a column, without building the table again. |
| `UNIQUE` | A rule in a table that refuses two rows with the same value, or with the same pair of values. |

STAGE 1 OF 4 · THIS YEAR'S CASE STUDY, MATCHED TO THE BIKE SHOP · SCREEN 2 OF 4

# Read this year's case study: The Gallery.

The Gallery sells art and antiques. It also runs pottery and photography, frames pictures, and sells art supplies. Read your copy of the case study first. The box below lists the parts of it that need a table or a stored procedure.

**What The Gallery needs your program to do**

Every artwork is one of a kind. Staff put a blue dot on an artwork when a customer reserves it, and a red dot when it is sold. A reserved artwork can go back to stock. Art supplies are counted. The Gallery orders pottery materials and photography materials from suppliers, and those orders are never written down anywhere at the moment. Customers book photo appointments, and the same photographer must never be booked twice at once. Pottery classes run for 1 day, 3 days or 5 days. A pottery consultation needs a deposit of 15 per cent, which is not paid back. Framing needs a deposit of 10 per cent. A customer who buys at an exhibition pays a deposit of 20 per cent and collects the artwork within 2 working days.

Every sentence in that box matches a shape you have already built. The next two screens go through them, one table at a time.

STAGE 1 OF 4 · THIS YEAR'S CASE STUDY, MATCHED TO THE BIKE SHOP · SCREEN 3 OF 4

# Match what The Gallery sells to the two kinds of stock in the bike shop.

**An artwork is a bike for sale**

A bike for sale is one of a kind, and so is an artwork. Your Artwork table is the Bike table with its own columns: the title, the artist and the price in place of the make, the model and the frame number. The Status column keeps the same three words. Available has no dot. Reserved is the blue dot. Sold is the red dot. Back to stock is what usp_Bike_Unreserve did: the status goes back to Available and the customer number is cleared.

**Art supplies are parts**

A part is counted stock, and so is a tube of paint or a bag of clay. Your Supply table is the Part table: a code, a name, a supplier, a cost, a selling price, a number in stock and a reorder level. The rule that stock is always right stays word for word. A sale is refused when there are not enough in stock, and the number in stock is only ever changed inside a stored procedure.

**Supplier orders are part orders**

The case study says that orders for pottery and photography materials are never written down. That is a problem your program solves. Your SupplyOrder table is the PartOrder table. Placing an order changes nothing in stock. Receiving the delivery closes the order and puts the number in stock up. Stage 3 shows an upgrade that lets one order hold many different supplies.

STAGE 1 OF 4 · THIS YEAR'S CASE STUDY, MATCHED TO THE BIKE SHOP · SCREEN 4 OF 4

# Match the bookings The Gallery takes to the bookings in the bike shop.

**A photo appointment is a workshop appointment**

A workshop appointment books one mechanic for a start time and an end time. A photo appointment books one photographer in the same way. Your PhotoAppointment table is the Appointment table, with PhotographerID in place of MechanicID. The two lines of the clash check that you typed in chapter 7 stay exactly as they are, so a photographer can never be booked twice at once.

**A pottery class is a count, not a clash**

A pottery class is different. Many customers book the same class, so one booking does not clash with another. Instead, the class has a number of places. Give your class table a Places column, which is its capacity. In the stored procedure that books a place, replace the clash check with a count: count the bookings for that class that are not cancelled, and refuse with the sentence That class is full. when the count has reached Places. The count is one line: `IF (SELECT COUNT(*) FROM ClassBooking WHERE ClassID = @ClassID AND Status <> 'Cancelled') ⩾ (SELECT Places FROM Class WHERE ClassID = @ClassID)`. The 1-day, 3-day and 5-day classes are three rows of a class type table, like the rows of ServiceType.

**Every deposit is the hire deposit rule**

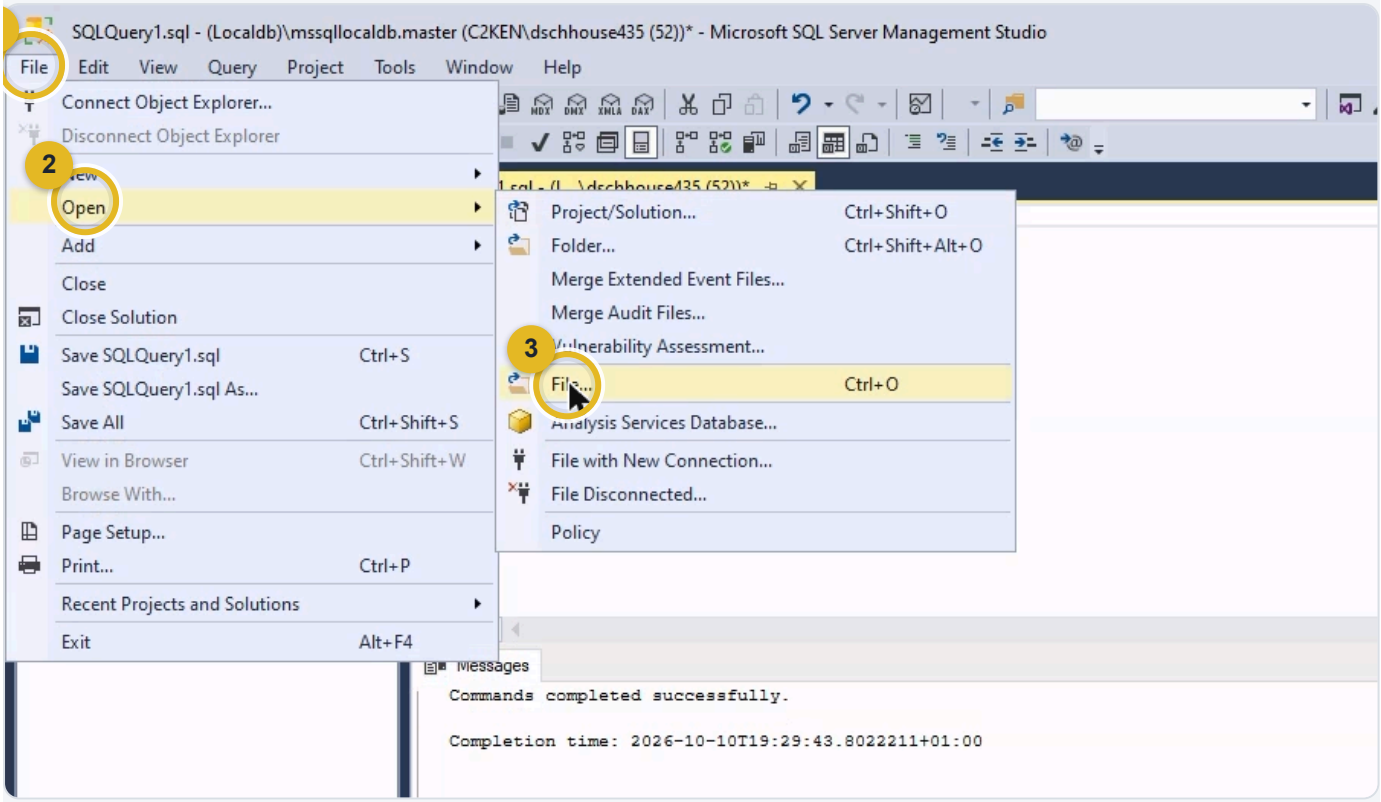
A bike hire needs a deposit of at least 20 per cent of the total cost, and usp_Hire_Add refuses anything less. The Gallery has three deposits, and each one is the same deposit lines with a different number. Framing uses 0.10 in place of 0.20. A pottery consultation uses 0.15. Collecting from an exhibition keeps 0.20. Change the number in the refusal sentence too, so that it says 10% or 15%. The consultation deposit is not paid back, so the stored procedure that cancels a consultation leaves the DepositPaid column as it is. The 2 working days to collect from an exhibition is a deadline, and stage 3 shows how to keep one.

# Learn how to turn one bike shop stored procedure into a Gallery one.

Every action in The Gallery is a stored procedure you have already written for Towpath Cycles. You do not start from nothing. You copy the bike shop's stored procedure into your own script and change the names. The checks, the IF lines, the refusal sentences and the order of the lines stay the same, because the rules are the same.

The next two screens hold a set of cards. Each card is one action The Gallery needs. It names the bike shop stored procedure to copy, then lists the names that change. Work through one card now, on paper, so that you know the method before your own project starts.

Click File, then Open, then File… in SQL Server Management Studio (SSMS), and choose BikeShop.sql from your OneDrive folder.




The File menu in SQL Server Management Studio (SSMS). Your script opens from here.

1. Click File at the top left.

2. Point at Open.

3. Click File… and choose BikeShop.sql from your OneDrive folder.

> **You should see** Your BikeShop script is in a tab, and the line USE BikeShop; is near the top.

Scroll down the script until you reach the line CREATE OR ALTER PROCEDURE usp_Bike_Reserve. Your stored procedures are in the order the chapters added them, so the bike ones are after the appointment ones.

> **You should see** The stored procedure that reserves a bike is on the screen, with @BikeID, @CustomerID and @Message at the top.

On your paper, write each name in usp_Bike_Reserve that belongs to the bike shop, one under another: usp_Bike_Reserve, Bike, @BikeID, BikeID, the Purpose test, and the refusal sentence.

> **You should see** Your paper has a column of six bike shop names.

Beside each of the six bike shop names on your paper, write its Gallery name from the first card on the next screen, Reserve an artwork with a blue dot.

> **You should see** Your paper has two columns. The left column holds usp_Bike_Reserve and the right column holds usp_Artwork_Reserve, and so on down the page. The two columns are the whole change.

STAGE 2 OF 4 · WHICH STORED PROCEDURE TO COPY, AND WHAT TO RENAME · SCREEN 2 OF 3

# Copy these stored procedures for the artworks and the art supplies.

**Reserve an artwork with a blue dot**

Copy usp_Bike_Reserve. Rename it usp_Artwork_Reserve. Bike becomes Artwork and @BikeID becomes @ArtworkID. Take out the test Purpose = 'Sale', because every artwork is for sale. The refusal sentence becomes Only an available artwork can be reserved.

**Take a reservation off, back to stock**

Copy usp_Bike_Unreserve. Rename it usp_Artwork_Unreserve. Bike becomes Artwork and @BikeID becomes @ArtworkID. The refusal sentence becomes That artwork is not reserved.

**Sell an artwork with a red dot**

Copy usp_Bike_Sell. Rename it usp_Artwork_Sell. Bike becomes Artwork and @BikeID becomes @ArtworkID. Take out the Purpose test again. Each sentence that says bike says artwork.

**Sell art supplies**

Copy usp_PartSale_Add. Rename it usp_SupplySale_Add. Part becomes Supply, PartSale becomes SupplySale and @PartID becomes @SupplyID. Keep QtyInStock, the check against @Quantity and the sentence that starts Only, word for word.

**Order pottery or photography materials**

Copy usp_PartOrder_Place. Rename it usp_SupplyOrder_Place. PartOrder becomes SupplyOrder and @PartID becomes @SupplyID. Placing an order still changes nothing in stock.

**Receive a delivery from a supplier**

Copy usp_PartOrder_Receive. Rename it usp_SupplyOrder_Receive. PartOrder becomes SupplyOrder, Part becomes Supply and @PartID becomes @SupplyID. Closing the order and putting the stock up still happen in the same stored procedure.

# Copy these stored procedures for the photo appointments, pottery, framing and collection.

**Book a photo appointment**

Copy usp_Appointment_Add. Rename it usp_PhotoAppointment_Add. Appointment becomes PhotoAppointment, @MechanicID becomes @PhotographerID and ServiceType becomes PhotoSessionType. Change the opening hours in the time check to The Gallery's hours. Keep the two lines of the clash check exactly as they are.

**Move a photo appointment**

Copy usp_Appointment_Move. Rename it usp_PhotoAppointment_Move, with the same changes. A booking may still overlap itself, so keep the line that leaves out its own number.

**Book a place on a pottery class**

Copy usp_Appointment_Add. Rename it usp_ClassBooking_Add. Appointment becomes ClassBooking. Replace the clash check with the count against Places from stage 1, and the refusal sentence with That class is full.

**Book a pottery consultation with a 15 per cent deposit**

Copy usp_Appointment_Add, then the deposit lines from usp_Hire_Add. Rename it usp_Consultation_Add. Add a @DepositPaid parameter, and copy in the deposit lines from usp_Hire_Add with 0.15 in place of 0.20.

**Book framing with a 10 per cent deposit**

Copy usp_Hire_Add. Rename it usp_Framing_Add. Hire becomes Framing and @BikeID goes. Keep the deposit lines with 0.10 in place of 0.20. Framing books a date range for a job, not a bike, so the clash check comes out.

**Hold an exhibition sale for collection, with a 20 per cent deposit**

Copy usp_Bike_Reserve, then the deposit lines from usp_Hire_Add. Rename it usp_Artwork_ReserveForCollection. Keep 0.20. Add an ExpiresOn date, 2 working days after today. Stage 3 shows how to let the sale go when that date has passed.

**Cancel any booking**

Copy usp_Appointment_Cancel. Rename it to match each booking table, such as usp_PhotoAppointment_Cancel. A cancel is still a change of Status, never a DELETE, because the row is The Gallery's history.

**Every report**

Copy usp_Report_MechanicWorkload and the other four reports. Rename them for The Gallery's own questions, such as usp_Report_PhotographerWorkload or usp_Report_IncomeBySource. Keep the LEFT JOIN with the date test after ON.

STAGE 3 OF 4 · TWO UPGRADES WORTH THE MARKS · SCREEN 1 OF 3

# Read why these two upgrades earn marks.

A project that copies the bike shop's shapes well earns good marks. The top marks go to a project that also handles something the bike shop did not need. The Gallery has two things like that, and each one is a small step on from code you already know.

**Upgrade one: one supplier order with many lines**

In the bike shop, one order is for one part. A real supplier order from The Gallery holds clay, glaze and photo paper on the same piece of paper. That needs a header table for the order and a line table for each thing on it, and the stored procedure that receives the delivery must put up the stock of every line at once.

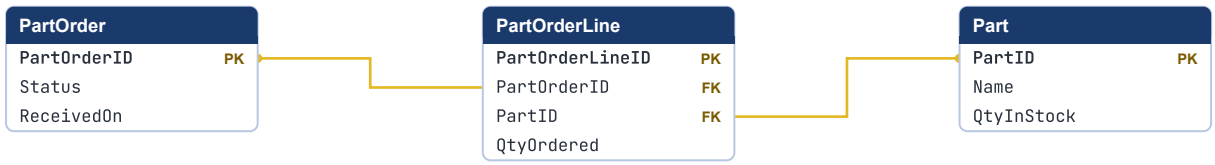
**Upgrade two: a booking held for a short time, then let go**

A customer who asks for a pottery consultation has not paid the deposit yet. An exhibition buyer has 2 working days to collect. In both cases the booking is held, but only until a deadline. That needs a status called Provisional, a column that holds the deadline, and a stored procedure that cancels every held booking whose deadline has passed.

Both upgrades are code blocks to read, not to copy. Read them here, part by part. Do not paste them into your BikeShop script, because the bike shop does not need them. You build them in your own project, with your own names, when you reach chapters 3 and 8.

# Read upgrade one: one supplier order with many lines.

The picture shows the three tables. The order table is on the left, the lines table is in the middle and the part table is on the right. Each line row points back to its order and across to one part. In your own project, the header table keeps the dates, the status and who placed the order. The part number and the quantity move into the lines.




One order on the left, many lines in the middle. Each line points back to its order and across to one part.

Read this one **PartOrderLine and usp_PartOrder_ReceiveLines: an order with many lines** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
-- Upgrade 1: one supplier order with many lines.
-- The lines table is a child of PartOrder, so it is dropped first.
DROP TABLE IF EXISTS PartOrderLine;
GO

CREATE TABLE PartOrderLine (
    PartOrderLineID INT IDENTITY(1,1) NOT NULL,
    PartOrderID     INT NOT NULL,
    PartID          INT NOT NULL,
    QtyOrdered      INT NOT NULL,
    CONSTRAINT PK_PartOrderLine PRIMARY KEY (PartOrderLineID),
    CONSTRAINT FK_PartOrderLine_PartOrder FOREIGN KEY (PartOrderID)
        REFERENCES PartOrder (PartOrderID),
    CONSTRAINT FK_PartOrderLine_Part FOREIGN KEY (PartID)
        REFERENCES Part (PartID),
    -- one line per part on each order
    CONSTRAINT UQ_PartOrderLine UNIQUE (PartOrderID, PartID),
    CONSTRAINT CK_PartOrderLine_Qty CHECK (QtyOrdered > 0)
);
GO

CREATE OR ALTER PROCEDURE usp_PartOrder_ReceiveLines
    @PartOrderID INT,
    @Message VARCHAR(200) OUTPUT
AS
BEGIN
    SET @Message = '';

    IF NOT EXISTS (SELECT * FROM PartOrder WHERE PartOrderID = @PartOrderID
        AND Status = 'Ordered')
    BEGIN
        SET @Message = 'That order is not open.';
        RETURN;
    END
    IF NOT EXISTS (SELECT * FROM PartOrderLine
        WHERE PartOrderID = @PartOrderID)
    BEGIN
        SET @Message = 'That order has no lines.';
        RETURN;
    END

    UPDATE PartOrder
    SET Status = 'Received', ReceivedOn = CAST(GETDATE() AS DATE)
    WHERE PartOrderID = @PartOrderID;

    -- Every line at once: each part goes up by its own line's quantity.
    UPDATE Part
    SET QtyInStock = QtyInStock + PartOrderLine.QtyOrdered
    FROM Part
        INNER JOIN PartOrderLine ON PartOrderLine.PartID = Part.PartID
    WHERE PartOrderLine.PartOrderID = @PartOrderID;
END
```

```
53  GO
```

**1 Drop the old lines table first**

Lines 3 to 4

A line row points at its order, so the lines table is a child of PartOrder. Children are dropped before their parents. In your own script, this line goes at the top of the list of DROP lines, above the order table.

**2 The lines table and its four columns**

Lines 6 to 10

Each line has its own number, the number of the order it belongs to, the part it is for, and how many were ordered. Three lines of the same order all hold the same PartOrderID.

**3 The key, and the two links**

Lines 11 to 15

The primary key is the line's own number. The first foreign key links each line to a real order. The second foreign key links each line to a real part, so a line can never be for a part that does not exist.

**4 One line per part, and never zero**

Lines 17 to 20

UNIQUE on the pair PartOrderID and PartID refuses a second line for the same part on the same order, so 10 tubes of paint are one line, not two lines of 5. The CHECK refuses a line for 0 or fewer.

**5 The name, and the one thing that goes in**

Lines 22 to 24

The stored procedure needs only the order number. Everything else it needs is already in the lines. @Message carries the refusal sentence back out, as in every stored procedure that changes data.

**6 AS, BEGIN and the blank message**

Lines 25 to 27

AS and BEGIN mark where the work starts. The message starts blank, and a blank message at the end means the delivery was received.

**7 Refuse an order that is not open**

Lines 29 to 34

An order that has already been received, or was cancelled, must not put the stock up a second time. If there is no open order with this number, the sentence That order is not open. goes back, and RETURN stops the stored procedure there.

**8 Refuse an order with no lines**

Lines 35 to 40

An order with nothing on it has nothing to receive. The sentence That order has no lines. goes back, and RETURN stops the stored procedure.

**9 Close the order**

Lines 42 to 44

The header row is marked Received, with today's date. This is the same UPDATE as in usp_PartOrder_Receive. This version receives every line in full.

**10 Put the stock up for every line at once**

Lines 47 to 53

This is the line that handles the whole order. The INNER JOIN matches each part to its own line on this order, and each matched part goes up by the quantity on its own line. An order with three lines puts three parts up, in one UPDATE. Parts that are not on the order do not match the join, so they are left as they were. END closes the stored procedure, and GO ends the batch.

To place an order with many lines, your own project needs one more stored procedure that adds one line to an open order. It is usp_PartOrder_Place with the part and the quantity moved into the line table.

STAGE 3 OF 4 · TWO UPGRADES WORTH THE MARKS · SCREEN 3 OF 3

# Read upgrade two: a booking held for a short time, then let go.

A provisional booking holds its slot like any other booking. The clash check counts every booking that is not cancelled, so nobody else can book that time while it is held. When the deadline passes, one stored procedure cancels it and the slot is free again. Your diary form runs that stored procedure each time it opens, so a booking that has run out is gone before anyone looks.

Read this one **A Provisional status, ExpiresOn and usp_Appointment_ExpireProvisional** SQL

**What is going on here?** Point at a line, or at a card, to see what goes with what.

```sql
-- Upgrade 2: a booking held for a short time, then let go.
-- In your own script these changes go inside CREATE TABLE.  Here, ALTER
-- TABLE makes them to the Appointment table that is already there.
ALTER TABLE Appointment ALTER COLUMN Status VARCHAR(12) NOT NULL;
ALTER TABLE Appointment ADD ExpiresOn DATETIME NULL;
ALTER TABLE Appointment DROP CONSTRAINT CK_Appointment_Status;
ALTER TABLE Appointment ADD CONSTRAINT CK_Appointment_Status
    CHECK (Status IN ('Provisional', 'Booked', 'Done', 'Cancelled'));
GO

CREATE OR ALTER PROCEDURE usp_Appointment_ExpireProvisional
    @Message VARCHAR(200) OUTPUT
AS
BEGIN
    SET @Message = '';
    -- A held booking whose time is up is cancelled.  The row stays as
    -- history, and the slot is free for someone else.
    UPDATE Appointment
    SET Status = 'Cancelled'
    WHERE Status = 'Provisional' AND ExpiresOn < GETDATE();
END
GO
```

**1 Make the Status column longer**

Line 4

The word Provisional has eleven letters, and the Status column holds only ten. This line makes it hold twelve. In your own script you type VARCHAR(12) in CREATE TABLE instead, and ALTER TABLE is not needed.

**2 A new column for the deadline**

Line 5

ExpiresOn holds the date and time when the hold runs out. It is empty for an ordinary booking, which never runs out.

**3 Add Provisional to the allowed statuses**

Lines 6 to 9

The CHECK on the Status column lists the words it allows. The old CHECK is dropped, and a new one is added with Provisional at the front of the list. In your own script, you add the word to the CHECK inside CREATE TABLE.

**4 The name, and the blank message**

Lines 11 to 15

Nothing goes in, because the stored procedure looks at every held booking. It still has @Message, because it changes data, and the message starts blank.

**5 Cancel every hold whose time is up**

Lines 18 to 22

> Every booking that is Provisional and whose ExpiresOn is before now becomes Cancelled. GETDATE() is the date and time at this moment. The row stays in the table as history, and the time slot is free for someone else. END closes the stored procedure, and GO ends the batch.

To make a provisional booking, the stored procedure that adds the booking sets Status to Provisional and ExpiresOn to `DATEADD(HOUR, 48, GETDATE())`, which is 48 hours from now. To confirm it when the deposit is paid, copy usp_Appointment_MarkDone and change two words: the status it looks for is Provisional, and the status it sets is Booked.

# Keep this table for testing your own project.

These are the tests an examiner would run on The Gallery. You cannot tick them yet, because your own project is not built. Come back to this table when your own program runs, and tick each row when you have seen it happen. Rows marked Must be refused are the most important ones. Each one proves that a rule holds, so the program says no in a plain sentence and nothing in the database changes.

| | THE TEST | WHAT MUST HAPPEN | HOW TO DO IT, AND WHAT YOU SHOULD SEE |
|---|---|---|---|
| | Reserve an available artwork | Must be allowed | Reserve an artwork with the status Available for a customer in your program. Its status becomes Reserved, the blue dot, with the customer's number beside it. |
| | Reserve an artwork that is already sold | Must be refused | Try to reserve an artwork with the status Sold. The message reads Only an available artwork can be reserved., and the artwork stays Sold. |
| | Take a reservation off | Must be allowed | Take the reservation off the artwork from the first row. Its status goes back to Available and the customer number is empty. |
| | Sell more art supplies than are in stock | Must be refused | Pick a supply with 2 in stock and try to sell 3. The message reads Only 2 in stock., and the number in stock is still 2. |
| | Receive a supplier order | Must be allowed | Place an order for 10 of one supply, then receive it. The order is marked Received, and the number in stock is 10 higher than before. |
| | Book one photographer twice at the same time | Must be refused | Book a photographer for 10:00 to 11:00, then try to book the same photographer for 10:30 to 11:30 on the same day. The second booking is refused with the clash sentence, and only one booking is in the table. |
| | Book one place more than a class has | Must be refused | Give a class 2 places, book 2 customers on it, then try to book a third. The third is refused with That class is full. |
| | Book framing with too small a deposit | Must be refused | Book a framing job that costs £50.00 with a deposit of £4.00. The message says the deposit must be at least £5.00, and no booking is made. |

| THE TEST | WHAT MUST HAPPEN | HOW TO DO IT, AND WHAT YOU SHOULD SEE |
|---|---|---|
| A provisional booking past its deadline is let go | **Must be allowed** | Make a provisional booking in your program. Then, in SQL Server Management Studio (SSMS), right-click your booking table, choose Edit Top 200 Rows and change that booking's ExpiresOn to yesterday's date. Then close your diary form and open it again. The booking now shows as Cancelled, and the time is free to book again. |
| Cancel a booking and keep its row | **Must be allowed** | Cancel any booking. It stays in the table with the status Cancelled. A consultation that is cancelled still shows its deposit as paid. |

STAGE 4 OF 4 · PROVE IT: YOUR OWN PROJECT · SCREEN 2 OF 2

## Start your own project.

You have built every shape a case study asks for, in a business made for practice. The Gallery uses the same shapes, with different names on them. Your two lists from this chapter are the plan: which bike shop table each Gallery table copies, and which stored procedure each Gallery action copies.

Go back to chapter 2 with The Gallery beside you. Work through each chapter again, and at each Make it yours box, build that part for The Gallery instead of the bike shop.

Your own project starts in chapter 2 with your own nouns.